

The Pragmatic Programmer

Bản dịch song ngữ Anh–Việt · 20th Anniversary Edition (2nd Edition)

Andrew Hunt & David Thomas (nguyên tác)

2026

Mục lục

A Pragmatic Philosophy — Một triết lý thực dụng	17
Topic 1. It's Your Life — Topic 1. Đời Bạn Là Của Bạn	18
Topic 2. The Cat Ate My Source Code — Topic 2. Mèo Ăn Mất Mã Nguồn Của Tôi	20
Topic 3. Software Entropy — Topic 3. Sự rối loạn phần mềm (Software Entropy)	23
Topic 4. Stone Soup and Boiled Frogs — Topic 4. Súp đá và những con ếch bị luộc	27
Topic 5. Good-Enough Software — Topic 5. Phần mềm Đủ tốt	30
Topic 6. Your Knowledge Portfolio — Topic 6. Danh mục Tri thức của bạn	34
Topic 7. Communicate! — Topic 7. Hãy giao tiếp!	41
A Pragmatic Approach — Một Cách Tiếp Cận Thực Dụng	51
Topic 8. The Essence of Good Design — Topic 8. Cốt Lõi của Thiết Kế Tốt	52
Topic 9. DRY—The Evils of Duplication — Topic 9. DRY — Tai họa của sự trùng lặp	55
Calculate the fees for this account. # * Each returned check costs \$20 # * If the account is in overdraft for more than 3 days, # charge \$10 for each day # * If the average account balance is greater than \$2,000 # reduce the fees by 50% — Calculate the fees for this account. # * Each returned check costs \$20 # * If the account is in overdraft for more than 3 days, # charge \$10 for each day # * If the average account balance is greater than \$2,000 # reduce the fees by 50%	61
Topic 10. Orthogonality — Topic 10. Tính trực giao	67
Topic 11. Reversibility — Topic 11. Tính khả nghịch	77
Topic 12. Tracer Bullets — Topic 12. Đạn vạch đường	82
Topic 13. Prototypes and Post-it Notes — Topic 13. Bản mẫu và Giấy ghi chú Post-it	89
Topic 14. Domain Languages — Topic 14. Ngôn ngữ miền	94
Topic 15. Estimating — Topic 15. Ước lượng	102
The Basic Tools — Những công cụ cơ bản	112
Topic 16. The Power of Plain Text — Topic 16. Sức mạnh của Văn bản thuần	114
Topic 17. Shell Games — Topic 17. Trò chơi với Shell	119
Topic 18. Power Editing — Topic 18. Soạn thảo Cao thủ	123
Topic 19. Version Control — Topic 19. Quản lý phiên bản	127
Topic 20. Debugging — Topic 20. Gỡ lỗi	133
Topic 21. Text Manipulation — Topic 21. Thao tác văn bản	145
Topic 22. Engineering Daybooks — Topic 22. Nhật ký kỹ thuật (Engineering Daybook)	148
Pragmatic Paranoia — Hoang tưởng kiểu Thực dụng	152
Topic 23. Design by Contract — Topic 23. Thiết kế theo hợp đồng	154

Extra Federal requirements for reporting # Some processing... end def accept_deposit(account_id,

amount) when (amount > 0) do # Some processing... end end — Extra Federal requirements for reporting # Some processing... end def accept_deposit(account_id, amount) when (amount > 0) do # Some processing... end end	158
Topic 24. Dead Programs Tell No Lies — Topic 24. Chương trình chết thì không nói dối . . .	166
Topic 25. Assertive Programming — Topic 25. Lập trình phòng vệ bằng assertion	169
Topic 26. How to Balance Resources — Topic 26. Cân Bằng Tài Nguyên	174
Topic 27. Don't Outrun Your Headlights — Topic 27. Đừng Chạy Vượt Đèn Pha Của Mình . .	182
Bend, or Break — Uốn cong, hoặc Gãy	187
Topic 28. Decoupling — Topic 28. Gỡ ghép nối	188
This is pretty poor style amount = customer.orders.last().totals().amount; — Đây là phong cách khá tệ amount = customer.orders.last().totals().amount;	194
and so is this... orders = customer.orders; last = orders.last(); totals = last.totals(); amount = totals.amount; — và đây cũng vậy... orders = customer.orders; last = orders.last(); totals = last.totals(); amount = totals.amount;	195
Topic 29. Juggling the Real World — Topic 29. Tung hứng với thế giới thực	199
current new state action to take # ————— — current new state action to take # —————	204
Topic 30. Transforming Programming — Topic 30. Lập trình theo lối Biến đổi	212
————— —————	224
————— —————	225
Topic 31. Inheritance Tax — Topic 31. Thuế Kế Thừa	229
top-level code my_ride = Car.new my_ride.move_at(30) — mã ở tầng top-level my_ride = Car.new my_ride.move_at(30)	232
Topic 32. Configuration — Topic 32. Cấu hình	240
Concurrency — Xử lý đồng thời	245
Topic 33. Breaking Temporal Coupling — Topic 33. Phá vỡ Ghép nối Thời gian	247
Topic 34. Shared State Is Incorrect State — Topic 34. Trạng Thái Chia Sẻ Là Trạng Thái Sai .	252
Topic 35. Actors and Processes — Topic 35. Actor và Tiến trình	260
Topic 36. Blackboards — Topic 36. Bảng đen	267
While You Are Coding — Khi Bạn Đang Viết Mã	274
Topic 37. Listen to Your Lizard Brain — Topic 37. Lắng Nghe Bộ Não Bò Sát Của Bạn	276
Topic 38. Programming by Coincidence — Topic 38. Lập trình theo may rủi	282
Topic 39. Algorithm Speed — Topic 39. Tốc độ thuật toán	290
Topic 40. Refactoring — Topic 40. Tái cấu trúc	297
Topic 41. Test to Code — Topic 41. Kiểm thử để viết mã	303
Topic 42. Property-Based Testing — Topic 42. Kiểm thử dựa trên thuộc tính	316
Topic 43. Stay Safe Out There — Topic 43. Hãy An Toàn Ngoài Kia	325
Topic 44. Naming Things — Topic 44. Đặt Tên Cho Mọi Thứ	334
Before the Project — Trước Khi Bắt Đầu Dự Án	343
Topic 45. The Requirements Pit — Topic 45. Cái Hố Yêu Cầu	344
Topic 46. Solving Impossible Puzzles — Topic 46. Giải những câu đố tưởng chừng bất khả thi	356
Topic 47. Working Together — Topic 47. Làm Việc Cùng Nhau	361

Topic 48. The Essence of Agility — Topic 48. Tinh túy của sự linh hoạt	365
Pragmatic Projects — Dự án Thực dụng	372
Topic 49. Pragmatic Teams — Topic 49. Nhóm Thực dụng	373
Topic 50. Coconuts Don't Cut It — Topic 50. Dừa Thì Không Ăn Thua	381
Topic 51. Pragmatic Starter Kit — Topic 51. Bộ Khởi Đầu Thực Dụng	386
Topic 52. Delight Your Users — Topic 52. Làm Hài Lòng Người Dùng	395
Topic 53. Pride and Prejudice — Topic 53. Niềm tự hào và Định kiến	397
Postface — Lời bạt	400
Bibliography	404
Possible Answers to the Exercises — Đáp án khả dĩ cho các bài tập	406

I remember when Dave and Andy first tweeted about the new edition of this book. It was big news. I watched as the coding community responded with excitement. My feed buzzed with anticipation. After twenty years, The **Pragmatic Programmer** is just as relevant today as it was back then.

Tôi vẫn nhớ lần đầu Dave và Andy tweet về ấn bản mới của cuốn sách này. Đó là một tin lớn. Tôi đã chứng kiến cộng đồng lập trình phản ứng đầy hào hứng. Dòng thời gian của tôi rộn ràng những chờ đợi. Sau hai mươi năm, The Pragmatic Programmer vẫn đúng đắn với hôm nay y như thuở ban đầu.

It says a lot that a book with such history had such a reaction. I had the privilege of reading an unreleased copy to write this foreword, and I understood why it created such a stir. While it's a technical book, calling it that does it a disservice. Technical books often intimidate. They're stuffed with big words, obscure terms, convoluted examples that, unintentionally, make you feel stupid. The more experienced the author, the easier it is to forget what it's like to learn new concepts, to be a beginner.

Một cuốn sách có bề dày lịch sử như vậy mà còn gây được phản ứng đến thế thì đã nói lên rất nhiều điều. Tôi may mắn được đọc một bản chưa phát hành để viết lời tựa này, và tôi hiểu vì sao nó tạo nên một làn sóng như thế. Tuy là sách kỹ thuật, nhưng gọi nó như vậy là làm nó thiệt thòi. Sách kỹ thuật thường khiến người ta e dè. Chúng nhồi nhét những từ to tát, những thuật ngữ tối nghĩa, những ví dụ rối rắm — vô tình làm bạn thấy mình ngu ngốc. Tác giả càng nhiều kinh nghiệm thì lại càng dễ quên cảm giác học một khái niệm mới, cảm giác làm một người mới bắt đầu là như thế nào.

Despite their decades of programming experience, Dave and Andy have conquered the difficult challenge of writing with the same excitement of people who've just learned these lessons. They don't talk down to you. They don't assume you are an expert. They don't even assume you've read the first edition. They take you as you are—programmers who just want to be better. They spend the pages of this book helping you get there, one actionable step at a time.

Dù đã có hàng chục năm kinh nghiệm lập trình, Dave và Andy vẫn chinh phục được thử thách khó nhằn: viết với chính sự hào hứng của những người vừa mới học được những bài học ấy. Họ không nói chuyện kiểu bề trên với bạn. Họ không mặc định bạn là chuyên gia. Họ thậm chí không mặc định bạn đã đọc ấn bản đầu tiên. Họ đón nhận bạn như bạn vốn có — những lập trình viên chỉ muốn giỏi hơn. Họ dùng cả cuốn sách này để giúp bạn đạt tới đó, từng bước hành động một.

To be fair, they'd already done this before. The original release was full of tangible examples, new ideas, and practical tips to build your coding muscles and develop your coding brain that still apply today. But this updated edition makes two improvements on the book.

Công bằng mà nói, họ đã làm được điều này từ trước rồi. Bản gốc đầy ắp những ví dụ cụ thể, những ý tưởng mới và những mẹo thiết thực để rèn cơ bắp lập trình và phát triển bộ não lập trình của bạn — những điều đến nay vẫn còn đúng. Nhưng ấn bản cập nhật này có hai cải tiến.

The first is the obvious one: it removes some of the older references, the out-of-date examples, and replaces them with fresh, modern content. You won't find examples of loop invariants or build machines. Dave and Andy have taken their powerful content and made sure the lessons still come through, free of the distractions of old examples. It dusts off old ideas like **DRY** (don't repeat yourself) and gives them a fresh coat of paint, really making them shine.

Cải tiến đầu tiên thì rõ ràng: nó loại bỏ một số tham chiếu cũ kỹ, những ví dụ đã lỗi thời, và thay bằng nội dung mới mẻ, hiện đại. Bạn sẽ không còn thấy ví dụ về bất biến vòng lặp (loop invariant) hay máy build nữa. Dave và Andy đã lấy phần nội dung đầy sức nặng của mình và đảm bảo các bài học vẫn truyền tải được trọn vẹn, không còn bị những ví dụ cũ làm nhiễu. Họ phủ bụi những ý tưởng cũ như DRY (Đừng Lặp Lại Chính Mình) và sơn cho chúng một lớp mới, làm chúng thực sự tỏa sáng.

But the second is what makes this release truly exciting. After writing the first edition, they had the chance to reflect on what they were trying to say, what they wanted their readers to take away, and how it was being received. They got feedback on those lessons. They saw what stuck, what needed refining, what was misunderstood. In the twenty years that this book has made its way through the hands and hearts of programmers all over the world, Dave and Andy have studied this response and formulated new ideas, new concepts.

Nhưng cải tiến thứ hai mới là thứ khiến lần tái bản này thực sự đáng háo hức. Sau khi viết ấn bản đầu, họ có cơ hội ngẫm lại điều mình muốn nói, điều mình muốn người đọc đem theo, và cách nó được đón nhận. Họ nhận được phản hồi về những bài học ấy. Họ thấy điều gì đọng lại, điều gì cần mài giũa, điều gì bị hiểu nhầm. Trong hai mươi năm cuốn sách này lưu chuyển qua tay và trái tim của những lập trình viên khắp thế giới, Dave và Andy đã nghiên cứu phản hồi đó và hình thành những ý tưởng mới, những khái niệm mới.

They've learned the importance of agency and recognized that developers have arguably more agency than most other professionals. They start this book with the simple but profound message: "it's your life." It reminds us of our own power in our code base, in our jobs, in our careers. It sets the tone for everything else in the book—that it's more than just another technical book filled with code examples.

Họ đã hiểu ra tầm quan trọng của quyền tự quyết (agency), và nhận ra rằng các lập trình viên hẳn còn có nhiều quyền tự quyết hơn hầu hết những người làm nghề khác. Họ mở đầu cuốn sách bằng một thông điệp giản dị mà sâu sắc: "đây là cuộc đời của bạn". Nó nhắc ta nhớ về sức mạnh của chính mình trong cơ sở mã (codebase), trong công việc, trong sự nghiệp. Nó đặt nền cho mọi thứ còn lại trong sách — rằng đây không chỉ là thêm một cuốn sách kỹ thuật nhồi đầy ví dụ mã.

What makes it truly stand out among the shelves of technical books is that it understands what it means to be a programmer. Programming is about trying to make the future less painful. It's about making things easier for our teammates. It's about getting things wrong and being able to bounce back. It's about forming good habits. It's about understanding your toolset. Coding is just part of the world of being a programmer, and this book explores that world.

Điều khiến nó thực sự nổi bật giữa những kệ sách kỹ thuật là nó hiểu được làm một lập trình viên nghĩa là gì. Lập trình là cố làm cho tương lai bớt đau đớn hơn. Là làm mọi thứ dễ dàng hơn cho đồng đội. Là làm sai rồi biết bật dậy. Là tạo dựng những thói quen tốt.

Là hiểu rõ bộ công cụ của mình. Viết mã chỉ là một phần trong thế giới của một lập trình viên, và cuốn sách này khám phá cái thế giới ấy.

I spend a lot of time thinking about the coding journey. I didn't grow up coding; I didn't study it in college. I didn't spend my teenage years tinkering with tech. I entered the coding world in my mid-twenties and had to learn what it meant to be a programmer. This community is very different from others I'd been a part of. There is a unique dedication to learning and practicality that is both refreshing and intimidating.

Tôi dành nhiều thời gian suy nghĩ về hành trình học lập trình. Tôi không lớn lên cùng việc code; tôi không học nó ở đại học. Tôi không dành những năm tuổi teen để mày mò công nghệ. Tôi bước vào thế giới lập trình ở quãng giữa tuổi đôi mươi và phải học xem làm một lập trình viên nghĩa là gì. Cộng đồng này rất khác những cộng đồng tôi từng thuộc về. Có một sự tận tụy độc đáo với việc học hỏi và tính thực tiễn, vừa khiến ta sáng khoái vừa khiến ta e dè.

For me, it really does feel like entering a new world. A new town, at least. I had to get to know the neighbors, pick my grocery store, find the best coffee shops. It took a while to get the lay of the land, to find the most efficient routes, to avoid the streets with the heaviest traffic, to know when traffic was likely to hit. The weather is different, I needed a new wardrobe.

Với tôi, nó thực sự giống như bước vào một thế giới mới. Hay ít nhất là một thị trấn mới. Tôi phải làm quen hàng xóm, chọn cửa hàng tạp hóa, tìm những quán cà phê ngon nhất. Phải mất một thời gian để nắm được địa thế, tìm ra những tuyến đường hiệu quả nhất, né những con phố kẹt xe nhất, biết khi nào thì hay tắc đường. Thời tiết khác đi, tôi cần một tủ quần áo mới.

The first few weeks, even months, in a new town can be scary. Wouldn't it be wonderful to have a friendly, knowledgeable neighbor who'd been living there a while? Who can give you a tour, show you those coffee shops? Someone who'd been there long enough to know the culture, understand the pulse of the town, so you not only feel at home, but become a contributing member as well? Dave and Andy are those neighbors.

Vài tuần đầu, thậm chí vài tháng đầu, ở một thị trấn mới có thể đáng sợ. Chẳng phải sẽ thật tuyệt nếu có một người hàng xóm thân thiện, am hiểu, đã sống ở đó một thời gian hay sao? Một người có thể dẫn bạn đi một vòng, chỉ cho bạn những quán cà phê đó? Một người đã ở đủ lâu để hiểu văn hóa, nắm được nhịp đập của thị trấn, để bạn không chỉ thấy như ở nhà mà còn trở thành một thành viên đóng góp? Dave và Andy chính là những người hàng xóm ấy.

As a relative newcomer, it's easy to be overwhelmed not by the

Là một người tương đối mới, ta dễ bị choáng ngợp không phải bởi

act of programming but the **process** of becoming a programmer. There is an entire mindset shift that needs to happen—a change in habits, behaviors, and expectations. The process of becoming a better programmer doesn't just happen because you know how to code; it must be met with intention and deliberate practice. This book is a guide to becoming a better programmer efficiently.

việc lập trình mà bởi quá trình trở thành một lập trình viên. Có cả một sự chuyển dịch tư duy cần diễn ra — một thay đổi trong thói quen, hành vi và kỳ vọng. Quá trình trở thành một lập trình viên giỏi hơn không tự xảy ra chỉ vì bạn biết code; nó phải được đáp lại bằng chủ đích và sự luyện tập có chủ ý. Cuốn sách này là kim chỉ nam để trở thành lập trình viên giỏi hơn một cách hiệu quả.

But make no mistake—it doesn’t tell you how programming should be. It’s not philosophical or judgmental in that way. It tells you, plain and simple, what a **Pragmatic** Programmer is— how they operate, and how they approach code. They leave it up to you to decide if you want to be one. If you feel it’s not for you, they won’t hold it against you. But if you decide it is, they’re your friendly neighbors, there to show you the way.

Nhưng đừng nhầm — nó không bảo bạn lập trình nên thế nào. Nó không triết lý hay phán xét theo kiểu đó. Nó nói với bạn, giản dị và rõ ràng, một Lập trình viên Thực dụng (Pragmatic Programmer) là gì — họ vận hành ra sao, và họ tiếp cận mã nguồn thế nào. Họ để bạn tự quyết có muốn trở thành một người như thế hay không. Nếu bạn thấy điều đó không hợp với mình, họ chẳng trách bạn đâu. Nhưng nếu bạn quyết là có, họ chính là những người hàng xóm thân thiện, sẵn sàng chỉ đường cho bạn.

Saron Yitbarek

Saron Yitbarek

Founder & CEO of CodeNewbie

Nhà sáng lập & CEO của CodeNewbie

Host of Command Line Heroes

Người dẫn chương trình Command Line Heroes

Copyright © 2020 Pearson Education, Inc.

Copyright © 2020 Pearson Education, Inc.

Back in the 1990s, we worked with companies whose projects were having problems. We found ourselves saying the same things to each: maybe you should test that before you ship it; why does the code only build on Mary’s machine? Why didn’t anyone ask the users?

Hồi những năm 1990, chúng tôi làm việc với các công ty có những dự án đang gặp trục trặc. Chúng tôi thấy mình cứ nói đi nói lại cùng những điều với từng bên: hay là bạn nên kiểm thử cái đó trước khi giao đi; tại sao mã chỉ build được trên máy của Mary thôi? Sao chẳng ai hỏi người dùng cả?

To save time with new clients, we started jotting down notes. And those notes became The Pragmatic Programmer . To our surprise the book seemed to strike a chord, and it has continued to be popular these last 20 years.

Để tiết kiệm thời gian với các khách hàng mới, chúng tôi bắt đầu ghi chú lại. Và những ghi chú ấy trở thành The Pragmatic Programmer. Thật bất ngờ, cuốn sách dường như chạm đúng mạch, và nó vẫn được ưa chuộng suốt 20 năm qua.

But 20 years is many lifetimes in terms of software. Take a developer from 1999 and drop them into a team today, and they’d struggle in this strange new world. But the world of the 1990s is equally foreign to today’s developer. The book’s references to things such as CORBA, CASE tools, and indexed loops were at best quaint and more likely confusing.

Nhưng 20 năm là cả nhiều kiếp người xét theo thước đo phần mềm. Lấy một lập trình viên từ năm 1999 thả vào một nhóm ngày nay, hẳn họ sẽ vật lộn trong cái thế giới mới lạ lùng này. Nhưng thế giới của những năm 1990 cũng xa lạ y như vậy với một lập trình viên ngày nay. Những tham chiếu trong sách tới các thứ như CORBA, công cụ CASE, và vòng lặp đánh chỉ số (indexed loop) giỏi lắm thì cổ kính, mà nhiều khả năng là khó hiểu.

At the same time, 20 years has had no impact whatsoever on common sense. Technology may have changed, but people haven't. Practices and approaches that were a good idea then remain a good idea now. Those aspects of the book aged well.

Đồng thời, 20 năm chẳng tác động gì đến lẽ thường (common sense) cả. Công nghệ có thể đã đổi thay, nhưng con người thì không. Những thực hành và cách tiếp cận từng là ý hay khi đó thì nay vẫn là ý hay. Những khía cạnh đó của cuốn sách vẫn già đi rất đẹp.

So when it came time to create this 20 Anniversary Edition, th

Vậy nên khi đến lúc làm Ấn bản Kỷ niệm 20 năm này,

we had to make a decision. We could go through and update the technologies we reference and call it a day. Or we could reexamine the assumptions behind the practices we recommended in the light of an additional two decades' worth of experience.

chúng tôi phải đưa ra một quyết định. Chúng tôi có thể rà soát và cập nhật các công nghệ mình tham chiếu rồi gọi thế là xong. Hoặc chúng tôi có thể xem xét lại những giả định đằng sau các thực hành mình từng khuyến nghị, dưới ánh sáng của thêm hai thập kỷ kinh nghiệm.

In the end, we did both.

Cuối cùng, chúng tôi làm cả hai.

As a result, this book is something of a Ship of Theseus . [1] Roughly one-third of the topics in the book are brand new. Of the rest, the majority have been rewritten, either partially or totally. Our intent was to make things clearer, more relevant, and hopefully somewhat timeless.

Kết quả là cuốn sách này phần nào giống Con tàu của Theseus (Ship of Theseus). [1] Khoảng một phần ba số mục trong sách là hoàn toàn mới. Trong phần còn lại, đa số đã được viết lại, một phần hoặc toàn bộ. Ý định của chúng tôi là làm cho mọi thứ rõ ràng hơn, sát thực hơn, và hy vọng phần nào vượt thời gian.

We made some difficult decisions. We dropped the Resources appendix, both because it would be impossible to keep up-to-date and because it's easier to search for what you want. We reorganized and rewrote topics to do with **concurrency**, given the current abundance of parallel hardware and the dearth of good ways of dealing with it. We added content to reflect changing attitudes and environments, from the **agile** movement which we helped launch, to the rising acceptance of functional programming idioms and the growing need to consider privacy and security.

Chúng tôi đã có vài quyết định khó khăn. Chúng tôi bỏ phụ lục Tài nguyên (Resources), vừa vì sẽ không tài nào cập nhật kịp, vừa vì giờ đây tìm kiếm thứ mình muốn dễ hơn nhiều. Chúng tôi tổ chức lại và viết lại các mục liên quan đến xử lý đồng thời (concurrency), trước sự dồi dào hiện nay của phần cứng song song và sự khan hiếm những cách hay để xoay sở với nó. Chúng tôi bổ sung nội dung phản ánh những thay đổi trong thái độ và môi trường, từ phong trào agile mà chính chúng tôi góp phần khởi xướng, đến sự chấp nhận ngày càng tăng của các thành ngữ lập trình hàm (functional programming), và nhu cầu ngày càng lớn về việc cân nhắc quyền riêng tư và bảo mật.

Interestingly, though, there was considerably less debate between us on the content of this edition than there was when we wrote the first. We both felt that the stuff that was important was easier to identify.

Điều thú vị là, giữa chúng tôi lại có ít tranh luận về nội dung của ấn bản này hơn nhiều so với khi viết ấn bản đầu. Cả hai chúng tôi đều cảm thấy những thứ thực sự quan trọng giờ đây dễ nhận ra hơn.

Anyway, this book is the result. Please enjoy it. Maybe adopt some new practices. Maybe decide that some of the stuff we suggest is wrong. Get involved in your craft. Give us feedback.

Dù sao thì, cuốn sách này chính là kết quả. Mong bạn thưởng thức nó. Có thể bạn sẽ áp dụng vài thực hành mới. Có thể bạn sẽ kết luận rằng vài điều chúng tôi gợi ý là sai. Hãy dẫn thân vào nghề của bạn. Hãy gửi phản hồi cho chúng tôi.

But, most important, remember to make it fun.

Nhưng, quan trọng nhất, đừng quên làm cho nó vui.

How the Book Is Organized

Cuốn sách được tổ chức như thế nào

This book is written as a collection of short topics. Each **topic** is self-contained, and addresses a particular theme. You'll find numerous cross references, which help put each topic in **context**. Feel free to read the topics in any order—this isn't a book you need to read front-to-back.

Cuốn sách này được viết thành một tập hợp các mục ngắn. Mỗi mục (topic) là độc lập, và đề cập một chủ đề riêng. Bạn sẽ thấy vô số tham chiếu chéo, giúp đặt mỗi mục vào ngữ cảnh. Bạn cứ thoải mái đọc các mục theo bất kỳ thứ tự nào — đây không phải cuốn sách phải đọc tuần tự từ đầu đến cuối.

Occasionally you'll come across a box labeled **Tip** nn (such as Tip 1, Care About Your Craft). As well as emphasizing points in the text, we feel the tips have a life of their own—we live by them daily. You'll find a summary of all the tips on a pull-out card inside the back cover.

Thi thoảng bạn sẽ bắt gặp một khung gắn nhãn Tip nn (chẳng hạn Tip 1, Hãy Quan Tâm Đến Nghề Của Bạn). Bên cạnh việc nhấn mạnh các ý trong văn bản, chúng tôi cảm thấy các mẹo có một đời sống riêng của chúng — chúng tôi sống theo chúng mỗi ngày. Bạn sẽ thấy bản tóm tắt toàn bộ các mẹo trên một tấm thẻ rút ra được ở mặt trong bìa sau.

We've included **exercises** and **challenges** where appropriate. Exercises normally have relatively straightforward answers, while the challenges are more open-ended. To give you an idea of our thinking, we've included our answers to the exercises in an appendix, but very few have a single correct solution. The challenges might form the basis of group discussions or essay work in advanced programming courses.

Chúng tôi cũng đưa vào các bài tập và thử thách ở những chỗ phù hợp. Bài tập thường có đáp án khá rõ ràng, đơn giản, còn thử thách thì mở hơn. Để bạn hình dung cách chúng tôi nghĩ, chúng tôi đã đưa lời giải các bài tập vào một phụ lục, nhưng rất ít bài có một lời giải đúng duy nhất. Các thử thách có thể làm nền cho những buổi thảo luận nhóm hoặc các bài luận trong các khóa lập trình nâng cao.

There's also a short bibliography listing the books and articles we explicitly reference.

Còn có một thư mục tài liệu ngắn liệt kê những sách và bài viết mà chúng tôi tham chiếu trực tiếp.

What's in a Name?

Cái Tên Có Gì?

Scattered throughout the book you'll find various bits of jargon —either perfectly good English words that have been corrupted to mean something technical, or horrendous made-up words that have been assigned meanings by computer scientists with a grudge against the language. The first time we use each of these jargon words, we try to define it, or at least give a hint to its meaning. However, we're sure that some have fallen through the cracks, and others, such as object and relational database,

are in common enough usage that adding a definition would be boring. If you do come across a term you haven't seen before, please don't just skip over it. Take time to look it up, perhaps on the web, or maybe in a computer science textbook. And, if you get a chance, drop us an email and complain, so we can add a definition to the next edition.

Rải rác khắp cuốn sách bạn sẽ thấy đủ thứ biệt ngữ (jargon) — hoặc là những từ tiếng Anh hoàn toàn tử tế đã bị bóp méo để mang một nghĩa kỹ thuật, hoặc là những từ bịa ra kinh khủng được các nhà khoa học máy tính có hiềm khích với ngôn ngữ gán cho ý nghĩa. Lần đầu dùng mỗi từ biệt ngữ này, chúng tôi cố định nghĩa nó, hoặc ít nhất gợi ý nghĩa của nó. Tuy vậy, chắc chắn có vài từ đã lọt khe, và những từ khác — như object (đối tượng) và cơ sở dữ liệu quan hệ (relational database) — thì đã thông dụng đến mức thêm định nghĩa cũng chán. Nếu bạn gặp một thuật ngữ chưa từng thấy, xin đừng cứ thế lướt qua. Hãy dành thời gian tra nó, có thể trên web, hoặc trong một giáo trình khoa học máy tính. Và, nếu có dịp, hãy gửi email cho chúng tôi mà phàn nàn, để chúng tôi thêm định nghĩa vào ấn bản sau.

Having said all this, we decided to get revenge against the computer scientists. Sometimes, there are perfectly good jargon words for concepts, words that we've decided to ignore. Why? Because the existing jargon is normally restricted to a particular problem domain, or to a particular phase of development. However, one of the basic philosophies of this book is that most of the techniques we're recommending are universal: modularity applies to code, designs, documentation, and team organization, for instance. When we wanted to use the conventional jargon word in a broader context, it got confusing — we couldn't seem to overcome the baggage the original term brought with it. When this happened, we contributed to the decline of the language by inventing our own terms.

Nói tất cả những điều đó rồi, chúng tôi quyết định trả thù các nhà khoa học máy tính. Đôi khi, đã có sẵn những từ biệt ngữ hoàn toàn tử tế cho một khái niệm, những từ mà chúng tôi quyết định phớt lờ. Vì sao? Bởi biệt ngữ sẵn có thường bị bó hẹp trong một miền bài toán cụ thể, hoặc một giai đoạn phát triển cụ thể. Tuy nhiên, một trong những triết lý nền tảng của cuốn sách này là phần lớn các kỹ thuật chúng tôi khuyến nghị đều mang tính phổ quát: chẳng hạn, tính mô-đun (modularity) áp dụng cho mã, cho thiết kế, cho tài liệu, và cho cả cách tổ chức nhóm. Khi chúng tôi muốn dùng từ biệt ngữ quy ước trong một ngữ cảnh rộng hơn, nó trở nên khó hiểu — chúng tôi dường như không vượt qua nổi cái gánh nặng mà thuật ngữ gốc mang theo. Mỗi khi điều đó xảy ra, chúng tôi lại góp phần làm suy đồi ngôn ngữ bằng cách bịa ra thuật ngữ của riêng mình.

Source Code and Other Resources

Mã Nguồn Và Các Tài Nguyên Khác

Most of the code shown in this book is extracted from compilable source files, available for download from our website. [2]

Hầu hết mã hiển thị trong sách này được trích từ các tệp mã nguồn biên dịch được, có thể tải xuống từ website của chúng tôi. [2]

There you'll also find links to resources we find useful, along with updates to the book and news of other Pragmatic Programmer developments.

Ở đó bạn cũng sẽ thấy các liên kết tới những tài nguyên chúng tôi thấy hữu ích, cùng với các bản cập nhật cho cuốn sách và tin tức về những phát triển khác của Pragmatic Programmer.

Send Us Feedback

Hãy Gửi Phản Hồi Cho Chúng Tôi

We'd appreciate hearing from you. Email us at ppbook@pragprog.com.

Chúng tôi rất mong được nghe từ bạn. Hãy email cho chúng tôi tại ppbook@pragprog.com.

Second Edition Acknowledgments

Lời Cảm Ơn Cho Ấn Bản Thứ Hai

We have enjoyed literally thousands of interesting conversations about programming over the last 20 years, meeting people at conferences, at courses, and sometimes even on the plane. Each one of these has added to our understanding of the development process, and has contributed to the updates in this edition. Thank you all (and keep telling us when we're wrong).

Suốt 20 năm qua chúng tôi đã có theo nghĩa đen là hàng nghìn cuộc trò chuyện thú vị về lập trình, gặp gỡ mọi người ở các hội nghị, các khóa học, và đôi khi ngay cả trên máy bay. Mỗi cuộc trong số đó đều bồi đắp thêm cho hiểu biết của chúng tôi về quá trình phát triển phần mềm, và đã góp phần vào những cập nhật trong ấn bản này. Cảm ơn tất cả các bạn (và cứ tiếp tục chỉ ra mỗi khi chúng tôi sai nhé).

Thanks to the participants in the book's beta process. Your questions and comments helped us explain things better.

Cảm ơn những người tham gia quá trình beta của cuốn sách. Những câu hỏi và bình luận của các bạn đã giúp chúng tôi giải thích mọi thứ tốt hơn.

Before we went beta, we shared the book with a few folks for comments. Thanks to VM (Vicky) Brasseur, Jeff Langr, and Kim Shrier for your detailed comments, and to José Valim and Nick Cuthbert for your technical reviews.

Trước khi vào beta, chúng tôi đã chia sẻ cuốn sách với một vài người để xin ý kiến. Cảm ơn VM (Vicky) Brasseur, Jeff Langr, và Kim Shrier vì những bình luận chi tiết của các bạn, và cảm ơn José Valim cùng Nick Cuthbert vì các bản đánh giá kỹ thuật.

Thanks to Ron Jeffries for letting us use the Sudoku example.

Cảm ơn Ron Jeffries đã cho chúng tôi dùng ví dụ Sudoku.

Much gratitude to the folks at Pearson who agreed to let us create this book our way.

Vô cùng biết ơn những con người ở Pearson đã đồng ý để chúng tôi làm cuốn sách này theo cách của riêng mình.

A special thanks to the indispensable Janet Furlow, who masters whatever she takes on and keeps us in line.

Một lời cảm ơn đặc biệt dành cho Janet Furlow không thể thiếu được, người làm chủ mọi việc mình nhận và luôn giữ chúng tôi vào khuôn phép.

And, finally, a shout out to all the Pragmatic Programmers out there who have been making programming better for everyone for the last twenty years. Here's to twenty more.

Và, cuối cùng, xin gửi lời chào đến tất cả các Lập trình viên Thực dụng ngoài kia, những người suốt hai mươi năm qua đã làm cho việc lập trình tốt đẹp hơn cho mọi người. Chúc mừng hai mươi năm nữa.

Footnotes

Chú thích

[1] If, over the years, every component of a ship is replaced as it fails, is the resulting vessel the same ship?

[1] Nếu, qua nhiều năm, từng bộ phận của một con tàu đều được thay thế khi hỏng, thì con tàu cuối cùng có còn là con tàu cũ không?

[2] <https://pragprog.com/titles/tpp20>

[2] <https://pragprog.com/titles/tpp20>

Copyright © 2020 Pearson Education, Inc.

Copyright © 2020 Pearson Education, Inc.

This book will help you become a better programmer.

Cuốn sách này sẽ giúp bạn trở thành một lập trình viên giỏi hơn.

You could be a lone developer, a member of a large project team, or a consultant working with many clients at once. It doesn't matter; this book will help you, as an individual, to do better work. This book isn't theoretical—we concentrate on practical topics, on using your experience to make more informed decisions. The word pragmatic comes from the Latin *pragmaticus* —“skilled in business”—which in turn is derived from the Greek *πραγματικός*, meaning “fit for use.”

Bạn có thể là một lập trình viên đơn độc, một thành viên của một nhóm dự án lớn, hay một chuyên gia tư vấn làm việc với nhiều khách hàng cùng lúc. Điều đó không quan trọng; cuốn sách này sẽ giúp bạn, với tư cách một cá nhân, làm việc tốt hơn. Cuốn sách này không mang tính lý thuyết — chúng tôi tập trung vào những chủ đề thiết thực, vào việc dùng kinh nghiệm của bạn để ra những quyết định sáng suốt hơn. Từ *pragmatic* (thực dụng) bắt nguồn từ tiếng Latin *pragmaticus* — “lành nghề trong công việc” — vốn lại bắt nguồn từ tiếng Hy Lạp *πραγματικός*, nghĩa là “thích hợp để dùng”.

This is a book about doing.

Đây là một cuốn sách về việc làm.

Programming is a craft. At its simplest, it comes down to getting a computer to do what you want it to do (or what your user wants it to do). As a programmer, you are part listener, part advisor, part interpreter, and part dictator. You try to capture elusive **requirements** and find a way of expressing them so that a mere machine can do them justice. You try to document your work so that others can understand it, and you try to engineer your work so that others can build on it. What's more, you try to do all this against the relentless ticking of the project clock. You

Lập trình là một nghề thủ công (craft). Đơn giản nhất, nó quy về việc khiến một chiếc máy tính làm điều bạn muốn nó làm (hoặc điều người dùng của bạn muốn nó làm). Là một lập trình viên, bạn một phần là người lắng nghe, một phần là người cố vấn, một phần là người thông dịch, và một phần là kẻ độc tài. Bạn cố nắm bắt những yêu cầu mơ hồ và tìm cách diễn đạt chúng sao cho một cỗ máy thuần túy có thể thực thi cho đến nơi đến chốn. Bạn cố ghi chép công việc của mình để người khác có thể hiểu, và bạn cố thiết kế công việc của mình để người khác có thể xây dựng tiếp lên đó. Hơn nữa, bạn cố làm tất cả những điều này trong tiếng tích tắc không ngừng của chiếc đồng hồ dự án. Bạn

work small miracles every day.

làm những phép màu nhỏ mỗi ngày.

It's a difficult job.

Đó là một công việc khó nhằn.

There are many people offering you help. Tool vendors tout the miracles their products perform. Methodology gurus promise that their techniques guarantee results. Everyone claims that their

programming language is the best, and every operating system is the answer to all conceivable ills.

Có rất nhiều người sẵn sàng giúp bạn. Các nhà bán công cụ rao giảng về những phép màu mà sản phẩm của họ làm được. Các bậc thầy phương pháp luận hứa hẹn kỹ thuật của họ bảo đảm kết quả. Ai cũng tuyên bố ngôn ngữ lập trình của mình là hay nhất, và mọi hệ điều hành đều là lời giải cho mọi căn bệnh có thể hình dung được.

Of course, none of this is true. There are no easy answers. There is no best solution, be it a tool, a language, or an operating system. There can only be systems that are more appropriate in a particular set of circumstances.

Tất nhiên, chẳng điều nào trong số đó là thật. Không có câu trả lời dễ dàng nào cả. Không có lời giải tốt nhất, dù là công cụ, ngôn ngữ, hay hệ điều hành. Chỉ có thể có những hệ thống phù hợp hơn trong một tập hợp hoàn cảnh cụ thể.

This is where pragmatism comes in. You shouldn't be wedded to any particular technology, but have a broad enough background and experience base to allow you to choose good solutions in particular situations. Your background stems from an understanding of the basic principles of computer science, and your experience comes from a wide range of practical projects. Theory and practice combine to make you strong.

Đây chính là chỗ chủ nghĩa thực dụng nhập cuộc. Bạn không nên gắn chặt mình với bất kỳ công nghệ cụ thể nào, mà phải có nền tảng và vốn kinh nghiệm đủ rộng để có thể chọn những lời giải tốt trong từng tình huống cụ thể. Nền tảng của bạn xuất phát từ việc hiểu các nguyên lý cơ bản của khoa học máy tính, còn kinh nghiệm của bạn đến từ một phạm vi rộng các dự án thực tế. Lý thuyết và thực hành kết hợp lại làm nên sức mạnh của bạn.

You adjust your approach to suit the current circumstances and environment. You judge the relative importance of all the factors affecting a project and use your experience to produce appropriate solutions. And you do this continuously as the work progresses. Pragmatic Programmers get the job done, and do it well.

Bạn điều chỉnh cách tiếp cận của mình cho hợp với hoàn cảnh và môi trường hiện tại. Bạn cân nhắc tầm quan trọng tương đối của mọi yếu tố ảnh hưởng đến một dự án và dùng kinh nghiệm để tạo ra những lời giải phù hợp. Và bạn làm điều này liên tục khi công việc tiến triển. Lập trình viên Thực dụng hoàn thành công việc, và làm tốt.

Who Should Read This Book?

Ai Nên Đọc Cuốn Sách Này?

This book is aimed at people who want to become more effective and more productive programmers. Perhaps you feel frustrated that you don't seem to be achieving your potential. Perhaps you look at colleagues who seem to be using tools to make themselves more productive than you. Maybe your current job uses older technologies, and you want to know how newer ideas can be applied to what you do.

Cuốn sách này hướng đến những người muốn trở thành lập trình viên hiệu quả hơn và năng suất hơn. Có lẽ bạn thấy bức bối vì dường như mình chưa phát huy được hết tiềm năng. Có lẽ bạn nhìn các đồng nghiệp dường như đang dùng công cụ để làm cho mình năng suất hơn bạn. Có thể công việc hiện tại của bạn dùng những công nghệ cũ hơn, và bạn muốn biết những ý tưởng mới hơn có thể áp dụng vào việc bạn làm như thế nào.

We don't pretend to have all (or even most) of the answers, nor are all of our ideas applicable in all situations. All we can say is that if you follow our approach, you'll gain experience rapidly, your productivity will increase, and you'll have a better understanding of the entire development process. And you'll write better software.

Chúng tôi không tỏ vẻ có tất cả (hay thậm chí phần lớn) câu trả lời, và cũng không phải mọi ý tưởng của chúng tôi đều áp dụng được trong mọi tình huống. Tất cả những gì chúng tôi có thể nói là nếu bạn theo cách tiếp cận của chúng tôi, bạn sẽ tích lũy kinh nghiệm nhanh chóng, năng suất của bạn sẽ tăng, và bạn sẽ hiểu rõ hơn toàn bộ quá trình phát triển phần mềm. Và bạn sẽ viết phần mềm tốt hơn.

What Makes a Pragmatic Programmer?

Điều Gì Làm Nên Một Lập Trình Viên Thực Dụng?

Each developer is unique, with individual strengths and weaknesses, preferences and dislikes. Over time, each will craft their own personal environment. That environment will reflect the programmer's individuality just as forcefully as his or her hobbies, clothing, or haircut. However, if you're a Pragmatic Programmer, you'll share many of the following characteristics:

Mỗi lập trình viên là độc nhất, với những điểm mạnh và điểm yếu, những sở thích và những điều ghét bỏ riêng. Theo thời gian, mỗi người sẽ tạo dựng môi trường cá nhân của riêng mình. Môi trường đó sẽ phản chiếu cá tính của lập trình viên mạnh mẽ chẳng kém gì sở thích, quần áo, hay kiểu tóc của họ. Tuy nhiên, nếu bạn là một Lập trình viên Thực dụng, bạn sẽ có chung nhiều đặc điểm dưới đây:

Early adopter/fast adapter You have an instinct for technologies and techniques, and you love trying things out. When given something new, you can grasp it quickly and integrate it with the rest of your knowledge. Your confidence is born of experience.

Người tiên phong/người thích nghi nhanh Bạn có bản năng với công nghệ và kỹ thuật, và bạn thích thử nghiệm các thứ. Khi được giao một thứ mới, bạn nắm bắt nhanh và tích hợp nó với phần còn lại trong kiến thức của mình. Sự tự tin của bạn sinh ra từ kinh nghiệm.

Inquisitive You tend to ask questions. That's neat—how did you do that? Did you have problems with that library? What's this quantum computing I've heard about? How are symbolic links implemented? You are a pack rat for little facts, each of which may affect some decision years from now.

Tò mò Bạn hay đặt câu hỏi. Hay đấy — bạn làm cái đó thế nào vậy? Bạn có gặp vấn đề gì với thư viện đó không? Cái máy tính lượng tử mà tôi nghe nói là gì thế? Liên kết tượng trưng (symbolic link) được hiện thực ra sao? Bạn là một kẻ tích góp những sự thật nhỏ nhặt, mỗi mẫu trong đó đều có thể ảnh hưởng đến một quyết định nào đó nhiều năm sau này.

Critical thinker You rarely take things as given without first getting the facts. When colleagues say “because that's the way it's done,” or a vendor promises the solution to all your problems, you smell a challenge.

Người tư duy phản biện Bạn hiếm khi chấp nhận mọi thứ như đã rồi mà chưa kiểm chứng sự thật trước. Khi đồng nghiệp nói “vì người ta vẫn làm thế”, hoặc một nhà bán hàng hứa hẹn giải pháp cho mọi vấn đề của bạn, bạn ngửi thấy mùi một thử thách.

Realistic You try to understand the underlying nature of each problem you face. This realism gives you a good feel for

Thực tế Bạn cố hiểu bản chất nằm dưới mỗi vấn đề mình đối mặt. Sự thực tế này cho bạn cảm nhận tốt về

how difficult things are, and how long things will take. Deeply understanding that a process should be difficult or will take a while to complete gives you the stamina to keep at it.

mọi thứ khó đến đâu, và sẽ mất bao lâu. Hiểu sâu rằng một quá trình sẽ khó hoặc sẽ mất một thời gian mới xong cho bạn sức bền để bám trụ với nó.

Jack of all trades You try hard to be familiar with a broad range of technologies and environments, and you work to keep abreast of new developments. Although your current job may require you to be a specialist, you will always be able to move on to new areas and new challenges.

Người biết tuốt Bạn nỗ lực để quen thuộc với một phạm vi rộng các công nghệ và môi trường, và bạn chịu khó cập nhật những phát triển mới. Mặc dù công việc hiện tại có thể đòi hỏi bạn là một chuyên gia, bạn sẽ luôn có thể chuyển sang những lĩnh vực mới và những thử thách mới.

We've left the most basic characteristics until last. All Pragmatic Programmers share them. They're basic enough to **state** as tips:

Chúng tôi để dành những đặc điểm cơ bản nhất đến cuối cùng. Mọi Lập trình viên Thực dụng đều có chung chúng. Chúng đủ cơ bản để được phát biểu thành các mẹo:

Tip 1: Care About Your Craft

Tip 1: Hãy Quan Tâm Đến Nghề Của Bạn

We feel that there is no point in developing software unless you care about doing it well.

Chúng tôi cảm thấy chẳng có ý nghĩa gì khi phát triển phần mềm nếu bạn không quan tâm đến việc làm nó cho tốt.

Tip 2: Think! About Your Work

Tip 2: Hãy Suy Nghĩ! Về Công Việc Của Bạn

In order to be a Pragmatic Programmer, we're challenging you to think about what you're doing while you're doing it. This isn't a one-time audit of current practices—it's an ongoing critical appraisal of every decision you make, every day, and on every project. Never run on auto-pilot. Constantly be thinking, critiquing your work in real time. The old IBM corporate motto, THINK! , is the Pragmatic Programmer's mantra.

Để trở thành một Lập trình viên Thực dụng, chúng tôi thách bạn suy nghĩ về điều mình đang làm ngay trong lúc làm nó. Đây không phải một cuộc kiểm toán một lần các thực hành hiện tại — mà là một sự đánh giá phản biện liên tục với mọi quyết định bạn đưa ra, mỗi ngày, và trên mọi dự án. Đừng bao giờ chạy bằng chế độ lái tự động. Hãy không ngừng suy nghĩ, phê phán công việc của mình theo thời gian thực. Câu khẩu hiệu cũ của tập đoàn IBM, THINK! (HÃY SUY NGHĨ!), chính là câu thần chú của Lập trình viên Thực dụng.

If this sounds like hard work to you, then you're exhibiting the realistic characteristic. This is going to take up some of your

Nếu điều này nghe có vẻ là việc vất vả với bạn, thì bạn đang thể hiện đặc điểm thực tế đấy. Việc này sẽ tốn của bạn một phần

valuable time—time that is probably already under tremendous pressure. The reward is a more active involvement with a job you love, a feeling of mastery over an increasing range of subjects, and pleasure in a feeling of continuous improvement. Over the long term, your time investment will be repaid as you and your team become more efficient, write code that's easier to maintain, and spend less time in meetings.

thời gian quý giá — thời gian mà có lẽ vốn đã chịu áp lực khủng khiếp rồi. Phần thưởng là sự gắn bó chủ động hơn với một công việc bạn yêu thích, cảm giác làm chủ một phạm vi chủ đề ngày càng rộng, và niềm vui trong cảm giác không ngừng tiến bộ. Về dài hạn,

khoản đầu tư thời gian của bạn sẽ được hoàn lại khi bạn và nhóm của mình trở nên hiệu quả hơn, viết mã dễ bảo trì hơn, và dành ít thời gian họp hành hơn.

Individual Pragmatists, Large Teams

Những Người Thực Dụng Đơn Lẻ, Những Nhóm Lớn

Some people feel that there is no room for individuality on large teams or complex projects. “Software is an engineering discipline,” they say, “that breaks down if individual team members make decisions for themselves.”

Một số người cảm thấy không có chỗ cho cá tính trong các nhóm lớn hay những dự án phức tạp. “Phần mềm là một bộ môn kỹ thuật,” họ nói, “sẽ đổ vỡ nếu từng thành viên trong nhóm tự ra quyết định cho mình.”

We strongly disagree.

Chúng tôi cực lực phản đối.

There should be engineering in software construction. However, this doesn’t preclude individual craftsmanship. Think about the large cathedrals built in Europe during the Middle Ages. Each took thousands of person-years of effort, spread over many decades. Lessons learned were passed down to the next set of builders, who advanced the state of structural engineering with their accomplishments. But the carpenters, stonecutters, carvers, and glass workers were all craftspeople, interpreting the engineering requirements to produce a whole that transcended the purely mechanical side of the construction. It was their belief in their individual contributions that sustained the projects: We who cut mere stones must always be envisioning cathedrals.

Cần phải có yếu tố kỹ thuật trong việc xây dựng phần mềm. Tuy nhiên, điều đó không loại trừ tay nghề thủ công của từng cá nhân. Hãy nghĩ về những thánh đường lớn được xây dựng ở châu Âu thời Trung cổ. Mỗi công trình tốn hàng nghìn người-năm công sức, trải dài qua nhiều thập kỷ. Những bài học rút ra được truyền lại cho lớp thợ xây kế tiếp, những người lại đẩy tới trạng thái của kỹ thuật kết cấu bằng các thành tựu của mình. Nhưng những người thợ mộc, thợ đẽo đá, thợ chạm khắc, và thợ kính đều là những nghệ nhân, diễn giải các yêu cầu kỹ thuật để tạo nên một tổng thể vượt lên trên khía cạnh thuần cơ học của công trình. Chính niềm tin vào sự đóng góp riêng của mình đã nâng đỡ các dự án ấy: Chúng ta, những kẻ chỉ đẽo những hòn đá đơn thuần, phải luôn hình dung ra những thánh đường.

Within the overall structure of a project there is always room for individuality and craftsmanship. This is particularly true given the **current state** of software engineering. One hundred years from now, our engineering may seem as archaic as the techniques used by medieval cathedral builders seem to today’s civil engineers, while our craftsmanship will still be honored.

Bên trong cấu trúc tổng thể của một dự án luôn có chỗ cho cá tính và tay nghề thủ công. Điều này đặc biệt đúng với trạng thái hiện tại của ngành kỹ nghệ phần mềm. Một trăm năm nữa, kỹ thuật của chúng ta có thể sẽ trông cổ lỗ chẳng khác gì những kỹ thuật mà các thợ xây thánh đường thời Trung cổ dùng so với các kỹ sư xây dựng ngày nay, trong khi tay nghề thủ công của chúng ta thì vẫn sẽ được tôn vinh.

It’s a Continuous Process

Đó Là Một Quá Trình Liên Tục

A tourist visiting England’s Eton College asked the gardener how he got the lawns so perfect. “That’s easy,” he replied, “You just brush off the dew every morning, mow them every other day, and roll them once a week.”

Một du khách đến thăm trường Eton College ở Anh hỏi người làm vườn làm sao ông ấy có được những bãi cỏ hoàn hảo đến thế. “Dễ thôi mà,” ông đáp, “Mỗi sáng cứ phủi sương đi, cách ngày thì cắt cỏ một lần, và mỗi tuần lăn cỏ một lần.”

“Is that all?” asked the tourist. “Absolutely,” replied the gardener. “Do that for 500 years and you’ll have a nice lawn, too.”

“Chỉ thế thôi sao?” du khách hỏi. “Đúng vậy,” người làm vườn đáp. “Cứ làm thế trong 500 năm thì anh cũng sẽ có một bãi cỏ đẹp thôi.”

Great lawns need small amounts of daily care, and so do great programmers. Management consultants like to drop the word kaizen in conversations. “Kaizen” is a Japanese term that captures the concept of continuously making many small improvements. It was considered to be one of the main reasons for the dramatic gains in productivity and quality in Japanese manufacturing and was widely copied throughout the world. Kaizen applies to individuals, too. Every day, work to refine the skills you have and to add new tools to your repertoire. Unlike the Eton lawns, you’ll start seeing results in a matter of days. Over the years, you’ll be amazed at how your experience has blossomed and how your skills have grown.

Những bãi cỏ tuyệt vời cần một chút chăm sóc mỗi ngày, và những lập trình viên tuyệt vời cũng vậy. Các nhà tư vấn quản lý thích thả từ kaizen vào các cuộc trò chuyện. “Kaizen” là một thuật ngữ tiếng Nhật nắm bắt khái niệm không ngừng thực hiện nhiều cải tiến nhỏ. Nó được xem là một trong những lý do chính cho những bước nhảy vọt ngoạn mục về năng suất và chất lượng trong ngành sản xuất Nhật Bản, và đã được sao chép rộng rãi khắp thế giới. Kaizen cũng áp dụng cho cá nhân. Mỗi ngày, hãy làm việc để mài giũa những kỹ năng bạn có và thêm những công cụ mới vào kho của mình. Không như những bãi cỏ ở Eton, bạn sẽ bắt đầu thấy kết quả chỉ sau vài ngày. Qua nhiều năm, bạn sẽ kinh ngạc khi thấy kinh nghiệm của mình đã nở rộ ra sao và kỹ năng của mình đã lớn mạnh thế nào.

Copyright © 2020 Pearson Education, Inc.

Copyright © 2020 Pearson Education, Inc.

A Pragmatic Philosophy — Một triết lý thực dụng

This book is about you.

Cuốn sách này nói về chính bạn.

Make no mistake, it is your career, and more importantly, **Topic 1, It's Your Life** . You own it. You're here because you know you can become a better developer and help others become better as well. You can become a **Pragmatic Programmer** .

Đừng nhầm lẫn, đây là sự nghiệp của bạn, và quan trọng hơn cả, như Mục 1, Đời Bạn Là Của Bạn đã nói. Bạn là chủ nhân của nó. Bạn ở đây vì bạn biết mình có thể trở thành một lập trình viên giỏi hơn, và giúp người khác cùng tiến bộ. Bạn có thể trở thành một Lập trình viên Thực dụng (Pragmatic Programmer).

What distinguishes **Pragmatic** Programmers? We feel it's an attitude, a style, a philosophy of approaching problems and their solutions. They think beyond the immediate problem, placing it in its larger **context** and seeking out the bigger picture. After all, without this larger context, how can you be pragmatic? How can you make intelligent compromises and informed decisions?

Điều gì làm nên một Lập trình viên Thực dụng? Chúng tôi cho rằng đó là một thái độ, một phong cách, một triết lý tiếp cận các vấn đề và lời giải cho chúng. Họ nghĩ xa hơn vấn đề trước mắt, đặt nó vào bối cảnh rộng lớn hơn và đi tìm bức tranh toàn cảnh. Bởi lẽ, thiếu cái bối cảnh rộng lớn đó thì làm sao bạn có thể thực dụng được? Làm sao bạn đưa ra được những thỏa hiệp khôn ngoan và những quyết định sáng suốt?

Another key to their success is that Pragmatic Programmers take responsibility for everything they do, which we discuss in Topic 2, **The Cat Ate My Source Code** . Being responsible, Pragmatic Programmers won't sit idly by and watch their projects fall apart through neglect. In Topic 3, **Software Entropy** , we tell you how to keep your projects pristine.

Một chìa khóa khác cho thành công của họ là Lập trình viên Thực dụng nhận trách nhiệm cho mọi việc mình làm, điều chúng ta sẽ bàn trong Mục 2, Con Mèo Đã Ăn Mất Mã Nguồn Của Tôi. Vì có tinh thần trách nhiệm, Lập trình viên Thực dụng sẽ không khoanh tay ngồi nhìn dự án của mình tan rã vì bị bỏ bê. Trong Mục 3, Sự Rối Loạn Phần Mềm (Software Entropy), chúng tôi sẽ chỉ cho bạn cách giữ cho dự án luôn nguyên vẹn.

Most people find change difficult, sometimes for good reasons,

Hầu hết mọi người đều thấy sự thay đổi là khó, đôi khi vì những lý do chính đáng,

sometimes because of plain old inertia. In Topic 4, **Stone Soup and Boiled Frogs** , we look at a strategy for instigating change and (in the interests of balance) present the cautionary tale of an

amphibian that ignored the dangers of gradual change.

đôi khi chỉ đơn thuần vì sức ỳ cố hữu. Trong Mục 4, Súp Đá Và Con Ếch Bị Luộc, chúng ta sẽ xem xét một chiến lược để khơi mào thay đổi và (để cho cân bằng) kể lại câu chuyện cảnh tỉnh về một chú lưỡng cư đã phớt lờ những hiểm họa của sự thay đổi từ từ.

One of the benefits of understanding the context in which you work is that it becomes easier to know just how good your software has to be. Sometimes near-perfection is the only option, but often there are trade-offs involved. We explore this in Topic 5, **Good-Enough Software**.

Một lợi ích của việc thấu hiểu bối cảnh nơi bạn làm việc là bạn sẽ dễ dàng biết được phần mềm của mình cần phải tốt đến mức nào. Đôi khi gần như hoàn hảo là lựa chọn duy nhất, nhưng thường thì sẽ có những đánh đổi. Chúng ta sẽ khám phá điều này trong Mục 5, Phần Mềm Đủ Tốt (Good-Enough Software).

Of course, you need to have a broad base of knowledge and experience to pull all of this off. Learning is a continuous and ongoing **process**. In Topic 6, **Your Knowledge Portfolio**, we discuss some strategies for keeping the momentum up.

Tất nhiên, để làm được tất cả những điều này, bạn cần có một nền tảng kiến thức và kinh nghiệm rộng. Học tập là một quá trình liên tục và không ngừng nghỉ. Trong Mục 6, Danh Mục Tri Thức Của Bạn (Knowledge Portfolio), chúng ta sẽ bàn về một vài chiến lược để duy trì đà tiến.

Finally, none of us works in a vacuum. We all spend a large amount of time interacting with others. Topic 7, **Communicate!** lists ways we can do this better.

Cuối cùng, không ai trong chúng ta làm việc trong chân không cả. Tất cả chúng ta đều dành rất nhiều thời gian để tương tác với người khác. Mục 7, Giao Tiếp! liệt kê những cách giúp ta làm việc đó tốt hơn.

Pragmatic programming stems from a philosophy of pragmatic thinking. This chapter sets the basis for that philosophy.

Lập trình thực dụng bắt nguồn từ một triết lý tư duy thực dụng. Chương này đặt nền móng cho triết lý ấy.

I'm not in this world to live up to your expectations and you're not in this world to live up to mine. Bruce Lee

Tôi không sống trên đời này để đáp ứng kỳ vọng của bạn, và bạn cũng không sống trên đời để đáp ứng kỳ vọng của tôi. Bruce Lee

Topic 1. It's Your Life — Topic 1. Đời Bạn Là Của Bạn

It is your life. You own it. You run it. You create it.

Đó là cuộc đời của bạn. Bạn sở hữu nó. Bạn điều khiển nó. Bạn tạo ra nó.

Many developers we talk to are frustrated. Their concerns are varied. Some feel they're stagnating in their job, others that technology has passed them by. Folks feel they are under appreciated, or underpaid, or that their teams are toxic. Maybe they want to move to Asia, or Europe, or work from home.

Rất nhiều lập trình viên mà chúng tôi trò chuyện đều đang bức bối. Những mối lo của họ muôn hình muôn vẻ. Người thì thấy mình đang giậm chân tại chỗ trong công việc, kẻ thì thấy công nghệ đã bỏ mình lại phía sau. Người ta cảm thấy mình không được trân trọng,

hay bị trả lương bèo bọt, hoặc đội nhóm của mình thì độc hại. Có thể họ muốn chuyển sang châu Á, sang châu Âu, hay làm việc tại nhà.

And the answer we give is always the same.

Và câu trả lời chúng tôi đưa ra luôn luôn giống nhau.

“Why can’t you change it?”

“Sao bạn không thay đổi nó đi?”

Software development must appear close to the top of any list of careers where you have control. Our skills are in demand, our knowledge crosses geographic boundaries, we can work remotely. We’re paid well. We really can do just about anything we want.

Phát triển phần mềm chắc chắn phải nằm gần đầu bất kỳ danh sách nghề nghiệp nào mà ở đó bạn nắm quyền chủ động. Kỹ năng của chúng ta đang được săn đón, kiến thức của chúng ta vượt qua mọi ranh giới địa lý, chúng ta có thể làm việc từ xa. Chúng ta được trả lương hậu hĩnh. Chúng ta thực sự có thể làm gần như bất cứ điều gì mình muốn.

But, for some reason, developers seem to resist change. They hunker down, and hope things will get better. They look on, passively, as their skills become dated and complain that their companies don’t train them. They look at ads for exotic locations on the bus, then step off into the chilling rain and

Thế nhưng, vì lý do nào đó, các lập trình viên dường như lại chống cự sự thay đổi. Họ co cụm lại, và hy vọng mọi thứ sẽ khá hơn. Họ thụ động đứng nhìn kỹ năng của mình lỗi thời dần và than phiền rằng công ty chẳng chịu đào tạo họ. Trên xe buýt, họ ngẩn nhìn những tấm quảng cáo về các vùng đất xa xôi đầy mê hoặc, rồi bước xuống giữa cơn mưa lạnh buốt và

trudge into work.

lê bước đến chỗ làm.

So here’s the most important **tip** in the book.

Vậy nên đây là mẹo quan trọng nhất trong cả cuốn sách.

Tip 3: You Have Agency

Tip 3: Bạn Có Quyền Tự Quyết

Does your work environment suck? Is your job boring? Try to fix it. But don’t try forever. As Martin Fowler says, “you can change your organization or change your organization.” [3]

Môi trường làm việc của bạn tệ hại? Công việc của bạn nhàm chán? Hãy cố sửa nó. Nhưng đừng cố mãi mãi. Như Martin Fowler nói, “bạn có thể thay đổi tổ chức của mình, hoặc thay đổi tổ chức của mình” (ý là: cải tổ nó, hoặc đổi sang chỗ khác). [3]

If technology seems to be passing you by, make time (in your own time) to study new stuff that looks interesting. You’re investing in yourself, so doing it while you’re off-the-clock is only reasonable.

Nếu công nghệ có vẻ đang bỏ bạn lại phía sau, hãy dành thời gian (thời gian của riêng bạn) để học những thứ mới mẻ trông thú vị. Bạn đang đầu tư cho chính mình, nên việc làm điều đó ngoài giờ làm cũng là hợp lý thôi.

Want to work remotely? Have you asked? If they say no, then find someone who says yes.

Muốn làm việc từ xa? Bạn đã hỏi chưa? Nếu họ nói không, thì hãy tìm người nói có.

This industry gives you a remarkable set of opportunities. Be proactive, and take them.

Ngành này trao cho bạn vô số cơ hội đáng kinh ngạc. Hãy chủ động, và nắm lấy chúng.

Related Sections Include — Các mục liên quan

Topic 4, **Stone Soup** and Boiled Frogs

Mục 4, Súp Đá Và Con Ếch Bị Luộc

Topic 6, Your **Knowledge Portfolio**

Mục 6, Danh Mục Tri Thức Của Bạn

The greatest of all weaknesses is the fear of appearing weak. J.B. Bossuet, Politics from Holy Writ, 1709

Điểm yếu lớn nhất trong tất cả các điểm yếu chính là nỗi sợ tỏ ra yếu đuối. J.B. Bossuet, Politics from Holy Writ, 1709

Topic 2. The Cat Ate My Source Code — Topic 2. Mèo Ăn Mất Mã Nguồn Của Tôi

One of the cornerstones of the **pragmatic** philosophy is the idea of taking responsibility for yourself and your actions in terms of your career advancement, your learning and education, your project, and your day-to-day work. Pragmatic Programmers take charge of their own career, and aren't afraid to admit ignorance or error. It's not the most pleasant aspect of programming, to be sure, but it will happen—even on the best of projects. Despite thorough testing, good documentation, and solid automation, things go wrong. Deliveries are late. Unforeseen technical problems come up.

Một trong những nền tảng của triết lý thực dụng là ý tưởng tự chịu trách nhiệm về bản thân và hành động của mình — trong chuyện thăng tiến sự nghiệp, trong học hành và trau dồi, trong dự án, và trong công việc thường ngày. Lập trình viên Thực dụng (Pragmatic Programmer) tự nắm lấy sự nghiệp của mình, và không ngại thừa nhận khi mình thiếu hiểu biết hay phạm sai lầm. Hẳn rồi, đây chẳng phải là khía cạnh thú vị nhất của nghề lập trình, nhưng nó sẽ xảy ra — kể cả trong những dự án tốt nhất. Dù có kiểm thử kỹ lưỡng, tài liệu đầy đủ và tự động hóa vững chắc đến đâu, mọi thứ vẫn cứ trục trặc. Bàn giao trễ hẹn. Những vấn đề kỹ thuật không lường trước cứ thế lộ ra.

These things happen, and we try to deal with them as professionally as we can. This means being honest and direct. We can be proud of our abilities, but we must own up to our shortcomings—our ignorance and our mistakes.

Những chuyện ấy vẫn xảy ra, và chúng tôi cố xử lý chúng một cách chuyên nghiệp nhất có thể. Điều đó nghĩa là thành thật và thẳng thắn. Chúng ta có quyền tự hào về năng lực của mình, nhưng cũng phải dám thừa nhận những thiếu sót — sự non kém và những lỗi lầm của bản thân.

Team Trust — Niềm tin trong nhóm

Above all, your team needs to be able to trust and rely on you— and you need to be comfortable relying on each of them as well. Trust in a team is absolutely essential for creativity and collaboration according to the research literature. In a healthy [4] environment based in trust, you can safely speak your mind, present your ideas, and rely on your team members who can in

Trên hết, nhóm của bạn cần có thể tin tưởng và trông cậy vào bạn — và bạn cũng cần thấy thoải mái khi trông cậy vào từng người trong số họ. Theo các nghiên cứu học thuật, niềm tin trong một nhóm là điều tuyệt đối thiết yếu cho sự sáng tạo và hợp tác. Trong một môi trường lành mạnh [4] đặt nền trên niềm tin, bạn có thể yên tâm nói thẳng suy nghĩ, trình bày ý tưởng, và dựa vào các đồng đội — những người, đến lượt mình,

turn rely on you. Without trust, well...

cũng có thể dựa vào bạn. Còn không có niềm tin thì, ờ thì...

Imagine a high-tech, stealth ninja team infiltrating the villain's evil lair. After months of planning and delicate execution, you've made it on site. Now it's your turn to set up the laser guidance grid: "Sorry, folks, I don't have the laser. The cat was playing with the red dot and I left it at home."

Hãy hình dung một nhóm ninja công nghệ cao, hành tung bí mật, đang đột nhập vào sào huyệt của kẻ phản diện. Sau bao tháng lên kế hoạch và thực thi tỉ mỉ, các bạn đã đặt chân tới nơi. Giờ tới lượt bạn dựng lưới dẫn đường bằng laser: "Xin lỗi anh em, tôi không mang laser theo. Con mèo nhà tôi nghịch cái chấm đỏ nên tôi để quên nó ở nhà mất rồi."

That sort of breach of trust might be hard to repair.

Kiểu phá vỡ niềm tin như thế thì khó mà hàn gắn lại được.

Take Responsibility — Hãy chịu trách nhiệm

Responsibility is something you actively agree to. You make a commitment to ensure that something is done right, but you don't necessarily have direct control over every aspect of it. In addition to doing your own personal best, you must analyze the situation for risks that are beyond your control. You have the right not to take on a responsibility for an impossible situation, or one in which the risks are too great, or the ethical implications too sketchy. You'll have to make the call based on your own values and judgment.

Trách nhiệm là thứ bạn chủ động nhận lấy. Bạn cam kết bảo đảm một việc được làm cho đúng, nhưng không nhất thiết bạn nắm quyền kiểm soát trực tiếp mọi khía cạnh của nó. Bên cạnh việc cố hết sức mình, bạn còn phải phân tích tình huống để tìm ra những rủi ro nằm ngoài tầm kiểm soát của bạn. Bạn có quyền không nhận trách nhiệm cho một tình huống bất khả thi, hoặc một việc có rủi ro quá lớn, hoặc những hệ lụy đạo đức quá mập mờ. Bạn sẽ phải tự quyết định dựa trên giá trị và phán đoán của chính mình.

When you do accept the responsibility for an outcome, you should expect to be held accountable for it. When you make a mistake (as we all do) or an error in judgment, admit it honestly and try to offer options.

Một khi bạn đã nhận trách nhiệm về một kết quả nào đó, bạn nên chuẩn bị tinh thần là mình sẽ phải gánh chịu nó. Khi bạn phạm sai lầm (mà ai chẳng có lúc sai) hay nhận định nhầm, hãy thành thật thừa nhận và cố đưa ra các phương án.

Don't blame someone or something else, or make up an excuse. Don't blame all the problems on a vendor, a programming language, management, or your coworkers. Any and all of these may play a role, but it is up to you to provide solutions, not excuses.

Đừng đổ lỗi cho ai đó hay cho thứ gì khác, cũng đừng bịa ra cái cớ. Đừng trút mọi vấn đề lên đầu một nhà cung cấp, một ngôn ngữ lập trình, ban quản lý, hay đồng nghiệp của bạn. Bất kỳ — và tất cả — những thứ đó đều có thể góp phần, nhưng việc đưa ra giải pháp, chứ không phải lời bào chữa, là phần của bạn.

If there was a risk that the vendor wouldn't come through for

Nếu có rủi ro rằng nhà cung cấp sẽ không kịp giao hàng cho

you, then you should have had a contingency plan. If your mass storage melts—taking all of your source code with it—and you don't have a backup, it's your fault. Telling your boss “**the cat ate my source code**” just won't cut it.

bạn, thì lẽ ra bạn phải có sẵn một kế hoạch dự phòng. Nếu ổ lưu trữ của bạn cháy rụi — mang theo toàn bộ mã nguồn — mà bạn lại không có bản sao lưu, thì đó là lỗi của bạn. Nói với sếp rằng “con mèo ăn mất mã nguồn của tôi rồi” thì chẳng ăn thua gì đâu.

Tip 4: Provide Options, Don't Make Lame Excuses

Tip 4: Đưa Ra Phương Án, Đừng Viện Cớ Vớ Vẩn

Before you approach anyone to tell them why something can't be done, is late, or is broken, stop and listen to yourself. Talk to the rubber duck on your monitor, or the cat. Does your excuse sound reasonable, or stupid? How's it going to sound to your boss?

Trước khi tìm đến ai đó để giải thích vì sao một việc không làm được, bị trễ, hay đang hỏng, hãy dừng lại và tự lắng nghe chính mình. Hãy nói với con vịt cao su trên màn hình, hoặc với con mèo. Cái cớ của bạn nghe có hợp lý không, hay nghe ngớ ngẩn? Nó sẽ nghe ra sao trong tai sếp của bạn?

Run through the conversation in your mind. What is the other person likely to say? Will they ask, “Have you tried this...” or “Didn't you consider that?” How will you respond? Before you go and tell them the bad news, is there anything else you can try? Sometimes, you just know what they are going to say, so save them the trouble.

Hãy diễn lại cuộc trò chuyện đó trong đầu. Người kia có lẽ sẽ nói gì? Họ sẽ hỏi “Cậu thử cách này chưa...” hay “Sao cậu không tính tới chuyện kia?” Bạn sẽ đáp lại thế nào? Trước khi đem tin xấu đi báo, liệu còn cách nào khác bạn có thể thử không? Đôi khi bạn thừa biết họ sắp nói gì, nên hãy đỡ cho họ khỏi phải nói ra.

Instead of excuses, provide options. Don't say it can't be done; explain what can be done to salvage the situation. Does code have to be deleted? Tell them so, and explain the value of **refactoring** (see **Topic 40, Refactoring**).

Thay vì viện cớ, hãy đưa ra phương án. Đừng nói rằng việc đó không làm được; hãy giải thích những gì có thể làm để cứu vãn tình hình. Có cần xóa bớt mã đi không? Hãy nói thẳng, và giải thích giá trị của việc tái cấu trúc (refactoring) (xem Topic 40, Tái cấu trúc).

Do you need to spend time prototyping to determine the best way to proceed (see Topic 13, Prototypes and Post-it Notes)? Do you need to introduce better testing (see Topic 41, **Test to Code** , and Ruthless and Continuous Testing) or automation to prevent it from happening again?

Bạn có cần dành thời gian dựng bản mẫu (prototype) để xác định cách tốt nhất nhằm tiến tới không (xem Topic 13, Bản mẫu và Giấy nhớ)? Bạn có cần áp dụng kiểm thử tốt hơn (xem Topic 41, Kiểm thử trước Mã nguồn, và Kiểm thử Liên tục và Không khoan nhượng) hay tự động hóa để chuyện đó không tái diễn không?

Perhaps you need additional resources to complete this task. Or

Có lẽ bạn cần thêm nguồn lực để hoàn thành nhiệm vụ này. Hoặc

maybe you need to spend more time with the users? Or maybe it's just you: do you need to learn some technique or technology in greater depth? Would a book or a course help? Don't be afraid to ask, or to admit that you need help.

có lẽ bạn cần dành thêm thời gian với người dùng? Hay biết đâu vấn đề nằm ở chính bạn: bạn có cần học sâu hơn về một kỹ thuật hay công nghệ nào đó không? Một cuốn sách hay một khóa học liệu có giúp ích? Đừng ngại hỏi, hay ngại thừa nhận rằng mình cần được giúp đỡ.

Try to flush out the lame excuses before voicing them aloud. If you must, tell your cat first. After all, if little Tiddles is going to take the blame....

Hãy cố gạt bỏ những cái cớ vớ vẩn trước khi nói chúng ra thành tiếng. Nếu buộc phải nói, hãy kể với con mèo của bạn trước. Suy cho cùng, nếu bé Tiddles sắp phải đứng ra chịu tội...

Related Sections Include — Các mục liên quan

Topic 49, Pragmatic Teams

Topic 49, Nhóm Thực dụng

Challenges — Thử thách

How do you react when someone—such as a bank teller, an auto mechanic, or a clerk—comes to you with a lame excuse? What do you think of them and their company as a result?

Bạn phản ứng thế nào khi ai đó — chẳng hạn một nhân viên ngân hàng, một thợ sửa xe, hay một anh nhân viên quầy — đến gặp bạn với một cái cớ vớ vẩn? Sau đó bạn nghĩ gì về họ và công ty của họ?

When you find yourself saying, “I don’t know,” be sure to follow it up with “—but I’ll find out.” It’s a great way to admit what you don’t know, but then take responsibility like a pro.

Khi thấy mình buột miệng nói “Tôi không biết”, hãy nhớ tiếp lời ngay bằng “—nhưng tôi sẽ tìm hiểu cho ra.” Đó là cách tuyệt vời để thừa nhận điều mình chưa biết, rồi gánh lấy trách nhiệm như một người chuyên nghiệp.

Topic 3. Software Entropy — Topic 3. Sự rối loạn phần mềm (Software Entropy)

While software development is immune from almost all physical laws, the inexorable increase in **entropy** hits us hard. Entropy is a term from physics that refers to the amount of “disorder” in a system. Unfortunately, the laws of thermodynamics guarantee that the entropy in the universe tends toward a maximum. When disorder increases in software, we call it “software rot.” Some folks might call it by the more optimistic term, “technical debt,” with the implied notion that they’ll pay it back someday. They probably won’t.

Dù việc phát triển phần mềm gần như miễn nhiễm với mọi định luật vật lý, sự gia tăng không thể tránh khỏi của entropy lại giáng vào chúng ta khá đau. Entropy là một thuật ngữ trong vật lý chỉ mức độ “hỗn loạn” trong một hệ thống. Đáng tiếc, các định luật nhiệt động lực học bảo đảm rằng entropy trong vũ trụ luôn có xu hướng tăng tới mức tối đa. Khi sự hỗn loạn tăng lên trong phần mềm, ta gọi đó là “software rot” (phần mềm mục ruỗng). Một số người có thể dùng cái tên lạc quan hơn — “technical debt” (nợ kỹ thuật) — với hàm ý rằng họ sẽ trả lại món nợ ấy vào một ngày nào đó. Mà thường thì họ chẳng trả đâu.

Whatever the name, though, both debt and rot can spread uncontrollably.

Dù tên gọi là gì đi nữa, cả nợ lẫn mục ruỗng đều có thể lan ra một cách mất kiểm soát.

There are many factors that can contribute to software rot. The most important one seems to be the psychology, or culture, at work on a project. Even if you are a team of one, your project's psychology can be a very delicate thing. Despite the best-laid plans and the best people, a project can still experience ruin and decay during its lifetime. Yet there are other projects that, despite enormous difficulties and constant setbacks, successfully fight nature's tendency toward disorder and manage to come out pretty well.

Có nhiều yếu tố góp phần làm phần mềm mục ruỗng. Yếu tố quan trọng nhất dường như là tâm lý, hay văn hóa, đang vận hành trong một dự án. Ngay cả khi bạn là một đội chỉ có một người, tâm lý của dự án vẫn có thể là thứ hết sức mong manh. Bất chấp những kế hoạch chu đáo nhất và những con người giỏi giang nhất, một dự án vẫn có thể lụi tàn và rệu rã trong vòng đời của nó. Vậy mà cũng có những dự án khác, dù gặp vô vàn khó khăn và liên tiếp vấp ngã, lại chiến đấu thành công với xu hướng tự nhiên hướng về hỗn loạn và vẫn xoay sở để về đích khá đẹp.

What makes the difference?

Điều gì tạo nên khác biệt?

In inner cities, some buildings are beautiful and clean, while others are rotting hulks. Why? Researchers in the field of crime

Trong các khu nội đô, có những tòa nhà đẹp đẽ, sạch sẽ, trong khi những tòa khác chỉ còn là cái vỏ mục nát. Vì sao? Các nhà nghiên cứu trong lĩnh vực tội phạm

and urban decay discovered a fascinating trigger mechanism, one that very quickly turns a clean, intact, inhabited building into a smashed and abandoned derelict. [5]

và suy tàn đô thị đã phát hiện một cơ chế kích hoạt hết sức thú vị, một cơ chế biến rất nhanh một tòa nhà sạch sẽ, nguyên vẹn, có người ở thành một đồng đống nát hoang phế, cửa kính vỡ tan. [5]

A broken window.

Một ô cửa sổ vỡ.

One broken window, left unrepaired for any substantial length of time, instills in the inhabitants of the building a sense of abandonment—a sense that the powers that be don't care about the building. So another window gets broken. People start littering. Graffiti appears. Serious structural damage begins. In a relatively short span of time, the building becomes damaged beyond the owner's desire to fix it, and the sense of abandonment becomes reality.

Một ô cửa sổ vỡ, bị bỏ mặc không sửa trong một khoảng thời gian đủ dài, gieo vào lòng những người sống trong tòa nhà cảm giác bị bỏ rơi — cảm giác rằng những kẻ có quyền chẳng hề bận tâm đến tòa nhà này. Thế là một ô cửa nữa vỡ. Người ta bắt đầu xả rác. Hình vẽ bậy xuất hiện. Hư hại kết cấu nghiêm trọng bắt đầu xảy ra. Trong một khoảng thời gian khá ngắn, tòa nhà hư hỏng tới mức vượt quá mong muốn sửa chữa của chủ nhân, và cảm giác bị bỏ rơi trở thành hiện thực.

Why would that make a difference? Psychologists have done studies that show hopelessness can be contagious. Think of [6] the flu virus in close quarters. Ignoring a clearly broken situation reinforces the ideas that perhaps nothing can be fixed, that no one cares, all is doomed; all negative thoughts which can spread among team members, creating a vicious spiral.

Tại sao điều đó lại tạo ra khác biệt? Các nhà tâm lý học đã làm những nghiên cứu cho thấy sự tuyệt vọng có thể lây lan. Hãy nghĩ tới [6] virus cúm trong một không gian chật hẹp.

Phớt lờ một tình huống rõ ràng đã hỏng càng củng cố cái ý nghĩ rằng có lẽ chẳng còn gì cứu vãn được, rằng chẳng ai thêm quan tâm, rằng tất cả đều vô vọng — toàn những suy nghĩ tiêu cực có thể lan ra giữa các thành viên trong đội, tạo nên một vòng xoáy tệ hại.

Tip 5: Don't Live with Broken Windows

Tip 5: Đừng Sống Chung Với Những Ô Cửa Sổ Vỡ

Don't leave **“broken windows”** (bad designs, wrong decisions, or poor code) unrepaired. Fix each one as soon as it is discovered. If there is insufficient time to fix it properly, then board it up. Perhaps you can comment out the offending code, or display a “Not Implemented” message, or substitute dummy data instead. Take some action to prevent further damage and to show that you're on top of the situation.

Đừng để những “ô cửa sổ vỡ” (broken windows) — những thiết kế tồi, những quyết định sai, hay mã nguồn dở — không được sửa chữa. Hãy sửa ngay từng cái một khi vừa phát hiện ra. Nếu không đủ thời gian để sửa cho đến nơi đến chốn, thì hãy bịt kín nó lại. Bạn có thể chú thích để vô hiệu hóa đoạn mã có vấn đề, hoặc hiển thị thông báo “Not Implemented” (Chưa hiện thực), hoặc thay vào đó bằng dữ liệu giả. Hãy làm gì đó để ngăn hư hại lan rộng và để cho thấy bạn đang kiểm soát được tình hình.

We've seen clean, functional systems deteriorate pretty quickly once windows start breaking. There are other factors that can contribute to software rot, and we'll touch on some of them elsewhere, but neglect accelerates the rot faster than any other factor.

Chúng tôi đã chứng kiến những hệ thống sạch sẽ, hoạt động tốt rệu rã đi rất nhanh một khi những ô cửa bắt đầu vỡ. Còn có những yếu tố khác góp phần làm phần mềm mục ruỗng, và chúng tôi sẽ đề cập tới một vài yếu tố ở chỗ khác, nhưng sự bỏ bê làm quá trình mục ruỗng tăng tốc nhanh hơn bất kỳ yếu tố nào khác.

You may be thinking that no one has the time to go around cleaning up all the broken glass of a project. If so, then you'd better plan on getting a dumpster, or moving to another neighborhood. Don't let entropy win.

Bạn có thể đang nghĩ rằng chẳng ai có thời gian để đi dọn dẹp hết đồng kính vỡ của một dự án. Nếu vậy thì tốt hơn hết bạn nên tính chuyện thuê một thùng rác cỡ lớn, hoặc dọn sang khu phố khác mà ở. Đừng để entropy thắng.

First, Do No Harm — Trước hết, đừng gây hại

Andy once had an acquaintance who was obscenely rich. His house was immaculate, loaded with priceless antiques, objets d'art, and so on. One day, a tapestry that was hanging a little too close to a fireplace caught on fire. The fire department rushed in to save the day—and his house. But before they dragged their big, dirty hoses into the house, they stopped—with the fire raging—to roll out a mat between the front door and the source of the fire.

Andy từng có một người quen giàu nứt đổ đổ vách. Ngôi nhà của ông ta sạch bong, chất đầy đồ cổ vô giá, các objets d'art (tác phẩm nghệ thuật), vân vân. Một hôm, một tấm thảm treo tường mắc treo hơi sát lò sưởi bắt lửa. Đội cứu hỏa lao tới để cứu vãn tình thế — và cứu ngôi nhà. Nhưng trước khi kéo những vòi nước to bự, lấm lem của họ vào trong nhà, họ dừng lại — giữa lúc đám cháy đang bùng dữ dội — để trải một tấm bạt giữa cửa trước và điểm phát hỏa.

They didn't want to mess up the carpet.

Họ không muốn làm bẩn tấm thảm.

Now that sounds pretty extreme. Surely the fire department's first priority is to put out the fire, collateral damage be damned. But they clearly had assessed the situation, were confident of their ability to manage the fire, and were careful not to inflict unnecessary damage to the property. That's the way it must be with software: don't cause collateral damage just because there's a crisis of some sort. One broken window is one too many.

Nghề thì có vẻ khá thái quá. Chắc chắn ưu tiên hàng đầu của đội cứu hỏa là dập lửa, kệ cho thiệt hại kèm theo. Nhưng rõ ràng họ đã đánh giá tình hình, tự tin vào khả năng khống chế đám cháy của mình, và cẩn thận để không gây thiệt hại không cần thiết cho tài sản. Với phần mềm cũng phải như vậy: đừng gây thiệt hại kèm theo chỉ vì đang có một cuộc khủng hoảng nào đó. Một ô cửa sổ vỡ đã là quá nhiều rồi.

One broken window—a badly designed piece of code, a poor management decision that the team must live with for the

Một ô cửa sổ vỡ — một đoạn mã thiết kế tồi, một quyết định quản lý dở mà cả đội phải sống chung với nó suốt

duration of the project—is all it takes to start the decline. If you find yourself working on a project with quite a few broken windows, it's all too easy to slip into the mindset of “All the rest of this code is crap, I'll just follow suit.” It doesn't matter if the project has been fine up to this point. In the original experiment leading to the “Broken Window Theory,” an abandoned car sat for a week untouched. But once a single window was broken, the car was stripped and turned upside down within hours .

quãng đường dài của dự án — chỉ cần thế thôi là đủ để khởi đầu cho sự xuống dốc. Nếu bạn thấy mình đang làm trong một dự án với khá nhiều ô cửa sổ vỡ, sẽ rất dễ trượt vào lối nghĩ “Cả đồng mã còn lại trong này đều là rác rưởi, thôi thì mình cứ làm theo cho xong.” Chuyện dự án từ trước tới giờ vẫn ổn cũng chẳng quan trọng gì. Trong thí nghiệm gốc dẫn tới “Thuyết Cửa Sổ Vỡ” (Broken Window Theory), một chiếc xe bị bỏ rơi đứng nguyên một tuần mà không ai đụng vào. Nhưng chỉ cần một ô cửa kính bị đập vỡ, chiếc xe đã bị lột sạch và lật ngửa chỉ trong vài giờ.

By the same token, if you find yourself on a project where the code is pristinely beautiful—cleanly written, well designed, and elegant—you will likely take extra special care not to mess it up, just like the firefighters. Even if there's a fire raging (deadline, release date, trade show demo, **etc.**), you don't want to be the first one to make a mess and inflict additional damage.

Cũng theo lẽ ấy, nếu bạn thấy mình đang ở trong một dự án mà mã nguồn đẹp để tinh khôi — viết sạch sẽ, thiết kế khéo léo, và thanh thoát — bạn sẽ có xu hướng đặc biệt cẩn thận để không làm hỏng nó, hết như những người lính cứu hỏa kia. Ngay cả khi có một đám cháy đang bùng (deadline, ngày phát hành, demo cho hội chợ thương mại, v.v.), bạn cũng chẳng muốn mình là kẻ đầu tiên bày bừa và gây thêm thiệt hại.

Just tell yourself, “No broken windows.”

Cứ tự nhủ với bản thân: “Không có ô cửa sổ vỡ nào hết.”

Related Sections Include — Các mục liên quan

Topic 10, Orthogonality

Topic 10, Tính trực giao

Topic 40, Refactoring

Topic 40, Tái cấu trúc

Topic 44, **Naming Things**

Topic 44, Đặt tên cho mọi thứ

Challenges — Thử thách

Help strengthen your team by surveying your project neighborhood. Choose two or three broken windows and discuss with your colleagues what the problems are and what could be done to fix them.

Hãy giúp đội của bạn vững mạnh hơn bằng cách khảo sát “khu phố” của dự án. Chọn ra hai hoặc ba ô cửa sổ vỡ và bàn với đồng nghiệp xem vấn đề là gì và có thể làm gì để sửa chúng.

Can you tell when a window first gets broken? What is your reaction? If it was the result of someone else’s decision, or a management edict, what can you do about it?

Bạn có nhận ra được khi một ô cửa sổ lần đầu bị vỡ không? Phản ứng của bạn là gì? Nếu đó là hệ quả từ quyết định của ai đó khác, hay một mệnh lệnh từ cấp quản lý, thì bạn có thể làm gì với nó?

Topic 4. Stone Soup and Boiled Frogs — Topic 4. Súp đá và những con ếch bị luộc

The three soldiers returning home from war were hungry. When they saw the village ahead their spirits lifted—they were sure the villagers would give them a meal. But when they got there, they found the doors locked and the windows closed. After many years of war, the villagers were short of food, and hoarded what they had.

Ba người lính trên đường từ chiến trường trở về nhà, bụng đói cồn cào. Khi nhìn thấy ngôi làng phía trước, tinh thần họ phấn chấn hẳn lên — họ chắc mẩm dân làng sẽ cho mình một bữa ăn. Nhưng khi tới nơi, họ thấy cửa nhà nào cũng khóa chặt, cửa sổ đóng im ỉm. Sau bao năm chiến tranh, dân làng thiếu ăn, ai nấy đều giấu nhem chỗ lương thực ít ỏi của mình.

Undeterred, the soldiers boiled a pot of water and carefully placed three stones into it. The amazed villagers came out to watch.

Không hề nản, mấy người lính đun một nồi nước rồi cẩn thận thả ba hòn đá vào. Dân làng ngạc nhiên kéo nhau ra xem.

“This is **stone soup**,” the soldiers explained. “Is that all you put in it?” asked the villagers. “Absolutely—although some say it tastes even better with a few carrots...” A villager ran off, returning in no time with a basket of carrots from his hoard.

“Đây là món súp đá,” mấy người lính giải thích. “Chỉ có thể thôi à?” dân làng hỏi. “Đúng vậy — dù có người bảo cho thêm vài củ cà rốt vào thì ngon hơn nhiều...” Một người dân vội chạy đi, lát sau quay lại với một rổ cà rốt lấy từ chỗ giấu của mình.

A couple of minutes later, the villagers again asked “Is that it?”

Được vài phút, dân làng lại hỏi “Thế là xong à?”

“Well,” said the soldiers, “a couple of potatoes give it body.” Off ran another villager.

“Ờ thì,” mấy người lính đáp, “cho thêm dăm củ khoai tây nữa cho đặc.” Lại một người dân khác chạy đi.

Over the next hour, the soldiers listed more ingredients that would enhance the soup: beef, leeks, **salt**, and herbs. Each time a different villager would run off to raid their personal stores.

Suốt một giờ sau đó, mấy người lính kể ra hết nguyên liệu này đến nguyên liệu khác để súp ngon hơn: thịt bò, tỏi tây, muối, rồi rau thơm. Mỗi lần như vậy lại có một người dân chạy về moi kho riêng của mình.

Eventually they had produced a large pot of steaming soup. The soldiers removed the stones, and they sat down with the entire village to enjoy the first square meal any of them had

Cuối cùng họ đã có cả một nồi súp lớn bốc khói nghi ngút. Mấy người lính vớt mấy hòn đá ra, rồi cùng ngồi xuống với cả làng để thưởng thức bữa ăn tử tế đầu tiên mà bất kỳ ai trong số họ đã

eaten in months.

ăn được trong suốt mấy tháng trời.

There are a couple of morals in the stone soup story. The villagers are tricked by the soldiers, who use the villagers’ curiosity to get food from them. But more importantly, the soldiers act as a **catalyst**, bringing the village together so they can jointly produce something that they couldn’t have done by themselves—a synergistic result. Eventually everyone wins.

Câu chuyện súp đá có vài bài học. Dân làng bị mấy người lính lừa, kẻ đã lợi dụng chính sự tò mò của dân làng để moi đồ ăn. Nhưng quan trọng hơn, mấy người lính đóng vai chất xúc tác (catalyst), gắn kết cả làng lại để cùng nhau làm nên một thứ mà tự thân từng người không thể làm được — một kết quả cộng hưởng. Rốt cuộc thì ai cũng được lợi.

Every now and then, you might want to emulate the soldiers.

Thỉnh thoảng, bạn cũng nên học theo mấy người lính ấy.

You may be in a situation where you know exactly what needs doing and how to do it. The entire system just appears before your eyes—you know it’s right. But ask permission to tackle the whole thing and you’ll be met with delays and blank stares. People will form committees, budgets will need approval, and things will get complicated. Everyone will guard their own resources. Sometimes this is called “start-up fatigue.”

Có thể bạn đang ở trong tình huống mình biết chính xác cần làm gì và làm như thế nào. Cả hệ thống hiện ra trước mắt bạn — bạn biết là nó đúng. Nhưng cứ thử xin phép để bắt tay vào làm toàn bộ xem, bạn sẽ chỉ nhận được sự trì hoãn và những ánh mắt ngơ ngác. Người ta sẽ lập hội đồng, ngân sách thì phải được duyệt, và mọi thứ sẽ trở nên rắc rối. Ai nấy đều khư khư giữ lấy nguồn lực của mình. Đôi khi người ta gọi đây là “sự ị ạch lúc khởi động” (start-up fatigue).

It’s time to bring out the stones. Work out what you can reasonably ask for. Develop it well. Once you’ve got it, show people, and let them marvel. Then say “of course, it would be better if we added...” Pretend it’s not important. Sit back and wait for them to start asking you to add the functionality you originally wanted. People find it easier to join an ongoing success. Show them a glimpse of the future and you’ll get them to rally around. [7]

Đã đến lúc lôi mấy hòn đá ra. Hãy tính xem bạn có thể xin được những gì một cách hợp lý. Làm thật tốt phần đó. Khi đã có rồi, cho mọi người xem, để họ trầm trồ. Rồi nói “tất nhiên, sẽ còn tốt hơn nếu ta thêm vào...” Giả vờ như nó chẳng quan trọng gì. Cứ ngồi yên chờ họ bắt đầu nhờ bạn thêm vào đúng những tính năng mà ngay từ đầu bạn đã muốn. Người ta

thấy dễ tham gia vào một thành công đang diễn ra hơn nhiều. Hãy cho họ thấy một thoáng tương lai, và bạn sẽ khiến họ xúm lại quanh mình. [7]

Tip 6: Be a Catalyst for Change

Tip 6: Hãy Là Chất Xúc Tác Cho Sự Thay Đổi

The Villagers' Side — Phía dân làng

On the other hand, the stone soup story is also about gentle and

Mặt khác, câu chuyện súp đá cũng nói về một kiểu lừa dối nhẹ nhàng và

gradual deception. It's about focusing too tightly. The villagers think about the stones and forget about the rest of the world. We all fall for it, every day. Things just creep up on us.

từ từ. Nó nói về việc tập trung quá hẹp. Dân làng cứ chăm chăm vào mấy hòn đá mà quên mất phần còn lại của thế giới. Tất cả chúng ta đều mắc bẫy đó, mỗi ngày. Mọi thứ cứ lặng lẽ bò tới mà ta không hay.

We've all seen the symptoms. Projects slowly and inexorably get totally out of hand. Most software disasters start out too small to notice, and most project overruns happen a day at a time. Systems drift from their specifications feature by feature, while patch after patch gets added to a piece of code until there's nothing of the original left. It's often the accumulation of small things that breaks morale and teams.

Tất cả chúng ta đều đã thấy những triệu chứng ấy. Các dự án cứ chậm rãi mà không sao cưỡng lại được tuột hẳn khỏi tầm kiểm soát. Hầu hết các thảm họa phần mềm khởi đầu nhỏ đến mức chẳng ai để ý, và hầu hết các dự án bị đội thời gian cũng diễn ra từng ngày một. Hệ thống trôi dạt khỏi đặc tả của nó, từng tính năng một, trong khi hết bản vá này đến bản vá khác cứ chồng lên một đoạn mã cho đến khi chẳng còn lại gì của bản gốc. Thường thì chính sự tích tụ của những điều nhỏ nhặt mới làm tan vỡ tinh thần và đội nhóm.

Tip 7: Remember the Big Picture

Tip 7: Hãy Nhớ Tới Bức Tranh Toàn Cảnh

We've never tried this—honest. But “they” say that if you take a frog and drop it into boiling water, it will jump straight back out again. However, if you place the frog in a pan of cold water, then gradually heat it, the frog won't notice the slow increase in temperature and will stay put until cooked.

Chúng tôi chưa bao giờ thử cái này đâu — thật đấy. Nhưng “người ta” bảo rằng nếu bạn bắt một con ếch thả vào nước đang sôi, nó sẽ nhảy phắt ra ngay. Tuy nhiên, nếu bạn đặt con ếch vào một nồi nước lạnh rồi đun nóng dần lên, con ếch sẽ không nhận ra nhiệt độ đang từ từ tăng lên và cứ nằm ì ra đó cho đến khi bị luộc chín.

Note that the frog's problem is different from the **broken windows** issue discussed in **Topic 3, Software Entropy**. In the Broken Window Theory, people lose the will to fight **entropy** because they perceive that no one else cares. The frog just doesn't notice the change.

Lưu ý rằng vấn đề của con ếch khác với chuyện cửa sổ vỡ (broken windows) đã bàn ở Topic 3, Sự rối loạn phần mềm. Trong Lý thuyết Cửa sổ vỡ, người ta mất ý chí chống lại sự rối loạn vì họ cảm thấy chẳng ai khác bận tâm. Còn con ếch thì đơn giản là không nhận ra sự thay đổi.

Don't be like the fabled frog. Keep an eye on the **big picture**. Constantly review what's happening around you, not just what you personally are doing.

Đừng giống như con ếch trong truyện ngụ ngôn. Hãy luôn để mắt tới bức tranh toàn cảnh. Liên tục xem xét những gì đang diễn ra quanh bạn, chứ không chỉ những gì đích thân bạn đang làm.

Related Sections Include — Các mục liên quan

Topic 1, It's Your Life

Topic 1, Cuộc đời là của bạn

Topic 38, **Programming by Coincidence**

Topic 38, Lập trình theo may rủi

Challenges — Thử thách

While reviewing a draft of the first edition, John Lakos raised the following issue: The soldiers progressively deceive the villagers, but the change they catalyze does them all good. However, by progressively deceiving the frog, you're doing it harm. Can you determine whether you're making stone soup or frog soup when you try to catalyze change? Is the decision subjective or objective?

Khi xem lại bản nháp của lần xuất bản đầu tiên, John Lakos đã nêu ra vấn đề sau: Mấy người lính dần dần lừa dối dân làng, nhưng sự thay đổi mà họ tạo ra lại tốt cho tất cả mọi người. Tuy nhiên, khi dần dần lừa con ếch, bạn lại đang gây hại cho nó. Liệu bạn có thể xác định được mình đang nấu súp đá hay súp ếch khi cố làm chất xúc tác cho sự thay đổi không? Quyết định đó là chủ quan hay khách quan?

Quick, without looking, how many lights are in the ceiling above you? How many exits in the room? How many people? Is there anything out of **context**, anything that looks like it doesn't belong? This is an exercise in **situational awareness**, a technique practiced by folks ranging from Boy and Girl Scouts to Navy SEALs. Get in the habit of really looking and noticing your surroundings. Then do the same for your project.

Nhanh nào, đừng nhìn lên, có bao nhiêu bóng đèn trên trần phía trên bạn? Căn phòng có mấy lối ra? Có bao nhiêu người? Có gì lạc lõng không, có gì trông như không thuộc về nơi đó không? Đây là một bài tập về nhận thức tình huống (situational awareness), một kỹ thuật được nhiều người rèn luyện, từ các Hướng đạo sinh nam nữ cho đến lính đặc nhiệm Navy SEAL. Hãy tập thói quen thực sự quan sát và để ý đến môi trường xung quanh. Rồi làm điều tương tự với dự án của bạn.

Striving to better, oft we mar what's well. Shakespeare, King Lear 1.4

Cố vươn tới điều tốt hơn, ta thường làm hỏng cả những gì vốn đã tốt đẹp.
Shakespeare, King Lear 1.4

Topic 5. Good-Enough Software — Topic 5. Phần mềm Đủ tốt

There's an old(ish) joke about a company that places an order for 100,000 ICs with a Japanese manufacturer. Part of the specification was the defect rate: one chip in 10,000. A few weeks later the order arrived: one large box containing thousands of ICs, and a small one containing just ten. Attached to the small box was a label that read: "These are the faulty ones."

Có một câu chuyện cười (cũng hơi cũ rồi) kể về một công ty đặt mua 100.000 con IC từ một nhà sản xuất Nhật Bản. Một phần trong bản đặc tả là tỷ lệ lỗi cho phép: cứ 10.000 con

thì được hổng một con. Vài tuần sau, đơn hàng về tới: một thùng to đựng hàng nghìn con IC, và một thùng nhỏ chỉ đựng đúng mười con. Dán trên thùng nhỏ là một mảnh giấy ghi: “Đây là những con bị lỗi.”

If only we really had this kind of control over quality. But the real world just won't let us produce much that's truly perfect, particularly not bug-free software. Time, technology, and temperament all conspire against us.

Giá mà chúng ta thực sự kiểm soát được chất lượng kiểu đó. Nhưng thế giới thực chẳng cho phép ta tạo ra mấy thứ thực sự hoàn hảo, nhất là phần mềm không một con bug. Thời gian, công nghệ và cả tính khí con người — tất cả đều hòa nhau chống lại ta.

However, this doesn't have to be frustrating. As Ed Yourdon described in an article in IEEE Software, When **good-enough software** is best [You95] , you can discipline yourself to write software that's good enough—good enough for your users, for future maintainers, for your own peace of mind. You'll find that you are more productive and your users are happier. And you may well find that your programs are actually better for their shorter incubation.

Tuy vậy, điều đó không nhất thiết phải khiến bạn bức bối. Như Ed Yourdon đã mô tả trong một bài viết đăng trên IEEE Software, When good-enough software is best [You95] , bạn hoàn toàn có thể rèn cho mình thói quen viết phần mềm đủ tốt — đủ tốt cho người dùng, cho những người bảo trì sau này, và cho chính sự thanh thản của bạn. Bạn sẽ thấy mình làm việc năng suất hơn và người dùng cũng hài lòng hơn. Thậm chí bạn còn có thể nhận ra rằng chương trình của mình thực sự tốt hơn nhờ thời gian thai nghén ngắn hơn.

Before we go any further, we need to qualify what we're about to say. The phrase “good enough” does not imply sloppy or poorly

Trước khi đi xa hơn, ta cần nói rõ điều sắp bàn tới. Cụm “đủ tốt” không hề ngụ ý là viết code cẩu thả hay làm

produced code. All systems must meet their users' **requirements** to be successful, and meet basic performance, privacy, and security standards. We are simply advocating that users be given an opportunity to participate in the **process** of deciding when what you've produced is good enough for their needs.

cho có. Mọi hệ thống đều phải đáp ứng yêu cầu của người dùng thì mới thành công, đồng thời phải đạt các chuẩn cơ bản về hiệu năng, quyền riêng tư và bảo mật. Chúng tôi chỉ đơn giản chủ trương rằng người dùng nên được trao cơ hội tham gia vào quá trình quyết định xem thứ bạn làm ra đã đủ tốt cho nhu cầu của họ hay chưa.

Involve Your Users In The Trade-Off — Lôi kéo Người dùng vào sự Đánh đổi

Normally you're writing software for other people. Often you'll remember to find out what they want. But do you ever ask [8] them how good they want their software to be? Sometimes there'll be no choice. If you're working on pacemakers, an autopilot, or a low-level library that will be widely disseminated, the requirements will be more stringent and your options more limited.

Thường thì bạn viết phần mềm cho người khác. Thường thì bạn sẽ nhớ tìm hiểu xem họ muốn gì. Nhưng đã bao giờ bạn hỏi [8] họ muốn phần mềm của mình tốt đến mức nào chưa? Đôi khi sẽ chẳng có lựa chọn nào cả. Nếu bạn đang làm máy tạo nhịp tim, hệ thống lái tự động, hay một thư viện cấp thấp sẽ được phát tán rộng rãi, thì yêu cầu sẽ ngặt nghèo hơn và các lựa chọn của bạn cũng bị bó hẹp hơn.

However, if you're working on a brand-new product, you'll have different constraints. The marketing people will have promises to keep, the eventual end users may have made plans based on a delivery schedule, and your company will certainly have cashflow constraints. It would be unprofessional to ignore these users' requirements simply to add new features to the program, or to polish up the code just one more time. We're not advocating panic: it is equally unprofessional to promise impossible time scales and to cut basic engineering corners to meet a deadline.

Tuy nhiên, nếu bạn đang làm một sản phẩm hoàn toàn mới, bạn sẽ vướng những ràng buộc khác. Đội marketing có những lời hứa phải giữ, người dùng cuối có thể đã lên kế hoạch dựa trên lịch bàn giao, và công ty bạn thì chắc chắn có những ràng buộc về dòng tiền. Sẽ là thiếu chuyên nghiệp nếu phớt lờ các yêu cầu này của người dùng chỉ để nhồi thêm tính năng mới vào chương trình, hay để mài giũa code thêm một lần nữa. Chúng tôi không cổ vũ chuyện cưỡng ép: cũng thiếu chuyên nghiệp y như vậy nếu hứa hẹn những thời hạn bất khả thi rồi cắt xén những phần kỹ thuật cơ bản cho kịp deadline.

The scope and quality of the system you produce should be discussed as part of that system's requirements.

Phạm vi và chất lượng của hệ thống bạn tạo ra nên được bàn bạc như một phần trong yêu cầu của chính hệ thống đó.

Tip 8: Make Quality a Requirements Issue

Tip 8: Biến Chất Lượng Thành Một Vấn Đề Yêu Cầu

Often you'll be in situations where trade-offs are involved.

Bạn sẽ thường xuyên rơi vào những tình huống phải đánh đổi.

Surprisingly, many users would rather use software with some rough edges today than wait a year for the shiny, bells-and-whistles version (and in fact what they will need a year from now may be completely different anyway). Many IT departments with tight budgets would agree. Great software today is often preferable to the fantasy of perfect software tomorrow. If you give your users something to play with early, their feedback will often lead you to a better eventual solution (see **Topic 12, Tracer Bullets**).

Đáng ngạc nhiên là rất nhiều người dùng thà dùng ngay hôm nay một phần mềm còn vài chỗ thô ráp còn hơn chờ cả năm trời để có bản bóng bẩy đủ chuông đủ còi (mà thật ra một năm nữa thứ họ cần có khi lại hoàn toàn khác). Nhiều phòng IT với ngân sách eo hẹp hẳn sẽ đồng tình. Phần mềm tuyệt vời có ngay hôm nay thường vẫn hơn cái ảo mộng về phần mềm hoàn hảo của ngày mai. Nếu bạn đưa cho người dùng thứ gì đó để vọc sớm, phản hồi của họ thường sẽ dẫn bạn tới một giải pháp cuối cùng tốt hơn (xem Topic 12, Đạn vạch đường (Tracer Bullets)).

Know When To Stop — Biết Khi nào nên Dừng tay

In some ways, programming is like painting. You start with a blank canvas and certain basic raw materials. You use a combination of science, art, and craft to determine what to do with them. You sketch out an overall shape, paint the underlying environment, then fill in the details. You constantly step back with a critical eye to view what you've done. Every now and then you'll throw a canvas away and start again.

Xét theo vài khía cạnh, lập trình cũng giống như hội họa. Bạn bắt đầu với một tấm toan trắng và một ít nguyên liệu thô cơ bản. Bạn dùng đến sự kết hợp của khoa học, nghệ thuật và tay nghề để quyết định sẽ làm gì với chúng. Bạn phác ra hình dáng tổng thể, vẽ phần

nền phía sau, rồi dần dần các chi tiết vào. Bạn liên tục lùi lại, nhìn bằng con mắt phê phán những gì mình đã làm. Thỉnh thoảng bạn lại vứt cả tấm toan đi và làm lại từ đầu.

But artists will tell you that all the hard work is ruined if you don't know when to stop. If you add layer upon layer, detail over detail, the painting becomes lost in the paint .

Nhưng các họa sĩ sẽ bảo bạn rằng mọi công sức vất vả đều hỏng bét nếu bạn không biết khi nào nên dừng tay. Nếu bạn cứ chồng lớp lên lớp, chi tiết đè chi tiết, thì bức tranh sẽ chìm ngấm trong chính những lớp sơn ấy.

Don't spoil a perfectly good program by overembellishment and overrefinement. Move on, and let your code stand in its own right for a while. It may not be perfect. Don't worry: it could never be perfect. (In Chapter 7, *While You Are Coding* , we'll discuss philosophies for developing code in an imperfect world.)

Đừng phá hỏng một chương trình vốn rất ổn bằng việc tô vẽ và chau chuốt thái quá. Hãy bước tiếp, và để code của bạn tự đứng vững một thời gian. Nó có thể chưa hoàn hảo. Đừng lo: nó vốn dĩ chẳng bao giờ hoàn hảo được. (Trong Chương 7, *Trong khi Bạn đang Code* , chúng tôi sẽ bàn về những triết lý phát triển code trong một thế giới không hoàn hảo.)

Related Sections Include — Các mục liên quan

Topic 45, The Requirements Pit

Topic 45, Cái hố Yêu cầu

Topic 46, Solving Impossible Puzzles

Topic 46, Giải những Câu đố Bất khả thi

Challenges — Thử thách

Look at the software tools and operating systems that you use regularly. Can you find any evidence that these organizations and/or developers are comfortable shipping software they know is not perfect? As a user, would you rather (1) wait for them to get all the bugs out, (2) have complex software and accept some bugs, or (3) opt for simpler software with fewer defects?

Hãy nhìn vào những công cụ phần mềm và hệ điều hành bạn dùng thường ngày. Bạn có tìm thấy bằng chứng nào cho thấy các tổ chức và/hoặc lập trình viên đó thoải mái phát hành phần mềm mà họ biết là chưa hoàn hảo không? Với tư cách người dùng, bạn muốn (1) chờ họ diệt sạch mọi con bug, (2) có phần mềm phức tạp và chấp nhận một ít bug, hay (3) chọn phần mềm đơn giản hơn nhưng ít lỗi hơn?

Consider the effect of modularization on the delivery of software. Will it take more or less time to get a tightly coupled monolithic block of software to the required quality compared with a system designed as very loosely coupled modules or microservices? What are the advantages or disadvantages of each approach?

Hãy nghĩ về tác động của việc mô-đun hóa lên quá trình bàn giao phần mềm. Sẽ tốn nhiều hay ít thời gian hơn để đưa một khối phần mềm nguyên khối, ghép nối chặt đạt tới chất lượng yêu cầu, so với một hệ thống được thiết kế thành các mô-đun hay microservice ghép nối rất lỏng? Mỗi cách tiếp cận có ưu, nhược điểm gì?

Can you think of popular software that suffers from **feature bloat** ? That is, software containing far more features than you would ever use, each feature introducing more opportunity for bugs and

security vulnerabilities, and making the features you do use harder to find and manage. Are you in danger of falling into this trap yourself?

Bạn có nghĩ ra phần mềm phổ biến nào mắc chứng phình tính năng (feature bloat) không? Tức là phần mềm chứa nhiều tính năng hơn hẳn mức bạn sẽ bao giờ dùng tới, mỗi tính năng lại mở thêm cơ hội cho bug và lỗ hổng bảo mật, đồng thời khiến những tính năng bạn thực sự dùng trở nên khó tìm và khó quản lý hơn. Liệu chính bạn có đang đứng trước nguy cơ sa vào cái bẫy này không?

An investment in knowledge always pays the best interest. Benjamin Franklin

Đầu tư vào tri thức luôn sinh lời tốt nhất. Benjamin Franklin

Topic 6. Your Knowledge Portfolio — Topic 6. Danh mục Tri thức của bạn

Ah, good old Ben Franklin—never at a loss for a pithy homily. Why, if we could just be early to bed and early to rise, we'd be great programmers—right? The early bird might get the worm, but what happens to the early worm?

À, ông Ben Franklin tốt bụng ngày xưa—chẳng bao giờ thiếu một câu châm ngôn súc tích. Nào, nếu chúng ta cứ ngủ sớm dậy sớm thì hẳn sẽ thành lập trình viên cừ khôi—đúng không? Con chim dậy sớm có thể bắt được sâu, nhưng còn con sâu dậy sớm thì sao?

In this case, though, Ben really hit the nail on the head. Your knowledge and experience are your most important day-to-day professional assets.

Tuy vậy, lần này thì Ben nói trúng phóc. Tri thức và kinh nghiệm chính là tài sản nghề nghiệp quan trọng nhất của bạn trong từng ngày làm việc.

Unfortunately, they're expiring assets. Your knowledge [9] becomes out of date as new techniques, languages, and environments are developed. Changing market forces may render your experience obsolete or irrelevant. Given the everincreasing pace of change in our technological society, this can happen pretty quickly.

Khốn nỗi, đó lại là những tài sản có hạn sử dụng. Tri thức của bạn [9] dần lỗi thời khi các kỹ thuật, ngôn ngữ và môi trường mới ra đời. Những biến động của thị trường có thể khiến kinh nghiệm của bạn trở nên lạc hậu hoặc chẳng còn liên quan. Với nhịp độ thay đổi ngày một nhanh trong xã hội công nghệ của chúng ta, điều này có thể xảy ra khá chóng vánh.

As the value of your knowledge declines, so does your value to your company or **client**. We want to prevent this from ever happening.

Khi giá trị tri thức của bạn sụt giảm, giá trị của bạn đối với công ty hay khách hàng cũng giảm theo. Chúng tôi muốn ngăn điều này xảy ra ngay từ đầu.

Your ability to learn new things is your most important strategic asset. But how do you learn how to learn, and how do you know what to learn?

Khả năng học những điều mới chính là tài sản chiến lược quan trọng nhất của bạn. Nhưng làm sao để học cách học, và làm sao để biết mình nên học gì?

Your Knowledge Portfolio — Danh mục Tri thức của bạn

We like to think of all the facts programmers know about computing, the application domains they work in, and all their experience as their knowledge portfolios. Managing a **knowledge portfolio** is very similar to managing a financial portfolio:

Chúng tôi thích hình dung tất cả những kiến thức mà các lập trình viên nắm về máy tính, về các lĩnh vực ứng dụng họ làm việc, cùng toàn bộ kinh nghiệm của họ như là một danh mục tri thức (knowledge portfolio). Quản lý một danh mục tri thức rất giống với quản lý một danh mục đầu tư tài chính:

1. Serious investors invest regularly—as a habit. 2. Diversification is the key to long-term success. 3. Smart investors balance their portfolios between conservative and high-risk, high-reward investments. 4. Investors try to buy low and sell high for maximum return. 5. Portfolios should be reviewed and rebalanced periodically.

1. Nhà đầu tư nghiêm túc đầu tư đều đặn—như một thói quen. 2. Đa dạng hóa là chìa khóa cho thành công dài hạn. 3. Nhà đầu tư khôn ngoan cân bằng danh mục giữa các khoản thận trọng và các khoản rủi ro cao, lợi nhuận cao. 4. Nhà đầu tư cố gắng mua thấp bán cao để tối đa hóa lợi nhuận. 5. Danh mục nên được rà soát và tái cân bằng định kỳ.

To be successful in your career, you must invest in **your knowledge portfolio** using these same guidelines.

Để thành công trong sự nghiệp, bạn phải đầu tư vào danh mục tri thức của mình theo đúng những nguyên tắc ấy.

The good news is that managing this kind of investment is a skill just like any other—it can be learned. The trick is to make yourself do it initially and form a habit. Develop a routine which you follow until your brain internalizes it. At that point, you'll find yourself sucking up new knowledge automatically.

Tin vui là việc quản lý kiểu đầu tư này cũng là một kỹ năng như mọi kỹ năng khác—nó học được. Mẹo nằm ở chỗ ban đầu phải tự ép mình làm cho thành thói quen. Hãy xây dựng một nếp sinh hoạt và bám theo nó cho tới khi bộ não thấm nhuần. Tới lúc đó, bạn sẽ thấy mình tự động hấp thụ tri thức mới.

Building Your Portfolio — Xây dựng Danh mục của bạn

Invest regularly Just as in financial investing, you must invest in your knowledge portfolio regularly, even if it's just a small amount. The habit is as important as the sums, so plan to use a consistent time and place, away from interruptions. A few sample goals are listed in the next section.

Đầu tư đều đặn Cũng như trong đầu tư tài chính, bạn phải đầu tư vào danh mục tri thức một cách đều đặn, dù chỉ là một khoản nhỏ. Thói quen quan trọng chẳng kém số tiền, vậy nên hãy định ra một thời điểm và một nơi chốn cố định, tránh xa các thứ làm gián đoạn. Một vài mục tiêu mẫu được liệt kê ở phần kế tiếp.

Diversify The more different things you know, the more valuable

Đa dạng hóa Càng biết nhiều thứ khác nhau, bạn

you are. As a baseline, you need to know the ins and outs of the particular technology you are working with currently. But don't stop there. The face of computing changes rapidly—hot technology today may well be close to useless (or at least not in demand) tomorrow. The more technologies you are

comfortable with, the better you will be able to adjust to change. And don't forget all the other skills you need, including those in nontechnical areas.

càng có giá trị. Ở mức cơ bản, bạn cần nắm tường tận ngọn ngành công nghệ cụ thể mình đang dùng hiện tại. Nhưng đừng dừng ở đó. Bộ mặt của ngành máy tính thay đổi nhanh chóng—công nghệ nóng hồi hôm nay rất có thể gần như vô dụng (hoặc ít ra là không còn được săn đón) vào ngày mai. Càng thoải mái với nhiều công nghệ, bạn càng dễ thích nghi với đổi thay. Và đừng quên mọi kỹ năng khác bạn cần, kể cả những kỹ năng thuộc lĩnh vực phi kỹ thuật.

Manage risk Technology exists along a spectrum from risky, potentially high-reward to low-risk, low-reward standards. It's not a good idea to invest all of your money in high-risk stocks that might collapse suddenly, nor should you invest all of it conservatively and miss out on possible opportunities. Don't put all your technical eggs in one basket.

Quản lý rủi ro Công nghệ trải dài trên một phổ, từ những thứ rủi ro nhưng tiềm năng lợi nhuận cao cho tới những chuẩn mực ít rủi ro, lợi nhuận thấp. Để hết tiền vào cổ phiếu rủi ro cao có thể sụp đổ bất thành hình thì không hay, mà rút tất cả một cách thận trọng để rồi bỏ lỡ các cơ hội cũng chẳng nên. Đừng đặt tất cả trứng kỹ thuật vào một giỏ.

Buy low, sell high Learning an emerging technology before it becomes popular can be just as hard as finding an undervalued stock, but the payoff can be just as rewarding. Learning Java back when it was first introduced and unknown may have been risky at the time, but it paid off handsomely for the early adopters when it became an industry mainstay later.

Mua thấp, bán cao Học một công nghệ đang manh nha trước khi nó trở nên phổ biến có thể khó chẳng kém việc tìm ra một cổ phiếu bị định giá thấp, nhưng phần thưởng cũng xứng đáng không kém. Học Java thuở nó mới ra đời và còn vô danh có lẽ là chuyện mạo hiểm lúc bấy giờ, nhưng nó đã mang lại quả ngọt hậu hĩnh cho những người tiên phong khi Java sau này trở thành trụ cột của ngành.

Review and rebalance This is a very dynamic industry. That hot technology you started investigating last month might be stone cold by now. Maybe you need to brush up on that database technology that you haven't used in a while. Or perhaps

Rà soát và tái cân bằng Đây là một ngành rất năng động. Cái công nghệ nóng hồi mà bạn bắt đầu tìm hiểu tháng trước giờ có thể đã nguội ngắt. Có lẽ bạn cần ôn lại công nghệ cơ sở dữ liệu mà đã lâu mình không động tới. Hay biết đâu bạn sẽ ở vị thế tốt hơn cho cơ hội việc làm mới kia

you could be better positioned for that new job opening if you tried out that other language....

nếu thử qua ngôn ngữ nọ....

Of all these guidelines, the most important one is the simplest to do:

Trong tất cả các nguyên tắc này, điều quan trọng nhất lại là điều dễ làm nhất:

Tip 9: Invest Regularly in Your Knowledge Portfolio

Tip 9: Đầu Tư Điều Đặn Vào Danh Mục Tri Thức Của Bạn

Goals — Mục tiêu

Now that you have some guidelines on what and when to add to your knowledge portfolio, what's the best way to go about acquiring intellectual capital with which to fund your portfolio? Here are a few suggestions:

Giờ bạn đã có vài nguyên tắc về việc nên thêm gì và thêm khi nào vào danh mục tri thức, vậy đâu là cách tốt nhất để thu nạp vốn trí tuệ làm nguồn vốn cho danh mục? Đây là vài gợi ý:

Learn at least one new language every year Different languages solve the same problems in different ways. By learning several different approaches, you can help broaden your thinking and avoid getting stuck in a rut. Additionally, learning many languages is easy thanks to the wealth of freely available software.

Học ít nhất một ngôn ngữ mới mỗi năm Những ngôn ngữ khác nhau giải cùng một bài toán theo những cách khác nhau. Bằng cách học vài hướng tiếp cận khác nhau, bạn có thể mở rộng lối tư duy và tránh sa vào lối mòn. Hơn nữa, học nhiều ngôn ngữ giờ rất dễ nhờ kho phần mềm miễn phí dồi dào.

Read a technical book each month While there's a glut of short-form essays and occasionally reliable answers on the web, for deep understanding you need long-form books. Browse the booksellers for technical books on interesting topics related to your current project. [10] Once you're in the habit, read a book a month. After you've mastered the technologies you're currently using, branch out and study some that don't relate to your project.

Đọc một cuốn sách kỹ thuật mỗi tháng Tuy có cả một biển bài luận ngắn và những câu trả lời thỉnh thoảng đáng tin trên web, nhưng để hiểu sâu bạn cần những cuốn sách dài hơi. Hãy lướt các hiệu sách tìm sách kỹ thuật về những chủ đề thú vị liên quan đến dự án hiện tại của bạn. [10] Khi đã thành nếp, mỗi tháng đọc một cuốn. Sau khi đã làm chủ những công nghệ đang dùng, hãy mở rộng ra và nghiên cứu cả những thứ chẳng liên quan đến dự án.

Read nontechnical books, too

Đọc cả sách phi kỹ thuật nữa

It is important to remember that computers are used by people —people whose needs you are trying to satisfy. You work with people, are employed by people, and get hacked by people. Don't forget the human side of the equation, as that requires an entirely different skill set (we ironically call these soft skills, but they are actually quite hard to master).

Quan trọng là phải nhớ rằng máy tính được con người sử dụng—những con người mà bạn đang cố đáp ứng nhu cầu. Bạn làm việc với con người, được con người thuê, và bị con người hack. Đừng quên khía cạnh con người trong phương trình, bởi điều đó đòi hỏi một bộ kỹ năng hoàn toàn khác (chúng tôi gọi đùa chúng là kỹ năng mềm, nhưng thực ra rất khó nhằn để thành thạo).

Take classes Look for interesting courses at a local or online college or university, or perhaps at the next nearby trade show or conference.

Tham gia các lớp học Tìm những khóa học thú vị ở một trường cao đẳng hay đại học gần nhà hoặc trực tuyến, hay có lẽ tại hội chợ thương mại hoặc hội thảo gần đó sắp diễn ra.

Participate in local user groups and meetups Isolation can be deadly to your career; find out what people are working on outside of your company. Don't just go and listen: actively participate.

Tham gia các nhóm người dùng và buổi gặp mặt ở địa phương Sự cô lập có thể là tử huyệt cho sự nghiệp của bạn; hãy tìm hiểu xem người ta đang làm gì bên ngoài công ty bạn. Đừng chỉ tới ngồi nghe: hãy chủ động tham gia.

Experiment with different environments If you've worked only in Windows, spend some time with Linux. If you've used only makefile s and an editor, try a sophisticated IDE with cutting-edge features,

and vice versa.

Thử nghiệm với các môi trường khác nhau Nếu bạn chỉ từng làm trên Windows, hãy dành chút thời gian với Linux. Nếu bạn chỉ dùng các makefile và một trình soạn thảo, hãy thử một IDE tinh vi với các tính năng tân tiến, và ngược lại.

Stay current Read news and posts online on technology different from that of your current project. It's a great way to find out what experiences other people are having with it, the particular jargon they use, and so on.

Bắt kịp thời cuộc Đọc tin tức và bài viết trực tuyến về công nghệ khác với công nghệ của dự án hiện tại. Đó là cách tuyệt vời để biết người khác đang có trải nghiệm gì với nó, họ dùng những thuật ngữ riêng nào, v.v.

It's important to continue investing. Once you feel comfortable with some new language or bit of technology, move on. Learn another one.

Quan trọng là phải tiếp tục đầu tư. Một khi bạn thấy thoải mái với một ngôn ngữ hay mẫu công nghệ mới nào đó, hãy đi tiếp. Học thêm cái khác.

It doesn't matter whether you ever use any of these technologies on a project, or even whether you put them on your resume. The **process** of learning will expand your thinking, opening you to new possibilities and new ways of doing things. The crosspollination of ideas is important; try to apply the lessons you've learned to your current project. Even if your project doesn't use that technology, perhaps you can borrow some ideas. Get familiar with object orientation, for instance, and you'll write procedural programs differently. Understand the functional programming paradigm and you'll write object-oriented code differently, and so on.

Chẳng quan trọng việc liệu bạn có bao giờ dùng những công nghệ này trong một dự án hay không, thậm chí có ghi chúng vào hồ sơ xin việc hay không. Chính quá trình học sẽ mở rộng tư duy của bạn, mở ra những khả năng mới và những cách làm mới. Sự thụ phấn chéo của các ý tưởng rất quan trọng; hãy thử áp dụng những bài học bạn đã rút ra vào dự án hiện tại. Ngay cả khi dự án không dùng công nghệ ấy, biết đâu bạn có thể vay mượn vài ý tưởng. Chẳng hạn, hãy làm quen với lập trình hướng đối tượng, và bạn sẽ viết chương trình thủ tục theo cách khác. Hiểu mô hình lập trình hàm, và bạn sẽ viết mã hướng đối tượng theo cách khác, và cứ thế.

Opportunities For Learning — Cơ hội Học hỏi

So you're reading voraciously, you're on top of all the latest breaking developments in your field (not an easy thing to do), and somebody asks you a question. You don't have the faintest idea what the answer is, and freely admit as much.

Vậy là bạn đọc ngấu ngiến, bạn nắm bắt mọi diễn biến mới nhất trong lĩnh vực của mình (chẳng phải chuyện dễ), thế rồi ai đó hỏi bạn một câu. Bạn chẳng có lấy một mảy may ý niệm câu trả lời là gì, và thành thật thừa nhận điều đó.

Don't let it stop there. Take it as a personal challenge to find the answer. Ask around. Search the web—the scholarly parts too, not just the consumer parts.

Đừng để mọi chuyện dừng ở đó. Hãy coi việc tìm ra câu trả lời như một thử thách cá nhân. Hỏi quanh. Tìm trên web—cả những phần học thuật chứ không chỉ phần dành cho người tiêu dùng.

If you can't find the answer yourself, find out who can. Don't let it rest. Talking to other people will help build your personal network, and you may surprise yourself by finding solutions to other,

unrelated problems along the way. And that old portfolio just keeps getting bigger....

Nếu tự mình không tìm được câu trả lời, hãy tìm xem ai tìm được. Đừng để nó yên đó. Nói chuyện với người khác sẽ giúp gây dựng mạng lưới quan hệ cá nhân của bạn, và bạn có thể tự làm mình ngạc nhiên khi trên đường đi tình cờ tìm ra lời giải cho những vấn đề khác chẳng liên quan. Và cái danh mục cũ kỹ kia cứ thế ngày một phình to....

All of this reading and researching takes time, and time is already in short supply. So you need to plan ahead. Always have something to read in an otherwise dead moment. Time spent waiting for doctors and dentists can be a great opportunity to catch up on your reading—but be sure to bring your own e-

Tất cả việc đọc và nghiên cứu này tốn thời gian, mà thời gian thì vốn đã eo hẹp. Vậy nên bạn cần lên kế hoạch trước. Hãy luôn có thứ gì đó để đọc trong những lúc rảnh rỗi vô vị. Thời gian ngồi chờ bác sĩ hay nha sĩ có thể là cơ hội tuyệt vời để đọc bù—nhưng nhớ mang theo máy đọc

reader with you, or you might find yourself thumbing through a dog-eared 1973 article about Papua New Guinea.

sách điện tử của riêng bạn, kéo lại thấy mình đang lật giở một bài báo nhàu nát năm 1973 viết về Papua New Guinea.

Critical Thinking — Tư duy Phản biện

The last important point is to think critically about what you read and hear. You need to ensure that the knowledge in your portfolio is accurate and unswayed by either vendor or media hype. Beware of the zealots who insist that their dogma provides the only answer—it may or may not be applicable to you and your project.

Điểm quan trọng cuối cùng là phải tư duy phản biện về những gì bạn đọc và nghe. Bạn cần đảm bảo rằng tri thức trong danh mục của mình là chính xác và không bị lung lay bởi chiêu trò thổi phồng của nhà cung cấp hay truyền thông. Hãy dè chừng những kẻ cuồng tín cứ khẳng định rằng giáo điều của họ là câu trả lời duy nhất—nó có thể đúng mà cũng có thể chẳng phù hợp với bạn và dự án của bạn.

Never underestimate the power of commercialism. Just because a web search engine lists a hit first doesn't mean that it's the best match; the content provider can pay to get top billing. Just because a bookstore features a book prominently doesn't mean it's a good book, or even popular; they may have been paid to place it there.

Đừng bao giờ đánh giá thấp sức mạnh của thương mại. Việc một công cụ tìm kiếm xếp một kết quả lên đầu chẳng có nghĩa nó là kết quả khớp nhất; bên cung cấp nội dung có thể trả tiền để được lên top. Việc một hiệu sách trưng bày một cuốn sách ở vị trí nổi bật chẳng có nghĩa đó là sách hay, hay thậm chí ấn khách; có khi họ được trả tiền để bày nó ở đó.

Tip 10: Critically Analyze What You Read and Hear

Tip 10: Phân Tích Phản Biện Những Gì Bạn Đọc Và Nghe

Critical thinking is an entire discipline unto itself, and we encourage you to read and study all you can about it. In the meantime, here's a head start with a few questions to ask and think about.

Tư duy phản biện tự nó là cả một bộ môn, và chúng tôi khuyến khích bạn đọc và học mọi thứ có thể về nó. Trong lúc chờ, đây là một cú khởi đầu với vài câu hỏi để đặt ra và suy ngẫm.

Ask the “Five Whys” A favorite consulting trick: ask “why?” at least five times. Ask a question, and get an answer. Dig deeper by asking “why?” Repeat as if you were a petulant four-year old (but a polite one). You might be able to get closer to a root cause this way.

Hỏi “Năm lần Tại sao” Một mảnh tư vấn được ưa chuộng: hỏi “tại sao?” ít nhất năm lần. Đặt một câu hỏi, nhận một câu trả lời. Đào sâu hơn bằng cách hỏi “tại sao?”. Lặp lại như thể bạn là một đứa trẻ bốn tuổi hay hơn (nhưng là một đứa lễ phép). Bằng cách này bạn có thể tiến gần hơn tới gốc rễ vấn đề.

Who does this benefit? It may sound cynical, but follow the money can be a very helpful path to analyze. The benefits to someone else or another organization may be aligned with your own, or not.

Việc này có lợi cho ai? Nghe có vẻ hoài nghi, nhưng “lần theo dòng tiền” có thể là một hướng phân tích rất hữu ích. Lợi ích cho một người khác hay một tổ chức khác có thể trùng khớp với lợi ích của bạn, mà cũng có thể không.

What’s the **context**? Everything occurs in its own context, which is why “one size fits all” solutions often don’t. Consider an article or book touting a “best practice.” Good questions to consider are “best for who?” What are the prerequisites, what are the consequences, short and long term?

Bối cảnh là gì? Mọi thứ đều diễn ra trong bối cảnh riêng của nó, chính vì thế mà các giải pháp “một cỡ vừa cho tất cả” thường không ổn. Hãy xét một bài báo hay cuốn sách ca ngợi một “thông lệ tốt nhất” (best practice). Những câu hỏi hay cần cân nhắc là “tốt nhất cho ai?”. Đây là các điều kiện tiên quyết, đây là hệ quả, ngắn hạn lẫn dài hạn?

When or Where would this work? Under what circumstances? Is it too late? Too early? Don’t stop with first-order thinking (what will happen next), but use second-order thinking: what will happen after that?

Khi nào hay Ở đâu thì việc này có tác dụng? Trong những hoàn cảnh nào? Liệu có quá muộn? Quá sớm? Đừng dừng ở tư duy bậc nhất (chuyện gì sẽ xảy ra tiếp theo), mà hãy dùng tư duy bậc hai: sau đó thì chuyện gì sẽ xảy ra?

Why is this a problem? Is there an underlying model? How does the underlying model work?

Tại sao đây lại là một vấn đề? Có một mô hình nền tảng nào không? Mô hình nền tảng ấy vận hành ra sao?

Unfortunately, there are very few simple answers anymore. But with your extensive portfolio, and by applying some critical analysis to the torrent of technical articles you will read, you can understand the complex answers.

Khốn nỗi, ngày nay rất hiếm câu trả lời đơn giản. Nhưng với cái danh mục đồ sộ của bạn, và bằng cách áp dụng chút phân tích phản biện vào dòng thác bài báo kỹ thuật mà bạn sẽ đọc, bạn có thể hiểu được những câu trả lời phức tạp.

Related Sections Include — Các mục liên quan

Topic 1, It’s Your Life

Topic 1, Đời bạn là của bạn

Topic 22, Engineering Daybooks

Topic 22, Nhật ký Kỹ thuật

Challenges — Thử thách

Start learning a new language this week. Always programmed in the same old language? Try Clojure, Elixir, Elm, F#, Go, Haskell, Python, R, ReasonML, Ruby, Rust, Scala, Swift, TypeScript, or anything else that appeals and/or looks as if you might like it. [11]

Tuần này hãy bắt đầu học một ngôn ngữ mới. Cứ mãi lập trình bằng đúng cái ngôn ngữ cũ rích? Hãy thử Clojure, Elixir, Elm, F#, Go, Haskell, Python, R, ReasonML, Ruby, Rust, Scala, Swift, TypeScript, hoặc bất cứ thứ gì cuốn hút bạn và/hoặc trông có vẻ bạn sẽ thích. [11]

Start reading a new book (but finish this one first!). If you are doing very detailed implementation and coding, read a book on design and architecture. If you are doing high-level design, read a book on coding techniques.

Hãy bắt đầu đọc một cuốn sách mới (nhưng đọc xong cuốn này đã!). Nếu bạn đang làm những việc cài đặt và viết mã rất chi tiết, hãy đọc một cuốn về thiết kế và kiến trúc. Nếu bạn đang làm thiết kế ở tầng cao, hãy đọc một cuốn về kỹ thuật viết mã.

Get out and talk technology with people who aren't involved in your current project, or who don't work for the same company. Network in your company cafeteria, or maybe seek out fellow enthusiasts at a local meetup.

Hãy ra ngoài và bàn chuyện công nghệ với những người không dính dáng đến dự án hiện tại của bạn, hoặc không làm cùng công ty. Kết nối ở căng-tin công ty, hay có lẽ tìm những người cùng đam mê tại một buổi gặp mặt địa phương.

I believe that it is better to be looked over than it is to be overlooked. Mae West, Belle of the Nineties, 1934

Tôi tin rằng được người ta soi xét vẫn hơn là bị ngó lơ. Mae West, Belle of the Nineties, 1934

Topic 7. Communicate! — Topic 7. Hãy giao tiếp!

Maybe we can learn a lesson from Ms. West. It's not just what you've got, but also how you package it. Having the best ideas, the finest code, or the most **pragmatic** thinking is ultimately sterile unless you can communicate with other people. A good idea is an **orphan** without effective communication.

Có lẽ chúng ta học được một bài học từ cô West. Vấn đề không chỉ nằm ở thứ bạn có, mà còn ở cách bạn gói ghém nó lại. Dù bạn có những ý tưởng hay nhất, đoạn mã tinh xảo nhất, hay lối tư duy thực dụng (pragmatic) nhất, tất cả rốt cuộc cũng vô sinh nếu bạn không thể truyền đạt với người khác. Một ý tưởng hay sẽ trở thành kẻ mồ côi nếu thiếu khả năng giao tiếp hiệu quả.

As developers, we have to communicate on many levels. We spend hours in meetings, listening and talking. We work with end users, trying to understand their needs. We write code, which communicates our intentions to a machine and documents our thinking for future generations of developers. We write proposals and memos requesting and justifying resources, reporting our status, and suggesting new approaches. And we work daily within our teams to advocate our ideas, modify existing practices, and suggest new ones. A large part of our day is spent communicating, so we need to do it well.

Là người phát triển phần mềm, chúng ta phải giao tiếp ở rất nhiều cấp độ. Chúng ta ngồi hàng giờ trong các cuộc họp, vừa nghe vừa nói. Chúng ta làm việc với người dùng cuối, cố gắng hiểu nhu cầu của họ. Chúng ta viết mã, thứ vừa truyền đạt ý định của ta cho máy

tính vừa ghi lại lối suy nghĩ của ta cho các thể hệ lập trình viên sau này. Chúng ta viết đề xuất và bản ghi nhớ để xin và biện minh cho nguồn lực, báo cáo tiến độ, và gợi ý những hướng đi mới. Và hằng ngày chúng ta làm việc trong nhóm để bảo vệ ý tưởng của mình, điều chỉnh các thói quen hiện có, và đề xuất những cách làm mới. Một phần lớn thời gian trong ngày của ta là để giao tiếp, nên ta cần làm điều đó cho thật tốt.

Treat English (or whatever your native tongue may be) as just another programming language. Write natural language as you would write code: honor the **DRY** principle, **ETC**, automation, and so on. (We discuss the DRY and ETC design principles in the next chapter.)

Hãy coi tiếng Anh (hay bất cứ tiếng mẹ đẻ nào của bạn) cũng chỉ là một ngôn ngữ lập trình khác. Hãy viết bằng ngôn ngữ tự nhiên như khi bạn viết mã: tôn trọng nguyên tắc DRY — Đừng Lập Lại Chính Mình, ETC — Dễ Thay Đổi Hơn, sự tự động hóa, v.v. (Chúng tôi bàn về hai nguyên tắc thiết kế DRY và ETC ở chương kế tiếp.)

Tip 11: English is Just Another Programming Language

Tip 11: Tiếng Anh cũng chỉ là một Ngôn ngữ Lập trình khác

We've put together a list of additional ideas that we find useful.

Chúng tôi đã tổng hợp một danh sách những ý tưởng bổ sung mà chúng tôi thấy hữu ích.

Know Your Audience — Hiểu khán giả của bạn

You're communicating only if you're conveying what you mean to convey—just talking isn't enough. To do that, you need to understand the needs, interests, and capabilities of your audience. We've all sat in meetings where a development geek glazes over the eyes of the vice president of marketing with a long monologue on the merits of some arcane technology. This isn't communicating: it's just talking, and it's annoying. [12]

Bạn chỉ thực sự giao tiếp khi bạn truyền tải được đúng điều mình muốn nói — chỉ nói thôi thì chưa đủ. Để làm được điều đó, bạn cần hiểu nhu cầu, mối quan tâm và năng lực của khán giả. Hẳn ai trong chúng ta cũng từng ngồi trong những cuộc họp mà một anh chàng mê công nghệ làm phó chủ tịch marketing đờ đẫn cả mắt bằng một bài độc thoại dài dằng dặc về cái hay của một công nghệ bí hiểm nào đó. Đây không phải là giao tiếp: đó chỉ là nói cho có, và nó thật phiền toái. [12]

Say you want to change your remote monitoring system to use a third-party message broker to disseminate status notifications. You can present this update in many different ways, depending on your audience. End users will appreciate that their systems can now interoperate with other services that use the broker. Your marketing department will be able to use this fact to boost sales. Development and operations managers will be happy because the care and maintenance of that part of the system is now someone else's problem. Finally, developers may enjoy getting experience with new APIs, and may even be able to find new uses for the message broker. By making the appropriate pitch to each group, you'll get them all excited about your project.

Giả sử bạn muốn thay đổi hệ thống giám sát từ xa của mình để dùng một message broker của bên thứ ba nhằm phát đi các thông báo trạng thái. Bạn có thể trình bày bản cập nhật này theo nhiều cách khác nhau, tùy vào khán giả. Người dùng cuối sẽ thích thú vì hệ thống của họ giờ có thể liên thông với các dịch vụ khác cũng dùng broker đó. Bộ phận marketing của bạn sẽ tận dụng được điều này để đẩy doanh số. Các quản lý phát triển và vận hành sẽ vui vì việc chăm sóc và bảo trì phần đó của hệ thống giờ đã là vấn đề của người khác. Cuối cùng, các lập trình viên có thể thích thú khi được trải nghiệm những API mới, và thậm chí

còn tìm ra những cách dùng mới cho message broker. Bằng cách đưa ra lời chào hàng phù hợp cho từng nhóm, bạn sẽ khiến tất cả họ đều háo hức với dự án của mình.

As with all forms of communication, the trick here is to gather feedback. Don't just wait for questions: ask for them. Look at body language, and facial expressions. One of the Neuro Linguistic Programming presuppositions is "The meaning of your communication is the response you get." Continuously improve your knowledge of your audience as you communicate.

Cũng như mọi hình thức giao tiếp khác, mảnh ở đây là thu thập phản hồi. Đừng chỉ ngồi chờ câu hỏi: hãy chủ động mời gọi chúng. Hãy quan sát ngôn ngữ cơ thể và biểu cảm gương mặt. Một trong những tiền đề của Lập trình Ngôn ngữ Tư duy (Neuro Linguistic Programming) là "Ý nghĩa của lời bạn nói chính là phản ứng mà bạn nhận được." Hãy liên tục cải thiện hiểu biết của bạn về khán giả ngay trong lúc giao tiếp.

Know What You Want To Say — Biết rõ bạn muốn nói gì

Probably the most difficult part of the more formal styles of communication used in business is working out exactly what it is you want to say. Fiction writers often plot out their books in detail before they start, but people writing technical documents are often happy to sit down at a keyboard, enter:

Phần khó nhất của những kiểu giao tiếp mang tính trang trọng hơn trong môi trường công sở có lẽ là xác định cho rõ chính xác điều bạn muốn nói. Các nhà văn viết tiểu thuyết thường dựng chi tiết cốt truyện sách của họ trước khi bắt tay viết, nhưng người viết tài liệu kỹ thuật lại thường hài lòng với việc ngồi xuống trước bàn phím, gõ:

1. Introduction

1. Giới thiệu

and start typing whatever comes into their heads next.

rồi bắt đầu gõ bất cứ điều gì lóe lên trong đầu tiếp theo.

Plan what you want to say. Write an outline. Then ask yourself, "Does this communicate what I want to express to my audience in a way that works for them?" Refine it until it does.

Hãy lên kế hoạch cho điều bạn muốn nói. Hãy viết một dàn ý. Rồi tự hỏi: "Cái này có truyền tải được điều mình muốn diễn đạt tới khán giả theo cách hợp với họ không?" Hãy mài giũa nó cho tới khi đạt.

This approach works for more than just documents. When you're faced with an important meeting or a chat with a major **client**, jot down the ideas you want to communicate, and plan a couple of strategies for getting them across.

Cách tiếp cận này không chỉ áp dụng cho tài liệu. Khi bạn sắp đối mặt với một cuộc họp quan trọng hay một buổi trò chuyện với khách hàng lớn, hãy ghi nhanh ra những ý mình muốn truyền đạt, và vạch sẵn vài chiến lược để chuyển tải chúng.

Now that you know what your audience wants, let's deliver it.

Giờ thì bạn đã biết khán giả muốn gì, hãy mang đến cho họ điều đó.

Choose Your Moment — Chọn đúng thời điểm

It's six o'clock on Friday afternoon, following a week when the auditors have been in. Your boss's youngest is in the hospital, it's pouring rain outside, and the commute home is guaranteed to be a

nightmare. This probably isn't a good time to ask her for a memory upgrade for your laptop.

Đang là sáu giờ chiều thứ Sáu, sau một tuần kiểm toán viên ra vào liên tục. Đứa con nhỏ nhất của sếp bạn đang nằm viện, ngoài trời mưa như trút, và chặng đường về nhà chắc chắn sẽ là một cơn ác mộng. Đây hẳn không phải lúc thích hợp để xin bà ấy nâng cấp bộ nhớ cho chiếc laptop của bạn.

As part of understanding what your audience needs to hear, you need to work out what their priorities are. Catch a manager who's just been given a hard time by her boss because some

Là một phần của việc hiểu xem khán giả cần nghe gì, bạn cần xác định đâu là ưu tiên của họ. Hãy bắt gặp một người quản lý vừa bị sếp của cô ấy mắng cho một trận vì một mớ

source code got lost, and you'll have a more receptive listener to your ideas on source code repositories. Make what you're saying relevant in time, as well as in content. Sometimes all it takes is the simple question, "Is this a good time to talk about...?"

mã nguồn bị thất lạc, và bạn sẽ có một người nghe cởi mở hơn nhiều với những ý tưởng của bạn về kho lưu mã nguồn. Hãy làm cho điều bạn nói trở nên phù hợp cả về thời điểm lẫn nội dung. Đôi khi tất cả những gì cần làm chỉ là một câu hỏi đơn giản: "Bây giờ có phải lúc tiện để nói về...?"

Choose A Style — Chọn một phong cách

Adjust the style of your delivery to suit your audience. Some people want a formal "just the facts" briefing. Others like a long, wide-ranging chat before getting down to business. What is their skill level and experience in this area? Are they experts? Newbies? Do they need hand-holding or just a quick tl;dr? If in doubt, ask.

Hãy điều chỉnh phong cách trình bày cho hợp với khán giả của bạn. Có người muốn một buổi báo cáo trang trọng kiểu "chỉ cần sự thật thôi". Người khác lại thích một cuộc chuyện trò dài, rộng đủ thứ trước khi vào việc chính. Trình độ và kinh nghiệm của họ trong lĩnh vực này ra sao? Họ là chuyên gia? Hay tay mơ? Họ cần được dắt tay từng bước hay chỉ muốn một bản tóm tắt tl;dr thật nhanh? Nếu phân vân, cứ hỏi.

Remember, however, that you are half of the communication transaction. If someone says they need a paragraph describing something and you can't see any way of doing it in less than several pages, tell them so. Remember, that kind of feedback is a form of communication, too.

Tuy nhiên, hãy nhớ rằng bạn là một nửa của cuộc giao dịch giao tiếp này. Nếu có ai bảo họ cần một đoạn văn mô tả điều gì đó mà bạn lại không thấy cách nào làm gọn dưới mấy trang, hãy nói thẳng với họ. Nhớ nhé, kiểu phản hồi đó cũng là một hình thức giao tiếp.

Make It Look Good — Làm cho nó trông đẹp mắt

Your ideas are important. They deserve a good-looking vehicle to convey them to your audience.

Ý tưởng của bạn rất quan trọng. Chúng xứng đáng có một phương tiện trông tươi tắn để chuyển tải tới khán giả.

Too many developers (and their managers) concentrate solely on content when producing written documents. We think this is a mistake. Any chef (or watcher of the Food Network) will tell you that you can slave in the kitchen for hours only to ruin your efforts with poor presentation.

Quá nhiều lập trình viên (và cả các quản lý của họ) chỉ chăm chăm vào nội dung khi soạn tài liệu. Chúng tôi cho rằng đó là một sai lầm. Bất kỳ đầu bếp nào (hoặc bất kỳ ai hay xem

kênh Food Network) cũng sẽ bảo bạn rằng bạn có thể đổ mồ hôi trong bếp hàng giờ liền chỉ để rồi phá hỏng mọi công sức bằng cách bày biện vụng về.

There is no excuse today for producing poor-looking printed documents. Modern software can produce stunning output, regardless of whether you're writing using Markdown or using a

Ngày nay không có lý do gì để cho ra đời những tài liệu in ấn trông cầu thả. Phần mềm hiện đại có thể tạo ra kết quả tuyệt đẹp, bất kể bạn viết bằng Markdown hay bằng

word processor. You need to learn just a few basic commands. If you're using a word processor, use its style sheets for consistency. (Your company may already have defined style sheets that you can use.) Learn how to set page headers and footers. Look at the sample documents included with your package to get ideas on style and layout. Check the spelling, first automatically and then by hand. After all, there are spelling miss steaks that the chequer can knot ketch.

một trình soạn thảo văn bản. Bạn chỉ cần học vài lệnh cơ bản. Nếu dùng trình soạn thảo văn bản, hãy dùng style sheet của nó để giữ tính nhất quán. (Công ty bạn có thể đã định sẵn các style sheet để bạn dùng.) Hãy học cách đặt phần đầu trang và chân trang. Hãy ngó qua các tài liệu mẫu đi kèm gói phần mềm để lấy ý tưởng về phong cách và bố cục. Hãy kiểm tra chính tả, đầu tiên là tự động rồi sau đó bằng tay. Bởi nhẽ, vẫn có nhữn lỗi chín tả mà trình kiểm chả thể bắt nủi đâu.

Involve Your Audience — Lôi kéo khán giả tham gia

We often find that the documents we produce end up being less important than the **process** we go through to produce them. If possible, involve your readers with early drafts of your document. Get their feedback, and pick their brains. You'll build a good working relationship, and you'll probably produce a better document in the process.

Chúng tôi thường thấy rằng những tài liệu mình tạo ra rốt cuộc lại kém quan trọng hơn so với chính quá trình chúng tôi đã trải qua để tạo ra chúng. Nếu có thể, hãy cho người đọc của bạn tham gia ngay từ những bản nháp đầu tiên. Hãy lấy phản hồi của họ, và khai thác chất xám của họ. Bạn sẽ xây dựng được một mối quan hệ làm việc tốt đẹp, và nhiều khả năng bạn còn tạo ra được một tài liệu hay hơn trong quá trình đó.

Be A Listener — Hãy là người biết lắng nghe

There's one technique that you must use if you want people to listen to you: listen to them. Even if this is a situation where you have all the information, even if this is a formal meeting with you standing in front of 20 suits—if you don't listen to them, they won't listen to you.

Có một kỹ thuật mà bạn nhất định phải dùng nếu muốn người ta lắng nghe mình: hãy lắng nghe họ. Ngay cả khi đây là tình huống bạn nắm trong tay mọi thông tin, ngay cả khi đây là một cuộc họp trang trọng với bạn đứng trước 20 bộ vest — nếu bạn không lắng nghe họ, họ cũng sẽ không lắng nghe bạn.

Encourage people to talk by asking questions, or ask them to restate the discussion in their own words. Turn the meeting into a dialog, and you'll make your point more effectively. Who knows, you might even learn something.

Hãy khuyến khích người ta nói bằng cách đặt câu hỏi, hoặc nhờ họ diễn đạt lại nội dung thảo luận theo cách của riêng mình. Hãy biến cuộc họp thành một cuộc đối thoại, và bạn sẽ truyền đạt quan điểm của mình hiệu quả hơn. Mà biết đâu đấy, bạn còn học được điều gì đó.

Get Back To People — Hãy hồi đáp người khác

If you ask someone a question, you feel they're impolite if they don't respond. But how often do you fail to get back to people

Khi bạn hỏi ai đó một câu, bạn sẽ thấy họ bất lịch sự nếu họ không trả lời. Nhưng bao nhiêu lần chính bạn lại quên hồi đáp người ta

when they send you an email or a memo asking for information or requesting some action? In the rush of everyday life, it's easy to forget. Always respond to emails and voicemails, even if the response is simply "I'll get back to you later." Keeping people informed makes them far more forgiving of the occasional slip, and makes them feel that you haven't forgotten them.

khi họ gửi cho bạn một email hay một bản ghi nhớ để hỏi thông tin hoặc yêu cầu làm một việc gì đó? Trong guồng quay tất bật của cuộc sống thường nhật, rất dễ quên. Hãy luôn phản hồi email và tin nhắn thoại, dù câu trả lời chỉ đơn giản là "Tôi sẽ liên hệ lại với bạn sau." Việc luôn cập nhật thông tin cho người khác khiến họ dễ thông cảm hơn nhiều với những sơ suất thi thoảng của bạn, và khiến họ cảm thấy bạn không hề bỏ quên họ.

Tip 12: It's Both What You Say and the Way You Say It

Tip 12: Vấn Đề Vừa Là Điều Bạn Nói, Vừa Là Cách Bạn Nói

Unless you work in a vacuum, you need to be able to communicate. The more effective that communication, the more influential you become.

Trừ phi bạn làm việc trong một khoảng chân không, bạn cần có khả năng giao tiếp. Giao tiếp càng hiệu quả, bạn càng có sức ảnh hưởng.

Documentation — Tài liệu

Finally, there's the matter of communicating via documentation. Typically, developers don't give much thought to documentation. At best it is an unfortunate necessity; at worst it is treated as a low-priority task in the hope that management will forget about it at the end of the project.

Cuối cùng, có một vấn đề là giao tiếp thông qua tài liệu. Thông thường, các lập trình viên chẳng mấy bận tâm đến tài liệu. Tốt lắm thì nó là một nỗi khổ bất đắc dĩ; tệ nhất thì nó bị xem như một việc ưu tiên thấp, với hy vọng rằng ban quản lý sẽ quên bống nó đi vào lúc kết thúc dự án.

Pragmatic Programmers embrace documentation as an integral part of the overall development process. Writing documentation can be made easier by not duplicating effort or wasting time, and by keeping documentation close at hand—in the code itself. In fact, we want to apply all of our pragmatic principles to documentation as well as to code.

Lập trình viên Thực dụng (Pragmatic Programmer) coi tài liệu là một phần không thể tách rời của toàn bộ quá trình phát triển. Việc viết tài liệu có thể được làm cho dễ dàng hơn bằng cách không lặp lại công sức hay phí phạm thời gian, và bằng cách giữ tài liệu luôn trong tầm tay — ngay trong chính mã nguồn. Thực vậy, chúng tôi muốn áp dụng tất cả các nguyên tắc thực dụng của mình cho tài liệu y như cho mã nguồn.

Tip 13: Build Documentation In, Don't Bolt It On

Tip 13: Hãy Gắn Tài Liệu Vào Trong, Đừng Chắp Nó Ở Ngoài

It's easy to produce good-looking documentation from the comments in source code, and we recommend adding

Việc tạo ra tài liệu trông đẹp mắt từ các chú thích trong mã nguồn là rất dễ, và chúng tôi khuyên bạn nên thêm

comments to modules and exported functions to give other developers a leg up when they come to use it.

chú thích vào các module và các hàm được export để giúp các lập trình viên khác có chỗ bám khi họ bắt tay vào dùng chúng.

However, this doesn't mean we agree with the folks who say that every function, data structure, type declaration, etc., needs its own comment. This kind of mechanical comment writing actually makes it more difficult to maintain code: now there are two things to update when you make a change. So restrict your non-API commenting to discussing why something is done, its purpose and its goal. The code already shows how it is done, so commenting on this is redundant—and is a violation of the DRY principle.

Tuy nhiên, điều này không có nghĩa là chúng tôi đồng tình với những người cho rằng mọi hàm, mọi cấu trúc dữ liệu, mọi khai báo kiểu, v.v. đều cần có chú thích riêng của nó. Kiểu viết chú thích máy móc này thực ra lại khiến việc bảo trì mã khó hơn: giờ đây mỗi khi bạn thay đổi một thứ, lại có hai chỗ phải cập nhật. Vậy nên hãy giới hạn các chú thích không thuộc API ở việc bàn về lý do tại sao một việc được làm, mục đích và mục tiêu của nó. Mã nguồn vốn đã cho thấy việc đó được làm như thế nào, nên chú thích về điều này là thừa — và là một sự vi phạm nguyên tắc DRY.

Commenting source code gives you the perfect opportunity to document those elusive bits of a project that can't be documented anywhere else: engineering trade-offs, why decisions were made, what other alternatives were discarded, and so on.

Việc chú thích mã nguồn mang lại cho bạn cơ hội tuyệt vời để ghi lại những mẫu thông tin khó nắm bắt của dự án mà chẳng thể ghi ở đâu khác: những đánh đổi về mặt kỹ thuật, vì sao các quyết định được đưa ra, những phương án nào khác đã bị loại bỏ, v.v.

Summary — Tóm tắt

Know what you want to say.

Biết rõ bạn muốn nói gì.

Know your audience.

Hiểu khán giả của bạn.

Choose your moment.

Chọn đúng thời điểm.

Choose a style.

Chọn một phong cách.

Make it look good.

Làm cho nó trông đẹp mắt.

Involve your audience.

Lôi kéo khán giả tham gia.

Be a listener.

Hãy là người biết lắng nghe.

Get back to people.

Hãy hồi đáp người khác.

Keep code and documentation together.

Giữ mã nguồn và tài liệu đi cùng nhau.

Related Sections Include — Các mục liên quan

Topic 15, Estimating

Topic 15, Ước lượng

Topic 18, Power Editing

Topic 18, Soạn thảo cao thủ

Topic 45, The **Requirements** Pit

Topic 45, Cái hố Yêu cầu

Topic 49, Pragmatic Teams

Topic 49, Nhóm Thực dụng

Online Communication

Giao tiếp Trực tuyến

Everything we’ve said about communicating in writing applies equally to email, social media posts, blogs, and so on. Email in particular has evolved to the point where it is a mainstay of corporate communications; it’s used to discuss contracts, to settle disputes, and as evidence in court. But for some reason, people who would never send out a shabby paper document are happy to fling nasty-looking, incoherent emails around the world.

Mọi điều chúng tôi đã nói về giao tiếp bằng văn bản đều áp dụng y hệt cho email, bài đăng mạng xã hội, blog, v.v. Riêng email đã tiến hóa đến mức trở thành trụ cột của giao tiếp doanh nghiệp; nó được dùng để bàn hợp đồng, để dàn xếp tranh chấp, và làm bằng chứng tại tòa. Nhưng vì lý do nào đó, những người không bao giờ chịu gửi đi một tài liệu giấy lộn thối lại sẵn sàng quăng những email trông nhếch nhác, lủng củng đi khắp thế giới.

Our tips are simple:

Lời khuyên của chúng tôi rất đơn giản:

Proofread before you hit SEND .

Hãy đọc soát lại trước khi bạn nhấn GỬI.

Check your spelling and look for any accidental auto-correct mishaps.

Kiểm tra chính tả và để mắt tới mọi sự cố tự sửa lỗi (auto-correct) vô tình.

Keep the format simple and clear.

Giữ định dạng đơn giản và rõ ràng.

Keep quoting to a minimum. No one likes to receive back their own 100-line email with “I agree” tacked on. If you’re quoting other people’s email, be sure to attribute it, and quote it inline (rather than as an attachment). Same when quoting on social media platforms.

Trích dẫn ở mức tối thiểu thôi. Không ai thích nhận lại chính cái email 100 dòng của mình với mỗi dòng “Tôi đồng ý” gắn thêm vào. Nếu bạn trích email của người khác, hãy nhớ ghi rõ nguồn, và trích ngay trong nội dung (chứ đừng để dưới dạng tệp đính kèm). Trên các nền tảng mạng xã hội cũng vậy.

Don’t flame or act like a troll unless you want it to come back and haunt you later. If you wouldn’t say it to someone’s face, don’t say it online. Check your list of recipients before sending. It’s become a cliché to criticize the boss over departmental email without realizing that the boss is on the cc list. Better yet, don’t criticize the boss over email.

Đừng buông lời cay nghiệt hay hành xử như một kẻ chuyên gây sự (troll) trừ phi bạn muốn nó quay lại ám mình về sau. Nếu bạn không dám nói điều đó trước mặt một ai đó, thì đừng nói nó trên mạng. Hãy kiểm tra danh sách người nhận trước khi gửi. Cái cảnh chê bai sếp qua email của phòng ban mà không nhận ra sếp đang nằm trong danh sách cc đã trở thành chuyện sáo mòn rồi. Tốt hơn nữa, đừng chê bai sếp qua email.

As countless large corporations and politicians have discovered, email and social media posts are forever. Try to give the same attention and care to email as you would to any written memo or report.

Như vô số tập đoàn lớn và các chính trị gia đã phát hiện ra, email và bài đăng mạng xã hội tồn tại mãi mãi. Hãy cố gắng dành cho email sự quan tâm và cẩn trọng y như khi bạn soạn bất kỳ bản ghi nhớ hay báo cáo viết tay nào.

Challenges — Thử thách

There are several good books that contain sections on communications within teams, including *The Mythical Man-Month: Essays on Software Engineering* [Bro96] and *Peopleware: Productive Projects and Teams* [DL13]. Make it a point to try to read these over the next 18 months. In addition, *Dinosaur Brains: Dealing with All Those Impossible People at Work* [BR89] discusses the emotional baggage we all bring to the work environment.

Có vài cuốn sách hay chứa các phần bàn về giao tiếp trong nội bộ nhóm, bao gồm *The Mythical Man-Month: Essays on Software Engineering* [Bro96] và *Peopleware: Productive Projects and Teams* [DL13]. Hãy tự đặt mục tiêu cố đọc chúng trong 18 tháng tới. Ngoài ra, *Dinosaur Brains: Dealing with All Those Impossible People at Work* [BR89] bàn về những hành trạng cảm xúc mà tất cả chúng ta mang theo vào môi trường làm việc.

The next time you have to give a presentation, or write a memo advocating some position, try working through the advice in this section before you start. Explicitly identify the audience and what you need to communicate. If appropriate, talk to your audience afterward and see how accurate your assessment of their needs was.

Lần tới khi bạn phải thuyết trình, hay viết một bản ghi nhớ để cổ vũ cho một quan điểm nào đó, hãy thử làm theo lời khuyên trong mục này trước khi bắt đầu. Hãy xác định rõ ràng khán giả là ai và bạn cần truyền đạt điều gì. Nếu thích hợp, hãy trò chuyện với khán giả sau đó và xem đánh giá của bạn về nhu cầu của họ chính xác đến đâu.

Footnotes

Footnotes

[3] <http://wiki.c2.com/?ChangeYourOrganization>

[3] <http://wiki.c2.com/?ChangeYourOrganization>

[4] See, for example, a good meta-analysis at Trust and team performance: A metaanalysis of main effects, moderators, and covariates , <http://dx.doi.org/10.1037/apl0000110>

[4] Xem, chẳng hạn, một bài phân tích tổng hợp hay tại Trust and team performance: A metaanalysis of main effects, moderators, and covariates, <http://dx.doi.org/10.1037/apl0000110>

[5] See The police and neighborhood safety [WH82]

[5] Xem The police and neighborhood safety [WH82]

[6] See Contagious depression: Existence, specificity to depressed symptoms, and the role of reassurance seeking [Joi94]

[6] Xem Contagious depression: Existence, specificity to depressed symptoms, and the role of reassurance seeking [Joi94]

[7] While doing this, you may be comforted by the line attributed to Rear Admiral Dr. Grace Hopper: “It’s easier to ask forgiveness than it is to get permission.”

[7] Trong lúc làm việc này, bạn có thể được an ủi bởi câu nói được cho là của Chuẩn Đô đốc Tiến sĩ Grace Hopper: “Xin tha thứ thì dễ hơn là xin phép.”

[8] That was supposed to be a joke!

[8] Đó vốn dĩ là một câu nói đùa thôi!

[9] An expiring asset is something whose value diminishes over time. Examples include a warehouse full of bananas and a ticket to a ball game.

[9] Tài sản hết hạn (expiring asset) là thứ mà giá trị của nó giảm dần theo thời gian. Ví dụ gồm một nhà kho đầy chuối và một tấm vé xem trận bóng.

[10] We may be biased, but there’s a fine selection available at <https://pragprog.com> .

[10] Có thể chúng tôi hơi thiên vị, nhưng có một tuyển tập rất hay sẵn có tại <https://pragprog.com>.

[11] Never heard of any of these languages? Remember, knowledge is an expiring asset, and so is popular technology. The list of hot new and experimental languages was very different for the first edition, and is probably different again by the time you read this. All the more reason to keep learning.

[11] Chưa từng nghe nói đến bất kỳ ngôn ngữ nào trong số này ư? Hãy nhớ, tri thức là một tài sản hết hạn, và công nghệ thời thượng cũng vậy. Danh sách các ngôn ngữ mới nóng hổi và thử nghiệm ở lần xuất bản đầu tiên đã rất khác, và đến lúc bạn đọc những dòng này thì hẳn lại khác nữa. Càng thêm lý do để không ngừng học hỏi.

[12] The word annoy comes from the Old French enui , which also means “to bore.”

[12] Từ “annoy” (làm phiền) bắt nguồn từ tiếng Pháp cổ enui, vốn cũng có nghĩa là “làm cho chán.”

Copyright © 2020 Pearson Education, Inc.

Copyright © 2020 Pearson Education, Inc.

A Pragmatic Approach — Một Cách Tiếp Cận Thực Dụng

There are certain tips and tricks that apply at all levels of software development, processes that are virtually universal, and ideas that are almost axiomatic. However, these approaches are rarely documented as such; you'll mostly find them written down as odd sentences in discussions of design, project management, or coding. But for your convenience, we'll bring these ideas and processes together here.

Có những mẹo và thủ thuật áp dụng được ở mọi cấp độ của phát triển phần mềm, những quy trình gần như mang tính phổ quát, và những ý tưởng gần như là tiên đề. Thế nhưng những cách tiếp cận này hiếm khi được ghi lại một cách tường minh; phần lớn bạn sẽ chỉ bắt gặp chúng nằm rải rác như những câu nói lẻ trong các cuộc bàn luận về thiết kế, quản lý dự án hay viết mã. Nhưng để cho bạn tiện theo dõi, ở đây chúng tôi sẽ gom tất cả các ý tưởng và quy trình ấy lại với nhau.

The first and maybe most important **topic** gets to the heart of software development: Topic 8, The Essence of Good Design . Everything follows from this.

Mục đầu tiên, và có lẽ cũng quan trọng nhất, đi thẳng vào trái tim của phát triển phần mềm: Topic 8, Cốt Lõi của Thiết Kế Tốt. Mọi thứ đều bắt nguồn từ đây.

The next two sections, Topic 9, **DRY**—The Evils of **Duplication** and Topic 10, **Orthogonality** , are closely related. The first warns you not to duplicate knowledge throughout your systems, the second not to split any one piece of knowledge across multiple system components.

Hai mục tiếp theo, Topic 9, DRY—Cái Ác của Sự Trùng Lặp và Topic 10, Tính Trực Giao, có quan hệ mật thiết với nhau. Mục đầu cảnh báo bạn đừng nhân bản tri thức khắp các hệ thống của mình, mục sau nhắc bạn đừng xé lẻ một mẫu tri thức ra thành nhiều thành phần hệ thống khác nhau.

As the pace of change increases, it becomes harder and harder to keep our applications relevant. In Topic 11, **Reversibility** , we'll look at some techniques that help insulate your projects from their changing environment.

Khi nhịp độ thay đổi ngày một tăng, việc giữ cho các ứng dụng luôn còn phù hợp trở nên khó khăn hơn bao giờ hết. Trong Topic 11, Tính Khả Nghịch, chúng ta sẽ xem xét một vài kỹ thuật giúp cách ly các dự án của bạn khỏi môi trường luôn biến đổi xung quanh chúng.

The next two sections are also related. In Topic 12, **Tracer Bullets** , we talk about a style of development that allows you to gather **requirements**, test designs, and implement code at the same time. It's the only way to keep up with the pace of modern life.

Hai mục sau đó cũng có liên hệ với nhau. Trong Topic 12, Đạn Vạch Đường (tracer bullets), chúng tôi bàn về một phong cách phát triển cho phép bạn vừa thu thập yêu cầu, vừa thử nghiệm thiết kế, vừa hiện thực hóa mã — tất cả cùng một lúc. Đó là cách duy nhất để theo kịp nhịp sống hiện đại.

Topic 13, Prototypes and Post-it Notes shows you how to use prototyping to test architectures, algorithms, interfaces, and ideas. In the modern world, it's critical to test ideas and get feedback before you commit to them whole-heartedly.

Topic 13, Bản Mẫu và Giấy Nhớ Post-it cho bạn thấy cách dùng việc dựng bản mẫu (prototype) để kiểm thử kiến trúc, thuật toán, giao diện và ý tưởng. Trong thế giới hiện đại, việc thử nghiệm ý tưởng và thu nhận phản hồi trước khi bạn dốc lòng cam kết với chúng là cực kỳ quan trọng.

As computer science slowly matures, designers are producing increasingly higher-level languages. While the compiler that accepts “make it so” hasn't yet been invented, in Topic 14, Domain Languages we present some more modest suggestions that you can implement for yourself.

Khi ngành khoa học máy tính dần trưởng thành, các nhà thiết kế đang tạo ra những ngôn ngữ ngày càng ở mức cao hơn. Dù trình biên dịch nào chịu nghe lệnh “cứ làm như thế đi” vẫn chưa được phát minh, thì trong Topic 14, Ngôn Ngữ Miền (domain languages), chúng tôi sẽ đưa ra vài gợi ý khiêm tốn hơn mà chính bạn có thể tự hiện thực.

Finally, we all work in a world of limited time and resources. You can survive these scarcities better (and keep your bosses or clients happier) if you get good at working out how long things will take, which we cover in Topic 15, **Estimating**.

Cuối cùng, tất cả chúng ta đều làm việc trong một thế giới mà thời gian và nguồn lực đều có hạn. Bạn sẽ sống sót qua những sự khan hiếm ấy tốt hơn (và làm cho sếp hay khách hàng của mình vui hơn) nếu bạn giỏi việc tính toán xem mọi thứ sẽ tốn bao nhiêu thời gian — chủ đề mà chúng ta bàn trong Topic 15, Ước Lượng.

Keep these fundamental principles in mind during development, and you'll write code that's better, faster, and stronger. You can even make it look easy.

Hãy ghi nhớ những nguyên tắc nền tảng này trong suốt quá trình phát triển, và bạn sẽ viết ra mã tốt hơn, nhanh hơn, mạnh hơn. Thậm chí bạn còn có thể khiến nó trông thật dễ dàng.

Topic 8. The Essence of Good Design — Topic 8. Cốt Lõi của Thiết Kế Tốt

The world is full of gurus and pundits, all eager to pass on their hard-earned wisdom when it comes to How to Design Software. There are acronyms, lists (which seem to favor five entries), patterns, diagrams, videos, talks, and (the internet being the internet) probably a cool series on the **Law of Demeter** explained using interpretive dance.

Thế giới đầy rẫy các bậc thầy (guru) và các chuyên gia chém gió, ai cũng háo hức truyền lại sự khôn ngoan đúc kết bằng mồ hôi nước mắt của mình về Cách Thiết Kế Phần Mềm. Nào là các từ viết tắt, các danh sách (mà chẳng hiểu sao cứ thích có đúng năm mục), các mẫu thiết kế (pattern), sơ đồ, video, các buổi nói chuyện, và (internet thì vẫn cứ là internet) có lẽ là cả một loạt video độc đáo giải thích Luật Demeter (Law of Demeter) bằng... múa đương đại diễn giải.

And we, your gentle authors, are guilty of this too. But we'd like to make amends by explaining something that only became apparent to us fairly recently. First, the general statement:

Và chúng tôi, hai tác giả hiền lành của bạn, cũng có tội trong chuyện này. Nhưng chúng tôi muốn chuộc lỗi bằng cách giải thích một điều mà mãi gần đây mới trở nên rõ ràng với chúng tôi. Trước hết, hãy nói phát biểu tổng quát:

Tip 14: Good Design Is Easier to Change Than Bad Design

Tip 14: Thiết Kế Tốt Thì Dễ Thay Đổi Hơn Thiết Kế Tồi

A thing is well designed if it adapts to the people who use it. For code, that means it must adapt by changing. So we believe in the **ETC** principle: Easier to Change . ETC . That's it.

Một thứ được thiết kế tốt nếu nó thích nghi được với những người sử dụng nó. Với mã nguồn, điều đó nghĩa là nó phải thích nghi bằng cách thay đổi. Vậy nên chúng tôi tin vào nguyên tắc ETC: Dễ Thay Đổi Hơn (Easier to Change). ETC. Chỉ vậy thôi.

As far as we can tell, every design principle out there is a special case of ETC.

Theo như chúng tôi thấy, mọi nguyên tắc thiết kế ngoài kia đều là một trường hợp đặc biệt của ETC.

Why is **decoupling** good? Because by isolating concerns we make each easier to change. ETC.

Vì sao gỡ ghép nối (decoupling) lại tốt? Vì khi cô lập từng mối quan tâm, ta khiến mỗi mối quan tâm dễ thay đổi hơn. ETC.

Why is the single responsibility principle useful? Because a change in requirements is mirrored by a change in just one module. ETC.

Vì sao nguyên tắc trách nhiệm đơn lẻ (single responsibility principle) lại hữu ích? Vì một thay đổi trong yêu cầu sẽ được phản ánh bằng một thay đổi ở đúng một module. ETC.

Why is naming important? Because good names make code easier to read, and you have to read it to change it. ETC!

Vì sao việc đặt tên lại quan trọng? Vì tên hay làm cho mã dễ đọc hơn, mà bạn thì phải đọc được nó mới sửa được nó. ETC!

Etc Is A Value, Not A Rule — ETC Là Một Giá Trị, Không Phải Một Quy Tắc

Values are things that help you make decisions: should I do this, or that? When it comes to thinking about software, ETC is a guide, helping you choose between paths. Just like all your other values, it should be floating just behind your conscious thought, subtly nudging you in the right direction.

Giá trị là những thứ giúp bạn đưa ra quyết định: mình nên làm cái này, hay cái kia? Khi nghĩ về phần mềm, ETC là một kim chỉ nam, giúp bạn chọn giữa các con đường. Cũng như mọi giá trị khác của bạn, nó nên lửng lơ ngay phía sau dòng suy nghĩ có ý thức, lặng lẽ huých bạn đi đúng hướng.

But how do you make that happen? Our experience is that it requires some initial conscious reinforcement. You may need to spend a week or so deliberately asking yourself “did the thing I just did make the overall system easier or harder to change?” Do it when you save a file. Do it when you write a test. Do it when you fix a bug.

Nhưng làm sao để điều đó xảy ra? Kinh nghiệm của chúng tôi là nó cần được củng cố có ý thức một chút lúc ban đầu. Bạn có thể phải dành ra cỡ một tuần để cố ý tự hỏi “việc mình

vừa làm khiến tổng thể hệ thống dễ thay đổi hơn hay khó thay đổi hơn?”. Hãy hỏi khi bạn lưu một file. Hãy hỏi khi bạn viết một bài kiểm thử. Hãy hỏi khi bạn sửa một lỗi.

There’s an implicit premise in ETC. It assumes that a person can tell which of many paths will be easier to change in the future. Much of the time, common sense will be correct, and you can make an educated guess.

Trong ETC có một tiền đề ngầm. Nó giả định rằng một người có thể nhận ra trong vô số con đường, con đường nào sẽ dễ thay đổi hơn trong tương lai. Phần lớn thời gian, lẽ thường (common sense) sẽ đúng, và bạn có thể đưa ra một phỏng đoán có cơ sở.

Sometimes, though, you won’t have a clue. That’s OK. In those cases, we think you can do two things.

Tuy nhiên, đôi khi bạn sẽ chẳng có manh mối nào. Không sao cả. Trong những trường hợp đó, chúng tôi nghĩ bạn có thể làm hai việc.

First, given that you’re not sure what form change will take, you can always fall back on the ultimate “easy to change” path: try to make what you write replaceable. That way, whatever happens in the future, this chunk of code won’t be a roadblock. It seems extreme, but actually it’s what you should be doing all the time, anyway. It’s really just thinking about keeping code decoupled and cohesive.

Thứ nhất, vì bạn không chắc thay đổi sẽ mang hình hài gì, bạn luôn có thể quay về với con đường “dễ thay đổi” tối thượng: hãy cố làm cho thứ bạn viết ra có thể thay thế được. Như vậy, dù tương lai có ra sao, mẫu mã này sẽ không trở thành một vật cản đường. Nghe có vẻ cực đoan, nhưng thật ra đó là điều bạn nên làm thường xuyên rồi. Nó thực chất chỉ là tư duy giữ cho mã luôn được gỡ ghép nối (decoupled) và gắn kết (cohesive).

Second, treat this as a way to develop instincts. Note the situation in your engineering day book: the choices you have, and some guesses about change. Leave a tag in the source. Then, later, when this code has to change, you’ll be able to look back and give yourself feedback. It might help the next time you reach a similar fork in the road.

Thứ hai, hãy xem đây như một cách để rèn luyện trực giác. Hãy ghi lại tình huống vào cuốn nhật ký kỹ thuật (engineering daybook) của bạn: bạn có những lựa chọn nào, và vài phỏng đoán về thay đổi sắp tới. Để lại một dấu (tag) trong mã nguồn. Rồi sau này, khi đoạn mã này buộc phải thay đổi, bạn sẽ có thể ngoái lại nhìn và tự cho mình phản hồi. Điều đó có thể có ích vào lần sau khi bạn đến một ngã rẽ tương tự.

The rest of the sections in this chapter have specific ideas on design, but all are motivated by this one principle.

Các mục còn lại trong chương này đưa ra những ý tưởng cụ thể về thiết kế, nhưng tất cả đều được thúc đẩy bởi đúng một nguyên tắc này.

Related Sections Include — Các mục liên quan

Topic 9, DRY—The Evils of Duplication

Topic 9, DRY—Cái Ác của Sự Trùng Lặp

Topic 10, Orthogonality

Topic 10, Tính Trực Giao

Topic 11, Reversibility

Topic 11, Tính Khả Nghịch

Topic 14, Domain Languages

Topic 14, Ngôn Ngữ Miền

Topic 28, Decoupling

Topic 28, Gỡ Ghép Nối

Topic 30, Transforming Programming

Topic 30, Lập Trình Biến Đổi

Topic 31, **Inheritance** Tax

Topic 31, Thuế Kế Thừa

Challenges — Thử thách

Think about a design principle you use regularly. Is it intended to make things easy-to-change?

Hãy nghĩ về một nguyên tắc thiết kế mà bạn dùng thường xuyên. Nó có nhằm làm cho mọi thứ dễ thay đổi hơn không?

Also think about languages and programming paradigms (OO, FP, Reactive, and so on). Do any have either big positives or big negatives when it comes to helping you write ETC code? Do any have both? When coding, what can you do to eliminate the negatives and accentuate the positives? [13]

Cũng hãy nghĩ về các ngôn ngữ và mô hình lập trình (OO, FP, Reactive, v.v.). Có cái nào mang lại điểm cộng lớn hoặc điểm trừ lớn khi giúp bạn viết mã theo tinh thần ETC không? Có cái nào có cả hai không? Khi viết mã, bạn có thể làm gì để loại bỏ điểm trừ và khuếch đại điểm cộng? [13]

Many editors have support (either built-in or via extensions) to run commands when you save a file. Get your editor to popup an ETC? message every time you save [14] and use it as a cue to think about the code you just wrote. Is it easy to change?

Nhiều trình soạn thảo có hỗ trợ (sẵn có hoặc qua tiện ích mở rộng) để chạy lệnh khi bạn lưu một file. Hãy bắt trình soạn thảo của bạn bật lên thông báo ETC? mỗi lần bạn lưu [14] và dùng nó như một tín hiệu nhắc bạn ngẫm về đoạn mã mình vừa viết. Nó có dễ thay đổi không?

Topic 9. DRY—The Evils of Duplication — Topic 9. DRY — Tai họa của sự trùng lặp

Giving a computer two contradictory pieces of knowledge was Captain James T. Kirk's preferred way of disabling a marauding artificial intelligence. Unfortunately, the same principle can be effective in bringing down your code.

Trao cho máy tính hai mẫu kiến thức mâu thuẫn nhau là chiêu ưa thích của Thuyền trưởng James T. Kirk để vô hiệu hóa một trí tuệ nhân tạo hung hăng. Đáng tiếc, chính nguyên tắc ấy cũng đủ sức hạ gục đoạn mã của bạn.

As programmers, we collect, organize, maintain, and harness knowledge. We document knowledge in specifications, we make it come alive in running code, and we use it to provide the checks needed during testing.

Là lập trình viên, chúng ta thu thập, tổ chức, duy trì và khai thác kiến thức. Chúng ta ghi lại kiến thức trong các đặc tả, làm nó sống dậy trong mã đang chạy, và dùng nó để cung cấp các kiểm tra cần thiết trong quá trình kiểm thử (testing).

Unfortunately, knowledge isn't stable. It changes—often rapidly. Your understanding of a requirement may change following a meeting with the **client**. The government changes a regulation and some business logic gets outdated. Tests may show that the chosen algorithm won't work. All this instability means that we spend a large part of our time in maintenance mode, reorganizing and reexpressing the knowledge in our systems.

Đáng tiếc, kiến thức không hề đứng yên. Nó thay đổi — thường là rất nhanh. Cách hiểu của bạn về một yêu cầu có thể đổi khác sau một cuộc họp với khách hàng. Chính phủ thay đổi một quy định và thế là vài đoạn logic nghiệp vụ trở nên lỗi thời. Kiểm thử có thể cho thấy thuật toán đã chọn không chạy được. Tất cả sự bất ổn ấy đồng nghĩa với việc chúng ta dành phần lớn thời gian ở chế độ bảo trì, sắp xếp lại và diễn đạt lại kiến thức trong hệ thống của mình.

Most people assume that maintenance begins when an application is released, that maintenance means fixing bugs and enhancing features. We think these people are wrong. Programmers are constantly in maintenance mode. Our understanding changes day by day. New **requirements** arrive and existing requirements evolve as we're heads-down on the project. Perhaps the environment changes. Whatever the reason, maintenance is not a discrete activity, but a routine part of the entire development **process**.

Đa số mọi người mặc định rằng bảo trì bắt đầu khi ứng dụng đã phát hành, rằng bảo trì là sửa lỗi và thêm tính năng. Chúng tôi cho rằng những người đó nhầm. Lập trình viên thường trực ở chế độ bảo trì. Cách hiểu của chúng ta thay đổi từng ngày. Yêu cầu mới ập đến và yêu cầu cũ tiến hóa ngay cả khi ta đang cắm đầu vào dự án. Có khi môi trường thay đổi. Dù lý do là gì, bảo trì không phải một hoạt động tách rời, mà là một phần thường nhật của toàn bộ quá trình phát triển.

When we perform maintenance, we have to find and change the representations of things—those capsules of knowledge embedded in the application. The problem is that it's easy to duplicate knowledge in the specifications, processes, and programs that we develop, and when we do so, we invite a maintenance nightmare—one that starts well before the application ships.

Khi bảo trì, chúng ta phải tìm và thay đổi cách biểu diễn của mọi thứ — những viên nang kiến thức gắn chìm trong ứng dụng. Vấn đề là kiến thức rất dễ bị nhân bản trong các đặc tả, quy trình và chương trình mà ta xây dựng, và khi làm vậy, ta tự rước về một cơn ác mộng bảo trì — cơn ác mộng bắt đầu từ rất lâu trước cả khi ứng dụng được giao.

We feel that the only way to develop software reliably, and to make our developments easier to understand and maintain, is to follow what we call the **DRY** principle:

Chúng tôi tin rằng cách duy nhất để phát triển phần mềm một cách đáng tin cậy, và để sản phẩm dễ hiểu, dễ bảo trì hơn, là tuân theo cái mà chúng tôi gọi là nguyên tắc DRY:

Every piece of knowledge must have a single, unambiguous, authoritative representation within a system.

Mỗi mẫu kiến thức phải có một cách biểu diễn duy nhất, rõ ràng, có thẩm quyền bên trong hệ thống.

Why do we call it DRY?

Tại sao chúng tôi gọi nó là DRY?

Tip 15: DRY—Don't Repeat Yourself

Tip 15: DRY — Đừng Lặp Lại Chính Mình

The alternative is to have the same thing expressed in two or more places. If you change one, you have to remember to change the others, or, like the alien computers, your program will be brought to its knees by a contradiction. It isn't a question of whether you'll remember: it's a question of when you'll forget.

Phương án thay thế là để cùng một thứ được diễn đạt ở hai nơi trở lên. Nếu sửa một chỗ, bạn phải nhớ sửa các chỗ còn lại, nếu không — y như những cỗ máy ngoài hành tinh kia — chương trình của bạn sẽ quy ngã vì một mâu thuẫn. Vấn đề không phải là bạn có nhớ hay không: vấn đề là bao giờ bạn sẽ quên.

You'll find the DRY principle popping up time and time again throughout this book, often in contexts that have nothing to do with coding. We feel that it is one of the most important tools in the **Pragmatic Programmer's** tool box.

Bạn sẽ thấy nguyên tắc DRY xuất hiện đi xuất hiện lại xuyên suốt cuốn sách này, thường là trong những ngữ cảnh chẳng dính dáng gì đến viết mã. Chúng tôi cho rằng nó là một trong những công cụ quan trọng nhất trong hộp đồ nghề của Lập trình viên Thực dụng (Pragmatic Programmer).

In this section we'll outline the problems of **duplication** and

Trong mục này chúng tôi sẽ phác ra những vấn đề của sự trùng lặp (duplication) và suggest general strategies for dealing with it.

gợi ý các chiến lược chung để xử lý nó.

Dry Is More Than Code — DRY Không Chỉ Là Mã

Let's get something out of the way up-front. In the first edition of this book we did a poor job of explaining just what we meant by Don't Repeat Yourself . Many people took it to refer to code only: they thought that DRY means “don't copy-and-paste lines of source.”

Hãy làm rõ một chuyện ngay từ đầu. Ở ấn bản đầu của cuốn sách này, chúng tôi đã giải thích khá tệ về điều mình muốn nói với “Đừng Lặp Lại Chính Mình”. Nhiều người tưởng nó chỉ nói về mã: họ nghĩ DRY nghĩa là “đừng copy-and-paste các dòng mã nguồn”.

That is part of DRY, but it's a tiny and fairly trivial part.

Đó cũng là một phần của DRY, nhưng là một phần tí hon và khá tầm thường.

DRY is about the duplication of knowledge , of intent . It's about expressing the same thing in two different places, possibly in two totally different ways.

DRY nói về sự trùng lặp của kiến thức, của ý định. Nó nói về việc diễn đạt cùng một thứ ở hai nơi khác nhau, thậm chí theo hai cách hoàn toàn khác nhau.

Here's the acid test: when some single facet of the code has to change, do you find yourself making that change in multiple places, and in multiple different formats? Do you have to change code and documentation, or a database **schema** and a structure that holds it, or...? If so, your code isn't DRY.

Đây là phép thử quyết định: khi một khía cạnh đơn lẻ nào đó của mã phải thay đổi, bạn có thấy mình phải thực hiện thay đổi đó ở nhiều nơi, và theo nhiều định dạng khác nhau không? Bạn có phải sửa cả mã lẫn tài liệu, hay cả lược đồ cơ sở dữ liệu (database schema) lẫn cấu trúc chứa nó, hay...? Nếu có, mã của bạn không hề DRY.

So let's look at some typical examples of duplication.

Vậy hãy cùng xem vài ví dụ trùng lặp tiêu biểu.

Duplication In Code — Trùng Lặp Trong Mã

It may be trivial, but code duplication is oh, so common. Here's an example:

Có thể là chuyện vặt, nhưng trùng lặp mã thì, ôi chao, phổ biến vô cùng. Đây là một ví dụ:

```
def print_balance(account) printf "Debits: %10.2f\n" , account.debits printf "Credits: %10.2f\n" ,
account.credits if account.fees < 0 printf "Fees: %10.2f\n" , -account.fees else printf "Fees: %10.2f\n"
, account.fees
```

```
def print_balance(account) printf "Debits: %10.2f\n" , account.debits printf "Credits:
%10.2f\n" , account.credits if account.fees < 0 printf "Fees: %10.2f\n" , -account.fees else
printf "Fees: %10.2f\n" , account.fees
```

```
end printf " ——\n" if account.balance < 0 printf "Balance: %10.2f\n" , -account.balance else printf
"Balance: %10.2f\n" , account.balance end end
```

```
end printf " ——\n" if account.balance < 0 printf "Balance: %10.2f\n" , -account.balance
else printf "Balance: %10.2f\n" , account.balance end end
```

For now ignore the implication that we're committing the newbie mistake of storing currencies in floats. Instead see if you can spot duplications in this code. (We can see at least three things, but you might see more.)

Tạm bỏ qua chuyện đoạn này hàm ý chúng ta đang phạm cái lỗi gà mờ là lưu tiền tệ bằng kiểu số thực (float). Thay vào đó, thử xem bạn có phát hiện được những chỗ trùng lặp trong đoạn mã này không. (Chúng tôi thấy ít nhất ba thứ, nhưng bạn có thể thấy nhiều hơn.)

What did you find? Here's our list.

Bạn tìm ra gì rồi? Đây là danh sách của chúng tôi.

First, there's clearly a copy-and-paste duplication of handling the negative numbers. We can fix that by adding another function:

Thứ nhất, rõ ràng có một sự trùng lặp kiểu copy-and-paste trong việc xử lý số âm. Ta có thể khắc phục bằng cách thêm một hàm nữa:

```
def format_amount(value) result = sprintf( "%10.2f" , value.abs) if value < 0 result + "-" else result + "
" end end
```

```
def format_amount(value) result = sprintf( "%10.2f" , value.abs) if value < 0 result + "-" else
result + " " end end
```

```
def print_balance(account) printf "Debits: %10.2f\n" , account.debits printf "Credits: %10.2f\n" ,
account.credits printf "Fees: %s\n" , format_amount(account.fees) printf " ——\n" printf "Balance:
%s\n" , format_amount(account.balance) end
```

```
def print_balance(account) printf "Debits: %10.2f\n" , account.debits printf "Credits:
%10.2f\n" , account.credits printf "Fees: %s\n" , format_amount(account.fees) printf "
——\n" printf "Balance: %s\n" , format_amount(account.balance) end
```

Another duplication is the repetition of the field width in all the printf calls. We could fix this by introducing a constant and

Một chỗ trùng lặp khác là việc lặp lại độ rộng trường (field width) trong tất cả các lời gọi printf. Ta có thể khắc phục bằng cách đưa vào một hằng số và

passing it to each call, but why not just use the existing function?

truyền nó cho từng lời gọi printf, nhưng sao không dùng luôn cái hàm đã có sẵn?

```
def format_amount(value) result = sprintf( "%10.2f" , value.abs) if value < 0 result + "-" else result + "
```

```
result + " " end end
```

```
def format_amount(value) result = sprintf( "%10.2f" , value.abs) if value < 0 result + "-" else
```

```
result + " " end end
```

```
def print_balance(account) printf "Debits: %s\n" , format_amount(account.debits) printf "Credits: %s\n" , format_amount(account.credits) printf "Fees: %s\n" , format_amount(account.fees) printf "———\n" printf "Balance: %s\n" , format_amount(account.balance) end
```

```
def print_balance(account) printf "Debits: %s\n" , format_amount(account.debits) printf "Credits: %s\n" , format_amount(account.credits) printf "Fees: %s\n" , format_amount(account.fees) printf "———\n" printf "Balance: %s\n" , format_amount(account.balance) end
```

Anything more? Well, what if the client asks for an extra space between the labels and the numbers? We'd have to change five lines. Let's remove that duplication:

Còn gì nữa không? À, lẽ khách hàng yêu cầu thêm một khoảng trắng giữa nhãn và con số thì sao? Ta sẽ phải sửa năm dòng. Hãy gỡ bỏ chỗ trùng lặp đó:

```
def format_amount(value) result = sprintf( "%10.2f" , value.abs) if value < 0 result + "-" else result + "
```

```
result + " " end end
```

```
def format_amount(value) result = sprintf( "%10.2f" , value.abs) if value < 0 result + "-" else
```

```
result + " " end end
```

```
def print_line(label, value) printf "%-9s%s\n" , label, value end
```

```
def print_line(label, value) printf "%-9s%s\n" , label, value end
```

```
def report_line(label, amount) print_line(label + ":", format_amount(amount)) end
```

```
def report_line(label, amount) print_line(label + ":", format_amount(amount)) end
```

```
def print_balance(account) report_line( "Debits" , account.debits) report_line( "Credits" , account.credits) report_line( "Fees" , account.fees) print_line( ":", "———" ) report_line( "Balance" , account.balance) end
```

If we have to change the formatting of amounts, we change format_amount . If we want to change the label format, we change report_line .

```
def print_balance(account) report_line( "Debits" , account.debits) report_line( "Credits" , account.credits) report_line( "Fees" , account.fees) print_line( ":", "———" ) report_line( "Balance" , account.balance) end
```

Nếu phải thay đổi cách định dạng các khoản tiền, ta sửa format_amount. Nếu muốn thay đổi cách định dạng nhãn, ta sửa report_line.

There's still an implicit DRY violation: the number of hyphens in the separator line is related to the width of the amount field. But it isn't an exact match: it's currently one character shorter, so any trailing minus signs extend beyond the column. This is the **customer's** intent, and it's a different intent to the actual formatting of amounts.

Vẫn còn một vi phạm DRY ngầm: số dấu gạch nối trong dòng phân cách có liên hệ với độ rộng của trường số tiền. Nhưng nó không khớp chính xác: hiện nó ngắn hơn một ký tự, nên mọi dấu trừ ở cuối sẽ thò ra ngoài cột. Đây là chủ ý của khách hàng, và là một chủ ý khác với việc định dạng thực tế của các khoản tiền.

Not All Code Duplication Is Knowledge Duplication As part of your online wine ordering application you're capturing and validating your user's age, along with the quantity they're ordering. According to the site owner, they should both be numbers, and both greater than zero. So you code up the validations:

Không Phải Mọi Trùng Lặp Mã Đều Là Trùng Lặp Kiến Thức. Là một phần của ứng dụng đặt rượu vang trực tuyến, bạn thu thập và xác thực (validate) tuổi của người dùng, cùng với số lượng họ đặt mua. Theo chủ sở hữu trang web, cả hai đều phải là số, và đều phải lớn hơn không. Thế là bạn viết các đoạn xác thực:

```
def validate_age(value): validate_type(value, :integer) validate_min_integer(value, 0)
    def validate_age(value): validate_type(value, :integer) validate_min_integer(value, 0)
def validate_quantity(value): validate_type(value, :integer) validate_min_integer(value, 0)
    def validate_quantity(value): validate_type(value, :integer) validate_min_integer(value, 0)
```

During code review, the resident know-it-all bounces this code, claiming it's a DRY violation: both function bodies are the same.

Trong buổi review mã, vị “biết tuốt” ngồi đó gạt phắt đoạn mã này đi, một mực bảo nó vi phạm DRY: thân của cả hai hàm giống hệt nhau.

They are wrong. The code is the same, but the knowledge they represent is different. The two functions validate two separate things that just happen to have the same rules. That's a coincidence, not a duplication.

Họ sai rồi. Mã thì giống nhau, nhưng kiến thức mà chúng biểu diễn lại khác nhau. Hai hàm xác thực hai thứ riêng biệt, chỉ tình cờ có cùng quy tắc thôi. Đó là sự trùng hợp ngẫu nhiên, không phải sự trùng lặp.

Duplication In Documentation — Trùng Lặp Trong Tài Liệu

Somehow the myth was born that you should comment all your functions. Those who believe in this insanity then produce something such as this:

Chẳng hiểu sao lại nảy ra cái huyền thoại rằng bạn nên viết chú thích (comment) cho mọi hàm. Những ai tin vào sự điên rồ ấy rồi sẽ để ra thứ đại loại như thế này:

**Calculate the fees for this account. #
 # * Each returned check costs \$20 # *
 If the account is in overdraft for
 more than 3 days, # charge \$10 for
 each day # * If the average account
 balance is greater that \$2,000 #
 reduce the fees by 50% — Calculate
 the fees for this account. # # * Each
 returned check costs \$20 # * If the
 account is in overdraft for more
 than 3 days, # charge \$10 for each
 day # * If the average account
 balance is greater that \$2,000 #
 reduce the fees by 50%**

`def fees(a) f = 0 if a.returned_check_count > 0 f += 20 * a.returned_check_count end if a.overdraft_days > 3 f += 10 * a.overdraft_days end if a.average_balance > 2_000 f /= 2 end f end` The intent of this function is given twice: once in the comment and again in the code. The customer changes a fee, and we have to update both. Given time, we can pretty much guarantee the comment and the code will get out of step.

`def fees(a) f = 0 if a.returned_check_count > 0 f += 20 * a.returned_check_count end if`

a.overdraft_days > 3 f += 10*a.overdraft_days end if a.average_balance > 2_000 f /= 2 end f end Ý định của hàm này được nêu ra hai lần: một lần trong chú thích và một lần nữa trong mã. Khách hàng đổi một khoản phí, và ta phải cập nhật cả hai chỗ. Theo thời gian, gần như chắc chắn chú thích và mã sẽ lệch nhịp với nhau.

Ask yourself what the comment adds to the code. From our

Hãy tự hỏi chú thích kia bổ sung được gì cho mã. Theo

point of view, it simply compensates for some bad naming and layout. How about just this:

quan điểm của chúng tôi, nó chỉ đơn thuần bù đắp cho việc đặt tên và bố cục tẻ nhạt. Còn cứ thế này thì sao:

```
def calculate_account_fees(account) fees = 20 * account.returned_check_count fees += 10 *
account.overdraft_days if account.overdraft_days > 3 fees /= 2 if account.average_balance >
2_000 fees end
```

```
def calculate_account_fees(account) fees = 20 * account.returned_check_count fees += 10 *
account.overdraft_days if account.overdraft_days > 3 fees /= 2 if account.average_balance
> 2_000 fees end
```

The name says what it does, and if someone needs details, they're laid out in the source. That's DRY!

Cái tên đã nói lên nó làm gì, còn nếu ai cần chi tiết thì chúng đã bày sẵn ngay trong mã nguồn. Thế mới là DRY!

DRY Violations in Data Our data structures represent knowledge, and they can fall afoul of the DRY principle. Let's look at a class representing a line:

Vi Phạm DRY Trong Dữ Liệu. Các cấu trúc dữ liệu của chúng ta biểu diễn kiến thức, và chúng cũng có thể vướng vào nguyên tắc DRY. Hãy xem một lớp biểu diễn một đoạn thẳng:

```
class Line { Point start; Point end; double length; };
```

```
class Line { Point start; Point end; double length; };
```

At first sight, this class might appear reasonable. A line clearly has a start and end, and will always have a length (even if it's zero). But we have duplication. The length is defined by the start and end points: change one of the points and the length changes. It's better to make the length a calculated field:

Thoạt nhìn, lớp này trông có vẻ hợp lý. Một đoạn thẳng rõ ràng có điểm đầu và điểm cuối, và luôn có một độ dài (kể cả khi bằng không). Nhưng ở đây có trùng lặp. Độ dài đã được xác định bởi điểm đầu và điểm cuối: đổi một trong hai điểm thì độ dài đổi theo. Tốt hơn nên biến độ dài thành một trường được tính toán:

```
class Line { Point start; Point end; double length() { return start.distanceTo(end); } };
```

```
class Line { Point start; Point end; double length() { return start.distanceTo(end); } };
```

Later on in the development process, you may choose to violate the DRY principle for performance reasons. Frequently this occurs when you need to cache data to avoid repeating

Về sau trong quá trình phát triển, có thể bạn sẽ chọn vi phạm nguyên tắc DRY vì lý do hiệu năng. Điều này thường xảy ra khi bạn cần lưu đệm (cache) dữ liệu để tránh lặp lại các

expensive operations. The trick is to localize the impact. The violation is not exposed to the outside world: only the methods within the class have to worry about keeping things straight:

thao tác tốn kém. Bí quyết là khoanh vùng tác động. Vì phạm này không bị phơi ra cho thế giới bên ngoài: chỉ các phương thức bên trong lớp mới phải bận tâm việc giữ mọi thứ nhất quán:

```
class Line { private double length; private Point start; private Point end;
    class Line { private double length; private Point start; private Point end;
public Line(Point start, Point end) { this .start = start; this .end = end; calculateLength(); }
    public Line(Point start, Point end) { this .start = start; this .end = end; calculateLength(); }
// public void setStart(Point p) { this .start = p; calculateLength(); } void setEnd(Point p) { this .end =
p; calculateLength(); }
    // public void setStart(Point p) { this .start = p; calculateLength(); } void setEnd(Point p) {
this .end = p; calculateLength(); }
Point getStart() { return start; } Point getEnd() { return end; }
    Point getStart() { return start; } Point getEnd() { return end; }
double getLength() { return length; }
    double getLength() { return length; }
private void calculateLength() { this .length = start.distanceTo(end); } };
    private void calculateLength() { this .length = start.distanceTo(end); } };
```

This example also illustrates an important issue: whenever a module exposes a data structure, you're **coupling** all the code that uses that structure to the implementation of that module. Where possible, always use accessor functions to read and write the attributes of objects. It will make it easier to add functionality in the future.

Ví dụ này cũng minh họa một vấn đề quan trọng: bất cứ khi nào một module phơi ra một cấu trúc dữ liệu, bạn đang ghép nối (couple) toàn bộ mã dùng cấu trúc đó với phần cài đặt của module ấy. Khi có thể, hãy luôn dùng các hàm truy cập (accessor) để đọc và ghi các thuộc tính của đối tượng. Điều đó sẽ giúp việc thêm chức năng trong tương lai dễ dàng hơn.

This use of accessor functions ties in with Meyer's **Uniform Access principle**, described in Object-Oriented Software Construction [Mey97], which states that

Việc dùng hàm truy cập này ăn khớp với nguyên tắc Truy cập Đồng nhất (Uniform Access) của Meyer, được mô tả trong cuốn Object-Oriented Software Construction [Mey97], với nội dung rằng

All services offered by a module should be available through a uniform notation, which does not betray whether they are implemented through storage or through computation.

Mọi dịch vụ mà một module cung cấp đều phải được tiếp cận thông qua một ký pháp đồng nhất, không để lộ ra việc chúng được hiện thực bằng cách lưu trữ hay bằng cách tính toán.

Representational Duplication — Trùng Lặp Trong Cách Biểu Diễn

Your code interfaces to the outside world: other libraries via APIs, other services via remote calls, data in external sources, and so on. And pretty much each time you do, you introduce some kind of DRY violation: your code has to have knowledge that is also present in the external thing. It needs to know the API, or the schema, or the meaning of error codes, or whatever. The duplication here is

that two things (your code and the external entity) have to have knowledge of the representation of their interface. Change it at one end, and the other end breaks.

Mã của bạn giao tiếp với thế giới bên ngoài: với các thư viện khác qua API, với các dịch vụ khác qua lời gọi từ xa, với dữ liệu ở các nguồn ngoài, và vân vân. Và gần như mỗi lần làm vậy, bạn lại đưa vào một kiểu vi phạm DRY nào đó: mã của bạn buộc phải nắm kiến thức vốn cũng có mặt trong cái thứ bên ngoài kia. Nó cần biết API, hoặc lược đồ (schema), hoặc ý nghĩa của các mã lỗi, hay bất cứ thứ gì. Sự trùng lặp ở đây là cả hai phía (mã của bạn và thực thể bên ngoài) đều phải nắm kiến thức về cách biểu diễn giao diện chung của chúng. Đổi nó ở một đầu, đầu kia sẽ vỡ.

This duplication is inevitable, but can be mitigated. Here are some strategies.

Sự trùng lặp này là không thể tránh khỏi, nhưng có thể giảm nhẹ. Đây là vài chiến lược.

Duplication Across Internal APIs For internal APIs, look for tools that let you specify the API in some kind of neutral format. These tools will typically generate documentation, mock APIs, functional tests, and API clients, the latter in a number of different languages. Ideally the tool will store all your APIs in a central repository, allowing them to be shared across teams.

Trùng Lặp Giữa Các API Nội Bộ. Với các API nội bộ, hãy tìm những công cụ cho phép bạn đặc tả API theo một định dạng trung lập nào đó. Các công cụ này thường sẽ sinh ra tài liệu, API giả lập (mock), kiểm thử chức năng, và client cho API — thứ cuối này còn ở nhiều ngôn ngữ khác nhau. Lý tưởng nhất là công cụ sẽ lưu toàn bộ API của bạn trong một kho tập trung, cho phép chia sẻ giữa các nhóm.

Duplication Across External APIs Increasingly, you'll find that public APIs are documented formally using something like OpenAPI. [15] This allows you to import the API spec into your local API tools and integrate more reliably with the service.

Trùng Lặp Giữa Các API Bên Ngoài. Ngày càng nhiều, bạn sẽ thấy các API công khai được tài liệu hóa một cách chính quy bằng thứ gì đó như OpenAPI. [15] Điều này cho phép bạn nhập đặc tả API vào các công cụ API cục bộ của mình và tích hợp với dịch vụ một cách đáng tin cậy hơn.

If you can't find such a specification, consider creating one and

Nếu không tìm thấy một đặc tả như vậy, hãy cân nhắc tự tạo một cái và

publishing it. Not only will others find it useful; you may even get help maintaining it.

công bố nó. Không chỉ người khác sẽ thấy nó hữu ích; bạn thậm chí còn có thể được giúp đỡ trong việc duy trì nó.

Duplication with Data Sources Many data sources allow you to introspect on their data schema. This can be used to remove much of the duplication between them and your code. Rather than manually creating the code to contain this stored data, you can generate the containers directly from the schema. Many persistence frameworks will do this heavy lifting for you.

Trùng Lặp Với Các Nguồn Dữ Liệu. Nhiều nguồn dữ liệu cho phép bạn tự khám phá (introspect) lược đồ dữ liệu của chúng. Điều này có thể dùng để loại bỏ phần lớn sự trùng lặp giữa chúng và mã của bạn. Thay vì viết tay đoạn mã để chứa dữ liệu đã lưu này, bạn có thể sinh các đối tượng chứa trực tiếp từ lược đồ. Nhiều framework lưu trữ bền (persistence) sẽ gánh phần việc nặng nhọc đó hộ bạn.

There's another option, and one we often prefer. Rather than writing code that represents external data in a fixed structure (an instance of a struct or class, for example), just stick it into a key/value data structure (your language might call it a map, **hash**, dictionary, or even object).

Còn một lựa chọn khác, và là lựa chọn chúng tôi thường thích hơn. Thay vì viết mã biểu diễn dữ liệu bên ngoài trong một cấu trúc cố định (chẳng hạn một thực thể của struct hay class), hãy cứ nhét nó vào một cấu trúc dữ liệu khóa/giá trị (ngôn ngữ của bạn có thể gọi nó là map, hash, dictionary, hay thậm chí object).

On its own this is risky: you lose a lot of the security of knowing just what data you're working with. So we recommend adding a second layer to this solution: a simple table-driven validation suite that verifies that the map you've created contains at least the data you need, in the format you need it. Your API documentation tool might be able to generate this.

Đứng riêng một mình thì cách này khá rủi ro: bạn đánh mất phần lớn sự an tâm khi biết chính xác mình đang làm việc với dữ liệu gì. Vậy nên chúng tôi khuyên thêm vào giải pháp này một lớp thứ hai: một bộ xác thực đơn giản theo kiểu điều khiển bằng bảng (table-driven), để kiểm chứng rằng cái map bạn tạo ra chứa ít nhất những dữ liệu bạn cần, theo đúng định dạng bạn cần. Công cụ tài liệu hóa API của bạn có thể sinh được bộ này.

Interdeveloper Duplication — Trùng Lặp Giữa Các Lập Trình Viên

Perhaps the hardest type of duplication to detect and handle occurs between different developers on a project. Entire sets of functionality may be inadvertently duplicated, and that duplication could go undetected for years, leading to maintenance problems. We heard firsthand of a U.S. **state** whose governmental computer systems were surveyed for Y2K compliance. The audit turned up more than 10,000 programs that each contained a different version of Social Security

Có lẽ loại trùng lặp khó phát hiện và khó xử lý nhất xảy ra giữa các lập trình viên khác nhau trong một dự án. Cả những mảng chức năng lớn có thể bị nhân bản mà chẳng ai hay, và sự trùng lặp đó có thể âm thầm tồn tại nhiều năm, dẫn đến các vấn đề bảo trì. Chúng tôi từng nghe kể trực tiếp về một bang ở Mỹ, nơi các hệ thống máy tính của chính quyền được rà soát để kiểm tra khả năng tương thích Y2K. Cuộc kiểm toán phát hiện hơn 10.000 chương trình mà mỗi chương trình chứa một phiên bản khác nhau của đoạn mã xác thực Mã An sinh Xã hội

Number validation code.

(Social Security Number).

At a high level, deal with the problem by building a strong, tight-knit team with good communications.

Ở mức cao, hãy đối phó với vấn đề bằng cách xây dựng một nhóm vững mạnh, gắn kết, với khả năng giao tiếp tốt.

However, at the module level, the problem is more insidious. Commonly needed functionality or data that doesn't fall into an obvious area of responsibility can get implemented many times over.

Tuy nhiên, ở mức module, vấn đề lại tinh vi hơn. Những chức năng hay dữ liệu thường được cần đến nhưng không rơi gọn vào một vùng trách nhiệm rõ ràng nào có thể bị hiện thực đi hiện thực lại nhiều lần.

We feel that the best way to deal with this is to encourage active and frequent communication between developers.

Chúng tôi cho rằng cách tốt nhất để đối phó với chuyện này là khuyến khích giao tiếp tích cực và thường xuyên giữa các lập trình viên.

Maybe run a daily scrum standup meeting. Set up forums (such as Slack channels) to discuss common problems. This provides a nonintrusive way of communicating—even across multiple sites—while

retaining a permanent history of everything said.

Có thể tổ chức một buổi họp scrum standup hằng ngày. Lập các diễn đàn (chẳng hạn các kênh Slack) để bàn về những vấn đề chung. Cách này cung cấp một phương tiện giao tiếp không gây phiền nhiễu — kể cả khi làm việc qua nhiều địa điểm — đồng thời lưu lại một lịch sử vĩnh viễn của mọi điều đã trao đổi.

Appoint a team member as the project librarian, whose job is to facilitate the exchange of knowledge. Have a central place in the source tree where utility routines and scripts can be deposited. And make a point of reading other people's source code and documentation, either informally or during code reviews. You're not snooping—you're learning from them. And remember, the access is reciprocal—don't get twisted about other people poring (pawing?) through your code, either.

Hãy chỉ định một thành viên làm thủ thư của dự án, người có nhiệm vụ tạo điều kiện cho việc trao đổi kiến thức. Hãy dành một chỗ trung tâm trong cây mã nguồn để gửi vào đó các thủ tục và script tiện ích. Và hãy tập thói quen đọc mã nguồn cũng như tài liệu của người khác, dù là một cách không chính thức hay trong các buổi review mã. Bạn không phải đang rình mò — bạn đang học hỏi từ họ. Và nhớ rằng, sự tiếp cận này là tương hỗ — cũng đừng cau có khi người khác lục lọi (sờ nắn?) mã của mình.

Tip 16: Make It Easy to Reuse

Tip 16: Hãy Làm Cho Việc Tái Sử Dụng Trở Nên Dễ Dàng

What you're trying to do is foster an environment where it's easier to find and reuse existing stuff than to write it yourself. If it isn't easy, people won't do it. And if you fail to reuse, you risk

Cái bạn đang cố làm là nuôi dưỡng một môi trường nơi tìm và tái sử dụng đồ có sẵn dễ hơn là tự tay viết lại. Nếu nó không dễ, người ta sẽ không làm. Còn nếu bạn không tái sử dụng được, bạn có nguy cơ

duplicating knowledge.

nhân bản kiến thức.

Related Sections Include — Các mục liên quan

Topic 8, The Essence of Good Design

Topic 8, Tinh túy của thiết kế tốt

Topic 28, Decoupling

Topic 28, Gỡ ghép nối

Topic 32, Configuration

Topic 32, Cấu hình

Topic 38, Programming by Coincidence

Topic 38, Lập trình theo may rủi

Topic 40, Refactoring

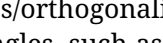
Topic 40, Tái cấu trúc

Topic 10. Orthogonality — Topic 10. Tính trực giao

Orthogonality is a critical concept if you want to produce systems that are easy to design, build, test, and extend. However, the concept of orthogonality is rarely taught directly. Often it is an implicit feature of various other methods and techniques you learn. This is a mistake. Once you learn to apply the principle of orthogonality directly, you'll notice an immediate improvement in the quality of systems you produce.

Tính trực giao (orthogonality) là một khái niệm then chốt nếu bạn muốn tạo ra những hệ thống dễ thiết kế, dễ xây dựng, dễ kiểm thử và dễ mở rộng. Vậy mà khái niệm này hiếm khi được dạy một cách trực tiếp. Thường thì nó chỉ là một đặc tính ngầm ẩn trong nhiều phương pháp và kỹ thuật khác mà bạn học. Đó là một sai lầm. Một khi bạn học được cách áp dụng nguyên tắc trực giao một cách trực tiếp, bạn sẽ thấy ngay chất lượng của những hệ thống mình tạo ra được cải thiện tức thì.

What Is Orthogonality? — Tính trực giao là gì?

“Orthogonality” is a term  borrowed from geometry. Two lines are orthogonal if they meet at right angles, such as the axes on a graph. In vector terms, the two lines are independent. As the number 1 on the diagram moves north, it doesn't change how far east or west it is. The number 2 moves east, but not north or south.

“Orthogonality” (tính trực giao) là một thuật ngữ  mượn từ hình học. Hai đường thẳng được gọi là trực giao nếu chúng gặp nhau tại một góc vuông, chẳng hạn như các trục trên một đồ thị. Theo ngôn ngữ vector, hai đường thẳng đó độc lập với nhau. Khi số 1 trên sơ đồ di chuyển về phía bắc, nó không làm thay đổi vị trí đông hay tây của mình. Số 2 di chuyển về phía đông, nhưng không lên bắc hay xuống nam.

In computing, the term has come to signify a kind of independence or **decoupling**. Two or more things are orthogonal if changes in one do not affect any of the others. In a well-designed system, the database code will be orthogonal to the user interface: you can change the interface without affecting the database, and swap databases without changing the interface.

Trong tin học, thuật ngữ này dần mang ý nghĩa một dạng độc lập hay tách rời. Hai hay nhiều thứ là trực giao nếu thay đổi ở cái này không ảnh hưởng đến bất kỳ cái nào khác. Trong một hệ thống được thiết kế tốt, phần mã database sẽ trực giao với giao diện người dùng: bạn có thể đổi giao diện mà không động đến database, và thay database mà không phải sửa giao diện.

Before we look at the benefits of orthogonal systems, let's first look at a system that isn't orthogonal.

Trước khi xem xét những lợi ích của các hệ thống trực giao, hãy nhìn vào một hệ thống không trực giao đã.

A Nonorthogonal System You're on a helicopter tour of the Grand Canyon when the pilot, who made the obvious mistake of eating fish for lunch, suddenly groans and faints. Fortunately, he left you hovering 100 feet above the ground.

Một hệ thống không trực giao Bạn đang trong chuyến bay trực thăng ngắm cảnh Grand Canyon thì viên phi công — người vừa phạm sai lầm hiển nhiên là ăn cá cho bữa trưa — đột nhiên rên lên một tiếng rồi ngất xỉu. May thay, anh ta để bạn lơ lửng ở độ cao 30 mét trên mặt đất.

As luck would have it, you had read a Wikipedia page about helicopters the previous night. You know that helicopters have four basic controls. The cyclic is the stick you hold in your right hand. Move

it, and the helicopter moves in the corresponding direction. Your left hand holds the collective pitch lever . Pull up on this and you increase the pitch on all the blades, generating lift. At the end of the pitch lever is the throttle . Finally you have two foot pedals , which vary the amount of tail rotor thrust and so help turn the helicopter.

Cũng may là tối hôm trước bạn vừa đọc một trang Wikipedia về trực thăng. Bạn biết rằng trực thăng có bốn bộ điều khiển cơ bản. Cần điều khiển chu kỳ (cyclic) là cái cần bạn cầm trong tay phải. Đẩy nó, trực thăng sẽ chuyển động theo hướng tương ứng. Tay trái bạn cầm cần điều khiển bước tổng (collective pitch lever). Kéo cần này lên thì bạn tăng góc nghiêng của tất cả các cánh quạt, tạo ra lực nâng. Ở đầu cần bước này là tay ga (throttle). Cuối cùng bạn có hai bàn đạp chân (foot pedals), dùng để thay đổi lực đẩy của cánh quạt đuôi và nhờ đó giúp xoay trực thăng.

“Easy!,” you think. “Gently lower the collective pitch lever and you’ll descend gracefully to the ground, a hero.” However, when you try it, you discover that life isn’t that simple. The helicopter’s nose drops, and you start to spiral down to the left. Suddenly you discover that you’re flying a system where every control input has secondary effects. Lower the left-hand lever and you need to add compensating backward movement to the right-hand stick and push the right pedal. But then each of these changes affects all of the other controls again. Suddenly you’re juggling an unbelievably complex system, where every change impacts all the other inputs. Your workload is phenomenal: your hands and feet are constantly moving, trying to balance all the interacting forces.

“Dễ thôi!,” bạn nghĩ. “Nhẹ nhàng hạ cần bước tổng xuống là bạn sẽ đáp xuống đất một cách duyên dáng, thành người hùng.” Tuy nhiên, khi thử làm, bạn phát hiện ra rằng đời chẳng đơn giản thế. Mũi trực thăng chúi xuống, và bạn bắt đầu xoáy tròn rơi về bên trái. Đột nhiên bạn nhận ra mình đang lái một hệ thống mà mỗi tác động điều khiển đều kéo theo những hệ quả phụ. Hạ cần bên trái xuống thì bạn phải bù lại bằng cách kéo cần bên phải về sau và đạp bàn đạp phải. Nhưng rồi mỗi thay đổi đó lại tác động đến tất cả các bộ điều khiển còn lại một lần nữa. Bỗng chốc bạn đang tung hứng một hệ thống phức tạp đến khó tin, nơi mỗi thay đổi đều ảnh hưởng tới mọi tác động khác. Khối lượng công việc của bạn lớn khủng khiếp: chân tay liên tục cử động, cố cân bằng tất cả các lực đang tương tác lẫn nhau.

Helicopter controls are decidedly not orthogonal.

Các bộ điều khiển của trực thăng dứt khoát là không trực giao.

Benefits Of Orthogonality — Lợi ích của tính trực giao

As the helicopter example illustrates, nonorthogonal systems are inherently more complex to change and control. When components of any system are highly interdependent, there is no such thing as a local fix.

Như ví dụ trực thăng cho thấy, các hệ thống không trực giao về bản chất khó thay đổi và khó kiểm soát hơn. Khi các thành phần của một hệ thống bất kỳ phụ thuộc lẫn nhau quá chặt, thì chẳng có thứ gì gọi là sửa cục bộ cả.

Tip 17: Eliminate Effects Between Unrelated Things

Tip 17: Hãy Loại Bỏ Ảnh Hưởng Giữa Những Thứ Không Liên Quan

We want to design components that are self-contained: independent, and with a single, well-defined purpose (what Yourdon and Constantine call **cohesion** in Structured Design: Fundamentals of a Discipline of Computer Program and Systems Design [YC79]). When components are isolated from one another, you know that you can change one without having to worry about the rest. As long as

you don't change that component's external interfaces, you can be confident that you won't cause problems that ripple through the entire system.

Chúng tôi muốn thiết kế những thành phần tự chứa: độc lập và có một mục đích duy nhất, được định nghĩa rõ ràng (cái mà Yourdon và Constantine gọi là tính kết dính (cohesion) trong cuốn *Structured Design: Fundamentals of a Discipline of Computer Program and Systems Design* [YC79]). Khi các thành phần được tách biệt với nhau, bạn biết rằng mình có thể thay đổi một cái mà không phải bận tâm tới phần còn lại. Miễn là bạn không thay đổi giao diện đối ngoại của thành phần đó, bạn có thể yên tâm rằng mình sẽ không gây ra những vấn đề lan tỏa khắp cả hệ thống.

You get two major benefits if you write orthogonal systems: increased productivity and reduced risk.

Bạn nhận được hai lợi ích lớn nếu viết các hệ thống trực giao: tăng năng suất và giảm rủi ro.

Gain Productivity

Tăng năng suất

Changes are localized, so development time and testing time are reduced. It is easier to write relatively small, self-contained components than a single large block of code. Simple components can be designed, coded, tested, and then forgotten—there is no need to keep changing existing code as you add new code.

Các thay đổi được khoanh vùng cục bộ, nên thời gian phát triển và thời gian kiểm thử giảm đi. Viết những thành phần tương đối nhỏ, tự chứa thì dễ hơn viết một khối mã lớn duy nhất. Các thành phần đơn giản có thể được thiết kế, viết mã, kiểm thử, rồi quên đi — không cần phải liên tục sửa mã cũ mỗi khi bạn thêm mã mới.

An orthogonal approach also promotes reuse. If components have specific, well-defined responsibilities, they can be combined with new components in ways that were not envisioned by their original

Cách tiếp cận trực giao cũng thúc đẩy việc tái sử dụng. Nếu các thành phần có trách nhiệm cụ thể, được định nghĩa rõ ràng, chúng có thể được kết hợp với các thành phần mới theo những cách mà những người triển khai ban đầu

implementors. The more loosely coupled your systems, the easier they are to reconfigure and reengineer.

chưa hề hình dung ra. Hệ thống của bạn càng ghép nối lỏng lẻo thì càng dễ tái cấu hình và tái thiết kế.

There is a fairly subtle gain in productivity when you combine orthogonal components. Assume that one component does distinct things and another does things. If they are orthogonal and you combine them, the result does things. However, if the two components are not orthogonal, there will be overlap, and the result will do less. You get more functionality per unit effort by combining orthogonal components.

Có một lợi ích về năng suất khá tinh tế khi bạn kết hợp các thành phần trực giao. Giả sử một thành phần làm được một số việc khác biệt, và một thành phần khác làm được một số việc nữa. Nếu chúng trực giao và bạn kết hợp chúng, kết quả làm được tích các việc. Tuy nhiên, nếu hai thành phần không trực giao, sẽ có sự chồng lấn, và kết quả làm được ít hơn. Bạn thu được nhiều chức năng hơn trên mỗi đơn vị công sức khi kết hợp các thành phần trực giao.

Reduce Risk An orthogonal approach reduces the risks inherent in any development.

Giảm rủi ro Cách tiếp cận trực giao làm giảm những rủi ro vốn có trong bất kỳ quá trình phát triển nào.

Diseased sections of code are isolated. If a module is sick, it is less likely to spread the symptoms around the rest of the system. It is also easier to slice it out and transplant in something new and healthy.

Những đoạn mã bệnh tật được cách ly. Nếu một module bị ốm, nó ít có khả năng lây lan triệu chứng ra phần còn lại của hệ thống. Cũng dễ hơn để cắt bỏ nó đi và cấy ghép vào một thứ gì đó mới và khỏe mạnh.

The resulting system is less fragile. Make small changes and fixes to a particular area, and any problems you generate will be restricted to that area.

Hệ thống thu được sẽ ít mong manh hơn. Hãy thực hiện những thay đổi và sửa lỗi nhỏ ở một khu vực cụ thể, và mọi vấn đề bạn gây ra sẽ chỉ giới hạn trong khu vực đó.

An orthogonal system will probably be better tested, because it will be easier to design and run tests on its components.

Một hệ thống trực giao có lẽ sẽ được kiểm thử tốt hơn, vì việc thiết kế và chạy các bài kiểm thử trên các thành phần của nó sẽ dễ dàng hơn.

You will not be as tightly tied to a particular vendor, product, or platform, because the interfaces to these third-party components will be isolated to smaller parts of the overall development.

Bạn sẽ không bị trói chặt vào một nhà cung cấp, sản phẩm hay nền tảng cụ thể nào đó, bởi vì các giao diện tới những thành phần của bên thứ ba này sẽ được cách ly vào những phần nhỏ hơn của toàn bộ quá trình phát triển.

Let's look at some of the ways you can apply the principle of orthogonality to your work.

Hãy cùng xem một vài cách bạn có thể áp dụng nguyên tắc trực giao vào công việc của mình.

Design — Thiết kế

Most developers are familiar with the need to design orthogonal systems, although they may use words such as modular ,

Hầu hết các nhà phát triển đều quen thuộc với nhu cầu thiết kế các hệ thống trực giao, dù họ có thể dùng những từ như mô-đun hóa (modular),

component-based , and layered to describe the **process**. Systems should be composed of a set of cooperating modules, each of which implements functionality independent of the others. Sometimes these components are organized into layers, each providing a level of abstraction. This layered approach is a powerful way to design orthogonal systems. Because each layer uses only the abstractions provided by the layers below it, you have great flexibility in changing underlying implementations without affecting code. Layering also reduces the risk of runaway dependencies between modules. You'll often see layering expressed in diagrams:

dựa trên thành phần (component-based), và phân lớp (layered) để mô tả quá trình đó. Hệ thống nên được cấu thành từ một tập hợp các module hợp tác với nhau, mỗi module hiện thực một phần chức năng độc lập với các module khác. Đôi khi các thành phần này được tổ chức thành các lớp, mỗi lớp cung cấp một mức trừu tượng. Cách tiếp cận phân lớp này là một phương pháp mạnh mẽ để thiết kế các hệ thống trực giao. Vì mỗi lớp chỉ sử dụng những trừu tượng do các lớp bên dưới nó cung cấp, bạn có sự linh hoạt lớn trong việc thay

đổi các hiện thực bên dưới mà không ảnh hưởng tới mã. Phân lớp cũng làm giảm rủi ro về những phụ thuộc lan tràn mất kiểm soát giữa các module. Bạn sẽ thường thấy việc phân lớp được biểu diễn trong các sơ đồ:

images/layer-diagram.png

images/layer-diagram.png

There is an easy test for orthogonal design. Once you have your components mapped out, ask yourself: If I dramatically change the **requirements** behind a particular function, how many modules are affected? In an orthogonal system, the answer should be “one.” [16] Moving a button on a GUI panel should not require a change in the database **schema**. Adding context-sensitive help should not change the billing subsystem.

Có một phép thử đơn giản cho thiết kế trực giao. Một khi đã vạch ra các thành phần của mình, hãy tự hỏi: Nếu tôi thay đổi mạnh mẽ các yêu cầu đằng sau một chức năng cụ thể nào đó, sẽ có bao nhiêu module bị ảnh hưởng? Trong một hệ thống trực giao, câu trả lời nên là “một.” [16] Việc di chuyển một nút bấm trên một bảng giao diện đồ họa không nên đòi hỏi thay đổi lược đồ database. Việc thêm trợ giúp theo ngữ cảnh không nên làm thay đổi hệ thống con tính cước.

Let’s consider a complex system for monitoring and controlling a heating plant. The original requirement called for a graphical user interface, but the requirements were changed to add a mobile interface that lets engineers monitor key values. In an orthogonally designed system, you would need to change only those modules associated with the user interface to handle this: the underlying logic of controlling the plant would remain unchanged. In fact, if you structure your system carefully, you should be able to support both interfaces with the same underlying code base.

Hãy xét một hệ thống phức tạp để giám sát và điều khiển một nhà máy nhiệt điện. Yêu cầu ban đầu đòi hỏi một giao diện người dùng đồ họa, nhưng sau đó yêu cầu được thay đổi để thêm một giao diện di động cho phép các kỹ sư giám sát các giá trị quan trọng. Trong một hệ thống được thiết kế trực giao, bạn sẽ chỉ cần thay đổi những module liên quan đến giao diện người dùng để xử lý việc này: phần logic nền tảng điều khiển nhà máy sẽ vẫn không đổi. Thực ra, nếu bạn cấu trúc hệ thống của mình một cách cẩn thận, bạn hẳn sẽ có thể hỗ trợ cả hai giao diện bằng cùng một cơ sở mã nền tảng.

Also ask yourself how decoupled your design is from changes in the real world. Are you using a telephone number as a **customer** identifier? What happens when the phone company reassigns area codes? Postal codes, Social Security Numbers or government IDs, email addresses, and domains are all external identifiers that you have no control over, and could change at any time for any reason. Don’t rely on the properties of things you can’t control.

Cũng hãy tự hỏi xem thiết kế của bạn tách rời đến mức nào khỏi những thay đổi trong thế giới thực. Bạn có đang dùng số điện thoại làm định danh khách hàng không? Chuyện gì sẽ xảy ra khi công ty điện thoại phân bổ lại các mã vùng? Mã bưu chính, số An sinh Xã hội hay số định danh do chính phủ cấp, địa chỉ email và tên miền đều là những định danh đối ngoại mà bạn không hề kiểm soát được, và chúng có thể thay đổi bất cứ lúc nào vì bất cứ lý do gì. Đừng dựa vào các thuộc tính của những thứ bạn không thể kiểm soát.

Toolkits And Libraries — Bộ công cụ và thư viện

Be careful to preserve the orthogonality of your system as you introduce third-party toolkits and libraries. Choose your technologies wisely.

Hãy cẩn thận gìn giữ tính trực giao của hệ thống khi bạn đưa vào các bộ công cụ và thư viện của bên thứ ba. Hãy chọn công nghệ một cách khôn ngoan.

When you bring in a toolkit (or even a library from other members of your team), ask yourself whether it imposes changes on your code that shouldn't be there. If an object persistence scheme is transparent, then it's orthogonal. If it requires you to create or access objects in a special way, then it's not. Keeping such details isolated from your code has the added benefit of making it easier to change vendors in the future.

Khi bạn mang vào một bộ công cụ (hoặc thậm chí một thư viện từ các thành viên khác trong nhóm), hãy tự hỏi liệu nó có áp đặt lên mã của bạn những thay đổi vốn không nên có ở đó không. Nếu một cơ chế lưu trữ bền vững (persistence) cho đối tượng là trong suốt, thì nó trực giao. Nếu nó đòi hỏi bạn phải tạo hoặc truy cập các đối tượng theo một cách đặc biệt, thì nó không trực giao. Giữ những chi tiết như vậy được cách ly khỏi mã của bạn còn có thêm lợi ích là giúp việc đổi nhà cung cấp trong tương lai dễ dàng hơn.

The Enterprise Java Beans (EJB) system is an interesting example of orthogonality. In most transaction-oriented systems, the application code has to delineate the start and end of each transaction. With EJB, this information is expressed declaratively as annotations, outside the methods that do the work. The same application code can run in different EJB transaction environments with no change.

Hệ thống Enterprise Java Beans (EJB) là một ví dụ thú vị về tính trực giao. Trong hầu hết các hệ thống hướng giao dịch (transaction), mã ứng dụng phải tự khoanh vùng điểm bắt đầu và kết thúc của mỗi giao dịch. Với EJB, thông tin này được biểu đạt một cách khai báo dưới dạng các chú thích (annotation), nằm bên ngoài các phương thức làm công việc thực sự. Cùng một mã ứng dụng có thể chạy trong những môi trường giao dịch EJB khác nhau mà không cần thay đổi.

In a way, EJB is an example of the Decorator Pattern: adding functionality to things without changing them. This style of programming can be used in just about every programming language, and doesn't necessarily require a framework or library. It just takes a little discipline when programming.

Theo một cách nào đó, EJB là một ví dụ của Mẫu Trang trí (Decorator Pattern): thêm chức năng vào các thứ mà không thay đổi chúng. Phong cách lập trình này có thể được dùng trong gần như mọi ngôn ngữ lập trình, và không nhất thiết đòi hỏi một framework hay thư viện. Nó chỉ cần một chút kỷ luật khi lập trình.

Coding — Viết mã

Every time you write code you run the risk of reducing the orthogonality of your application. Unless you constantly monitor not just what you are doing but also the larger **context** of the application, you might unintentionally duplicate functionality in some other module, or express existing knowledge twice.

Mỗi lần viết mã, bạn đều mang rủi ro làm giảm tính trực giao của ứng dụng. Trừ phi bạn liên tục để mắt không chỉ tới việc mình đang làm mà cả tới bối cảnh lớn hơn của ứng dụng, bạn có thể vô tình nhân bản chức năng trong một module nào đó khác, hoặc biểu đạt cùng một tri thức hiện có tới hai lần.

There are several techniques you can use to maintain orthogonality:

Có một số kỹ thuật bạn có thể dùng để duy trì tính trực giao:

Keep your code decoupled Write **shy code**—modules that don't reveal anything unnecessary to other modules and that don't rely on other modules' implementations. Try the **Law of Demeter**, which we discuss in **Topic 28, Decoupling**. If you need to change an object's **state**, get the object to do it for you. This way your code remains isolated from the other code's implementation and increases the chances

Giữ cho mã của bạn được gỡ ghép nối Hãy viết mã nhút nhát — những module không tiết lộ bất cứ điều gì không cần thiết cho các module khác và không dựa vào hiện thực của các module khác. Hãy thử Luật Demeter (Law of Demeter), thứ chúng tôi bàn tới trong Topic 28, Gỡ ghép nối. Nếu bạn cần thay đổi trạng thái của một đối tượng, hãy để chính đối tượng đó làm việc ấy giúp bạn. Bằng cách này, mã của bạn vẫn được cách ly khỏi hiện thực của mã khác và tăng cơ hội

that you'll remain orthogonal.

rằng bạn vẫn giữ được tính trực giao.

Avoid global data Every time your code references global data, it ties itself into the other components that share that data. Even globals that you intend only to read can lead to trouble (for example, if you suddenly need to change your code to be multithreaded). In general, your code is easier to understand and maintain if you explicitly pass any required context into your modules. In object-oriented applications, context is often passed as parameters to objects' constructors. In other code, you can create structures containing the context and pass around references to them.

Tránh dữ liệu toàn cục Mỗi lần mã của bạn tham chiếu tới dữ liệu toàn cục, nó tự buộc mình vào những thành phần khác cùng chia sẻ dữ liệu đó. Ngay cả những biến toàn cục mà bạn chỉ định đọc cũng có thể dẫn tới rắc rối (ví dụ, nếu bỗng dưng bạn cần sửa mã để chạy đa luồng). Nhìn chung, mã của bạn sẽ dễ hiểu và dễ bảo trì hơn nếu bạn truyền tường minh mọi bối cảnh cần thiết vào các module của mình. Trong các ứng dụng hướng đối tượng, bối cảnh thường được truyền dưới dạng tham số cho các hàm khởi tạo của đối tượng. Trong mã khác, bạn có thể tạo những cấu trúc chứa bối cảnh và truyền đi các tham chiếu tới chúng.

The Singleton pattern in Design Patterns: Elements of Reusable Object-Oriented Software [GHJV95] is a way of ensuring that there is only one instance of an object of a particular class. Many people use these singleton objects as a kind of global variable (particularly in languages, such as Java, that otherwise do not support the concept of globals). Be careful with singletons—they can also lead to unnecessary linkage.

Mẫu Singleton trong cuốn Design Patterns: Elements of Reusable Object-Oriented Software [GHJV95] là một cách để đảm bảo rằng chỉ có duy nhất một thể hiện của một đối tượng thuộc một lớp cụ thể. Nhiều người dùng các đối tượng singleton này như một dạng biến toàn cục (đặc biệt trong những ngôn ngữ, chẳng hạn như Java, vốn không hỗ trợ khái niệm biến toàn cục). Hãy cẩn thận với singleton — chúng cũng có thể dẫn tới những liên kết không cần thiết.

Avoid similar functions Often you'll come across a set of functions that all look similar—maybe they share common code at the start and end, but each has a different central algorithm. Duplicate code is a symptom of structural problems. Have a look at the Strategy pattern in Design Patterns for a better implementation.

Tránh các hàm giống nhau Bạn sẽ thường bắt gặp một tập các hàm trông na ná nhau — có thể chúng dùng chung đoạn mã ở đầu và cuối, nhưng mỗi hàm lại có một thuật toán cốt lõi khác nhau. Mã trùng lặp là một triệu chứng của các vấn đề về cấu trúc. Hãy xem qua mẫu Strategy trong Design Patterns để có một cách hiện thực tốt hơn.

Get into the habit of being constantly critical of your code. Look

Hãy tập thói quen luôn phê phán mã của chính mình. Hãy tìm

for any opportunities to reorganize it to improve its structure and orthogonality. This process is called **refactoring**, and it's so important that we've dedicated a section to it (see Topic 40, Refactoring).

mọi cơ hội để tổ chức lại nó nhằm cải thiện cấu trúc và tính trực giao. Quá trình này gọi là tái cấu trúc (refactoring), và nó quan trọng đến mức chúng tôi đã dành hẳn một mục cho nó (xem Topic 40, Tái cấu trúc).

Testing — Kiểm thử

An orthogonally designed and implemented system is easier to test. Because the interactions between the system's components are formalized and limited, more of the system testing can be performed at the individual module level. This is good news, because module level (or unit) testing is considerably easier to specify and perform than integration testing. In fact, we suggest that these tests be performed automatically as part of the regular build process (see Topic 41, **Test to Code**).

Một hệ thống được thiết kế và hiện thực theo lối trực giao thì dễ kiểm thử hơn. Vì các tương tác giữa các thành phần của hệ thống được hình thức hóa và giới hạn lại, nên phần lớn việc kiểm thử hệ thống có thể được thực hiện ngay ở cấp độ từng module. Đây là tin tốt, bởi vì kiểm thử ở cấp module (hay kiểm thử đơn vị) dễ đặc tả và thực hiện hơn nhiều so với kiểm thử tích hợp. Thực ra, chúng tôi đề xuất rằng những bài kiểm thử này nên được chạy tự động như một phần của quy trình build thông thường (xem Topic 41, Kiểm thử cho mã).

Writing unit tests is itself an interesting test of orthogonality. What does it take to get a unit test to build and run? Do you have to import a large percentage of the rest of the system's code? If so, you've found a module that is not well decoupled from the rest of the system.

Việc viết các bài kiểm thử đơn vị tự nó đã là một phép thử thú vị về tính trực giao. Cần những gì để một bài kiểm thử đơn vị build và chạy được? Bạn có phải import một tỷ lệ lớn phần còn lại trong mã của hệ thống không? Nếu có, bạn vừa tìm ra một module chưa được gỡ ghép nối tốt khỏi phần còn lại của hệ thống.

Bug fixing is also a good time to assess the orthogonality of the system as a whole. When you come across a problem, assess how localized the fix is. Do you change just one module, or are the changes scattered throughout the entire system? When you make a change, does it fix everything, or do other problems mysteriously arise? This is a good opportunity to bring automation to bear. If you use a **version control** system (and you will after reading Topic 19, Version Control), tag bug fixes when you check the code back in after testing. You can then run monthly reports analyzing trends in the number of source files affected by each bug fix.

Sửa lỗi cũng là một thời điểm tốt để đánh giá tính trực giao của toàn bộ hệ thống. Khi bạn gặp một vấn đề, hãy đánh giá xem cách sửa nó cục bộ đến mức nào. Bạn chỉ thay đổi đúng một module, hay các thay đổi rải rác khắp cả hệ thống? Khi bạn thực hiện một thay đổi, nó có sửa được mọi thứ không, hay những vấn đề khác lại bí ẩn xuất hiện? Đây là cơ hội tốt để áp dụng tự động hóa. Nếu bạn dùng một hệ thống quản lý phiên bản (và bạn sẽ dùng sau khi đọc Topic 19, Quản lý phiên bản), hãy gắn thẻ các lần sửa lỗi khi bạn check mã trở lại sau khi kiểm thử. Sau đó bạn có thể chạy các báo cáo hằng tháng phân tích xu hướng về số lượng tệp mã nguồn bị ảnh hưởng bởi mỗi lần sửa lỗi.

Documentation — Tài liệu

Perhaps surprisingly, orthogonality also applies to documentation. The axes are content and presentation. With truly orthogonal documentation, you should be able to change the appearance dramatically without changing the content. Word processors provide style sheets and macros that help. We personally prefer using a markup system such as Markdown: when writing we focus only on the content, and leave the presentation to whichever tool we use to render it. [17]

Có lẽ đáng ngạc nhiên là tính trực giao cũng áp dụng cho tài liệu. Các trục ở đây là nội dung và cách trình bày. Với tài liệu thực sự trực giao, bạn hẳn có thể thay đổi diện mạo một cách mạnh mẽ mà không thay đổi nội dung. Các trình xử lý văn bản cung cấp các style sheet và macro giúp làm việc này. Bản thân chúng tôi thì thích dùng một hệ thống đánh dấu (markup) như Markdown: khi viết, chúng tôi chỉ tập trung vào nội dung, và để phần trình bày cho bất kỳ công cụ nào chúng tôi dùng để kết xuất nó. [17]

Living With Orthogonality — Sống chung với tính trực giao

Orthogonality is closely related to the **DRY** principle . With DRY, you're looking to minimize **duplication** within a system, whereas with orthogonality you reduce the interdependency among the system's components. It may be a clumsy word, but if you use the principle of orthogonality, combined closely with the DRY principle, you'll find that the systems you develop are more flexible, more understandable, and easier to debug, test, and maintain.

Tính trực giao có liên hệ mật thiết với nguyên tắc DRY. Với DRY, bạn nhắm tới việc giảm thiểu sự trùng lặp bên trong một hệ thống, trong khi với tính trực giao bạn giảm bớt sự phụ thuộc lẫn nhau giữa các thành phần của hệ thống. Có thể đó là một từ vựng về, nhưng nếu bạn dùng nguyên tắc trực giao, kết hợp chặt chẽ với nguyên tắc DRY, bạn sẽ thấy rằng những hệ thống mình phát triển trở nên linh hoạt hơn, dễ hiểu hơn, và dễ gỡ lỗi, kiểm thử cùng bảo trì hơn.

If you're brought into a project where people are desperately struggling to make changes, and where every change seems to cause four other things to go wrong, remember the nightmare with the helicopter. The project probably is not orthogonally designed and coded. It's time to refactor.

Nếu bạn được đưa vào một dự án mà ở đó người ta đang vật vã tuyệt vọng để thực hiện các thay đổi, và mỗi thay đổi dường như lại khiến bốn thứ khác hỏng theo, hãy nhớ lại cơn ác mộng với chiếc trực thăng. Dự án đó hẳn không được thiết kế và viết mã một cách trực giao. Đã đến lúc tái cấu trúc.

And, if you're a helicopter pilot, don't eat the fish....

Và, nếu bạn là phi công trực thăng, đừng có ăn cá nhé....

Related Sections Include — Các mục liên quan

Topic 3, **Software Entropy**

Topic 3, Sự rối loạn phần mềm

Topic 8, The Essence of Good Design

Topic 8, Tinh hoa của Thiết kế Tốt

Topic 11, **Reversibility**

Topic 11, Tính khả nghịch

Topic 28, Decoupling

Topic 28, Gỡ ghép nối

Topic 31, **Inheritance** Tax

Topic 31, Thuế Kế thừa

Topic 33, Breaking **Temporal Coupling**

Topic 33, Phá vỡ Ghép nối Thời gian

Topic 34, **Shared State Is Incorrect State**

Topic 34, Trạng thái Chia sẻ là Trạng thái Sai

Topic 36, Blackboards

Topic 36, Bảng đen

Challenges

Thử thách

Consider the difference between tools which have a graphical user interface and small but combinable command-line utilities used at **shell** prompts. Which set is more orthogonal, and why? Which is easier to use for exactly the purpose for which it was intended? Which set is easier to combine with other tools to meet new challenges? Which set is easier to learn?

Hãy xét sự khác biệt giữa những công cụ có giao diện người dùng đồ họa và những tiện ích dòng lệnh nhỏ nhưng có thể kết hợp được, dùng tại các dấu nhắc shell. Tập công cụ nào trực giao hơn, và vì sao? Tập nào dễ dùng hơn cho đúng mục đích mà nó được tạo ra? Tập nào dễ kết hợp với các công cụ khác hơn để đáp ứng những thử thách mới? Tập nào dễ học hơn?

C++ supports multiple inheritance, and Java allows a class to implement multiple interfaces. Ruby has mixins. What impact does using these facilities have on orthogonality? Is there a difference in impact between using multiple inheritance and multiple interfaces? Is there a difference between using delegation and using inheritance?

C++ hỗ trợ đa kế thừa, và Java cho phép một lớp hiện thực nhiều giao diện. Ruby có mixin. Việc sử dụng những tính năng này có tác động gì lên tính trực giao? Có khác biệt nào về tác động giữa việc dùng đa kế thừa và dùng nhiều giao diện không? Có khác biệt nào giữa việc dùng ủy quyền (delegation) và dùng kế thừa không?

Exercises Exercise 1 (possible answer)

Bài tập Bài tập 1 (câu trả lời gợi ý)

You're asked to read a file a line at a time. For each line, you have to split it into fields. Which of the following sets of pseudo class definitions is likely to be more orthogonal?

Bạn được yêu cầu đọc một tệp theo từng dòng một. Với mỗi dòng, bạn phải tách nó thành các trường. Tập định nghĩa lớp giả mã nào sau đây có khả năng trực giao hơn?

```
class Split1 { constructor(fileName) # opens the file for reading def readNextLine() # moves to the next line def getField(n) # returns nth field in current line
```

```
class Split1 { constructor(fileName) # mở tệp để đọc def readNextLine() # chuyển sang dòng tiếp theo def getField(n) # trả về trường thứ n trong dòng hiện tại
```

} or

} hoặc

```
class Split2 { constructor(line) # splits a line def getField(n) # returns nth field in current line }  
class Split2 { constructor(line) # tách một dòng def getField(n) # trả về trường thứ n trong  
dòng hiện tại }
```

Exercise 2 (possible answer)

Bài tập 2 (câu trả lời gợi ý)

What are the differences in orthogonality between objectoriented and functional languages? Are these differences inherent in the languages themselves, or just in the way people use them?

Những khác biệt về tính trực giao giữa các ngôn ngữ hướng đối tượng và các ngôn ngữ hàm là gì? Những khác biệt này là vốn có trong bản thân các ngôn ngữ, hay chỉ nằm ở cách người ta sử dụng chúng?

Nothing is more dangerous than an idea if it's the only one you have. Emil-Auguste Chartier (Alain), *Propos sur la religion*,

Không có gì nguy hiểm hơn một ý tưởng nếu đó là ý tưởng duy nhất bạn có. Emil-Auguste Chartier (Alain), *Propos sur la religion*,

Topic 11. Reversibility — Topic 11. Tính khả nghịch

Engineers prefer simple, singular solutions to problems. Math tests that allow you to proclaim with great confidence that are much more comfortable than fuzzy, warm essays about the myriad causes of the French Revolution. Management tends to agree with the engineers: singular, easy answers fit nicely on spreadsheets and project plans.

Các kỹ sư thích những giải pháp đơn giản, duy nhất cho mỗi vấn đề. Những bài kiểm tra toán cho phép bạn đồng ý tuyên bố một đáp án chắc nịch thì dễ chịu hơn nhiều so với những bài luận mơ hồ, ấm áp về vô vàn nguyên nhân của Cách mạng Pháp. Giới quản lý cũng có xu hướng đồng tình với các kỹ sư: những câu trả lời đơn lẻ, dễ dàng thì vừa khít một cách gọn gàng trên bảng tính và kế hoạch dự án.

If only the real world would cooperate! Unfortunately, while is today, it may need to be tomorrow, and next week. Nothing is forever—and if you rely heavily on some fact, you can almost guarantee that it will change.

Giá mà thế giới thực chịu hợp tác! Tiếc thay, cái gì đúng hôm nay thì ngày mai có thể cần thay đổi, rồi tuần sau lại đổi nữa. Chẳng có gì là vĩnh viễn cả — và nếu bạn dựa dẫm quá nhiều vào một sự thật nào đó, bạn gần như chắc chắn rằng nó sẽ thay đổi.

There is always more than one way to implement something, and there is usually more than one vendor available to provide a third-party product. If you go into a project hampered by the myopic notion that there is only one way to do it, you may be in for an unpleasant surprise. Many project teams have their eyes forcibly opened as the future unfolds:

Luôn có nhiều hơn một cách để hiện thực hóa một thứ gì đó, và thường cũng có nhiều hơn một nhà cung cấp sẵn sàng đưa ra sản phẩm của bên thứ ba. Nếu bạn bước vào một dự án với cái nhìn thiên cận rằng chỉ có duy nhất một cách để làm, bạn có thể sẽ phải nhận một bất ngờ chẳng dễ chịu chút nào. Nhiều nhóm dự án bị buộc phải mở to mắt khi tương lai dần hé lộ:

“But you said we’d use database XYZ! We are 85% done coding the project, we can’t change now!” the programmer protested. “Sorry, but our company decided to standardize on database

“Nhưng anh đã nói chúng ta sẽ dùng cơ sở dữ liệu XYZ mà! Chúng tôi đã code xong 85% dự án rồi, giờ không đổi được đâu!” lập trình viên phản đối. “Xin lỗi, nhưng công ty ta đã quyết định chuẩn hóa sang cơ sở dữ liệu

PDQ instead—for all projects. It’s out of my hands. We’ll just have to recode. All of you will be working weekends until further notice.”

PDQ — cho mọi dự án. Việc này nằm ngoài tầm tay tôi. Chúng ta sẽ phải code lại thôi. Tất cả các cậu sẽ làm việc cả cuối tuần cho tới khi có thông báo mới.”

Changes don’t have to be that Draconian, or even that immediate. But as time goes by, and your project progresses, you may find yourself stuck in an untenable position. With every critical decision, the project team commits to a smaller target—a narrower version of reality that has fewer options.

Thay đổi không nhất thiết phải hà khắc đến thế, hay thậm chí cũng chẳng cần đến ngay lập tức. Nhưng theo thời gian, khi dự án của bạn tiến triển, bạn có thể thấy mình mắc kẹt ở một vị thế không thể nào trụ nổi. Với mỗi quyết định then chốt, nhóm dự án lại cam kết với một mục tiêu nhỏ hơn — một phiên bản hẹp hơn của thực tại, với ít lựa chọn hơn.

By the time many critical decisions have been made, the target becomes so small that if it moves, or the wind changes direction, or a butterfly in Tokyo flaps its wings, you miss. [18] And you may miss by a huge amount.

Đến khi nhiều quyết định then chốt đã được đưa ra, cái đích trở nên nhỏ đến mức chỉ cần nó nhúc nhích, hay gió đổi chiều, hay một con bướm ở Tokyo vỗ cánh, là bạn trượt mất. [18] Và bạn có thể trượt một khoảng rất xa.

The problem is that critical decisions aren’t easily reversible.

Vấn đề là ở chỗ những quyết định then chốt không dễ gì đảo ngược được.

Once you decide to use this vendor’s database, or that architectural pattern, or a certain deployment model, you are committed to a course of action that cannot be undone, except at great expense.

Một khi bạn đã quyết định dùng cơ sở dữ liệu của nhà cung cấp này, hay mẫu kiến trúc kia, hay một mô hình triển khai nào đó, bạn đã cam kết với một lộ trình hành động không thể hoàn tác — trừ phi chịu trả giá rất đắt.

Reversibility — Tính khả nghịch

Many of the topics in this book are geared to producing flexible, adaptable software. By sticking to their recommendations— especially the **DRY** principle , **decoupling** , and use of external **configuration** —we don’t have to make as many critical, irreversible decisions. This is a good thing, because we don’t always make the best decisions the first time around. We commit to a certain technology only to discover we can’t hire enough people with the necessary skills. We lock in a certain third-party vendor just before they get bought out by their competitor. **Requirements**, users, and hardware change faster

Nhiều chủ đề trong cuốn sách này hướng tới việc tạo ra phần mềm mềm dẻo, dễ thích nghi. Bằng cách bám sát những khuyến nghị đó — đặc biệt là nguyên tắc DRY — Đừng Lặp Lại Chính Mình, việc gỡ ghép nối (decoupling), và việc dùng cấu hình bên ngoài — chúng ta không phải đưa ra quá nhiều quyết định then chốt, bất khả nghịch. Đây là điều tốt, bởi chúng ta không phải lúc nào cũng đưa ra quyết định tối ưu ngay từ lần đầu. Ta cam kết với

một công nghệ nào đó rồi mới phát hiện không thể tuyển đủ người có kỹ năng cần thiết. Ta khóa chặt vào một nhà cung cấp bên thứ ba ngay trước khi họ bị đối thủ thâm tóm. Yêu cầu, người dùng và phần cứng thay đổi nhanh hơn

than we can get the software developed.

so với tốc độ chúng ta phát triển được phần mềm.

Suppose you decide, early in the project, to use a relational database from vendor A. Much later, during performance testing, you discover that the database is simply too slow, but that the document database from vendor B is faster. With most conventional projects, you'd be out of luck. Most of the time, calls to third-party products are entangled throughout the code. But if you really abstracted the idea of a database out—to the point where it simply provides persistence as a service—then you have the flexibility to change horses in midstream.

Giả sử ngay từ đầu dự án, bạn quyết định dùng một cơ sở dữ liệu quan hệ (relational database) từ nhà cung cấp A. Mãi về sau, trong lúc kiểm thử hiệu năng, bạn phát hiện ra cơ sở dữ liệu đó đơn giản là quá chậm, còn cơ sở dữ liệu dạng tài liệu (document database) của nhà cung cấp B lại nhanh hơn. Với hầu hết các dự án thông thường, bạn sẽ hết đường xoay xở. Phần lớn thời gian, các lời gọi đến sản phẩm bên thứ ba bị đan xen rối rắm khắp mã nguồn. Nhưng nếu bạn thực sự đã trừu tượng hóa ý niệm về cơ sở dữ liệu — đến mức nó chỉ đơn thuần cung cấp tính bền vững như một dịch vụ — thì bạn có sự mềm dẻo để đổi ngựa giữa dòng.

Similarly, suppose the project begins as a browser-based application, but then, late in the game, marketing decides that what they really want is a mobile app. How hard would that be for you? In an ideal world, it shouldn't impact you too much, at least on the server side. You'd be stripping out some HTML rendering and replacing it with an API.

Tương tự, giả sử dự án khởi đầu là một ứng dụng chạy trên trình duyệt, nhưng rồi đến phút chót, bộ phận marketing quyết định rằng thứ họ thực sự muốn lại là một ứng dụng di động. Việc đó sẽ khó đến đâu với bạn? Trong một thế giới lý tưởng, nó chẳng nên ảnh hưởng đến bạn nhiều lắm, ít nhất là ở phía máy chủ. Bạn chỉ việc lột bỏ phần kết xuất HTML và thay nó bằng một API.

The mistake lies in assuming that any decision is cast in stone— and in not preparing for the contingencies that might arise. Instead of carving decisions in stone, think of them more as being written in the sand at the beach. A big wave can come along and wipe them out at any time.

Sai lầm nằm ở chỗ cho rằng mọi quyết định đều được đúc bằng đá — và ở chỗ không hề chuẩn bị cho những tình huống bất ngờ có thể nảy sinh. Thay vì khắc quyết định vào đá, hãy nghĩ về chúng như những dòng chữ viết trên cát ngoài bãi biển. Một con sóng lớn có thể ập đến và xóa sạch chúng bất cứ lúc nào.

Tip 18: There Are No Final Decisions

Tip 18: Không Có Quyết Định Nào Là Cuối Cùng

Flexible Architecture — Kiến trúc mềm dẻo

While many people try to keep their code flexible, you also need to think about maintaining flexibility in the areas of architecture, deployment, and vendor integration.

Trong khi nhiều người cố giữ cho mã nguồn của mình mềm dẻo, bạn cũng cần nghĩ đến việc duy trì sự mềm dẻo trong các mảng kiến trúc, triển khai và tích hợp với nhà cung cấp.

We're writing this in 2019. Since the turn of the century we've seen the following “best practice” server-side architectures:

Chúng tôi viết những dòng này vào năm 2019. Kể từ đầu thế kỷ đến nay, chúng tôi đã chứng kiến những kiến trúc phía máy chủ được coi là “best practice” sau đây:

Big hunk of iron

Một khối sắt to đùng

Federations of big iron

Liên minh của những khối sắt to đùng

Load-balanced clusters of commodity hardware

Các cụm máy thương phẩm cân bằng tải

Cloud-based virtual machines running applications

Máy ảo trên đám mây chạy ứng dụng

Cloud-based virtual machines running services

Máy ảo trên đám mây chạy dịch vụ

Containerized versions of the above

Phiên bản đóng gói container của những thứ trên

Cloud-supported serverless applications

Ứng dụng serverless do đám mây hậu thuẫn

And, inevitably, an apparent move back to big hunks of iron for some tasks

Và, tất yếu, một động thái rõ ràng quay trở lại với những khối sắt to đùng cho một số tác vụ

Go ahead and add the very latest and greatest fads to this list, and then regard it with awe: it's a miracle that anything ever worked.

Cứ thoải mái thêm vào danh sách này những trào lưu mới nhất, “hot” nhất, rồi nhìn nó mà kinh ngạc: thật là một phép màu khi mọi thứ vẫn từng chạy được.

How can you plan for this kind of architectural volatility? You can't.

Làm sao bạn có thể lên kế hoạch cho kiểu biến động kiến trúc như thế này? Bạn không thể.

What you can do is make it easy to change. Hide third-party APIs behind your own abstraction layers. Break your code into components: even if you end up deploying them on a single massive server, this approach is a lot easier than taking a monolithic application and splitting it. (We have the scars to prove it.)

Thứ bạn có thể làm là khiến nó dễ thay đổi. Giấu các API của bên thứ ba sau những tầng trừu tượng của riêng bạn. Tách mã nguồn thành các thành phần: kể cả khi cuối cùng bạn triển khai tất cả chúng trên một máy chủ khổng lồ duy nhất, cách làm này vẫn dễ hơn nhiều so với việc đem một ứng dụng nguyên khối ra mà chẻ nó ra. (Chúng tôi có sẹo để làm chứng cho điều đó.)

And, although this isn't particularly a **reversibility** issue, one final piece of advice.

Và, dù điều này không hẳn là chuyện về tính khả nghịch, đây là một lời khuyên cuối cùng.

Tip 19: Forgo Following Fads

Tip 19: Hãy Bỏ Qua Việc Chạy Theo Trào Lưư

No one knows what the future may hold, especially not us! So enable your code to rock-n-roll: to “rock on” when it can, to roll with the punches when it must.

Chẳng ai biết tương lai cất giấu điều gì, đặc biệt là chúng tôi! Vậy nên hãy cho mã nguồn của bạn biết “rock-n-roll”: “quẩy” hết mình khi có thể, và uyển chuyển né đòn khi buộc phải vậy.

Related Sections Include — Các mục liên quan

Topic 8, The Essence of Good Design

Topic 8, Cốt lõi của Thiết kế Tốt

Topic 10, **Orthogonality**

Topic 10, Tính trực giao

Topic 19, **Version Control**

Topic 19, Quản lý phiên bản

Topic 28, Decoupling

Topic 28, Gỡ ghép nối

Topic 45, The Requirements Pit

Topic 45, Cái hố Yêu cầu

Topic 51, **Pragmatic Starter Kit**

Topic 51, Bộ khởi đầu Thực dụng

Challenges — Thử thách

Time for a little quantum mechanics with Schrödinger’s cat. Suppose you have a cat in a closed box, along with a radioactive particle. The particle has exactly a 50% chance of fissioning into two. If it does, the cat will be killed. If it doesn’t, the cat will be okay. So, is the cat dead or alive? According to Schrödinger, the correct answer is both (at least while the box remains closed). Every time a subnuclear reaction takes place that has two possible outcomes, the universe is cloned. In one, the **event** occurred, in the other it didn’t. The cat’s alive in one universe, dead in another. Only when you open the box do you know which universe you are in. No wonder coding for the future is difficult. But think of code evolution along the same lines as a box full of Schrödinger’s cats: every decision results in a different version of the future. How many possible futures can your code support? Which ones are more likely? How hard will it be to support them when the time comes? Dare you open the box?

Đến giờ chơi một chút cơ học lượng tử với con mèo của Schrödinger nào. Giả sử bạn có một con mèo trong một cái hộp kín, cùng với một hạt phóng xạ. Hạt này có đúng 50% khả năng phân hạch thành hai. Nếu nó phân hạch, con mèo sẽ chết. Nếu không, con mèo vẫn bình an. Vậy thì, con mèo đang chết hay đang sống? Theo Schrödinger, câu trả lời đúng là cả hai (ít nhất là chừng nào cái hộp còn đóng). Mỗi lần một phản ứng hạ nguyên tử có hai kết cục khả dĩ xảy ra, vũ trụ lại được nhân bản. Ở vũ trụ này, sự kiện đã xảy ra; ở vũ trụ kia, nó đã không. Con mèo sống ở vũ trụ này, chết ở vũ trụ khác. Chỉ khi mở hộp ra bạn mới

biết mình đang ở vũ trụ nào. Chẳng trách viết mã cho tương lai lại khó đến vậy. Nhưng hãy hình dung sự tiến hóa của mã nguồn theo cùng cách ấy, như một cái hộp đầy những con mèo của Schrödinger: mỗi quyết định cho ra một phiên bản khác nhau của tương lai. Mã của bạn có thể hỗ trợ bao nhiêu tương lai khả dĩ? Cái nào dễ xảy ra hơn? Sẽ khó đến mức nào để hỗ trợ chúng khi thời điểm ấy đến? Bạn có dám mở hộp ra không?

Ready, fire, aim... Anon

Sẵn sàng, bắn, ngắm... Khuyết danh

Topic 12. Tracer Bullets — Topic 12. Đạn vạch đường

We often talk about hitting targets when we develop software. We're not actually firing anything at the shooting range, but it's still a useful and very visual metaphor. In particular, it's interesting to consider how to hit a target in a complex and shifting world.

Khi phát triển phần mềm, chúng ta hay nói về chuyện bắn trúng mục tiêu. Thật ra chẳng có ai nổ súng ở trường bắn cả, nhưng đây vẫn là một phép ẩn dụ hữu ích và rất trực quan. Đặc biệt, thật thú vị khi suy ngẫm xem làm thế nào để bắn trúng mục tiêu trong một thế giới phức tạp và luôn dịch chuyển.

The answer, of course, depends on the nature of the device you're aiming with. With many you only get one chance to aim, and then get to see if you hit the bullseye or not. But there's a better way.

Câu trả lời, dĩ nhiên, phụ thuộc vào bản chất của thứ vũ khí bạn dùng để ngắm. Với nhiều loại, bạn chỉ có một cơ hội duy nhất để ngắm, rồi sau đó mới biết mình có bắn trúng hồng tâm hay không. Nhưng có một cách hay hơn.

You know all those movies, TV shows, and video games where people are shooting machine guns? In these scenes, you'll often see the path of bullets as bright streaks in the air. These streaks come from **tracer bullets**.

Bạn có nhớ tất cả những bộ phim, chương trình truyền hình và trò chơi điện tử có cảnh người ta xả súng máy không? Trong những cảnh đó, bạn thường thấy đường đi của viên đạn hiện thành những vệt sáng vạch ngang không trung. Những vệt sáng ấy đến từ đạn vạch đường (tracer bullets).

Tracer bullets are loaded at intervals alongside regular ammunition. When they're fired, their phosphorus ignites and leaves a pyrotechnic trail from the gun to whatever they hit. If the tracers are hitting the target, then so are the regular bullets. Soldiers use these tracer rounds to refine their aim: it's **pragmatic**, real-time feedback under actual conditions.

Đạn vạch đường được nạp xen kẽ giữa những viên đạn thường theo từng quãng nhất định. Khi bắn ra, lớp phốt-pho của chúng bốc cháy và để lại một vệt lửa kéo dài từ nòng súng đến bất cứ thứ gì chúng trúng phải. Nếu đạn vạch đường trúng mục tiêu, thì đạn thường cũng trúng. Lính dùng những viên đạn vạch này để chỉnh đường ngắm: đó là phản hồi thực dụng (pragmatic), tức thời, ngay trong điều kiện thực chiến.

That same principle applies to projects, particularly when you're building something that hasn't been built before. We use the

Cùng một nguyên lý ấy cũng áp dụng cho các dự án, đặc biệt khi bạn đang xây dựng một thứ chưa từng được xây dựng bao giờ. Chúng tôi dùng

term tracer bullet development to visually illustrate the need for immediate feedback under actual conditions with a moving goal.

thuật ngữ phát triển kiểu đạn vạch đường để minh họa một cách trực quan cho nhu cầu nhận phản hồi tức thời trong điều kiện thực tế, với một mục tiêu không ngừng di chuyển.

Like the gunners, you're trying to hit a target in the dark. Because your users have never seen a system like this before, their **requirements** may be vague. Because you may be using algorithms, techniques, languages, or libraries you aren't familiar with, you face a large number of unknowns. And because projects take time to complete, you can pretty much guarantee the environment you're working in will change before you're done.

Cũng như những xạ thủ kia, bạn đang cố bắn trúng một mục tiêu trong bóng tối. Vì người dùng chưa từng thấy một hệ thống nào như thế này, các yêu cầu của họ có thể rất mơ hồ. Vì bạn có thể đang dùng những thuật toán, kỹ thuật, ngôn ngữ hay thư viện mà mình chưa quen, bạn phải đối mặt với vô số ẩn số. Và vì các dự án cần thời gian để hoàn thành, gần như chắc chắn môi trường bạn đang làm việc sẽ thay đổi trước khi bạn kịp xong.

The classic response is to specify the system to death. Produce reams of paper itemizing every requirement, tying down every unknown, and constraining the environment. Fire the gun using dead reckoning. One big calculation up front, then shoot and hope.

Phản ứng kinh điển là đặc tả hệ thống đến chết. Sản xuất hàng đống giấy tờ liệt kê từng yêu cầu một, trói chặt mọi ẩn số, và ràng buộc cả môi trường. Bóp cò bằng cách nhắm chừng. Một phép tính lớn ngay từ đầu, rồi bắn và cầu may.

Pragmatic Programmers, however, tend to prefer using the software equivalent of tracer bullets.

Tuy nhiên, Lập trình viên Thực dụng (Pragmatic Programmer) lại có xu hướng thích dùng thứ tương đương của đạn vạch đường trong phần mềm.

Code That Glows In The Dark — Đoạn mã phát sáng trong bóng tối

Tracer bullets work because they operate in the same environment and under the same constraints as the real bullets. They get to the target fast, so the gunner gets immediate feedback. And from a practical standpoint they're a relatively cheap solution.

Đạn vạch đường phát huy tác dụng vì chúng hoạt động trong cùng một môi trường và chịu cùng những ràng buộc như đạn thật. Chúng lao tới mục tiêu thật nhanh, nên xạ thủ nhận phản hồi tức thời. Và xét về mặt thực tế, đó là một giải pháp tương đối rẻ.

To get the same effect in code, we look for something that gets us from a requirement to some aspect of the final system quickly, visibly, and repeatably.

Để đạt được hiệu ứng tương tự trong mã nguồn, chúng ta tìm kiếm một thứ gì đó đưa ta đi từ một yêu cầu đến một khía cạnh nào đó của hệ thống cuối cùng một cách nhanh chóng, rõ ràng và có thể lặp lại.

Look for the important requirements, the ones that define the system. Look for the areas where you have doubts, and where you see the biggest risks. Then prioritize your development so that these are the first areas you code.

Hãy tìm những yêu cầu quan trọng, những yêu cầu định hình nên hệ thống. Hãy tìm những vùng mà bạn còn nghi ngờ, những nơi bạn thấy rủi ro lớn nhất. Rồi ưu tiên việc phát triển sao cho đây chính là những vùng bạn viết mã trước tiên.

Tip 20: Use Tracer Bullets to Find the Target

Tip 20: Dùng Đạn Vạch Đường Để Tìm Mục Tiêu

In fact, given the complexity of today’s project setup, with swarms of external dependencies and tools, tracer bullets become even more important. For us, the very first tracer bullet is simply create the project, add a “hello world!” and make sure it compiles and runs. Then we look for areas of uncertainty in the overall application and add the skeleton needed to make it work.

Thực ra, với độ phức tạp của việc thiết lập dự án ngày nay, với vô số phụ thuộc bên ngoài (external dependencies) và công cụ vây quanh, đạn vạch đường lại càng trở nên quan trọng hơn. Với chúng tôi, viên đạn vạch đầu tiên đơn giản chỉ là tạo dự án, thêm một dòng “hello world!”, và bảo đảm nó biên dịch và chạy được. Sau đó chúng tôi tìm những vùng còn bất định trong toàn bộ ứng dụng và thêm bộ khung tối thiểu cần thiết để nó hoạt động.

Have a look at the following diagram. This system has five architectural layers. We have some concerns about how they’d integrate, so we look for a simple feature that lets us exercise them together. The diagonal line shows the path that feature takes through the code. To make it work, we just have to implement the solidly shaded areas in each layer: the stuff with the squiggles will be done later.

Hãy xem sơ đồ dưới đây. Hệ thống này có năm tầng kiến trúc. Chúng tôi có chút lo ngại về cách chúng tích hợp với nhau, nên chúng tôi tìm một tính năng đơn giản cho phép vận hành tất cả các tầng cùng lúc. Đường chéo cho thấy lối đi mà tính năng đó xuyên qua mã nguồn. Để nó chạy được, chúng ta chỉ cần hiện thực những vùng được tô đặc trong mỗi tầng: phần có những đường loằng ngoằng sẽ làm sau.

images/layer-diagram-tracer.png

images/layer-diagram-tracer.png

We once undertook a complex **client**-server database marketing project. Part of its requirement was the ability to specify and execute temporal queries. The servers were a range of relational and specialized databases. The client UI, written in random language A, used a set of libraries written in a different language to provide an interface to the servers. The user’s query was stored on the server in a Lisp-like notation before being converted to optimized SQL just prior to execution. There were many unknowns and many different environments, and no one was too sure how the UI should behave.

Có lần chúng tôi từng nhận một dự án marketing trên cơ sở dữ liệu kiểu khách-chủ (client-server) phức tạp. Một phần yêu cầu của nó là khả năng đặc tả và thực thi các truy vấn theo thời gian. Phía máy chủ là một loạt cơ sở dữ liệu quan hệ và chuyên dụng. Giao diện người dùng phía khách, viết bằng một ngôn ngữ A bất kỳ, dùng một tập thư viện viết bằng ngôn ngữ khác để cung cấp giao tiếp với các máy chủ. Truy vấn của người dùng được lưu trên máy chủ dưới dạng ký hiệu kiểu Lisp trước khi được chuyển thành SQL tối ưu ngay trước khi thực thi. Có rất nhiều ẩn số và rất nhiều môi trường khác nhau, và chẳng ai dám chắc giao diện người dùng nên hành xử ra sao.

This was a great opportunity to use tracer code. We developed the framework for the front end, libraries for representing the queries, and a structure for converting a stored query into a database-specific query. Then we put it all together and checked that it worked. For that initial build, all we could do was submit

Đây là một cơ hội tuyệt vời để dùng mã vạch đường. Chúng tôi xây dựng khung sườn cho phần giao diện đầu (front end), các thư viện để biểu diễn truy vấn, và một cấu trúc để chuyển một truy vấn đã lưu thành truy vấn đặc thù cho từng cơ sở dữ liệu. Rồi chúng tôi ghép tất cả lại và kiểm tra xem nó có chạy không. Với bản dựng đầu tiên đó, tất cả những gì chúng tôi làm được là gửi đi

a query that listed all the rows in a table, but it proved that the UI could talk to the libraries, the libraries could serialize and unserialize a query, and the server could generate SQL from the result. Over the following months we gradually fleshed out this basic structure, adding new functionality by augmenting each component of the tracer code in parallel. When the UI added a new query type, the library grew and the SQL generation was made more sophisticated.

một truy vấn liệt kê toàn bộ các hàng trong một bảng, nhưng nó đã chứng minh rằng giao diện người dùng có thể nói chuyện với các thư viện, các thư viện có thể tuần tự hóa và giải tuần tự hóa một truy vấn, và máy chủ có thể sinh ra SQL từ kết quả ấy. Trong những tháng tiếp theo, chúng tôi dần dần đắp da đắp thịt cho cái cấu trúc cơ bản này, bổ sung chức năng mới bằng cách mở rộng song song từng thành phần của mã vạch đường. Khi giao diện người dùng thêm một loại truy vấn mới, thư viện lớn lên và phần sinh SQL trở nên tinh vi hơn.

Tracer code is not disposable: you write it for keeps. It contains all the error checking, structuring, documentation, and selfchecking that any piece of production code has. It simply is not fully functional. However, once you have achieved an end-to-end connection among the components of your system, you can check how close to the target you are, adjusting if necessary. Once you're on target, adding functionality is easy.

Mã vạch đường không phải loại dùng một lần rồi vứt: bạn viết nó để giữ lại lâu dài. Nó chứa đầy đủ kiểm tra lỗi, cấu trúc, tài liệu và tự kiểm tra như bất kỳ đoạn mã sản xuất nào. Chỉ là nó chưa đầy đủ chức năng mà thôi. Tuy nhiên, một khi bạn đã thiết lập được kết nối đầu-cuối giữa các thành phần của hệ thống, bạn có thể kiểm tra xem mình đã gần mục tiêu đến đâu và điều chỉnh nếu cần. Một khi đã đúng hướng, việc thêm chức năng trở nên dễ dàng.

Tracer development is consistent with the idea that a project is never finished: there will always be changes required and functions to add. It is an incremental approach.

Phát triển kiểu vạch đường nhất quán với ý tưởng rằng một dự án không bao giờ kết thúc: sẽ luôn có những thay đổi cần làm và những hàm cần thêm. Đây là một cách tiếp cận tăng dần.

The conventional alternative is a kind of heavy engineering approach: code is divided into modules, which are coded in a vacuum. Modules are combined into subassemblies, which are then further combined, until one day you have a complete application. Only then can the application as a whole be presented to the user and tested.

Phương án quy ước thay thế lại là một kiểu tiếp cận kỹ thuật nặng nề: mã được chia thành các mô-đun, mỗi mô-đun được viết trong chân không. Các mô-đun được ghép thành cụm nhỏ, rồi tiếp tục được ghép lại, cho đến một ngày bạn có một ứng dụng hoàn chỉnh. Chỉ đến lúc đó toàn bộ ứng dụng mới được trình ra cho người dùng và đem đi kiểm thử.

The tracer code approach has many advantages:

Cách tiếp cận mã vạch đường có nhiều ưu điểm:

Users get to see something working early If you have successfully communicated what you are doing (see **Topic 52**, *Delight Your Users*), your users will

Người dùng được thấy thứ gì đó chạy được từ sớm Nếu bạn đã truyền đạt thành công những gì mình đang làm (xem Topic 52, *Làm người dùng vui sướng*), người dùng sẽ

know they are seeing something immature. They won't be disappointed by a lack of functionality; they'll be ecstatic to see some visible progress toward their system. They also get to contribute as the

project progresses, increasing their buy-in. These same users will likely be the people who'll tell you how close to the target each iteration is.

biết họ đang xem một thứ còn non. Họ sẽ không thất vọng vì thiếu chức năng; họ sẽ phấn khích khi thấy có tiến triển hữu hình hướng tới hệ thống của mình. Họ cũng được góp phần khi dự án tiến triển, làm tăng sự gắn kết của họ. Cũng chính những người dùng này nhiều khả năng sẽ là người cho bạn biết mỗi vòng lặp đã gần mục tiêu đến mức nào.

Developers build a structure to work in The most daunting piece of paper is the one with nothing written on it. If you have worked out all the end-to-end interactions of your application, and have embodied them in code, then your team won't need to pull as much out of thin air. This makes everyone more productive, and encourages consistency.

Lập trình viên có sẵn một bộ khung để làm việc Tờ giấy đáng sợ nhất là tờ giấy chẳng có gì viết trên đó. Nếu bạn đã hình dung ra mọi tương tác đầu-cuối của ứng dụng và đã hiện thân chúng thành mã, thì nhóm của bạn sẽ không phải bịa ra quá nhiều thứ từ hư không. Điều này khiến mọi người làm việc hiệu quả hơn và khuyến khích sự nhất quán.

You have an integration platform As the system is connected end-to-end, you have an environment to which you can add new pieces of code once they have been unit-tested. Rather than attempting a big-bang integration, you'll be integrating every day (often many times a day). The impact of each new change is more apparent, and the interactions are more limited, so **debugging** and testing are faster and more accurate.

Bạn có một nền tảng tích hợp Khi hệ thống đã được nối thông đầu-cuối, bạn có sẵn một môi trường để thêm vào những đoạn mã mới ngay khi chúng được kiểm thử đơn vị. Thay vì cố tích hợp một cú “nổ lớn”, bạn sẽ tích hợp mỗi ngày (thường là nhiều lần một ngày). Tác động của từng thay đổi mới hiện rõ hơn, các tương tác cũng giới hạn hơn, nên việc gỡ lỗi và kiểm thử nhanh và chính xác hơn.

You have something to demonstrate Project sponsors and top brass have a tendency to want to see demos at the most inconvenient times. With tracer code, you'll always have something to show them.

Bạn có thứ để trình diễn Các nhà tài trợ dự án và sếp lớn thường có thói quen muốn xem demo vào những lúc bất tiện nhất. Với mã vạch đường, bạn sẽ luôn có sẵn thứ gì đó để cho họ xem.

You have a better feel for progress In a tracer code development, developers tackle use cases one by one. When one is done, they move to the next. It is

Bạn cảm nhận tiến độ rõ hơn Trong lối phát triển bằng mã vạch đường, lập trình viên xử lý từng ca sử dụng một. Xong một ca, họ chuyển sang ca tiếp theo. Việc

far easier to measure performance and to demonstrate progress to your user. Because each individual development is smaller, you avoid creating those monolithic blocks of code that are reported as 95% complete week after week.

đo lường hiệu năng và trình diễn tiến độ cho người dùng dễ hơn nhiều. Vì mỗi phần phát triển riêng lẻ đều nhỏ hơn, bạn tránh được việc tạo ra những khối mã khổng lồ cứ tuần này qua tuần khác bị báo cáo là “hoàn thành 95%”.

Tracer Bullets Don'T Always Hit Their — Đạn vạch đường không phải lúc nào cũng trúng

Target — mục tiêu

Tracer bullets show what you're hitting. This may not always be the target. You then adjust your aim until they're on target. That's the point.

Đạn vạch đường cho thấy bạn đang bắn trúng cái gì. Cái đó có thể không phải lúc nào cũng là mục tiêu. Bạn liền chỉnh lại đường ngắm cho đến khi chúng trúng đích. Đó chính là điểm mấu chốt.

It's the same with tracer code. You use the technique in situations where you're not 100% certain of where you're going. You shouldn't be surprised if your first couple of attempts miss: the user says "that's not what I meant," or data you need isn't available when you need it, or performance problems seem likely. So change what you've got to bring it nearer the target, and be thankful that you've used a lean development methodology; a small body of code has low inertia—it is easy and quick to change. You'll be able to gather feedback on your application and generate a new, more accurate version quickly and cheaply. And because every major application component is represented in your tracer code, your users can be confident that what they're seeing is based on reality, not just a paper specification.

Với mã vạch đường cũng vậy. Bạn dùng kỹ thuật này trong những tình huống mà bạn chưa chắc 100% mình đang đi về đâu. Bạn đừng nên ngạc nhiên nếu vài lần thử đầu tiên trượt: người dùng bảo "đây không phải ý tôi", hoặc dữ liệu bạn cần lại không có sẵn vào lúc bạn cần, hoặc có vẻ sắp phát sinh vấn đề về hiệu năng. Vậy thì hãy thay đổi những gì bạn đang có để kéo nó lại gần mục tiêu hơn, và hãy mừng vì bạn đã dùng một phương pháp luận phát triển gọn nhẹ; một lượng mã nhỏ có quán tính thấp — dễ và nhanh để thay đổi. Bạn sẽ có thể thu thập phản hồi về ứng dụng và tạo ra một phiên bản mới chính xác hơn một cách nhanh chóng và rẻ tiền. Và vì mọi thành phần chính của ứng dụng đều có mặt trong mã vạch đường của bạn, người dùng có thể yên tâm rằng thứ họ đang thấy dựa trên thực tế, chứ không phải chỉ là một bản đặc tả trên giấy.

Tracer Code Versus Prototyping — Mã vạch đường so với làm bản mẫu

You might think that this tracer code concept is nothing more than prototyping under an aggressive name. There is a difference. With a **prototype**, you're aiming to explore specific aspects of the final system. With a true prototype, you will

Bạn có thể nghĩ rằng cái khái niệm mã vạch đường này chẳng qua chỉ là làm bản mẫu (prototype) với một cái tên nghe hung hăng hơn. Nhưng có một khác biệt. Với một bản mẫu, mục tiêu của bạn là khám phá những khía cạnh cụ thể của hệ thống cuối cùng. Với một bản mẫu đúng nghĩa, bạn sẽ

throw away whatever you lashed together when trying out the concept, and recode it properly using the lessons you've learned.

vứt bỏ bất cứ thứ gì bạn chắp vá lại khi đang thử nghiệm ý tưởng, rồi viết lại nó cho đàng hoàng bằng những bài học bạn đã rút ra.

For example, say you're producing an application that helps shippers determine how to pack odd-sized boxes into containers. Among other problems, the user interface needs to be intuitive and the algorithms you use to determine optimal packing are very complex.

Ví dụ, giả sử bạn đang làm một ứng dụng giúp các hãng vận chuyển xác định cách xếp những thùng có kích thước kỳ quặc vào container. Trong số nhiều vấn đề, giao diện người dùng cần phải trực quan và các thuật toán bạn dùng để tìm cách xếp tối ưu lại rất phức tạp.

You could prototype a user interface for your end users in a UI tool. You code only enough to make the interface responsive to user actions. Once they've agreed to the layout, you might throw it away and recode it, this time with the business logic behind it, using the target language. Similarly, you might want to prototype a number of algorithms that perform the actual packing. You might code functional tests in a high-level, forgiving language such as Python, and code low-level performance tests in something closer to the machine. In any case, once you'd made your decision, you'd start again and code the algorithms in their final environment, interfacing to the real world. This is prototyping, and it is very useful.

Bạn có thể làm bản mẫu giao diện người dùng cho người dùng cuối bằng một công cụ thiết kế giao diện. Bạn chỉ viết vừa đủ mã để giao diện phản hồi được các thao tác của người dùng. Một khi họ đã đồng ý với bố cục, bạn có thể vứt nó đi rồi viết lại, lần này có logic nghiệp vụ phía sau, bằng ngôn ngữ đích. Tương tự, bạn có thể muốn làm bản mẫu cho một số thuật toán thực hiện việc xếp thùng. Bạn có thể viết các bài kiểm thử chức năng bằng một ngôn ngữ cấp cao, dễ tính như Python, và viết các bài kiểm thử hiệu năng cấp thấp bằng một thứ gì đó gần với máy hơn. Dù thế nào đi nữa, một khi đã quyết định xong, bạn sẽ bắt đầu lại từ đầu và viết các thuật toán trong môi trường cuối cùng của chúng, giao tiếp với thế giới thực. Đây là làm bản mẫu, và nó rất hữu ích.

The tracer code approach addresses a different problem. You need to know how the application as a whole hangs together. You want to show your users how the interactions will work in practice, and you want to give your developers an architectural skeleton on which to hang code. In this case, you might construct a tracer consisting of a trivial implementation of the container packing algorithm (maybe something like first-come, first-served) and a simple but working user interface. Once you have all the components in the application plumbed together, you have a framework to show your users and your developers. Over time, you add to this framework with new functionality,

Cách tiếp cận mã vạch đường giải quyết một vấn đề khác. Bạn cần biết toàn bộ ứng dụng gắn kết với nhau ra sao. Bạn muốn cho người dùng thấy các tương tác sẽ hoạt động thế nào trong thực tế, và bạn muốn trao cho lập trình viên một bộ khung kiến trúc để treo mã lên đó. Trong trường hợp này, bạn có thể dựng một bản vạch đường gồm một hiện thực tầm thường của thuật toán xếp container (chẳng hạn kiểu ai đến trước phục vụ trước) và một giao diện người dùng đơn giản nhưng chạy được. Một khi đã nối thông tất cả các thành phần trong ứng dụng, bạn có một bộ khung để trình cho người dùng và lập trình viên xem. Theo thời gian, bạn bổ sung vào bộ khung này những chức năng mới,

completing stubbed routines. But the framework stays intact, and you know the system will continue to behave the way it did when your first tracer code was completed.

hoàn thiện những hàm mới chỉ là khúc xương (stub). Nhưng bộ khung vẫn nguyên vẹn, và bạn biết hệ thống sẽ tiếp tục hành xử đúng như cách nó đã hành xử khi mã vạch đường đầu tiên của bạn hoàn thành.

The distinction is important enough to warrant repeating. Prototyping generates **disposable code**. Tracer code is lean but complete, and forms part of the skeleton of the final system. Think of prototyping as the reconnaissance and intelligence gathering that takes place before a single tracer bullet is fired.

Sự phân biệt này quan trọng đến mức đáng nhắc lại. Làm bản mẫu sinh ra mã dùng một

lần rồi vớt. Mã vạch đường thì gọn nhẹ nhưng hoàn chỉnh, và tạo thành một phần bộ xương của hệ thống cuối cùng. Hãy coi việc làm bản mẫu như công tác trinh sát và thu thập tình báo diễn ra trước khi bắt đi viên đạn vạch đường đầu tiên.

Related Sections Include — Các mục liên quan

Topic 13, Prototypes and Post-it Notes

Topic 13, Bản mẫu và giấy nhớ Post-it

Topic 27, Don't Outrun Your Headlights

Topic 27, Đừng chạy nhanh hơn đèn pha

Topic 40, **Refactoring**

Topic 40, Tái cấu trúc

Topic 49, Pragmatic Teams

Topic 49, Nhóm thực dụng

Topic 50, Coconuts Don't Cut It

Topic 50, Dừa thì không ăn thua

Topic 51, Pragmatic **Starter Kit**

Topic 51, Bộ khởi đầu thực dụng

Topic 52, Delight Your Users

Topic 52, Làm người dùng vui sướng

Topic 13. Prototypes and Post-it Notes — Topic 13. Bản mẫu và Giấy ghi chú Post-it

Many industries use prototypes to try out specific ideas; prototyping is much cheaper than full-scale production. Car makers, for example, may build many different prototypes of a new car design. Each one is designed to test a specific aspect of the car—the aerodynamics, styling, structural characteristics, and so on. Old school folks might use a clay model for wind tunnel testing, maybe a balsa wood and duct tape model will do for the art department, and so on. The less romantic will do their modeling on a computer screen or in virtual reality, reducing costs even further. In this way, risky or uncertain elements can be tried out without committing to building the real item.

Nhiều ngành công nghiệp dùng bản mẫu (prototype) để thử nghiệm những ý tưởng cụ thể; làm bản mẫu rẻ hơn nhiều so với sản xuất ở quy mô đầy đủ. Chẳng hạn, các hãng xe hơi có thể dựng ra rất nhiều bản mẫu khác nhau cho một mẫu xe mới. Mỗi bản được thiết kế để kiểm tra một khía cạnh cụ thể của chiếc xe — khí động học, kiểu dáng, đặc tính kết cấu, vân vân. Dân làm theo lối cũ có thể dùng mô hình đất sét để thử trong hầm gió, một mô hình gỗ balsa dán băng keo có lẽ là đủ cho bộ phận thiết kế mỹ thuật, và cứ thế. Những người ít lãng mạn hơn sẽ dựng mô hình ngay trên màn hình máy tính hoặc trong thực tế ảo, cắt giảm chi phí hơn nữa. Bằng cách này, các yếu tố rủi ro hay chưa chắc chắn có thể được thử nghiệm mà không phải cam kết bắt tay vào dựng món đồ thật.

We build software prototypes in the same fashion, and for the same reasons—to analyze and expose risk, and to offer chances for correction at a greatly reduced cost. Like the car makers, we can target a **prototype** to test one or more specific aspects of a project.

Chúng ta xây dựng bản mẫu phần mềm theo đúng cách đó, và cũng vì những lý do đó — để phân tích và phơi bày rủi ro, đồng thời tạo cơ hội sửa chữa với chi phí rẻ hơn rất nhiều. Giống như các hãng xe, ta có thể nhắm một bản mẫu vào việc kiểm tra một hay vài khía cạnh cụ thể của dự án.

We tend to think of prototypes as code-based, but they don't always have to be. Like the car makers, we can build prototypes out of different materials. Post-it notes are great for prototyping dynamic things such as workflow and application logic. A user interface can be prototyped as a drawing on a whiteboard, as a nonfunctional mock-up drawn with a paint program, or with an interface builder.

Ta hay nghĩ bản mẫu là thứ dựa trên mã nguồn, nhưng không nhất thiết phải vậy. Giống các hãng xe, ta có thể làm bản mẫu từ nhiều chất liệu khác nhau. Giấy ghi chú Post-it rất tuyệt để làm bản mẫu cho những thứ động như luồng công việc và logic ứng dụng. Một giao diện người dùng có thể được làm bản mẫu dưới dạng bản vẽ trên bảng trắng, một mô hình tĩnh không hoạt động vẽ bằng phần mềm đồ họa, hoặc bằng một công cụ dựng giao diện.

Prototypes are designed to answer just a few questions, so they are much cheaper and faster to develop than applications that go into production. The code can ignore unimportant details—unimportant to you at the moment, but probably very important to the user later on. If you are prototyping a UI, for instance, you can get away with incorrect results or data. On the other hand, if you're just investigating computational or performance aspects, you can get away with a pretty poor UI, or perhaps even no UI at all.

Bản mẫu được thiết kế chỉ để trả lời vài câu hỏi, nên chúng rẻ và nhanh hơn nhiều so với các ứng dụng đưa vào sản xuất. Mã có thể bỏ qua những chi tiết không quan trọng — không quan trọng với bạn ở thời điểm này, nhưng có lẽ rất quan trọng với người dùng về sau. Chẳng hạn nếu bạn đang làm bản mẫu giao diện, bạn có thể chấp nhận kết quả hay dữ liệu sai. Mặt khác, nếu bạn chỉ đang khảo sát khía cạnh tính toán hay hiệu năng, bạn có thể chấp nhận một giao diện khá tệ, hoặc thậm chí chẳng cần giao diện nào cả.

But if you find yourself in an environment where you cannot give up the details, then you need to ask yourself if you are really building a prototype at all. Perhaps a tracer bullet style of development would be more appropriate in this case (see **Topic 12, Tracer Bullets**).

Nhưng nếu bạn thấy mình rơi vào tình huống không thể bỏ qua các chi tiết, thì bạn cần tự hỏi liệu mình có đang thực sự làm bản mẫu hay không. Có lẽ trong trường hợp này, lối phát triển kiểu đạn vạch đường (tracer bullets) sẽ phù hợp hơn (xem Topic 12, Đạn vạch đường).

Things To Prototype — Những thứ nên làm bản mẫu

What sorts of things might you choose to investigate with a prototype? Anything that carries risk. Anything that hasn't been tried before, or that is absolutely critical to the final system. Anything unproven, experimental, or doubtful. Anything you aren't comfortable with. You can prototype:

Bạn có thể chọn khảo sát những thứ gì bằng bản mẫu? Bất cứ thứ gì mang theo rủi ro. Bất cứ thứ gì chưa từng được thử, hoặc tuyệt đối thiết yếu với hệ thống cuối cùng. Bất cứ thứ gì chưa được kiểm chứng, mang tính thử nghiệm, hay còn đáng ngờ. Bất cứ thứ gì khiến bạn chưa yên tâm. Bạn có thể làm bản mẫu cho:

Architecture

Kiến trúc

New functionality in an existing system

Tính năng mới trong một hệ thống đã có

Structure or contents of external data

Cấu trúc hoặc nội dung của dữ liệu bên ngoài

Third-party tools or components

Công cụ hoặc thành phần của bên thứ ba

Performance issues

Các vấn đề về hiệu năng

User interface design

Thiết kế giao diện người dùng

Prototyping is a learning experience. Its value lies not in the code produced, but in the lessons learned. That's really the point of prototyping.

Làm bản mẫu là một trải nghiệm học hỏi. Giá trị của nó không nằm ở mã được tạo ra, mà nằm ở những bài học rút ra được. Đó mới thực sự là điểm cốt lõi của việc làm bản mẫu.

Tip 21: Prototype to Learn

Tip 21: Làm Bản Mẫu Để Học

How To Use Prototypes — Cách dùng bản mẫu

When building a prototype, what details can you ignore?

Khi dựng một bản mẫu, bạn có thể bỏ qua những chi tiết nào?

Correctness You may be able to use dummy data where appropriate.

Tính đúng đắn (Correctness) Bạn có thể dùng dữ liệu giả ở những chỗ phù hợp.

Completeness The prototype may function only in a very limited sense, perhaps with only one preselected piece of input data and one menu item.

Tính đầy đủ (Completeness) Bản mẫu có thể chỉ hoạt động trong một phạm vi rất hạn chế, có lẽ chỉ với một mẫu dữ liệu đầu vào được chọn sẵn và một mục menu.

Robustness Error checking is likely to be incomplete or missing entirely. If you stray from the predefined path, the prototype may crash and burn in a glorious display of pyrotechnics. That's okay.

Tính bền bỉ (Robustness) Việc kiểm tra lỗi có lẽ sẽ không đầy đủ hoặc thiếu hẳn. Nếu bạn đi lệch khỏi con đường đã định trước, bản mẫu có thể sụp đổ và bốc cháy trong một màn pháo hoa hoành tráng. Vậy cũng chẳng sao.

Style Prototype code shouldn't have much in the way of comments or documentation (although you may produce reams of documentation as a result of your experience with the prototype).

Phong cách (Style) Mã của bản mẫu không nên có nhiều chú thích hay tài liệu (mặc dù bạn có thể tạo ra cả đống tài liệu nhờ trải nghiệm với bản mẫu).

Prototypes gloss over details, and focus in on specific aspects of the system being considered, so you may want to implement them using a high-level **scripting language**—higher than the rest

Bản mẫu lướt qua các chi tiết và tập trung vào những khía cạnh cụ thể của hệ thống đang xem xét, nên bạn có thể muốn hiện thực chúng bằng một ngôn ngữ kịch bản (scripting language) ở mức cao — cao hơn so với phần còn lại

of the project (maybe a language such as Python or Ruby), as these languages can get out of your way. You may choose to continue to develop in the language used for the prototype, or you can switch; after all, you're going to throw the prototype away anyway.

của dự án (có lẽ một ngôn ngữ như Python hoặc Ruby), vì những ngôn ngữ này có thể tránh đường cho bạn. Bạn có thể chọn tiếp tục phát triển bằng ngôn ngữ đã dùng cho bản mẫu, hoặc đổi sang ngôn ngữ khác; dù sao đi nữa bạn cũng sẽ vứt bỏ bản mẫu mà.

To prototype user interfaces, use a tool that lets you focus on the appearance and/or interactions without worrying about code or markup.

Để làm bản mẫu giao diện người dùng, hãy dùng một công cụ cho phép bạn tập trung vào diện mạo và/hoặc các tương tác mà không phải lo về mã hay markup.

Scripting languages also work well as the “glue” to combine lowlevel pieces into new combinations. Using this approach, you can rapidly assemble existing components into new configurations to see how things work.

Các ngôn ngữ kịch bản cũng hoạt động tốt như chất “keo” để ghép những mẫu cấp thấp lại thành các tổ hợp mới. Theo cách này, bạn có thể nhanh chóng lắp ráp những thành phần sẵn có thành các cấu hình mới để xem mọi thứ vận hành ra sao.

Prototyping Architecture — Làm bản mẫu cho kiến trúc

Many prototypes are constructed to model the entire system under consideration. As opposed to tracer bullets, none of the individual modules in the prototype system need to be particularly functional. In fact, you may not even need to code in order to prototype architecture—you can prototype on a whiteboard, with Post-it notes or index cards. What you are looking for is how the system hangs together as a whole, again deferring details. Here are some specific areas you may want to look for in the architectural prototype:

Nhiều bản mẫu được dựng để mô hình hóa toàn bộ hệ thống đang xem xét. Khác với đạn vạch đường, không một mô-đun riêng lẻ nào trong hệ thống bản mẫu cần phải thực sự hoạt động đầy đủ. Thực ra, bạn thậm chí có thể chẳng cần viết mã để làm bản mẫu cho kiến trúc — bạn có thể làm bản mẫu trên bảng trắng, bằng giấy Post-it hay thẻ ghi chú. Thứ bạn đang tìm kiếm là cách cả hệ thống gắn kết với nhau như một tổng thể, một lần nữa tạm gác lại các chi tiết. Dưới đây là vài khía cạnh cụ thể bạn có thể muốn xem xét trong bản mẫu kiến trúc:

Are the responsibilities of the major areas well defined and appropriate?

Trách nhiệm của các mảng chính đã được định nghĩa rõ ràng và phù hợp chưa?

Are the collaborations between major components well defined?

Sự cộng tác giữa các thành phần chính đã được định nghĩa rõ ràng chưa?

Is **coupling** minimized?

Sự ghép nối (coupling) đã được giảm thiểu chưa?

Can you identify potential sources of **duplication**?

Bạn có thể nhận diện những nguồn trùng lặp tiềm tàng không?

Are interface definitions and constraints acceptable?

Các định nghĩa giao diện và ràng buộc có chấp nhận được không?

Does every module have an access path to the data it needs during execution? Does it have that access when it needs it?

Mỗi mô-đun có một đường truy cập tới dữ liệu nó cần trong lúc thực thi không? Nó có được quyền truy cập đó đúng lúc cần không?

This last item tends to generate the most surprises and the most valuable results from the prototyping experience.

Mục cuối cùng này thường tạo ra nhiều bất ngờ nhất và mang lại những kết quả giá trị nhất từ trải nghiệm làm bản mẫu.

How Not To Use Prototypes — Đừng dùng bản mẫu như thế nào

Before you embark on any code-based prototyping, make sure that everyone understands that you are writing **disposable code**. Prototypes can be deceptively attractive to people who don't know that they are just prototypes. You must make it very clear that this code is disposable, incomplete, and unable to be completed.

Trước khi bắt tay vào bất kỳ việc làm bản mẫu dựa trên mã nào, hãy chắc rằng mọi người đều hiểu bạn đang viết mã dùng-một-lần-rồi-bỏ. Bản mẫu có thể hấp dẫn một cách lừa mị với những người không biết rằng đó chỉ là bản mẫu. Bạn phải nói thật rõ rằng đoạn mã này là loại bỏ đi được, chưa hoàn chỉnh, và không thể hoàn chỉnh.

It's easy to become misled by the apparent completeness of a demonstrated prototype, and project sponsors or management may insist on deploying the prototype (or its progeny) if you don't set the right expectations. Remind them that you can build a great prototype of a new car out of balsa wood and duct tape, but you wouldn't try to drive it in rush-hour traffic!

Người ta dễ bị đánh lừa bởi vẻ ngoài có vẻ hoàn chỉnh của một bản mẫu được trình diễn, và nhà tài trợ dự án hay ban quản lý có thể khăng khăng đòi triển khai bản mẫu (hoặc hậu duệ của nó) nếu bạn không đặt ra kỳ vọng đúng đắn. Hãy nhắc họ rằng bạn có thể dựng một bản mẫu tuyệt vời của chiếc xe mới bằng gỗ balsa và băng keo, nhưng bạn sẽ chẳng dám lái nó vào dòng xe giờ cao điểm đâu!

If you feel there is a strong possibility in your environment or culture that the purpose of prototype code may be misinterpreted, you may be better off with the tracer bullet approach. You'll end up with a solid framework on which to base future development.

Nếu bạn cảm thấy trong môi trường hay văn hóa của mình có khả năng cao là mục đích của mã bản mẫu sẽ bị hiểu sai, thì có lẽ bạn nên chọn lối đạn vạch đường thì hơn. Bạn sẽ có một khung sườn vững chắc làm nền cho việc phát triển về sau.

Properly used prototypes can save you huge amounts of time, money, and pain by identifying and correcting potential problem spots early in the development cycle—the time when fixing mistakes is both cheap and easy.

Được dùng đúng cách, bản mẫu có thể tiết kiệm cho bạn lượng lớn thời gian, tiền bạc và phiền toái nhờ nhận diện và sửa chữa những điểm có thể gặp vấn đề ngay từ sớm trong chu kỳ phát triển — thời điểm mà việc sửa sai vừa rẻ vừa dễ.

Related Sections Include — Các mục liên quan

Topic 12, Tracer Bullets

[Topic 12, Đạn vạch đường](#)

Topic 14, Domain Languages

[Topic 14, Ngôn ngữ miền](#)

Topic 17, **Shell** Games

[Topic 17, Trò chơi Shell](#)

Topic 27, Don't Outrun Your Headlights

[Topic 27, Đừng vượt quá tầm đèn pha](#)

Topic 37, Listen to Your Lizard Brain

[Topic 37, Hãy lắng nghe Não bò sát của bạn](#)

Topic 45, The **Requirements** Pit

[Topic 45, Cái hố Yêu cầu](#)

Topic 52, Delight Your Users

[Topic 52, Làm người dùng vui sướng](#)

Exercises — Bài tập

Exercise 3 (possible answer)

[Bài tập 3 \(đáp án khả dĩ\)](#)

Marketing would like to sit down and brainstorm a few web page designs with you. They are thinking of clickable image maps to take you to other pages, and so on. But they can't decide on a model for the image—maybe it's a car, or a phone, or a house. You have a list of target pages and content; they'd like to see a few prototypes. Oh, by the way, you have 15 minutes. What tools might you use?

Bộ phận tiếp thị muốn ngồi xuống cùng bạn động não vài thiết kế trang web. Họ đang nghĩ tới các bản đồ ảnh có thể nhấp vào để đưa bạn sang những trang khác, và đại loại vậy. Nhưng họ chưa quyết được mô hình cho hình ảnh — có thể là chiếc xe, hoặc cái điện thoại, hoặc ngôi nhà. Bạn có một danh sách các trang đích và nội dung; họ muốn xem vài bản mẫu. À mà tiện thể, bạn có 15 phút. Bạn sẽ dùng những công cụ nào?

The limits of language are the limits of one's world. Ludwig Wittgenstein

Giới hạn của ngôn ngữ là giới hạn của thế giới một con người. Ludwig Wittgenstein

Topic 14. Domain Languages — Topic 14. Ngôn ngữ miền

Computer languages influence how you think about a problem, and how you think about communicating. Every language comes with a list of features: buzzwords such as static versus dynamic typing, early versus late binding, functional versus OO, **inheritance** models, mixins, macros—all of which may suggest or obscure certain solutions. Designing a solution with C++ in mind will produce different results than a solution based on Haskell-style thinking, and vice versa.

Conversely, and we think more importantly, the language of the problem domain may also suggest a programming solution.

Ngôn ngữ máy tính ảnh hưởng đến cách bạn suy nghĩ về một vấn đề, và cách bạn nghĩ về việc giao tiếp. Mỗi ngôn ngữ đi kèm một danh sách đặc tính: những thuật ngữ thời thượng như định kiểu tĩnh so với động, ràng buộc sớm so với ràng buộc muộn, hướng hàm so với hướng đối tượng, các mô hình kế thừa (inheritance), mixin, macro—tất cả đều có thể gợi mở hoặc che khuất một số giải pháp nhất định. Thiết kế một lời giải với tư duy C++ trong đầu sẽ cho ra kết quả khác hẳn với lời giải dựa trên lối nghĩ kiểu Haskell, và ngược lại. Ngược lại nữa, mà theo chúng tôi điều này quan trọng hơn, chính ngôn ngữ của miền vấn đề (problem domain) cũng có thể gợi ý một giải pháp lập trình.

We always try to write code using the vocabulary of the application domain (see Maintain a Glossary). In some cases, **Pragmatic** Programmers can go to the next level and actually program using the vocabulary, syntax, and semantics—the language—of the domain.

Chúng tôi luôn cố gắng viết mã bằng từ vựng của miền ứng dụng (xem Duy trì một bảng thuật ngữ). Trong một số trường hợp, các Lập trình viên Thực dụng (Pragmatic Programmer) có thể tiến lên một tầm cao mới và thực sự lập trình bằng từ vựng, cú pháp và ngữ nghĩa—tức là ngôn ngữ—của chính miền đó.

Tip 22: Program Close to the Problem Domain

Tip 22: Hãy lập trình sát với miền vấn đề

Some Real-World Domain Languages — Một vài ngôn ngữ miền trong thực tế

Let's look at a few examples where folks have done just that.

Hãy cùng xem vài ví dụ nơi người ta đã làm đúng như vậy.

RSpec RSpec [19] is a testing library for Ruby. It inspired versions for most other modern languages. A test in RSpec is intended to reflect the behavior you expect from your code.

RSpec — RSpec [19] là một thư viện kiểm thử dành cho Ruby. Nó đã truyền cảm hứng cho những phiên bản tương tự ở hầu hết các ngôn ngữ hiện đại khác. Một bài test trong RSpec được viết ra để phản chiếu đúng cái hành vi mà bạn kỳ vọng ở mã của mình.

```
describe BowlingScore do it "totals 12 if you score 3 four times" do score = BowlingScore.new 4.times { score.add_pins(3) } expect(score.total).to eq(12) end end
```

```
describe BowlingScore do it "totals 12 if you score 3 four times" do score = BowlingScore.new 4.times { score.add_pins(3) } expect(score.total).to eq(12) end end
```

Cucumber Cucumber [20] is programming-language neutral way of specifying tests. You run the tests using a version of Cucumber appropriate to the language you're using. In order to support the natural-language like syntax, you also have to write specific matchers that recognize phrases and extract parameters for the tests.

Cucumber — Cucumber [20] là một cách đặc tả các bài kiểm thử độc lập với ngôn ngữ lập trình. Bạn chạy các bài kiểm thử ấy bằng một phiên bản Cucumber phù hợp với ngôn ngữ mình đang dùng. Để đỡ được cái cú pháp trông như ngôn ngữ tự nhiên kia, bạn còn phải viết thêm những bộ so khớp (matcher) chuyên biệt để nhận ra các cụm từ và bóc tham số ra cho bài kiểm thử.

Feature : Scoring

Feature : Scoring

Background : Given an empty scorecard

Background : Given an empty scorecard

Scenario : bowling a lot of 3s Given I throw a 3 And I throw a 3 And I throw a 3 And I throw a 3 Then the score should be 12 Cucumber tests were intended to be read by the customers of the software (although that happens fairly rarely in practice; the following aside considers why that might be).

Scenario : bowling a lot of 3s Given I throw a 3 And I throw a 3 And I throw a 3 And I throw a 3 Then the score should be 12 — Các test Cucumber vốn được thiết kế để khách hàng của phần mềm đọc được (mặc dù trên thực tế điều đó hiếm khi xảy ra; phần luận bàn bên dưới sẽ xét xem tại sao lại thế).

Why Don't Many Business Users Read Cucumber Features?

Vì sao ít người dùng nghiệp vụ chịu đọc các Feature của Cucumber?

One of the reasons that the classic gather **requirements**, design, code, ship approach doesn't work is that it is anchored by the concept that we know what the requirements are. But we rarely do. Your business users will have a vague idea of what they want to achieve, but they neither know nor care about the details. That's part of our value: we intuit intent and convert it to code.

Một trong những lý do khiến cách tiếp cận kinh điển “thu thập yêu cầu, thiết kế, viết mã, giao hàng” không hiệu quả là vì nó neo vào niềm tin rằng chúng ta biết các yêu cầu là gì. Nhưng chúng ta hiếm khi biết. Người dùng nghiệp vụ của bạn sẽ chỉ có một ý niệm mơ hồ về điều họ muốn đạt được, còn các chi tiết thì họ vừa không biết vừa chẳng quan tâm. Đó chính là một phần giá trị của chúng ta: chúng ta cảm nhận ý định và chuyển hóa nó thành mã.

So when you force a business person to sign off on a requirements document, or get them to agree to a set of Cucumber features, you're doing the equivalent of getting them to check the spelling in an essay written in Sumerian. They'll make some random changes to save face and sign it off to get you out of their office.

Cho nên khi bạn ép một người làm nghiệp vụ ký duyệt một tài liệu yêu cầu, hay buộc họ đồng ý với một bộ feature Cucumber, thì bạn đang làm điều tương đương với việc bắt họ soát lỗi chính tả trong một bài luận viết bằng tiếng Sumer cổ. Họ sẽ sửa vài chỗ ngẫu nhiên cho đỡ mất mặt rồi ký duyệt cho xong để tổng bạn ra khỏi phòng.

Give them code that runs, however, and they can play with it. That's where their real needs will surface.

Thế nhưng, hãy đưa cho họ mã chạy được, và họ có thể nghịch với nó. Đó mới chính là nơi nhu cầu thực sự của họ lộ diện.

Phoenix Routes Many web frameworks have a routing facility, mapping incoming HTTP requests onto handler functions in the code. Here's an example from Phoenix. [21]

Routes của Phoenix — Nhiều web framework có sẵn cơ chế định tuyến (routing), ánh xạ các yêu cầu HTTP đi vào tới những hàm xử lý trong mã. Dưới đây là một ví dụ lấy từ Phoenix. [21]

```
scope "/" , HelloPhoenix do pipe_through :browser # Use the default browser stack
```

```
  scope "/" , HelloPhoenix do pipe_through :browser # Use the default browser stack
```

```
  get "/" , PageController, :index resources "/users" , UserController end
```

```
  get "/" , PageController, :index resources "/users" , UserController end
```

This says that requests starting “/” will be run through a series of filters appropriate for browsers. A request to “/” itself will be handled by the index function in the PageController module. The UsersController implements the functions needed to manage a resource accessible via the url /users

Đoạn này nói rằng những yêu cầu bắt đầu bằng “/” sẽ được cho chạy qua một loạt bộ lọc phù hợp với trình duyệt. Còn bản thân một yêu cầu tới “/” sẽ do hàm index trong module PageController xử lý. UsersController hiện thực những hàm cần thiết để quản lý một tài nguyên truy cập được qua url /users.

Ansible Ansible [22] is a tool that configures software, typically on a bunch of remote servers. It does this by reading a specification that you provide, then doing whatever is needed on the servers to make them mirror that spec. The specification can be written in YAML, [23] a language that builds data structures from text

Ansible — Ansible [22] là một công cụ dùng để cấu hình phần mềm, thường là trên một mớ máy chủ từ xa. Nó làm việc đó bằng cách đọc bản đặc tả mà bạn cung cấp, rồi làm bất cứ điều gì cần thiết trên các máy chủ để chúng phản chiếu đúng bản đặc tả ấy. Bản đặc tả có thể được viết bằng YAML, [23] một ngôn ngữ dựng nên các cấu trúc dữ liệu từ những mô tả

descriptions:

bằng văn bản:

— - name: install nginx apt: name=nginx **state**=latest

— - name: install nginx apt: name=nginx state=latest

- name: ensure nginx is running (and enable it at boot) service: name=nginx state=started enabled=yes
 - name: ensure nginx is running (and enable it at boot) service: name=nginx state=started enabled=yes
- name: write the nginx config file template: src=templates/nginx.conf.j2 dest=/etc/nginx/nginx.conf notify: - restart nginx
 - name: write the nginx config file template: src=templates/nginx.conf.j2 dest=/etc/nginx/nginx.conf notify: - restart nginx

This example ensures that the latest version of nginx is installed on my servers, that it is started by default, and that it uses a **configuration** file that you’ve provided.

Ví dụ này đảm bảo rằng phiên bản nginx mới nhất được cài trên các máy chủ của tôi, rằng nó được khởi động mặc định, và rằng nó dùng đúng tệp cấu hình mà bạn đã cung cấp.

Characteristics Of Domain Languages — Đặc điểm của các ngôn ngữ miền

Let’s look at these examples more closely.

Hãy cùng xem các ví dụ trên kỹ hơn một chút.

RSpec and the Phoenix **router** are written in their host languages (Ruby and Elixir). They employ some fairly devious code, including **metaprogramming** and macros, but ultimately they are compiled and run as regular code.

RSpec và bộ định tuyến của Phoenix được viết bằng chính ngôn ngữ chủ (host language) của chúng (Ruby và Elixir). Chúng dùng tới một số đoạn mã khá ranh ma, bao gồm cả siêu

lập trình (metaprogramming) và macro, nhưng rốt cuộc chúng vẫn được biên dịch và chạy như mã thông thường.

Cucumber tests and Ansible configurations are written in their own languages. A Cucumber test is converted into code to be run or into a datastructure, whereas Ansible specs are always converted into a data structure that is run by Ansible itself.

Test Cucumber và cấu hình Ansible thì được viết bằng ngôn ngữ riêng của chúng. Một test Cucumber được chuyển hóa thành mã để chạy hoặc thành một cấu trúc dữ liệu, trong khi đặc tả Ansible thì luôn được chuyển thành một cấu trúc dữ liệu do chính Ansible chạy.

As a result, RSpec and the router code are embedded into the code you run: they are true extensions to your code's vocabulary. Cucumber and Ansible are read by code and converted into some form the code can use.

Kết quả là, RSpec và mã định tuyến được nhúng thẳng vào mã mà bạn chạy: chúng là phần mở rộng thực sự cho từ vựng của mã bạn. Còn Cucumber và Ansible thì được mã đọc vào rồi chuyển sang một dạng nào đó mà mã có thể dùng.

We call RSpec and the router examples of **internal** domain languages, while Cucumber and Ansible use external languages.

Chúng tôi gọi RSpec và bộ định tuyến là các ví dụ về ngôn ngữ miền (domain language) nội bộ, còn Cucumber và Ansible thì dùng ngôn ngữ bên ngoài.

Trade-Offs Between Internal And External — Đánh đổi giữa ngôn ngữ nội bộ và ngôn ngữ

Languages — bên ngoài

In general, an internal **domain language** can take advantage of the features of its **host language**: the domain language you create is more powerful, and that power comes for free. For example, you could use some Ruby code to create a bunch of RSpec tests automatically. In this case we can test scores where there are no spares or strikes:

Nhìn chung, một ngôn ngữ miền nội bộ có thể tận dụng các đặc tính của ngôn ngữ chủ: ngôn ngữ miền mà bạn tạo ra trở nên mạnh mẽ hơn, và sức mạnh đó đến mà chẳng tốn công gì thêm. Chẳng hạn, bạn có thể dùng một ít mã Ruby để tự động sinh ra cả đồng test RSpec. Trong trường hợp này, ta có thể kiểm thử các điểm số mà không có lượt spare hay strike nào:

```
describe BowlingScore do (0..4).each do |pins| (1..20).each do |throws| target = pins * throws
```

```
  describe BowlingScore do (0..4).each do |pins| (1..20).each do |throws| target = pins * throws
```

```
    it "totals #{ target } if you score #{ pins } #{ throws } times" do score = BowlingScore.new throws.times { score.add_pins(pins) } expect(score.total).to eq(target) end end end end That's 100 tests you just wrote. Take the rest of the day off.
```

```
    it "totals #{ target } if you score #{ pins } #{ throws } times" do score = BowlingScore.new throws.times { score.add_pins(pins) } expect(score.total).to eq(target) end end end end —  
    Bạn vừa viết xong 100 bài test đấy. Nghỉ ngơi nốt phần còn lại của ngày đi.
```

The downside of internal domain languages is that you're bound by the syntax and semantics of that language. Although some languages are remarkably flexible in this regards, you're still forced to compromise between the language you want and the language you can implement.

Mặt trái của ngôn ngữ miền nội bộ là bạn bị trói buộc bởi cú pháp và ngữ nghĩa của ngôn ngữ đó. Dù một số ngôn ngữ linh hoạt đến mức đáng kinh ngạc về khoản này, bạn vẫn buộc phải thỏa hiệp giữa ngôn ngữ mà bạn muốn và ngôn ngữ mà bạn có thể hiện thực được.

Ultimately, whatever you come up with must still be valid syntax in your target language. Languages with macros (such as

Suy cho cùng, bất cứ thứ gì bạn nghĩ ra cũng vẫn phải là cú pháp hợp lệ trong ngôn ngữ đích. Các ngôn ngữ có macro (như

Elixir, Clojure, and Crystal) gives you a little more flexibility, but ultimately syntax is syntax.

Elixir, Clojure và Crystal) cho bạn thêm chút linh hoạt, nhưng rốt cuộc cú pháp vẫn là cú pháp.

External languages have no such restrictions. As long as you can write a **parser** for the language, you're good to go. Sometimes you can use someone else's parser (as Ansible did by using YAML), but then you're back to making a compromise.

Ngôn ngữ bên ngoài thì không bị những ràng buộc như vậy. Miễn là bạn viết được một bộ phân tích cú pháp (parser) cho ngôn ngữ đó, là bạn sẵn sàng lên đường. Đôi khi bạn có thể dùng parser của người khác (như Ansible đã làm khi mượn YAML), nhưng khi đó bạn lại quay về với chuyện phải thỏa hiệp.

Writing a parser probably means adding new libraries and possibly tools to your application. And writing a good parser is not a trivial job. But, if you're feeling stout of heart, you could look at parser generators such as bison or ANTLR, and parsing frameworks such as the many PEG parsers out there.

Viết một parser nhiều khả năng đồng nghĩa với việc thêm các thư viện mới và có thể cả công cụ mới vào ứng dụng của bạn. Mà viết được một parser tốt thì chẳng phải việc tầm thường. Nhưng nếu thấy mình gan dạ, bạn có thể ngó qua các bộ sinh parser như bison hay ANTLR, và các framework phân tích cú pháp như vô số bộ parser PEG đang có ngoài kia.

Our suggestion is fairly simple: don't spend more effort than you save. Writing a domain language adds some cost to your project, and you'll need to be convinced that there are offsetting savings (potentially in the long term).

Gợi ý của chúng tôi khá đơn giản: đừng bỏ ra nhiều công sức hơn số công bạn tiết kiệm được. Viết một ngôn ngữ miền sẽ thêm chút chi phí cho dự án, và bạn cần được thuyết phục rằng có những khoản tiết kiệm bù lại (tiềm năng là về dài hạn).

In general, use off-the-shelf external languages (such as YAML, JSON, or CSV) if you can. If not, look at internal languages. We'd recommend using external languages only in cases where your language will be written by the users of your application.

Nhìn chung, hãy dùng các ngôn ngữ bên ngoài có sẵn (như YAML, JSON hay CSV) nếu có thể. Nếu không, hãy xét đến ngôn ngữ nội bộ. Chúng tôi khuyên chỉ nên dùng ngôn ngữ bên ngoài trong những trường hợp ngôn ngữ của bạn sẽ do chính người dùng ứng dụng của bạn viết ra.

An Internal Domain Language On The Cheap — Một ngôn ngữ miền nội bộ kiểu giá rẻ

Finally, there's a cheat for creating internal domain languages if you don't mind the host language syntax leaking through. Don't do a bunch of metaprogramming. Instead, just write functions to do the work. In fact, this is pretty much what RSpec does:

Cuối cùng, có một mảnh để tạo ngôn ngữ miền nội bộ nếu bạn không ngại để cú pháp của ngôn ngữ chủ lộ ra. Đùng sa vào một mớ siêu lập trình. Thay vào đó, cứ viết các hàm để làm việc. Thực ra, đó gần như chính là những gì RSpec làm:

```
describe BowlingScore do it "totals 12 if you score 3 four times" do score = BowlingScore.new 4.times { score.add_pins(3) }
```

```
  describe BowlingScore do it "totals 12 if you score 3 four times" do score = BowlingScore.new 4.times { score.add_pins(3) }
```

```
    expect(score.total).to eq(12) end end
```

```
      expect(score.total).to eq(12) end end
```

In this code, `describe`, `it`, `expect`, `to`, and `eq` are just Ruby methods. There's a little plumbing behind the scenes in terms of how objects are passed around, but it's all just code. We'll explore that a little in the **exercises**.

Trong đoạn mã này, `describe`, `it`, `expect`, `to` và `eq` chỉ đơn thuần là các phương thức Ruby. Có một chút “đường ống” chạy ở hậu trường, liên quan đến cách các đối tượng được chuyển qua lại, nhưng rốt cuộc tất cả cũng chỉ là mã mà thôi. Chúng ta sẽ khám phá điều đó một chút ở phần bài tập.

Related Sections Include — Các mục liên quan

Topic 8, The Essence of Good Design

Topic 8, Tinh túy của thiết kế tốt

Topic 13, Prototypes and Post-it Notes

Topic 13, Bản mẫu và giấy ghi nhớ Post-it

Topic 32, Configuration

Topic 32, Cấu hình

Challenges — Thử thách

Could some of the requirements of your current project be expressed in a **domain-specific language**? Would it be possible to write a compiler or translator that could generate most of the code required?

Liệu một số yêu cầu của dự án hiện tại của bạn có thể được diễn đạt bằng một ngôn ngữ chuyên biệt cho miền (domain-specific language) không? Liệu có khả thi để viết một trình biên dịch hay trình chuyển đổi có thể sinh ra phần lớn mã cần thiết không?

If you decide to adopt mini-languages as a way of programming closer to the problem domain, you're accepting that some effort will be required to implement them. Can you see ways in which the framework you develop for one project can be reused in others?

Nếu bạn quyết định áp dụng các ngôn ngữ thu nhỏ (mini-language) như một cách lập trình sát hơn với miền vấn đề, bạn đang chấp nhận rằng sẽ cần bỏ ra một số công sức để hiện thực chúng. Bạn có thấy được những cách nào để khung sườn (framework) bạn phát triển cho một dự án có thể tái sử dụng ở các dự án khác không?

Exercises — Bài tập

Exercise 4 (possible answer)

Bài tập 4 (lời giải khả dĩ)

We want to implement a **mini-language** to control a simple turtle-graphics system. The language consists of single-letter commands, some followed by a single number. For example, the following input would draw a rectangle:

Chúng ta muốn hiện thực một ngôn ngữ thu nhỏ để điều khiển một hệ đồ họa rùa (turtle-graphics) đơn giản. Ngôn ngữ này gồm các lệnh một chữ cái, một số lệnh theo sau là một con số duy nhất. Ví dụ, đầu vào sau đây sẽ vẽ một hình chữ nhật:

P 2 # select pen 2 D # pen down W 2 # draw west 2cm N 1 # then north 1 E 2 # then east 2 S 1 # then back south U # pen up

P 2 # select pen 2 D # pen down W 2 # draw west 2cm N 1 # then north 1 E 2 # then east 2 S 1 # then back south U # pen up

Implement the code that parses this language. It should be designed so that it is simple to add new commands.

Hãy hiện thực đoạn mã phân tích cú pháp ngôn ngữ này. Nó nên được thiết kế sao cho việc thêm các lệnh mới là đơn giản.

Exercise 5 (possible answer)

Bài tập 5 (lời giải khả dĩ)

In the previous exercise we implemented a parser for the drawing language—it was an **external domain language**. Now implement it again as an internal language. Don't do anything clever: just write a function for each of the commands. You may have to change the names of the commands to lower case, and maybe to wrap them inside something to provide some **context**.

Ở bài tập trước, chúng ta đã hiện thực một parser cho ngôn ngữ vẽ—đó là một ngôn ngữ miền bên ngoài. Bây giờ hãy hiện thực lại nó dưới dạng một ngôn ngữ nội bộ. Đừng làm gì cao siêu: cứ viết một hàm cho mỗi lệnh. Bạn có thể phải đổi tên các lệnh sang chữ thường, và có lẽ phải gói chúng vào bên trong cái gì đó để cung cấp một chút ngữ cảnh.

Exercise 6 (possible answer)

Bài tập 6 (lời giải khả dĩ)

Design a BNF grammar to parse a time specification. All of the following examples should be accepted:

Hãy thiết kế một văn phạm BNF để phân tích một đặc tả thời gian. Tất cả các ví dụ sau đây đều phải được chấp nhận:

4pm, 7:38pm, 23:42, 3:16, 3:16am Exercise 7 (possible answer)

4pm, 7:38pm, 23:42, 3:16, 3:16am — Bài tập 7 (lời giải khả dĩ)

Implement a parser for the BNF grammar in the previous exercise using a PEG parser generator in the language of your choice. The output should be an integer containing the number of minutes past midnight.

Hãy hiện thực một parser cho văn phạm BNF ở bài tập trước, dùng một bộ sinh parser PEG bằng ngôn ngữ bạn chọn. Đầu ra phải là một số nguyên chứa số phút tính từ nửa đêm.

Exercise 8 (possible answer)

Bài tập 8 (lời giải khả dĩ)

Implement the time parser using a **scripting language** and regular expressions.

Hãy hiện thực bộ phân tích thời gian bằng một ngôn ngữ kịch bản (scripting language) và biểu thức chính quy (regular expression).

Topic 15. Estimating — Topic 15. Ước lượng

The Library of Congress in Washington, DC, currently has about 75 terabytes of digital information online. Quick! How long will it take to send all that information over a 1Gbps network? How much storage will you need for a million names and addresses? How long does it take to compress 100Mb of text? How many months will it take to deliver your project?

Thư viện Quốc hội Hoa Kỳ ở Washington, DC hiện có khoảng 75 terabyte thông tin số đặt trực tuyến. Nhanh nào! Sẽ mất bao lâu để gửi toàn bộ chỗ thông tin đó qua một đường mạng 1Gbps? Bạn cần bao nhiêu dung lượng lưu trữ cho một triệu cái tên và địa chỉ? Mất bao lâu để nén 100Mb văn bản? Còn bao nhiêu tháng nữa thì dự án của bạn được bàn giao?

At one level, these are all meaningless questions—they are all missing information. And yet they can all be answered, as long as you are comfortable **estimating**. And, in the **process** of producing an estimate, you'll come to understand more about the world your programs inhabit.

Ở một góc nhìn nào đó, đây đều là những câu hỏi vô nghĩa — tất cả đều thiếu thông tin. Vậy mà chúng đều có thể trả lời được, miễn là bạn thấy thoải mái với việc ước lượng (estimating). Và trong quá trình tạo ra một con số ước lượng, bạn sẽ hiểu thêm về cái thế giới mà những chương trình của bạn đang sống trong đó.

By learning to estimate, and by developing this skill to the point where you have an intuitive feel for the magnitudes of things, you will be able to show an apparent magical ability to determine their feasibility. When someone says “we'll send the backup over a network connection to S3,” you'll be able to know intuitively whether this is practical. When you're coding, you'll be able to know which subsystems need optimizing and which ones can be left alone.

Bằng cách học ước lượng, và rèn kỹ năng này tới mức bạn có một cảm giác trực giác về độ lớn của mọi thứ, bạn sẽ thể hiện được một khả năng tưởng như có phép màu: phán đoán xem một việc có khả thi hay không. Khi ai đó nói “ta sẽ gửi bản sao lưu qua mạng lên S3”, bạn sẽ biết ngay theo bản năng liệu điều này có thực tế không. Khi đang viết code, bạn sẽ biết hệ thống con nào cần tối ưu và hệ thống con nào thì cứ để yên đó cũng được.

Tip 23: Estimate to Avoid Surprises

Tip 23: Hãy ước lượng để tránh bất ngờ

As a bonus, at the end of this section we'll reveal the single correct answer to give whenever anyone asks you for an estimate.

Thêm một phần thưởng nữa: ở cuối mục này, chúng tôi sẽ tiết lộ câu trả lời đúng duy nhất mà bạn nên đưa ra mỗi khi có người hỏi bạn một con số ước lượng.

How Accurate Is Accurate Enough? — Chính xác đến mức nào thì đủ chính xác?

To some extent, all answers are estimates. It's just that some are more accurate than others. So the first question you have to ask yourself when someone asks you for an estimate is the **context** in which your answer will be taken. Do they need high accuracy, or are they looking for a ballpark figure?

Ở một mức độ nào đó, mọi câu trả lời đều là ước lượng. Chỉ là có cái chính xác hơn cái khác mà thôi. Vậy nên câu hỏi đầu tiên bạn phải tự hỏi khi ai đó muốn bạn ước lượng là: câu trả lời của bạn sẽ được hiểu trong bối cảnh nào? Họ cần độ chính xác cao, hay chỉ tìm một con số áng chừng?

One of the interesting things about estimating is that the units you use make a difference in the interpretation of the result. If you say that something will take about 130 working days, then people will be expecting it to come in pretty close. However, if you say “Oh, about six months,” then they know to look for it any time between five and seven months from now. Both numbers represent the same duration, but “130 days” probably implies a higher degree of accuracy than you feel. We recommend that you scale time estimates as follows:

Một điều thú vị về ước lượng là đơn vị bạn dùng sẽ làm thay đổi cách người ta diễn giải kết quả. Nếu bạn nói một việc gì đó sẽ mất khoảng 130 ngày làm việc, người ta sẽ kỳ vọng nó về đích khá sát con số ấy. Còn nếu bạn nói “Ồ, khoảng sáu tháng”, thì họ hiểu là có thể trông chờ nó vào bất cứ lúc nào trong khoảng từ năm đến bảy tháng tới. Cả hai con số biểu thị cùng một khoảng thời gian, nhưng “130 ngày” có lẽ ngụ ý một độ chính xác cao hơn mức bạn thực sự cảm thấy. Chúng tôi khuyên bạn nên co giãn các ước lượng thời gian như sau:

Duration Quote estimate in

Khoảng thời gian — Nêu ước lượng theo

1–15 days Days

1–15 ngày — Ngày

3–6 weeks Weeks

3–6 tuần — Tuần

8–20 weeks Months

8–20 tuần — Tháng

20+ weeks Think hard before giving an estimate

Trên 20 tuần — Cân nhắc thật kỹ trước khi đưa ra ước lượng

So, if after doing all the necessary work, you decide that a project will take 125 working days (25 weeks), you might want

Vậy nên, sau khi đã làm hết mọi việc cần thiết, nếu bạn quyết định rằng một dự án sẽ mất 125 ngày làm việc (25 tuần), bạn có lẽ nên

to deliver an estimate of “about six months.”

đưa ra ước lượng là “khoảng sáu tháng”.

The same concepts apply to estimates of any quantity: choose the units of your answer to reflect the accuracy you intend to convey.

Cùng những khái niệm này áp dụng cho việc ước lượng bất kỳ đại lượng nào: hãy chọn đơn vị cho câu trả lời sao cho phản ánh đúng độ chính xác mà bạn muốn truyền đạt.

Where Do Estimates Come From? — Các con số ước lượng đến từ đâu?

All estimates are based on models of the problem. But before we get too deeply into the techniques of building models, we have to mention a basic estimating trick that always gives good answers: ask someone who's already done it. Before you get too committed to model building, cast around for someone who's been in a similar situation in the past. See how their problem got solved. It's unlikely you'll ever find an exact match, but you'd be surprised how many times you can successfully draw on others' experiences.

Mọi ước lượng đều dựa trên các mô hình của bài toán. Nhưng trước khi đi quá sâu vào kỹ thuật dựng mô hình, chúng tôi phải nhắc đến một mẹo ước lượng cơ bản luôn cho câu trả lời tốt: hỏi người đã từng làm việc đó. Trước khi bạn lao vào dựng mô hình, hãy ngó quanh tìm ai đó từng ở trong tình huống tương tự trong quá khứ. Xem họ đã giải quyết vấn đề ra sao. Chẳng mấy khi bạn tìm được một trường hợp khớp hệt, nhưng bạn sẽ ngạc nhiên về số lần mình có thể tận dụng thành công kinh nghiệm của người khác.

Understand What's Being Asked The first part of any **estimation** exercise is building an understanding of what's being asked. As well as the accuracy issues discussed above, you need to have a grasp of the scope of the domain. Often this is implicit in the question, but you need to make it a habit to think about the scope before starting to guess. Often, the scope you choose will form part of the answer you give: “Assuming there are no traffic accidents and there's gas in the car, I should be there in 20 minutes.”

Hiểu rõ người ta đang hỏi gì. Phần đầu tiên của bất kỳ bài tập ước lượng nào là xây dựng sự hiểu biết về điều đang được hỏi. Ngoài chuyện độ chính xác đã bàn ở trên, bạn cần nắm được phạm vi của miền vấn đề. Thường thì điều này nằm ẩn trong câu hỏi, nhưng bạn cần tập thói quen nghĩ về phạm vi trước khi bắt đầu đoán. Phạm vi bạn chọn thường sẽ là một phần của câu trả lời bạn đưa ra: “Giả sử không có tai nạn giao thông và xe còn xăng, tôi sẽ tới đó trong 20 phút.”

Build a Model of the System This is the fun part of estimating. From your understanding of the question being asked, build a rough-and-ready bare-bones mental model. If you're estimating response times, your model may involve a server and some kind of arriving traffic. For a project, the model may be the steps that your organization uses

Dựng một mô hình của hệ thống. Đây là phần vui của việc ước lượng. Từ chỗ hiểu được câu hỏi đang đặt ra, hãy dựng một mô hình tinh thần thô sơ, gọn ghẽ tới mức trơ xương. Nếu đang ước lượng thời gian phản hồi, mô hình của bạn có thể gồm một máy chủ và một dòng lưu lượng (traffic) đang ủa tới. Với một dự án, mô hình có thể là các bước mà tổ chức của bạn áp dụng

during development, along with a very rough picture of how the system might be implemented.

trong quá trình phát triển, kèm theo một bức tranh rất sơ lược về cách hệ thống có thể được hiện thực.

Model building can be both creative and useful in the long term. Often, the process of building the model leads to discoveries of underlying patterns and processes that weren't apparent on the surface. You may even want to reexamine the original question: "You asked for an estimate to do X. However, it looks like Y, a variant of X, could be done in about half the time, and you lose only one feature."

Việc dựng mô hình có thể vừa sáng tạo vừa hữu ích về lâu dài. Thường thì quá trình dựng mô hình dẫn tới những khám phá về các mẫu hình (pattern) và tiến trình ẩn bên dưới mà nhìn bề ngoài không thấy. Bạn thậm chí có thể muốn xem lại câu hỏi ban đầu: "Bạn yêu cầu ước lượng để làm X. Tuy nhiên, có vẻ Y, một biến thể của X, có thể làm xong chỉ trong khoảng một nửa thời gian, mà bạn chỉ mất đúng một tính năng."

Building the model introduces inaccuracies into the estimating process. This is inevitable, and also beneficial. You are trading off model simplicity for accuracy. Doubling the effort on the model may give you only a slight increase in accuracy. Your experience will tell you when to stop refining.

Việc dựng mô hình đưa các sai số vào quá trình ước lượng. Điều này là không thể tránh khỏi, và cũng có lợi. Bạn đang đánh đổi sự đơn giản của mô hình lấy độ chính xác. Bỏ gấp đôi công sức vào mô hình có thể chỉ đem lại một chút xíu chính xác hơn. Kinh nghiệm sẽ mách bạn khi nào nên ngừng tinh chỉnh.

Break the Model into Components Once you have a model, you can decompose it into components. You'll need to discover the mathematical rules that describe how these components interact. Sometimes a component contributes a single value that is added into the result. Some components may supply multiplying factors, while others may be more complicated (such as those that simulate the arrival of traffic at a node).

Chia mô hình thành các thành phần. Một khi đã có mô hình, bạn có thể phân rã nó thành các thành phần. Bạn sẽ cần tìm ra các quy tắc toán học mô tả cách những thành phần này tương tác với nhau. Đôi khi một thành phần đóng góp một giá trị đơn lẻ được cộng vào kết quả. Một số thành phần có thể cung cấp các hệ số nhân, trong khi những thành phần khác có thể phức tạp hơn (chẳng hạn loại mô phỏng lưu lượng ủa tới một nút mạng).

You'll find that each component will typically have parameters that affect how it contributes to the overall model. At this stage, simply identify each parameter.

Bạn sẽ thấy mỗi thành phần thường có những tham số ảnh hưởng tới mức độ nó đóng góp vào mô hình tổng thể. Ở giai đoạn này, chỉ cần nhận diện từng tham số là đủ.

Give Each Parameter a Value Once you have the parameters broken out, you can go through and assign each one a value. You expect to introduce some

Gán cho mỗi tham số một giá trị. Một khi đã tách bạch các tham số, bạn có thể lần lượt gán cho mỗi cái một giá trị. Bạn nên lường trước rằng mình sẽ đưa thêm vào một số

errors in this step. The trick is to work out which parameters have the most impact on the result, and concentrate on getting them about right. Typically, parameters whose values are added into a result are less significant than those that are multiplied or divided. Doubling a line speed may double the amount of data received in an hour, while adding a 5ms transit delay will have no noticeable effect.

sai số ở bước này. Mẹo ở đây là tìm ra những tham số nào tác động mạnh nhất tới kết quả, rồi tập trung làm cho chúng tạm đúng. Thông thường, các tham số có giá trị được cộng vào kết quả ít quan trọng hơn các tham số được nhân hoặc chia. Tăng gấp đôi tốc độ đường truyền có thể làm tăng gấp đôi lượng dữ liệu nhận được trong một giờ, trong khi cộng thêm 5ms độ trễ truyền dẫn sẽ chẳng có tác động đáng kể nào.

You should have a justifiable way of calculating these critical parameters. For the queuing example, you might want to measure the actual transaction arrival rate of the existing system, or find a similar

system to measure. Similarly, you could measure the current time taken to serve a request, or come up with an estimate using the techniques described in this section. In fact, you'll often find yourself basing an estimate on other subestimates. This is where your largest errors will creep in.

Bạn nên có một cách tính các tham số then chốt này một cách có cơ sở. Với ví dụ hàng đợi, bạn có thể muốn đo tốc độ giao dịch đến thực tế của hệ thống hiện có, hoặc tìm một hệ thống tương tự để đo. Tương tự, bạn có thể đo thời gian hiện tại để phục vụ một yêu cầu, hoặc đưa ra một ước lượng bằng các kỹ thuật mô tả trong mục này. Thực tế, bạn sẽ thường thấy mình đặt một ước lượng dựa trên những ước lượng con khác. Đây chính là chỗ những sai số lớn nhất của bạn sẽ len lỏi vào.

Calculate the Answers Only in the simplest of cases will an estimate have a single answer. You might be happy to say “I can walk five cross-town blocks in 15 minutes.” However, as the systems get more complex, you'll want to hedge your answers. Run multiple calculations, varying the values of the critical parameters, until you work out which ones really drive the model. A spreadsheet can be a big help. Then couch your answer in terms of these parameters. “The response time is roughly three quarters of a second if the system has SSDs and 32GB of memory, and one second with 16GB memory.” (Notice how “three quarters of a second” conveys a different feeling of accuracy than 750ms.)

Tính ra đáp số. Chỉ trong những trường hợp đơn giản nhất thì một ước lượng mới có một đáp số duy nhất. Bạn có thể thoải mái nói “Tôi đi bộ qua năm dãy phố trong khu là mất 15 phút.” Tuy nhiên, khi hệ thống trở nên phức tạp hơn, bạn sẽ muốn rào trước câu trả lời của mình. Hãy chạy nhiều phép tính, thay đổi giá trị của các tham số then chốt, cho đến khi bạn tìm ra những tham số nào thực sự chi phối mô hình. Một bảng tính có thể giúp ích rất nhiều. Rồi hãy diễn đạt câu trả lời theo các tham số đó. “Thời gian phản hồi tầm ba phần tư giây nếu hệ thống có SSD và 32GB bộ nhớ, và một giây với 16GB bộ nhớ.” (Để ý xem “ba phần tư giây” gợi một cảm giác chính xác khác hẳn so với 750ms.)

During the calculation phase, you get answers that seem strange. Don't be too quick to dismiss them. If your arithmetic

Trong giai đoạn tính toán, bạn sẽ nhận được những đáp số trông kỳ quặc. Đừng vội gạt bỏ chúng. Nếu phép tính số học của bạn

is correct, your understanding of the problem or your model is probably wrong. This is valuable information.

đúng, thì có lẽ cách bạn hiểu vấn đề hoặc mô hình của bạn đang sai. Đây là thông tin quý giá.

Keep Track of Your Estimating Prowess We think it's a great idea to record your estimates so you can see how close you were. If an overall estimate involved calculating subestimates, keep track of these as well. Often you'll find your estimates are pretty good—in fact, after a while, you'll come to expect this.

Theo dõi tài nghệ ước lượng của bạn. Chúng tôi nghĩ ghi lại các ước lượng của mình là một ý hay, để bạn thấy mình đã đoán sát đến đâu. Nếu một ước lượng tổng thể có liên quan tới việc tính các ước lượng con, hãy theo dõi cả những cái đó nữa. Bạn sẽ thường thấy các ước lượng của mình khá tốt — thực ra, một thời gian sau, bạn sẽ bắt đầu kỳ vọng điều đó.

When an estimate turns out wrong, don't just shrug and walk away—find out why. Maybe you chose some parameters that didn't match the reality of the problem. Maybe your model was wrong. Whatever the reason, take some time to uncover what happened. If you do, your next estimate will be better.

Khi một ước lượng hóa ra sai, đừng chỉ nhún vai rồi bỏ đi — hãy tìm cho ra tại sao. Có thể

bạn đã chọn vài tham số không khớp với thực tế của vấn đề. Có thể mô hình của bạn sai. Dù lý do là gì, hãy dành chút thời gian khám phá xem chuyện gì đã xảy ra. Nếu làm vậy, ước lượng lần sau của bạn sẽ tốt hơn.

Estimating Project Schedules — Ước lượng lịch trình dự án

Normally you'll be asked to estimate how long something will take. If that “something” is complex, the estimate can be very difficult to produce. In this section, we'll look at two techniques for reducing that uncertainty.

Thông thường người ta sẽ nhờ bạn ước lượng một việc gì đó sẽ mất bao lâu. Nếu “việc gì đó” ấy phức tạp, con số ước lượng có thể rất khó đưa ra. Trong mục này, chúng tôi sẽ xem xét hai kỹ thuật để giảm bớt sự bất định ấy.

Painting the Missile

Sơn quả tên lửa

“How long will it take to paint the house?”

“Sơn xong căn nhà thì mất bao lâu?”

“Well, if everything goes right, and this paint has the coverage they claim, it might be as few as 10 hours. But that's unlikely: I'd guess a more realistic figure is closer to 18 hours. And, of course, if the weather turns bad, that could push it out to 30 or more.”

“Ồ, nếu mọi thứ suôn sẻ, và loại sơn này phủ được như họ quảng cáo, thì có khi chỉ mất ít ỏi 10 giờ. Nhưng khả năng đó hơi thấp: tôi đoán một con số thực tế hơn là khoảng 18 giờ. Và tất nhiên, nếu thời tiết xấu đi, nó có thể bị đẩy lên 30 giờ hoặc hơn.”

That's how people estimate in the real world. Not with a single

Đó là cách người ta ước lượng trong thế giới thực. Không phải bằng một con số

number (unless you force them to give you one) but with a range of scenarios.

duy nhất (trừ khi bạn ép họ phải nêu ra một con số) mà bằng một dải các kịch bản.

When the U.S. Navy needed to plan the Polaris submarine project, they adopted this style of estimating with a methodology they called the Program Evaluation Review Technique , or PERT .

Khi Hải quân Hoa Kỳ cần lập kế hoạch cho dự án tàu ngầm Polaris, họ đã áp dụng kiểu ước lượng này với một phương pháp luận họ gọi là Kỹ thuật Đánh giá và Rà soát Chương trình (Program Evaluation Review Technique), hay PERT.

Every PERT task has an optimistic , a most likely , and a pessimistic estimate . The tasks are arranged into a dependency network, and then you use some simple statistics to identify likely best and worst times for the overall project.

Mỗi tác vụ trong PERT có một ước lượng lạc quan, một ước lượng khả dĩ nhất, và một ước lượng bi quan. Các tác vụ được sắp xếp thành một mạng lưới phụ thuộc, rồi bạn dùng vài phép thống kê đơn giản để xác định khoảng thời gian tốt nhất và xấu nhất khả dĩ cho toàn bộ dự án.

Using a range of values like this is a great way to avoid one of the most common causes of estimation error: padding a number because you're unsure. Instead, the statistics behind PERT spreads the uncertainty out for you, giving you better estimations of the whole project.

Dùng một dải giá trị như vậy là một cách tuyệt vời để tránh một trong những nguyên nhân phổ biến nhất gây sai sót ước lượng: độn thêm vào một con số chỉ vì bạn không chắc chắn. Thay vào đó, phép thống kê đứng sau PERT trải đều sự bất định ra giúp bạn, cho bạn những ước lượng tốt hơn cho toàn dự án.

However, we're not big fans of this. People tend to produce wall-sized charts of all the tasks in a project, and implicitly believe that, just because they used a formula, they have an accurate estimate. The chances are they don't, because they have never done this before.

Tuy nhiên, chúng tôi không phải fan ruột của cách này. Người ta hay làm ra những biểu đồ to bằng cả bức tường liệt kê mọi tác vụ trong một dự án, rồi ngẫm tin rằng, chỉ vì đã dùng một công thức, là họ có một ước lượng chính xác. Khả năng cao là không, bởi họ chưa từng làm việc này bao giờ.

Eating the Elephant We find that often the only way to determine the timetable for a project is by gaining experience on that same project. This needn't be a paradox if you practice incremental development, repeating the following steps with very thin slices of functionality:

Ăn con voi. Chúng tôi thấy rằng thường thì cách duy nhất để xác định thời gian biểu cho một dự án là tích lũy kinh nghiệm trên chính dự án đó. Điều này không hề mâu thuẫn nếu bạn thực hành phát triển tăng dần (incremental development), lặp lại các bước sau với những lát cắt chức năng cực kỳ mỏng:

Check **requirements**

Kiểm tra yêu cầu

Analyze risk (and prioritize riskiest items earlier)

Phân tích rủi ro (và ưu tiên những hạng mục rủi ro nhất lên sớm hơn)

Design, implement, integrate

Thiết kế, hiện thực, tích hợp

Validate with the users

Kiểm chứng với người dùng

Initially, you may have only a vague idea of how many iterations will be required, or how long they may be. Some methods require you to nail this down as part of the initial plan; however, for all but the most trivial of projects this is a mistake. Unless you are doing an application similar to a previous one, with the same team and the same technology, you'd just be guessing.

Ban đầu, bạn có thể chỉ có một ý niệm mơ hồ về việc sẽ cần bao nhiêu vòng lặp, hay mỗi vòng dài bao lâu. Một số phương pháp đòi hỏi bạn phải chốt cứng điều này như một phần của kế hoạch khởi đầu; tuy nhiên, với mọi dự án trừ những dự án tầm phào nhất, làm vậy là một sai lầm. Trừ phi bạn đang làm một ứng dụng tương tự một ứng dụng trước đó, với cùng đội ngũ và cùng công nghệ, thì bạn chỉ đang đoán mò mà thôi.

So you complete the coding and testing of the initial functionality and mark this as the end of the first iteration. Based on that experience, you can refine your initial guess on the number of iterations and what can be included in each. The refinement gets better and better each time, and confidence in the schedule grows along with it. This kind of estimating is often done during the team's review at the end of each iterative cycle.

Vậy là bạn hoàn thành việc viết code và kiểm thử cho khối chức năng đầu tiên rồi đánh dấu đây là điểm kết thúc của vòng lặp thứ nhất. Dựa trên kinh nghiệm đó, bạn có thể tinh chỉnh phỏng đoán ban đầu về số vòng lặp và những gì có thể gói vào mỗi vòng. Sự tinh

chỉnh càng ngày càng tốt hơn, và lòng tin vào lịch trình cũng lớn lên theo. Kiểu ước lượng này thường được thực hiện trong buổi rà soát của đội ngũ ở cuối mỗi chu kỳ lặp.

That's also how the old joke says to eat an elephant: one bite at a time.

Đó cũng chính là cách mà câu đùa xưa bảo ta ăn một con voi: gặm từng miếng một.

Tip 24: Iterate the Schedule with the Code

Tip 24: Hãy tinh chỉnh lịch trình song song với mã nguồn

This may not be popular with management, who typically want a single, hard-and-fast number before the project even starts. You'll have to help them understand that the team, their productivity, and the environment will determine the schedule. By formalizing this, and refining the schedule as part of each iteration, you'll be giving them the most accurate scheduling estimates you can.

Điều này có thể không được lòng cấp quản lý, những người thường muốn có một con số duy nhất, chắc nịch và bất di bất dịch ngay từ trước khi dự án bắt đầu. Bạn sẽ phải giúp họ hiểu rằng chính đội ngũ, năng suất của họ, và môi trường mới là thứ quyết định lịch trình. Bằng cách chính thức hóa điều này, và tinh chỉnh lịch trình như một phần của mỗi vòng lặp, bạn sẽ đem lại cho họ những ước lượng lịch trình chính xác nhất có thể.

What To Say When Asked For An Estimate — Nên nói gì khi bị hỏi một con số ước lượng

You say “I’ll get back to you.”

Bạn nói “Để tôi trả lời sau.”

You almost always get better results if you slow the process down and spend some time going through the steps we describe in this section. Estimates given at the coffee machine will (like the coffee) come back to haunt you.

Bạn gần như luôn nhận được kết quả tốt hơn nếu làm chậm quá trình lại và dành chút thời gian đi qua các bước chúng tôi mô tả trong mục này. Những ước lượng buột miệng bên máy pha cà phê thì (giống như cà phê) sẽ quay lại ám bạn.

Related Sections Include — Các mục liên quan

Topic 7, Communicate!

Topic 7, Giao Tiếp!

Topic 39, Algorithm Speed

Topic 39, Tốc độ của thuật toán

Challenges — Thử thách

Start keeping a log of your estimates. For each, track how accurate you turned out to be. If your error was greater than 50%, try to find out where your estimate went wrong.

Hãy bắt đầu ghi nhật ký các ước lượng của bạn. Với mỗi cái, theo dõi xem bạn đã đoán chính xác đến đâu. Nếu sai số của bạn lớn hơn 50%, hãy thử tìm xem ước lượng của mình đã chệch hướng ở chỗ nào.

Exercises — Bài tập

Exercise 9 (possible answer)

Bài 9 (đáp án khả dĩ)

You are asked “Which has a higher bandwidth: a 1Gbps net connection or a person walking between two computers with a full 1TB of storage device in their pocket?” What constraints will you put on your answer to ensure that the scope of your response is correct? (For example, you might say that the time taken to access the storage device is ignored.)

Bạn được hỏi “Cái nào có băng thông cao hơn: một đường mạng 1Gbps, hay một người đi bộ giữa hai máy tính với một thiết bị lưu trữ 1TB đầy ắp dữ liệu trong túi?” Bạn sẽ đặt những ràng buộc nào lên câu trả lời để đảm bảo phạm vi phản hồi của mình là đúng đắn? (Ví dụ, bạn có thể nói rằng thời gian truy cập thiết bị lưu trữ được bỏ qua.)

Exercise 10 (possible answer)

Bài 10 (đáp án khả dĩ)

So, which has the higher bandwidth?

Vậy, cái nào có băng thông cao hơn?

Footnotes

Chú thích

[13] To paraphrase the old Arlen/Mercer song...

[13] Mượn ý từ ca khúc xưa của Arlen/Mercer...

[14] Or, perhaps, to keep your sanity, every 10th time...

[14] Hoặc, có lẽ, để giữ cho mình còn tỉnh táo, cứ 10 lần thì một lần...

[15] <https://github.com/OAI/OpenAPI-Specification>

[15] <https://github.com/OAI/OpenAPI-Specification>

[16] In reality, this is naive. Unless you are remarkably lucky, most real-world requirements changes will affect multiple functions in the system. However, if you analyze the change in terms of functions, each functional change should still ideally affect just one module.

[16] Thực tế, suy nghĩ này còn ngây thơ. Trừ phi bạn may mắn đến lạ thường, hầu hết các thay đổi yêu cầu trong thế giới thực sẽ ảnh hưởng tới nhiều hàm trong hệ thống. Tuy nhiên, nếu bạn phân tích sự thay đổi theo từng hàm, thì lý tưởng nhất mỗi thay đổi chức năng vẫn chỉ nên ảnh hưởng tới đúng một mô-đun.

[17] In fact, this book is written in Markdown, and typeset directly from the Markdown source.

[17] Thực tế, cuốn sách này được viết bằng Markdown, và được dàn trang trực tiếp từ mã nguồn Markdown.

[18] Take a nonlinear, or chaotic, system and apply a small change to one of its inputs. You may get a large and often unpredictable result. The clichéd butterfly flapping its wings in Tokyo could be the start of a chain of events that ends up generating a tornado in Texas. Does this sound like any projects you know?

[18] Lấy một hệ phi tuyến, hay hỗn loạn, rồi áp một thay đổi nhỏ vào một trong các đầu vào của nó. Bạn có thể nhận được một kết quả lớn và thường là khó lường. Cái cảnh con

bướm vồ cánh ở Tokyo đã thành sáo mồn có thể là khởi đầu của một chuỗi sự kiện rất cuộc sinh ra một cơn lốc xoáy ở Texas. Nghe có giống dự án nào bạn từng biết không?

[19] <https://rspec.info>

[19] <https://rspec.info>

[20] <https://cucumber.io/>

[20] <https://cucumber.io/>

[21] <https://phoenixframework.org/>

[21] <https://phoenixframework.org/>

[22] <https://www.ansible.com/>

[22] <https://www.ansible.com/>

[23] <https://yaml.org/>

[23] <https://yaml.org/>

Copyright © 2020 Pearson Education, Inc.

Copyright © 2020 Pearson Education, Inc.

The Basic Tools — Những công cụ cơ bản

Every maker starts their journey with a basic set of good-quality tools. A woodworker might need rules, gauges, a couple of saws, some good planes, fine chisels, drills and braces, mallets, and clamps. These tools will be lovingly chosen, will be built to last, will perform specific jobs with little overlap with other tools, and, perhaps most importantly, will feel right in the budding woodworker's hands.

Mọi người thợ đều khởi đầu hành trình của mình với một bộ công cụ tốt căn bản. Một người thợ mộc có thể cần thước, dũa, đo, vài cái cưa, mấy cái bào ngon lành, đục tinh xảo, khoan và tay quay, vồ gỗ, và kẹp. Những công cụ này được chọn lựa đầy năng niu, được làm ra để dùng bền lâu, làm những việc cụ thể ít chồng chéo với công cụ khác, và — có lẽ quan trọng nhất — phải cảm thấy “vừa tay” với người thợ mộc mới vào nghề.

Then begins a **process** of learning and adaptation. Each tool will have its own personality and quirks, and will need its own special handling. Each must be sharpened in a unique way, or held just so. Over time, each will wear according to use, until the grip looks like a mold of the woodworker's hands and the cutting surface aligns perfectly with the angle at which the tool is held. At this point, the tools become conduits from the maker's brain to the finished product—they have become extensions of their hands. Over time, the woodworker will add new tools, such as biscuit cutters, laser-guided miter saws, dovetail jigs—all wonderful pieces of technology. But you can bet that they'll be happiest with one of those original tools in hand, feeling the plane sing as it slides through the wood.

Rồi bắt đầu một quá trình học hỏi và thích nghi. Mỗi công cụ có cá tính và những nét lập dị riêng, cần được xử lý theo cách riêng của nó. Mỗi cái phải được mài theo một kiểu độc nhất, hoặc cầm theo một góc nhất định. Theo thời gian, mỗi công cụ sẽ mòn theo cách sử dụng, cho đến khi cái cán trông như được đúc khuôn theo bàn tay người thợ và lưỡi cắt khớp hoàn hảo với góc mà công cụ thường được giữ. Đến lúc đó, các công cụ trở thành ống dẫn từ bộ não người thợ tới sản phẩm hoàn thiện — chúng đã trở thành phần nối dài của đôi bàn tay. Theo thời gian, người thợ mộc sẽ sắm thêm công cụ mới, như máy cắt mộng dẹt, máy cưa góc dẫn hướng bằng laser, đồ gá mộng đuôi én — toàn những tuyệt tác công nghệ. Nhưng bạn có thể đặt cược rằng họ sẽ hạnh phúc nhất khi cầm trong tay một trong những công cụ nguyên thủy ấy, cảm nhận cái bào ngân lên khi nó lướt qua thớ gỗ.

Tools amplify your talent. The better your tools, and the better

Công cụ khuếch đại tài năng của bạn. Công cụ càng tốt, và bạn

you know how to use them, the more productive you can be. Start with a basic set of generally applicable tools. As you gain experience, and as you come across special **requirements**, you'll add to this basic set. Like the maker, expect to add to your toolbox regularly. Always be on the lookout for better ways of doing things. If you come across a situation where you feel your current tools can't cut

it, make a note to look for something different or more powerful that would have helped. Let need drive your acquisitions.

càng biết cách dùng chúng, bạn càng làm việc năng suất hơn. Hãy bắt đầu với một bộ công cụ căn bản có tính ứng dụng rộng. Khi tích lũy kinh nghiệm, và khi gặp những yêu cầu đặc thù, bạn sẽ bổ sung thêm vào bộ căn bản này. Giống như người thợ, hãy sẵn sàng đều đặn thêm công cụ vào hộp đồ nghề của mình. Luôn dò tìm những cách làm việc tốt hơn. Nếu gặp tình huống mà bạn cảm thấy bộ công cụ hiện tại “không kham nổi”, hãy ghi chú lại để đi tìm một thứ khác hoặc mạnh hơn lẽ ra đã giúp được bạn. Hãy để nhu cầu dẫn dắt việc sắm sửa.

Many new programmers make the mistake of adopting a single power tool, such as a particular integrated development environment (IDE), and never leave its cozy interface. This really is a mistake. You need to be comfortable beyond the limits imposed by an IDE. The only way to do this is to keep the basic tool set sharp and ready to use.

Nhiều lập trình viên mới mắc sai lầm là chọn lấy một công cụ mạnh duy nhất, chẳng hạn một môi trường phát triển tích hợp (IDE) cụ thể nào đó, rồi không bao giờ rời khỏi giao diện ấm cúng của nó. Đây thực sự là một sai lầm. Bạn cần thấy thoải mái cả ở bên ngoài giới hạn mà một IDE áp đặt. Cách duy nhất để làm được điều đó là giữ cho bộ công cụ căn bản luôn sắc bén và sẵn sàng dùng.

In this chapter we'll talk about investing in your own basic toolbox. As with any good discussion on tools, we'll start (in **Topic 16, The Power of Plain Text**) by looking at your raw materials, the stuff you'll be shaping. From there we'll move to the workbench, or in our case the computer. How can you use your computer to get the most out of the tools you use? We'll discuss this in **Topic 17, Shell Games**. Now that we have material and a bench to work on, we'll turn to the tool you'll probably use more than any other, your editor. In **Topic 18, Power Editing**, we'll suggest ways of making you more efficient.

Trong chương này, chúng ta sẽ bàn về việc đầu tư cho hộp công cụ căn bản của riêng bạn. Như mọi cuộc bàn luận tử tế về công cụ, chúng ta sẽ khởi đầu (ở **Topic 16, Sức mạnh của Văn bản thuần**) bằng việc nhìn vào nguyên liệu thô của bạn, thứ mà bạn sẽ tạo hình. Từ đó chúng ta chuyển sang bàn làm việc, hay trong trường hợp của ta là chiếc máy tính. Làm sao để bạn khai thác được nhiều nhất từ những công cụ mình dùng? Chúng ta sẽ bàn điều này ở **Topic 17, Trò chơi với Shell**. Giờ đã có nguyên liệu và một bàn làm việc, ta sẽ quay sang công cụ mà bạn hẳn sẽ dùng nhiều hơn bất kỳ thứ gì khác: trình soạn thảo của bạn. Ở **Topic 18, Soạn thảo Cao thủ**, chúng tôi sẽ gợi ý các cách giúp bạn làm việc hiệu quả hơn.

To ensure that we never lose any of our precious work, we should always use a **Topic 19, Version Control** system—even for personal things such as recipes or notes. And, since Murphy was really an optimist after all, you can't be a great programmer until you become highly skilled at **Topic 20, Debugging**.

Để đảm bảo không bao giờ mất đi bất kỳ thành quả quý giá nào, ta nên luôn dùng một hệ thống **Topic 19, Quản lý phiên bản** — kể cả cho những thứ cá nhân như công thức nấu ăn hay ghi chú. Và, vì Murphy rốt cuộc lại là một người lạc quan, bạn không thể trở thành một lập trình viên cừ khôi cho tới khi thành thạo **Topic 20, Gỡ lỗi**.

You'll need some glue to bind much of the magic together. We discuss some possibilities in **Topic 21, Text Manipulation**.

Bạn sẽ cần chút “keo” để gắn phần lớn ma thuật ấy lại với nhau. Chúng tôi bàn vài khả năng ở **Topic 21, Thao tác Văn bản**.

Finally, the palest ink is still better than the best memory. Keep track of your thoughts and your history, as we describe in Topic 22, Engineering Daybooks .

Cuối cùng, mực nhạt nhất vẫn còn hơn trí nhớ tốt nhất. Hãy ghi lại những suy nghĩ và lịch sử làm việc của bạn, như chúng tôi mô tả ở Topic 22, Nhật ký Kỹ thuật.

Spend time learning to use these tools, and at some point you'll be surprised to discover your fingers moving over the keyboard, manipulating text without conscious thought. The tools will have become extensions of your hands.

Hãy dành thời gian học cách dùng những công cụ này, và đến một lúc nào đó bạn sẽ ngạc nhiên nhận ra ngón tay mình lướt trên bàn phím, thao tác văn bản mà chẳng cần suy nghĩ có ý thức. Các công cụ đã trở thành phần nối dài của đôi bàn tay bạn.

Topic 16. The Power of Plain Text — Topic 16. Sức mạnh của Văn bản thuần

As **Pragmatic** Programmers, our base material isn't wood or iron, it's knowledge. We gather requirements as knowledge, and then express that knowledge in our designs, implementations, tests, and documents. And we believe that the best format for storing knowledge persistently is plain text . With plain text, we give ourselves the ability to manipulate knowledge, both manually and programmatically, using virtually every tool at our disposal.

Là những Lập trình viên Thực dụng (Pragmatic Programmer), nguyên liệu nền của chúng ta không phải gỗ hay sắt, mà là tri thức. Ta thu thập yêu cầu dưới dạng tri thức, rồi diễn đạt tri thức ấy trong các thiết kế, hiện thực, kiểm thử và tài liệu của mình. Và chúng tôi tin rằng định dạng tốt nhất để lưu trữ tri thức một cách bền vững chính là văn bản thuần (plain text). Với văn bản thuần, ta tự trao cho mình khả năng thao tác tri thức — cả thủ công lẫn bằng chương trình — bằng gần như mọi công cụ trong tầm tay.

The problem with most binary formats is that the **context** necessary to understand the data is separate from the data itself. You are artificially divorcing the data from its meaning. The data may as well be encrypted; it is absolutely meaningless without the application logic to parse it. With plain text, however, you can achieve a self-describing data **stream** that is independent of the application that created it.

Vấn đề với hầu hết các định dạng nhị phân là phần ngữ cảnh cần thiết để hiểu dữ liệu lại tách rời khỏi chính dữ liệu. Bạn đang chia lìa dữ liệu khỏi ý nghĩa của nó một cách giả tạo. Dữ liệu đó cũng coi như bị mã hóa vậy; nó hoàn toàn vô nghĩa nếu thiếu logic của ứng dụng để phân tích cú pháp. Còn với văn bản thuần, bạn có thể đạt được một luồng dữ liệu tự mô tả, độc lập với ứng dụng đã tạo ra nó.

What Is Plain Text? — Văn bản thuần là gì?

Plain text is made up of printable characters in a form that conveys information. It can be as simple as a shopping list:

Văn bản thuần được tạo nên từ các ký tự in được, ở một dạng truyền tải thông tin. Nó có thể đơn giản như một danh sách đi chợ:

- milk * lettuce * coffee
 - sữa * xà lách * cà phê

or as complex as the source of this book (yes, it's in plain text, much to the chagrin of the **publisher**, who wanted us to use a word processor).

hoặc phức tạp như mã nguồn của chính cuốn sách này (vâng, nó được viết bằng văn bản thuần, làm phật lòng không ít nhà xuất bản, những người muốn chúng tôi dùng một trình xử lý văn bản).

The information part is important. The following is not useful plain text:

Phần “thông tin” mới là điều quan trọng. Đoạn dưới đây không phải là văn bản thuần hữu ích:

hlj;uijn bfjxrrectvh jkni'pio6p7gu;vh bjxr di5rgv h j

hlj;uijn bfjxrrectvh jkni'pio6p7gu;vh bjxr di5rgv h j

Neither is this:

Mà đoạn này cũng không:

Field19=467abe

Field19=467abe

The reader has no idea what the significance of 467abe may be. We like our plain text to be understandable to humans.

Người đọc chẳng thể nào đoán được 467abe có ý nghĩa gì. Chúng tôi muốn văn bản thuần của mình phải là thứ con người hiểu được.

Tip 25: Keep Knowledge in Plain Text

Tip 25: Hãy lưu giữ tri thức bằng văn bản thuần

The Power Of Text — Sức mạnh của Văn bản

Plain text doesn't mean that the text is unstructured; HTML, JSON, YAML, and so on are all plain text. So are the majority of the fundamental protocols on the net, such as HTTP, SMTP, IMAP, and so on. And that's for some good reasons:

Văn bản thuần không có nghĩa là văn bản không có cấu trúc; HTML, JSON, YAML, v.v. đều là văn bản thuần. Phần lớn các giao thức nền tảng trên mạng cũng vậy, như HTTP, SMTP, IMAP, v.v. Và điều đó có những lý do chính đáng:

Insurance against obsolescence

Bảo hiểm chống lỗi thời

Leverage existing tools

Tận dụng các công cụ sẵn có

Easier testing

Kiểm thử dễ dàng hơn

Insurance Against Obsolescence Human-readable forms of data, and self-describing data, will outlive all other forms of data and the applications that created them. Period. As long as the data survives, you will have a chance to be able to use it—potentially long after the original application that wrote it is defunct.

Bảo hiểm chống lỗi thời — Những dạng dữ liệu con người đọc được, và dữ liệu tự mô tả, sẽ sống lâu hơn mọi dạng dữ liệu khác và lâu hơn cả những ứng dụng đã tạo ra chúng. Chấm hết. Chừng nào dữ liệu còn tồn tại, bạn còn có cơ hội dùng được nó — có khi là rất lâu sau khi ứng dụng gốc viết ra nó đã “về vườn”.

You can parse such a file with only partial knowledge of its

Bạn có thể phân tích cú pháp một tập tin như thế chỉ với hiểu biết một phần về

format; with most binary files, you must know all the details of the entire format in order to parse it successfully.

định dạng của nó; còn với phần lớn tập tin nhị phân, bạn phải biết hết mọi chi tiết của toàn bộ định dạng thì mới phân tích cú pháp thành công.

Consider a data file from some legacy system that you are given. [24] You know little about the original application; all that’s important to you is that it maintained a list of clients’ Social Security numbers, which you need to find and extract. Among the data, you see

Hãy hình dung một tập tin dữ liệu từ hệ thống cũ kỹ (legacy) nào đó được giao cho bạn. [24] Bạn biết rất ít về ứng dụng gốc; tất cả những gì bạn quan tâm là nó lưu một danh sách số An sinh Xã hội (Social Security) của khách hàng, thứ mà bạn cần tìm và trích ra. Lần trong đồng dữ liệu, bạn thấy

123-45-6789 ... 567-89-0123 ... 901-23-4567

123-45-6789 ... 567-89-0123 ... 901-23-4567

Recognizing the format of a Social Security number, you can quickly write a small program to extract that data—even if you have no information on anything else in the file.

Nhận ra định dạng của một số An sinh Xã hội, bạn có thể nhanh chóng viết một chương trình nhỏ để trích dữ liệu đó ra — kể cả khi bạn không có chút thông tin nào về phần còn lại trong tập tin.

But imagine if the file had been formatted this way instead:

Nhưng hãy tưởng tượng nếu tập tin lại được định dạng theo kiểu này:

AC27123456789B11P ... XY43567890123QTYL ... 6T2190123456788AM

AC27123456789B11P ... XY43567890123QTYL ... 6T2190123456788AM

You may not have recognized the significance of the numbers quite as easily. This is the difference between human readable and human understandable .

Hẳn bạn sẽ không nhận ra ý nghĩa của những con số này dễ dàng đến thế. Đây chính là khác biệt giữa con người đọc được và con người hiểu được.

While we’re at it, FIELD10 doesn’t help much either. Something like

Nhân tiện, cái tên FIELD10 cũng chẳng giúp được gì mấy. Một thứ đại loại như

123-45-6789

123-45-6789

makes the exercise a no-brainer—and ensures that the data will

biến bài toán thành chuyện chẳng cần nghĩ ngợi — và đảm bảo dữ liệu sẽ outlive any project that created it.

sống lâu hơn bất kỳ dự án nào đã tạo ra nó.

Leverage Virtually every tool in the computing universe, from version control systems to editors to command-line tools, can operate on plain text.

Tận dụng — Gần như mọi công cụ trong vũ trụ điện toán, từ hệ thống quản lý phiên bản đến trình soạn thảo đến công cụ dòng lệnh, đều có thể làm việc trên văn bản thuần.

The Unix Philosophy

Triết lý Unix

Unix is famous for being designed around the philosophy of small, sharp tools, each intended to do one thing well. This philosophy is enabled by using a common underlying format—the line-oriented, plain-text file. Databases used for system administration (users and passwords, networking **configuration**, and so on) are all kept as plain-text files. (Some systems also maintain a binary form of certain databases as a performance optimization. The plain-text version is kept as an interface to the binary version.)

Unix nổi tiếng vì được thiết kế quanh triết lý những công cụ nhỏ, sắc bén, mỗi cái nhằm làm thật tốt một việc. Triết lý này khả thi nhờ việc dùng một định dạng nền tảng chung — tập tin văn bản thuần, theo dòng. Các cơ sở dữ liệu dùng cho quản trị hệ thống (người dùng và mật khẩu, cấu hình mạng, v.v.) đều được giữ dưới dạng tập tin văn bản thuần. (Một số hệ thống còn duy trì thêm một dạng nhị phân của vài cơ sở dữ liệu nhất định để tối ưu hiệu năng. Bản văn bản thuần được giữ làm giao diện cho bản nhị phân.)

When a system crashes, you may be faced with only a minimal environment to restore it (you may not be able to access graphics drivers, for instance). Situations such as this can really make you appreciate the simplicity of plain text.

Khi một hệ thống sập, có thể bạn chỉ còn một môi trường tối thiểu để khôi phục nó (chẳng hạn bạn không truy cập được driver đồ họa). Những tình huống như thế thực sự khiến bạn biết trân trọng sự đơn giản của văn bản thuần.

Plain text is also easier to search. If you can't remember which configuration file manages your system backups, a quick `grep -r backup /etc` should tell you.

Văn bản thuần cũng dễ tìm kiếm hơn. Nếu bạn không nhớ nổi tập tin cấu hình nào quản lý việc sao lưu hệ thống, một lệnh `grep -r backup /etc` chớp nhoáng sẽ cho bạn câu trả lời.

For instance, suppose you have a production deployment of a large application with a complex site-specific configuration file. If this file is in plain text, you could place it under a version control system (see Topic 19, Version Control), so that you automatically keep a history of all changes. File comparison tools such as `diff` and `fc` allow you to see at a glance what changes have been made, while `sum` allows you to generate a checksum to monitor the file for accidental (or malicious) modification.

Chẳng hạn, giả sử bạn có một bản triển khai production của một ứng dụng lớn kèm theo một tệp cấu hình phức tạp riêng cho từng nơi cài đặt. Nếu tệp này ở dạng văn bản thuần, bạn có thể đặt nó dưới một hệ thống quản lý phiên bản (version control) (xem Topic 19, Quản lý phiên bản), để tự động giữ lại lịch sử của mọi thay đổi. Các công cụ so sánh tệp như `diff` và `fc` cho phép bạn liếc một cái là thấy ngay những gì đã bị sửa, còn `sum` thì cho phép bạn sinh ra một checksum để canh chừng tệp trước những sửa đổi ngoài ý muốn (hoặc có ác ý).

Easier Testing If you use plain text to create synthetic data to drive system tests, then it is a simple matter to add, update, or modify the test data without having to create any special tools to do so.

Kiểm thử dễ dàng hơn — Nếu bạn dùng văn bản thuần để tạo dữ liệu tổng hợp phục vụ kiểm thử hệ thống, thì việc thêm, cập nhật, hay chỉnh sửa dữ liệu kiểm thử trở nên đơn giản, chẳng cần phải làm ra công cụ đặc biệt nào.

Similarly, plain-text output from regression tests can be trivially analyzed with shell commands or a simple script.

Tương tự, đầu ra dạng văn bản thuần từ các bài kiểm thử hồi quy (regression test) có thể được phân tích một cách dễ như bốn bằng lệnh shell hoặc một script đơn giản.

Lowest Common Denominator — Mẫu số chung thấp nhất

Even in the future of blockchain-based intelligent agents that travel the wild and dangerous internet autonomously, negotiating data interchange among themselves, the ubiquitous text file will still be there. In fact, in heterogeneous environments the advantages of plain text can outweigh all of the drawbacks. You need to ensure that all parties can communicate using a common standard. Plain text is that standard.

Ngay cả trong tương lai của những tác tử thông minh dựa trên blockchain tự do rong ruổi khắp internet hoang dã và nguy hiểm, tự thương lượng trao đổi dữ liệu với nhau, thì tập tin văn bản hiện diện khắp nơi vẫn sẽ còn đó. Thực vậy, trong những môi trường không đồng nhất, các ưu điểm của văn bản thuần có thể lấn át mọi nhược điểm. Bạn cần đảm bảo mọi bên đều có thể giao tiếp bằng một chuẩn chung. Văn bản thuần chính là cái chuẩn đó.

Related Sections Include — Các mục liên quan

Topic 17, Shell Games

Topic 17, Trò chơi với Shell

Topic 21, Text Manipulation

Topic 21, Thao tác Văn bản

Topic 32, Configuration

Topic 32, Cấu hình

Challenges — Thử thách

Design a small address book database (name, phone number, and so on) using a straightforward binary representation in your language of choice. Do this before reading the rest of this challenge.

Hãy thiết kế một cơ sở dữ liệu sổ địa chỉ nhỏ (tên, số điện thoại, v.v.) bằng một cách biểu diễn nhị phân thẳng thắn trong ngôn ngữ bạn chọn. Hãy làm việc này trước khi đọc phần còn lại của thử thách.

Translate that format into a plain-text format using XML or JSON.

Dịch định dạng đó sang một định dạng văn bản thuần dùng XML hoặc JSON.

For each version, add a new, variable-length field called directions in which you might enter directions to each person's house.

Với mỗi phiên bản, hãy thêm một trường mới có độ dài thay đổi, gọi là directions, để bạn có thể nhập chỉ đường đến nhà từng người.

What issues come up regarding versioning and extensibility? Which

Những vấn đề gì nảy sinh liên quan đến quản lý phiên bản và khả năng mở rộng? Dạng nào

form was easier to modify? What about converting existing data?

để chỉnh sửa hơn? Còn việc chuyển đổi dữ liệu hiện có thì sao?

Topic 17. Shell Games — Topic 17. Trò chơi với Shell

Every woodworker needs a good, solid, reliable workbench, somewhere to hold work pieces at a convenient height while they're being shaped. The workbench becomes the center of the woodshop, the maker returning to it time and time again as a piece takes shape.

Mỗi người thợ mộc đều cần một bàn thợ tốt, chắc chắn và đáng tin cậy, một nơi để giữ phôi gỗ ở độ cao thuận tiện trong lúc đẽo gọt tạo hình. Bàn thợ trở thành trung tâm của xưởng mộc, người thợ cứ quay về với nó hết lần này đến lần khác khi tác phẩm dần thành hình.

For a programmer manipulating files of text, that workbench is the command **shell**. From the shell prompt, you can invoke your full repertoire of tools, using pipes to combine them in ways never dreamt of by their original developers. From the shell, you can launch applications, debuggers, browsers, editors, and utilities. You can search for files, query the status of the system, and filter output. And by programming the shell, you can build complex macro commands for activities you perform often.

Với một lập trình viên đang thao tác trên các tệp văn bản, cái bàn thợ ấy chính là command shell. Từ dấu nhắc của shell, bạn có thể gọi ra toàn bộ kho công cụ của mình, dùng pipe để kết hợp chúng theo những cách mà chính các nhà phát triển ra chúng cũng chưa từng mơ tới. Từ shell, bạn có thể khởi chạy ứng dụng, trình gỡ lỗi (debugger), trình duyệt, trình soạn thảo và đủ thứ tiện ích. Bạn có thể tìm tệp, truy vấn trạng thái hệ thống, và lọc đầu ra. Và bằng cách lập trình cho shell, bạn có thể dựng nên những macro phức tạp cho những việc bạn làm thường xuyên.

For programmers raised on GUI interfaces and integrated development environments (IDEs), this might seem an extreme position. After all, can't you do everything equally well by pointing and clicking?

Với những lập trình viên lớn lên cùng giao diện đồ họa (GUI) và môi trường phát triển tích hợp (IDE), quan điểm này nghe có vẻ hơi cực đoan. Suy cho cùng, chẳng phải bạn vẫn làm được mọi thứ ngon lành bằng cách trỏ và nhấp đó sao?

The simple answer is “no.” GUI interfaces are wonderful, and they can be faster and more convenient for some simple operations. Moving files, reading and writing email, and building and deploying your project are all things that you might want to do in a graphical environment. But if you do all your work using GUIs, you are missing out on the full capabilities of your environment. You won't be able to automate

Câu trả lời đơn giản là “không”. Giao diện GUI tuyệt vời thật, và với vài thao tác đơn giản thì chúng có thể nhanh và tiện hơn. Di chuyển tệp, đọc và viết email, build rồi triển khai dự án — tất cả đều là những việc bạn có thể muốn làm trong môi trường đồ họa. Nhưng nếu bạn làm tất tần tật mọi việc bằng GUI, bạn đang bỏ lỡ toàn bộ sức mạnh của môi trường mình đang dùng. Bạn sẽ không thể tự động hóa

common tasks, or use the full power of the tools available to you. And you won't be able to combine your tools to create customized macro tools. A benefit of GUIs is WYSIWYG — what you see is what you get. The disadvantage is WYSIAYG — what you see is all you get.

những tác vụ thường nhật, hay tận dụng hết sức mạnh của các công cụ trong tầm tay. Và bạn cũng không thể ghép các công cụ lại để tạo ra những macro tùy biến của riêng mình. Cái hay của GUI là WYSIWYG — thấy sao được vậy (what you see is what you get). Cái dở của nó là WYSIAYG — thấy gì thì chỉ có nhiều đó (what you see is all you get).

GUI environments are normally limited to the capabilities that their designers intended. If you need to go beyond the model the designer provided, you are usually out of luck—and more often than not, you do need to go beyond the model. **Pragmatic** Programmers don't just cut code, or develop object models, or write documentation, or automate the build **process**—we do all of these things. The scope of any one tool is usually limited to the tasks that the tool is expected to perform. For instance, suppose you need to integrate a code preprocessor (to implement design-by-contract, or multi-processing pragmas, or some such) into your IDE. Unless the designer of the IDE explicitly provided hooks for this capability, you can't do it.

Môi trường GUI thường bị giới hạn trong những khả năng mà người thiết kế ra nó dự liệu trước. Nếu bạn cần vượt ra ngoài mô hình mà người thiết kế đã vạch ra, thường thì bạn hết cách — và oái oăm thay, phần nhiều là bạn lại thật sự cần vượt ra ngoài đó. Lập trình viên Thực dụng (Pragmatic Programmer) không chỉ gõ code, hay dựng mô hình đối tượng, hay viết tài liệu, hay tự động hóa quy trình build — chúng tôi làm tất cả những việc đó. Phạm vi của bất kỳ công cụ nào cũng thường bị bó hẹp trong những tác vụ mà nó được kỳ vọng phải làm. Chẳng hạn, giả sử bạn cần tích hợp một bộ tiền xử lý code (để hiện thực thiết kế theo hợp đồng (design by contract), hay các pragma đa xử lý, hay đại loại vậy) vào IDE của mình. Trừ phi người thiết kế IDE đã cố tình chừa sẵn các móc nối (hook) cho khả năng này, bạn sẽ không làm được.

Tip 26: Use the Power of Command Shells

Tip 26: Hãy tận dụng sức mạnh của shell dòng lệnh

Gain familiarity with the shell, and you'll find your productivity soaring. Need to create a list of all the unique package names explicitly imported by your Java code? The following stores it in a file called "list":

Hãy làm quen với shell, rồi bạn sẽ thấy năng suất của mình bay vút lên. Cần lập một danh sách mọi tên gói (package) duy nhất được code Java của bạn import tường minh? Dòng lệnh sau sẽ lưu nó vào một tệp tên là "list":

```
sh/packages.sh
```

```
sh/packages.sh
```

```
grep '^import' *.java | sed -e 's/.import //' -e 's/;.*$//' | sort -u >list
```

```
grep '^import' *.java | sed -e 's/.import //' -e 's/;.*$//' | sort -u >list
```

If you haven't spent much time exploring the capabilities of the command shell on the systems you use, this might appear

Nếu bạn chưa dành nhiều thời gian khám phá khả năng của command shell trên các hệ thống mình đang dùng, dòng lệnh này trông có vẻ

daunting. However, invest some energy in becoming familiar with your shell and things will soon start falling into place. Play around with your command shell, and you'll be surprised at how much more productive it makes you.

đáng sợ. Tuy vậy, hãy đầu tư chút công sức làm quen với shell, mọi thứ rồi sẽ sớm đâu vào đó. Cứ nghịch ngợm với command shell đi, và bạn sẽ ngạc nhiên vì nó giúp bạn làm việc năng suất hơn đến nhường nào.

A Shell Of Your Own — Một cái Shell của riêng bạn

In the same way that a woodworker will customize their workspace, a developer should customize their shell. This typically also involves changing the **configuration** of the terminal program you use.

Hệt như người thợ mộc tùy biến không gian làm việc của mình, một lập trình viên cũng nên tùy biến cái shell của mình. Việc này thường cũng kéo theo chuyện thay đổi cấu hình của chương trình terminal bạn dùng.

Common changes include:

Những tùy chỉnh phổ biến gồm:

Setting color themes. Many, many hours can be spent trying out every single theme that's available online for your particular shell.

Đặt bộ màu (color theme). Bạn có thể đốt cả tá giờ đồng hồ để thử từng bộ màu một có sẵn trên mạng dành cho cái shell cụ thể của mình.

Configuring a prompt . The prompt that tells you the shell is ready for you to type a command can be configured to display just about any information you might want (and a bunch of stuff you'd never want). Personal preferences are everything here: we tend to like simple prompts, with a shortened current directory name and **version control** status along with the time.

Cấu hình dấu nhắc (prompt). Cái dấu nhắc báo cho bạn biết shell đã sẵn sàng nhận lệnh có thể được cấu hình để hiển thị gần như bất cứ thông tin nào bạn muốn (và cả đồng thứ bạn chẳng bao giờ muốn). Ở đây sở thích cá nhân là tất cả: chúng tôi thường thích dấu nhắc gọn gàng, với tên thư mục hiện tại được rút ngắn cùng trạng thái quản lý phiên bản (version control) và giờ giấc.

Aliases and shell functions . Simplify your workflow by turning commands you use a lot into simple aliases. Maybe you regularly update your Linux box, but can never remember whether you update and upgrade, or upgrade and update. Create an alias:

Bí danh (alias) và hàm shell. Hãy đơn giản hóa luồng làm việc bằng cách biến những lệnh bạn dùng nhiều thành các alias gọn nhẹ. Có thể bạn thường xuyên cập nhật máy Linux của mình, nhưng chẳng bao giờ nhớ nổi là update rồi upgrade, hay upgrade rồi update. Vậy thì tạo một alias:

```
alias apt-up= 'sudo apt-get update && sudo apt-get upgrade'
```

```
alias apt-up= 'sudo apt-get update && sudo apt-get upgrade'
```

Maybe you've accidentally deleted files with the rm command just one time too often. Write an alias so that it will always prompt in future:

Có lẽ bạn đã lỡ tay xóa mất tệp bằng lệnh rm thêm một lần nữa — và thế là quá nhiều lần rồi đấy. Hãy viết một alias để từ nay về sau nó luôn hỏi lại bạn:

```
alias rm = 'rm -iv'
```

```
alias rm = 'rm -iv'
```

Command completion . Most shells will complete the names of commands and files: type the first few characters, hit tab, and it'll fill in what it can. But you can take this a lot further, configuring the

Tự động hoàn tất lệnh (command completion). Phần lớn shell sẽ tự hoàn tất tên lệnh và tên tệp: gõ vài ký tự đầu, nhấn tab, nó sẽ điền nốt phần còn lại trong khả năng của nó. Nhưng bạn còn có thể đẩy việc này đi xa hơn nhiều, cấu hình cho

shell to recognize the command you're entering and offer contextspecific completions. Some even customize the completion depending on the current directory.

shell nhận ra lệnh bạn đang nhập và gợi ý những phần hoàn tất phù hợp với ngữ cảnh. Một số shell thậm chí còn tùy biến phần hoàn tất tùy theo thư mục hiện tại.

You'll spend a lot of time living in one of these shells. Be like a hermit crab and make it your own home.

Bạn sẽ sống trong một trong những cái shell này rất nhiều thời gian. Hãy giống như chú cua ẩn sĩ mà biến nó thành mái nhà của riêng mình.

Related Sections Include — Các mục liên quan

Topic 13, Prototypes and Post-it Notes

Topic 13, Bản mẫu và Giấy nhớ Post-it

Topic 16, The Power of Plain Text

Topic 16, Sức mạnh của Văn bản thuần

Topic 21, Text Manipulation

Topic 21, Thao tác trên Văn bản

Topic 30, Transforming Programming

Topic 30, Lập trình Biến đổi

Topic 51, Pragmatic Starter Kit

Topic 51, Bộ khởi đầu Thực dụng

Challenges — Thử thách

Are there things that you're currently doing manually in a GUI? Do you ever pass instructions to colleagues that involve a number of individual “click this button,” “select this item” steps? Could these be automated?

Có việc gì hiện bạn đang làm thủ công trong GUI không? Bạn có bao giờ truyền cho đồng nghiệp những chỉ dẫn gồm một loạt các bước riêng lẻ kiểu “nhấp nút này”, “chọn mục kia” không? Liệu chúng có thể tự động hóa được không?

Whenever you move to a new environment, make a point of finding out what shells are available. See if you can bring your current shell with you.

Mỗi khi chuyển sang một môi trường mới, hãy chú tâm tìm hiểu xem ở đó có những shell nào. Thử xem bạn có thể mang theo cái shell hiện tại của mình đi cùng không.

Investigate alternatives to your current shell. If you come across a problem your shell can't address, see if an alternative shell would cope better.

Hãy khảo sát các lựa chọn thay thế cho shell hiện tại của bạn. Nếu gặp một vấn đề mà shell của bạn không kham nổi, hãy xem liệu một shell khác có xoay sở tốt hơn không.

Topic 18. Power Editing — Topic 18. Soạn thảo Cao thủ

We’ve talked before about tools being an extension of your hand. Well, this applies to editors more than to any other software tool. You need to be able to manipulate text as effortlessly as possible, because text is the basic raw material of programming.

Chúng tôi đã nói trước đây rằng công cụ là phần kéo dài của bàn tay bạn. Điều này đúng với trình soạn thảo (editor) hơn bất kỳ công cụ phần mềm nào khác. Bạn cần thao tác với văn bản (text) thật mượt mà, càng ít tốn sức càng tốt, bởi văn bản chính là thứ nguyên liệu thô cơ bản của nghề lập trình.

In the first edition of this book we recommended using a single editor for everything: code, documentation, memos, system administration, and so on. We’ve softened that position a little. We’re happy for you to use as many editors as you want. We’d just like you to be working toward fluency in each.

Trong lần xuất bản đầu tiên của cuốn sách này, chúng tôi từng khuyên dùng một trình soạn thảo duy nhất cho mọi việc: viết mã, viết tài liệu, ghi chú, quản trị hệ thống, v.v. Nay chúng tôi đã nói lỏng quan điểm đó một chút. Bạn cứ thoải mái dùng bao nhiêu trình soạn thảo tùy thích. Chúng tôi chỉ mong bạn hướng tới sự thành thạo với từng cái một.

Tip 27: Achieve Editor Fluency

Tip 27: Hãy đạt tới độ thuần thục với trình soạn thảo

Why is this a big deal? Are we saying you’ll save lots of time? Actually yes: over the course of a year, you might actually gain an additional week if you make your editing just 4% more efficient and you edit for 20 hours a week.

Tại sao chuyện này lại quan trọng đến vậy? Chúng tôi muốn nói rằng bạn sẽ tiết kiệm được khối thời gian ư? Đúng thế thật: tính ra cả một năm, bạn có thể được lợi thêm tới cả tuần lễ nếu việc soạn thảo của bạn chỉ cần hiệu quả hơn 4% thôi, trong khi bạn ngồi soạn thảo 20 giờ mỗi tuần.

But that’s not the real benefit. No, the major gain is that by becoming fluent, you no longer have to think about the mechanics of editing. The distance between thinking something and having it appear in an editor buffer drop way down. Your thoughts will flow, and your programming will benefit. (If you’ve ever taught someone to drive, then you’ll understand the difference between someone who has to think about every action they take and a more experienced driver who controls the

Nhưng đó chưa phải lợi ích thực sự. Không, cái lợi lớn nhất là khi đã thành thạo, bạn không còn phải bận tâm tới cơ chế soạn thảo nữa. Khoảng cách giữa lúc bạn nghĩ ra điều gì đó và lúc nó hiện lên trong buffer của trình soạn thảo sẽ rút ngắn lại rất nhiều. Dòng suy nghĩ của bạn sẽ trôi chảy, và việc lập trình của bạn sẽ được hưởng lợi. (Nếu bạn từng dạy ai đó lái xe, hẳn bạn hiểu sự khác biệt giữa một người phải nghĩ về từng động tác mình làm với một tài xế lão luyện điều khiển chiếc

car instinctively.)

xe một cách bản năng.)

What Does “Fluent” Mean? — “Thành thạo” nghĩa là sao?

What counts as being fluent? Here’s the challenge list:

Thế nào thì được coi là thành thạo? Đây là danh sách thử thách:

When editing text, move and make selections by character, word, line, and paragraph.

Khi soạn thảo văn bản, di chuyển và chọn theo từng ký tự, từ, dòng, và đoạn.

When editing code, move by various syntactic units (matching delimiters, functions, modules, ...).

Khi soạn thảo mã, di chuyển theo các đơn vị cú pháp khác nhau (cặp dấu phân tách, hàm, mô-đun, ...).

Reindent code following changes.

Thụt lề lại cho mã sau khi chỉnh sửa.

Comment and uncomment blocks of code with a single command.

Thêm và bỏ chú thích (comment) cho cả khối mã chỉ bằng một lệnh.

Undo and redo changes.

Hoàn tác (undo) và làm lại (redo) các thay đổi.

Split the editor window into multiple panels, and navigate between them.

Chia cửa sổ soạn thảo thành nhiều khung và di chuyển qua lại giữa chúng.

Navigate to a particular line number.

Nhảy tới một số dòng cụ thể.

Sort selected lines.

Sắp xếp các dòng đã chọn.

Search for both strings and regular expressions, and repeat previous searches.

Tìm kiếm cả chuỗi lẫn biểu thức chính quy (regular expression), và lặp lại các lần tìm trước đó.

Temporarily create multiple cursors based on a selection or on a pattern match, and edit the text at each in parallel.

Tạm thời tạo nhiều con trỏ dựa trên một vùng chọn hoặc một mẫu khớp (pattern match), rồi sửa văn bản tại từng chỗ cùng lúc.

Display compilation errors in the current project.

Hiển thị các lỗi biên dịch trong dự án hiện tại.

Run the current project's tests.

Chạy các bài kiểm thử của dự án hiện tại.

Can you do all this without using a mouse/trackpad?

Bạn có làm được tất cả những việc này mà không dùng tới chuột/trackpad không?

You might say that your current editor can't do some of these things. Maybe it's time to switch?

Bạn có thể bảo rằng trình soạn thảo hiện tại của bạn không làm được vài việc trong số đó. Vậy thì có lẽ đã đến lúc đổi sang cái khác?

Moving Toward Fluency — Tiến tới sự Thành thạo

We doubt there are more than a handful of people who know all the commands in any particular powerful editor. We don't expect you to, either. Instead, we suggest a more **pragmatic** approach: learn the commands that make your life easier.

Chúng tôi ngờ rằng chẳng có quá vài người biết hết mọi lệnh trong bất kỳ một trình soạn thảo mạnh mẽ nào. Chúng tôi cũng không kỳ vọng bạn làm được vậy. Thay vào đó, chúng tôi gợi ý một hướng tiếp cận thực dụng hơn: học những lệnh giúp cuộc sống của bạn dễ thở hơn.

The recipe for this is fairly simple.

Công thức cho việc này khá đơn giản.

First, look at yourself while you're editing. Every time you find yourself doing something repetitive, get into the habit of thinking "there must be a better way." Then find it.

Đầu tiên, hãy tự quan sát mình khi đang soạn thảo. Mỗi lần thấy bản thân làm một việc lặp đi lặp lại, hãy tập thói quen nghĩ "phải có cách nào hay hơn chứ." Rồi đi tìm cách đó.

Once you've discovered a new, useful feature, you now need to get it installed into your muscle memory, so you can use it without thinking. The only way we know to do that is through repetition. Consciously look for opportunities to use your new superpower, ideally many times a day. After a week or so, you'll find you use it without thinking.

Một khi đã khám phá ra một tính năng mới hữu ích, giờ bạn cần cài nó vào trí nhớ cơ bắp (muscle memory) của mình, để dùng được mà không cần nghĩ ngợi. Cách duy nhất chúng tôi biết để làm điều đó là lặp đi lặp lại. Hãy chú ý tìm cơ hội dùng siêu năng lực mới của mình, lý tưởng nhất là nhiều lần mỗi ngày. Sau chừng một tuần, bạn sẽ thấy mình dùng nó mà chẳng cần suy nghĩ.

Growing Your Editor Most of the powerful code editors are built around a basic core that is then augmented through extensions. Many are supplied with the editor, and others can be added later.

Nuôi lớn Trình soạn thảo của bạn Hầu hết các trình soạn thảo mã mạnh mẽ đều được xây quanh một lõi cơ bản, rồi được bồi đắp thêm thông qua các phần mở rộng (extension). Nhiều cái đi kèm sẵn với trình soạn thảo, số khác có thể thêm vào sau.

When you bump into some apparent limitation of the editor you're using, search around for an extension that will do the job. The chances are that you are not alone in needing that capability, and if you're lucky someone else will have published their solution.

Khi bạn vấp phải một giới hạn nào đó của trình soạn thảo đang dùng, hãy lục tìm xung quanh xem có phần mở rộng nào làm được việc đó không. Nhiều khả năng bạn không phải người duy nhất cần khả năng ấy, và nếu may mắn, sẽ có người khác đã công bố lời giải của họ.

Take this a step further. Dig into your editor's extension language. Work out how to use it to automate some of the repetitive things you do. Often you'll just need a line or two of code.

Hãy tiến thêm một bước nữa. Đào sâu vào ngôn ngữ viết phần mở rộng của trình soạn thảo. Tìm hiểu cách dùng nó để tự động hóa vài việc lặp đi lặp lại mà bạn vẫn làm. Thường thì bạn chỉ cần một hai dòng mã thôi.

Sometimes you might take it further still, and you'll find yourself writing a full-blown extension. If so, **publish** it: if you had a need for it, other people will, too.

Đôi khi bạn có thể đi xa hơn nữa, và rồi thấy mình đang viết cả một phần mở rộng hoàn chỉnh. Nếu vậy, hãy công bố nó: nếu bạn đã cần đến nó, người khác cũng sẽ cần.

Related Sections Include — Các mục liên quan

Topic 7, Communicate!

Topic 7, Hãy Giao tiếp!

Challenges — Thử thách

No more autorepeat. Everyone does it: you need to delete the last word you typed, so you press down on backspace and wait for autorepeat to kick in. In fact, we bet that your brain has done this so much that you can judge pretty much exactly when to release the key. So turn off autorepeat, and instead learn the key sequences to move, select, and delete by characters, words, lines, and blocks.

Bỏ hẳn chế độ tự lặp phím (autorepeat) đi. Ai cũng làm thế cả: bạn cần xóa cái từ vừa gõ, thế là bạn nhấn giữ phím backspace rồi ngồi chờ chế độ tự lặp kích hoạt. Thật ra, chúng tôi dám cá là não bạn đã làm việc này nhiều đến mức bạn canh được gần như chính xác lúc nào thì thả phím. Vậy thì hãy tắt tự lặp phím, và thay vào đó học các tổ hợp phím để di chuyển, chọn và xóa theo ký tự, theo từ, theo dòng và theo khối.

This one is going to hurt. Lose the mouse/trackpad. For one whole week, edit using just the keyboard. You'll discover a bunch of stuff that you can't do without pointing and clicking, so now's the time to learn. Keep notes (we recommend going old-school and using pencil and paper) of the key sequences you learn. You'll take a productivity hit for a few days. But, as you learn to do stuff without moving your hands away from the home position, you'll find that your editing becomes faster and more fluent than it ever was in the past.

Cái này sẽ đau đấy. Bỏ chuột/trackpad đi. Suốt trọn một tuần, hãy soạn thảo chỉ bằng bàn phím. Bạn sẽ phát hiện ra cả đồng thứ không làm được nếu thiếu trỏ-và-nhấp, vậy thì giờ là lúc để học. Hãy ghi chú lại (chúng tôi khuyên cứ theo lối cổ điển, dùng bút chì và giấy) những tổ hợp phím bạn học được. Bạn sẽ bị sụt năng suất trong vài ngày. Nhưng khi đã học được cách làm mọi việc mà không phải rời tay khỏi vị trí gốc trên bàn phím, bạn sẽ thấy việc soạn thảo của mình trở nên nhanh hơn và mượt mà hơn bao giờ hết.

Look for integrations. While writing this chapter, Dave wondered if he could preview the final layout (a PDF file) in an editor buffer. One download later, the layout is sitting alongside the original text, all in the editor. Keep a list of things you'd like to bring into your editor, then look for them.

Hãy tìm các tích hợp. Trong lúc viết chương này, Dave tự hỏi liệu mình có thể xem trước bố cục cuối cùng (một file PDF) ngay trong buffer của trình soạn thảo không. Một lượt tải về sau, bố cục đã nằm ngay cạnh văn bản gốc, tất cả ở trong trình soạn thảo. Hãy giữ một danh sách những thứ bạn muốn đưa vào trình soạn thảo, rồi đi tìm chúng.

Somewhat more ambitiously, if you can't find a plugin or extension that does what you want, write one. Andy is fond of making custom, local file-based Wiki plugins for his favorite editors. If you can't

Tham vọng hơn một chút, nếu không tìm được plugin hay phần mở rộng nào làm đúng điều bạn muốn, hãy tự viết một cái. Andy rất khoái tự làm các plugin Wiki dạng tệp cục bộ tùy biến cho những trình soạn thảo ưa thích của mình. Nếu không

find it, build it!

tìm được nó, hãy tự dựng nó nên!

Progress, far from consisting in change, depends on retentiveness. Those who cannot remember the past are condemned to repeat it. George Santayana, *Life of Reason*

Tiến bộ, không hề nằm ở sự đổi thay, mà phụ thuộc vào khả năng ghi nhớ. Những kẻ không thể nhớ về quá khứ ắt bị kết án phải lặp lại nó. George Santayana, *Life of Reason*

Topic 19. Version Control — Topic 19. Quản lý phiên bản

One of the important things we look for in a user interface is the undo key—a single button that forgives us our mistakes. It's even better if the environment supports multiple levels of undo and redo, so you can go back and recover from something that happened a couple of minutes ago.

Một trong những thứ quan trọng mà chúng tôi tìm kiếm ở một giao diện người dùng là phím undo — một cái nút duy nhất tha thứ cho những sai lầm của ta. Còn tuyệt hơn nữa nếu môi trường đó hỗ trợ undo và redo nhiều cấp, để bạn có thể quay ngược lại và cứu vãn một chuyện đã xảy ra từ vài phút trước.

But what if the mistake happened last week, and you've turned your computer on and off ten times since then? Well, that's one of the many benefits of using a **version control** system (VCS): it's a giant undo key—a project-wide time machine that can return you to those halcyon days of last week, when the code actually compiled and ran.

Nhưng nếu sai lầm ấy xảy ra từ tuần trước, và từ bấy đến giờ bạn đã tắt máy bật máy cả chục lần rồi thì sao? Đó chính là một trong vô vàn lợi ích của việc dùng hệ thống quản lý phiên bản (version control system — VCS): nó là một phím undo khổng lồ — một cỗ máy thời gian cho cả dự án, có thể đưa bạn trở lại những ngày tháng êm đềm của tuần trước, khi mà mã còn biên dịch được và chạy được.

For many folks, that's the limit of their VCS usage. Those folks are missing out on a whole bigger world of collaboration, deployment pipelines, issue tracking, and general team interaction.

Với nhiều người, đó là giới hạn của việc dùng VCS. Những người đó đang bỏ lỡ cả một thế giới rộng lớn hơn nhiều: cộng tác, đường ống triển khai (deployment pipeline), theo dõi vấn đề (issue tracking) và tương tác nhóm nói chung.

So let's take a look at VCS, first as a repository of changes, and then as a central meeting place for your team and their code.

Vậy nên hãy cùng nhìn vào VCS, trước hết như một kho lưu trữ các thay đổi, rồi sau đó như một nơi gặp gỡ trung tâm cho cả nhóm bạn và mã nguồn của họ.

Shared Directories Are NOT Version Control

Thư mục dùng chung KHÔNG phải là quản lý phiên bản

We still come across the occasional team who share their project source files across a network: either internally or using some kind of cloud storage.

Thỉnh thoảng chúng tôi vẫn bắt gặp những nhóm chia sẻ tệp mã nguồn của dự án qua mạng: hoặc trong nội bộ, hoặc dùng một dạng lưu trữ đám mây nào đó.

This is not viable.

Cách này không khả thi.

Teams that do this are constantly messing up each other's work, losing changes, breaking builds, and getting into fist fights in the car park. It's like writing concurrent code with shared data and no synchronization mechanism. Use version control.

Những nhóm làm vậy liên tục phá hỏng công việc của nhau, mất thay đổi, làm vỡ bản build, và đánh nhau túi bụi ngoài bãi đỗ xe. Nó chẳng khác gì viết mã đồng thời với dữ liệu chia sẻ mà chẳng có cơ chế đồng bộ nào. Hãy dùng quản lý phiên bản.

But there's more! Some folks do use version control, and keep their main repository on a network or cloud drive. They reason that this is the best of both worlds: their files are accessible anywhere and (in the case of cloud storage) it's backed up off-site.

Nhưng chưa hết! Một số người có dùng quản lý phiên bản, nhưng lại đặt kho chính (main repository) của mình trên ổ đĩa mạng hoặc đám mây. Họ lý luận rằng đây là điều tốt nhất của cả hai thế giới: tệp của họ truy cập được từ bất cứ đâu, và (trong trường hợp lưu trữ đám mây) còn được sao lưu ở nơi khác.

Turns out that this is even worse, and you risk losing everything. The version control software uses a set of interacting files and directories. If two instances simultaneously make changes, the overall **state** can become corrupted, and there's no telling how much damage will be done. And no one likes seeing developers cry.

Hóa ra cách này còn tệ hơn, và bạn có nguy cơ mất sạch mọi thứ. Phần mềm quản lý phiên bản dùng một tập hợp các tệp và thư mục tương tác lẫn nhau. Nếu hai bản chạy đồng thời cùng tạo thay đổi, trạng thái tổng thể có thể bị hỏng, và chẳng ai biết được thiệt hại sẽ lớn đến đâu. Mà chẳng ai muốn thấy lập trình viên khóc cả.

It Starts At The Source — Mọi thứ bắt đầu từ mã nguồn

Version control systems keep track of every change you make in your source code and documentation. With a properly configured source code control system, you can always go back to a previous version of your software.

Các hệ thống quản lý phiên bản theo dõi mọi thay đổi bạn thực hiện trong mã nguồn và tài liệu. Với một hệ thống quản lý mã nguồn được cấu hình đúng đắn, bạn luôn có thể quay về một phiên bản trước đó của phần mềm.

But a version control system does far more than undo mistakes. A good VCS will let you track changes, answering questions such as: Who made changes in this line of code? What's the difference between the current version and last week's? How many lines of code did we change in this release? Which files get changed most often? This kind of information is invaluable for bug-tracking, audit, performance, and quality purposes.

Nhưng một hệ thống quản lý phiên bản làm được nhiều hơn việc undo sai lầm rất nhiều. Một VCS tốt sẽ cho bạn theo dõi các thay đổi, trả lời những câu hỏi như: Ai đã sửa dòng mã này? Phiên bản hiện tại khác gì so với phiên bản tuần trước? Trong bản phát hành này chúng ta đã thay đổi bao nhiêu dòng mã? Tệp nào bị sửa thường xuyên nhất? Loại thông tin này vô giá cho việc theo dõi lỗi (bug-tracking), kiểm toán, đánh giá hiệu năng và chất lượng.

A VCS will also let you identify releases of your software. Once identified, you will always be able to go back and regenerate the release, independent of changes that may have occurred later.

Một VCS cũng cho bạn định danh các bản phát hành (release) của phần mềm. Một khi đã định danh, bạn luôn có thể quay lại và tái tạo bản phát hành đó, độc lập với những thay

đổi có thể xảy ra sau này.

Version control systems may keep the files they maintain in a central repository—a great candidate for archiving.

Các hệ thống quản lý phiên bản có thể giữ những tệp chúng quản lý trong một kho lưu trữ trung tâm — một ứng viên tuyệt vời cho việc lưu trữ dài hạn.

Finally, version control systems allow two or more users to be working concurrently on the same set of files, even making concurrent changes in the same file. The system then manages the merging of these changes when the files are sent back to the repository. Although seemingly risky, such systems work well in practice on projects of all sizes.

Cuối cùng, các hệ thống quản lý phiên bản cho phép hai hay nhiều người làm việc đồng thời trên cùng một tập hợp tệp, thậm chí cùng tạo thay đổi đồng thời trong cùng một tệp. Hệ thống sau đó sẽ quản lý việc hợp nhất (merge) các thay đổi này khi tệp được gửi trở lại kho. Tuy có vẻ mạo hiểm, những hệ thống như vậy hoạt động tốt trong thực tế trên các dự án ở mọi quy mô.

Tip 28: Always Use Version Control

Tip 28: Luôn luôn dùng quản lý phiên bản

Always. Even if you are a single-person team on a one-week project. Even if it's a “throw-away” **prototype**. Even if the stuff you're working on isn't source code. Make sure that everything is under version control: documentation, phone number lists, memos to vendors, makefiles, build and release procedures, that little **shell** script that tidies up log files—everything. We routinely use version control on just about everything we type (including the text of this book). Even if we're not working on a project, our day-to-day work is secured in a repository.

Luôn luôn. Ngay cả khi bạn là nhóm một người trên một dự án kéo dài một tuần. Ngay cả khi đó là một bản mẫu (prototype) “vứt-đi”. Ngay cả khi thứ bạn đang làm chẳng phải mã nguồn. Hãy đảm bảo mọi thứ đều nằm dưới sự quản lý phiên bản: tài liệu, danh sách số điện thoại, ghi nhớ gửi nhà cung cấp, makefile, quy trình build và phát hành, cái shell script bé tí dọn dẹp tệp log — tất tần tật. Chúng tôi thường xuyên dùng quản lý phiên bản cho gần như mọi thứ mình gõ ra (kể cả phần văn bản của cuốn sách này). Ngay cả khi không làm dự án, công việc hằng ngày của chúng tôi vẫn được giữ an toàn trong một kho lưu trữ.

Branching Out — Phân nhánh

Version control systems don't just keep a single history of your project. One of their most powerful and useful features is the way they let you isolate islands of development into things called branches. You can create a branch at any point in your project's history, and any work you do in that branch will be isolated from all other branches. At some time in the future you can merge the branch you're working on back into another branch, so the target branch now contains the changes you made in your branch. Multiple people can even be working on a

Các hệ thống quản lý phiên bản không chỉ giữ một lịch sử duy nhất cho dự án của bạn. Một trong những tính năng mạnh mẽ và hữu dụng nhất của chúng là cách chúng cho bạn cô lập từng hòn đảo phát triển vào những thứ gọi là nhánh (branch). Bạn có thể tạo một nhánh tại bất kỳ điểm nào trong lịch sử dự án, và mọi việc bạn làm trong nhánh đó sẽ được cô lập khỏi tất cả các nhánh khác. Vào một lúc nào đó trong tương lai, bạn có thể hợp nhất nhánh mình đang làm trở lại vào một nhánh khác, để nhánh đích giờ chứa luôn những

thay đổi bạn đã thực hiện trong nhánh của mình. Thậm chí nhiều người có thể cùng làm việc trên một

branch: in a way, branches are like little clone projects.

nhánh: theo một cách nào đó, các nhánh giống như những dự án nhân bản tí hon.

One benefit of branches is the isolation they give you. If you develop feature A in one branch, and a teammate works on feature B in another, you're not going to interfere with each other.

Một lợi ích của nhánh là sự cô lập mà chúng mang lại. Nếu bạn phát triển tính năng A trong một nhánh, còn đồng đội làm tính năng B trong một nhánh khác, hai bạn sẽ không giẫm chân lên nhau.

A second benefit, which may be surprising, is that branches are often at the heart of a team's project workflow.

Lợi ích thứ hai, có lẽ gây bất ngờ, là các nhánh thường nằm ở trung tâm quy trình làm việc của cả nhóm trên dự án.

And this is where things get a little confusing. Version control branches and test organization have something in common: they both have thousands of people out there telling you how you should do it. And that advice is largely meaningless, because what they're really saying is "this is what worked for me."

Và đây là chỗ mọi thứ trở nên hơi rối. Nhánh quản lý phiên bản và cách tổ chức kiểm thử có một điểm chung: cả hai đều có hàng nghìn người ngoài kia bảo bạn nên làm thế nào. Mà lời khuyên ấy phần lớn vô nghĩa, bởi điều họ thực sự đang nói là "đây là cái đã hiệu quả với tôi".

So use version control in your project, and if you bump into workflow issues, search for possible solutions. And remember to review and adjust what you're doing as you gain experience.

Vậy nên hãy dùng quản lý phiên bản trong dự án của bạn, và nếu vướng vào vấn đề quy trình, hãy tìm kiếm các giải pháp khả dĩ. Và nhớ rà soát rồi điều chỉnh những gì bạn đang làm khi tích lũy thêm kinh nghiệm.

A Thought Experiment

Một thí nghiệm tưởng tượng

Spill an entire cup of tea (English breakfast, with a little milk) onto your laptop keyboard. Take the machine to the smart-person bar, and have them tut and frown. Buy a new computer. Take it home.

Hãy hắt cả một tách trà (loại English breakfast, thêm chút sữa) lên bàn phím laptop của bạn. Mang máy đến quầy của mấy chuyên gia, để họ tặc lưỡi và cau mày. Mua một máy tính mới. Mang về nhà.

How long would it take to get that machine back to the same state it was in (with all the SSH keys, editor **configuration**, shell setup, installed applications, and so on) at the point where you first lifted that fateful cup? This was an issue one of us faced recently.

Sẽ mất bao lâu để đưa máy đó trở lại đúng trạng thái như lúc trước (với tất cả khóa SSH, cấu hình trình soạn thảo, thiết lập shell, các ứng dụng đã cài đặt, v.v.) tại thời điểm bạn vừa nhấc cái tách trà định mệnh kia lên? Đây là tình huống mà một trong hai chúng tôi vừa gặp gần đây.

Just about everything that defined the configuration and usage of the original machine was stored in version control, including:

Gần như mọi thứ định nghĩa nên cấu hình và cách sử dụng của chiếc máy gốc đều đã được lưu trong quản lý phiên bản, bao gồm:

All the user preferences and dotfiles

Tất cả các tùy chọn người dùng và dotfile

The editor configuration

Cấu hình trình soạn thảo

The list of software installed using Homebrew

Danh sách phần mềm đã cài đặt bằng Homebrew

The Ansible script used to configure apps

Script Ansible dùng để cấu hình các ứng dụng

All current projects

Tất cả các dự án hiện tại

The machine was restored by the end of the afternoon.

Chiếc máy đã được khôi phục xong trước khi hết buổi chiều.

Version Control As A Project Hub — Quản lý phiên bản như một trung tâm điều phối dự án

Although version control is incredibly useful on personal projects, it really comes into its own when working with a team. And much of this value comes from how you host your repository.

Tuy quản lý phiên bản cực kỳ hữu dụng trên các dự án cá nhân, nó mới thực sự phát huy hết sức mạnh khi làm việc với cả một nhóm. Và phần lớn giá trị ấy đến từ cách bạn lưu trữ (host) kho của mình.

Now, many version control systems don't need any hosting. They are completely decentralized, with each developer cooperating on a peer-to-peer basis. But even with these systems, it's worth looking into having a central repository, because once you do, you can take advantage of a ton of integrations to make the project flow easier.

Giờ thì, nhiều hệ thống quản lý phiên bản chẳng cần lưu trữ tập trung gì cả. Chúng hoàn toàn phi tập trung, với mỗi lập trình viên cộng tác trên cơ sở ngang hàng (peer-to-peer). Nhưng ngay cả với những hệ thống này, vẫn đáng để cân nhắc có một kho trung tâm, bởi một khi có nó, bạn có thể tận dụng cả tá tích hợp để dòng chảy công việc của dự án trôi chảy hơn.

Many of the repository systems are open source, so you can install and run them in your company. But that's not really your line of business, so we'd recommend most people host with a third party. Look for features such as:

Nhiều hệ thống kho lưu trữ là mã nguồn mở, nên bạn có thể tự cài và chạy chúng trong công ty mình. Nhưng đó không thực sự là lĩnh vực kinh doanh của bạn, nên chúng tôi khuyên hầu hết mọi người nên lưu trữ với một bên thứ ba. Hãy tìm những tính năng như:

Good security and access control

Bảo mật và kiểm soát truy cập tốt

Intuitive UI

Giao diện trực quan

The ability to do everything from the command line, too (because you may need to automate it)

Khả năng làm được mọi thứ từ dòng lệnh nữa (vì có thể bạn sẽ cần tự động hóa nó)

Automated builds and tests

Build và kiểm thử tự động

Good support for branch merging (sometimes called pull requests)

Hỗ trợ tốt cho việc hợp nhất nhánh (đôi khi gọi là pull request)

Issue management (ideally integrated into commits and merges, so

Quản lý vấn đề (lý tưởng là tích hợp vào các commit và merge, để

you can keep metrics)

bạn có thể giữ được số liệu thống kê)

Good reporting (a Kanban board-like display of pending issues and tasks can be very useful)

Báo cáo tốt (một bảng hiển thị kiểu Kanban cho các vấn đề và việc đang chờ xử lý có thể rất hữu ích)

Good team communications: emails or other notifications on changes, a wiki, and so on

Giao tiếp nhóm tốt: email hoặc các thông báo khác khi có thay đổi, một wiki, v.v.

Many teams have their VCS configured so that a push to a particular branch will automatically build the system, run the tests, and if successful deploy the new code into production.

Nhiều nhóm cấu hình VCS của họ sao cho việc push lên một nhánh nhất định sẽ tự động build hệ thống, chạy các bài kiểm thử, và nếu thành công thì triển khai mã mới vào môi trường production.

Sound scary? Not when you realize you're using version control. You can always roll it back.

Nghe đáng sợ chứ? Không hề, một khi bạn nhận ra mình đang dùng quản lý phiên bản. Bạn luôn có thể quay ngược (roll back) lại.

Related Sections Include — Các mục liên quan

Topic 11, **Reversibility**

Topic 11, Tính khả nghịch

Topic 49, **Pragmatic** Teams

Topic 49, Nhóm thực dụng

Topic 51, Pragmatic **Starter Kit**

Topic 51, Bộ khởi đầu thực dụng

Challenges — Thử thách

Knowing you can roll back to any previous state using the VCS is one thing, but can you actually do it? Do you know the commands to do it properly? Learn them now, not when disaster strikes and you're under pressure.

Biết rằng mình có thể quay ngược về bất kỳ trạng thái trước đó bằng VCS là một chuyện, nhưng bạn có thực sự làm được không? Bạn có biết những lệnh để làm việc đó cho đúng cách không? Hãy học chúng ngay bây giờ, đừng đợi đến khi thảm họa ập đến và bạn đang chịu áp lực.

Spend some time thinking about recovering your own laptop environment in case of a disaster. What would you need to recover? Many of the things you need are just text files. If they're not in a VCS (hosted off your laptop), find a way to add them. Then think about the other stuff: installed applications, system configuration, and so on. How can you express all that stuff in text files so it, too, can be saved?

Hãy dành chút thời gian nghĩ về việc khôi phục lại chính môi trường laptop của bạn trong trường hợp gặp thảm họa. Bạn sẽ cần khôi phục những gì? Phần lớn những thứ bạn cần chỉ là các tệp văn bản. Nếu chúng chưa nằm trong VCS (được lưu trữ bên ngoài laptop), hãy tìm cách thêm chúng vào. Rồi nghĩ tới những thứ còn lại: các ứng dụng đã cài, cấu hình hệ thống, v.v. Làm sao bạn có thể biểu diễn tất cả những thứ đó dưới dạng tệp văn bản để chúng cũng được lưu lại?

An interesting experiment, once you've made some progress, is to find an old computer you no longer use and see if your new system can be used to set it up.

Một thí nghiệm thú vị, sau khi bạn đã có chút tiến triển, là tìm một máy tính cũ bạn không còn dùng nữa và xem liệu hệ thống mới của bạn có dùng được để thiết lập lại nó không.

Consciously explore the features of your current VCS and hosting provider that you're not using. If your team isn't using feature branches, experiment with introducing them. The same with pull/merge requests. Continuous integration. Build pipelines. Even continuous deployment. Look into the team communication tools, too: wikis, Kanban boards, and the like. You don't have to use any of it. But you do need to know what it does so you can make that decision.

Hãy chủ động khám phá những tính năng của VCS và nhà cung cấp lưu trữ hiện tại mà bạn chưa dùng đến. Nếu nhóm bạn chưa dùng nhánh tính năng (feature branch), hãy thử nghiệm đưa chúng vào. Tương tự với pull/merge request. Tích hợp liên tục (continuous integration). Đường ống build. Thậm chí cả triển khai liên tục (continuous deployment). Hãy ngó qua cả các công cụ giao tiếp nhóm nữa: wiki, bảng Kanban, và những thứ tương tự. Bạn không bắt buộc phải dùng bất kỳ thứ nào trong số đó. Nhưng bạn cần biết chúng làm được gì để có thể đưa ra quyết định ấy.

Use version control for nonproject things, too.

Hãy dùng quản lý phiên bản cho cả những thứ ngoài dự án nữa.

It is a painful thing To look at your own trouble and know That you yourself and no one else has made it Sophocles, Ajax

Thật là một điều đau đớn Khi nhìn vào nỗi khổ của chính mình và biết rằng Chính bạn, chứ không ai khác, đã tự gây ra nó Sophocles, Ajax

Topic 20. Debugging — Topic 20. Gỡ lỗi

The word bug has been used to describe an “object of terror” ever since the fourteenth century. Rear Admiral Dr. Grace Hopper, the inventor of COBOL, is credited with observing the first computer bug—literally, a moth caught in a relay in an early computer system. When asked to explain why the machine wasn't behaving as intended, a technician reported that there was “a bug in the system,” and dutifully taped it—wings and all—into the log book.

Từ “bug” (con bọ) đã được dùng để chỉ một “thứ gây kinh hãi” từ tận thế kỷ mười bốn. Chuẩn Đô đốc, Tiến sĩ Grace Hopper — người phát minh ra COBOL — được ghi nhận là người đầu tiên quan sát thấy một con bug máy tính: theo nghĩa đen, một con bướm đêm mắc kẹt trong một rơ-le của một hệ thống máy tính thời sơ khai. Khi được yêu cầu giải thích vì sao chiếc máy không hoạt động như ý, một kỹ thuật viên báo cáo rằng có “một con bug trong hệ thống”, rồi cẩn thận dán nó — nguyên cả cánh — vào cuốn sổ nhật ký.

Regrettably, we still have bugs in the system, albeit not the flying kind. But the fourteenth century meaning—a bogeyman—is perhaps even more applicable now than it was then. Software defects manifest themselves in a variety of ways, from misunderstood **requirements** to coding errors. Unfortunately, modern computer systems are still limited to doing what you tell them to do, not necessarily what you want them to do.

Đáng tiếc là chúng ta vẫn còn bug trong hệ thống, dù không phải loại biết bay. Nhưng cái nghĩa từ thế kỷ mười bốn — một con ngáo ộp — có lẽ giờ còn đúng hơn cả ngày xưa. Khiếm khuyết phần mềm biểu hiện theo đủ kiểu, từ hiểu sai yêu cầu cho tới lỗi viết mã. Thật không may, máy tính hiện đại vẫn chỉ làm đúng những gì bạn bảo nó làm, chứ không nhất thiết là những gì bạn muốn nó làm.

No one writes perfect software, so it’s a given that **debugging** will take up a major portion of your day. Let’s look at some of the issues involved in debugging and some general strategies for finding elusive bugs.

Chẳng ai viết được phần mềm hoàn hảo cả, nên cứ coi như chuyện gỡ lỗi (debugging) sẽ tốn một phần lớn thời gian trong ngày của bạn là điều hiển nhiên. Hãy cùng xem xét vài vấn đề liên quan đến gỡ lỗi và một số chiến lược chung để truy lùng những con bug khó nắm bắt.

Psychology Of Debugging — Tâm lý học của việc gỡ lỗi

Debugging is a sensitive, emotional subject for many developers. Instead of attacking it as a puzzle to be solved, you may encounter denial, finger pointing, lame excuses, or just plain apathy.

Gỡ lỗi là một chủ đề nhạy cảm, dễ động chạm cảm xúc với nhiều lập trình viên. Thay vì xông vào nó như một câu đố cần giải, bạn có thể gặp phải đủ thứ: chối bay chối biến, đổ lỗi qua lại, viện cớ lảng nhách, hay đơn giản là thái độ mặc kệ.

Embrace the fact that debugging is just problem solving , and attack it as such.

Hãy chấp nhận một sự thật: gỡ lỗi chỉ là giải quyết vấn đề, và hãy tiếp cận nó đúng như vậy.

Having found someone else’s bug, you can spend time and energy laying blame on the filthy culprit who created it. In some workplaces this is part of the culture, and may be cathartic. However, in the technical arena, you want to concentrate on fixing the problem , not the blame.

Khi tìm ra một con bug của người khác, bạn có thể tốn thời gian và công sức để đổ lỗi cho cái kẻ bẩn thỉu đã gây ra nó. Ở một số nơi làm việc, đây là một phần của văn hóa và có thể giúp xả stress. Tuy nhiên, trong địa hạt kỹ thuật, điều bạn nên tập trung là sửa vấn đề, chứ không phải sửa trách nhiệm.

Tip 29: Fix the Problem, Not the Blame

Tip 29: Hãy sửa vấn đề, đừng sửa trách nhiệm

It doesn’t really matter whether the bug is your fault or someone else’s. It is still your problem.

Con bug là lỗi của bạn hay của ai khác thật ra chẳng quan trọng. Dù sao thì nó vẫn là vấn đề của bạn.

A Debugging Mindset — Tư duy để gỡ lỗi

Before you start debugging, it's important to adopt the right mindset. You need to turn off many of the defenses you use each day to protect your ego, tune out any project pressures you may be under, and get yourself comfortable. Above all, remember the first rule of debugging:

Trước khi bắt tay vào gỡ lỗi, điều quan trọng là phải có một tư duy đúng đắn. Bạn cần tắt bớt phần nhiều những cơ chế phòng vệ mà bạn dùng mỗi ngày để bảo vệ cái tôi của mình, gạt sang một bên mọi áp lực dự án mà bạn đang gánh, và để bản thân thoải mái. Trên hết, hãy nhớ quy tắc đầu tiên của việc gỡ lỗi:

Tip 30: Don't Panic

Tip 30: Đừng hoảng loạn

It's easy to get into a panic, especially if you are facing a deadline, or have a nervous boss or **client** breathing down your neck while you are trying to find the cause of the bug. But it is very important to step back a pace, and actually think about

Rất dễ rơi vào trạng thái hoảng loạn, nhất là khi bạn đang cận kề một deadline, hoặc có một ông sếp hay một khách hàng đứng thở phì phò sau gáy trong lúc bạn cố tìm nguyên nhân của con bug. Nhưng điều cực kỳ quan trọng là phải lùi lại một bước, và thật sự suy nghĩ về

what could be causing the symptoms that you believe indicate a bug.

điều gì có thể đang gây ra những triệu chứng mà bạn tin là dấu hiệu của một con bug.

If your first reaction on witnessing a bug or seeing a bug report is “that’s impossible,” you are plainly wrong. Don’t waste a single neuron on the train of thought that begins “but that can’t happen” because quite clearly it can, and has.

Nếu phản ứng đầu tiên của bạn khi chứng kiến một con bug hay đọc một báo cáo bug là “điều đó là không thể”, thì bạn rõ ràng đã sai. Đừng phí một tế bào thần kinh nào cho cái dòng suy nghĩ bắt đầu bằng “nhưng chuyện đó không thể xảy ra được” — bởi rõ ràng là nó có thể, và nó đã xảy ra rồi.

Beware of myopia when debugging. Resist the urge to fix just the symptoms you see: it is more likely that the actual fault may be several steps removed from what you are observing, and may involve a number of other related things. Always try to discover the root cause of a problem, not just this particular appearance of it.

Hãy cảnh giác với tật cận thị khi gỡ lỗi. Đừng để cái thôi thúc chỉ sửa đúng những triệu chứng bạn nhìn thấy lấn át bạn: nhiều khả năng lỗi thật sự nằm cách chỗ bạn quan sát vài bước, và có thể dính dáng tới một loạt thứ liên quan khác. Hãy luôn cố gắng tìm ra nguyên nhân gốc rễ của vấn đề, chứ không phải chỉ cái biểu hiện cụ thể lần này của nó.

Where To Start — Bắt đầu từ đâu

Before you start to look at the bug, make sure that you are working on code that built cleanly—without warnings. We routinely set compiler warning levels as high as possible. It doesn’t make sense to waste time trying to find a problem that the computer could find for you! We need to concentrate on the harder problems at hand.

Trước khi bắt tay xem xét con bug, hãy chắc chắn rằng bạn đang làm việc trên mã được build sạch — không có cảnh báo nào. Chúng tôi luôn thường trực đặt mức cảnh báo của trình biên dịch cao nhất có thể. Chẳng có lý do gì để phí thời gian đi tìm một vấn đề mà chính cái máy có thể tìm hộ bạn! Chúng ta cần dành sức cho những bài toán khó nhằn hơn đang chờ phía trước.

When trying to solve any problem, you need to gather all the relevant data. Unfortunately, bug reporting isn't an exact science. It's easy to be misled by coincidences, and you can't afford to waste time debugging coincidences. You first need to be accurate in your observations.

Khi cố giải bất kỳ vấn đề nào, bạn cần thu thập tất cả dữ liệu liên quan. Tiếc thay, báo cáo bug không phải là một môn khoa học chính xác. Rất dễ bị dẫn dắt lạc lối bởi những sự trùng hợp, và bạn không thể phí thời gian đi gỡ lỗi cho các trùng hợp ngẫu nhiên. Trước hết bạn cần chính xác trong những gì mình quan sát.

Accuracy in bug reports is further diminished when they come through a third party—you may actually need to watch the user who reported the bug in action to get a sufficient level of detail.

Độ chính xác trong các báo cáo bug còn giảm thêm khi chúng đi qua một bên thứ ba — bạn có thể sẽ phải tận mắt quan sát người dùng đã báo lỗi đang thao tác để nắm đủ mức chi tiết cần thiết.

Andy once worked on a large graphics application. Nearing release, the testers reported that the application crashed every time they painted a stroke with a particular brush. The programmer responsible argued that there was nothing wrong with it; he had tried painting with it, and it worked just fine. This dialog went back and forth for several days, with tempers rapidly rising.

Có lần Andy làm việc trên một ứng dụng đồ họa lớn. Gần đến ngày phát hành, các tester báo rằng ứng dụng cứ bị crash mỗi khi họ quét một nét bằng một loại cọ cụ thể. Lập trình viên phụ trách cãi rằng chẳng có gì sai cả; anh ta đã thử vẽ bằng cọ đó, và nó chạy ngon lành. Cuộc đôi co này qua lại suốt mấy ngày, với cái đầu hai bên ngày một nóng.

Finally, we got them together in the same room. The tester selected the brush tool and painted a stroke from the upper right corner to the lower left corner. The application exploded. “Oh,” said the programmer, in a small voice, who then sheepishly admitted that he had made test strokes only from the lower left to the upper right, which did not expose the bug.

Cuối cùng, chúng tôi gom họ lại trong cùng một phòng. Anh tester chọn công cụ cọ và quét một nét từ góc trên bên phải xuống góc dưới bên trái. Ứng dụng nổ tung. “Ồ,” lập trình viên nói khẽ, rồi ngượng ngùng thừa nhận rằng anh ta chỉ thử quét nét từ dưới trái lên trên phải, mà cách đó không làm lộ con bug ra.

There are two points to this story:

Câu chuyện này có hai điểm cần rút ra:

You may need to interview the user who reported the bug in order to gather more data than you were initially given.

Bạn có thể cần phỏng vấn người dùng đã báo bug để thu thập thêm dữ liệu, nhiều hơn so với những gì bạn được cung cấp ban đầu.

Artificial tests (such as the programmer's single brush stroke from bottom to top) don't exercise enough of an application. You must brutally test both boundary conditions and realistic end-user usage patterns. You need to do this systematically (see Ruthless and Continuous Testing).

Các phép thử giả tạo (như cái nét cọ duy nhất từ dưới lên trên của lập trình viên kia) không vận hành đủ rộng phần ứng dụng. Bạn phải kiểm thử một cách tàn nhẫn cả các điều kiện

biên lẫn những kiểu sử dụng thực tế của người dùng cuối. Bạn cần làm việc này một cách có hệ thống (xem Kiểm thử Không khoan nhượng và Liên tục).

Debugging Strategies — Các chiến lược gỡ lỗi

Once you think you know what is going on, it's time to find out what the program thinks is going on.

Một khi bạn nghĩ là mình đã biết chuyện gì đang xảy ra, đã đến lúc tìm hiểu xem chương trình nghĩ rằng chuyện gì đang xảy ra.

Reproducing Bugs No, our bugs aren't really multiplying (although some of them are probably old enough to do it legally). We're talking about a different kind of reproduction.

Tái tạo bug — Không, đám bug của chúng ta không thật sự đang sinh sản đâu (mặc dù vài con chắc cũng đủ tuổi để làm chuyện đó một cách hợp pháp rồi). Chúng tôi đang nói về một kiểu “sinh sản” khác.

The best way to start fixing a bug is to make it reproducible. After all, if you can't reproduce it, how will you know if it is ever fixed?

Cách tốt nhất để bắt đầu sửa một con bug là làm cho nó có thể tái tạo được. Suy cho cùng, nếu không tái tạo được nó, làm sao bạn biết được khi nào nó đã được sửa?

But we want more than a bug that can be reproduced by following some long series of steps; we want a bug that can be reproduced with a single command . It's a lot harder to fix a bug if you have to go through 15 steps to get to the point where the bug shows up.

Nhưng chúng ta muốn nhiều hơn một con bug chỉ tái tạo được bằng cách lần theo một chuỗi dài các bước; chúng ta muốn một con bug tái tạo được chỉ với một câu lệnh duy nhất. Sửa một con bug sẽ khó hơn nhiều nếu bạn phải đi qua 15 bước mới tới được cái điểm bug lộ ra.

So here's the most important rule of debugging:

Vậy nên đây là quy tắc quan trọng nhất của việc gỡ lỗi:

Tip 31: Failing Test Before Fixing Code

Tip 31: Bài kiểm thử fail trước, sửa mã sau

Sometimes by forcing yourself to isolate the circumstances that display the bug, you'll even gain an insight on how to fix it. The act of writing the test informs the solution.

Đôi khi, bằng cách ép mình cô lập đúng những tình huống làm con bug lộ diện, bạn thậm chí sẽ vỡ ra cách sửa nó. Chính cái việc viết bài kiểm thử lại gợi ý cho lời giải.

Coder In A Strange Land — Lập trình viên ở miền đất lạ

All this talk about isolating the bug is fine, when faced with 50,000 lines of code and a ticking clock, what's a poor coder to do?

Tất cả những lời bàn về việc cô lập con bug thì hay ho đấy, nhưng khi đối mặt với 50.000 dòng mã và một chiếc đồng hồ đang tích tắc, một lập trình viên khốn khổ biết phải làm gì đây?

First, look at the problem. Is it a crash? It's always surprising when we teach courses that involve programming how many developers see an **exception** pop up in red and immediately tab across to the code.

Đầu tiên, hãy nhìn vào vấn đề. Có phải là crash không? Lúc dạy các khóa học liên quan đến lập trình, chúng tôi luôn thấy bất ngờ là có bao nhiêu lập trình viên để thấy một ngoại lệ (exception) hiện lên màu đỏ là lập tức nhảy ngay sang phần mã.

Tip 32: Read the Damn Error Message

Tip 32: Hãy đọc cái thông báo lỗi chết tiệt ấy đi

'nuf said.

Khỏi nói thêm.

Bad Results What if it's not a crash? What if it's just a bad result?

Kết quả sai — Còn nếu không phải crash thì sao? Nếu nó chỉ đơn giản là cho ra một kết quả sai thì sao?

Get in there with a debugger and use your failing test to trigger the problem.

Hãy nhảy vào đó với một trình gỡ lỗi (debugger) và dùng bài kiểm thử lỗi của bạn để kích hoạt vấn đề.

Before anything else, make sure that you're also seeing the incorrect value in the debugger. We've both wasted hours trying to track down a bug only to discover that this particular run of the code worked fine.

Trước hết mọi thứ, hãy chắc chắn rằng bạn cũng đang thấy đúng cái giá trị sai đó trong trình gỡ lỗi. Cả hai chúng tôi đều từng phí hàng giờ cố truy một con bug, để rồi phát hiện ra rằng cái lần chạy mã cụ thể này lại chạy hoàn toàn ổn.

Sometimes the problem is obvious: `interest_rate` is 4.5 and should be 0.045 . More often you have to look deeper to find out why the value is wrong in the first place. Make sure you know how to move up and down the call stack and examine the local stack environment.

Đôi khi vấn đề hiển nhiên: `interest_rate` đang là 4.5 trong khi lẽ ra phải là 0.045. Nhưng thường xuyên hơn, bạn phải đào sâu hơn nữa mới tìm ra vì sao ngay từ đầu giá trị ấy đã sai. Hãy chắc rằng bạn biết cách đi lên đi xuống dọc ngăn xếp lời gọi (call stack) và xem xét môi trường ngăn xếp cục bộ.

We find it often helps to keep pen and paper nearby so we can jot down notes. In particular we often come across a clue and chase it down, only to find it didn't pan out. If we didn't jot down where we were when we started the chase, we could lose a lot of time getting back there.

Chúng tôi thấy rằng để sẵn bút và giấy bên cạnh thường rất có ích, để có thể ghi vội vài dòng. Cụ thể là chúng tôi hay vớ được một manh mối rồi đuổi theo nó, chỉ để rồi thấy nó chẳng đi đến đâu. Nếu lúc bắt đầu cuộc rượt đuổi mà không ghi lại mình đang đứng ở đâu, ta có thể mất khối thời gian để quay lại được chỗ cũ.

Sometimes you're looking at a stack trace that seems to scroll on forever. In this case, there's often a quicker way to find the problem than examining each and every stack frame: use a **binary chop** . But before we discuss that, let's look at two other common bug scenarios.

Đôi khi bạn đang nhìn một stack trace dài như thể cuộn mãi không hết. Trong trường hợp này, thường có một cách tìm vấn đề nhanh hơn là sẫm soi từng khung ngăn xếp một: dùng phép chia đôi nhị phân (binary chop). Nhưng trước khi bàn về nó, hãy xem qua hai kịch bản bug phổ biến khác.

Sensitivity to Input Values You've been there. Your program works fine with all the test data, and survives its first week in production with honor. Then it suddenly crashes when fed a particular

dataset.

Nhảy cảm với giá trị đầu vào — Bạn từng ở trong tình huống này rồi. Chương trình của bạn chạy ngon lành với mọi dữ liệu thử nghiệm, và sống sót qua tuần đầu tiên trong môi trường vận hành một cách vẻ vang. Rồi đột nhiên nó crash khi được nạp một tập dữ liệu cụ thể.

You can try looking at the place it crashes and work backwards. But sometimes it's easier to start with the data. Get a copy of the dataset and feed it through a locally running copy of the app, making sure it still crashes. Then binary chop the data until you isolate exactly which input values are leading to the crash.

Bạn có thể thử nhìn vào chỗ nó crash rồi lần ngược lại. Nhưng đôi khi sẽ dễ hơn nếu bắt đầu từ dữ liệu. Lấy một bản sao của tập dữ liệu rồi nạp nó qua một bản app chạy cục bộ, chắc chắn rằng nó vẫn crash. Sau đó chia đôi nhị phân tập dữ liệu cho đến khi bạn cô lập được chính xác những giá trị đầu vào nào dẫn đến crash.

Regressions Across Releases You're on a good team, and you release your software into production. At some point a bug pops up in code that worked OK a week ago. Wouldn't it be nice if you could identify the specific change that introduced it? Guess what? Binary chop time.

Hồi quy giữa các bản phát hành — Bạn đang ở trong một đội tốt, và bạn phát hành phần mềm của mình lên môi trường vận hành. Tại một thời điểm nào đó, một con bug bỗng xuất hiện trong đoạn mã mà tuần trước còn chạy ổn. Chẳng phải sẽ tuyệt sao nếu bạn có thể xác định được đúng cái thay đổi đã gây ra nó? Đoán xem? Lại đến lúc chia đôi nhị phân rồi đấy.

The Binary Chop — Phép chia đôi nhị phân

Every CS undergraduate has been forced to code a binary chop (sometimes called a **binary search**). The idea is simple. You're looking for a particular value in a sorted array. You could just look at each value in turn, but you'd end up looking at roughly half the entries on average until you either found the value you wanted, or you found a value greater than it, which would mean the value's not in the array.

Mọi sinh viên đại học ngành Khoa học Máy tính đều từng bị bắt phải viết một phép chia đôi nhị phân (binary chop, đôi khi gọi là tìm kiếm nhị phân — binary search). Ý tưởng rất đơn giản. Bạn đang tìm một giá trị cụ thể trong một mảng đã sắp xếp. Bạn có thể cứ xem lần lượt từng giá trị, nhưng trung bình bạn sẽ phải xem khoảng một nửa số phần tử cho đến khi hoặc tìm được giá trị mình muốn, hoặc gặp một giá trị lớn hơn nó — nghĩa là giá trị đó không có trong mảng.

But it's faster to use a **divide and conquer** approach. Choose a value in the middle of the array. If it's the one you're looking for, stop. Otherwise you can chop the array in two. If the value you find is greater than the target then you know it must be in the first half of the array, otherwise it's in the second half. Repeat the procedure in the appropriate subarray, and in no time you'll have a result. (As we'll see when we talk about **Big-O Notation**, a linear search is , and a binary chop is).

Nhưng dùng cách chia để trị thì nhanh hơn. Chọn một giá trị ở giữa mảng. Nếu nó đúng là cái bạn đang tìm, thì dừng lại. Bằng không, bạn có thể chặt mảng làm đôi. Nếu giá trị bạn vừa tìm thấy lớn hơn mục tiêu thì bạn biết mục tiêu phải nằm ở nửa đầu của mảng, ngược lại thì nó ở nửa sau. Lập lại quy trình trong mảng con tương ứng, và chỉ trong nháy mắt bạn sẽ có kết quả. (Như chúng ta sẽ thấy khi bàn về Ký hiệu Big-O, một phép tìm kiếm tuyến tính là $O(n)$, còn một phép chia đôi nhị phân là $O(\log n)$).

So, the binary chop is way, way faster on any decent sized problem. Let's see how to apply it to debugging.

Vậy nên, với bất kỳ bài toán có kích thước kha khá nào, phép chia đôi nhị phân nhanh hơn rất, rất nhiều. Hãy xem cách áp dụng nó vào việc gỡ lỗi.

When you're facing a massive stacktrace and you're trying to find out exactly which function mangled the value in error, you do a chop by choosing a stack frame somewhere in the middle and seeing if the error is manifest there. If it is, then you know to focus on the frames before, otherwise the problem is in the frames after. Chop again. Even if you have 64 frames in the stacktrace, this approach will give you an answer after at most six attempts.

Khi bạn đối mặt với một stack trace khổng lồ và đang cố tìm xem chính xác hàm nào đã làm hỏng cái giá trị bị lỗi, bạn thực hiện một phép chia bằng cách chọn một khung ngăn xếp đầu đó ở giữa và xem lỗi đã biểu hiện ở đó chưa. Nếu có rồi, thì bạn biết phải tập trung vào các khung phía trước, bằng không thì vấn đề nằm ở các khung phía sau. Lại chia tiếp. Cho dù bạn có 64 khung trong stack trace, cách làm này sẽ cho bạn câu trả lời sau nhiều nhất sáu lần thử.

If you find bugs that appear on certain datasets, you might be able to do the same thing. Split the dataset into two, and see if the problem occurs if you feed one or the other through the app. Keep dividing the data until you get a minimum set of values that exhibit the problem.

Nếu bạn gặp những con bug chỉ xuất hiện với một số tập dữ liệu nhất định, bạn có thể làm điều tương tự. Chia tập dữ liệu làm hai, rồi xem vấn đề có xảy ra không khi bạn nạp nửa này hay nửa kia qua app. Cứ tiếp tục chia nhỏ dữ liệu cho đến khi bạn có được một tập giá trị tối thiểu vẫn còn thể hiện vấn đề.

If your team has introduced a bug during a set of releases, you can use the same type of technique. Create a test that causes the current release to fail. Then choose a half-way release between now and the last known working version. Run the test again, and decide how to narrow your search. Being able to do this is just one of the many benefits of having good **version control** in your projects. Indeed, many version control systems will take this further and will automate the **process**, picking releases for you depending on the result of the test.

Nếu đội của bạn đã đưa vào một con bug qua một loạt các bản phát hành, bạn có thể dùng đúng kiểu kỹ thuật này. Hãy tạo một bài kiểm thử khiến bản phát hành hiện tại bị fail. Rồi chọn một bản phát hành nằm ở giữa khoảng từ hiện tại đến phiên bản gần nhất được biết là còn chạy tốt. Chạy lại bài kiểm thử, và quyết định cách thu hẹp phạm vi tìm kiếm. Làm được điều này chỉ là một trong vô số lợi ích của việc có hệ thống quản lý phiên bản (version control) tốt trong các dự án của bạn. Thực vậy, nhiều hệ thống quản lý phiên bản còn đẩy việc này đi xa hơn và sẽ tự động hóa cả quy trình, tự chọn các bản phát hành cho bạn tùy theo kết quả của bài kiểm thử.

Logging and/or Tracing Debuggers generally focus on the **state** of the program now . Sometimes you need more—you need to watch the state of a program or a data structure over time. Seeing a stack trace can only tell you how you got here directly. It typically can't tell you what you were doing prior to this call chain, especially in eventbased systems. [25]

Ghi log và/hoặc theo vết (Logging and/or Tracing) — Trình gỡ lỗi nói chung chỉ tập trung vào trạng thái của chương trình *ngay lúc này*. Đôi khi bạn cần nhiều hơn thế — bạn cần theo dõi trạng thái của một chương trình hay một cấu trúc dữ liệu diễn biến theo thời gian. Nhìn vào một stack trace chỉ cho bạn biết bạn đã đi thẳng tới đây bằng đường nào. Thường thì nó không cho bạn biết được bạn đã làm gì trước cả cái chuỗi lời gọi ấy, nhất là trong những hệ thống dựa trên sự kiện. [25]

Tracing statements are those little diagnostic messages you print to the screen or to a file that say things such as “got here” and “value of $x = 2$.” It’s a primitive technique compared with IDE-style debuggers, but it is peculiarly effective at diagnosing several classes of errors that debuggers can’t. Tracing is invaluable in any system where time itself is a factor: concurrent processes, real-time systems, and **event**-based applications.

Câu lệnh theo vết (tracing) là những mẫu thông điệp chẩn đoán nhỏ mà bạn in ra màn hình hoặc ra file, đại loại như “tới đây rồi” và “giá trị của $x = 2$ ”. Đây là một kỹ thuật nguyên thủy so với các trình gỡ lỗi kiểu IDE, nhưng nó lại đặc biệt hiệu quả trong việc chẩn đoán một số lớp lỗi mà trình gỡ lỗi bỏ tay. Theo vết là vô giá trong bất kỳ hệ thống nào mà bản thân thời gian là một yếu tố: các tiến trình chạy đồng thời, hệ thống thời gian thực, và các ứng dụng dựa trên sự kiện.

You can use tracing statements to drill down into the code. That is, you can add tracing statements as you descend the call tree.

Bạn có thể dùng các câu lệnh theo vết để khoan sâu vào trong mã. Tức là, bạn có thể thêm các câu lệnh theo vết theo từng bước đi xuống dọc cây lời gọi.

Trace messages should be in a regular, consistent format as you may want to parse them automatically. For instance, if you needed to track down a resource leak (such as unbalanced file opens/closes), you could trace each open and each close in a log file. By processing the log file with text processing tools or **shell** commands, you can easily identify where the offending open was occurring.

Các thông điệp theo vết nên có định dạng đều đặn, nhất quán, bởi có thể bạn sẽ muốn phân tích chúng một cách tự động. Chẳng hạn, nếu cần truy ra một chỗ rò rỉ tài nguyên (như số lần mở tệp và đóng tệp không khớp nhau), bạn có thể ghi vết mỗi lần mở và mỗi lần đóng vào một tệp log. Rồi bằng cách xử lý tệp log ấy bằng các công cụ xử lý văn bản hay lệnh shell, bạn dễ dàng chỉ ra được cái lệnh mở gây chuyện nằm ở đâu.

Rubber Ducking A very simple but particularly useful technique for finding the cause of a problem is simply to explain it to someone else. The other person should look over your shoulder at the screen, and nod his or her head constantly (like a rubber duck bobbing up and down in a bathtub). They do not need to say a word; the simple act of explaining, step by step, what the code is supposed to do often causes the problem to leap off the screen and announce itself. [26]

Vịt cao su (Rubber Ducking) — Một kỹ thuật rất đơn giản nhưng đặc biệt hữu dụng để tìm ra nguyên nhân của một vấn đề là chỉ cần giải thích nó cho người khác nghe. Người kia nên nhìn qua vai bạn vào màn hình, và gật gù liên tục (như một con vịt cao su nhấp nhô trong bồn tắm). Họ chẳng cần nói lấy một lời; chính cái việc bạn phải giải thích từng bước một xem đoạn mã lẽ ra phải làm gì lại thường khiến vấn đề nhảy bật ra khỏi màn hình và tự xưng danh. [26]

It sounds simple, but in explaining the problem to another person you must explicitly state things that you may take for granted when going through the code yourself. By having to

Nghe thì đơn giản, nhưng khi giải thích vấn đề cho người khác, bạn buộc phải nói rõ ra những điều mà lúc tự mình rà mã bạn có thể đã mặc nhiên coi là đúng. Bằng cách phải

verbalize some of these assumptions, you may suddenly gain new insight into the problem. And if you don’t have a person, a rubber duck, or teddy bear, or potted plant will do. [27]

nói thành lời một vài trong số các giả định ấy, có thể bạn sẽ đột nhiên vỡ ra điều gì đó mới mẻ về vấn đề. Còn nếu chẳng có ai bên cạnh, thì một con vịt cao su, một chú gấu bông, hay một chậu cây cảnh cũng xong. [27]

Process of Elimination In most projects, the code you are debugging may be a mixture of application code written by you and others on your project team, third-party products (database, connectivity, web framework, specialized communications or algorithms, and so on) and the platform environment (operating system, system libraries, and compilers).

Loại trừ dần (Process of Elimination) — Trong hầu hết các dự án, đoạn mã bạn đang gỡ lỗi có thể là một mớ hỗn hợp gồm: mã ứng dụng do chính bạn và những người khác trong đội dự án viết, các sản phẩm của bên thứ ba (cơ sở dữ liệu, kết nối, web framework, các thuật toán hay giao tiếp chuyên biệt, v.v.) và môi trường nền tảng (hệ điều hành, thư viện hệ thống, và trình biên dịch).

It is possible that a bug exists in the OS, the compiler, or a thirdparty product—but this should not be your first thought. It is much more likely that the bug exists in the application code under development. It is generally more profitable to assume that the application code is incorrectly calling into a library than to assume that the library itself is broken. Even if the problem does lie with a third party, you'll still have to eliminate your code before submitting the bug report.

Có khả năng một con bug nằm trong hệ điều hành, trong trình biên dịch, hoặc trong một sản phẩm bên thứ ba — nhưng đó không nên là ý nghĩ đầu tiên của bạn. Khả năng cao hơn nhiều là con bug nằm trong chính mã ứng dụng đang được phát triển. Nhìn chung, giả định rằng mã ứng dụng đang gọi vào thư viện một cách sai sót thì có lợi hơn là giả định rằng bản thân thư viện bị hỏng. Mà ngay cả khi vấn đề đúng là nằm ở một bên thứ ba, bạn vẫn sẽ phải loại trừ mã của mình ra trước khi nộp báo cáo bug.

We worked on a project where a senior engineer was convinced that the select system call was broken on a Unix system. No amount of persuasion or logic could change his mind (the fact that every other networking application on the box worked fine was irrelevant). He spent weeks writing workarounds, which, for some odd reason, didn't seem to fix the problem. When finally forced to sit down and read the documentation on select, he discovered the problem and corrected it in a matter of minutes. We now use the phrase “select is broken” as a gentle reminder whenever one of us starts blaming the system for a fault that is likely to be our own.

Chúng tôi từng làm một dự án mà ở đó có một kỹ sư kỳ cựu đinh ninh rằng lời gọi hệ thống select bị hỏng trên một hệ thống Unix. Chẳng lời thuyết phục hay lập luận nào lay chuyển nổi ông ta (chuyện mọi ứng dụng mạng khác trên chính cái máy đó đều chạy ngon lành bị coi là không liên quan). Ông ta bỏ ra hàng tuần liền viết các giải pháp đi vòng, mà vì lý do quái lạ nào đó, chẳng cái nào có vẻ sửa được vấn đề. Đến khi cuối cùng bị buộc phải ngồi xuống đọc tài liệu về select, ông ta phát hiện ra vấn đề và sửa nó chỉ trong vài phút. Giờ đây chúng tôi vẫn dùng câu “select bị hỏng” như một lời nhắc nhở nhẹ nhàng mỗi khi một trong hai đứa bắt đầu đổ lỗi cho hệ thống về một sự cố mà nhiều khả năng là do chính mình.

Tip 33: “select” Isn't Broken

Tip 33: “select” không hỏng đâu

Remember, if you see hoof prints, think horses—not zebras. The OS is probably not broken. And select is probably just fine.

Hãy nhớ, thấy dấu móng guốc thì hãy nghĩ tới ngựa — chứ đừng nghĩ tới ngựa vằn. Hệ điều hành nhiều khả năng không hỏng. Và select nhiều khả năng cũng chẳng làm sao cả.

If you “changed only one thing” and the system stopped working, that one thing was likely to be responsible, directly or indirectly, no matter how farfetched it seems. Sometimes the thing that

changed is outside of your control: new versions of the OS, compiler, database, or other third-party software can wreak havoc with previously correct code. New bugs might show up. Bugs for which you had a workaround get fixed, breaking the workaround. APIs change, functionality changes; in short, it's a whole new ball game, and you must retest the system under these new conditions. So keep a close eye on the schedule when considering an upgrade; you may want to wait until after the next release.

Nếu bạn “chỉ thay đổi đúng một thứ” mà hệ thống ngừng hoạt động, thì cái một thứ đó rất có thể là thủ phạm, trực tiếp hay gián tiếp, dù nó nghe có vẻ xa vời đến đâu. Đôi khi cái thứ thay đổi đó nằm ngoài tầm kiểm soát của bạn: phiên bản mới của hệ điều hành, trình biên dịch, cơ sở dữ liệu, hay phần mềm bên thứ ba khác có thể tàn phá đoạn mã trước đó vốn chạy đúng. Bug mới có thể trôi lên. Những bug mà bạn từng có giải pháp đi vòng nay được sửa, làm hỏng luôn cái giải pháp đi vòng đó. API thay đổi, chức năng thay đổi; nói ngắn gọn, đó là cả một ván bài hoàn toàn mới, và bạn phải kiểm thử lại hệ thống dưới những điều kiện mới này. Vậy nên hãy để mắt thật kỹ tới lịch trình khi cân nhắc một bản nâng cấp; bạn có thể sẽ muốn chờ đến sau bản phát hành kế tiếp.

The Element Of Surprise — Yếu tố bất ngờ

When you find yourself surprised by a bug (perhaps even muttering “that’s impossible” under your breath where we can’t hear you), you must reevaluate truths you hold dear. In that discount calculation algorithm—the one you knew was bulletproof and couldn’t possibly be the cause of this bug—did you test all the boundary conditions? That other piece of code you’ve been using for years—it couldn’t possibly still have a bug in it. Could it?

Khi thấy mình bị một con bug làm cho bất ngờ (có lẽ thậm chí còn lẩm bẩm “điều đó là không thể” trong miệng ở chỗ chúng tôi không nghe thấy), bạn phải đánh giá lại những chân lý mình hằng tin yêu. Trong cái thuật toán tính chiết khấu kia — cái mà bạn định ninh là bất khả xâm phạm và không đời nào có thể là nguyên nhân của con bug này — bạn đã kiểm thử hết mọi điều kiện biên chưa? Còn cái đoạn mã khác mà bạn dùng đã bao năm nay — nó không thể nào vẫn còn bug được. Hay là vẫn có?

Of course it can. The amount of surprise you feel when something goes wrong is proportional to the amount of trust and faith you have in the code being run. That’s why, when faced with a “surprising” failure, you must accept that one or

Tất nhiên là có thể chứ. Mức độ bất ngờ bạn cảm thấy khi có gì đó trục trặc tỷ lệ thuận với mức độ tin tưởng và lòng tin bạn đặt vào đoạn mã đang chạy. Đó là lý do vì sao, khi đối mặt với một sự cố “bất ngờ”, bạn phải chấp nhận rằng một hay

more of your assumptions is wrong. Don’t gloss over a routine or piece of code involved in the bug because you “know” it works. Prove it. Prove it in this **context**, with this data, with these boundary conditions.

nhiều giả định của mình là sai. Đừng cho qua một thủ tục hay một đoạn mã dính dáng tới con bug chỉ vì bạn “biết” là nó chạy đúng. Hãy chứng minh đi. Chứng minh trong đúng bối cảnh này, với đúng dữ liệu này, với đúng các điều kiện biên này.

Tip 34: Don’t Assume It—Prove It

Tip 34: Đừng giả định — hãy chứng minh

When you come across a surprise bug, beyond merely fixing it, you need to determine why this failure wasn’t caught earlier. Consider whether you need to amend the unit or other tests so that they would have caught it.

Khi bạn vấp phải một con bug bất ngờ, ngoài việc chỉ sửa nó, bạn cần xác định xem vì sao sự cố này lại không bị bắt sớm hơn. Hãy cân nhắc xem bạn có cần bổ sung các bài kiểm thử đơn vị hay các bài kiểm thử khác để chúng có thể bắt được nó hay không.

Also, if the bug is the result of bad data that was propagated through a couple of levels before causing the explosion, see if better parameter checking in those routines would have isolated it earlier (see the discussions on crashing early and assertions [here](#) and [here](#), respectively).

Ngoài ra, nếu con bug là kết quả của dữ liệu xấu được lan truyền qua vài tầng trước khi gây ra vụ nổ, hãy xem liệu việc kiểm tra tham số tốt hơn ở những thủ tục đó có cô lập được nó sớm hơn không (xem các thảo luận về việc cho crash sớm và về assertion, lần lượt ở đây và đây).

While you're at it, are there any other places in the code that may be susceptible to this same bug? Now is the time to find and fix them. Make sure that whatever happened, you'll know if it happens again.

Nhân tiện, liệu còn chỗ nào khác trong mã có thể dính phải đúng con bug này không? Bây giờ chính là lúc tìm và sửa chúng. Hãy chắc chắn rằng dù chuyện gì đã xảy ra, bạn sẽ biết nếu nó xảy ra lần nữa.

If it took a long time to fix this bug, ask yourself why. Is there anything you can do to make fixing this bug easier the next time around? Perhaps you could build in better testing hooks, or write a log file analyzer.

Nếu phải mất nhiều thời gian mới sửa được con bug này, hãy tự hỏi vì sao. Có điều gì bạn có thể làm để lần sau việc sửa con bug này dễ dàng hơn không? Có lẽ bạn có thể xây sẵn các móc kiểm thử (testing hook) tốt hơn, hay viết một bộ phân tích file log.

Finally, if the bug is the result of someone's wrong assumption, discuss the problem with the whole team: if one person misunderstands, then it's possible many people do.

Cuối cùng, nếu con bug là kết quả của một giả định sai của ai đó, hãy bàn về vấn đề với cả đội: nếu một người hiểu sai, thì có khả năng nhiều người cũng hiểu sai.

Do all this, and hopefully you won't be surprised next time.

Hãy làm hết những điều này, và hy vọng là lần sau bạn sẽ không còn bị bất ngờ nữa.

Debugging Checklist — Danh sách kiểm tra khi gỡ lỗi

Is the problem being reported a direct result of the underlying bug, or merely a symptom?

Vấn đề đang được báo cáo là hậu quả trực tiếp của con bug nằm sâu bên dưới, hay chỉ là một triệu chứng?

Is the bug really in the framework you're using? Is it in the OS? Or is it in your code?

Con bug có thật sự nằm trong cái framework bạn đang dùng không? Nó nằm trong hệ điều hành? Hay nó nằm trong mã của bạn?

If you explained this problem in detail to a coworker, what would you say?

Nếu bạn giải thích chi tiết vấn đề này cho một đồng nghiệp, bạn sẽ nói gì?

If the suspect code passes its unit tests, are the tests complete enough? What happens if you run the tests with this data?

Nếu đoạn mã bị nghi ngờ vượt qua được các bài kiểm thử đơn vị của nó, thì những bài kiểm thử đó đã đủ đầy đủ chưa? Chuyện gì xảy ra nếu bạn chạy các bài kiểm thử đó với dữ liệu này?

Do the conditions that caused this bug exist anywhere else in the system? Are there other bugs still in the larval stage, just waiting to hatch?

Những điều kiện gây ra con bug này có tồn tại ở bất kỳ chỗ nào khác trong hệ thống không? Liệu còn những con bug khác đang ở giai đoạn ấu trùng, chỉ chờ chờ nổ ra không?

Related Sections Include — Các mục liên quan

Topic 24, Dead Programs Tell No Lies

Topic 24, Chương trình Chết Chẳng Biết Nói Dối

Challenges — Thử thách

Debugging is challenge enough.

Gỡ lỗi đã đủ là một thử thách rồi.

Topic 21. Text Manipulation — Topic 21. Thao tác văn bản

Pragmatic Programmers manipulate text the same way woodworkers shape wood. In previous sections we discussed some specific tools—shells, editors, debuggers—that we use. These are similar to a woodworker’s chisels, saws, and planes— tools specialized to do one or two jobs well. However, every now and then we need to perform some **transformation** not readily handled by the basic tool set. We need a general-purpose text manipulation tool.

Lập trình viên Thực dụng (Pragmatic Programmer) thao tác với văn bản giống như thợ mộc tạo hình cho gỗ. Ở những mục trước, chúng tôi đã bàn về một vài công cụ cụ thể — shell, trình soạn thảo, trình gỡ lỗi (debugger) — mà chúng tôi vẫn dùng. Chúng tựa như đục, cưa và bào của người thợ mộc — những công cụ được chuyên biệt hóa để làm tốt một hai việc. Tuy vậy, thỉnh thoảng ta lại cần thực hiện một phép biến đổi nào đó mà bộ công cụ cơ bản không xử lý nổi một cách dễ dàng. Ta cần một công cụ thao tác văn bản đa dụng.

Text manipulation languages are to programming what routers [28] are to woodworking. They are noisy, messy, and somewhat brute force. Make mistakes with them, and entire pieces can be ruined. Some people swear they have no place in the toolbox. But in the right hands, both routers and text manipulation languages can be incredibly powerful and versatile. You can quickly trim something into shape, make joints, and carve. Used properly, these tools have surprising finesse and subtlety. But they take time to master.

Các ngôn ngữ thao tác văn bản đối với lập trình cũng giống như máy phay (router) [28] đối với nghề mộc. Chúng ồn ào, bừa bộn, và có phần thô bạo. Sảy tay với chúng, cả một chi tiết có thể hỏng bét. Có người thề rằng chúng chẳng có chỗ nào trong hộp đồ nghề cả. Nhưng vào tay người biết dùng, cả máy phay lẫn ngôn ngữ thao tác văn bản đều có thể mạnh mẽ và linh hoạt đến khó tin. Bạn có thể chớp nhoáng gọt một thứ vào đúng hình dáng, làm mộng ghép, rồi chạm khắc. Dùng đúng cách, những công cụ này có độ tinh tế và tinh vi đến bất ngờ. Nhưng phải mất thời gian mới làm chủ được chúng.

Fortunately, there are a number of great text manipulation languages. Unix developers (and we include macOS users here) often like to use the power of their command shells, augmented with tools such as `awk` and `sed`. People who prefer a more structured tool may prefer languages such as Python or Ruby.

May thay, có khá nhiều ngôn ngữ thao tác văn bản tuyệt vời. Các lập trình viên Unix (ở đây chúng tôi tính cả người dùng macOS) thường thích tận dụng sức mạnh của shell dòng lệnh, được hỗ trợ bằng những công cụ như `awk` và `sed`. Ai chuộng một công cụ có cấu trúc hơn thì có lẽ sẽ thích những ngôn ngữ như Python hay Ruby.

These languages are important enabling technologies. Using them, you can quickly hack up utilities and **prototype** ideas—

Đây là những công nghệ “bê phóng” quan trọng. Nhờ chúng, bạn có thể nhanh chóng chấp vá ra các tiện ích và dựng bản mẫu (prototype) cho ý tưởng —

jobs that might take five or ten times as long using conventional languages. And that multiplying factor is crucially important to the kind of experimenting that we do. Spending 30 minutes trying out a crazy idea is a whole lot better than spending five hours. Spending a day automating important components of a project is acceptable; spending a week might not be. In their book *The Practice of Programming* [KP99], Kernighan and Pike built the same program in five different languages. The Perl version was the shortest (17 lines, compared with C’s 150). With Perl you can manipulate text, interact with programs, talk over networks, drive web pages, perform arbitrary precision arithmetic, and write programs that look like Snoopy swearing.

những việc mà nếu dùng các ngôn ngữ thông thường có thể tốn gấp năm tới mười lần thời gian. Và cái hệ số nhân ấy cực kỳ quan trọng với kiểu thử nghiệm mà chúng tôi vẫn làm. Bỏ ra 30 phút để thử một ý tưởng điên rồ thì dễ chịu hơn nhiều so với bỏ ra năm tiếng. Dành một ngày để tự động hóa các thành phần quan trọng của một dự án là chấp nhận được; dành cả tuần thì có lẽ không. Trong cuốn *The Practice of Programming* [KP99], Kernighan và Pike đã xây dựng cùng một chương trình bằng năm ngôn ngữ khác nhau. Bản viết bằng Perl ngắn nhất (17 dòng, so với 150 dòng của C). Với Perl, bạn có thể thao tác văn bản, tương tác với các chương trình, trò chuyện qua mạng, điều khiển trang web, làm số học với độ chính xác tùy ý, và viết những chương trình trông như Snoopy đang chửi thề.

Tip 35: Learn a Text Manipulation Language

Tip 35: Hãy học một ngôn ngữ thao tác văn bản

To show the wide-ranging applicability of text manipulation languages, here’s a sample of some stuff we’ve done with Ruby and Python just related to the creation of this book:

Để cho thấy phạm vi ứng dụng rộng lớn của các ngôn ngữ thao tác văn bản, đây là vài thứ chúng tôi đã làm bằng Ruby và Python chỉ riêng cho việc tạo ra cuốn sách này:

Building the Book The build system for the Pragmatic Bookshelf is written in Ruby. Authors, editors, layout people, and support folks use Rake tasks to coordinate the building of PDFs and ebooks.

Dựng cuốn sách Hệ thống build của Pragmatic Bookshelf được viết bằng Ruby. Tác giả, biên tập viên, người dàn trang và đội hỗ trợ dùng các tác vụ Rake để phối hợp việc dựng PDF và ebook.

Code inclusion and highlighting We think it is important that any code presented in a book should have been tested first. Most of the code in this book has been. However, using the **DRY** principle (see **Topic 9**, DRY—The Evils of **Duplication**) we didn’t want to copy and paste lines of code from the tested programs into the book. That would mean we’d be

Chèn và tô màu mã nguồn Chúng tôi cho rằng bất kỳ đoạn mã nào xuất hiện trong sách cũng nên được kiểm thử trước. Phần lớn mã trong cuốn này đã được như vậy. Tuy nhiên, theo nguyên tắc DRY (xem Topic 9, DRY — Cái Ác của sự Trùng Lặp), chúng tôi không muốn sao chép rồi dán các dòng mã từ những chương trình đã kiểm thử vào sách. Làm thế nghĩa là chúng tôi sẽ

duplicating code, virtually guaranteeing that we'd forget to update an example when the corresponding program was changed. For some examples, we also didn't want to bore you with all the framework code needed to make our example compile and run. We turned to Ruby. A relatively simple script is invoked when we format the book—it extracts a named segment of a source file, does syntax highlighting, and converts the result into the typesetting language we use.

trùng lặp mã, gần như chắc chắn rằng sẽ quên cập nhật một ví dụ khi chương trình tương ứng thay đổi. Với một số ví dụ, chúng tôi cũng không muốn làm bạn buồn ngủ với toàn bộ phần mã khung cần thiết để ví dụ biên dịch và chạy được. Chúng tôi tìm đến Ruby. Một script khá đơn giản được gọi lúc chúng tôi định dạng cuốn sách — nó trích ra một đoạn được đặt tên trong một tệp mã nguồn, tô màu cú pháp, rồi chuyển kết quả sang ngôn ngữ dàn chữ mà chúng tôi dùng.

Website update We have a simple script that does a partial book build, extracts the table of contents, then uploads it to the book's page on our website. We also have a script that extracts sections of a book and uploads them as samples.

Cập nhật website Chúng tôi có một script đơn giản thực hiện việc build sách một phần, trích ra mục lục, rồi tải nó lên trang của cuốn sách trên website. Chúng tôi cũng có một script trích ra các phần của cuốn sách và tải lên làm bản đọc thử.

Including equations There's a Python script that converts LaTeX math markup into nicely formatted text.

Chèn công thức Có một script Python chuyển markup toán học LaTeX thành văn bản được định dạng đẹp.

Index generation Most indexes are created as separate documents (which makes maintaining them difficult if a document changes). Ours are marked up in the text itself, and a Ruby script collates and formats the entries.

Tạo mục lục tra cứu Hầu hết mục lục tra cứu (index) được tạo thành tài liệu riêng (điều này khiến việc bảo trì chúng trở nên khó khăn khi tài liệu thay đổi). Của chúng tôi thì được đánh dấu ngay trong văn bản, và một script Ruby sẽ gom lại và định dạng các mục từ.

And so on. In a very real way, the Pragmatic Bookshelf is built around text manipulation. And if you follow our advice to keep things in **plain text**, then using these languages to manipulate that text will bring a whole host of benefits.

Và còn nữa. Theo một nghĩa rất thật, Pragmatic Bookshelf được xây dựng quanh việc thao tác văn bản. Và nếu bạn nghe theo lời khuyên của chúng tôi mà giữ mọi thứ ở dạng văn bản thuần (plain text), thì việc dùng những ngôn ngữ này để thao tác văn bản đó sẽ mang lại cả một loạt lợi ích.

Related Sections Include — Các mục liên quan

Topic 16, The Power of Plain Text

Topic 16, Sức mạnh của Văn bản Thuần

Topic 17, **Shell** Games

Topic 17, Trò chơi với Shell

Exercises — Bài tập

Exercise 11

Bài tập 11

You're rewriting an application that used to use YAML as a **configuration** language. Your company has now standardized on JSON, so you have a bunch of .yaml files that need to be turned into .json . Write a script that takes a directory and converts each .yaml file into a corresponding .json file (so database.yaml becomes database.json , and the contents are valid JSON).

Bạn đang viết lại một ứng dụng vốn dùng YAML làm ngôn ngữ cấu hình. Công ty bạn nay đã chuẩn hóa sang JSON, nên bạn còn cả đống tệp .yaml cần được biến thành .json. Hãy viết một script nhận vào một thư mục và chuyển mỗi tệp .yaml thành một tệp .json tương ứng (nghĩa là database.yaml trở thành database.json, với nội dung là JSON hợp lệ).

Exercise 12

Bài tập 12

Your team initially chose to use camelCase names for variables, but then changed their collective mind and switched to snake_case . Write a script that scans all the source files for camelCase names and reports on them.

Ban đầu đội của bạn chọn đặt tên biến theo kiểu camelCase, nhưng rồi cả đội đổi ý và chuyển sang snake_case. Hãy viết một script quét toàn bộ các tệp mã nguồn để tìm những cái tên viết theo kiểu camelCase và báo cáo lại.

Exercise 13

Bài tập 13

Following on from the previous exercise, add the ability to change those variable names automatically in one or more files. Remember to keep a backup of the originals in case something goes horribly, horribly wrong.

Tiếp nối bài tập trước, hãy bổ sung khả năng tự động đổi những tên biến đó trong một hoặc nhiều tệp. Nhớ giữ một bản sao lưu của các tệp gốc, phòng khi có gì đó hỏng bét, hỏng kinh khủng.

Topic 22. Engineering Daybooks — Topic 22. Nhật ký kỹ thuật (Engineering Daybook)

Dave once worked for a small computer manufacturer, which meant working alongside electronic and sometimes mechanical engineers.

Có thời Dave làm cho một hãng sản xuất máy tính nhỏ, nghĩa là làm việc cạnh đám kỹ sư điện tử, đôi khi cả kỹ sư cơ khí.

Many of them walked around with a paper notebook, normally with a pen stuffed down the spine. Every now and then when we were talking, they'd pop the notebook open and scribble something.

Nhiều người trong số họ đi đâu cũng kè kè một cuốn sổ giấy, thường có cây bút nhét dọc theo gáy. Thi thoảng đang trò chuyện, họ lại lật sổ ra và nguệch ngoạc ghi cái gì đó.

Eventually Dave asked the obvious question. It turned out that they'd been trained to keep an **engineering daybook**, a kind of journal in which they recorded what they did, things they'd learned, sketches of ideas, readings from meters: basically anything to do with their work. When the notebook became full, they'd write the date range on the spine, then stick it on the shelf next to previous daybooks. There may have been a gentle competition going on for whose set of books took the most shelf space.

Cuối cùng Dave hỏi đúng cái câu ai cũng nghĩ tới. Hóa ra họ được rèn thói quen giữ một cuốn nhật ký kỹ thuật (engineering daybook), một dạng nhật trình ghi lại những gì họ làm, những điều học được, phác thảo ý tưởng, số đo trên đồng hồ đo: nói chung là bất cứ thứ gì liên quan đến công việc. Khi cuốn sổ đầy, họ ghi khoảng thời gian lên gáy rồi xếp nó lên kệ cạnh những cuốn nhật ký trước. Hình như còn có một cuộc thi đua ngầm ngầm xem bộ sổ của ai chiếm nhiều chỗ trên kệ nhất.

We use daybooks to take notes in meetings, to jot down what we're working on, to note variable values when **debugging**, to leave reminders where we put things, to record wild ideas, and sometimes just to doodle. [29]

Chúng tôi dùng nhật ký để ghi chép trong các cuộc họp, để ghi vội xem mình đang làm gì, để ghi lại giá trị các biến lúc gỡ lỗi, để nhắc mình đã cất thứ gì ở đâu, để lưu lại những ý tưởng điên rồ, và đôi khi chỉ để vẽ nguệch ngoạc. [29]

The daybook has three main benefits:

Cuốn nhật ký có ba lợi ích chính:

It is more reliable than memory. People might ask “What was the name of that company you called last week about the power supply

Nó đáng tin hơn trí nhớ. Người ta có thể hỏi “Cái công ty mà cậu gọi tuần trước về vụ trục trặc bộ nguồn tên là gì ấy nhỉ?”

problem?” and you can flip back a page or so and give them the name and number.

và bạn chỉ cần lật lại một trang hay đại loại thế là đọc ra ngay tên cùng số điện thoại cho họ.

It gives you a place to store ideas that aren't immediately relevant to the task at hand. That way you can continue to concentrate on what you are doing, knowing that the great idea won't be forgotten.

Nó cho bạn một chỗ để cất những ý tưởng chưa liên quan ngay đến việc đang làm. Nhờ vậy bạn có thể tiếp tục tập trung vào việc trước mắt, yên tâm rằng ý tưởng hay ho kia sẽ không bị quên lãng.

It acts as a kind of rubber duck (described here). When you stop to write something down, your brain may switch gears, almost as if talking to someone—a great chance to reflect. You may start to make a note and then suddenly realize that what you'd just done, the **topic** of the note, is just plain wrong.

Nó hoạt động như một kiểu vịt cao su (rubber duck) (đã mô tả ở đây). Khi bạn dừng lại để viết ra điều gì đó, não bạn có thể chuyển số, gần như thể đang nói chuyện với ai đó, một cơ hội tuyệt vời để ngầm nghĩ. Bạn có thể bắt đầu ghi chú rồi đột nhiên nhận ra cái việc mình vừa làm, chính cái chủ đề của ghi chú ấy, đơn giản là sai bét.

There's an added benefit, too. Every now and then you can look back at what you were doing oh-so-many-years-ago and think about the people, the projects, and the awful clothes and hairstyles.

Còn có thêm một lợi ích nữa. Thi thoảng bạn có thể ngó lại xem mình đã làm gì hồi-xa-lắc-xa-lơ rồi nghĩ về những con người, những dự án, cùng đám quần áo và kiểu tóc kinh dị thuở ấy.

So, try keeping an engineering daybook. Use paper, not a file or a wiki: there's something special about the act of writing compared to typing. Give it a month, and see if you're getting any benefits.

Vậy nên, hãy thử giữ một cuốn nhật ký kỹ thuật. Dùng giấy, đừng dùng file hay wiki: hành động viết tay có gì đó đặc biệt so với gõ phím. Cho nó một tháng, rồi xem bạn có thu được lợi ích gì không.

If nothing else, it'll make writing your memoir easier when you're rich and famous.

Nếu chẳng được gì khác, thì ít nhất nó cũng giúp bạn viết hồi ký dễ dàng hơn khi bạn trở nên giàu có và nổi tiếng.

Related Sections Include — Các mục liên quan

Topic 6, **Your Knowledge Portfolio**

Topic 6, Danh mục tri thức của bạn

Topic 37, Listen to Your Lizard Brain

Topic 37, Lắng nghe Não Thằn Lằn của bạn

Footnotes

Chú thích

[24] All software becomes legacy software as soon as it's written.

[24] Mọi phần mềm đều trở thành phần mềm cũ (legacy) ngay khi vừa được viết ra.

[25] Although the Elm language does have a time-traveling debugger.

[25] Mặc dù ngôn ngữ Elm thực sự có một trình gỡ lỗi du hành thời gian.

Why “rubber ducking”? While an undergraduate at Imperial College in London, Dave [26] did a lot of work with a research assistant named Greg Pugh, one of the best developers Dave has known. For several months Greg carried around a small yellow rubber duck, which he'd place on his terminal while coding. It was a while before Dave had the courage to ask....

Tại sao lại gọi là “rubber ducking”? Hồi còn là sinh viên đại học ở Imperial College London, Dave [26] đã làm rất nhiều việc cùng một trợ lý nghiên cứu tên Greg Pugh, một trong những lập trình viên giỏi nhất Dave từng biết. Suốt mấy tháng trời, Greg lúc nào cũng mang theo một chú vịt cao su nhỏ màu vàng, đặt lên màn hình mỗi khi lập trình. Phải một lúc lâu sau Dave mới đủ can đảm để hỏi cho ra lẽ....

[27] Earlier versions of the book talked about talking to your pot plant . It was a typo. Honest.

[27] Các phiên bản trước của cuốn sách nói về việc trò chuyện với cây cảnh trong chậu (pot plant) của bạn. Đó là lỗi đánh máy. Thật đấy.

[28] Here **router** means the tool that spins cutting blades very, very fast, not a device for interconnecting networks.

[28] Ở đây router nghĩa là cái máy quay lưỡi cắt thật, thật nhanh, chứ không phải thiết bị để liên kết các mạng với nhau.

[29] There is some evidence that doodling helps focus and improves cognitive skills, for example, see What does doodling do? [And10] .

[29] Có một số bằng chứng cho thấy việc vẽ nguệch ngoạc giúp tập trung và cải thiện kỹ năng nhận thức, ví dụ, xem What does doodling do? [And10].

Copyright © 2020 Pearson Education, Inc.

Copyright © 2020 Pearson Education, Inc.

Pragmatic Paranoia — Hoang tưởng kiểu Thực dụng

Tip 36: You Can't Write Perfect Software

Tip 36: Bạn không thể viết ra phần mềm hoàn hảo

Did that hurt? It shouldn't. Accept it as an axiom of life. Embrace it. Celebrate it. Because perfect software doesn't exist. No one in the brief history of computing has ever written a piece of perfect software. It's unlikely that you'll be the first. And unless you accept this as a fact, you'll end up wasting time and energy chasing an impossible dream.

Có đau không? Đáng ra thì không. Hãy chấp nhận nó như một tiên đề của cuộc đời. Hãy ôm lấy nó. Hãy ăn mừng nó. Bởi vì phần mềm hoàn hảo không tồn tại. Trong suốt lịch sử ngắn ngủi của ngành điện toán, chưa có ai từng viết ra một phần mềm hoàn hảo. Khó mà bạn lại là người đầu tiên. Và trừ phi bạn chấp nhận điều này như một sự thật, bạn sẽ chỉ phí thời gian lẫn sức lực đuổi theo một giấc mơ bất khả thi.

So, given this depressing reality, how does a **Pragmatic Programmer** turn it into an advantage? That's the **topic** of this chapter.

Vậy thì, với thực tế đáng buồn ấy, làm sao một Lập trình viên Thực dụng (Pragmatic Programmer) biến nó thành lợi thế? Đó là chủ đề của chương này.

Everyone knows that they personally are the only good driver on Earth. The rest of the world is out there to get them, blowing through stop signs, weaving between lanes, not indicating turns, texting on the phone, and just generally not living up to our standards. So we drive defensively. We look out for trouble before it happens, anticipate the unexpected, and never put ourselves into a position from which we can't extricate ourselves.

Ai cũng tin rằng bản thân mình là tài xế giỏi duy nhất trên Trái Đất. Cả phần còn lại của thế giới thì đang nhằm nhe hại mình, lao qua biển báo dừng, lượn lách giữa các làn, rẽ mà chẳng xi nhan, nhắn tin trên điện thoại, và nói chung là chẳng đạt nổi chuẩn mực của ta. Thế nên ta lái xe phòng thủ. Ta cảnh giác với rắc rối trước khi nó xảy ra, lường trước những điều bất ngờ, và không bao giờ tự đặt mình vào thế không thể thoát ra.

The analogy with coding is pretty obvious. We are constantly interfacing with other people's code—code that might not live up to our high standards—and dealing with inputs that may or may not be valid. So we are taught to code defensively. If there's any doubt, we validate all information we're given. We use assertions to detect bad data, and distrust data from potential attackers or trolls. We check for consistency, put constraints on database columns, and generally feel pretty good about ourselves.

Phép so sánh với việc viết mã thì khá rõ ràng. Chúng ta liên tục giao tiếp với mã của người khác — thứ mã có thể chẳng đạt nổi chuẩn mực cao của ta — và xử lý những dữ liệu đầu vào có thể hợp lệ hoặc không. Thế nên ta được dạy phải viết mã phòng thủ. Để có chút nghi ngờ, ta kiểm chứng mọi thông tin được trao cho. Ta dùng các khẳng định (assertion) để phát hiện dữ liệu xấu, và không tin tưởng dữ liệu đến từ những kẻ tấn công hay phá rối tiềm tàng. Ta kiểm tra tính nhất quán, đặt ràng buộc lên các cột cơ sở dữ liệu, và nhìn chung là khá hài lòng về bản thân.

But **Pragmatic** Programmers take this a step further. They don't trust themselves, either. Knowing that no one writes perfect code, including themselves, Pragmatic Programmers build in defenses against their own mistakes. We describe the first defensive measure in Topic 23, **Design by Contract** : clients and suppliers must agree on rights and responsibilities.

Nhưng Lập trình viên Thực dụng còn đi xa hơn một bước. Họ cũng chẳng tin chính mình. Biết rằng không ai viết được mã hoàn hảo, kể cả bản thân, Lập trình viên Thực dụng xây dựng sẵn các tuyến phòng thủ chống lại chính những sai lầm của mình. Chúng tôi mô tả biện pháp phòng thủ đầu tiên ở Topic 23, Thiết kế theo hợp đồng (Design by Contract): bên dùng và bên cung cấp phải thống nhất với nhau về quyền lợi và trách nhiệm.

In Topic 24, Dead Programs Tell No Lies , we want to ensure that we do no damage while we're working the bugs out. So we try to check things often and terminate the program if things go awry.

Ở Topic 24, Chương trình chết không biết nói dối (Dead Programs Tell No Lies), chúng tôi muốn đảm bảo rằng không gây ra thiệt hại gì trong lúc đang dò tìm lỗi. Vì vậy ta cố gắng kiểm tra mọi thứ thường xuyên và cho chương trình dừng hẳn nếu có gì trục trặc.

Topic 25, **Assertive Programming** describes an easy method of checking along the way—write code that actively verifies your assumptions.

Topic 25, Lập trình bằng khẳng định (Assertive Programming) mô tả một phương pháp dễ dàng để kiểm tra trong suốt quá trình — viết mã chủ động xác minh các giả định của bạn.

As your programs get more dynamic, you'll find yourself juggling system resources—memory, files, devices, and the like. In Topic 26, How to Balance Resources , we'll suggest ways of ensuring that you don't drop any of the balls.

Khi chương trình của bạn trở nên động hơn, bạn sẽ thấy mình phải tung hứng các tài nguyên hệ thống — bộ nhớ, tệp tin, thiết bị, vân vân. Trong Topic 26, Cách cân bằng tài nguyên (How to Balance Resources), chúng tôi sẽ gợi ý vài cách để bạn không làm rơi quả nào.

And most importantly, we stick to small steps always, as described in Topic 27, Don't Outrun Your Headlights , so we

Và quan trọng nhất, ta luôn bám lấy những bước nhỏ, như mô tả ở Topic 27, Đừng vượt quá tầm đèn pha (Don't Outrun Your Headlights), để ta

don't fall off the edge of the cliff.

không rơi khỏi mép vực.

In a world of imperfect systems, ridiculous time scales, laughable tools, and impossible **requirements**, let's play it safe. As Woody Allen said, "When everybody actually is out to get you, paranoia is just good thinking."

Trong một thế giới đầy những hệ thống không hoàn hảo, những mốc thời gian lơ lửng, những công cụ buồn cười và những yêu cầu bất khả thi, hãy chơi an toàn. Như Woody

Allen từng nói: “Khi cả thế giới thật sự đang nhắm nhe hại bạn, thì hoang tưởng chỉ là một suy nghĩ sáng suốt mà thôi.”

Nothing astonishes men so much as common sense and plain dealing. Ralph Waldo Emerson, Essays

Không gì khiến con người kinh ngạc bằng lẽ thường và sự thẳng thắn. Ralph Waldo Emerson, Essays

Topic 23. Design by Contract — Topic 23. Thiết kế theo hợp đồng

Dealing with computer systems is hard. Dealing with people is even harder. But as a species, we’ve had longer to figure out issues of human interactions. Some of the solutions we’ve come up with during the last few millennia can be applied to writing software as well. One of the best solutions for ensuring plain dealing is the contract .

Làm việc với hệ thống máy tính đã khó. Làm việc với con người còn khó hơn. Nhưng với tư cách một giống loài, ta đã có nhiều thời gian hơn để tìm cách gỡ rối những chuyện tương tác giữa người với người. Một số giải pháp ta nghĩ ra suốt vài thiên niên kỷ qua cũng có thể áp dụng vào việc viết phần mềm. Một trong những giải pháp tốt nhất để đảm bảo sự thẳng thắn chính là hợp đồng.

A contract defines your rights and responsibilities, as well as those of the other party. In addition, there is an agreement concerning repercussions if either party fails to abide by the contract.

Một hợp đồng định nghĩa quyền lợi và trách nhiệm của bạn, cũng như của bên kia. Ngoài ra, còn có thỏa thuận về hậu quả nếu một trong hai bên không tuân thủ hợp đồng.

Maybe you have an employment contract that specifies the hours you’ll work and the rules of conduct you must follow. In return, the company pays you a salary and other perks. Each party meets its obligations and everyone benefits.

Có lẽ bạn có một hợp đồng lao động quy định số giờ làm việc và những quy tắc ứng xử bạn phải theo. Đổi lại, công ty trả lương cùng các phúc lợi khác cho bạn. Mỗi bên đều làm tròn nghĩa vụ và ai cũng được lợi.

It’s an idea used the world over—both formally and informally— to help humans interact. Can we use the same concept to help software modules interact? The answer is “yes.”

Đây là một ý tưởng được dùng khắp thế giới — cả chính thức lẫn không chính thức — để giúp con người tương tác với nhau. Liệu ta có thể dùng cùng khái niệm ấy để giúp các mô-đun phần mềm tương tác không? Câu trả lời là “có.”

DBC

DBC

Bertrand Meyer (*Object-Oriented Software Construction* [Mey97]) developed the concept of Design by Contract for the language Eiffel. [30] It is a simple yet powerful technique that focuses on documenting (and agreeing to) the rights and responsibilities of software modules to ensure program correctness. What is a correct program? One that does no more and no less than it claims to do. Documenting and verifying that claim is the heart of Design by Contract (DBC, for short).

Bertrand Meyer (*Object-Oriented Software Construction* [Mey97]) đã phát triển khái niệm Thiết kế theo Hợp đồng (Design by Contract) cho ngôn ngữ Eiffel. [30] Đó là một kỹ thuật đơn giản mà mạnh mẽ, tập trung vào việc ghi thành tài liệu (và thỏa thuận về) các quyền

và trách nhiệm của những module phần mềm nhằm đảm bảo tính đúng đắn của chương trình. Thế nào là một chương trình đúng đắn? Là chương trình làm không nhiều hơn cũng không ít hơn những gì nó tuyên bố sẽ làm. Ghi thành tài liệu và kiểm chứng cái tuyên bố ấy chính là trái tim của Thiết kế theo Hợp đồng (gọi tắt là DBC).

Every function and method in a software system does something . Before it starts that something , the function may have some expectation of the **state** of the world, and it may be able to make a statement about the state of the world when it concludes. Meyer describes these expectations and claims as follows:

Mọi hàm và phương thức trong một hệ thống phần mềm đều làm một việc gì đó. Trước khi bắt đầu việc gì đó ấy, hàm có thể có một kỳ vọng nào đó về trạng thái của thế giới, và khi kết thúc, nó có thể đưa ra một phát biểu về trạng thái của thế giới. Meyer mô tả những kỳ vọng và tuyên bố này như sau:

Preconditions What must be true in order for the routine to be called; the routine's requirements. A routine should never get called when its preconditions would be violated. It is the caller's responsibility to pass good data (see the box here).

Tiền điều kiện (Preconditions) Điều gì phải đúng để chương trình con được gọi; đó là yêu cầu của chương trình con. Một chương trình con không bao giờ nên bị gọi khi các tiền điều kiện của nó bị vi phạm. Việc truyền dữ liệu tốt là trách nhiệm của bên gọi (xem khung ở đây).

Postconditions What the routine is guaranteed to do; the state of the world when the routine is done. The fact that the routine has a postcondition implies that it will conclude: infinite loops aren't allowed.

Hậu điều kiện (Postconditions) Điều mà chương trình con được đảm bảo sẽ làm; trạng thái của thế giới khi chương trình con kết thúc. Việc chương trình con có một hậu điều kiện ngụ ý rằng nó sẽ kết thúc: không cho phép các vòng lặp vô hạn.

Class invariants A class ensures that this condition is always true from the perspective of a caller. During **internal** processing of a

Bất biến của lớp (Class invariants) Một lớp đảm bảo rằng điều kiện này luôn đúng dưới góc nhìn của bên gọi. Trong quá trình xử lý nội bộ của một

routine, the invariant may not hold, but by the time the routine exits and control returns to the caller, the invariant must be true. (Note that a class cannot give unrestricted write-access to any data member that participates in the invariant.)

chương trình con, bất biến có thể không đúng, nhưng đến khi chương trình con thoát ra và quyền điều khiển trả về cho bên gọi, bất biến phải đúng. (Lưu ý rằng một lớp không thể cấp quyền ghi không hạn chế cho bất kỳ thành viên dữ liệu nào tham gia vào bất biến.)

The contract between a routine and any potential caller can thus be read as

Hợp đồng giữa một chương trình con và bất kỳ bên gọi tiềm năng nào do đó có thể đọc là

If all the routine's preconditions are met by the caller, the routine shall guarantee that all postconditions and invariants will be true when it completes.

Nếu mọi tiền điều kiện của chương trình con đều được bên gọi đáp ứng, thì chương trình con phải đảm bảo rằng mọi hậu điều kiện và bất biến sẽ đúng khi nó hoàn tất.

If either party fails to live up to the terms of the contract, then a remedy (which was previously agreed to) is invoked—maybe an **exception** is raised, or the program terminates. Whatever happens, make

no mistake that failure to live up to the contract is a bug. It is not something that should ever happen, which is why preconditions should not be used to perform things such as user-input validation.

Nếu một trong hai bên không tuân thủ các điều khoản của hợp đồng, thì một biện pháp khắc phục (đã được thỏa thuận trước) sẽ được kích hoạt — có thể là ném ra một ngoại lệ, hoặc chương trình dừng lại. Dù điều gì xảy ra đi nữa, đừng nhầm lẫn: việc không tuân thủ hợp đồng là một lỗi (bug). Đó là điều không bao giờ nên xảy ra, và đó cũng là lý do tiền điều kiện không nên được dùng để làm những việc như kiểm chứng dữ liệu nhập của người dùng.

Some languages have better support for these concepts than others. Clojure, for example, supports preand post-conditions as well as the more comprehensive instrumentation provided by specs . Here's an example of a banking function to make a deposit using simple preand post-conditions:

Một số ngôn ngữ hỗ trợ những khái niệm này tốt hơn các ngôn ngữ khác. Chẳng hạn Clojure hỗ trợ cả tiền và hậu điều kiện, cũng như cơ chế kiểm soát toàn diện hơn do specs cung cấp. Dưới đây là ví dụ về một hàm ngân hàng thực hiện việc nạp tiền dùng tiền và hậu điều kiện đơn giản:

```
(defn accept-deposit [account-id amount] { :pre [ (> amount 0.00) (account-open? account-id) ] :post [ (contains? (account-transactions account-id) %) ] } "Accept a deposit and return the new transaction id" ;; Some other processing goes here... ;; Return the newly created transaction:
```

```
(defn accept-deposit [account-id amount] { :pre [ (> amount 0.00) (account-open? account-id) ] :post [ (contains? (account-transactions account-id) %) ] } "Accept a deposit and return the new transaction id" ;; Some other processing goes here... ;; Return the newly created transaction:
```

```
(create-transaction account-id :deposit amount))
```

```
(create-transaction account-id :deposit amount))
```

There are two preconditions for the accept-deposit function. The first is that the amount is greater than zero, and the second is that the account is open and valid, as determined by some function named account-open? . There is also a postcondition: the function guarantees that the new transaction (the return value of this function, represented here by '%') can be found among the transactions for this account.

Hàm `accept-deposit` có hai tiền điều kiện. Thứ nhất là số tiền phải lớn hơn không, và thứ hai là tài khoản phải đang mở và hợp lệ, do một hàm nào đó tên là `account-open?` xác định. Ngoài ra còn có một hậu điều kiện: hàm bảo đảm rằng giao dịch mới (giá trị trả về của hàm này, ở đây được biểu diễn bằng '%') có thể tìm thấy trong số các giao dịch của tài khoản đó.

If you call `accept-deposit` with a positive amount for the deposit and a valid account, it will proceed to create a transaction of the appropriate type and do whatever other processing it does. However, if there's a bug in the program and you somehow passed in a negative amount for the deposit, you'll get a runtime exception:

Nếu bạn gọi `accept-deposit` với một số tiền nạp dương và một tài khoản hợp lệ, nó sẽ tiến hành tạo một giao dịch thuộc loại thích hợp và làm mọi xử lý khác mà nó vẫn làm. Tuy nhiên, nếu chương trình có bug và bằng cách nào đó bạn truyền vào một số tiền âm cho khoản nạp, bạn sẽ nhận được một ngoại lệ lúc chạy:

Exception in thread "main"... Caused by: java.lang.AssertionError: Assert failed: (> amount 0.0)

```
Exception in thread "main"... Caused by: java.lang.AssertionError: Assert failed: (>
amount 0.0)
```

Similarly, this function requires that the specified account is open and valid. If it's not, you'll see that exception instead:

Tương tự, hàm này yêu cầu tài khoản chỉ định phải đang mở và hợp lệ. Nếu không, bạn sẽ thấy ngoại lệ này thay vào đó:

```
Exception in thread "main"... Caused by: java.lang.AssertionError: Assert failed: (account-open?
accountid)
```

```
Exception in thread "main"... Caused by: java.lang.AssertionError: Assert failed: (account-
open? accountid)
```

Other languages have features that, while not DBC-specific, can still be used to good effect. For example, Elixir uses guard clauses to dispatch function calls against several available bodies:

Các ngôn ngữ khác có những tính năng tuy không chuyên cho DBC nhưng vẫn dùng được rất hiệu quả. Chẳng hạn, Elixir dùng các mệnh đề bảo vệ (guard clause) để điều phối lời gọi hàm tới một trong nhiều thân hàm có sẵn:

```
defmodule Deposits do
  def accept_deposit(account_id, amount) when (amount > 100000) do # Call
    the manager!
  end
  def accept_deposit(account_id, amount) when (amount > 10000) do
    defmodule Deposits do
      def accept_deposit(account_id, amount) when (amount > 100000)
      do # Call the manager!
      end
      def accept_deposit(account_id, amount) when (amount > 10000)
      do

```

**Extra Federal requirements for reporting # Some processing... end
def accept_deposit(account_id, amount) when (amount > 0) do #
Some processing... end end — Extra
Federal requirements for reporting
Some processing... end def
accept_deposit(account_id, amount)
when (amount > 0) do # Some
processing... end end**

In this case, calling `accept_deposit` with a large enough amount may trigger additional steps and processing. Try to call it with an amount less than or equal to zero, however, and you'll get an exception informing you that you can't:

Trong trường hợp này, gọi `accept_deposit` với một số tiền đủ lớn có thể kích hoạt thêm những bước và xử lý bổ sung. Nhưng cứ thử gọi nó với số tiền nhỏ hơn hoặc bằng không mà xem, bạn sẽ nhận được một ngoại lệ báo cho biết là bạn không được phép:

**** (FunctionClauseError) no function clause matching in Deposits.accept_deposit/2**

**** (FunctionClauseError) no function clause matching in Deposits.accept_deposit/2**

This is a better approach than simply checking your inputs; in this case, you simply can not call this function if your arguments are out of range.

Đây là một cách tiếp cận tốt hơn so với việc chỉ kiểm tra dữ liệu nhập; trong trường hợp này, bạn đơn giản là không thể gọi hàm nếu các đối số nằm ngoài khoảng cho phép.

Tip 37: Design with Contracts

Tip 37: Hãy thiết kế bằng hợp đồng

In Topic 10, **Orthogonality**, we recommended writing “shy” code. Here, the emphasis is on “lazy” code: be strict in what you will accept before you begin, and promise as little as possible in return. Remember, if your contract indicates that you’ll accept anything and promise the world in return, then you’ve got a lot of code to write!

Ở Topic 10, Tính trực giao (Orthogonality), chúng tôi đã khuyên bạn viết mã “nhút nhát” (shy). Ở đây, trọng tâm là mã “lười biếng” (lazy): hãy nghiêm khắc với những gì bạn chấp nhận trước khi bắt đầu, và hứa hẹn càng ít càng tốt ở chiều ngược lại. Nhớ rằng, nếu hợp đồng của bạn tuyên bố sẽ chấp nhận bất cứ thứ gì và hứa hẹn cả thế giới ở chiều ngược lại, thì bạn sẽ có cả đống mã phải viết đấy!

In any programming language, whether it’s functional, objectoriented, or procedural, DBC forces you to think.

Trong bất kỳ ngôn ngữ lập trình nào, dù là hàm, hướng đối tượng hay thủ tục, DBC buộc bạn phải tư duy.

DBC and Test-Driven Development Is Design by Contract needed in a world where developers practice unit testing, testdriven development (TDD), **property-based testing**, or **defensive programming**?

DBC và Phát triển hướng kiểm thử (Test-Driven Development) Liệu Thiết kế theo hợp đồng có cần thiết trong một thế giới mà các lập trình viên thực hành kiểm thử đơn vị, phát triển hướng kiểm thử (TDD), kiểm thử dựa trên thuộc tính (property-based testing), hay lập trình phòng thủ không?

The short answer is “yes.”

Câu trả lời ngắn gọn là “có.”

DBC and testing are different approaches to the broader topic of program correctness. They both have value and both have uses in different situations. DBC offers several advantages over specific testing approaches:

DBC và kiểm thử là những cách tiếp cận khác nhau cho chủ đề rộng hơn là tính đúng đắn của chương trình. Cả hai đều có giá trị và đều có công dụng trong những tình huống khác nhau. DBC mang lại vài lợi thế so với các cách tiếp cận kiểm thử cụ thể:

DBC doesn’t require any setup or mocking

DBC không đòi hỏi bất kỳ thiết lập hay giả lập (mocking) nào

DBC defines the parameters for success or failure in all cases, whereas testing can only target one specific case at a time

DBC định nghĩa các tham số cho thành công hay thất bại trong mọi trường hợp, trong khi kiểm thử chỉ có thể nhắm vào một trường hợp cụ thể mỗi lần

TDD and other testing happens only at “test time” within the build cycle. But DBC and assertions are forever: during design, development, deployment, and maintenance

TDD và các kiểu kiểm thử khác chỉ diễn ra vào “lúc kiểm thử” trong chu trình build. Nhưng DBC và các khẳng định thì tồn tại mãi mãi: trong lúc thiết kế, phát triển, triển khai và bảo trì

TDD does not focus on checking internal invariants within the code under test, it’s more black-box style to check the public interface DBC is more efficient (and **DRY**-er) than defensive programming,

where everyone has to validate data in case no one else does.

TDD không tập trung vào việc kiểm tra các bất biến nội bộ bên trong mã đang được kiểm thử, nó thiên về kiểu hộp đen để kiểm tra giao diện công khai. DBC hiệu quả hơn (và “DRY” hơn) so với lập trình phòng thủ, nơi ai cũng phải kiểm chứng dữ liệu phòng khi không ai khác làm việc đó.

TDD is a great technique, but as with many techniques, it might invite you to concentrate on the “happy path,” and not the real world full of bad data, bad actors, bad versions, and bad specifications.

TDD là một kỹ thuật tuyệt vời, nhưng như nhiều kỹ thuật khác, nó có thể dụ bạn chỉ tập trung vào “con đường suôn sẻ” (happy path), chứ không phải thế giới thực đầy dữ liệu xấu, kẻ xấu, phiên bản xấu và đặc tả xấu.

Class Invariants and Functional Languages It’s a naming thing. Eiffel is an object-oriented language, so Meyer named this idea “class invariant.” But, really, it’s more general than that. What this idea really refers to is state. In an object-oriented language, the state is associated with instances of classes. But other languages have state, too.

Bất biến của lớp và Ngôn ngữ hàm Đây là chuyện đặt tên. Eiffel là một ngôn ngữ hướng đối tượng, nên Meyer đặt tên ý tưởng này là “bất biến của lớp.” Nhưng thật ra nó tổng quát hơn thế. Điều mà ý tưởng này thực sự ám chỉ là trạng thái (state). Trong một ngôn ngữ hướng đối tượng, trạng thái gắn với các thể hiện của lớp. Nhưng các ngôn ngữ khác cũng có trạng thái.

In a functional language, you typically pass state to functions and receive updated state as a result. The concepts of invariants is just as useful in these circumstances.

Trong một ngôn ngữ hàm, bạn thường truyền trạng thái vào các hàm và nhận lại trạng thái đã cập nhật làm kết quả. Khái niệm bất biến cũng hữu ích y như vậy trong những hoàn cảnh này.

Implementing Dbc — Triển khai DBC

Simply enumerating what the input domain range is, what the boundary conditions are, and what the routine promises to deliver—or, more importantly, what it doesn’t promise to deliver—before you write the code is a huge leap forward in writing

Chỉ riêng việc liệt kê khoảng miền đầu vào là gì, các điều kiện biên là gì, và chương trình con hứa sẽ trả về điều gì—hay quan trọng hơn, điều gì nó không hứa sẽ trả về—trước khi bạn viết mã đã là một bước nhảy vọt khổng lồ trong việc viết

better software. By not stating these things, you are back to **programming by coincidence** (see the discussion here), which is where many projects start, finish, and fail.

phần mềm tốt hơn. Bằng cách không phát biểu những điều này, bạn quay về với lập trình theo may rủi (programming by coincidence) (xem thảo luận ở đây), nơi mà nhiều dự án bắt đầu, kết thúc và thất bại.

In languages that do not support DBC in the code, this might be as far as you can go—and that’s not too bad. DBC is, after all, a design technique. Even without automatic checking, you can put the contract in the code as comments or in the unit tests and still get a very real benefit.

Trong những ngôn ngữ không hỗ trợ DBC ngay trong mã, có lẽ đây là điểm xa nhất bạn có thể đi tới—và thế cũng không tệ lắm. Suy cho cùng, DBC là một kỹ thuật thiết kế. Ngay cả

khi không có kiểm tra tự động, bạn vẫn có thể đặt hợp đồng vào trong mã dưới dạng chú thích hoặc trong các bài kiểm thử đơn vị và vẫn thu được lợi ích rất thực tế.

Assertions While documenting these assumptions is a great start, you can get much greater benefit by having the compiler check your contract for you. You can partially emulate this in some languages by using assertions: runtime checks of logical conditions (see Topic 25, Assertive Programming). Why only partially? Can't you use assertions to do everything DBC can do?

Các khẳng định (Assertions) Tuy việc lập tài liệu cho những giả định này là một khởi đầu tuyệt vời, bạn có thể thu lợi nhiều hơn hẳn nếu để trình biên dịch kiểm tra hợp đồng giúp bạn. Bạn có thể mô phỏng một phần điều đó trong vài ngôn ngữ bằng cách dùng các khẳng định: những lần kiểm tra điều kiện logic lúc chạy (xem Topic 25, Lập trình bằng khẳng định). Tại sao chỉ một phần? Chẳng lẽ bạn không thể dùng khẳng định để làm mọi thứ DBC làm được sao?

Unfortunately, the answer is no. To begin with, in objectoriented languages there probably is no support for propagating assertions down an **inheritance** hierarchy. This means that if you override a base class method that has a contract, the assertions that implement that contract will not be called correctly (unless you duplicate them manually in the new code). You must remember to call the class invariant (and all base class invariants) manually before you exit every method. The basic problem is that the contract is not automatically enforced.

Tiếc thay, câu trả lời là không. Trước hết, trong các ngôn ngữ hướng đối tượng, có lẽ chẳng có hỗ trợ cho việc lan truyền khẳng định xuống theo cây phân cấp kế thừa. Điều này nghĩa là nếu bạn ghi đè một phương thức của lớp cơ sở vốn có hợp đồng, thì các khẳng định cài đặt hợp đồng đó sẽ không được gọi đúng cách (trừ phi bạn tự tay nhân bản chúng trong mã mới). Bạn phải nhớ tự tay gọi bất biến của lớp (và mọi bất biến của các lớp cơ sở) trước khi thoát khỏi mỗi phương thức. Vấn đề căn bản là hợp đồng không được tự động thực thi.

In other environments, the exceptions generated from DBCstyle assertions might be turned off globally or ignored in the code.

Trong những môi trường khác, các ngoại lệ sinh ra từ những khẳng định kiểu DBC có thể bị tắt trên toàn cục hoặc bị phớt lờ trong mã.

Also, there is no built-in concept of “old” values; that is, values

Ngoài ra, không có khái niệm dựng sẵn nào về giá trị “cũ”; tức là các giá trị

as they existed at the entry to a method. If you're using assertions to enforce contracts, you must add code to the precondition to save any information you'll want to use in the postcondition, if the language will even allow that. In the Eiffel language, where DBC was born, you can just use old expression .

như chúng tồn tại tại thời điểm bước vào một phương thức. Nếu bạn dùng assertion để thực thi hợp đồng, bạn phải thêm mã vào tiền điều kiện để lưu lại mọi thông tin mà bạn sẽ muốn dùng ở hậu điều kiện — nếu như ngôn ngữ còn cho phép làm thế. Trong ngôn ngữ Eiffel, nơi DBC ra đời, bạn chỉ việc dùng biểu thức old.

Finally, conventional runtime systems and libraries are not designed to support contracts, so these calls are not checked. This is a big loss, because it is often at the boundary between your code and the libraries it uses that the most problems are detected (see Topic 24, Dead Programs Tell No Lies for a more detailed discussion).

Cuối cùng, các hệ thống và thư viện chạy lúc thực thi thông thường không được thiết kế để hỗ trợ hợp đồng, nên những lời gọi này không được kiểm tra. Đây là một mất mát lớn, bởi vì thường chính tại ranh giới giữa mã của bạn và các thư viện mà nó dùng là nơi phát

hiện ra nhiều vấn đề nhất (xem Topic 24, Chương trình chết không biết nói dối để có thảo luận chi tiết hơn).

Who's Responsible?

Ai chịu trách nhiệm?

Who is responsible for checking the precondition, the caller or the routine being called? When implemented as part of the language, the answer is neither: the precondition is tested behind the scenes after the caller invokes the routine but before the routine itself is entered. Thus if there is any explicit checking of parameters to be done, it must be performed by the caller, because the routine itself will never see parameters that violate its precondition. (For languages without built-in support, you would need to bracket the called routine with a preamble and/or postamble that checks these assertions.)

Ai chịu trách nhiệm kiểm tra tiền điều kiện, bên gọi hay chương trình con được gọi? Khi được cài đặt như một phần của ngôn ngữ, câu trả lời là chẳng bên nào: tiền điều kiện được kiểm tra ở hậu trường sau khi bên gọi gọi chương trình con nhưng trước khi chính chương trình con được bước vào. Do đó nếu có bất kỳ việc kiểm tra tham số tường minh nào cần làm, nó phải do bên gọi thực hiện, vì bản thân chương trình con sẽ không bao giờ thấy những tham số vi phạm tiền điều kiện của nó. (Với những ngôn ngữ không có hỗ trợ dựng sẵn, bạn sẽ cần kẹp chương trình con được gọi giữa một đoạn mở đầu và/hoặc đoạn kết để kiểm tra những khẳng định này.)

Consider a program that reads a number from the console, calculates its square root (by calling `sqrt`), and prints the result. The `sqrt` function has a precondition—its argument must not be negative. If the user enters a negative number at the console, it is up to the calling code to ensure that it never gets passed to `sqrt`. This calling code has many options: it could terminate, it could issue a warning and read another number, or it could make the number positive and append an `i` to the result returned by `sqrt`. Whatever its choice, this is definitely not `sqrt`'s problem.

Hãy xét một chương trình đọc một con số từ console, tính căn bậc hai của nó (bằng cách gọi `sqrt`), rồi in kết quả ra. Hàm `sqrt` có một tiền điều kiện — đối số của nó không được âm. Nếu người dùng nhập một số âm ở console, thì việc bảo đảm con số đó không bao giờ bị truyền vào `sqrt` là trách nhiệm của mã gọi. Mã gọi ấy có nhiều lựa chọn: nó có thể kết thúc chương trình, có thể phát một cảnh báo rồi đọc một con số khác, hoặc có thể đổi con số thành dương rồi thêm chữ `i` vào sau kết quả mà `sqrt` trả về. Nhưng dù chọn cách nào đi nữa, đây dứt khoát không phải là vấn đề của `sqrt`.

By expressing the domain of the square root function in the precondition of the `sqrt` routine, you shift the burden of correctness to the caller—where it belongs. You can then design the `sqrt` routine secure in the knowledge that its input will be in range.

Bằng cách diễn đạt miền xác định của hàm căn bậc hai trong tiền điều kiện của chương trình con `sqrt`, bạn dịch chuyển gánh nặng về tính đúng đắn sang cho bên gọi — nơi nó thuộc về. Sau đó bạn có thể thiết kế chương trình con `sqrt` một cách an tâm, biết chắc rằng đầu vào của nó sẽ nằm trong khoảng cho phép.

Dbc And Crashing Early — DBC và việc sụp sớm

DBC fits in nicely with our concept of crashing early (see Topic 24, Dead Programs Tell No Lies). By using an `assert` or DBC

DBC ăn khớp đẹp đẽ với khái niệm sụp sớm (crashing early) của chúng tôi (xem Topic 24, Chương trình chết không biết nói dối). Bằng cách dùng cơ chế `assert` hoặc DBC

mechanism to validate the preconditions, postconditions, and invariants, you can **crash early** and report more accurate information about the problem.

để kiểm chứng các tiền điều kiện, hậu điều kiện và bất biến, bạn có thể sụp sớm và báo cáo thông tin chính xác hơn về vấn đề.

For example, suppose you have a method that calculates square roots. It needs a DBC precondition that restricts the domain to positive numbers. In languages that support DBC, if you pass `sqrt` a negative parameter, you'll get an informative error such as `sqrt_arg_must_be_positive`, along with a stack trace.

Chẳng hạn, giả sử bạn có một phương thức tính căn bậc hai. Nó cần một tiền điều kiện DBC để giới hạn miền xác định trong các số dương. Ở những ngôn ngữ có hỗ trợ DBC, nếu bạn truyền cho `sqrt` một tham số âm, bạn sẽ nhận được một lỗi giàu thông tin, đại loại như `sqrt_arg_must_be_positive`, kèm theo một stack trace.

This is better than the alternative in other languages such as Java, C, and C++ where passing a negative number to `sqrt` returns the special value NaN (Not a Number). It may be some time later in the program that you attempt to do some math on NaN, with surprising results.

Như thế còn hơn hẳn cách làm ở những ngôn ngữ khác như Java, C và C++, nơi truyền một số âm cho `sqrt` sẽ trả về giá trị đặc biệt NaN (Not a Number). Có khi mãi về sau trong chương trình bạn mới thử làm phép toán nào đó trên NaN, với những kết quả đầy bất ngờ.

It's much easier to find and diagnose the problem by crashing early, at the site of the problem.

Tìm ra và chẩn đoán vấn đề sẽ dễ hơn nhiều khi sụp sớm, ngay tại nơi xảy ra vấn đề.

Semantic Invariants — Bất biến ngữ nghĩa

You can use semantic invariants to express inviolate requirements, a kind of “philosophical contract.”

Bạn có thể dùng các bất biến ngữ nghĩa (semantic invariants) để diễn đạt những yêu cầu bất khả xâm phạm, một kiểu “hợp đồng triết lý.”

We once wrote a debit card transaction switch. A major requirement was that the user of a debit card should never have the same transaction applied to their account twice. In other words, no matter what sort of failure mode might happen, the error should be on the side of not processing a transaction rather than processing a duplicate transaction.

Có lần chúng tôi viết một bộ chuyển mạch giao dịch thẻ ghi nợ. Một yêu cầu quan trọng là người dùng thẻ ghi nợ không bao giờ được bị áp cùng một giao dịch vào tài khoản hai lần. Nói cách khác, bất kể kiểu hỏng hóc nào có thể xảy ra, sai sót phải nghiêng về phía không xử lý một giao dịch chứ không phải xử lý một giao dịch trùng lặp.

This simple law, driven directly from the requirements, proved to be very helpful in sorting out complex error recovery

Quy luật đơn giản này, được suy ra trực tiếp từ yêu cầu, hóa ra lại cực kỳ hữu ích trong việc gỡ rối những kịch bản khôi phục lỗi

scenarios, and guided the detailed design and implementation in many areas.

phức tạp, và đã định hướng cho thiết kế chi tiết lẫn việc cài đặt ở nhiều mảng.

Be sure not to confuse requirements that are fixed, inviolate laws with those that are merely policies that might change with a new management regime. That's why we use the term semantic invariants—it must be central to the very meaning of a thing, and not subject to the whims of **policy** (which is what more dynamic business rules are for).

Hãy chắc chắn đừng nhầm lẫn giữa những yêu cầu là quy luật cố định, bất khả xâm phạm với những thứ chỉ là chính sách có thể thay đổi khi có một ê-kíp quản lý mới. Đó là lý do chúng tôi dùng thuật ngữ bất biến ngữ nghĩa — nó phải là cốt lõi trong chính ý nghĩa của một sự vật, chứ không phụ thuộc vào những thay đổi tùy hứng của chính sách (vốn là việc của những quy tắc nghiệp vụ động hơn).

When you find a requirement that qualifies, make sure it becomes a well-known part of whatever documentation you are producing—whether it is a bulleted list in the requirements document that gets signed in triplicate or just a big note on the common whiteboard that everyone sees. Try to state it clearly and unambiguously. For example, in the debit card example, we might write

Khi bạn tìm thấy một yêu cầu đủ tư cách, hãy chắc chắn rằng nó trở thành một phần ai cũng biết trong bất kỳ tài liệu nào bạn đang tạo ra — dù đó là một gạch đầu dòng trong tài liệu yêu cầu được ký ba bản hay chỉ là một dòng ghi chú to tướng trên tấm bảng chung mà ai cũng thấy. Hãy cố diễn đạt nó rõ ràng và không mập mờ. Chẳng hạn, trong ví dụ thẻ ghi nợ, ta có thể viết

Err in favor of the consumer.

Sai thì hãy sai về phía có lợi cho người tiêu dùng.

This is a clear, concise, unambiguous statement that's applicable in many different areas of the system. It is our contract with all users of the system, our guarantee of behavior.

Đây là một phát biểu rõ ràng, súc tích, không mập mờ, áp dụng được ở nhiều mảng khác nhau của hệ thống. Nó là hợp đồng của chúng tôi với mọi người dùng hệ thống, là sự đảm bảo về hành vi của chúng tôi.

Dynamic Contracts And Agents — Hợp đồng động và các tác nhân

Until now, we have talked about contracts as fixed, immutable specifications. But in the landscape of autonomous agents, this doesn't need to be the case. By the definition of “autonomous,” agents are free to reject requests that they do not want to honor. They are free to renegotiate the contract—“I can't provide that, but if you give me this, then I might provide something else.”

Cho đến giờ, ta đã nói về hợp đồng như những đặc tả cố định, bất biến. Nhưng trong bối cảnh các tác nhân (agent) tự trị, điều này không nhất thiết phải vậy. Theo đúng định nghĩa của “tự trị,” các tác nhân được tự do từ chối những yêu cầu mà chúng không muốn đáp ứng. Chúng được tự do thương lượng lại hợp đồng — “Tôi không cung cấp được cái đó, nhưng nếu anh đưa tôi cái này, thì tôi có thể cung cấp một thứ khác.”

Certainly any system that relies on agent technology has a

Chắc chắn rằng bất kỳ hệ thống nào dựa vào công nghệ tác nhân đều có

critical dependence on contractual arrangements—even if they are dynamically generated.

một sự phụ thuộc cốt yếu vào các dàn xếp hợp đồng — ngay cả khi chúng được sinh ra một cách động.

Imagine: with enough components and agents that can negotiate their own contracts among themselves to achieve a goal, we might just solve the software productivity crisis by letting software solve it for us.

Hãy tưởng tượng: với đủ thành phần và tác nhân có thể tự thương lượng hợp đồng giữa chúng với nhau để đạt một mục tiêu, biết đâu ta có thể giải quyết được cuộc khủng hoảng năng suất phần mềm bằng cách để phần mềm giải nó hộ ta.

But if we can't use contracts by hand, we won't be able to use them automatically. So next time you design a piece of software, design its contract as well.

Nhưng nếu ta không thể dùng hợp đồng bằng tay, thì ta sẽ không thể dùng chúng một cách tự động. Vậy nên lần tới khi bạn thiết kế một phần mềm, hãy thiết kế cả hợp đồng cho nó nữa.

Related Sections Include — Các mục liên quan

Topic 24, Dead Programs Tell No Lies

Topic 24, Chương trình chết không biết nói dối

Topic 25, Assertive Programming

Topic 25, Lập trình bằng khẳng định

Topic 38, Programming by Coincidence

Topic 38, Lập trình theo may rủi

Topic 42, Property-Based Testing

Topic 42, Kiểm thử dựa trên thuộc tính

Topic 43, Stay Safe Out There

Topic 43, Giữ an toàn ngoài kia

Topic 45, The Requirements Pit

Topic 45, Cái hố yêu cầu

Challenges — Thử thách

Points to ponder: If DBC is so powerful, why isn't it used more widely? Is it hard to come up with the contract? Does it make you think about issues you'd rather ignore for now? Does it force you to THINK! ? Clearly, this is a dangerous tool!

Vài điều để ngẫm nghĩ: Nếu DBC mạnh mẽ đến vậy, sao nó lại không được dùng rộng rãi hơn? Có phải vì khó nghĩ ra hợp đồng không? Có phải nó bắt bạn nghĩ về những vấn đề mà bạn thà lờ đi lúc này hơn? Có phải nó buộc bạn phải TƯ DUY!? Rõ ràng, đây là một công cụ nguy hiểm!

Exercises — Bài tập

Exercise 14 (possible answer)

Bài tập 14 (gợi ý đáp án)

Design an interface to a kitchen blender. It will eventually be a web-based, IoT-enabled blender, but for now we just need the interface to control it. It has ten speed settings (0 means off). You can't operate it empty, and you can change the speed only one unit at a time (that is, from 0 to 1, and from 1 to 2, not from 0 to 2).

Hãy thiết kế một giao diện cho một chiếc máy xay sinh tố trong bếp. Cuối cùng nó sẽ là một chiếc máy xay dùng nền web, hỗ trợ IoT, nhưng hiện giờ ta chỉ cần giao diện để điều khiển nó. Nó có mười mức tốc độ (0 nghĩa là tắt). Bạn không thể vận hành nó khi rỗng, và bạn chỉ có thể thay đổi tốc độ mỗi lần một bậc (tức là từ 0 lên 1, từ 1 lên 2, chứ không phải từ 0 lên 2).

Here are the methods. Add appropriate preand postconditions and an invariant.

Đây là các phương thức. Hãy thêm các tiền và hậu điều kiện thích hợp cùng một bất biến.

```
int getSpeed() void setSpeed(int x) boolean isFull() void fill() void empty()
```

```
int getSpeed() void setSpeed(int x) boolean isFull() void fill() void empty()
```

Exercise 15 (possible answer)

Bài tập 15 (gợi ý đáp án)

How many numbers are in the series 0, 5, 10, 15, ..., 100?

Có bao nhiêu con số trong dãy 0, 5, 10, 15, ..., 100?

Topic 24. Dead Programs Tell No Lies — Topic 24. Chương trình chết thì không nói dối

Have you noticed that sometimes other people can detect that things aren't well with you before you're aware of the problem yourself? It's the same with other people's code. If something is starting to go awry with one of our programs, sometimes it is a library or framework routine that catches it first. Maybe we've passed in a nil value, or an empty list. Maybe there's a missing key in that **hash**, or the value we thought contained a hash really contains a list instead. Maybe there was a network error or filesystem error that we didn't catch, and we've got empty or corrupted data. A logic error a couple of million instructions ago means that the selector for a case statement is no longer the expected 1, 2, or 3. We'll hit the default case unexpectedly. That's also one reason why each and every case/switch statement needs to have a default clause: we want to know when the "impossible" has happened.

Bạn có để ý rằng đôi khi người khác nhận ra bạn đang có chuyện không ổn trước cả khi chính bạn ý thức được vấn đề không? Với code của người khác cũng vậy. Nếu có gì đó bắt đầu trục trặc trong một chương trình của ta, đôi khi chính một hàm thư viện hay hàm framework mới là thứ bắt được nó trước tiên. Có thể ta đã truyền vào một giá trị nil, hay một danh sách rỗng. Có thể cái hash kia thiếu mất một khóa, hoặc cái giá trị mà ta tưởng chứa một hash thì thật ra lại chứa một danh sách. Có thể đã có một lỗi mạng hay lỗi hệ thống tệp mà ta không bắt, và thế là ta ôm phải dữ liệu rỗng hoặc hỏng. Một lỗi logic từ vài triệu chỉ thị trước đó khiến bộ chọn của một câu lệnh case không còn là 1, 2 hay 3 như kỳ vọng nữa. Ta sẽ rơi vào nhánh default một cách bất ngờ. Đó cũng là một lý do vì sao mỗi và mọi câu lệnh case/switch đều cần có mệnh đề default: chúng ta muốn biết khi nào thì điều "không thể xảy ra" đã xảy ra.

It's easy to fall into the "it can't happen" mentality. Most of us have written code that didn't check that a file closed successfully, or that a trace statement got written as we expected. And all things being equal, it's likely that we didn't need to—the code in question wouldn't fail under any normal

conditions. But we're coding defensively. We're making sure that the data is what we think it is, that the code in production is the code we think it is. We're checking that the correct versions of dependencies were actually loaded.

Thật dễ rơi vào lối nghĩ “chuyện đó không thể xảy ra”. Hầu hết chúng ta đều từng viết code mà không kiểm tra xem một tập tin đã đóng thành công chưa, hay một câu lệnh trace có được ghi ra đúng như ta tưởng không. Và nói cho công bằng, rất có thể là ta chẳng cần kiểm tra thật — đoạn code ấy sẽ không hỏng trong bất kỳ điều kiện bình thường nào. Nhưng chúng ta đang lập trình một cách phòng vệ. Chúng ta đảm bảo rằng dữ liệu đúng là thứ ta nghĩ, rằng code chạy trên môi trường production đúng là code ta nghĩ. Chúng ta kiểm tra rằng đúng phiên bản của các phụ thuộc đã thực sự được nạp.

All errors give you information. You could convince yourself

Mọi lỗi đều trao cho bạn thông tin. Bạn có thể tự thuyết phục mình

that the error can't happen, and choose to ignore it. Instead, **Pragmatic** Programmers tell themselves that if there is an error, something very, very bad has happened. Don't forget to Read the Damn Error Message (see Coder in a Strange Land).

rằng lỗi đó không thể xảy ra, rồi chọn cách phớt lờ nó. Thay vào đó, Lập trình viên Thực dụng (Pragmatic Programmer) tự nhủ rằng nếu có lỗi, thì một chuyện rất, rất tệ vừa xảy ra. Đừng quên Đọc Cái Thông Báo Lỗi Chết Tiệt Đó (xem mục Coder in a Strange Land).

Catch And Release Is For Fish — Bắt rồi thả lại là chuyện của câu cá

Some developers feel that is it good style to catch or rescue all exceptions, re-raising them after writing some kind of message. Their code is full of things like this (where a bare raise statement reraises the current **exception**):

Một số lập trình viên cho rằng bắt (hoặc rescue) mọi ngoại lệ rồi ném lại chúng sau khi đã ghi ra một thông báo nào đó mới là phong cách tốt. Mã của họ đầy rẫy những thứ như thế này (ở đây câu lệnh raise trần trụi sẽ ném lại ngoại lệ hiện tại):

```
try do add_score_to_board(score); rescue InvalidScore Logger.error( "Can't add invalid score. Exiting" ); raise rescue BoardServerDown Logger.error( "Can't add score: board is down. Exiting" ); raise rescue StaleTransaction Logger.error( "Can't add score: stale transaction. Exiting" ); raise end
```

```
try do add_score_to_board(score); rescue InvalidScore Logger.error( "Can't add invalid score. Exiting" ); raise rescue BoardServerDown Logger.error( "Can't add score: board is down. Exiting" ); raise rescue StaleTransaction Logger.error( "Can't add score: stale transaction. Exiting" ); raise end
```

Here's how Pragmatic Programmers would write this:

Còn đây là cách Lập trình viên Thực dụng viết đoạn này:

```
add_score_to_board(score);  
add_score_to_board(score);
```

We prefer it for two reasons. First, the application code isn't eclipsed by the error handling. Second, and perhaps more important, the code is less coupled. In the verbose example, we have to list every exception the add_score_to_board method could raise. If the writer of that method adds another exception, our code is subtly out of date. In the more pragmatic second version, the new exception is automatically propagated.

Chúng tôi thích cách này hơn, vì hai lý do. Thứ nhất, mã ứng dụng không bị phần xử lý lỗi che lấp. Thứ hai, và có lẽ quan trọng hơn, mã ít bị ghép nối hơn. Ở ví dụ dài dòng kia, ta buộc phải liệt kê mọi ngoại lệ mà phương thức `add_score_to_board` có thể ném ra. Nếu người viết phương thức ấy thêm vào một ngoại lệ nữa, mã của ta sẽ lỗi thời một cách âm thầm. Còn ở phiên bản thứ hai thực dụng hơn, ngoại lệ mới sẽ tự động được truyền lên.

Tip 38: Crash Early

Tip 38: Hãy crash sớm

Crash, Don'T Trash — Hãy crash, đừng phá phách

One of the benefits of detecting problems as soon as you can is that you can crash earlier, and crashing is often the best thing you can do. The alternative may be to continue, writing corrupted data to some vital database or commanding the washing machine into its twentieth consecutive spin cycle.

Một trong những cái lợi của việc phát hiện sự cố càng sớm càng tốt là bạn có thể crash sớm hơn, và crash thường là điều tốt nhất bạn có thể làm. Phương án thay thế có thể là tiếp tục chạy, ghi dữ liệu hỏng vào một cơ sở dữ liệu trọng yếu nào đó, hoặc ra lệnh cho cái máy giặt quay vắt sang vòng thứ hai mươi liên tiếp.

The Erlang and Elixir languages embrace this philosophy. Joe Armstrong, inventor of Erlang and author of *Programming Erlang: Software for a Concurrent World* [Arm07], is often quoted as saying, “**Defensive programming** is a waste of time. **Let it crash!**” In these environments, programs are designed to fail, but that failure is managed with supervisors. A **supervisor** is responsible for running code and knows what to do in case the code fails, which could include cleaning up after it, restarting it, and so on. What happens when the supervisor itself fails? Its own supervisor manages that **event**, leading to a design composed of supervisor trees. The technique is very effective and helps to account for the use of these languages in high-availability, fault-tolerant systems.

Ngôn ngữ Erlang và Elixir ôm trọn triết lý này. Joe Armstrong, người phát minh ra Erlang và tác giả cuốn *Programming Erlang: Software for a Concurrent World* [Arm07], thường được trích lại câu nói: “Lập trình phòng vệ là phí thời gian. Cứ để nó crash!”. Trong những môi trường này, các chương trình được thiết kế để hỏng, nhưng cái hỏng đó được quản lý bằng những supervisor (bộ giám sát). Một supervisor chịu trách nhiệm chạy code và biết phải làm gì khi code hỏng, có thể bao gồm dọn dẹp sau nó, khởi động lại nó, v.v. Vậy chuyện gì xảy ra khi chính supervisor cũng hỏng? Supervisor của riêng nó sẽ quản lý sự kiện đó, dẫn đến một thiết kế tạo thành cây supervisor (supervisor tree). Kỹ thuật này rất hiệu quả và phần nào lý giải vì sao những ngôn ngữ này được dùng trong các hệ thống tính sẵn sàng cao, chịu lỗi tốt.

In other environments, it may be inappropriate simply to exit a running program. You may have claimed resources that might not get released, or you may need to write log messages, tidy up open transactions, or interact with other processes.

Ở những môi trường khác, đơn giản thoát khỏi một chương trình đang chạy có thể là không phù hợp. Bạn có thể đã chiếm dụng những tài nguyên mà có nguy cơ không được giải phóng, hoặc bạn cần ghi các thông điệp log, dọn gọn những giao dịch đang mở, hay tương tác với các tiến trình khác.

However, the basic principle stays the same—when your code discovers that something that was supposed to be impossible just happened, your program is no longer viable. Anything it does from this point forward becomes suspect, so terminate it

Tuy vậy, nguyên tắc cơ bản vẫn không đổi — khi code của bạn phát hiện ra một điều lẽ ra không thể xảy ra vừa xảy ra, thì chương trình của bạn không còn sống được nữa. Mọi việc nó làm từ thời điểm này trở đi đều đáng ngờ, vậy nên hãy kết liễu nó

as soon as possible.

càng sớm càng tốt.

A dead program normally does a lot less damage than a crippled one.

Một chương trình đã chết thường gây ít thiệt hại hơn nhiều so với một chương trình lê lét tật nguyên.

Related Sections Include — Các mục liên quan

Topic 20, Debugging

Topic 20, Gỡ lỗi

Topic 23, Design by Contract

Topic 23, Thiết kế theo hợp đồng

Topic 25, Assertive Programming

Topic 25, Lập trình bằng khẳng định

Topic 26, How to Balance Resources

Topic 26, Cách cân đối tài nguyên

Topic 43, Stay Safe Out There

Topic 43, Giữ an toàn ngoài kia

There is a luxury in self-reproach. When we blame ourselves we feel no one else has a right to blame us. Oscar Wilde, *The Picture of Dorian Gray*

Có một thứ xa xỉ trong việc tự dằn vặt. Khi ta tự trách mình, ta cảm thấy không ai khác có quyền trách ta nữa. Oscar Wilde, *The Picture of Dorian Gray*

Topic 25. Assertive Programming — Topic 25. Lập trình phòng vệ bằng assertion

It seems that there's a mantra that every programmer must memorize early in his or her career. It is a fundamental tenet of computing, a core belief that we learn to apply to **requirements**, designs, code, comments, just about everything we do. It goes

Dường như có một câu thần chú mà mọi lập trình viên đều phải thuộc lòng từ sớm trong sự nghiệp. Nó là một giáo lý nền tảng của ngành tính toán, một niềm tin cốt lõi mà ta học cách áp dụng cho yêu cầu, thiết kế, mã nguồn, chú thích, gần như mọi thứ ta làm. Nó như thế này

This can never happen...

Chuyện này không bao giờ xảy ra được đâu...

“This application will never be used abroad, so why internationalize it?” “count can’t be negative.”
“Logging can’t fail.”

“Ứng dụng này sẽ chẳng bao giờ được dùng ở nước ngoài, vậy quốc tế hóa làm gì?” “count không thể âm được.” “Ghi log thì không thể hỏng được.”

Let’s not practice this kind of self-deception, particularly when coding.

Đừng tự lừa dối mình kiểu đó, nhất là khi viết mã.

Tip 39: Use Assertions to Prevent the Impossible

Tip 39: Dùng Assertion Để Ngăn Điều Bất Khả

Whenever you find yourself thinking “but of course that could never happen,” add code to check it. The easiest way to do this is with assertions. In many language implementations, you’ll find some form of assert that checks a Boolean condition. [31] These checks can be invaluable. If a parameter or a result should never be null , then check for it explicitly:

Mỗi khi bạn thấy mình đang nghĩ “nhưng dĩ nhiên chuyện đó chẳng bao giờ xảy ra được”, hãy thêm mã để kiểm tra nó. Cách dễ nhất là dùng assertion. Trong nhiều cài đặt ngôn ngữ, bạn sẽ thấy có một dạng assert nào đó dùng để kiểm tra một điều kiện Boolean. [31] Những phép kiểm tra này có thể vô giá. Nếu một tham số hay một kết quả lẽ ra không bao giờ được là null , thì hãy kiểm tra nó một cách tường minh:

```
assert (result != null );
```

```
assert (result != null );
```

In the Java implementation, you can (and should) add a descriptive string:

Trong cài đặt Java, bạn có thể (và nên) thêm một chuỗi mô tả:

```
assert result != null && result.size() > 0 : “Empty result from XYZ” ;
```

```
assert result != null && result.size() > 0 : “Empty result from XYZ” ;
```

Assertions are also useful checks on an algorithm’s operation. Maybe you’ve written a clever sort algorithm, named `my_sort` . Check that it works:

Assertion cũng là phép kiểm tra hữu ích cho hoạt động của một thuật toán. Có thể bạn vừa viết một thuật toán sắp xếp thông minh, đặt tên là `my_sort` . Hãy kiểm tra xem nó có chạy đúng không:

```
books = my_sort(find( “scifi” )) assert(is_sorted?(books))
```

```
books = my_sort(find( “scifi” )) assert(is_sorted?(books))
```

Don’t use assertions in place of real error handling. Assertions check for things that should never happen: you don’t want to be writing code such as the following:

Đừng dùng assertion để thay cho việc xử lý lỗi thực sự. Assertion kiểm tra những thứ lẽ ra không bao giờ được xảy ra: bạn không muốn viết mã kiểu như sau đây:

```
puts( “Enter ‘Y’ or ‘N’:” ) ans = gets[0] # Grab first character of response assert((ch == ‘Y’ ) || (ch == ‘N’ )) # Very bad idea!
```

```
puts( “Enter ‘Y’ or ‘N’:” ) ans = gets[0] # Lấy ký tự đầu tiên của phản hồi assert((ch == ‘Y’ ) || (ch == ‘N’ )) # Ý tưởng cực tệ!
```

And just because most assert implementations will terminate the **process** when an **assertion** fails, there’s no reason why versions you write should. If you need to free resources, catch the assertion’s **exception** or trap the exit, and run your own error handler. Just make sure the code you execute in

those dying milliseconds doesn't rely on the information that triggered the assertion failure in the first place.

Và chỉ vì phần lớn các cài đặt assert sẽ kết thúc tiến trình khi một assertion thất bại, không có nghĩa là phiên bản do bạn tự viết cũng phải làm thế. Nếu bạn cần giải phóng tài nguyên, hãy bắt ngoại lệ của assertion hoặc chặn lệnh thoát, rồi chạy trình xử lý lỗi của riêng mình. Chỉ cần đảm bảo rằng đoạn mã bạn chạy trong những mili-giây hấp hối ấy không dựa vào chính thông tin đã làm assertion thất bại ngay từ đầu.

Assertions And Side Effects — Assertion và tác dụng phụ

It's embarrassing when the code we add to detect errors actually ends up creating new errors. This can happen with assertions if evaluating the condition has side effects. For example, it would be a bad idea to code something such as

Thật ngượng khi mà ta thêm vào để phát hiện lỗi lại rồi cuộc để ra lỗi mới. Điều này có thể xảy ra với assertion nếu việc đánh giá điều kiện gây ra tác dụng phụ. Ví dụ, sẽ là một ý tồi nếu viết một thứ kiểu như

```
while (iter.hasMoreElements()) { assert (iter.nextElement() != null ); Object obj = iter.nextElement(); // .... }
```

```
while (iter.hasMoreElements()) { assert (iter.nextElement() != null ); Object obj = iter.nextElement(); // .... }
```

The `.nextElement()` call in the assertion has the side effect of moving the iterator past the element being fetched, and so the loop will process only half the elements in the collection. It would be better to write

Lời gọi `.nextElement()` trong assertion có tác dụng phụ là đẩy iterator vượt qua phần tử đang được lấy ra, và vì thế vòng lặp sẽ chỉ xử lý một nửa số phần tử trong tập hợp. Sẽ tốt hơn nếu viết

```
while (iter.hasMoreElements()) { Object obj = iter.nextElement(); assert (obj != null ); // .... } This problem is a kind of Heisenbug [32] —debugging that changes the behavior of the system being debugged.
```

```
while (iter.hasMoreElements()) { Object obj = iter.nextElement(); assert (obj != null ); // .... } Vấn đề này là một dạng Heisenbug [32] —việc gỡ lỗi làm thay đổi hành vi của chính hệ thống đang được gỡ lỗi.
```

(We also believe that nowadays, when most languages have decent support for iterating functions over collections, this kind of explicit loop is unnecessary and bad form.)

(Chúng tôi cũng tin rằng ngày nay, khi hầu hết các ngôn ngữ đều hỗ trợ tử tế việc duyệt hàm trên các tập hợp, kiểu vòng lặp tường minh này là không cần thiết và là phong cách dở.)

Leave Assertions Turned On — Cứ để assertion luôn bật

There is a common misunderstanding about assertions. It goes something like this:

Có một hiểu lầm phổ biến về assertion. Nó đại loại thế này:

Assertions add some overhead to code. Because they check for things that should never happen, they'll get triggered only by a bug in the code. Once the code has been tested and shipped, they are

no longer needed, and should be turned off to make the code run faster. Assertions are a debugging facility.

Assertion thêm chút chi phí cho mã. Vì chúng kiểm tra những thứ lẽ ra không bao giờ xảy ra, nên chúng chỉ bị kích hoạt bởi một lỗi (bug) trong mã. Một khi mã đã được kiểm thử và giao đi, chúng không còn cần nữa, và nên tắt đi để mã chạy nhanh hơn. Assertion là một công cụ gỡ lỗi.

There are two patently wrong assumptions here. First, they assume that testing finds all the bugs. In reality, for any

Ở đây có hai giả định sai bét. Thứ nhất, chúng giả định rằng kiểm thử tìm ra mọi lỗi. Trên thực tế, với bất kỳ

complex program you are unlikely to test even a minuscule percentage of the permutations your code will be put through. Second, the optimists are forgetting that your program runs in a dangerous world. During testing, rats probably won't gnaw through a communications cable, someone playing a game won't exhaust memory, and log files won't fill the storage partition. These things might happen when your program runs in a production environment. Your first line of defense is checking for any possible error, and your second is using assertions to try to detect those you've missed.

chương trình phức tạp nào, bạn khó lòng kiểm thử nổi dù chỉ một phần trăm cực nhỏ các hoán vị mà mã của bạn sẽ phải trải qua. Thứ hai, những người lạc quan đang quên rằng chương trình của bạn chạy trong một thế giới đầy hiểm họa. Khi kiểm thử, có lẽ chuột sẽ không gặm đứt cáp truyền thông, ai đó chơi game sẽ không làm cạn bộ nhớ, và file log sẽ không lấp đầy phân vùng lưu trữ. Những chuyện này có thể xảy ra khi chương trình của bạn chạy trong môi trường sản xuất (production). Tuyến phòng thủ đầu tiên của bạn là kiểm tra mọi lỗi có thể, và tuyến thứ hai là dùng assertion để cố phát hiện những lỗi bạn đã bỏ sót.

Turning off assertions when you deliver a program to production is like crossing a high wire without a net because you once made it across in practice. There's dramatic value, but it's hard to get life insurance.

Tắt assertion khi giao chương trình lên production cũng giống như đi trên dây cao mà không có lưới chỉ vì có lần bạn đã qua trót lọt lúc tập luyện. Nó có giá trị kịch tính đấy, nhưng khó mà mua được bảo hiểm nhân thọ.

Even if you do have performance issues, turn off only those assertions that really hit you. The sort example above may be a critical part of your application, and may need to be fast. Adding the check means another pass through the data, which might be unacceptable. Make that particular check optional, but leave the rest in.

Kể cả khi bạn thực sự gặp vấn đề về hiệu năng, cũng chỉ tắt những assertion thực sự đè nặng lên bạn thôi. Ví dụ sắp xếp ở trên có thể là một phần then chốt của ứng dụng, và có thể cần phải nhanh. Thêm phép kiểm tra nghĩa là thêm một lượt duyệt qua dữ liệu, điều này có thể không chấp nhận được. Hãy làm cho riêng phép kiểm tra đó thành tùy chọn, nhưng giữ những cái còn lại.

Use Assertions in Production, Win Big Money

Dùng assertion trong production, thắng đậm

A former neighbor of Andy's headed up a small startup company that made network devices. One of their secrets to success was the decision to leave assertions in place in production releases. These assertions were well crafted to report all the pertinent data leading to the failure, and presented via a nice-looking UI to the end user. This level of feedback, from real users under actual conditions,

allowed the developers to plug the holes and fix these obscure, hard-to-reproduce bugs, resulting in remarkably stable, bullet-proof software.

Một người hàng xóm cũ của Andy điều hành một công ty khởi nghiệp nhỏ chuyên làm thiết bị mạng. Một trong những bí quyết thành công của họ là quyết định giữ nguyên các assertion trong các bản phát hành production. Những assertion này được làm rất khéo để báo cáo mọi dữ liệu liên quan dẫn tới sự cố, và trình bày qua một giao diện người dùng đẹp mắt cho người dùng cuối. Mức độ phản hồi này, từ người dùng thật trong điều kiện thật, cho phép các lập trình viên bịt kín các lỗ hổng và sửa những lỗi tối nghĩa, khó tái hiện đó, kết quả là phần mềm ổn định và chống đạn một cách đáng kinh ngạc.

This small, unknown company had such a solid product, it was soon acquired for hundreds of millions of dollars.

Công ty nhỏ vô danh này có một sản phẩm vững chắc đến mức chẳng bao lâu nó được mua lại với giá hàng trăm triệu đô la.

Just sayin’.

Nói vậy thôi.

Exercise 16 (possible answer)

Bài tập 16 (đáp án khả dĩ)

A quick reality check. Which of these “impossible” things can happen?

Một phép kiểm tra thực tế nhanh. Trong những điều “bất khả” sau, điều nào có thể xảy ra?

A month with fewer than 28 days

Một tháng có ít hơn 28 ngày

Error code from a system call: can’t access the current directory

Mã lỗi từ một lời gọi hệ thống: không thể truy cập thư mục hiện hành

In C++: $a = 2$; $b = 3$; but $(a + b)$ does not equal

Trong C++: $a = 2$; $b = 3$; nhưng $(a + b)$ không bằng

A triangle with an interior angle sum $\neq 180^\circ$

Một tam giác có tổng các góc trong $\neq 180^\circ$

A minute that doesn’t have 60 seconds

Một phút không có đủ 60 giây

$(a + 1) \leq a$

$(a + 1) \leq a$

Related Sections Include — Các mục liên quan

Topic 23, Design by Contract

Topic 23, Thiết kế theo hợp đồng

Topic 24, Dead Programs Tell No Lies

Topic 24, Chương trình chết chẳng nói dối

Topic 42, **Property-Based Testing**

Topic 42, Kiểm thử dựa trên thuộc tính

Topic 43, Stay Safe Out There

Topic 43, Hãy giữ an toàn ngoài kia

To light a candle is to cast a shadow... Ursula K. Le Guin, A Wizard of Earthsea

Thắp một ngọn nến cũng là gieo một bóng tối... Ursula K. Le Guin, A Wizard of Earthsea

Topic 26. How to Balance Resources — Topic 26. Cân Bằng Tài Nguyên

We all manage resources whenever we code: memory, transactions, threads, network connections, files, timers—all kinds of things with limited availability. Most of the time, resource usage follows a predictable pattern: you **allocate** the resource, use it, and then **deallocate** it.

Mỗi khi viết code, tất cả chúng ta đều phải quản lý tài nguyên: bộ nhớ, giao dịch, luồng, kết nối mạng, tệp tin, bộ định giờ — đủ thứ chỉ có sẵn ở mức hữu hạn. Phần lớn thời gian, việc sử dụng tài nguyên đi theo một khuôn mẫu dễ đoán: bạn cấp phát tài nguyên, dùng nó, rồi giải phóng nó.

However, many developers have no consistent plan for dealing with resource allocation and deallocation. So let us suggest a simple **tip**:

Tuy nhiên, nhiều lập trình viên lại không có một kế hoạch nhất quán nào để xử lý chuyện cấp phát và giải phóng tài nguyên. Vậy nên chúng tôi xin đề xuất một mẹo (tip) đơn giản:

Tip 40: Finish What You Start

Tip 40: Hoàn Thành Những Gì Bạn Bắt Đầu

This tip is easy to apply in most circumstances. It simply means that the function or object that allocates a resource should be responsible for deallocating it. Let's see how it applies by looking at an example of some bad code—part of a Ruby program that opens a file, reads **customer** information from it, updates a field, and writes the result back. We've eliminated error handling to make the example clearer:

Mẹo này dễ áp dụng trong hầu hết hoàn cảnh. Nó đơn giản có nghĩa là: hàm hoặc đối tượng nào cấp phát một tài nguyên thì phải có trách nhiệm giải phóng tài nguyên đó. Hãy xem nó được áp dụng ra sao qua một ví dụ code dở — một phần của chương trình Ruby mở một tệp, đọc thông tin khách hàng từ đó, cập nhật một trường, rồi ghi kết quả trở lại. Chúng tôi đã lược bỏ phần xử lý lỗi để ví dụ rõ ràng hơn:

```
def read_customer @customer_file = File.open(@name + ".rec", "r") @balance = BigDecimal(@customer_file.gets)
end
```

```
def read_customer @customer_file = File.open(@name + ".rec", "r") @balance =
BigDecimal(@customer_file.gets) end
```

```
def write_customer @customer_file.rewind @customer_file.puts @balance.to_s @customer_file.close
end
```

```
def write_customer @customer_file.rewind @customer_file.puts @balance.to_s @customer_file.close
end
```

```
def update_customer(transaction_amount) read_customer @balance = @balance.add(transaction_amount,2)
write_customer end
```

```
def update_customer(transaction_amount) read_customer @balance = @balance.add(transaction_amount,2)
write_customer end
```

At first sight, the routine `update_customer` looks reasonable. It seems to implement the logic we require—reading a record, updating the balance, and writing the record back out. However, this tidiness hides a major problem. The routines `read_customer` and `write_customer` are tightly coupled [33] —they share the instance variable `customer_file`. `read_customer` opens the file and stores the file reference in `customer_file`, and then `write_customer` uses that stored reference to close the file when it finishes. This shared variable doesn't even appear in the `update_customer` routine.

Thoạt nhìn, hàm `update_customer` trông có vẻ hợp lý. Nó dường như cài đặt đúng logic ta cần—đọc một bản ghi, cập nhật số dư, rồi ghi bản ghi trở lại. Tuy nhiên, vẻ gọn gàng ấy che giấu một vấn đề lớn. Hai hàm `read_customer` và `write_customer` bị ghép nối chặt với nhau [33] —chúng dùng chung biến instance `customer_file`. `read_customer` mở tệp và lưu tham chiếu tệp vào `customer_file`, rồi `write_customer` dùng tham chiếu đã lưu đó để đóng tệp khi nó xong việc. Biến dùng chung này thậm chí còn chẳng hề xuất hiện trong hàm `update_customer`.

Why is this bad? Let's consider the unfortunate maintenance programmer who is told that the specification has changed—the balance should be updated only if the new value is not negative. They go into the source and change `update_customer`:

Vì sao chuyện này lại tệ? Hãy nghĩ đến anh lập trình viên bảo trì xấu số được báo rằng đặc tả đã thay đổi—số dư chỉ nên được cập nhật nếu giá trị mới không âm. Anh ta mở mã nguồn ra và sửa `update_customer`:

```
def update_customer(transaction_amount) read_customer if (transaction_amount >= 0.00) @balance
= @balance.add(transaction_amount,2) write_customer end end
```

```
def update_customer(transaction_amount) read_customer if (transaction_amount >= 0.00)
@balance = @balance.add(transaction_amount,2) write_customer end end
```

All seems fine during testing. However, when the code goes into production, it collapses after several hours, complaining of too

Mọi thứ có vẻ ổn trong quá trình kiểm thử. Tuy nhiên, khi code lên môi trường sản xuất, nó sụp đổ sau vài giờ, kêu ca rằng có quá

many open files. It turns out that `write_customer` is not getting called in some circumstances. When that happens, the file is not getting closed.

nhiều tệp đang mở. Hóa ra trong một số tình huống, `write_customer` không hề được gọi. Khi điều đó xảy ra, tệp không được đóng lại.

A very bad solution to this problem would be to deal with the special case in `update_customer`:

Một giải pháp rất tệ cho vấn đề này là xử lý trường hợp đặc biệt ngay trong `update_customer`:

```
def update_customer(transaction_amount) read_customer if (transaction_amount >= 0.00) @balance
+= BigDecimal(transaction_amount, 2) write_customer else @customer_file.close # Bad idea! end end
```

```
def update_customer(transaction_amount) read_customer if (transaction_amount >= 0.00)
@balance += BigDecimal(transaction_amount, 2) write_customer else @customer_file.close
# Ý tưởng tồi! end end
```

This will fix the problem—the file will now get closed regardless of the new balance—but the fix now means that three routines are coupled through the shared variable `customer_file`, and keeping track of when the file is open or not is going to start to get messy. We’re falling into a trap, and things are going to start going downhill rapidly if we continue on this course. This is not balanced!

Cách này sẽ vá được vấn đề—tệp giờ sẽ được đóng bất kể số dư mới là bao nhiêu—nhưng bản vá ấy lại khiến ba hàm bị ghép nối với nhau qua biến dùng chung `customer_file`, và việc theo dõi xem tệp đang mở hay đã đóng sẽ bắt đầu trở nên rối rắm. Ta đang sa vào một cái bẫy, và mọi thứ sẽ tuột dốc rất nhanh nếu cứ tiếp tục đi theo hướng này. Thế này là không cân bằng!

The **finish what you start** tip tells us that, ideally, the routine that allocates a resource should also free it. We can apply it here by **refactoring** the code slightly:

Mẹo “hoàn thành những gì bạn bắt đầu” nói với ta rằng, lý tưởng nhất là hàm nào cấp phát tài nguyên thì cũng nên giải phóng nó. Ta có thể áp dụng điều đó ở đây bằng cách tái cấu trúc (refactoring) đoạn code lại một chút:

```
def read_customer(file) @balance=BigDecimal(file.gets) end
  def read_customer(file) @balance=BigDecimal(file.gets) end
def write_customer(file) file.rewind file.puts @balance.to_s end
  def write_customer(file) file.rewind file.puts @balance.to_s end
def update_customer(transaction_amount) file=File.open(@name + “.rec”, “r+”) #>- read_customer(file)
# | @balance = @balance.add(transaction_amount,2) # | file.close # <- end Instead of holding on to
the file reference, we’ve changed the code to pass it as a parameter. [34] Now all the responsibility
for the file is in the update_customer routine. It opens the file and (finishing what it starts) closes it
before returning. The routine balances the use of the file: the open and close are in the same place,
and it is apparent that for every open there will be a corresponding close. The refactoring also
removes an ugly shared variable.
```

```
def update_customer(transaction_amount) file=File.open(@name + “.rec”, “r+”) #>-
read_customer(file) # | @balance = @balance.add(transaction_amount,2) # | file.close #
<- end Thay vì ôm khư khư tham chiếu tệp, ta đã đổi code để truyền nó vào như một tham
số. [34] Giờ toàn bộ trách nhiệm với tệp nằm gọn trong hàm update_customer. Nó mở tệp
và (hoàn thành những gì mình bắt đầu) đóng tệp lại trước khi trả về. Hàm này giữ cân
bằng cho việc sử dụng tệp: lệnh open và lệnh close nằm cùng một chỗ, và ta thấy rõ rằng
cứ mỗi lần open thì sẽ có một lần close tương ứng. Việc tái cấu trúc (refactoring) này còn
loại bỏ được một biến dùng chung xấu xí.
```

There’s another small but important improvement we can make. In many modern languages, you can scope the lifetime of a resource to an enclosed block of some sort. In Ruby, there’s a variation of the file open that passes in the open file reference to a block, shown here between the `do` and the `end`:

Còn một cải tiến nhỏ nhưng quan trọng nữa mà ta có thể làm. Trong nhiều ngôn ngữ hiện đại, bạn có thể giới hạn vòng đời của một tài nguyên vào trong một khối lệnh bao quanh nào đó. Trong Ruby, có một biến thể của lệnh mở tệp cho phép truyền tham chiếu tệp đang mở vào một khối, ở đây nằm giữa `do` và `end`:

```
def update_customer(transaction_amount) File.open(@name + “.rec”, “r+”) do |file| #>-
read_customer(file) # | @balance = @balance.add(transaction_amount,2) # | write_customer(file) #
| end # <- end
```



```
def update_customer(transaction_amount) File.open(@name + ".rec" , "r+" ) do |file|
# >- read_customer(file) # | @balance = @balance.add(transaction_amount,2) # |
write_customer(file) # | end # <- end
```

In this case, at the end of the block the file variable goes out of scope and the external file is closed. Period. No need to remember to close the file and release the source, it is guaranteed to happen for you.

Trong trường hợp này, đến cuối khối lệnh thì biến file ra khỏi phạm vi và tệp bên ngoài được đóng lại. Chấm hết. Không cần phải nhớ đóng tệp và trả lại tài nguyên, điều đó được bảo đảm sẽ tự xảy ra cho bạn.

When in doubt, it always pays to reduce scope.

Khi còn nghi ngờ, thu hẹp phạm vi luôn là điều có lợi.

Tip 41: Act Locally

Tip 41: Hành Động Cục Bộ

Balancing Over Time

Cân Bằng Theo Thời Gian

In this **topic** we're mostly looking at **ephemeral** resources used by your running **process**. But you might want to consider what other messes you might be leaving behind.

Trong mục này, chúng ta chủ yếu nhìn vào các tài nguyên phù du (ephemeral) mà tiến trình đang chạy của bạn sử dụng. Nhưng có thể bạn cũng nên cân nhắc xem mình còn để lại những đồng lõa nào khác.

For instance, how are your logging files handled? You are creating data and using up storage space. Is there something in place to rotate the logs and clean them up? How about for your unofficial debug files you're dropping? If you're adding logging records in a database, is there a similar process in place to expire them? For anything that you create that takes up a finite resource, consider how to balance it.

Chẳng hạn, các tệp log của bạn được xử lý thế nào? Bạn đang tạo ra dữ liệu và ngốn dần dung lượng lưu trữ. Có cơ chế nào để xoay vòng log và dọn dẹp chúng không? Còn những tệp debug không chính thức mà bạn rải ra thì sao? Nếu bạn thêm các bản ghi log vào cơ sở dữ liệu, liệu có một quy trình tương tự để cho chúng hết hạn không? Với bất cứ thứ gì bạn tạo ra mà chiếm dụng một tài nguyên hữu hạn, hãy cân nhắc cách cân bằng nó.

What else are you leaving behind?

Bạn còn để lại những gì phía sau nữa?

Nest Allocations — Lồng Các Lần Cấp Phát

The basic pattern for resource allocation can be extended for routines that need more than one resource at a time. There are just two more suggestions:

Khuôn mẫu cơ bản cho việc cấp phát tài nguyên có thể được mở rộng cho những hàm cần nhiều hơn một tài nguyên cùng lúc. Chỉ có thêm hai gợi ý:

Deallocate resources in the opposite order to that in which you allocate them. That way you won't **orphan** resources if one resource contains references to another.

Hãy giải phóng tài nguyên theo thứ tự ngược lại với thứ tự bạn cấp phát chúng. Như vậy bạn sẽ không bỏ rơi (orphan) tài nguyên nào nếu một tài nguyên chứa tham chiếu đến tài nguyên khác.

When allocating the same set of resources in different places in your code, always allocate them in the same order. This will reduce the possibility of **deadlock**. (If process A claims resource1 and is about to claim resource2, while process B has claimed resource2 and is trying to get resource1, the two processes will wait forever.)

Khi cấp phát cùng một bộ tài nguyên ở những chỗ khác nhau trong code, hãy luôn cấp phát chúng theo cùng một thứ tự. Việc này sẽ giảm khả năng xảy ra kẹt khóa (deadlock). (Nếu tiến trình A đã giành resource1 và sắp giành resource2, trong khi tiến trình B đã giành resource2 và đang cố lấy resource1, thì cả hai tiến trình sẽ chờ nhau mãi mãi.)

It doesn't matter what kind of resources we're using— transactions, network connections, memory, files, threads, windows—the basic pattern applies: whoever allocates a resource should be responsible for deallocating it. However, in some languages we can develop the concept further.

Không quan trọng ta đang dùng loại tài nguyên nào — giao dịch, kết nối mạng, bộ nhớ, tệp, luồng, cửa sổ — khuôn mẫu cơ bản vẫn áp dụng: ai cấp phát một tài nguyên thì phải chịu trách nhiệm giải phóng nó. Tuy nhiên, trong một số ngôn ngữ, ta có thể phát triển khái niệm này xa hơn.

Objects And Exceptions — Đối Tượng Và Ngoại Lệ

The equilibrium between allocations and deallocations is reminiscent of an object-oriented class's **constructor** and **destructor**. The class represents a resource, the constructor gives you a particular object of that resource type, and the destructor removes it from your scope.

Thế cân bằng giữa cấp phát và giải phóng gợi nhớ đến hàm khởi tạo (constructor) và hàm hủy (destructor) của một lớp hướng đối tượng. Lớp đại diện cho một tài nguyên, hàm khởi tạo trao cho bạn một đối tượng cụ thể thuộc loại tài nguyên đó, còn hàm hủy thì gỡ nó khỏi phạm vi của bạn.

If you are programming in an object-oriented language, you may find it useful to encapsulate resources in classes. Each time you need a particular resource type, you instantiate an object of that class. When the object goes out of scope, or is reclaimed by the **garbage collector**, the object's destructor then deallocates the wrapped resource.

Nếu bạn đang lập trình bằng một ngôn ngữ hướng đối tượng, bạn có thể thấy việc đóng gói tài nguyên vào trong các lớp là hữu ích. Mỗi lần cần một loại tài nguyên cụ thể, bạn tạo một thực thể (instance) của lớp đó. Khi đối tượng ra khỏi phạm vi, hoặc bị bộ thu gom rác (garbage collector) thu hồi, thì hàm hủy của đối tượng sẽ giải phóng tài nguyên được bọc bên trong.

This approach has particular benefits when you're working with languages where exceptions can interfere with resource deallocation.

Cách tiếp cận này đặc biệt có lợi khi bạn làm việc với những ngôn ngữ mà ngoại lệ (exception) có thể cản trở việc giải phóng tài nguyên.

Balancing And Exceptions — Cân Bằng Và Ngoại Lệ

Languages that support exceptions can make resource deallocation tricky. If an **exception** is thrown, how do you guarantee that everything allocated prior to the exception is tidied up? The answer

depends to some extent on the language support. You generally have two choices:

Những ngôn ngữ hỗ trợ ngoại lệ có thể khiến việc giải phóng tài nguyên trở nên hóc búa. Nếu một ngoại lệ bị ném ra, làm sao bạn đảm bảo rằng mọi thứ đã cấp phát trước đó đều được dọn dẹp gọn gàng? Câu trả lời phần nào phụ thuộc vào sự hỗ trợ của ngôn ngữ. Nhìn chung bạn có hai lựa chọn:

1. Use variable scope (for example, stack variables in C++ or Rust) 2. Use a finally clause in a try ... catch block

1. Dùng phạm vi của biến (ví dụ, biến trên stack trong C++ hay Rust) 2. Dùng mệnh đề finally trong khối try ... catch

With usual scoping rules in languages such as C++ or Rust, the variable's memory will be reclaimed when the variable goes out of scope via a return, block exit, or exception. But you can also hook in to the variable's destructor to cleanup any external

Với các quy tắc phạm vi thông thường trong những ngôn ngữ như C++ hay Rust, bộ nhớ của biến sẽ được thu hồi khi biến ra khỏi phạm vi qua một lệnh return, thoát khỏi khối lệnh, hoặc một ngoại lệ. Nhưng bạn cũng có thể móc nối vào hàm hủy của biến để dọn dẹp bất kỳ tài nguyên ngoài nào. Trong ví dụ này, biến Rust tên là accounts

resources. In this example, the Rust variable named accounts will automatically close the associated file when it goes out of scope:

tài nguyên. Trong ví dụ này, biến Rust tên là accounts sẽ tự động đóng tệp tương ứng khi nó ra khỏi phạm vi:

```
{ let mut accounts = File::open( "mydata.txt" )?; // >- // use 'accounts' // | ... // | } // <- // 'accounts' is now out of scope, and the file is // automatically closed
```

```
{ let mut accounts = File::open( "mydata.txt" )?; // >- // dùng 'accounts' // | ... // | } // <- // 'accounts' giờ đã ra khỏi phạm vi, và tệp được // tự động đóng
```

The other option, if the language supports it, is the finally clause. A finally clause will ensure that the specified code will run whether or not an exception was raised in the try ... catch block:

Lựa chọn còn lại, nếu ngôn ngữ có hỗ trợ, là mệnh đề finally . Mệnh đề finally bảo đảm rằng đoạn code được chỉ định sẽ chạy, bất kể có ngoại lệ nào bị ném ra trong khối try ... catch hay không:

```
try // some dodgy stuff catch // exception was raised finally // clean up in either case
```

```
try // làm vài việc rủi ro catch // một ngoại lệ đã bị ném ra finally // dọn dẹp trong cả hai trường hợp
```

However, there is a catch.

Tuy nhiên, có một cái bẫy ở đây.

An Exception **Antipattern** We commonly see folks writing something like this:

Một Phản Mẫu Về Ngoại Lệ Chúng tôi thường thấy người ta viết kiểu thế này:

```
begin thing = allocate_resource() process(thing) finally deallocate(thing) end
```

```
begin thing = allocate_resource() process(thing) finally deallocate(thing) end
```

Can you see what's wrong?

Bạn có thấy chỗ nào sai không?

What happens if the resource allocation fails and raises an exception? The finally clause will catch it, and try to deallocate a

Chuyện gì xảy ra nếu việc cấp phát tài nguyên thất bại và ném ra một ngoại lệ? Mệnh đề finally sẽ bắt được nó, và cố giải phóng một

thing that was never allocated.

thing chưa hề được cấp phát.

The correct pattern for handling resource deallocation in an environment with exceptions is

Khuôn mẫu đúng để xử lý việc giải phóng tài nguyên trong một môi trường có ngoại lệ là:

thing = allocate_resource() begin process(thing) finally deallocate(thing) end

thing = allocate_resource() begin process(thing) finally deallocate(thing) end

When You Can'T Balance Resources — Khi Bạn Không Thể Cân Bằng Tài Nguyên

There are times when the basic resource allocation pattern just isn't appropriate. Commonly this is found in programs that use dynamic data structures. One routine will allocate an area of memory and link it into some larger structure, where it may stay for some time.

Có những lúc khuôn mẫu cấp phát tài nguyên cơ bản đơn giản là không phù hợp. Thường thì điều này gặp ở những chương trình dùng cấu trúc dữ liệu động. Một hàm sẽ cấp phát một vùng nhớ và móc nó vào một cấu trúc lớn hơn, nơi nó có thể nằm lại đó một thời gian.

The trick here is to establish a **semantic invariant** for memory allocation. You need to decide who is responsible for data in an aggregate data structure. What happens when you deallocate the top-level structure? You have three main options:

Mẹo ở đây là thiết lập một bất biến ngữ nghĩa (semantic invariant) cho việc cấp phát bộ nhớ. Bạn cần quyết định ai chịu trách nhiệm cho dữ liệu trong một cấu trúc dữ liệu tổng hợp. Điều gì xảy ra khi bạn giải phóng cấu trúc cấp cao nhất? Bạn có ba lựa chọn chính:

The top-level structure is also responsible for freeing any substructures that it contains. These structures then recursively delete data they contain, and so on.

Cấu trúc cấp cao nhất cũng chịu trách nhiệm giải phóng mọi cấu trúc con mà nó chứa. Các cấu trúc này lại đệ quy xóa dữ liệu chúng chứa, và cứ thế tiếp tục.

The top-level structure is simply deallocated. Any structures that it pointed to (that are not referenced elsewhere) are orphaned.

Cấu trúc cấp cao nhất đơn giản được giải phóng. Mọi cấu trúc mà nó trỏ tới (mà không được tham chiếu ở nơi nào khác) sẽ bị bỏ rơi.

The top-level structure refuses to deallocate itself if it contains any substructures.

Cấu trúc cấp cao nhất từ chối tự giải phóng nếu nó còn chứa bất kỳ cấu trúc con nào.

The choice here depends on the circumstances of each individual data structure. However, you need to make it explicit

Lựa chọn ở đây tùy thuộc vào hoàn cảnh của từng cấu trúc dữ liệu cụ thể. Tuy nhiên, bạn cần làm cho nó tường minh

for each, and implement your decision consistently. Implementing any of these options in a procedural language such as C can be a problem: data structures themselves are not active. Our preference in these circumstances is to write a module for each major structure that provides standard allocation and deallocation facilities for that structure. (This module can also provide facilities such as debug printing, serialization, deserialization, and traversal hooks.)

cho từng cấu trúc, và cài đặt quyết định của mình một cách nhất quán. Cài đặt bất kỳ lựa chọn nào trong số này bằng một ngôn ngữ thủ tục như C có thể là một vấn đề: bản thân các cấu trúc dữ liệu không chủ động. Trong những hoàn cảnh này, sở thích của chúng tôi là viết một module cho mỗi cấu trúc chính, cung cấp các tiện ích cấp phát và giải phóng tiêu chuẩn cho cấu trúc đó. (Module này cũng có thể cung cấp những tiện ích như in debug, tuần tự hóa (serialization), giải tuần tự hóa (deserialization), và các móc duyệt cấu trúc.)

Checking The Balance — Kiểm Tra Sự Cân Bằng

Because **Pragmatic** Programmers trust no one, including ourselves, we feel that it is always a good idea to build code that actually checks that resources are indeed freed appropriately. For most applications, this normally means producing wrappers for each type of resource, and using these wrappers to keep track of all allocations and deallocations. At certain points in your code, the program logic will dictate that the resources will be in a certain **state**: use the wrappers to check this. For example, a long-running program that services requests will probably have a single point at the top of its main processing loop where it waits for the next request to arrive. This is a good place to ensure that resource usage has not increased since the last execution of the loop.

Vì Lập trình viên Thực dụng (Pragmatic Programmer) không tin tưởng ai cả, kể cả chính mình, nên chúng tôi cho rằng luôn là ý hay khi xây dựng code thực sự kiểm tra xem tài nguyên có thực sự được giải phóng đúng cách hay không. Với hầu hết ứng dụng, điều này thường có nghĩa là tạo ra các lớp bọc (wrapper) cho từng loại tài nguyên, rồi dùng các lớp bọc đó để theo dõi mọi lần cấp phát và giải phóng. Tại một số điểm nhất định trong code, logic chương trình sẽ quy định rằng tài nguyên phải ở một trạng thái nhất định: hãy dùng các lớp bọc để kiểm tra điều này. Ví dụ, một chương trình chạy lâu dài phục vụ các yêu cầu có lẽ sẽ có một điểm duy nhất ở đầu vòng lặp xử lý chính của nó, nơi nó chờ yêu cầu tiếp theo đến. Đây là chỗ tốt để bảo đảm rằng mức sử dụng tài nguyên không hề tăng lên kể từ lần chạy trước của vòng lặp.

At a lower, but no less useful level, you can invest in tools that (among other things) check your running programs for memory leaks.

Ở một mức thấp hơn, nhưng không kém phần hữu ích, bạn có thể đầu tư vào những công cụ (mà trong số nhiều việc khác) kiểm tra chương trình đang chạy của bạn xem có rò rỉ bộ nhớ hay không.

Related Sections Include — Các mục liên quan

Topic 24, Dead Programs Tell No Lies

Topic 24, Chương Trình Chết Không Biết Nói Dối

Topic 30, Transforming Programming

Topic 30, Biến Đổi Lập Trình

Topic 33, Breaking **Temporal Coupling**

Topic 33, Phá Vỡ Ghép Nối Thời Gian

Challenges — Thử thách

Although there are no guaranteed ways of ensuring that you always free resources, certain design techniques, when applied consistently, will help. In the text we discussed how establishing a semantic invariant for major data structures could direct memory deallocation decisions. Consider how Topic 23, **Design by Contract**, could help refine this idea.

Mặc dù không có cách nào đảm bảo chắc chắn rằng bạn sẽ luôn giải phóng tài nguyên, nhưng một số kỹ thuật thiết kế, khi được áp dụng nhất quán, sẽ giúp ích. Trong phần trên, chúng ta đã bàn về việc thiết lập một bất biến ngữ nghĩa cho các cấu trúc dữ liệu chính có thể định hướng các quyết định giải phóng bộ nhớ ra sao. Hãy cân nhắc xem Topic 23, Thiết Kế Theo Hợp Đồng, có thể giúp tinh chỉnh ý tưởng này như thế nào.

Exercise 17 (possible answer)

Bài tập 17 (đáp án khả dĩ)

Some C and C++ developers make a point of setting a pointer to NULL after they deallocate the memory it references. Why is this a good idea?

Một số lập trình viên C và C++ có thói quen gán con trỏ về NULL sau khi đã giải phóng vùng nhớ mà nó tham chiếu tới. Vì sao đó là một ý hay?

Exercise 18 (possible answer)

Bài tập 18 (đáp án khả dĩ)

Some Java developers make a point of setting an object variable to NULL after they have finished using the object. Why is this a good idea?

Một số lập trình viên Java có thói quen gán biến đối tượng về NULL sau khi đã dùng xong đối tượng đó. Vì sao đó là một ý hay?

It's tough to make predictions, especially about the future. Lawrence “Yogi” Berra, after a Danish Proverb

Thật khó để đưa ra dự đoán, nhất là về tương lai. Lawrence “Yogi” Berra, theo một câu tục ngữ Đan Mạch

Topic 27. Don't Outrun Your Headlights — Topic 27. Đừng Chạy Vượt Đèn Pha Của Mình

It's late at night, dark, pouring rain. The two-seater whips around the tight curves of the twisty little mountain roads, barely holding the corners. A hairpin comes up and the car misses it, crashing though the skimpy guardrail and soaring to a fiery crash in the valley below. **State** troopers arrive on the scene, and the senior officer sadly shakes their head. “Must have outrun their headlights.”

Đã khuya, trời tối đen, mưa như trút. Chiếc xe hai chỗ lao vun vút qua những khúc cua gấp trên con đường núi nhỏ ngoằn ngoèo, bám đường một cách chật vật. Một khúc cua chữ chi hiện ra và chiếc xe trượt khỏi nó, tông xuyên qua lan can mỏng manh rồi bay xuống thung lũng bên dưới trong một vụ tai nạn rục rủa. Cảnh sát tuần tra tiểu bang đến hiện trường, viên sĩ quan cấp cao buồn bã lắc đầu. “Chắc lại chạy vượt đèn pha của mình rồi.”

Had the speeding two-seater been going faster than the speed of light? No, that speed limit is firmly fixed. What the officer referred to was the driver's ability to stop or steer in time in response to the headlight's illumination.

Phải chăng chiếc xe hai chỗ đang phóng nhanh hơn cả tốc độ ánh sáng? Không, giới hạn tốc độ đó là cố định bất di bất dịch. Điều viên sĩ quan muốn nói là khả năng của tài xế trong việc dừng hay đánh lái kịp thời để phản ứng với vùng sáng mà đèn pha rọi tới.

Headlights have a certain limited range, known as the throw distance . Past that point, the light spread is too diffuse to be effective. In addition, headlights only project in a straight line, and won't illuminate anything off-axis, such as curves, hills, or dips in the road. According to the National Highway Traffic Safety Administration, the average distance illuminated by lowbeam headlights is about 160 feet. Unfortunately, stopping distance at 40mph is 189 feet, and at 70mph a whopping 464 feet. [35] So indeed, it's actually pretty easy to outrun your headlights.

Đèn pha có một tầm chiếu hữu hạn, gọi là quãng hắt sáng (throw distance). Quá điểm đó, chùm sáng đã tán quá loãng để còn hiệu quả. Thêm nữa, đèn pha chỉ rọi theo đường thẳng, nên sẽ không soi tỏ bất cứ thứ gì nằm lệch trục, chẳng hạn khúc cua, đoạn dốc hay chỗ trũng trên đường. Theo Cục Quản lý An toàn Giao thông Đường bộ Quốc gia Hoa Kỳ (NHTSA), quãng đường mà đèn cốt chiếu sáng được trung bình vào khoảng 160 feet. Đáng tiếc, quãng đường dừng ở tốc độ 40 dặm/giờ là 189 feet, còn ở 70 dặm/giờ thì lên tới những 464 feet. [35] Vậy nên đúng là chạy vượt đèn pha của mình hóa ra khá dễ.

In software development, our “headlights” are similarly limited. We can't see too far ahead into the future, and the further offaxis you look, the darker it gets. So **Pragmatic** Programmers have a firm rule:

Trong phát triển phần mềm, “đèn pha” của chúng ta cũng bị giới hạn tương tự. Chúng ta không thể nhìn quá xa vào tương lai, và càng nhìn lệch khỏi trục, trời càng tối hơn. Vậy nên các Lập trình viên Thực dụng (Pragmatic Programmer) có một quy tắc cứng:

Tip 42: Take Small Steps—Always

Tip 42: Hãy Bước Những Bước Nhỏ—Luôn Luôn Như Vậy

Always take small, deliberate steps, checking for feedback and adjusting before proceeding. Consider that the rate of feedback is your speed limit. You never take on a step or a task that's “too big.”

Hãy luôn bước những bước nhỏ, có chủ đích, kiểm tra phản hồi và điều chỉnh trước khi đi tiếp. Hãy xem nhịp độ phản hồi chính là giới hạn tốc độ của bạn. Bạn không bao giờ nhận một bước hay một nhiệm vụ “quá lớn”.

What do we mean exactly by feedback? Anything that independently confirms or disproves your action. For example:

Chính xác thì chúng tôi muốn nói gì khi nhắc tới phản hồi? Bất cứ thứ gì xác nhận hoặc bác bỏ hành động của bạn một cách độc lập. Ví dụ:

Results in a REPL provide feedback on your understanding of APIs and algorithms

Kết quả trong một REPL cho bạn phản hồi về mức độ bạn hiểu các API và thuật toán

Unit tests provide feedback on your last code change

Các kiểm thử đơn vị (unit test) cho bạn phản hồi về thay đổi mã nguồn vừa rồi của bạn

User demo and conversation provide feedback on features and usability

Buổi demo cho người dùng và trò chuyện với họ cho bạn phản hồi về tính năng và khả năng sử dụng

What's a task that's too big? Any task that requires “fortune telling.” Just as the car headlights have limited throw, we can only see into the future perhaps one or two steps, maybe a few hours or days

at most. Beyond that, you can quickly get past educated guess and into wild speculation. You might find yourself slipping into fortune telling when you have to:

Thế nào là một nhiệm vụ quá lớn? Bất kỳ nhiệm vụ nào đòi hỏi “bói toán”. Cũng như đèn pha của xe có tầm hắt sáng giới hạn, chúng ta chỉ có thể nhìn vào tương lai chừng một hai bước, nhiều lắm là vài giờ hay vài ngày. Vượt quá đó, bạn nhanh chóng rời khỏi vùng phỏng đoán có cơ sở mà bước vào địa hạt suy đoán hoang đường. Bạn có thể thấy mình đang sa vào trò bói toán khi phải:

Estimate completion dates months in the future

Ước lượng (estimating) ngày hoàn thành cho mấy tháng sau

Plan a design for future maintenance or extendability

Lên kế hoạch một thiết kế để bảo trì hay mở rộng trong tương lai

Guess user’s future needs

Đoán nhu cầu tương lai của người dùng

Guess future tech availability

Đoán công nghệ nào sẽ có trong tương lai

But, we hear you cry, aren’t we supposed to design for future maintenance? Yes, but only to a point: only as far ahead as you can see. The more you have to predict what the future will look like, the more risk you incur that you’ll be wrong. Instead of wasting effort designing for an uncertain future, you can always fall back on designing your code to be replaceable. Make it easy to throw out your code and replace it with something better suited. Making code replaceable will also help with **cohesion**, **coupling**, and **DRY**, leading to a better design overall.

Nhưng, chúng tôi nghe bạn kêu lên, chẳng phải chúng ta được khuyên nên thiết kế để bảo trì về sau hay sao? Đúng, nhưng chỉ tới một mức thôi: chỉ xa đúng tới chừng bạn còn nhìn thấy được. Bạn càng phải dự đoán tương lai trông sẽ ra sao, bạn càng gánh thêm rủi ro là mình đoán sai. Thay vì phí công thiết kế cho một tương lai bất định, bạn luôn có thể lùi về phương án thiết kế mã nguồn của mình sao cho dễ thay thế. Hãy làm sao để có thể vứt bỏ mã của bạn và thay nó bằng thứ gì đó phù hợp hơn một cách dễ dàng. Khiến mã dễ thay thế cũng sẽ giúp ích cho tính cố kết (cohesion), sự ghép nối (coupling) và DRY — Đừng Lặp Lại Chính Mình, dẫn tới một thiết kế tổng thể tốt hơn.

Even though you may feel confident of the future, there’s always the chance of a black swan around the corner.

Dù bạn có thể thấy tự tin về tương lai, vẫn luôn có khả năng một con thiên nga đen đang chực chờ ngay góc khuất.

Black Swans — Những Con Thiên Nga Đen

In his book, *The Black Swan: The Impact of the Highly Improbable* [Tal10], Nassim Nicholas Taleb posits that all significant events in history have come from high-profile, hard-to-predict, and rare events that are beyond the realm of normal expectations. These outliers, while statistically rare, have disproportionate effects. In addition, our own cognitive biases tend to blind us to changes creeping up on the edges of our work (see **Topic 4, Stone Soup and Boiled Frogs**).

Trong cuốn sách *The Black Swan: The Impact of the Highly Improbable* [Tal10] (Thiên Nga Đen: Tác Động Của Những Điều Cực Kỳ Khó Xảy Ra), Nassim Nicholas Taleb lập luận rằng mọi sự kiện trọng đại trong lịch sử đều đến từ những biến cố nổi bật, khó dự đoán và

hiếm gặp, nằm ngoài địa hạt của những kỳ vọng thông thường. Những điểm dị biệt này, dấu hiếm về mặt thống kê, lại gây ra những tác động không cân xứng. Thêm nữa, chính các thiên kiến nhận thức của ta có xu hướng khiến ta mù lòa trước những thay đổi đang lặng lẽ bò tới từ rìa công việc của mình (xem Topic 4, Súp Đá Và Con Ếch Bị Luộc).

Around the time of the first edition of *The Pragmatic Programmer*, debate raged in computer magazines and online forums over the burning question: “Who would win the desktop GUI wars, Motif or OpenLook?” [36] It was the wrong question. Odds are you’ve probably never heard of these technologies as neither “won” and the browser-centric web quickly dominated the landscape.

Vào quãng thời gian ấn bản đầu tiên của *The Pragmatic Programmer* ra đời, các tạp chí máy tính và diễn đàn trực tuyến tranh cãi nảy lửa quanh một câu hỏi nóng bỏng: “Ai sẽ thắng trong cuộc chiến GUI trên máy để bàn, Motif hay OpenLook?” [36] Đó là một câu hỏi sai. Nhiều khả năng bạn còn chưa từng nghe tới hai công nghệ này, bởi chẳng bên nào “thắng” cả, và web lấy trình duyệt làm trung tâm đã nhanh chóng thống trị toàn cảnh.

Tip 43: Avoid Fortune-Telling

Tip 43: Tránh Bói Toán

Much of the time, tomorrow looks a lot like today. But don’t count on it.

Phần lớn thời gian, ngày mai trông na ná ngày hôm nay. Nhưng đừng đặt cược vào điều đó.

Related Sections Include — Các mục liên quan

Topic 12, **Tracer Bullets**

Topic 12, Đạn Vạch Đường (Tracer Bullets)

Topic 13, Prototypes and Post-it Notes

Topic 13, Bản Mẫu Và Giấy Ghi Chú Post-it

Topic 40, **Refactoring**

Topic 40, Tái Cấu Trúc (Refactoring)

Topic 41, **Test to Code**

Topic 41, Kiểm Thử Đối Chiếu Mã

Topic 48, The Essence of **Agility**

Topic 48, Bản Chất Của Sự Linh Hoạt

Topic 50, Coconuts Don’t Cut It

Topic 50, Dừa Thì Không Ăn Thua

Footnotes

Footnotes

[30] Based in part on earlier work by Dijkstra, Floyd, Hoare, Wirth, and others.

[30] Dựa một phần vào công trình trước đó của Dijkstra, Floyd, Hoare, Wirth và những người khác.

[31] In C and C++ these are usually implemented as macros. In Java, assertions are disabled by default. Invoke the Java VM with the `-enableassertions` flag to enable them, and leave them enabled.

[31] Trong C và C++, chúng thường được cài đặt dưới dạng macro. Trong Java, assertion bị tắt mặc định. Hãy khởi động máy ảo Java với cờ `-enableassertions` để bật chúng lên, và cứ để chúng bật.

[32] <http://www.eps.mcgill.ca/jargon/jargon.html#heisenbug>

[32] <http://www.eps.mcgill.ca/jargon/jargon.html#heisenbug>

[33] For a discussion of the dangers of coupled code, see Topic 28, **Decoupling**.

[33] Để bàn về những mối nguy của mã bị ghép nối chặt, xem Topic 28, Gỡ Ghép Nối (Decoupling).

[34] See the **tip** here.

[34] Xem mẹo ở đây.

[35] Per the NHTSA, Stopping Distance = Reaction Distance + Braking Distance, assuming an average reaction time of 1.5s and deceleration of 17.02ft/s².

[35] Theo NHTSA, Quãng Đường Dừng = Quãng Đường Phản Xạ + Quãng Đường Phanh, giả định thời gian phản xạ trung bình 1,5 giây và gia tốc giảm 17,02 ft/s².

[36] Motif and OpenLook were GUI standards for X-Windows based Unix workstations.

[36] Motif và OpenLook là các chuẩn GUI dành cho máy trạm Unix trên nền X-Windows.

Copyright © 2020 Pearson Education, Inc.

Copyright © 2020 Pearson Education, Inc.

Bend, or Break — Uốn cong, hoặc Gãy

Life doesn't stand still. Neither can the code that we write. In order to keep up with today's near-frantic pace of change, we need to make every effort to write code that's as loose—as flexible—as possible. Otherwise we may find our code quickly becoming outdated, or too brittle to fix, and may ultimately be left behind in the mad dash toward the future.

Cuộc đời không bao giờ đứng yên. Mã chúng ta viết cũng vậy. Để bắt kịp nhịp thay đổi gần như điên cuồng của thời nay, ta phải nỗ lực hết sức để viết mã thật lỏng lẻo—thật linh hoạt—càng nhiều càng tốt. Nếu không, ta có thể thấy mã của mình nhanh chóng lỗi thời, hoặc giòn đến mức không sửa nổi, và rốt cuộc bị bỏ lại phía sau trong cuộc chạy đua hối hả về phía tương lai.

Back in **Topic 11, Reversibility** we talked about the perils of irreversible decisions. In this chapter, we'll tell you how to make reversible decisions, so your code can stay flexible and adaptable in the face of an uncertain world.

Quay lại Topic 11, Reversibility, chúng ta đã bàn về hiểm họa của những quyết định không thể đảo ngược. Trong chương này, chúng tôi sẽ chỉ cho bạn cách đưa ra những quyết định có thể đảo ngược, để mã của bạn luôn linh hoạt và thích nghi được trước một thế giới đầy bất định.

First we look at **coupling**—the dependencies between bits of code. Topic 28, **Decoupling** shows how to keep separate concepts separate, decreasing coupling.

Trước hết, ta xem xét sự ghép nối (coupling)—những phụ thuộc giữa các mẫu mã. Topic 28, Decoupling cho thấy cách giữ các khái niệm riêng biệt nằm tách bạch, qua đó giảm sự ghép nối.

Next, we'll look at different techniques you can use when Topic 29, Juggling the Real World . We'll examine four different strategies to help manage and react to events—a critical aspect of modern software applications.

Tiếp theo, ta sẽ xem các kỹ thuật khác nhau bạn có thể dùng trong Topic 29, Juggling the Real World. Chúng ta sẽ khảo sát bốn chiến lược khác nhau giúp quản lý và phản ứng với các sự kiện—một khía cạnh trọng yếu của ứng dụng phần mềm hiện đại.

Traditional procedural and object-oriented code might be too tightly coupled for your purposes. In Topic 30, Transforming

Mã thủ tục và hướng đối tượng truyền thống có thể bị ghép nối quá chặt so với mục đích của bạn. Trong Topic 30, Transforming

Programming , we'll take advantage of the more flexible and clearer style offered by function pipelines, even if your language doesn't support them directly.

Programming, chúng ta sẽ tận dụng phong cách linh hoạt và sáng sủa hơn mà các đường ống hàm (function pipelines) mang lại, ngay cả khi ngôn ngữ của bạn không hỗ trợ chúng trực tiếp.

Common object-oriented style can tempt you with another trap. Don't fall for it, or you'll end up paying a hefty Topic 31, **Inheritance Tax** . We'll explore better alternatives to keep your code flexible and easier to change.

Phong cách hướng đối tượng phổ biến có thể dụ bạn vào một cái bẫy khác. Đừng mắc bẫy, kẻo bạn sẽ phải trả một khoản nặng nề ở Topic 31, Inheritance Tax. Chúng ta sẽ khám phá những lựa chọn thay thế tốt hơn để giữ mã linh hoạt và dễ thay đổi hơn.

And of course a good way to stay flexible is to write less code. Changing code leaves you open to the possibility of introducing new bugs. Topic 32, **Configuration** will explain how to move details out of the code completely, where they can be changed more safely and easily.

Và dĩ nhiên, một cách hay để giữ sự linh hoạt là viết ít mã hơn. Thay đổi mã khiến bạn dễ vô tình đưa thêm lỗi mới vào. Topic 32, Configuration sẽ giải thích cách đưa các chi tiết ra hẳn khỏi mã, nơi chúng có thể được thay đổi an toàn và dễ dàng hơn.

All these techniques will help you write code that bends and doesn't break.

Tất cả những kỹ thuật này sẽ giúp bạn viết được loại mã uốn cong mà không gãy.

When we try to pick out anything by itself, we find it hitched to everything else in the Universe. John Muir, My First Summer in the Sierra

Khi ta cố tách riêng một thứ gì đó, ta lại thấy nó gắn liền với mọi thứ khác trong Vũ trụ. John Muir, My First Summer in the Sierra

Topic 28. Decoupling — Topic 28. Gỡ ghép nối

In Topic 8, The Essence of Good Design we claim that using good design principles will make the code you write easy to change. Coupling is the enemy of change, because it links together things that must change in parallel. This makes change more difficult: either you spend time tracking down all the parts that need changing, or you spend time wondering why things broke when you changed “just one thing” and not the other things to which it was coupled.

Trong Topic 8, The Essence of Good Design, chúng tôi khẳng định rằng dùng các nguyên tắc thiết kế tốt sẽ khiến mã bạn viết dễ thay đổi. Sự ghép nối (coupling) là kẻ thù của sự thay đổi, bởi nó cột chặt với nhau những thứ buộc phải thay đổi song song. Điều này khiến việc thay đổi khó hơn: hoặc bạn mất thời gian truy lùng tất cả các phần cần đổi, hoặc bạn mất thời gian băn khoăn tại sao mọi thứ vỡ tung khi bạn chỉ đổi “đúng một thứ” mà không đổi những thứ khác đang ghép nối với nó.

When you are designing something you want to be rigid, a bridge or a tower perhaps, you couple the components together:

Khi bạn thiết kế một thứ muốn nó cứng cáp, ví dụ một cây cầu hay một tòa tháp, bạn ghép các thành phần lại với nhau:

images/links-01.png

images/links-01.png

The links work together to make the structure rigid.

Các mối liên kết phối hợp với nhau để làm cho kết cấu cứng vững.

Compare that with something like this:

Hãy so sánh với một thứ kiểu như thế này:

images/links-02.png

images/links-02.png

Here there's no structural rigidity: individual links can change and others just accommodate it.

Ở đây không có độ cứng kết cấu nào: từng mắt xích có thể thay đổi và những mắt còn lại chỉ việc thích ứng theo.

When you're designing bridges, you want them to hold their shape; you need them to be rigid. But when you're designing software that you'll want to change, you want exactly the opposite: you want it to be flexible. And to be flexible, individual components should be coupled to as few other components as possible.

Khi thiết kế cầu, bạn muốn chúng giữ nguyên hình dạng; bạn cần chúng cứng vững. Nhưng khi thiết kế phần mềm mà bạn sẽ muốn thay đổi, bạn lại muốn điều ngược lại hoàn toàn: bạn muốn nó linh hoạt. Và để linh hoạt, mỗi thành phần nên được ghép nối với càng ít thành phần khác càng tốt.

And, to make matters worse, coupling is transitive: if A is coupled to B and C, and B is coupled to M and N, and C to X and Y, then A is actually coupled to B, C, M, N, X, and Y.

Và, tệ hơn nữa, sự ghép nối có tính bắc cầu: nếu A ghép nối với B và C, B ghép nối với M và N, còn C ghép nối với X và Y, thì thực ra A đang ghép nối với cả B, C, M, N, X và Y.

This means there's a simple principle you should follow:

Điều này dẫn đến một nguyên tắc đơn giản mà bạn nên tuân theo:

Tip 44: Decoupled Code Is Easier to Change

Tip 44: Mã Được Gỡ Ghép Nối Thì Dễ Thay Đổi Hơn

Given that we don't normally code using steel beams and rivets, just what does it mean to decouple code? In this section we'll talk about:

Vì ta thường không lập trình bằng dầm thép và đinh tán, vậy gỡ ghép nối (decoupling) mã rốt cuộc nghĩa là gì? Trong mục này, chúng ta sẽ bàn về:

Train wrecks—chains of method calls

Tai nạn tàu hỏa (train wrecks)—những chuỗi lời gọi phương thức.

Globalization—the dangers of static things

Sự toàn cầu hóa (globalization)—những hiểm họa của thứ tĩnh (static).

Inheritance—why subclassing is dangerous

Kế thừa (inheritance)—vì sao tạo lớp con lại nguy hiểm.

To some extent this list is artificial: coupling can occur just about any time two pieces of code share something, so as you read what follows keep an eye out for the underlying patterns so you can apply them to your code. And keep a lookout for some of the symptoms of coupling:

Ở một mức độ nào đó, danh sách này có phần khiên cưỡng: sự ghép nối có thể xảy ra gần như bất cứ khi nào hai mẫu mã cùng chia sẻ một thứ gì đó, nên khi đọc phần sau, hãy để mắt tới các khuôn mẫu nền tảng để có thể áp dụng vào mã của bạn. Và hãy canh chừng vài triệu chứng của sự ghép nối:

Wacky dependencies between unrelated modules or libraries.

Những phụ thuộc kỳ quặc giữa các mô-đun hay thư viện chẳng liên quan gì đến nhau.

“Simple” changes to one module that propagate through unrelated modules in the system or break stuff elsewhere in the system.

Những thay đổi “đơn giản” ở một mô-đun lại lan truyền qua các mô-đun không liên quan trong hệ thống hoặc làm hỏng thứ khác ở nơi khác trong hệ thống.

Developers who are afraid to change code because they aren’t sure what might be affected.

Các lập trình viên sợ thay đổi mã vì họ không chắc điều gì có thể bị ảnh hưởng.

Meetings where everyone has to attend because no one is sure who will be affected by a change.

Những cuộc họp mà ai cũng phải dự vì chẳng ai chắc một thay đổi sẽ ảnh hưởng đến ai.

Train Wrecks — Tai nạn tàu hỏa

We’ve all seen (and probably written) code like this:

Tất cả chúng ta đều từng thấy (và có lẽ từng viết) loại mã thế này:

```
public void applyDiscount(customer, order_id, discount) { totals = customer .orders .find(order_id)
.getTotals(); totals.grandTotal = totals.grandTotal - discount; totals.discount = discount; }

public void applyDiscount(customer, order_id, discount) { totals = customer .orders
.find(order_id) .getTotals(); totals.grandTotal = totals.grandTotal - discount; totals.discount
= discount; }
```

We’re getting a reference to some orders from a customer object, using that to find a particular order, and then getting the set of totals for the order. Using those totals, we subtract the discount from the order grand total and also update them with that discount.

Chúng ta lấy một tham chiếu tới một số đơn hàng (orders) từ một đối tượng customer, dùng nó để tìm một đơn hàng cụ thể, rồi lấy bộ tổng (totals) của đơn hàng đó. Dùng các tổng ấy, ta trừ khoản giảm giá khỏi tổng cộng (grand total) của đơn hàng và cũng cập nhật chúng với khoản giảm giá đó.

This chunk of code is traversing five levels of abstraction, from customer to total amounts. Ultimately our top-level code has to

Đoạn mã này đang đi xuyên qua năm tầng trừu tượng, từ customer đến các khoản tổng tiền. Rốt cuộc, đoạn mã cấp cao nhất của ta phải

know that a customer object exposes orders, that the orders have a find method that takes an order id and returns an order, and that the order object has a totals object which has getters and setters for grand totals and discounts. That’s a lot of implicit knowledge. But worse, that’s a lot of things that cannot change in the future if this code is to continue to work. All the cars in a train are coupled together, as are all the methods and attributes in a train wreck.

biết rằng một đối tượng customer phơi ra orders, rằng orders có một phương thức find nhận vào một order id và trả về một order, và rằng đối tượng order lại có một đối tượng

totals với các getter và setter cho tổng cộng (grand total) và khoản giảm giá. Đó là cả một đồng tri thức ngầm. Nhưng tệ hơn, đó là cả một đồng thứ không được phép thay đổi trong tương lai nếu muốn đoạn mã này còn chạy được. Mọi toa trong một đoàn tàu đều móc nối vào nhau, và mọi phương thức cùng thuộc tính trong một “tai nạn tàu hỏa” cũng vậy.

Let’s imagine that the business decides that no order can have a discount of more than 40%. Where would we put the code that enforces that rule?

Hãy hình dung doanh nghiệp quyết định rằng không đơn hàng nào được giảm giá quá 40%. Ta sẽ đặt đoạn mã thực thi quy tắc đó ở đâu?

You might say it belongs in the `applyDiscount` function we just wrote. That’s certainly part of the answer. But with the code the way it is now, you can’t know that this is the whole answer. Any piece of code, anywhere, could set fields in the totals object, and if the maintainer of that code didn’t get the memo, it wouldn’t be checking against the new **policy**.

Bạn có thể nói rằng nó thuộc về hàm `applyDiscount` mà ta vừa viết. Đó chắc chắn là một phần của câu trả lời. Nhưng với đoạn mã như hiện tại, bạn không thể biết đó đã là toàn bộ câu trả lời hay chưa. Bất kỳ mẫu mã nào, ở bất kỳ đâu, cũng có thể gán các trường trong đối tượng totals, và nếu người bảo trì đoạn mã đó không nhận được thông báo, nó sẽ chẳng kiểm tra gì theo quy tắc mới cả.

One way to look at this is to think about responsibilities. Surely the totals object should be responsible for managing the totals. And yet it isn’t: it’s really just a container for a bunch of fields that anyone can query and update.

Một cách nhìn nhận chuyện này là nghĩ về trách nhiệm. Chắc chắn đối tượng totals phải là nơi chịu trách nhiệm quản lý các khoản tổng. Vậy mà nó lại không: thực chất nó chỉ là một cái hộp chứa một mớ trường mà ai cũng có thể truy vấn và cập nhật.

The fix for that is to apply something we call:

Cách sửa cho điều đó là áp dụng thứ mà chúng tôi gọi là:

Tip 45: Tell, Don’t Ask

Tip 45: Ra Lệnh, Đừng Hỏi

This principle says that you shouldn’t make decisions based on the **internal state** of an object and then update that object. Doing so totally destroys the benefits of encapsulation and, in

Nguyên tắc này nói rằng bạn không nên ra quyết định dựa trên trạng thái nội bộ của một đối tượng rồi cập nhật chính đối tượng đó. Làm vậy phá hủy hoàn toàn lợi ích của đóng gói (encapsulation) và, qua đó,

doing so, spreads the knowledge of the implementation throughout the code. So the first fix for our train wreck is to delegate the discounting to the total object:

làm vậy còn rải tri thức về phần cài đặt ra khắp toàn bộ mã. Vậy nên cách sửa đầu tiên cho “tai nạn tàu hỏa” của ta là ủy thác việc giảm giá cho đối tượng total:

```
public void applyDiscount(customer, order_id, discount) { customer.orders.find(order_id).getTotals().applyDiscount(discount); }
```

```
public void applyDiscount(customer, order_id, discount) { customer.orders.find(order_id).getTotals().applyDiscount(discount); }
```

We have the same kind of tell-don’t-ask (TDA) issue with the customer object and its orders: we shouldn’t fetch its list of orders and search them. We should instead get the order we want directly

from the customer:

Ta cũng gặp đúng kiểu vấn đề “ra lệnh đừng hỏi” (tell-don’t-ask, TDA) với đối tượng customer và các orders của nó: ta không nên lấy danh sách orders rồi tìm kiếm trong đó. Thay vào đó, ta nên lấy thẳng đơn hàng mình cần từ customer:

```
public void applyDiscount(customer, order_id, discount) { customer .findOrder(order_id) .getTotals()
    .applyDiscount(discount); }
```

```
public void applyDiscount(customer, order_id, discount) { customer .findOrder(order_id)
    .getTotals() .applyDiscount(discount); }
```

The same thing applies to our order object and its totals. Why should the outside world have to know that the implementation of an order uses a separate object to store its totals?

Điều tương tự cũng áp dụng cho đối tượng order và các totals của nó. Tại sao thế giới bên ngoài lại phải biết rằng phần cài đặt của một order dùng một đối tượng riêng để lưu các totals của nó?

```
public void applyDiscount(customer, order_id, discount) { customer .findOrder(order_id) .applyDiscount(discount);
    }
```

```
public void applyDiscount(customer, order_id, discount) { customer .findOrder(order_id)
    .applyDiscount(discount); }
```

And this is where we’d probably stop.

Và đây có lẽ là chỗ ta sẽ dừng lại.

At this point you might be thinking that TDA would make us add an `applyDiscountToOrder(order_id)` method to customers. And, if

Đến đây có thể bạn đang nghĩ rằng TDA sẽ bắt ta thêm một phương thức `applyDiscountToOrder(order_id)` vào customer. Và, nếu

followed slavishly, it would.

làm theo một cách máy móc, thì đúng là vậy.

But TDA is not a law of nature; it’s just a pattern to help us recognize problems. In this case, we’re comfortable exposing the fact that a customer has orders, and that we can find one of those orders by asking the customer object for it. This is a **pragmatic** decision.

Nhưng TDA không phải một định luật tự nhiên; nó chỉ là một khuôn mẫu giúp ta nhận diện vấn đề. Trong trường hợp này, ta thấy thoải mái khi phơi ra sự thật rằng một customer có orders, và rằng ta có thể tìm một trong các orders đó bằng cách hỏi chính đối tượng customer. Đây là một quyết định thực dụng.

In every application there are certain top-level concepts that are universal. In this application, those concepts include customers and orders . It makes no sense to hide orders totally inside customer objects: they have an existence of their own. So we have no problem creating APIs that expose order objects.

Trong mọi ứng dụng đều có những khái niệm cấp cao nhất mang tính phổ quát. Trong ứng dụng này, những khái niệm đó gồm customers và orders. Việc giấu orders hoàn toàn bên trong các đối tượng customer là vô nghĩa: chúng có sự tồn tại riêng. Vậy nên ta chẳng có vấn đề gì khi tạo các API phơi ra các đối tượng order.

The **Law of Demeter** People often talk about something called the Law of Demeter , or LoD, in relation to coupling. The LoD is a set of guidelines [37] written in the late '80s by Ian Holland. He created them to help developers on the Demeter Project keep their functions cleaner and decoupled.

Luật Demeter Người ta thường nhắc tới một thứ gọi là Luật Demeter (Law of Demeter), hay LoD, khi bàn về sự ghép nối. LoD là một tập hợp các hướng dẫn [37] do Ian Holland viết vào cuối thập niên 1980. Ông tạo ra chúng để giúp các lập trình viên trong Dự án Demeter giữ cho hàm của họ sạch sẽ hơn và ít ghép nối hơn.

The LoD says that a method defined in a class C should only call:

LoD nói rằng một phương thức được định nghĩa trong một lớp C chỉ nên gọi:

Other instance methods in C

Các phương thức instance khác trong C.

Its parameters

Các tham số của chính nó.

Methods in objects that it creates, both on the stack and in the heap

Các phương thức trong những đối tượng mà nó tạo ra, cả trên stack lẫn trên heap.

Global variables

Các biến toàn cục (global variables).

In the first edition of this book we spent some time describing the LoD. In the intervening 20 years the bloom has faded on that particular rose. We now don't like the "global variable"

Trong ấn bản đầu của cuốn sách này, chúng tôi đã dành chút thời gian mô tả LoD. Trong 20 năm xen giữa, đóa hồng ấy đã phai sắc. Giờ đây chúng tôi không còn thích điều khoản "biến toàn cục"

clause (for reasons we'll go into in the next section). We also discovered that it's difficult to use this in practice: it's a little like having to parse a legal document whenever you call a method.

(vì những lý do sẽ bàn ở mục tiếp theo). Chúng tôi cũng nhận ra rằng dùng nó trong thực tế khá khó: nó hơi giống việc phải đọc và phân tích một văn bản pháp lý mỗi lần bạn gọi một phương thức.

However, the principle is still sound. We just recommend a somewhat simpler way of expressing almost the same thing:

Tuy nhiên, nguyên tắc thì vẫn vững. Chúng tôi chỉ khuyến nghị một cách diễn đạt giản dị hơn đôi chút cho gần như cùng một ý:

Tip 46: Don't Chain Method Calls

Tip 46: Đừng Nối Chuỗi Lời Gọi Phương Thức

Try not to have more than one "." when you access something. And access something also covers cases where you use intermediate variables, as in the following code:

Cố đừng có quá một dấu "." khi bạn truy cập một thứ gì đó. Và "truy cập một thứ gì đó" cũng bao gồm cả những trường hợp bạn dùng biến trung gian, như trong đoạn mã sau:

**This is pretty poor style amount =
customer.orders.last().totals().amount;
— Đây là phong cách khá tệ amount =
customer.orders.last().totals().amount;**

**and so is this... orders =
customer.orders; last = orders.last();
totals = last.totals(); amount =
totals.amount; — và đây cũng vậy...
orders = customer.orders; last =
orders.last(); totals = last.totals();
amount = totals.amount;**

There's a big **exception** to the one-dot rule: the rule doesn't apply if the things you're chaining are really, really unlikely to change. In practice, anything in your application should be considered likely to change. Anything in a third-party library should be considered volatile, particularly if the maintainers of that library are known to change APIs between releases. Libraries that come with the language, however, are probably pretty stable, and so we'd be happy with code such as:

Có một ngoại lệ lớn cho quy tắc một-dấu-chấm: quy tắc không áp dụng nếu những thứ bạn nối chuỗi thực sự, thực sự khó mà thay đổi. Trên thực tế, mọi thứ trong ứng dụng của bạn nên được coi là có khả năng thay đổi. Mọi thứ trong một thư viện bên thứ ba nên được coi là dễ biến động, đặc biệt nếu người bảo trì thư viện đó nổi tiếng hay thay đổi API giữa các bản phát hành. Tuy nhiên, các thư viện đi kèm ngôn ngữ thì có lẽ khá ổn định, nên chúng tôi sẽ thoải mái với đoạn mã như:

```
people.sort_by {|person| person.age }.first(10).map {| person | person.name }  
people.sort_by {|person| person.age }.first(10).map {| person | person.name }
```

That Ruby code worked when we wrote the first edition, 20 years ago, and will likely still work when we enter the home for old programmers (any day now...).

Đoạn mã Ruby đó chạy được khi chúng tôi viết ấn bản đầu, 20 năm trước, và có lẽ vẫn sẽ chạy được khi chúng tôi vào viện dưỡng lão dành cho lập trình viên già (chắc cũng nay mai thôi...).

Chains and Pipelines In Topic 30, Transforming Programming we talk about composing functions into pipelines. These pipelines transform data, passing it from one function to the next. This is not the same as a train wreck of method calls, as we are not relying on hidden implementation details.

Chuỗi và Đường ống Trong Topic 30, Transforming Programming, chúng ta bàn về việc kết hợp các hàm thành đường ống (pipelines). Những đường ống này biến đổi dữ liệu, chuyển nó từ hàm này sang hàm kế tiếp. Đây không giống một “tai nạn tàu hỏa” gồm các lời gọi phương thức, vì ta không dựa vào những chi tiết cài đặt bị che giấu.

That’s not to say that pipelines don’t introduce some coupling: they do. The format of the data returned by one function in a **pipeline** must be compatible with the format accepted by the next.

Nói vậy không có nghĩa là đường ống chẳng tạo ra chút ghép nối nào: chúng có đấy. Định dạng dữ liệu mà một hàm trong đường ống trả về phải tương thích với định dạng mà hàm kế tiếp chấp nhận.

Our experience is that this form of coupling is far less a barrier to changing the code than the form introduced by train wrecks.

Kinh nghiệm của chúng tôi là dạng ghép nối này ít cản trở việc thay đổi mã hơn nhiều so với dạng ghép nối do “tai nạn tàu hỏa” gây ra.

The Evils Of Globalization — Những điều xấu xa của Toàn cầu hóa

Globally accessible data is an insidious source of coupling between application components. Each piece of global data acts as if every method in your application suddenly gained an additional parameter: after all, that global data is available inside every method.

Dữ liệu truy cập được toàn cục là một nguồn ghép nối ngấm ngầm giữa các thành phần ứng dụng. Mỗi mẫu dữ liệu toàn cục hành xử như thể mọi phương thức trong ứng dụng của bạn bỗng dưng có thêm một tham số nữa: suy cho cùng, dữ liệu toàn cục ấy luôn sẵn có bên trong mọi phương thức.

Globals couple code for many reasons. The most obvious is that a change to the implementation of the global potentially affects all the code in the system. In practice, of course, the impact is fairly limited; the problem really comes down to knowing that you’ve found every place you need to change.

Dữ liệu toàn cục ghép nối mã vì nhiều lý do. Hiển nhiên nhất là một thay đổi ở phần cài đặt của dữ liệu toàn cục có thể ảnh hưởng đến toàn bộ mã trong hệ thống. Trên thực tế, dĩ nhiên, tác động khá hạn chế; vấn đề thật sự rắc rối nằm ở chỗ làm sao biết chắc bạn đã tìm ra mọi nơi cần thay đổi.

Global data also creates coupling when it comes to teasing your

Dữ liệu toàn cục cũng tạo ra ghép nối khi ta muốn tách rời code apart.

mã của mình ra.

Much has been made of the benefits of code reuse. Our experience has been that reuse should probably not be a primary concern when creating code, but the thinking that goes into making code reusable should be part of your coding routine. When you make code reusable, you give it clean interfaces, decoupling it from the rest of your code. This allows you to extract a method or module without dragging everything else along with it. And if your code uses global data, then it becomes difficult to split it out from the rest.

Người ta đã ca ngợi rất nhiều về lợi ích của việc tái sử dụng mã. Kinh nghiệm của chúng tôi là tái sử dụng có lẽ không nên là mối bận tâm hàng đầu khi tạo mã, nhưng lối tư duy hướng tới làm cho mã có thể tái sử dụng nên là một phần trong thói quen viết mã của bạn. Khi bạn làm cho mã tái sử dụng được, bạn trao cho nó những giao diện sạch sẽ, gỡ ghép

nổi nó khỏi phần còn lại của mã. Điều này cho phép bạn trích xuất một phương thức hay mô-đun mà không phải lồi theo mọi thứ khác. Và nếu mã của bạn dùng dữ liệu toàn cục, thì việc tách nó ra khỏi phần còn lại sẽ trở nên khó khăn.

You'll see this problem when you're writing unit tests for code that uses global data. You'll find yourself writing a bunch of setup code to create a global environment just to allow your test to run.

Bạn sẽ thấy vấn đề này khi viết kiểm thử đơn vị (unit test) cho mã dùng dữ liệu toàn cục. Bạn sẽ thấy mình phải viết cả một mớ mã thiết lập để dựng nên một môi trường toàn cục chỉ để bài kiểm thử chạy được.

Tip 47: Avoid Global Data

Tip 47: Tránh Dữ Liệu Toàn Cục

Global Data Includes Singletons In the previous section we were careful to talk about global data and not global variables. That's because people often tell us "Look! No global variables. I wrapped it all as instance data in a singleton object or global module."

Dữ liệu toàn cục bao gồm cả Singleton Ở mục trước, chúng tôi đã cẩn thận nói "dữ liệu toàn cục" chứ không phải "biến toàn cục". Đó là vì người ta thường bảo chúng tôi "Nhìn này! Không có biến toàn cục nào hết. Tôi đã gói tất cả thành dữ liệu instance trong một đối tượng singleton hay một mô-đun toàn cục rồi."

Try again, Skippy. If all you have is a singleton with a bunch of exported instance variables, then it's still just global data. It just has a longer name.

Thử lại đi, anh bạn. Nếu tất cả những gì bạn có là một singleton với một mớ biến instance được phơi ra, thì nó vẫn chỉ là dữ liệu toàn cục thôi. Nó chỉ có cái tên dài hơn mà thôi.

So then folks take this singleton and hide all the data behind methods. Instead of coding `Config.log_level` they now say `Config.log_level()` or `Config.getLogLevel()`. This is better, because it

Thế là người ta lấy cái singleton ấy và giấu toàn bộ dữ liệu phía sau các phương thức. Thay vì viết `Config.log_level`, giờ họ viết `Config.log_level()` hoặc `Config.getLogLevel()`. Như vậy thì tốt hơn, bởi nó

means that your global data has a bit of intelligence behind it. If you decide to change the representation of log levels, you can maintain compatibility by mapping between the new and old in the Config API. But you still have only the one set of configuration data.

có nghĩa là dữ liệu toàn cục của bạn giờ có chút thông minh đứng sau. Nếu bạn quyết định đổi cách biểu diễn các mức log, bạn có thể duy trì tính tương thích bằng cách ánh xạ giữa kiểu mới và kiểu cũ trong API của Config. Nhưng bạn vẫn chỉ có duy nhất một bộ dữ liệu cấu hình.

Global Data Includes External Resources Any mutable external resource is global data. If your application uses a database, datastore, file system, service API, and so on, it risks falling into the globalization trap. Again, the solution is to make sure you always wrap these resources behind code that you control.

Dữ liệu toàn cục bao gồm cả Tài nguyên bên ngoài Bất kỳ tài nguyên bên ngoài nào có thể thay đổi (mutable) đều là dữ liệu toàn cục. Nếu ứng dụng của bạn dùng cơ sở dữ liệu (database), kho dữ liệu (datastore), hệ thống tệp, API dịch vụ, v.v., thì nó có nguy cơ rơi vào cái bẫy toàn cầu hóa. Một lần nữa, giải pháp là đảm bảo bạn luôn gói các tài nguyên này phía sau đoạn mã do chính bạn kiểm soát.

If It's Important Enough to Be Global, Wrap It in — Nếu Nó Đủ Quan Trọng Để Là Toàn Cục, Hãy Bọc Nó Trong

Tip 48: an API

Tip 48: một API

Inheritance Adds Coupling — Kế thừa làm tăng sự ghép nối

The misuse of subclassing, where a class inherits state and behavior from another class, is so important that we discuss it in its own section, Topic 31, Inheritance Tax .

Việc lạm dụng tạo lớp con (subclassing), khi một lớp kế thừa trạng thái và hành vi từ một lớp khác, quan trọng đến mức chúng tôi bàn riêng trong một mục của nó, Topic 31, Inheritance Tax.

Again, It'S All About Change — Một lần nữa, tất cả vẫn là về Sự thay đổi

Coupled code is hard to change: alterations in one place can have secondary effects elsewhere in the code, and often in hard-to-find places that only come to light a month later in production.

Mã bị ghép nối thì khó thay đổi: những sửa đổi ở một nơi có thể gây ra hiệu ứng thứ cấp ở nơi khác trong mã, và thường ở những chỗ khó tìm chỉ lộ ra một tháng sau khi đã chạy trên môi trường thật (production).

Keeping your code shy: having it only deal with things it directly knows about, will help keep your applications decoupled, and that will make them more amenable to change.

Hãy giữ cho mã của bạn nhút nhát (shy code): chỉ cho nó dính líu tới những thứ nó trực tiếp biết, điều đó sẽ giúp giữ cho ứng dụng được gỡ ghép nối, và như thế sẽ khiến chúng dễ chiều theo sự thay đổi hơn.

Related Sections Include — Các mục liên quan

Topic 8, The Essence of Good Design

[Topic 8, The Essence of Good Design](#)

Topic 9, **DRY**—The Evils of **Duplication**

[Topic 9, DRY—The Evils of Duplication](#)

Topic 10, **Orthogonality**

[Topic 10, Orthogonality](#)

Topic 11, Reversibility

[Topic 11, Reversibility](#)

Topic 29, Juggling the Real World

[Topic 29, Juggling the Real World](#)

Topic 30, Transforming Programming

[Topic 30, Transforming Programming](#)

Topic 31, Inheritance Tax

Topic 31, Inheritance Tax

Topic 32, Configuration

Topic 32, Configuration

Topic 33, Breaking **Temporal Coupling**

Topic 33, Breaking Temporal Coupling

Topic 34, **Shared State Is Incorrect State**

Topic 34, Shared State Is Incorrect State

Topic 35, Actors and Processes

Topic 35, Actors and Processes

Topic 36, Blackboards

Topic 36, Blackboards

We discuss Tell, Don't Ask in our 2003 Software Construction article The Art of Enbugging . [38]

Chúng tôi bàn về Ra Lệnh, Đừng Hỏi (Tell, Don't Ask) trong bài The Art of Enbugging thuộc chuyên mục Software Construction năm 2003. [38]

Things don't just happen; they are made to happen. John F. Kennedy

Mọi chuyện không tự dưng xảy ra; chúng được làm cho xảy ra. John F. Kennedy

Topic 29. Juggling the Real World — Topic 29. Tung hứng với thế giới thực

In the old days, when your authors still had their boyish good looks, computers were not particularly flexible. We'd typically organize the way we interacted with them based on their limitations.

Ngày xưa, hồi hai tác giả của bạn vẫn còn nét đẹp trai trẻ trung, máy tính chẳng linh hoạt cho lắm. Chúng tôi thường phải tổ chức cách tương tác với chúng dựa trên những giới hạn của chính chúng.

Today, we expect more: computers have to integrate into our world, not the other way around. And our world is messy: things are constantly happening, stuff gets moved around, we change our minds, And the applications we write somehow have to work out what to do.

Ngày nay, ta kỳ vọng nhiều hơn: máy tính phải hòa nhập vào thế giới của ta, chứ không phải ngược lại. Mà thế giới của ta thì lộn xộn: mọi thứ liên tục xảy ra, đồ đạc bị xô dịch lung tung, ta thay đổi ý định, ... Và bằng cách nào đó, các ứng dụng ta viết phải tự xoay sở để biết phải làm gì.

This section is all about writing these responsive applications.

Mục này nói về chuyện viết những ứng dụng biết phản hồi như thế.

We'll start off with the concept of an **event** .

Ta sẽ bắt đầu với khái niệm sự kiện (event).

Events — Sự kiện

An event represents the availability of information. It might come from the outside world: a user clicking a button, or a stock quote update. It might be **internal**: the result of a calculation is ready, a search finishes. It can even be something as trivial as fetching the next element in a list.

Một sự kiện đại diện cho việc một thông tin nào đó trở nên sẵn sàng. Nó có thể đến từ thế giới bên ngoài: người dùng bấm một nút, hay một bản cập nhật giá cổ phiếu. Nó cũng có thể đến từ bên trong: kết quả của một phép tính đã sẵn sàng, một tác vụ tìm kiếm vừa kết thúc. Thậm chí nó có thể tầm thường đến mức chỉ là việc lấy ra phần tử kế tiếp trong một danh sách.

Whatever the source, if we write applications that respond to events, and adjust what they do based on those events, those

Dù đến từ nguồn nào đi nữa, nếu ta viết những ứng dụng biết phản hồi sự kiện, và biết điều chỉnh hành vi dựa trên các sự kiện ấy, thì những

applications will work better in the real world. Their users will find them to be more interactive, and the applications themselves will make better use of resources.

ứng dụng đó sẽ hoạt động tốt hơn trong thế giới thực. Người dùng sẽ thấy chúng tương tác hơn, và bản thân ứng dụng cũng tận dụng tài nguyên hiệu quả hơn.

But how can we write these kinds of applications? Without some kind of strategy, we'll quickly find ourselves confused, and our applications will be a mess of tightly coupled code.

Nhưng làm sao để viết được những kiểu ứng dụng này? Nếu không có một chiến lược nào đó, ta sẽ nhanh chóng rối tung lên, và ứng dụng của ta sẽ trở thành một mớ code ghép nối chặt chẽ với nhau.

Let's look at four strategies that help.

Hãy cùng xem bốn chiến lược giúp ích cho việc này.

1. Finite **State** Machines
2. The **Observer Pattern**
3. **Publish/Subscribe**
4. **Reactive Programming** and Streams

1. Máy trạng thái hữu hạn (Finite State Machine)
2. Mẫu Observer (The Observer Pattern)
3. Publish/Subscribe
4. Lập trình phản ứng và luồng dữ liệu (Reactive Programming and Streams)

Finite State Machines — Máy trạng thái hữu hạn

Dave finds that he writes code using a **Finite State Machine** (FSM) just about every week. Quite often, the FSM implementation will be just a couple of lines of code, but those few lines help untangle a whole lot of potential mess.

Dave nhận ra rằng gần như tuần nào anh ấy cũng viết code dùng đến Máy trạng thái hữu hạn (Finite State Machine — FSM). Khá thường xuyên, phần cài đặt FSM chỉ vỏn vẹn vài dòng code, nhưng mấy dòng ấy giúp gỡ rối cả một đống bừa bộn tiềm tàng.

Using an FSM is trivially easy, and yet many developers shy away from them. There seems to be a belief that they are difficult, or that they only apply if you're working with hardware, or that you need to use some hard-to-understand library. None of these are true.

Dùng FSM dễ đến mức tầm thường, vậy mà nhiều lập trình viên lại né tránh nó. Hình như có một niềm tin rằng FSM khó hiểu, hoặc chỉ áp dụng được khi làm việc với phần cứng,

hoặc rằng bạn cần đến một thư viện nào đó khó nhằn. Chẳng điều nào trong số đó là đúng cả.

The Anatomy of a **Pragmatic** FSM A state machine is basically just a specification of how to handle events. It consists of a set of states, one of which is the **current state**. For each state, we list the events that are significant to that state. For each of those events, we define the new current state of the system.

Giải phẫu một FSM thực dụng Một máy trạng thái về cơ bản chỉ là bản đặc tả về cách xử lý các sự kiện. Nó gồm một tập các trạng thái, trong đó có một trạng thái là trạng thái hiện tại (current state). Với mỗi trạng thái, ta liệt kê những sự kiện có ý nghĩa đối với trạng thái đó. Với mỗi sự kiện trong số đó, ta định nghĩa trạng thái hiện tại mới của hệ thống.

For example, we may be receiving multipart messages from a websocket. The first message is a header. This is followed by any number of data messages, followed by a trailing message. This could be represented as an FSM like this:

Ví dụ, giả sử ta đang nhận các thông điệp gồm nhiều phần từ một websocket. Thông điệp đầu tiên là phần tiêu đề (header). Tiếp sau đó là một số bất kỳ các thông điệp dữ liệu, rồi đến một thông điệp kết thúc. Có thể biểu diễn việc này bằng một FSM như sau:

images/events_simple_fsm.png

images/events_simple_fsm.png

We start in the “Initial state.” If we receive a header message, we **transition** to the “Reading message” state. If we receive anything else while we’re in the initial state (the line labeled with an asterisk) we transition to the “Error” state and we’re done.

Ta khởi đầu ở “Trạng thái ban đầu.” Nếu nhận được một thông điệp header, ta chuyển sang trạng thái “Đang đọc thông điệp.” Nếu nhận được bất cứ thứ gì khác khi đang ở trạng thái ban đầu (chính là đường được gắn nhãn dấu hoa thị), ta chuyển sang trạng thái “Lỗi” và thế là xong.

While we’re in the “Reading message” state, we can accept either data messages, in which case we continue reading in the same state, or we can accept a trailer message, which transitions us to the “Done” state. Anything else causes a transition to the error state.

Khi đang ở trạng thái “Đang đọc thông điệp,” ta có thể chấp nhận thông điệp dữ liệu — trong trường hợp đó ta tiếp tục đọc và giữ nguyên trạng thái — hoặc có thể chấp nhận một thông điệp kết thúc, đưa ta sang trạng thái “Xong.” Bất cứ thứ gì khác đều gây ra chuyển dịch sang trạng thái lỗi.

The neat thing about FSMs is that we can express them purely as data. Here’s a table representing our message **parser**:

Điều hay ho ở FSM là ta có thể biểu diễn chúng hoàn toàn dưới dạng dữ liệu. Đây là một bảng biểu diễn bộ phân tích thông điệp của ta:

images/event_simple_fsm_table.png

images/event_simple_fsm_table.png

The rows in the table represent the states. To find out what to do when an event occurs, look up the row for the current state, scan along for the column representing the event, the contents of that cell are the new state.

Các hàng trong bảng biểu diễn các trạng thái. Để biết phải làm gì khi một sự kiện xảy ra, hãy tra hàng ứng với trạng thái hiện tại, dò theo cột biểu diễn sự kiện đó, và nội dung của ô đó chính là trạng thái mới.

The code that handles it is equally simple:

Đoạn code xử lý việc này cũng đơn giản y như vậy:

```
event/simple_fsm.rb
event/simple_fsm.rb
TRANSITIONS = {
  TRANSITIONS = {
:
:
initial: { header: :reading }, -
  initial: { header: :reading }, -
reading: { data: :reading , trailer: :done }, -
  reading: { data: :reading , trailer: :done }, -
} -
} -
:
:
state = :initial -
  state = :initial -
•
•
while state != :done && state != :error -
  while state != :done && state != :error -
msg = get_next_message() -
  msg = get_next_message() -
state = TRANSITIONS[state][msg.msg_type] || :error
  state = TRANSITIONS[state][msg.msg_type] || :error
:
:
end -
  end -
```

The code that implements the transitions between states is on line 10. It indexes the transition table using the current state, and then indexes the transitions for that state using the message type. If there is no matching new state, it sets the state to :error .

Đoạn code cài đặt các chuyển dịch giữa các trạng thái nằm ở dòng 10. Nó dùng trạng thái hiện tại để truy bảng chuyển dịch, rồi dùng kiểu thông điệp để truy các chuyển dịch của trạng thái đó. Nếu không có trạng thái mới nào khớp, nó đặt trạng thái thành :error .

Adding Actions A pure FSM, such as the one we were just looking at, is an event **stream** parser. Its only output is the final state. We can beef it up by adding actions that are triggered on certain transitions.

Thêm hành động Một FSM thuần túy, như cái ta vừa xem, là một bộ phân tích luồng sự kiện. Đầu ra duy nhất của nó là trạng thái cuối cùng. Ta có thể tăng cường sức mạnh cho nó bằng cách thêm các hành động được kích hoạt trên một số chuyển dịch nhất định.

For example, we might need to extract all of the strings in a source file. A string is text between quotes, but a backslash in a string escapes the next character, so “Ignore ”quotes”” is a single string. Here’s an FSM that does this:

Ví dụ, giả sử ta cần trích xuất tất cả các chuỗi trong một file mã nguồn. Một chuỗi là phần văn bản nằm giữa hai dấu nháy, nhưng một dấu gạch chéo ngược trong chuỗi sẽ thoát (escape) ký tự ngay sau nó, nên “Ignore ”quotes”” là một chuỗi duy nhất. Đây là một FSM làm việc đó:

images/event_string_fsm.png

images/event_string_fsm.png

This time, each transition has two labels. The top one is the

Lần này, mỗi chuyển dịch có hai nhãn. Nhãn trên là

event that triggers it, and the bottom one is the action to take as we move between states.

sự kiện kích hoạt chuyển dịch, còn nhãn dưới là hành động cần thực hiện khi ta dịch chuyển giữa các trạng thái.

We’ll express this in a table, as we did last time. However, in this case each entry in the table is a two-element list containing the next state and the name of an action:

Ta sẽ biểu diễn việc này bằng một bảng, như lần trước. Tuy nhiên, lần này mỗi ô trong bảng là một danh sách hai phần tử chứa trạng thái kế tiếp và tên của một hành động:

event/strings_fsm.rb

event/strings_fsm.rb

TRANSITIONS = {

TRANSITIONS = {

current new state action to take #—

**— current new state action to take
#**

```
look_for_string: { "" => [ :in_string , :start_new_string ], :default => [ :look_for_string , :ignore ], },
  look_for_string: { "" => [ :in_string , :start_new_string ], :default => [ :look_for_string ,
    :ignore ], },
in_string: { "" => [ :look_for_string , :finish_current_string ], '\' => [ :copy_next_char , :add_current_to_string
  ], :default => [ :in_string , :add_current_to_string ], },
  in_string: { "" => [ :look_for_string , :finish_current_string ], '\' => [ :copy_next_char ,
    :add_current_to_string ], :default => [ :in_string , :add_current_to_string ], },
copy_next_char: { :default => [ :in_string , :add_current_to_string ], }, }
  copy_next_char: { :default => [ :in_string , :add_current_to_string ], }, }
```

We've also added the ability to specify a default transition, taken if the event doesn't match any of the other transitions for this state.

Ta cũng đã thêm khả năng chỉ định một chuyển dịch mặc định (default), được dùng khi sự kiện không khớp với bất kỳ chuyển dịch nào khác của trạng thái này.

Now let's look at the code:

Giờ thì hãy xem đoạn code:

```
event/strings_fsm.rb
  event/strings_fsm.rb
state = :look_for_string
  state = :look_for_string
result = []
  result = []
```

```
while ch = STDIN.getc state, action = TRANSITIONS[state][ch] || TRANSITIONS[state][:default] case
action when :ignore when :start_new_string result = [] when :add_current_to_string result « ch when
:finish_current_string puts result.join end end
```

```
while ch = STDIN.getc state, action = TRANSITIONS[state][ch] || TRANSITIONS[state][:default] case action when :ignore when :start_new_string result = [] when :add_current_to_string
result « ch when :finish_current_string puts result.join end end
```

This is similar to the previous example, in that we loop through the events (the characters in the input), triggering transitions. But it does more than the previous code. The result of each transition is both a new state and the name of an action. We use the action name to select the code to run before we go back around the loop.

Đoạn này tương tự ví dụ trước, ở chỗ ta lặp qua các sự kiện (các ký tự trong đầu vào) để kích hoạt các chuyển dịch. Nhưng nó làm nhiều việc hơn đoạn code trước. Kết quả của mỗi chuyển dịch là cả một trạng thái mới lẫn tên của một hành động. Ta dùng tên hành động để chọn ra đoạn code cần chạy trước khi quay lại vòng lặp.

This code is very basic, but it gets the job done. There are many other variants: the transition table could use anonymous functions or function pointers for the actions, you could wrap the code that implements the state machine in a separate class, with its own state, and so on.

Đoạn code này rất cơ bản, nhưng nó làm xong việc. Còn vô số biến thể khác: bảng chuyển dịch có thể dùng hàm vô danh hoặc con trỏ hàm cho các hành động, bạn có thể gói đoạn code cài đặt máy trạng thái vào một lớp riêng với trạng thái của riêng nó, v.v.

There's nothing to say that you have to **process** all the state transitions at the same time. If you're going through the steps to sign up a user on your app, there's likely to be a number of transitions as they enter their details, validate their email, agree to the 107 different legislated warnings that online apps must now give, and so on. Keeping the state in external storage, and using it to drive a state machine, is a great way to handle these kind of workflow **requirements**.

Chẳng có luật nào bắt bạn phải xử lý tất cả các chuyển dịch trạng thái cùng một lúc cả. Nếu bạn đang đi qua các bước đăng ký người dùng trên ứng dụng của mình, hẳn sẽ có một loạt chuyển dịch khi họ nhập thông tin, xác thực email, đồng ý với 107 loại cảnh báo do luật pháp quy định mà các ứng dụng trực tuyến giờ buộc phải hiển thị, v.v. Lưu trạng thái ở một nơi lưu trữ bên ngoài, rồi dùng nó để điều khiển một máy trạng thái, là một cách tuyệt vời để xử lý những kiểu yêu cầu luồng công việc (workflow) như thế.

State Machines Are a Start State machines are underused by developers, and we'd like to encourage you to look for opportunities to apply them. But they don't solve all the problems associated with events. So let's move on to some other ways of looking at the problems of juggling events.

Máy trạng thái mới chỉ là khởi đầu Máy trạng thái bị các lập trình viên dùng quá ít, và chúng tôi muốn khích lệ bạn để mắt tìm cơ hội áp dụng chúng. Nhưng chúng không giải quyết được mọi vấn đề liên quan đến sự kiện. Vậy nên hãy chuyển sang vài cách nhìn khác về bài toán tung hứng với các sự kiện.

The Observer Pattern — Mẫu Observer

In the **observer** pattern we have a source of events, called the **observable** and a list of clients, the **observers**, who are interested in those events.

Trong mẫu observer, ta có một nguồn phát sinh sự kiện, gọi là đối tượng được quan sát (observable), và một danh sách các bên khách, tức những đối tượng quan sát (observer),

những kẻ quan tâm đến các sự kiện đó.

An observer registers its interest with the observable, typically by passing a reference to a function to be called. Subsequently, when the event occurs, the observable iterates down its list of observers and calls the function that each passed it. The event is given as a parameter to that call.

Một observer đăng ký mối quan tâm của mình với observable, thường là bằng cách truyền vào một tham chiếu đến một hàm sẽ được gọi. Sau đó, khi sự kiện xảy ra, observable duyệt qua danh sách các observer của mình và gọi hàm mà mỗi observer đã truyền cho nó. Sự kiện được đưa vào lệnh gọi đó dưới dạng một tham số.

Here's a simple example in Ruby. The Terminator module is used to terminate the application. Before it does so, however, it notifies all its observers that the application is going to exit. [39] They might use this notification to tidy up temporary resources, commit data, and so on:

Đây là một ví dụ đơn giản bằng Ruby. Module Terminator được dùng để kết thúc ứng dụng. Tuy nhiên, trước khi làm vậy, nó thông báo cho tất cả các observer của mình rằng ứng dụng sắp thoát. [39] Chúng có thể dùng thông báo này để dọn dẹp tài nguyên tạm, ghi (commit) dữ liệu, v.v.:

```
event/observer.rb
```

```
event/observer.rb
```

```
module Terminator CALLBACKS = []
```

```
module Terminator CALLBACKS = []
```

```
def self.register(callback) CALLBACKS « callback end
```

```
def self.register(callback) CALLBACKS « callback end
```

```
def self.exit(exit_status) CALLBACKS.each { |callback| callback.(exit_status) }
```

```
def self.exit(exit_status) CALLBACKS.each { |callback| callback.(exit_status) }
```

```
exit!(exit_status) end end
```

```
exit!(exit_status) end end
```

```
Terminator.register(-> (status) { puts "callback 1 sees #{ status }" }) Terminator.register(-> (status) {  
puts "callback 2 sees #{ status }" })
```

```
Terminator.register(-> (status) { puts "callback 1 sees #{ status }" }) Terminator.register(->  
(status) { puts "callback 2 sees #{ status }" })
```

```
Terminator.exit(99)
```

```
Terminator.exit(99)
```

```
$ ruby event/observer.rb callback 1 sees 99 callback 2 sees 99
```

```
$ ruby event/observer.rb callback 1 sees 99 callback 2 sees 99
```

There's not much code involved in creating an observable: you push a function reference onto a list, and then call those functions when the event occurs. This is a good example of when not to use a library.

Tạo ra một observable chẳng tốn mấy code: bạn đẩy một tham chiếu hàm vào một danh sách, rồi gọi các hàm đó khi sự kiện xảy ra. Đây là một ví dụ điển hình về lúc không nên dùng thư viện.

The observer/observable pattern has been used for decades, and it has served us well. It is particularly prevalent in user interface systems, where the callbacks are used to inform the application that some interaction has occurred.

Mẫu observer/observable đã được dùng suốt nhiều thập kỷ, và nó phục vụ ta khá tốt. Nó đặc biệt phổ biến trong các hệ thống giao diện người dùng, nơi các hàm callback được dùng để báo cho ứng dụng biết rằng một tương tác nào đó vừa xảy ra.

But the observer pattern has a problem: because each of the observers has to register with the observable, it introduces **coupling**. In addition, because in the typical implementation the callbacks are handled inline by the observable, synchronously, it can introduce performance bottlenecks.

Nhưng mẫu observer có một vấn đề: vì mỗi observer phải đăng ký với observable, nó tạo ra sự ghép nối (coupling). Thêm nữa, vì trong cách cài đặt điển hình các callback được observable xử lý nội tuyến (inline), một cách đồng bộ, nó có thể tạo ra các nút thắt cổ chai về hiệu năng.

This is solved by the next strategy, Publish/Subscribe.

Điều này được giải quyết bởi chiến lược kế tiếp, Publish/Subscribe.

Publish/Subscribe — Publish/Subscribe

Publish/Subscribe (pubsub) generalizes the observer pattern, at the same time solving the problems of coupling and performance.

Publish/Subscribe (pubsub) tổng quát hóa mẫu observer, đồng thời giải quyết cả vấn đề ghép nối lẫn hiệu năng.

In the pubsub model, we have publishers and subscribers. These are connected via channels. The channels are implemented in a separate body of code: sometimes a library, sometimes a process, and sometimes a distributed infrastructure. All this implementation detail is hidden from your code.

Trong mô hình pubsub, ta có các bên phát hành (publisher) và các bên đăng ký (subscriber). Chúng được kết nối với nhau qua các kênh (channel). Các kênh được cài đặt trong một khối code riêng biệt: đôi khi là một thư viện, đôi khi là một tiến trình, và đôi khi là cả một hạ tầng phân tán. Mọi chi tiết cài đặt này đều được giấu khỏi code của bạn.

Every **channel** has a name. Subscribers register interest in one or more of these named channels, and publishers write events to them. Unlike the observer pattern, the communication between the **publisher** and **subscriber** is handled outside your code, and is potentially **asynchronous**.

Mỗi kênh có một cái tên. Các subscriber đăng ký quan tâm đến một hoặc nhiều kênh được đặt tên này, còn các publisher thì ghi sự kiện vào chúng. Khác với mẫu observer, việc giao tiếp giữa publisher và subscriber được xử lý bên ngoài code của bạn, và có khả năng diễn ra bất đồng bộ.

Although you could implement a very basic pubsub system yourself, you probably don't want to. Most cloud service providers have pubsub offerings, allowing you to connect applications around the world. Every popular language will have at least one pubsub library.

Tuy bạn hoàn toàn có thể tự cài đặt một hệ thống pubsub rất cơ bản, nhưng có lẽ bạn chẳng muốn làm vậy đâu. Hầu hết các nhà cung cấp dịch vụ đám mây đều có sản phẩm pubsub, cho phép bạn kết nối các ứng dụng ở khắp nơi trên thế giới. Mọi ngôn ngữ phổ biến đều sẽ có ít nhất một thư viện pubsub.

Pubsub is a good technology for **decoupling** the handling of asynchronous events. It allows code to be added and replaced, potentially while the application is running, without altering existing code. The downside is that it can be hard to see what is going on in a system that uses pubsub heavily: you can't look at a publisher and immediately see which subscribers are involved with a particular message.

Pubsub là một công nghệ tốt để gỡ ghép nối việc xử lý các sự kiện bất đồng bộ. Nó cho phép thêm và thay thế code, thậm chí ngay trong khi ứng dụng đang chạy, mà không phải sửa code hiện có. Nhược điểm là rất khó nhìn ra điều gì đang diễn ra trong một hệ thống dùng pubsub dày đặc: bạn không thể nhìn vào một publisher rồi lập tức thấy ngay những subscriber nào liên quan đến một thông điệp cụ thể.

Compared to the observer pattern, pubsub is a great example of reducing coupling by abstracting up through a shared interface (the channel). However, it is still basically just a message passing system. Creating systems that respond to combinations of events will need more than this, so let's look at ways we can add a time dimension to event processing.

So với mẫu observer, pubsub là một ví dụ tuyệt vời về việc giảm ghép nối bằng cách trừu tượng hóa lên thông qua một giao diện dùng chung (chính là kênh). Tuy nhiên, về cơ bản nó vẫn chỉ là một hệ thống truyền thông điệp. Việc tạo ra những hệ thống biết phản hồi các tổ hợp sự kiện sẽ cần nhiều hơn thế, vậy nên hãy xem những cách ta có thể thêm chiều thời gian vào việc xử lý sự kiện.

Reactive Programming, Streams, And Events — Lập trình phản ứng, luồng dữ liệu và sự kiện

If you've ever used a spreadsheet, then you'll be familiar with reactive programming . If a cell contains a formula which refers to a second cell, then updating that second cell causes the first to update as well. The values react as the values they use change.

Nếu bạn từng dùng bảng tính, thì hẳn bạn đã quen với lập trình phản ứng (reactive programming). Nếu một ô chứa một công thức tham chiếu đến một ô thứ hai, thì cập nhật ô thứ hai đó sẽ khiến ô thứ nhất cập nhật theo. Các giá trị phản ứng lại khi những giá trị mà chúng dùng thay đổi.

There are many frameworks that can help with this kind of data-level reactivity: in the realm of the browser React and Vue.js are current favorites (but, this being JavaScript, this information will be out-of-date before this book is even printed).

Có nhiều framework có thể giúp ích cho kiểu phản ứng ở mức dữ liệu này: trong địa hạt trình duyệt thì React và Vue.js đang là những lựa chọn được ưa chuộng hiện nay (nhưng, vì đây là JavaScript, thông tin này chắc sẽ lỗi thời ngay trước cả khi cuốn sách này được in ra).

It's clear that events can also be used to trigger reactions in code, but it isn't necessarily easy to plumb them in. That's where streams come in.

Rõ ràng là sự kiện cũng có thể được dùng để kích hoạt các phản ứng trong code, nhưng đầu nối chúng vào không hẳn là chuyện dễ. Đó chính là lúc luồng dữ liệu (stream) ra tay.

Streams let us treat events as if they were a collection of data. It's as if we had a list of events, which got longer when new events arrive. The beauty of that is that we can treat streams just like any other collection: we can manipulate, combine, filter, and do all the other data-ish things we know so well. We can even combine event streams and regular collections. And streams can be asynchronous, which means your code gets the opportunity to respond to events as they arrive.

Luồng dữ liệu cho phép ta đối xử với các sự kiện như thể chúng là một tập hợp dữ liệu. Cứ như thể ta có một danh sách các sự kiện, và danh sách ấy dài thêm mỗi khi có sự kiện mới đến. Cái hay là ta có thể đối xử với luồng y như mọi tập hợp khác: ta có thể thao tác, kết hợp, lọc, và làm đủ thứ trò “dữ liệu” mà ta đã quá rành. Ta thậm chí còn có thể kết hợp luồng sự kiện với các tập hợp thông thường. Và luồng có thể là bất đồng bộ, nghĩa là code của bạn có cơ hội phản hồi các sự kiện ngay khi chúng đến.

The current de facto baseline for reactive event handling is defined on the site <http://reactivex.io>, which defines a language-agnostic set of principles and documents some common implementations. Here we'll use the RxJs library for JavaScript.

Chuẩn nền tảng thực tế (de facto) hiện nay cho việc xử lý sự kiện theo kiểu phản ứng được định nghĩa trên trang <http://reactivex.io>, nơi đặt ra một tập các nguyên tắc không phụ thuộc ngôn ngữ và mô tả một số cách cài đặt phổ biến. Ở đây ta sẽ dùng thư viện RxJs cho JavaScript.

Our first example takes two streams and zips them together: the

Ví dụ đầu tiên của ta lấy hai luồng rồi “khóa kéo” (zip) chúng lại với nhau: kết

result is a new stream where each element contains one item from the first input stream and one item from the other. In this case, the first stream is simply a list of five animal names. The second stream is more interesting: it's an interval timer which generates an event every 500ms. Because the streams are zipped together, a result is only generated when data is available on both, and so our result stream only emits a value every half second:

quả là một luồng mới mà mỗi phần tử chứa một mục từ luồng đầu vào thứ nhất và một mục từ luồng kia. Trong trường hợp này, luồng thứ nhất chỉ đơn giản là một danh sách năm tên loài vật. Luồng thứ hai thú vị hơn: nó là một bộ đếm thời gian theo khoảng (interval timer) sinh ra một sự kiện cứ mỗi 500ms. Vì hai luồng được khóa kéo với nhau, một kết quả chỉ được sinh ra khi cả hai luồng đều có dữ liệu, nên luồng kết quả của ta chỉ phát ra một giá trị mỗi nửa giây:

event/rx0/index.js

event/rx0/index.js

```
import * as Observable from 'rxjs' import { logValues } from "../rxcommon/logger.js"
```

```
import * as Observable from 'rxjs' import { logValues } from "../rxcommon/logger.js"
```

```
let animals = Observable.of( "ant", "bee", "cat", "dog", "elk" ) let ticker = Observable.interval(500)
```

```
let animals = Observable.of( "ant", "bee", "cat", "dog", "elk" ) let ticker = Observable.interval(500)
```

```
let combined = Observable.zip(animals, ticker)
```

```
let combined = Observable.zip(animals, ticker)
```

combined.subscribe(next => logValues(JSON.stringify(next))) This code uses a simple logging function [40] which adds items to a list in the browser window. Each item is timestamped with the time in milliseconds since the program started to run. Here's what it shows for our code:

combined.subscribe(next => logValues(JSON.stringify(next))) Đoạn code này dùng một hàm ghi log đơn giản [40], hàm này thêm các mục vào một danh sách trong cửa sổ trình duyệt. Mỗi mục được đóng dấu thời gian theo số mili-giây tính từ lúc chương trình bắt đầu chạy. Đây là những gì nó hiển thị cho code của ta:

images/events_rxjs_0.png

images/events_rxjs_0.png

Notice the timestamps: we're getting one event from the stream every 500ms. Each event contains a serial number (created by the interval observable) and the name of the next animal from the list. Watching it live in a browser, the log lines appear at every half second.

Để ý các dấu thời gian: ta nhận được một sự kiện từ luồng cứ mỗi 500ms. Mỗi sự kiện chứa một số thứ tự (do observable interval tạo ra) và tên loài vật kế tiếp lấy từ danh sách. Theo dõi trực tiếp trong trình duyệt, các dòng log hiện ra đều đặn mỗi nửa giây.

Event streams are normally populated as events occur, which implies that the observables that populate them can run in parallel. Here's an example that fetches information about users from a remote site. For this we'll use <https://reqres.in>, a public site that provides an open REST interface. As part of its API, we can fetch data on a particular (fake) user by performing a GET request to `users/<id>`. Our code fetches the users with the IDs 3, 2, and 1:

Luồng sự kiện thường được đổ đầy ngay khi các sự kiện xảy ra, điều này ngụ ý rằng các observable đổ dữ liệu vào chúng có thể chạy song song. Đây là một ví dụ lấy thông tin về người dùng từ một trang web ở xa. Để làm việc này ta sẽ dùng <https://reqres.in>, một trang công khai cung cấp một giao diện REST mở. Trong API của nó, ta có thể lấy dữ liệu về một người dùng (giả) cụ thể bằng cách thực hiện một yêu cầu GET tới `users/<id>`. Đoạn code của ta lấy về những người dùng có ID là 3, 2 và 1:

event/rx1/index.js

event/rx1/index.js

```
import * as Observable from 'rxjs' import { mergeMap } from 'rxjs/operators' import { ajax } from 'rxjs/ajax'
```

```
import * as Observable from 'rxjs' import { mergeMap } from 'rxjs/operators' import { ajax } from 'rxjs/ajax'
```

```
import { logValues } from "../rxcommon/logger.js"
```

```
import { logValues } from "../rxcommon/logger.js"
```

```
let users = Observable.of(3, 2, 1)
```

```
let users = Observable.of(3, 2, 1)
```

```
let result = users.pipe( mergeMap((user) => ajax.getJSON( https://reqres.in/api/users/ ${user} )))
```

```
let result = users.pipe( mergeMap((user) => ajax.getJSON( https://reqres.in/api/users/ ${user} )))
```

```
result.subscribe( resp => logValues(JSON.stringify(resp.data)), err => console.error(JSON.stringify(err)) )
```

```
result.subscribe( resp => logValues(JSON.stringify(resp.data)), err => console.error(JSON.stringify(err)) )
```

The internal details of the code are not too important. What's exciting is the result, shown in the following screenshot:

Các chi tiết bên trong của đoạn code không quan trọng lắm. Điều thú vị nằm ở kết quả, hiển thị trong ảnh chụp màn hình dưới đây:

images/events_three_users.png

[images/events_three_users.png](#)

Look at the timestamps: the three requests, or three separate streams, were processed in parallel. The first to come back, for id 2, took 82ms, and the next two came back 50 and 51ms later.

Hãy nhìn các dấu thời gian: ba yêu cầu, hay ba luồng riêng biệt, đã được xử lý song song. Cái về trước tiên, cho id 2, mất 82ms, và hai cái kế tiếp về sau đó 50 và 51ms.

Streams of Events Are Asynchronous Collections In the previous example, our list of user IDs (in the observable users) was static. But it doesn't have to be. Perhaps we want to collect this information when people log in to our site. All we have to do is to generate an observable event containing their user ID when their session is created, and use that observable instead of the static one. We'd then be fetching details about the

Luồng sự kiện là những tập hợp bất đồng bộ Ở ví dụ trước, danh sách các ID người dùng (trong observable users) là tĩnh. Nhưng nó không nhất thiết phải vậy. Có lẽ ta muốn thu thập thông tin này khi người ta đăng nhập vào trang của mình. Tất cả những gì ta phải làm là sinh ra một sự kiện observable chứa ID người dùng khi phiên của họ được tạo, rồi dùng observable đó thay cho cái tĩnh kia. Khi ấy ta sẽ lấy về thông tin chi tiết về

users as we received these IDs, and presumably storing them somewhere.

người dùng ngay khi nhận được các ID này, và phỏng đoán là sẽ lưu chúng ở đâu đó.

This is a very powerful abstraction: we no longer need to think about time as being something we have to manage. Event streams unify **synchronous** and asynchronous processing behind a common, convenient API.

Đây là một trừu tượng cực kỳ mạnh: ta không còn phải coi thời gian là thứ ta phải tự tay quản lý nữa. Luồng sự kiện hợp nhất việc xử lý đồng bộ và bất đồng bộ phía sau một API chung, tiện lợi.

Events Are Ubiquitous — Sự kiện có ở khắp nơi

Events are everywhere. Some are obvious: a button click, a timer expiring. Other are less so: someone logging in, a line in a file matching a pattern. But whatever their source, code that's crafted around events can be more responsive and better decoupled than its more linear counterpart.

Sự kiện có ở khắp mọi nơi. Một số thì hiển nhiên: một cú bấm nút, một bộ đếm giờ hết hạn. Số khác thì kém rõ ràng hơn: ai đó đăng nhập, một dòng trong file khớp với một mẫu. Nhưng dù đến từ nguồn nào, code được nhào nặn xoay quanh sự kiện sẽ có thể phản hồi nhanh nhạy hơn và được gỡ ghép nối tốt hơn so với người anh em tuyến tính của nó.

Related Sections Include — Các mục liên quan

Topic 28, Decoupling

Topic 28, Gỡ ghép nối

Topic 36, Blackboards

Topic 36, Bảng đen

Exercises — Bài tập

Exercise 19 (possible answer)

Bài tập 19 (gợi ý đáp án)

In the FSM section we mentioned that you could move the generic state machine implementation into its own class. That class would probably be initialized by passing in a table of transitions and an initial state.

Ở phần FSM ta có nhắc rằng bạn có thể chuyển phần cài đặt máy trạng thái tổng quát vào một lớp riêng. Lớp đó hẳn sẽ được khởi tạo bằng cách truyền vào một bảng chuyển dịch và một trạng thái ban đầu.

Try implementing the string extractor that way.

Hãy thử cài đặt bộ trích xuất chuỗi theo cách đó.

Exercise 20 (possible answer)

Bài tập 20 (gợi ý đáp án)

Which of these technologies (perhaps in combination) would be a good fit for the following situations:

Trong các công nghệ sau đây (có thể kết hợp nhiều cái), cái nào phù hợp với từng tình huống dưới đây:

If you receive three network interface down events within five minutes, notify the operations staff.

Nếu bạn nhận được ba sự kiện “giao diện mạng sập” trong vòng năm phút, hãy báo cho bộ phận vận hành.

If it is after sunset, and there is motion detected at the bottom of the stairs followed by motion detected at the top of the stairs, turn on the upstairs lights.

Nếu đã qua hoàng hôn, và phát hiện có chuyển động ở chân cầu thang rồi tiếp đó là chuyển động ở đầu cầu thang, hãy bật đèn tầng trên.

You want to notify various reporting systems that an order was completed.

Bạn muốn thông báo cho nhiều hệ thống báo cáo khác nhau rằng một đơn hàng đã hoàn tất.

In order to determine whether a **customer** qualifies for a car loan, the application needs to send requests to three backend services and wait for the responses.

Để xác định xem một khách hàng có đủ điều kiện vay mua xe hay không, ứng dụng cần gửi yêu cầu tới ba dịch vụ backend rồi chờ phản hồi.

If you can't describe what you are doing as a process, you don't know what you're doing.
W. Edwards Deming, (attr)

Nếu bạn không thể mô tả việc mình đang làm như một quy trình, thì bạn không hiểu mình đang làm gì. W. Edwards Deming, (attr)

Topic 30. Transforming Programming — Topic 30. Lập trình theo lối Biến đổi

All programs transform data, converting an input into an output. And yet when we think about design, we rarely think about creating transformations. Instead we worry about classes and modules, data structures and algorithms, languages and frameworks.

Mọi chương trình đều biến đổi dữ liệu, chuyển một đầu vào thành một đầu ra. Vậy mà khi nghĩ về thiết kế, ta hiếm khi nghĩ tới chuyện tạo ra các biến đổi (transformation). Thay vào đó ta lo lắng về các lớp và mô-đun, về cấu trúc dữ liệu và giải thuật, về ngôn ngữ và framework.

We think that this focus on code often misses the point: we need to get back to thinking of programs as being something that transforms inputs into outputs. When we do, many of the details we previously worried about just evaporate. The structure becomes clearer, the error handling more consistent, and the **coupling** drops way down.

Chúng tôi cho rằng cái lối chăm chăm vào code này thường làm ta lạc mất trọng tâm: ta cần quay lại với cách nghĩ rằng chương trình là một thứ biến đổi đầu vào thành đầu ra. Khi làm được vậy, nhiều chi tiết mà trước đó ta lo lắng bỗng tự bốc hơi. Cấu trúc trở nên rõ ràng hơn, việc xử lý lỗi nhất quán hơn, và sự ghép nối (coupling) tụt xuống thấp hẳn.

To start our investigation, let's take the time machine back to the 1970s and ask a Unix programmer to write us a program that lists the five longest files in a directory tree, where longest means "having the largest number of lines."

Để bắt đầu cuộc khảo sát, hãy lên cỗ máy thời gian quay về thập niên 1970 và nhờ một lập trình viên Unix viết cho ta một chương trình liệt kê năm tệp dài nhất trong một cây thư mục, trong đó "dài nhất" nghĩa là "có nhiều dòng nhất".

You might expect them to reach for an editor and start typing in C. But they wouldn't, because they are thinking about this in terms of what we have (a directory tree) and what we want (a list of files). Then they'd go to a terminal and type something like:

Bạn có thể tưởng họ sẽ với lấy trình soạn thảo và bắt đầu gõ C. Nhưng họ sẽ không làm vậy, vì họ đang nghĩ theo kiểu: ta có cái gì (một cây thư mục) và ta muốn cái gì (một danh sách tệp). Rồi họ ra terminal gõ đại loại như sau:

```
$ find . -type f | xargs wc -l | sort -n | tail -5
```

```
$ find . -type f | xargs wc -l | sort -n | tail -5
```

This is a series of transformations:

Đây là một chuỗi các biến đổi:

`find . -type f` Write a list of all the files (`-type f`) in or below the current directory (`.`) to standard output.

`find . -type f` Ghi ra đầu ra chuẩn (standard output) danh sách tất cả các tệp (`-type f`) nằm trong hoặc dưới thư mục hiện tại (`.`).

`xargs wc -l` Read lines from standard input and arrange for them all to be passed as arguments to the command `wc -l` . The `wc` program with the `-l` option counts the number of lines in each of its arguments and writes each result as "count filename" to standard output.

`xargs wc -l` Đọc các dòng từ đầu vào chuẩn (standard input) và thu xếp để truyền tất cả chúng làm tham số cho lệnh `wc -l` . Chương trình `wc` với tùy chọn `-l` đếm số dòng trong mỗi tham số của nó và ghi từng kết quả dưới dạng "số_dòng tên_tệp" ra đầu ra chuẩn.

`sort -n` Sort standard input assuming each line starts with a number (`-n`), writing the result to standard output.

`sort -n` Sắp xếp đầu vào chuẩn với giả định mỗi dòng bắt đầu bằng một con số (`-n`), ghi kết quả ra đầu ra chuẩn.

tail -5 Read standard input and write just the last five lines to standard output.

tail -5 Đọc đầu vào chuẩn và chỉ ghi ra năm dòng cuối cùng tới đầu ra chuẩn.

Run this in our book's directory and we get

Chạy nó trong thư mục cuốn sách của chúng tôi, ta được:

```
470 ./test_to_build.pml 487 ./dbc.pml 719 ./domain_languages.pml 727 ./dry.pml 9561 total
```

```
470 ./test_to_build.pml 487 ./dbc.pml 719 ./domain_languages.pml 727 ./dry.pml 9561 total
```

That last line is the total number of lines in all the files (not just those shown), because that's what wc does. We can strip it off by requesting one more line from tail , and then ignoring the last line:

Dòng cuối cùng đó là tổng số dòng trong tất cả các tệp (không chỉ những tệp được hiển thị), vì đó là việc wc làm. Ta có thể bỏ nó đi bằng cách yêu cầu thêm một dòng từ tail , rồi lờ đi dòng cuối:

```
$ find . -type f | xargs wc -l | sort -n | tail -6 | head -5 470 ./debug.pml 470 ./test_to_build.pml 487 ./dbc.pml 719 ./domain_languages.pml 727 ./dry.pml
```

```
$ find . -type f | xargs wc -l | sort -n | tail -6 | head -5 470 ./debug.pml 470 ./test_to_build.pml 487 ./dbc.pml 719 ./domain_languages.pml 727 ./dry.pml
```

images/wc-pipeline.png

images/wc-pipeline.png

Figure 1. The find pipeline as a series of transformations

Hình 1. Đường ống (pipeline) find như một chuỗi các biến đổi

Let's look at this in terms of the data that flows between the individual steps. Our original requirement, "top 5 files in terms of lines," becomes a series of transformations (also show in the figure).

Hãy nhìn vào chuyện này theo góc độ dữ liệu chảy giữa từng bước. Yêu cầu ban đầu của ta, "5 tệp nhiều dòng nhất", trở thành một chuỗi các biến đổi (cũng được vẽ trong hình).

directory name → list of files → list with line numbers → sorted list → highest five + total → highest five

tên thư mục → danh sách tệp → danh sách kèm số dòng → danh sách đã sắp xếp → năm cao nhất + tổng → năm cao nhất

It's almost like an industrial assembly line: feed raw data in one

Nó gần như một dây chuyền lắp ráp công nghiệp: nạp dữ liệu thô vào một đầu

end and the finished product (information) comes out the other.

còn thành phẩm (thông tin) thì lòi ra ở đầu kia.

And we like to think about all code this way.

Và chúng tôi thích nghĩ về mọi đoạn code theo lối này.

Programming Is About Code, But Programs Are — Lập trình là Về Code, Nhưng Chương trình thì

Tip 49: About Data

Tip 49: Là Về Dữ liệu

Finding Transformations — Tìm các Biến đổi

Sometimes the easiest way to find the transformations is to start with the requirement and determine its inputs and outputs. Now you’ve defined the function representing the overall program. You can then find steps that lead you from input to output. This is a top-down approach.

Đôi khi cách dễ nhất để tìm ra các biến đổi là bắt đầu từ yêu cầu rồi xác định đầu vào và đầu ra của nó. Thế là bạn đã định nghĩa được hàm đại diện cho toàn bộ chương trình. Sau đó bạn có thể tìm ra các bước dẫn bạn đi từ đầu vào tới đầu ra. Đây là cách tiếp cận từ trên xuống (top-down).

For example, you want to create a website for folks playing word games that finds all the words that can be made from a set of letters. Your input here is a set of letters, and your output is a list of three-letter words, four-letter words, and so on:

Ví dụ, bạn muốn tạo một website cho những người chơi trò chơi chữ, website tìm tất cả các từ có thể ghép được từ một tập chữ cái. Đầu vào ở đây là một tập chữ cái, còn đầu ra là một danh sách các từ ba chữ cái, bốn chữ cái, v.v.:

“lvyin” is transformed to $\rightarrow 3 \Rightarrow$ ivy, lin, nil, yin

“lvyin” được biến đổi thành $\rightarrow 3 \Rightarrow$ ivy, lin, nil, yin

4 $\Rightarrow$ inly, liny, viny

4 $\Rightarrow$ inly, liny, viny

5 $\Rightarrow$ vinyl

5 $\Rightarrow$ vinyl

(Yes, they are all words, at least according to the macOS dictionary.)

(Vâng, tất cả đều là từ thật, ít nhất là theo từ điển macOS.)

The trick behind the overall application is simple: we have a dictionary which groups words by a **signature**, chosen so that all

Mánh khéo đằng sau toàn bộ ứng dụng rất đơn giản: ta có một từ điển gom các từ theo một chữ ký (signature), được chọn sao cho tất cả các

words containing the same letters will have the same signature. The simplest signature function is just the sorted list of letters in the word. We can then look up an input string by generating a signature for it, and then seeing which words (if any) in the dictionary have that same signature.

từ chứa cùng những chữ cái sẽ có cùng một chữ ký. Hàm chữ ký đơn giản nhất chính là danh sách các chữ cái trong từ đã được sắp xếp. Khi đó ta có thể tra một chuỗi đầu vào bằng cách sinh ra chữ ký cho nó, rồi xem từ nào (nếu có) trong từ điển có cùng chữ ký ấy.

Thus the **anagram finder** breaks down into four separate transformations:

Như vậy, bộ tìm đảo chữ (anagram finder) tách ra thành bốn biến đổi riêng biệt:

Step **Transformation** Sample data p

Bước Biến đổi Dữ liệu mẫu c

Step Initial input “ylvin” 0:

Bước Đầu vào ban đầu “ylvin” 0:

Step All combinations of three or more letters 1: vin, viy, vil, vny, vnl, vyl, iny, inl, iyl, nyl, viny, vinl, viyl, vnyl, inyl, vinyl

Bước Tất cả tổ hợp từ ba chữ cái trở lên 1: vin, viy, vil, vny, vnl, vyl, iny, inl, iyl, nyl, viny, vinl, viyl, vnyl, inyl, vinyl

Step Signatures of the combinations 2: inv, ivy, ilv, nvy, lnv, lvy, iny, iln, ily, lny, invy, ilnv, ilvy, lnvy, ilny, ilnv

Bước Chữ ký của các tổ hợp 2: inv, ivy, ilv, nvy, lnv, lvy, iny, iln, ily, lny, invy, ilnv, ilvy, lnvy, ilny, ilnv

Step 3: List of all dictionary words which match ivy, yin, nil, lin, viny, liny, inly, any of the vinyl signatures

Bước 3: Danh sách tất cả các từ trong từ điển khớp ivy, yin, nil, lin, viny, liny, inly, với bất kỳ chữ ký nào vinyl

Step Words grouped by length 4: 3 => ivy, lin, nil, yin

Bước Các từ được gom theo độ dài 4: 3 => ivy, lin, nil, yin

4 => inly, liny, viny

4 => inly, liny, viny

5 => vinyl

5 => vinyl

Transformations All the Way Down

Toàn Biến đổi từ Trên xuống Dưới

Let's start by looking at step 1, which takes a word and creates a list of all combinations of three or more letters. This step can itself be expressed as a list of transformations:

Hãy bắt đầu nhìn vào bước 1, bước nhận một từ và tạo ra danh sách tất cả các tổ hợp từ ba chữ cái trở lên. Bản thân bước này lại có thể được biểu diễn thành một danh sách các biến đổi:

Step Transformation Sample data

Bước Biến đổi Dữ liệu mẫu

Step 1.0: Initial input “vinyl”

Bước 1.0: Đầu vào ban đầu “vinyl”

Step 1.1: Convert to characters v, i, n, y, l

Bước 1.1: Chuyển thành các ký tự v, i, n, y, l

Step 1.2: Get all subsets

Bước 1.2: Lấy tất cả tập con

[], [v], [i], ... [v,i], [v,n], [v,y], ... [v,i,n], [v,i,y], ... [v,n,y], [i,n,y], [v,i,n,y],

[], [v], [i], ... [v,i], [v,n], [v,y], ... [v,i,n], [v,i,y], ... [v,n,y], [i,n,y], [v,i,n,y],

Step 1.3: Only those longer than three characters [v,i,n], [v,i,y], ... [i,n,y], [v,i,n,y]

Bước 1.3: Chỉ giữ những tập dài hơn ba ký tự [v,i,n], [v,i,y], ... [i,n,y,l], [v,i,n,y,l]

Step 1.4: Convert back to strings [vin,viy, ... inyl,vinyl]

Bước 1.4: Chuyển ngược lại thành chuỗi [vin,viy, ... inyl,vinyl]

We’ve now reached the point where we can easily implement each transformation in code (using Elixir in this case):

Giờ ta đã tới điểm có thể dễ dàng cài đặt từng biến đổi bằng code (ở đây dùng Elixir):

function-pipelines/anagrams/lib/anagrams.ex

function-pipelines/anagrams/lib/anagrams.ex

```
defp all_subsets_longer_than_three_characters(word) do word |> String.codepoints() |> Comb.subsets()
|> Stream.filter( fn subset -> length(subset) >= 3 end ) |> Stream.map(&List.to_string(&1)) end

defp all_subsets_longer_than_three_characters(word) do word |> String.codepoints() |>
Comb.subsets() |> Stream.filter( fn subset -> length(subset) >= 3 end ) |> Stream.map(&List.to_string(&1))
end
```

What’s with the |> Operator? Elixir, along with many other functional languages, has a pipeline operator, sometimes called a forward pipe or just a pipe . [41] All it does is take the value on its left and insert it as the first parameter of the function on its right, so

Toán tử |> này là gì vậy? Elixir, cùng nhiều ngôn ngữ hàm khác, có một toán tử đường ống (pipeline operator), đôi khi gọi là forward pipe hay đơn giản là pipe . [41] Tất cả những gì nó làm là lấy giá trị ở bên trái và chèn nó làm tham số đầu tiên của hàm ở bên phải, nên

“vinyl” |> String.codepoints |> Comb.subsets()

“vinyl” |> String.codepoints |> Comb.subsets()

is the same as writing

giống hệt như viết

Comb.subsets(String.codepoints(“vinyl”))

Comb.subsets(String.codepoints(“vinyl”))

(Other languages may inject this piped value as the last parameter of the next function—it largely depends on the style of the built-in libraries.)

(Các ngôn ngữ khác có thể chèn giá trị được đẩy qua ống này làm tham số cuối cùng của hàm kế tiếp—điều đó phần lớn tùy vào phong cách của các thư viện dựng sẵn.)

You might think that this is just syntactic sugar. But in a very real way the pipeline operator is a revolutionary opportunity to think differently. Using a pipeline means that you’re automatically thinking in terms of transforming data; each time you see |> you’re actually seeing a place where data is flowing between one transformation and the next.

Bạn có thể nghĩ đây chỉ là cú pháp ngọt (syntactic sugar). Nhưng theo một nghĩa rất thực, toán tử pipeline là một cơ hội mang tính cách mạng để nghĩ khác đi. Dùng pipeline nghĩa là bạn tự động nghĩ theo kiểu biến đổi dữ liệu; mỗi lần bạn thấy |> là bạn đang thực sự nhìn thấy một nơi mà dữ liệu đang chảy giữa biến đổi này và biến đổi kế tiếp.

Many languages have something similar: Elm, F#, and Swift have |> , Clojure has -> and -> (which work a little differently), R has %>% . Haskell both has pipe operators and makes it easy to declare new ones. As we write this, there’s talk of adding |> to JavaScript.

Nhiều ngôn ngữ có thứ tự tương tự: Elm, F# và Swift có `|>`, Clojure có `->` và `->>` (hoạt động khác đi đôi chút), R có `%>%`. Haskell vừa có các toán tử pipe vừa khiến việc khai báo toán tử mới trở nên dễ dàng. Khi chúng tôi viết những dòng này, người ta đang bàn chuyện thêm `|>` vào JavaScript.

If your current language supports something similar, you're in luck. If it doesn't, see [Language X Doesn't Have Pipelines](#).

Nếu ngôn ngữ hiện tại của bạn hỗ trợ thứ tự tương tự, bạn gặp may rồi. Nếu không, hãy xem [Ngôn ngữ X Không Có Pipeline](#).

Anyway, back to the code.

Dù sao thì, quay lại với code.

Keep on Transforming... Now look at Step 2 of the main program, where we convert the subsets into signatures. Again, it's a simple transformation—a list of subsets becomes a list of signatures:

Tiếp tục Biến đổi... Giờ hãy nhìn Bước 2 của chương trình chính, nơi ta chuyển các tập con thành chữ ký. Lại một lần nữa, đó là một biến đổi đơn giản—một danh sách tập con trở thành một danh sách chữ ký:

Step Transformation Sample data

Bước Biến đổi Dữ liệu mẫu

Step 2.0: initial input vin, viy, ... inyl, vinyl

Bước 2.0: đầu vào ban đầu vin, viy, ... inyl, vinyl

Step 2.1: convert to signatures inv, ivy ... ilny, inlv

Bước 2.1: chuyển thành chữ ký inv, ivy ... ilny, inlv

The Elixir code in the following listing is just as simple:

Đoạn code Elixir trong danh sách dưới đây cũng đơn giản y như vậy:

```
function-pipelines/anagrams/lib/anagrams.ex
```

```
function-pipelines/anagrams/lib/anagrams.ex
```

```
defp as_unique_signatures(subsets) do subsets |> Stream.map(&Dictionary.signature_of/1) end
      defp as_unique_signatures(subsets) do subsets |> Stream.map(&Dictionary.signature_of/1)
      end
```

Now we transform that list of signatures: each signature gets mapped to the list of known words with the same signature, or nil if there are no such words. We then have to remove the nils and flatten the nested lists into a single level:

Giờ ta biến đổi cái danh sách chữ ký đó: mỗi chữ ký được ánh xạ tới danh sách các từ đã biết có cùng chữ ký, hoặc nil nếu không có từ nào như vậy. Sau đó ta phải loại bỏ các nil và làm phẳng các danh sách lồng nhau thành một cấp duy nhất:

```
function-pipelines/anagrams/lib/anagrams.ex
```

```
function-pipelines/anagrams/lib/anagrams.ex
```

```
defp find_in_dictionary(signatures) do signatures |> Stream.map(&Dictionary.lookup_by_signature/1)
      defp find_in_dictionary(signatures) do signatures |> Stream.map(&Dictionary.lookup_by_signature/1)
```



```
|> Stream.reject(&is_nil/1) |> Stream.concat(&(&1)) end
```

Step 4, grouping the words by length, is another simple transformation, converting our list into a map where the keys are the lengths, and the values are all words with that length:

Bước 4, gom các từ theo độ dài, là một biến đổi đơn giản nữa, chuyển danh sách của ta thành một map trong đó khóa là độ dài, còn giá trị là tất cả các từ có độ dài đó:

```
function-pipelines/anagrams/lib/anagrams.ex
```

```
function-pipelines/anagrams/lib/anagrams.ex
```

```
defp group_by_length(words) do words |> Enum.sort() |> Enum.group_by(&String.length/1) end

defp group_by_length(words) do words |> Enum.sort() |> Enum.group_by(&String.length/1)
end
```

Language X Doesn't Have Pipelines

Ngôn ngữ X Không Có Pipeline

Pipelines have been around for a long time, but only in niche languages. They've only moved into the mainstream recently, and many popular languages still don't support the concept.

Pipeline đã có mặt từ lâu, nhưng chỉ trong các ngôn ngữ ngách. Mãi gần đây chúng mới chen được vào dòng chính, và nhiều ngôn ngữ phổ biến vẫn chưa hỗ trợ khái niệm này.

The good news is that thinking in transformations doesn't require a particular language syntax: it's more a philosophy of design. You still construct your code as transformations, but you write them as a series of assignments:

Tin tốt là việc nghĩ theo lối biến đổi không đòi hỏi một cú pháp ngôn ngữ cụ thể nào: nó giống một triết lý thiết kế hơn. Bạn vẫn dựng code của mình thành các biến đổi, chỉ là bạn viết chúng thành một chuỗi các phép gán:

```
const content = File.read(file_name);

const content = File.read(file_name);
const lines = find_matching_lines(content, pattern)
const lines = find_matching_lines(content, pattern)
const result = truncate_lines(lines)
const result = truncate_lines(lines)
```

It's a little more tedious, but it gets the job done.

Hơi lè mè hơn một chút, nhưng vẫn xong việc.

Putting It All Together We've written each of the individual transformations. Now it's time to string them all together into our main function:

Ráp Tất cả Lại với Nhau Ta đã viết từng biến đổi riêng lẻ. Giờ là lúc nối hết chúng lại với nhau thành hàm chính của ta:

```
function-pipelines/anagrams/lib/anagrams.ex
```

```
function-pipelines/anagrams/lib/anagrams.ex
```

```
def anagrams_in(word) do
  def anagrams_in(word) do
```

```
word |> all_subsets_longer_than_three_characters() |> as_unique_signatures() |> find_in_dictionary()
|> group_by_length() end
```

```
word |> all_subsets_longer_than_three_characters() |> as_unique_signatures() |>
find_in_dictionary() |> group_by_length() end
```

Does it work? Let's try it:

Nó có chạy không? Hãy thử xem:

```
iex(1)> Anagrams.anagrams_in "lyvin" %{ 3 => ["ivy", "lin", "nil", "yin"], 4 => ["inly", "liny", "viny"],
5 => ["vinyl"] }
```

```
iex(1)> Anagrams.anagrams_in "lyvin" %{ 3 => ["ivy", "lin", "nil", "yin"], 4 => ["inly", "liny",
"viny"], 5 => ["vinyl"] }
```

Why Is This So Great? — Tại sao Cái này lại Tuyệt đến vậy?

Let's look at the body of the main function again:

Hãy nhìn lại thân của hàm chính một lần nữa:

```
word |> all_subsets_longer_than_three_characters() |> as_unique_signatures() |> find_in_dictionary()
|> group_by_length()
```

```
word |> all_subsets_longer_than_three_characters() |> as_unique_signatures() |>
find_in_dictionary() |> group_by_length()
```

It's simply a chain of the transformations needed to meet our requirement, each taking input from the previous transformation and passing output to the next. That comes about as close to **literate code** as you can get.

Nó đơn giản là một chuỗi các biến đổi cần thiết để đáp ứng yêu cầu của ta, mỗi cái nhận đầu vào từ biến đổi trước và truyền đầu ra cho cái kế tiếp. Cái đó tiến gần tới mức code “đọc như văn” (literate code) nhất mà bạn có thể đạt được.

But there's something deeper, too. If your background is objectoriented programming, then your reflexes demand that you hide data, encapsulating it inside objects. These objects then chatter back and forth, changing each other's **state**. This introduces a lot of coupling, and it is a big reason that OO systems can be hard to change.

Nhưng còn có cái gì đó sâu hơn nữa. Nếu xuất thân của bạn là lập trình hướng đối tượng, thì phản xạ của bạn sẽ đòi giấu dữ liệu đi, đóng gói (encapsulating) nó vào trong các đối tượng. Rồi những đối tượng này tán gẫu qua lại với nhau, làm thay đổi trạng thái của nhau. Điều này đưa vào rất nhiều ghép nối, và là một lý do lớn khiến các hệ thống OO có thể rất khó thay đổi.

Tip 50: Don't Hoard State; Pass It Around

Tip 50: Đừng Tích trữ Trạng thái; Hãy Chuyển Nó Đi

In the transformational model, we turn that on its head. Instead of little pools of data spread all over the system, think of data as a mighty river, a flow . Data becomes a peer to functionality: a pipeline is a sequence of code $\rightarrow$ data $\rightarrow$ code $\rightarrow$ data.... The data is no longer tied to a particular group of functions, as it is in a class definition. Instead it is free to represent the unfolding progress of our application as it transforms its inputs into its outputs. This means that we can greatly reduce coupling: a function can be used (and reused) anywhere its parameters match the output of some other function.

Trong mô hình biến đổi, ta lật ngược điều đó lại. Thay vì những vũng dữ liệu nhỏ rải rác khắp hệ thống, hãy nghĩ về dữ liệu như một con sông hùng vĩ, một dòng chảy (flow). Dữ liệu trở thành ngang hàng với chức năng: một pipeline là một dãy code → dữ liệu → code → dữ liệu.... Dữ liệu không còn bị trói vào một nhóm hàm cụ thể, như trong một định nghĩa lớp. Thay vào đó, nó tự do biểu diễn tiến trình đang mở dần ra của ứng dụng khi nó biến đổi đầu vào thành đầu ra. Điều này nghĩa là ta có thể giảm ghép nối đi rất nhiều: một hàm có thể được dùng (và tái dùng) ở bất cứ đâu mà tham số của nó khớp với đầu ra của một hàm nào đó khác.

Yes, there is still a degree of coupling, but in our experience it's more manageable than the OO-style of command and control. And, if you're using a language with type checking, you'll get compile-time warnings when you try to connect two incompatible things.

Vâng, vẫn còn một mức độ ghép nối nhất định, nhưng theo kinh nghiệm của chúng tôi thì nó dễ quản hơn so với kiểu “ra lệnh và điều khiển” của OO. Và, nếu bạn dùng một ngôn ngữ có kiểm tra kiểu (type checking), bạn sẽ nhận được cảnh báo lúc biên dịch khi cố nối hai thứ không tương thích với nhau.

What About Error Handling? — Còn Xử lý Lỗi thì sao?

So far our transforms have worked in a world where nothing goes wrong. How can we use them in the real world, though? If we can only build linear chains, how can we add all that conditional logic that we need for error checking?

Cho tới giờ, các biến đổi của ta vẫn hoạt động trong một thế giới mà không gì trục trặc. Nhưng làm sao ta dùng chúng trong thế giới thực? Nếu ta chỉ có thể dựng các chuỗi tuyến tính, thì làm sao thêm vào được tất cả cái logic điều kiện mà ta cần để kiểm tra lỗi?

There are many ways of doing this, but they all rely on a basic convention: we never pass raw values between transformations. Instead, we wrap them in a data structure (or type) which also tells us if the contained value is valid. In Haskell, for example, this **wrapper** is called `Maybe`. In F# and Scala it's `Option`.

Có nhiều cách làm chuyện này, nhưng tất cả đều dựa trên một quy ước cơ bản: ta không bao giờ chuyển các giá trị thô giữa các biến đổi. Thay vào đó, ta bọc chúng trong một cấu trúc dữ liệu (hay kiểu) mà cũng cho ta biết giá trị bên trong có hợp lệ hay không. Trong Haskell chẳng hạn, cái bọc này được gọi là `Maybe`. Trong F# và Scala nó là `Option`.

How you use this concept is language specific. In general, though, there are two basic ways of writing the code: you can

Cách bạn dùng khái niệm này thì tùy ngôn ngữ. Nhưng nhìn chung, có hai cách cơ bản để viết code: bạn có thể

handle checking for errors inside your transformations or outside them.

xử lý việc kiểm tra lỗi bên trong các biến đổi hoặc bên ngoài chúng.

Elixir, which we've used so far, doesn't have this support built in. For our purposes this is a good thing, as we get to show an implementation from the ground up. Something similar should work in most other languages.

Elixir, ngôn ngữ ta đã dùng tới giờ, không có sẵn sự hỗ trợ này. Với mục đích của ta thì đây lại là chuyện tốt, vì ta được dịp trình bày một cách cài đặt từ con số không. Một thứ tương tự sẽ chạy được trong hầu hết các ngôn ngữ khác.

First, Choose a Representation We need a representation for our wrapper (the data structure that carries around a value or an error indication). You can use structures for this, but Elixir already has a pretty strong convention: functions tend to return a tuple containing either `{:ok, value}` or `{:error, reason}`. For example, `File.open` returns either `:ok` and an IO **process** or `:error` and a reason code:

Trước hết, Chọn một Cách Biểu diễn Ta cần một cách biểu diễn cho cái bọc của mình (cấu trúc dữ liệu mang theo một giá trị hoặc một chỉ báo lỗi). Bạn có thể dùng struct cho việc này, nhưng Elixir vốn đã có một quy ước khá mạnh: các hàm thường trả về một tuple chứa hoặc `{:ok, value}` hoặc `{:error, reason}`. Ví dụ, `File.open` trả về hoặc `:ok` và một tiến trình IO, hoặc `:error` và một mã lý do:

```
iex(1)> File.open( "/etc/passwd" ) {:ok, #PID<0.109.0>} iex(2)> File.open( "/etc/wombat" ) {:error, :enoent}
```

```
iex(1)> File.open( "/etc/passwd" ) {:ok, #PID<0.109.0>} iex(2)> File.open( "/etc/wombat" )
{:error, :enoent}
```

We'll use the `:ok / :error` tuple as our wrapper when passing things through a pipeline.

Ta sẽ dùng tuple `:ok / :error` làm cái bọc khi chuyển mọi thứ qua một pipeline.

Then Handle It Inside Each Transformation Let's write a function that returns all the lines in a file that contain a given string, truncated to the first 20 characters. We want to write it as a transformation, so the input will be a file name and a string to match, and the output will be either an `:ok` tuple with a list of lines or an `:error` tuple with some kind of reason. The top-level function should look something like this:

Rồi Xử lý Nó Bên trong Mỗi Biến đổi Hãy viết một hàm trả về tất cả các dòng trong một tệp có chứa một chuỗi cho trước, được cắt bớt còn 20 ký tự đầu. Ta muốn viết nó như một biến đổi, nên đầu vào sẽ là một tên tệp và một chuỗi cần khớp, còn đầu ra sẽ hoặc là một tuple `:ok` kèm danh sách các dòng, hoặc một tuple `:error` kèm một kiểu lý do nào đó. Hàm ở cấp cao nhất nên trông đại loại như sau:

```
function-pipelines/anagrams/lib/grep.ex
```

```
function-pipelines/anagrams/lib/grep.ex
```

```
def find_all(file_name, pattern) do
```

```
  def find_all(file_name, pattern) do
```

```
    File.read(file_name) |> find_matching_lines(pattern) |> truncate_lines() end
```

```
    File.read(file_name) |> find_matching_lines(pattern) |> truncate_lines() end
```

There's no explicit error checking here, but if any step in the pipeline returns an error tuple then the pipeline will return that error without executing the functions that follow. [42] We do this using Elixir's pattern matching:

Ở đây không có kiểm tra lỗi tường minh nào, nhưng nếu bất kỳ bước nào trong pipeline trả về một tuple lỗi thì pipeline sẽ trả về cái lỗi đó mà không thực thi các hàm theo sau. [42] Ta làm chuyện này bằng cách khớp mẫu (pattern matching) của Elixir:

```
function-pipelines/anagrams/lib/grep.ex
```

```
function-pipelines/anagrams/lib/grep.ex
```

```
defp find_matching_lines({:ok, content}, pattern) do content |> String.split(~r/\n/) |> Enum.filter(&String.match?(&1, pattern)) |> ok_unless_empty() end
```

```
defp find_matching_lines({ :ok , content}, pattern) do content |> String.split( ~r/\n/ ) |>
Enum.filter(&String.match?(&1, pattern)) |> ok_unless_empty() end
defp find_matching_lines(error, _), do : error
defp find_matching_lines(error, _), do : error
```

```
defp truncate_lines({ :ok , lines }) do lines |> Enum.map(&String.slice(&1, 0, 20)) |> ok() end
  defp truncate_lines({ :ok , lines }) do lines |> Enum.map(&String.slice(&1, 0, 20)) |> ok()
  end
defp truncate_lines(error), do : error
  defp truncate_lines(error), do : error
```

```
defp ok_unless_empty([], do : error( "nothing found" ) defp ok_unless_empty(result), do : ok(result)
  defp ok_unless_empty([], do : error( "nothing found" ) defp ok_unless_empty(result), do :
    ok(result)
defp ok(result), do : { :ok , result } defp error(reason), do : { :error , reason }
  defp ok(result), do : { :ok , result } defp error(reason), do : { :error , reason }
```

Have a look at the function `find_matching_lines` . If its first parameter is an `:ok` tuple, it uses the content in that tuple to find

Hãy nhìn hàm `find_matching_lines` . Nếu tham số đầu tiên của nó là một tuple `:ok` , nó dùng nội dung trong tuple đó để tìm

lines matching the pattern. However, if the first parameter is not an `:ok` tuple, the second version of the function runs, which just returns that parameter. This way the function simply forwards an error down the pipeline. The same thing applies to `truncate_lines` .

các dòng khớp với mẫu. Tuy nhiên, nếu tham số đầu tiên không phải là một tuple `:ok` , thì phiên bản thứ hai của hàm chạy, nó chỉ đơn giản trả về tham số đó. Bằng cách này, hàm chỉ đơn giản chuyển tiếp một lỗi xuống dưới pipeline. Điều tương tự cũng áp dụng cho `truncate_lines` .

We can play with this at the console:

Ta có thể nghịch thử nó ở console:

```
iex> Grep.find_all "/etc/passwd" , ~r/www/ { :ok, [ "_www: * :70:70:World W", "_wwwproxy: * :252:252:" ] }
```

```
iex> Grep.find_all "/etc/passwd" , ~r/www/ { :ok, [ "_www: * :70:70:World W", "_wwwproxy: * :252:252:" ] }
```

```
iex> Grep.find_all "/etc/passwd" , ~r/wombat/ { :error, "nothing found" } iex> Grep.find_all "/etc/koala" , ~r/www/ { :error, :enoent }
```

```
iex> Grep.find_all "/etc/passwd" , ~r/wombat/ { :error, "nothing found" } iex> Grep.find_all "/etc/koala" , ~r/www/ { :error, :enoent }
```

You can see that an error anywhere in the pipeline immediately becomes the value of the pipeline.

Bạn có thể thấy rằng một lỗi ở bất cứ đâu trong pipeline lập tức trở thành giá trị của cả pipeline.

Or Handle It in the Pipeline You might be looking at the `find_matching_lines` and `truncate_lines` functions thinking that we've moved the burden of error handling into the transformations. You'd

be right. In a language which uses pattern matching in function calls, such as Elixir, the effect is lessened, but it's still ugly.

Hoặc Xử lý Nó Trong Pipeline Có thể bạn đang nhìn hai hàm `find_matching_lines` và `truncate_lines` mà nghĩ rằng ta đã dồn gánh nặng xử lý lỗi vào trong các biến đổi. Bạn nghĩ đúng đấy. Trong một ngôn ngữ dùng khớp mẫu trong lời gọi hàm, như Elixir, hiệu ứng có nhẹ đi, nhưng nó vẫn xấu.

It would be nice if Elixir had a version of the pipeline operator `|>` that knew about the `:ok / :error` tuples and which short-circuited execution when an error occurred. [43] But the fact that it doesn't allows us to add something similar, and in a way that is applicable to a number of other languages.

Sẽ thật hay nếu Elixir có một phiên bản của toán tử pipeline `|>` biết về các tuple `:ok / :error` và biết đoản mạch (short-circuit) việc thực thi khi có lỗi xảy ra. [43] Nhưng chính cái việc nó không có lại cho phép ta tự thêm một thứ tương tự, và theo một cách áp dụng được cho khá nhiều ngôn ngữ khác.

The problem we face is that when an error occurs we don't want to run code further down the pipeline, and that we don't want that code to know that this is happening. This means that we

Vấn đề ta đối mặt là khi có lỗi xảy ra, ta không muốn chạy code ở xa hơn nữa trong pipeline, và ta không muốn cái code đó biết rằng chuyện này đang diễn ra. Điều này nghĩa là ta

need to defer running pipeline functions until we know that previous steps in the pipeline were successful. To do this, we'll need to change them from function calls into function values that can be called later. Here's one implementation:

cần hoãn việc chạy các hàm trong pipeline lại cho đến khi ta biết rằng các bước trước đó trong pipeline đã thành công. Để làm được vậy, ta cần đổi chúng từ những lời gọi hàm thành những giá trị hàm có thể được gọi sau này. Đây là một cách cài đặt:

```
function-pipelines/anagrams/lib/grep1.ex
```

```
function-pipelines/anagrams/lib/grep1.ex
```

```
defmodule Grep1 do
```

```
  defmodule Grep1 do
```

```
    def and_then({ :ok , value }, func), do : func.(value) def and_then(anything_else, _func), do :  
    anything_else
```

```
      def and_then({ :ok , value }, func), do : func.(value) def and_then(anything_else, _func), do  
      : anything_else
```

```
    def find_all(file_name, pattern) do File.read(file_name) |> and_then(&find_matching_lines(&1,  
    pattern)) |> and_then(&truncate_lines(&1)) end
```

```
      def find_all(file_name, pattern) do File.read(file_name) |> and_then(&find_matching_lines(&1,  
      pattern)) |> and_then(&truncate_lines(&1)) end
```

```
    defp find_matching_lines(content, pattern) do content |> String.split(~r/\n/) |> Enum.filter(&String.match?(&1,  
    pattern)) |> ok_unless_empty() end
```

```
      defp find_matching_lines(content, pattern) do content |> String.split( ~r/\n/ ) |>  
      Enum.filter(&String.match?(&1, pattern)) |> ok_unless_empty() end
```

```
    defp truncate_lines(lines) do lines |> Enum.map(&String.slice(&1, 0, 20)) |> ok() end
```

```
      defp truncate_lines(lines) do lines |> Enum.map(&String.slice(&1, 0, 20)) |> ok() end
```

```
defp ok_unless_empty([], do : error( "nothing found" ) defp ok_unless_empty(result), do : ok(result)
  defp ok_unless_empty([], do : error( "nothing found" ) defp ok_unless_empty(result), do :
    ok(result)
```

defp ok(result), do : { :ok , result } defp error(reason), do : { :error , reason } end The and_then function is an example of a **bind function**: it takes a value wrapped in something, then applies a function to that

```
defp ok(result), do : { :ok , result } defp error(reason), do : { :error , reason } end Hàm
and_then là một ví dụ về hàm bind (bind function): nó nhận một giá trị được bọc trong
thứ gì đó, rồi áp dụng một hàm lên giá trị
```

value, returning a new wrapped value. Using the and_then function in the pipeline takes a little extra punctuation because Elixir needs to be told to convert function calls into function values, but that extra effort is offset by the fact that the transforming functions become simple: each just takes a value (and any extra parameters) and returns {ok, new_value} or {error, reason} .

đó, trả về một giá trị được bọc mới. Việc dùng hàm and_then trong pipeline tốn thêm chút dấu câu vì Elixir cần được bảo phải chuyển các lời gọi hàm thành các giá trị hàm, nhưng công sức thêm đó được bù lại bằng việc các hàm biến đổi trở nên đơn giản: mỗi hàm chỉ nhận một giá trị (và bất kỳ tham số phụ nào) rồi trả về {ok, new_value} hoặc {error, reason} .

Transformations Transform Programming — Biến đổi làm Biến đổi cả Cách Lập trình

Thinking of code as a series of (nested) transformations can be a liberating approach to programming. It takes a while to get used to, but once you've developed the habit you'll find your code becomes cleaner, your functions shorter, and your designs flatter.

Nghĩ về code như một chuỗi các biến đổi (lồng nhau) có thể là một cách tiếp cận giải phóng cho việc lập trình. Phải mất một thời gian mới quen, nhưng một khi đã tạo được thói quen, bạn sẽ thấy code của mình trở nên sạch hơn, hàm của bạn ngắn hơn, và thiết kế của bạn phẳng hơn.

Give it a try.

Hãy thử xem.

Related Sections Include — Các mục liên quan

Topic 8, The Essence of Good Design

Topic 8, Tinh túy của Thiết kế Tốt

Topic 17, **Shell** Games

Topic 17, Trò chơi Shell

Topic 26, How to Balance Resources

Topic 26, Cách Cân bằng Tài nguyên

Topic 28, **Decoupling**

Topic 28, Gỡ ghép nối

Exercises — Bài tập

Exercise 21 (possible answer)

Bài tập 21 (gợi ý đáp án)

Can you express the following **requirements** as a top-level transformation? That is, for each, identify the input and the output.

Bạn có thể diễn đạt các yêu cầu sau thành một biến đổi cấp cao nhất không? Tức là, với mỗi yêu cầu, hãy xác định đầu vào và đầu ra.

1. Shipping and sales tax are added to an order
2. Your application loads **configuration** information from a named file
3. Someone logs in to a web application

1. Phí vận chuyển và thuế bán hàng được cộng vào một đơn hàng
2. Ứng dụng của bạn nạp thông tin cấu hình từ một tệp có tên cho trước
3. Có người đăng nhập vào một ứng dụng web

Exercise 22 (possible answer)

Bài tập 22 (gợi ý đáp án)

You've identified the need to validate and convert an input field from a string into an integer between 18 and 150. The overall transformation is described by

Bạn đã xác định được nhu cầu cần kiểm tra hợp lệ và chuyển một trường đầu vào từ một chuỗi thành một số nguyên nằm giữa 18 và 150. Toàn bộ biến đổi được mô tả bằng

field contents as string $\rightarrow$ [validate & convert] $\rightarrow$ { :ok, value } | { :error, reason }

nội dung trường dưới dạng chuỗi $\rightarrow$ [kiểm tra & chuyển đổi] $\rightarrow$ { :ok, value } | { :error, reason }

Write the individual transformations that make up validate & convert .

Hãy viết các biến đổi riêng lẻ tạo nên kiểm tra & chuyển đổi .

Exercise 23 (possible answer)

Bài tập 23 (gợi ý đáp án)

In Language X Doesn't Have Pipelines we wrote:

Trong Ngôn ngữ X Không Có Pipeline ta đã viết:

```
const content = File.read(file_name); const lines = find_matching_lines(content, pattern) const result = truncate_lines(lines)
```

```
const content = File.read(file_name); const lines = find_matching_lines(content, pattern)
const result = truncate_lines(lines)
```

Many people write OO code by chaining together method calls, and might be tempted to write this as something like:

Nhiều người viết code OO bằng cách nối chuỗi các lời gọi phương thức, và có thể bị cám dỗ viết cái này thành đại loại như:

```
const result = content_of(file_name) .find_matching_lines(pattern) .truncate_lines()
```

```
const result = content_of(file_name) .find_matching_lines(pattern) .truncate_lines()
```

What's the difference between these two pieces of code? Which do you think we prefer?

Khác biệt giữa hai mẫu code này là gì? Bạn nghĩ chúng tôi thích cái nào hơn?

You wanted a banana but what you got was a gorilla holding the banana and the entire jungle. Joe Armstrong

Bạn muốn một quả chuối nhưng cái bạn nhận được lại là một con khỉ đột cầm quả chuối và cả khu rừng đi kèm. Joe Armstrong

Topic 31. Inheritance Tax — Topic 31. Thuế Kế Thừa

Do you program in an objectoriented language? Do you use **inheritance**?

Bạn có lập trình bằng ngôn ngữ hướng đối tượng không? Bạn có dùng kế thừa (inheritance) không?

If so, stop! It probably isn't what you want to do.

Nếu có, hãy dừng lại! Rất có thể đó không phải điều bạn thực sự muốn làm.

Let's see why.

Hãy cùng xem tại sao.

Some Background — Đôi Nét Bối Cảnh

Inheritance first appeared in Simula 67 in 1969. It was an elegant solution to the problem of queuing multiple types of events on the same list. The Simula approach was to use something called prefix classes . You could write something like this:

Kế thừa xuất hiện lần đầu trong Simula 67 vào năm 1969. Đó là một giải pháp tao nhã cho bài toán xếp hàng nhiều loại sự kiện khác nhau trên cùng một danh sách. Cách tiếp cận của Simula là dùng thứ gọi là lớp tiền tố (prefix classes). Bạn có thể viết kiểu như thế này:

link CLASS car; ... implementation of car

link CLASS car; ... cài đặt của car

link CLASS bicycle; ... implementation of bicycle

link CLASS bicycle; ... cài đặt của bicycle

The link is a prefix class that adds the functionality of linked lists. This lets you add both cars and bicycles to the list of things waiting at (say) a traffic light. In current terminology, link would be a parent class.

link là một lớp tiền tố bổ sung chức năng của danh sách liên kết. Nhờ đó bạn có thể thêm cả car lẫn bicycle vào danh sách những thứ đang chờ ở (chẳng hạn) một đèn giao thông. Theo thuật ngữ ngày nay, link sẽ là một lớp cha.

The mental model used by Simula programmers was that the instance data and implementation of class link was prepended to the implementation of classes car and bicycle . The link part was almost viewed as being a container that carried around cars and bicycles. This gave them a form of

polymorphism: cars and bicycles both implemented the link interface because they both contained the link code.

Mô hình tư duy mà các lập trình viên Simula dùng là: dữ liệu thể hiện và phần cài đặt của lớp link được gắn vào phía trước phần cài đặt của các lớp car và bicycle. Phần link gắn như được xem là một cái thùng chứa mang theo các car và bicycle. Điều này cho họ một dạng đa hình: cả car lẫn bicycle đều hiện thực giao diện link, vì cả hai đều chứa mã của link.

After Simula came Smalltalk. Alan Kay, one of the creators of Smalltalk, describes in a 2019 Quora answer [44] why Smalltalk has inheritance:

Sau Simula là Smalltalk. Alan Kay, một trong những người tạo ra Smalltalk, đã mô tả trong một câu trả lời trên Quora năm 2019 [44] vì sao Smalltalk có kế thừa:

So when I designed Smalltalk-72—and it was a lark for fun while thinking about Smalltalk-71—I thought it would be fun to use its Lisp-like dynamics to do experiments with “differential programming” (meaning: various ways to accomplish “this is like that except”).

Vậy nên khi tôi thiết kế Smalltalk-72 — và đó chỉ là một trò vui trong lúc nghĩ về Smalltalk-71 — tôi nghĩ sẽ thú vị nếu dùng tính động kiểu Lisp của nó để làm thí nghiệm với “lập trình theo sai biệt” (differential programming) (nghĩa là: đủ kiểu cách để diễn đạt “cái này giống cái kia, ngoại trừ...”).

This is subclassing purely for behavior.

Đây thuần túy là chuyển kế thừa lớp con (subclassing) vì hành vi.

These two styles of inheritance (which actually had a fair amount in common) developed over the following decades. The Simula approach, which suggested inheritance was a way of combining types, continued in languages such as C++ and Java. The Smalltalk school, where inheritance was a dynamic organization of behaviors, was seen in languages such as Ruby and JavaScript.

Hai phong cách kế thừa này (thực ra có khá nhiều điểm chung) phát triển qua mấy thập kỷ tiếp theo. Cách tiếp cận của Simula, vốn ngụ ý rằng kế thừa là một cách kết hợp các kiểu (type), tiếp tục trong những ngôn ngữ như C++ và Java. Trường phái Smalltalk, nơi kế thừa là một cách tổ chức hành vi một cách động, xuất hiện trong những ngôn ngữ như Ruby và JavaScript.

So, now we’re faced with a generation of OO developers who use inheritance for one of two reasons: they don’t like typing, or they like types.

Thế là giờ đây ta đối mặt với cả một thế hệ lập trình viên hướng đối tượng dùng kế thừa vì một trong hai lý do: hoặc họ lười gõ, hoặc họ thích kiểu (type).

Those who don’t like typing save their fingers by using

Những người lười gõ tiết kiệm mấy ngón tay của mình bằng cách

inheritance to add common functionality from a base class into child classes: class User and class Product are both subclasses of ActiveRecord::Base .

dùng kế thừa để đưa chức năng chung từ một lớp cơ sở vào các lớp con: class User và class Product đều là lớp con của ActiveRecord::Base.

Those who like types use inheritance to express the relationship between classes: a Car is-a-kind-of Vehicle .

Còn những người thích kiểu thì dùng kế thừa để diễn đạt quan hệ giữa các lớp: một Car là-một-loại-của Vehicle.

Unfortunately both kinds of inheritance have problems.

Đáng tiếc, cả hai kiểu kế thừa đều có vấn đề.

Problems Using Inheritance To Share Code — Những Vấn Đề Khi Dùng Kế Thừa Để Dùng Chung Mã

Inheritance is **coupling**. Not only is the child class coupled to the parent, the parent's parent, and so on, but the code that uses the child is also coupled to all the ancestors. Here's an example:

Kế thừa là sự ghép nối (coupling). Không chỉ lớp con bị ghép nối với lớp cha, rồi cha của cha, và cứ thế, mà cả đoạn mã sử dụng lớp con cũng bị ghép nối với tất cả các tổ tiên của nó. Đây là một ví dụ:

```
class Vehicle def initialize @speed = 0 end def stop @speed = 0 end def move_at(speed) @speed = speed end end
```

```
class Vehicle def initialize @speed = 0 end def stop @speed = 0 end def move_at(speed) @speed = speed end end
```

```
class Car < Vehicle def info "I'm car driving at #{ @speed }" end end
```

```
class Car < Vehicle def info "I'm car driving at #{ @speed }" end end
```

**top-level code my_ride = Car.new
my_ride.move_at(30) — mã ở tầng
top-level my_ride = Car.new
my_ride.move_at(30)**

When the top-level calls `my_car.move_at`, the method being invoked is in `Vehicle`, the parent of `Car`.

Khi mã top-level gọi `my_car.move_at`, phương thức thực sự được gọi lại nằm trong `Vehicle`, lớp cha của `Car`.

Now the developer in charge of `Vehicle` changes the API, so `move_at` becomes `set_velocity`, and the instance variable `@speed` becomes `@velocity`.

Giờ lập trình viên phụ trách `Vehicle` thay đổi API: `move_at` trở thành `set_velocity`, còn biến thể hiện `@speed` trở thành `@velocity`.

An API change is expected to break clients of `Vehicle` class. But the top-level is not: as far as it is concerned it is using a `Car`. What the `Car` class does in terms of implementation is not the concern of the top-level code, but it still breaks.

Một thay đổi API thì đương nhiên được dự liệu là sẽ làm hỏng những nơi dùng lớp `Vehicle`. Nhưng mã top-level đâu nằm trong số đó: theo cách nó hiểu thì nó đang dùng một `Car`. Lớp `Car` cài đặt ra sao không phải là chuyện mã top-level phải bận tâm, vậy mà nó vẫn hỏng.

Similarly the name of an instance variable is purely an **internal** implementation detail, but when `Vehicle` changes it also (silently) breaks `Car`.

Tương tự, tên của một biến thể hiện thuần túy là chi tiết cài đặt nội bộ, vậy mà khi `Vehicle` đổi nó thì `Car` cũng hỏng theo (và hỏng một cách âm thầm).

So much coupling.

Ghép nối ới là ghép nối.

Problems Using Inheritance to Build Types Some folks view inheritance as a way of defining new types. Their favorite design diagram shows class hierarchies. They view problems the way Victorian gentleman scientists viewed nature, as something to be broken down into categories.

Những Vấn Đề Khi Dùng Kế Thừa Để Định Kiểu Một số người xem kế thừa như một cách định nghĩa kiểu mới. Sơ đồ thiết kế ưa thích của họ là những cây phân cấp lớp. Họ nhìn các bài toán giống như mấy quý ông khoa học gia thời Victoria nhìn tự nhiên: như một thứ cần được chia nhỏ thành từng phạm trù.

Unfortunately, these diagrams soon grow into wall-covering monstrosities, layer-upon-layer added in order to express the smallest nuance of differentiation between classes. This added complexity can make the application more brittle, as changes can ripple up and down many layers.

Đáng tiếc, mấy sơ đồ này chẳng mấy chốc phình to thành những con quái vật phủ kín tường, lớp chồng lên lớp được thêm vào chỉ để diễn đạt khác biệt bé tí xiu giữa các lớp. Độ phức tạp tăng thêm này có thể khiến ứng dụng dễ vỡ hơn, vì một thay đổi có thể lan truyền lên xuống qua nhiều tầng.

Even worse, though, is the multiple inheritance issue. A Car may be a kind of Vehicle , but it can also be a kind of Asset , InsuredItem , LoanCollateral and so on. Modeling this correctly would need

Nhưng tệ hơn nữa là vấn đề đa kế thừa. Một Car có thể là một loại Vehicle, nhưng nó cũng có thể là một loại Asset, InsuredItem, LoanCollateral, v.v. Mô hình hóa chuyện này cho đúng thì cần đến

multiple inheritance.

đa kế thừa.

C++ gave multiple inheritance a bad name in the 1990s because of some questionable disambiguation semantics. As a result, many current OO languages don't offer it. So, even if you're happy with complex type trees, you won't be able to model your domain accurately anyway.

C++ làm cho đa kế thừa mang tiếng xấu hồi thập niên 1990 vì một vài ngữ nghĩa khử nhập nhằng đáng ngờ. Hệ quả là nhiều ngôn ngữ hướng đối tượng hiện nay không hề cung cấp nó. Vậy nên, kể cả khi bạn thấy hài lòng với những cây kiểu phức tạp, bạn vẫn sẽ không thể mô hình hóa miền nghiệp vụ của mình một cách chính xác.

Tip 51: Don't Pay Inheritance Tax

Tip 51: Đừng Đóng Thuế Kế Thừa

The Alternatives Are Better — Các Lựa Chọn Thay Thế Tốt Hơn

Let us suggest three techniques that mean you should never need to use inheritance again:

Hãy để chúng tôi gợi ý ba kỹ thuật giúp bạn không bao giờ còn cần dùng kế thừa nữa:

Interfaces and protocols

Giao diện và giao thức (interfaces and protocols)

Delegation

Ủy quyền (delegation)

Mixins and traits

Mixin và trait

Interfaces and Protocols

Giao Diện và Giao Thức

Most OO languages allow you to specify that a class implements one or more sets of behaviors. You could say, for example, that a Car class implements the Drivable behavior and the Locatable behavior. The syntax used for doing this varies: in Java, it might look like this:

Hầu hết các ngôn ngữ hướng đối tượng cho phép bạn khai báo rằng một lớp hiện thực một hay nhiều tập hành vi. Chẳng hạn, bạn có thể nói rằng lớp Car hiện thực hành vi Drivable và hành vi Locatable. Cú pháp để làm việc này thì mỗi ngôn ngữ mỗi khác; trong Java, nó có thể trông như thế này:

```
public class Car implements Drivable, Locatable {  
    public class Car implements Drivable, Locatable {  
// Code for class Car. This code must include // the functionality of both Drivable // and Locatable  
    // Mã cho lớp Car. Mã này phải bao gồm // chức năng của cả Drivable // lẫn Locatable  
}  
}
```

Drivable and Locatable are what Java calls interfaces ; other languages call them protocols , and some call them traits (although this is not what we'll be calling a trait later).

Drivable và Locatable chính là cái mà Java gọi là interface (giao diện); các ngôn ngữ khác gọi chúng là protocol (giao thức), và một số gọi là trait (dù đây không phải thứ mà lát nữa chúng tôi sẽ gọi là trait).

Interfaces are defined like this:

Giao diện được định nghĩa như thế này:

```
public interface Drivable { double getSpeed(); void stop(); }  
    public interface Drivable { double getSpeed(); void stop(); }  
public interface Locatable { Coordinate getLocation(); boolean locationIsValid(); }  
    public interface Locatable { Coordinate getLocation(); boolean locationIsValid(); }
```

These declarations create no code: they simply say that any class that implements Drivable must implement the two methods getSpeed and stop , and a class that's Locatable must implement getLocation and locationIsValid . This means that our previous class definition of Car will only be valid if it includes all four of these methods.

Những khai báo này chẳng sinh ra mã nào cả: chúng chỉ đơn giản nói rằng bất kỳ lớp nào hiện thực Drivable đều phải cài đặt hai phương thức getSpeed và stop, còn một lớp là Locatable thì phải cài đặt getLocation và locationIsValid. Nghĩa là định nghĩa lớp Car ở trên chỉ hợp lệ nếu nó có đủ cả bốn phương thức này.

What makes interfaces and protocols so powerful is that we can use them as types, and any class that implements the appropriate interface will be compatible with that type. If Car and Phone both implement Locatable , we could store both in a list of locatable items:

Điều làm cho giao diện và giao thức mạnh mẽ đến vậy là ta có thể dùng chúng như những kiểu, và bất kỳ lớp nào hiện thực đúng giao diện tương ứng sẽ tương thích với kiểu ấy. Nếu cả Car lẫn Phone đều hiện thực Locatable, ta có thể cất cả hai vào cùng một danh sách các phần tử locatable:

```
List items = new ArrayList<>();  
    List items = new ArrayList<>();  
items.add( new Car(...)); items.add( new Phone(...)); items.add( new Car(...)); // ...  
    items.add( new Car(...)); items.add( new Phone(...)); items.add( new Car(...)); // ...
```

We can then **process** that list, safe in the knowledge that every item has getLocation and locationIsValid :

Sau đó ta có thể xử lý danh sách ấy, yên tâm rằng mọi phần tử đều có getLocation và locationIsValid:

```
void printLocation(Locatable item) { if (item.locationIsValid() { print(item.getLocation().asString()); }  
    void printLocation(Locatable item) { if (item.locationIsValid() { print(item.getLocation().asString());  
    }  
// ...  
// ...  
items.forEach(printLocation);  
    items.forEach(printLocation);
```

Tip 52: Prefer Interfaces to Express Polymorphism

Tip 52: Ưu Tiên Giao Diện Để Diễn Đạt Tính Đa Hình

Interfaces and protocols give us polymorphism without inheritance.

Giao diện và giao thức cho ta tính đa hình mà không cần kế thừa.

Delegation Inheritance encourages developers to create classes whose objects have large numbers of methods. If a parent class has 20 methods, and the subclass wants to make use of just two of them, its objects will still have the other 18 just lying around and callable. The class has lost control of its interface. This is a

Ủy Quyền Kế thừa khuyến khích lập trình viên tạo ra những lớp mà đối tượng của chúng có hằng hà sa số phương thức. Nếu một lớp cha có 20 phương thức, và lớp con chỉ muốn dùng đúng hai trong số đó, thì các đối tượng của nó vẫn cứ kéo theo 18 phương thức kia, nằm chình ịch và gọi được. Lớp đã đánh mất quyền kiểm soát giao diện của chính nó. Đây là một

common problem—many persistence and UI frameworks insist that application components subclass some supplied base class:

vấn đề phổ biến — nhiều framework lưu trữ và giao diện người dùng cứ khăng khăng bắt các thành phần ứng dụng phải kế thừa từ một lớp cơ sở nào đó mà chúng cung cấp:

```
class Account < PersistenceBaseClass end
```

```
class Account < PersistenceBaseClass end
```

The Account class now carries all of the persistence class's API around with it. Instead, imagine an alternative using delegation, as in the following example:

Giờ thì lớp Account phải vác theo toàn bộ API của lớp lưu trữ đi khắp nơi. Thay vào đó, hãy hình dung một cách khác dùng ủy quyền, như ví dụ sau:

```
class Account def initialize(. .) @repo = Persister.for(self) end  
    class Account def initialize(. .) @repo = Persister.for(self) end  
def save @repo.save() end end  
    def save @repo.save() end end
```

We now expose none of the framework API to the clients of our Account class: that **decoupling** is now broken. But there's more. Now that we're no longer constrained by the API of the framework we're using, we're free to create the API we need. Yes, we could do that before, but we always ran the risk that the interface we wrote can be bypassed, and the persistence API used instead. Now we control everything.

Giờ ta chẳng phơi bày chút nào API của framework ra cho những nơi dùng lớp Account của mình: mối ghép nối đó nay đã bị cắt đứt. Nhưng chưa hết. Vì không còn bị trói buộc bởi API của framework đang dùng, ta được tự do tạo ra đúng cái API mà mình cần. Đúng là trước kia ta cũng làm được vậy, nhưng luôn có nguy cơ giao diện ta viết ra bị đi vòng qua, và người ta dùng thẳng API lưu trữ. Còn giờ thì ta kiểm soát mọi thứ.

Tip 53: Delegate to Services: Has-A Trumps Is-A

Tip 53: Hãy Ủy Quyền Cho Dịch Vụ: Has-A Thắng Is-A

In fact, we can take this a step further. Why should an Account have to know how to persist itself? Isn't its job to know and enforce the account business rules?

Thật ra, ta còn có thể đẩy chuyện này đi xa thêm một bước. Tại sao một Account lại phải biết cách tự lưu trữ chính nó? Chẳng phải việc của nó là biết và thực thi các quy tắc nghiệp vụ của tài khoản hay sao?

```
class Account # nothing but account stuff
```

```
    class Account # không gì ngoài chuyện tài khoản
```

```
end
```

```
    end
```

```
class AccountRecord # wraps an account with the ability # to be fetched and stored end
```

```
    class AccountRecord # bọc một account với khả năng # được lấy về và lưu trữ end
```

Now we're really decoupled, but it has come at a cost. We're having to write more code, and typically some of it will be boilerplate: it's likely that all our record classes will need a find method, for example.

Giờ thì ta thực sự đã gỡ ghép nối, nhưng cái giá phải trả là có. Ta phải viết nhiều mã hơn, và thường thì một phần trong đó sẽ là mã khuôn mẫu (boilerplate): chẳng hạn, nhiều khả năng mọi lớp record của ta đều cần một phương thức find.

Fortunately, that's what mixins and traits do for us.

May thay, đó chính là việc mà mixin và trait làm cho ta.

Mixins, Traits, Categories, Protocol Extensions, ... As an industry, we love to give things names. Quite often we'll give the same thing many names. More is better, right?

Mixin, Trait, Category, Protocol Extension, ... Trong ngành này, ta mê đặt tên cho mọi thứ. Khá thường xuyên ta đặt nhiều cái tên cho cùng một thứ. Nhiều thì tốt hơn, đúng không?

That's what we're dealing with when we look at mixins. The basic idea is simple: we want to be able to extend classes and objects with new functionality without using inheritance. So we create a set of these functions, give that set a name, and then somehow extend a class or object with them. At that point, you've created a new class or object that combines the capabilities of the original and all its mixins. In most cases, you'll be able to make this extension even if you don't have access to the source code of the class you're extending.

Đó chính là tình cảnh ta gặp phải khi nhìn vào mixin. Ý tưởng cơ bản thì đơn giản: ta muốn có thể mở rộng các lớp và đối tượng bằng chức năng mới mà không cần dùng kế thừa. Vậy nên ta tạo ra một tập hàm, đặt cho tập đó một cái tên, rồi bằng cách nào đó mở rộng một lớp hay đối tượng bằng chúng. Tới lúc ấy, bạn đã tạo ra một lớp hay đối tượng mới kết hợp năng lực của bản gốc và tất cả các mixin của nó. Trong hầu hết trường hợp, bạn sẽ làm được phép mở rộng này ngay cả khi không có quyền truy cập mã nguồn của lớp mà bạn đang mở rộng.

Now the implementation and name of this feature varies between languages. We'll tend to call them mixins here, but we really want you to think of this as a language-agnostic feature. The important thing is the capability that all these implementations have: merging functionality between existing things and new things .

Giờ thì cách cài đặt và tên gọi của tính năng này khác nhau tùy ngôn ngữ. Ở đây ta sẽ thiên về gọi chúng là mixin, nhưng ta thực sự muốn bạn nghĩ về nó như một tính năng không phụ thuộc ngôn ngữ. Điều quan trọng là năng lực mà tất cả các cách cài đặt này đều có: hợp nhất chức năng giữa thứ đã có sẵn và thứ mới.

As an example, let's go back to our AccountRecord example. As we left it, an AccountRecord needed to know about both accounts and about our persistence framework. It also needed to delegate all the methods in the persistence layer that it wanted to expose to the outside world.

Lấy ví dụ, hãy quay lại ví dụ AccountRecord của chúng ta. Ở chỗ ta bỏ dở, một AccountRecord cần biết cả về account lẫn về framework lưu trữ của ta. Nó còn phải ủy quyền lại tất cả những phương thức của tầng lưu trữ mà nó muốn phơi ra cho thế giới bên ngoài.

Mixins give us an alternative. First, we could write a mixin that implements (for example) two of three of the standard finder methods. We could then add them into AccountRecord as a mixin. And, as we write new classes for persisted things, we can add the mixin to them, too:

Mixin cho ta một lựa chọn khác. Trước hết, ta có thể viết một mixin hiện thực (chẳng hạn) hai trong ba phương thức tìm kiếm tiêu chuẩn. Rồi ta thêm chúng vào AccountRecord dưới dạng một mixin. Và khi viết những lớp mới cho các thứ cần lưu trữ, ta cũng có thể thêm mixin ấy vào chúng luôn:

```
mixin CommonFinders { def find(id) { ... } def findAll() { ... } end
```

```
mixin CommonFinders { def find(id) { ... } def findAll() { ... } end
```

```
class AccountRecord extends BasicRecord with CommonFinders class OrderRecord extends BasicRecord with CommonFinders
```

```
class AccountRecord extends BasicRecord with CommonFinders class OrderRecord extends BasicRecord with CommonFinders
```

We can take this a lot further. For example, we all know our business objects need validation code to prevent bad data from infiltrating our calculations. But exactly what do we mean by validation ?

Ta có thể đẩy chuyện này đi xa hơn nhiều. Ví dụ, ai cũng biết các đối tượng nghiệp vụ của mình cần mã kiểm tra hợp lệ (validation) để ngăn dữ liệu xấu len lỏi vào các phép tính. Nhưng chính xác thì ta hiểu validation là gì?

If we take an account, for example, there are probably many different layers of validation that could be applied:

Lấy ví dụ một tài khoản, có lẽ có nhiều tầng kiểm tra hợp lệ khác nhau có thể áp dụng:

Validating that a hashed password matches one entered by the user

Kiểm tra rằng mật khẩu đã băm khớp với mật khẩu người dùng nhập vào

Validating form data entered by the user when an account is created

Kiểm tra dữ liệu biểu mẫu mà người dùng nhập khi một tài khoản được tạo

Validating form data entered by an admin person updating the user details

Kiểm tra dữ liệu biểu mẫu mà một quản trị viên nhập khi cập nhật thông tin người dùng

Validating data added to the account by other system components

Kiểm tra dữ liệu được các thành phần khác của hệ thống thêm vào tài khoản

Validating data for consistency before it is persisted

Kiểm tra tính nhất quán của dữ liệu trước khi nó được lưu trữ

A common (and we believe less-than-ideal) approach is to bundle all the validations into a single class (the business object/persistence object) and then add flags to control which fire in which circumstances.

Một cách làm phổ biến (mà chúng tôi cho là chưa lý tưởng) là gom tất cả các kiểm tra hợp lệ vào một lớp duy nhất (đối tượng nghiệp vụ kiêm đối tượng lưu trữ) rồi thêm các cờ để điều khiển cái nào kích hoạt trong hoàn cảnh nào.

We think a better way is to use mixins to create specialized classes for appropriate situations:

Chúng tôi nghĩ cách hay hơn là dùng mixin để tạo ra các lớp chuyên biệt cho từng tình huống phù hợp:

class AccountForCustomer extends Account with AccountValidations,AccountCustomerValidations

class AccountForCustomer extends Account with AccountValidations,AccountCustomerValidations

class AccountForAdmin extends Account with AccountValidations,AccountAdminValidations

class AccountForAdmin extends Account with AccountValidations,AccountAdminValidations

Here, both derived classes include validations common to all account objects. The **customer** variant also includes validations appropriate for the customer-facing APIs, while the admin variant contained (the presumably less restrictive) admin validations.

Ở đây, cả hai lớp dẫn xuất đều bao gồm các kiểm tra hợp lệ chung cho mọi đối tượng tài khoản. Biến thể dành cho khách hàng còn bao gồm các kiểm tra phù hợp cho các API hướng tới khách hàng, trong khi biến thể dành cho admin chứa các kiểm tra dành cho admin (mà có lẽ ít ràng buộc hơn).

Now, by passing instances of AccountForCustomer or AccountForAdmin back and forth, our code automatically ensures the correct validation is applied.

Giờ đây, chỉ cần truyền qua lại các thể hiện của AccountForCustomer hay AccountForAdmin, mã của ta tự động bảo đảm đúng bộ kiểm tra hợp lệ được áp dụng.

Tip 54: Use Mixins to Share Functionality

Tip 54: Dùng Mixin Để Dùng Chung Chức Năng

Inheritance Is Rarely The Answer — Kế Thừa Hiếm Khi Là Câu Trả Lời

We've had a quick look at three alternatives to traditional class inheritance:

Ta đã lướt qua nhanh ba lựa chọn thay thế cho kế thừa lớp truyền thống:

Interfaces and protocols

Giao diện và giao thức

Delegation

Ủy quyền

Mixins and traits

Mixin và trait

Each of these methods may be better for you in different circumstances, depending on whether your goal is sharing type information, adding functionality, or sharing methods. As with anything in programming, aim to use the technique that best expresses your intent.

Mỗi phương pháp trong số này có thể tốt hơn cho bạn trong những hoàn cảnh khác nhau, tùy vào việc mục tiêu của bạn là dùng chung thông tin kiểu, bổ sung chức năng, hay dùng chung phương thức. Như với mọi thứ trong lập trình, hãy hướng tới việc dùng kỹ thuật diễn đạt ý định của bạn rõ ràng nhất.

And try not to drag the whole jungle along for the ride.

Và cố đừng kéo lê cả khu rừng theo cho chuyến đi.

Related Sections Include — Các mục liên quan

Topic 8, The Essence of Good Design

Topic 8, Tinh Túy Của Thiết Kế Tốt

Topic 10, **Orthogonality**

Topic 10, Tính Trực Giao

Topic 28, Decoupling

Topic 28, Gỡ Ghép Nối

Challenges — Thử thách

The next time you find yourself subclassing, take a minute to examine the options. Can you achieve what you want with interfaces, delegation, and/or mixins? Can you reduce coupling by doing so?

Lần tới khi bạn thấy mình đang viết lớp con, hãy dành một phút xem xét các lựa chọn. Bạn có thể đạt được điều mình muốn bằng giao diện, ủy quyền, và/hoặc mixin không? Làm vậy bạn có giảm được sự ghép nối không?

Let all your things have their places; let each part of your business have its time. Benjamin Franklin, *Thirteen Virtues*, autobiography

Hãy để mọi thứ của bạn có chỗ của nó; hãy để mỗi phần việc của bạn có thời gian của nó. Benjamin Franklin, *Mười Ba Đức Hạnh*, tự truyện

Topic 32. Configuration — Topic 32. Cấu hình

When code relies on values that may change after the application has gone live, keep those values external to the app. When your application will run in different environments, and potentially for different customers, keep the environment and customer specific values outside the app. In this way, you're parameterizing your application; the code adapts to the places it runs.

Khi mã lệ thuộc vào những giá trị có thể thay đổi sau khi ứng dụng đã chạy thật, hãy giữ những giá trị đó ở bên ngoài ứng dụng. Khi ứng dụng của bạn sẽ chạy trong nhiều môi trường khác nhau, và có thể là cho nhiều khách hàng khác nhau, hãy giữ các giá trị riêng theo môi trường và theo khách hàng ở bên ngoài ứng dụng. Làm vậy là bạn đang tham số hóa ứng dụng của mình; mã sẽ thích nghi với nơi nó chạy.

Parameterize Your App Using External — Tham số hóa ứng dụng bằng cấu hình bên ngoài

Tip 55: Configuration

Tip 55: Tham số hóa ứng dụng bằng cấu hình bên ngoài

Common things you will probably want to put in **configuration** data include:

Những thứ thường gặp mà bạn nhiều khả năng sẽ muốn đặt vào dữ liệu cấu hình (configuration) gồm có:

Credentials for external services (database, third party APIs, and so on)

Thông tin xác thực cho các dịch vụ bên ngoài (cơ sở dữ liệu, API của bên thứ ba, v.v.)

Logging levels and destinations

Mức độ và đích đến của log (logging)

Port, IP address, machine, and cluster names the app uses

Cổng, địa chỉ IP, tên máy và tên cụm (cluster) mà ứng dụng sử dụng

Environment-specific validation parameters

Các tham số kiểm định riêng theo môi trường

Externally set parameters, such as tax rates

Các tham số được đặt từ bên ngoài, chẳng hạn như thuế suất

Site-specific formatting details

Chi tiết định dạng riêng theo từng nơi triển khai

License keys

Khóa bản quyền (license key)

Basically, look for anything that you know will have to change that you can express outside your main body of code, and slap it into some configuration bucket.

Về cơ bản, hãy tìm bất cứ thứ gì mà bạn biết chắc rồi sẽ phải thay đổi và có thể diễn đạt được bên ngoài phần thân mã chính, rồi tống nó vào một cái thùng cấu hình nào đó.

Static Configuration — Cấu hình tĩnh

Many frameworks, and quite a few custom applications, keep configuration in either flat files or database tables. If the information is in flat files, the trend is to use some off-the-shelf plain-text format. Currently YAML and JSON are popular for this. Sometimes applications written in scripting languages use special purpose source-code files, dedicated to containing just configuration. If the information is structured, and is likely to be changed by the **customer** (sales tax rates, for example), it might be better to store it in a database table. And, of course, you can use both, splitting the configuration information according to use.

Nhiều framework, và khá nhiều ứng dụng tự viết, giữ cấu hình trong các tệp phẳng (flat file) hoặc các bảng cơ sở dữ liệu. Nếu thông tin nằm trong tệp phẳng, xu hướng hiện nay là dùng một định dạng văn bản thuần (plain text) có sẵn. Hiện tại YAML và JSON đang được ưa chuộng cho việc này. Đôi khi các ứng dụng viết bằng ngôn ngữ kịch bản (scripting language) dùng những tệp mã nguồn chuyên dụng, chỉ để chứa cấu hình mà thôi. Nếu thông tin có cấu trúc, và có khả năng bị khách hàng thay đổi (chẳng hạn thuế suất bán hàng), thì lưu nó trong một bảng cơ sở dữ liệu có lẽ tốt hơn. Và tất nhiên, bạn có thể dùng cả hai, chia thông tin cấu hình theo mục đích sử dụng.

Whatever form you use, the configuration is read into your application as a data structure, normally when the application starts. Commonly, this data structure is made global, the thinking being that this makes it easier for any part of the code to get to the values it holds.

Dù dùng dạng nào, cấu hình cũng được đọc vào ứng dụng dưới dạng một cấu trúc dữ liệu, thường là lúc ứng dụng khởi động. Phổ biến là người ta biến cấu trúc dữ liệu này thành toàn cục (global), với suy nghĩ rằng làm vậy sẽ giúp mọi phần của mã dễ dàng lấy được các giá trị mà nó chứa.

We prefer that you don't do that. Instead, wrap the configuration information behind a (thin) API. This decouples your code from the details of the representation of configuration.

Chúng tôi thì không muốn bạn làm thế. Thay vào đó, hãy bọc thông tin cấu hình sau một API (mỏng). Việc này gỡ ghép nối (decoupling) mã của bạn khỏi các chi tiết về cách biểu diễn cấu hình.

Configuration-As-A-Service — Cấu hình dưới dạng dịch vụ

While static configuration is common, we currently favor a

Tuy cấu hình tĩnh rất phổ biến, hiện tại chúng tôi nghiêng về một

different approach. We still want configuration data kept external to the application, but rather than in a flat file or database, we'd like to see it stored behind a service API. This has a number of benefits:

cách tiếp cận khác. Chúng tôi vẫn muốn giữ dữ liệu cấu hình ở bên ngoài ứng dụng, nhưng thay vì nằm trong một tệp phẳng hay cơ sở dữ liệu, chúng tôi muốn thấy nó được lưu phía sau một API dịch vụ. Cách này có một số lợi ích:

Multiple applications can share configuration information, with **authentication** and access control limiting what each can see

Nhiều ứng dụng có thể dùng chung thông tin cấu hình, với cơ chế xác thực và kiểm soát truy cập giới hạn những gì mỗi ứng dụng được thấy

Configuration changes can be made globally

Có thể thay đổi cấu hình trên phạm vi toàn cục

The configuration data can be maintained via a specialized UI

Dữ liệu cấu hình có thể được bảo trì qua một giao diện chuyên dụng

The configuration data becomes dynamic

Dữ liệu cấu hình trở nên động

That last point, that configuration should be dynamic, is critical as we move toward highly available applications. The idea that we should have to stop and restart an application to change a single parameter is hopelessly out of touch with modern realities. Using a configuration service, components of the application could register for notifications of updates to parameters they use, and the service could send them messages containing new values if and when they are changed.

Điểm cuối cùng đó — rằng cấu hình nên động — là cốt yếu khi chúng ta hướng tới các ứng dụng có tính sẵn sàng cao. Cái ý tưởng rằng ta phải dừng và khởi động lại cả một ứng dụng chỉ để đổi một tham số đã lạc hậu một cách vô vọng so với thực tế hiện đại. Khi dùng một dịch vụ cấu hình, các thành phần của ứng dụng có thể đăng ký nhận thông báo khi những tham số chúng dùng được cập nhật, và dịch vụ có thể gửi cho chúng các thông điệp chứa giá trị mới mỗi khi có thay đổi.

Whatever form it takes, configuration data drives the runtime behavior of an application. When configuration values change, there's no need to rebuild the code.

Dù mang hình thức nào, dữ liệu cấu hình cũng điều khiển hành vi lúc chạy của một ứng dụng. Khi các giá trị cấu hình thay đổi, không cần phải biên dịch lại mã.

Don'T Write Dodo-Code — Đừng viết mã kiểu chim dodo

Without external configuration, your code is not as adaptable or flexible as it could be. Is this a bad thing? Well, out here in the real world, species that don't adapt die.

Không có cấu hình bên ngoài, mã của bạn sẽ không thể thích nghi hay linh hoạt như đáng lẽ nó có thể. Điều đó có tệ không? À, ngoài thế giới thực này, loài nào không thích nghi thì chết.

The dodo didn't adapt to the presence of humans and their livestock on the island of Mauritius, and quickly became extinct.

Loài chim dodo đã không thích nghi được với sự xuất hiện của con người và gia súc của họ trên đảo Mauritius, và nhanh chóng tuyệt chủng.

[45]

[45]

[45] It was the first documented extinction of a species at the hand of man.

[45] Đó là vụ tuyệt chủng của một loài đầu tiên được ghi chép lại do bàn tay con người gây ra.

Don't let your project (or your career) go the way of the dodo.

Đừng để dự án của bạn (hay sự nghiệp của bạn) đi theo vết xe của loài dodo.

images/dodo.png

images/dodo.png

Related Sections Include — Các mục liên quan

Topic 9, **DRY**—The Evils of **Duplication**

Topic 9, DRY — Tội lỗi của sự trùng lặp

Topic 14, Domain Languages

Topic 14, Ngôn ngữ miền

Topic 16, The Power of **Plain Text**

Topic 16, Sức mạnh của văn bản thuần

Topic 28, **Decoupling**

Topic 28, Gỡ ghép nối

Don't Overdo It

Đừng lạm dụng

In the first edition of this book, we suggested using configuration instead of code in a similar fashion, but apparently should have been a little more specific in our instructions. Any advice can be taken to extremes or used inappropriately, so here are a few cautions:

Trong lần xuất bản đầu tiên của cuốn sách này, chúng tôi đã gợi ý dùng cấu hình thay cho mã theo kiểu tương tự, nhưng có vẻ lẽ ra phải hướng dẫn cụ thể hơn một chút. Bất kỳ lời khuyên nào cũng có thể bị đẩy đến mức cực đoan hoặc dùng sai chỗ, nên đây là vài điều cần thận trọng:

Don't overdo it. One early **client** of ours decided that every single field in their application should be configurable. As a result, it took weeks to make even the smallest change, as you had to implement both the field and all the admin code to save and edit it. They had some 40,000 configuration variables and a coding nightmare on their hands.

Đừng lạm dụng. Một khách hàng thời kỳ đầu của chúng tôi quyết định rằng mọi trường (field) trong ứng dụng của họ đều phải cấu hình được. Hậu quả là phải mất hàng tuần mới làm xong dù chỉ một thay đổi nhỏ nhất, vì bạn phải cài đặt cả bản thân cái trường đó lẫn toàn bộ mã quản trị để lưu và chỉnh sửa nó. Họ có chừng 40.000 biến cấu hình và một cơn ác mộng lập trình trong tay.

Don't push decisions to configuration out of laziness. If there's genuine debate about whether a feature should work this way or that, or if it should be the users' choice, try it out one way and get feedback on whether the decision was a good one.

Đừng đẩy quyết định sang cấu hình chỉ vì lười. Nếu thật sự còn tranh cãi xem một tính năng nên hoạt động theo cách này hay cách kia, hay liệu có nên để người dùng tự chọn, thì hãy thử làm theo một cách rồi thu thập phản hồi xem quyết định đó có ổn không.

Footnotes

Footnotes

[37] So it's not really a law. It's more like The Jolly Good Idea of Demeter.

[37] Vậy nên nó không thực sự là một định luật. Nó giống một Ý Tưởng Hay Tuyệt của Demeter thì đúng hơn.

[38] https://media.pragprog.com/articles/jan_03_enbug.pdf

[38] https://media.pragprog.com/articles/jan_03_enbug.pdf

[39] Yes, we know that Ruby already has this capability with its `at_exit` function.

[39] Vâng, chúng tôi biết Ruby vốn đã có sẵn khả năng này với hàm `at_exit` của nó.

[40] <https://media.pragprog.com/titles/tpp20/code/event/rxcommon/logger.js>

[40] <https://media.pragprog.com/titles/tpp20/code/event/rxcommon/logger.js>

[41] It seems that the first use of the characters `|>` as a pipe dates to 1994, in a discussion about the language Isabelle/ML, archived at <https://blogs.msdn.microsoft.com/dsyme/2011/05/17/archeological-semiotics-thebirth-of-the-pipeline-symbol-1994/>

[41] Có vẻ lần đầu tiên cặp ký tự `|>` được dùng làm pipe là vào năm 1994, trong một cuộc thảo luận về ngôn ngữ Isabelle/ML, được lưu trữ tại <https://blogs.msdn.microsoft.com/dsyme/2011/05/17/archeological-semiotics-thebirth-of-the-pipeline-symbol-1994/>

[42] We've taken a liberty here. Technically we do execute the following functions. We just don't execute the code in them.

[42] Ở đây chúng tôi đã nói hơi phóng khoáng một chút. Về mặt kỹ thuật thì chúng tôi có thực thi các hàm tiếp theo. Chúng tôi chỉ là không thực thi đoạn mã bên trong chúng thôi.

[43] In fact you could add such an operator to Elixir using its macro facility; an example of this is the Monad library in hex. You could also use Elixir's `with` construct, but then you lose much of the sense of writing transformations that you get with pipelines.

[43] Thực ra bạn có thể thêm một toán tử như vậy vào Elixir bằng cơ chế macro của nó; thư viện Monad trên hex là một ví dụ. Bạn cũng có thể dùng cấu trúc `with` của Elixir, nhưng khi đó bạn đánh mất phần lớn cái cảm giác đang viết các phép biến đổi mà pipeline mang lại.

[44] <https://www.quora.com/What-does-Alan-Kay-think-about-inheritance-in-objectoriented-programming>

[44] <https://www.quora.com/What-does-Alan-Kay-think-about-inheritance-in-objectoriented-programming>

[45] It didn't help that the settlers beat the placid (read: stupid) birds to death with clubs for sport.

[45] Cũng chẳng giúp ích gì khi đám di dân lấy gậy đập chết những con chim hiền lành (đọc là: ngốc nghếch) ấy để mua vui.

Copyright © 2020 Pearson Education, Inc.

Copyright © 2020 Pearson Education, Inc.

Concurrency — Xử lý đồng thời

Just so we're all on the same page, let's start with some definitions:

Để chúng ta cùng thống nhất một góc nhìn, hãy bắt đầu bằng vài định nghĩa:

Concurrency is when the execution of two or more pieces of code act as if they run at the same time. Parallelism is when they do run at the same time.

Xử lý đồng thời (concurrency) là khi việc thực thi hai hay nhiều đoạn mã diễn ra *như thể* chúng chạy cùng lúc. Còn song song (parallelism) là khi chúng *thực sự* chạy cùng lúc.

To have concurrency, you need to run code in an environment that can switch execution between different parts of your code when it is running. This is often implemented using things such as fibers, threads, and processes.

Để có được xử lý đồng thời, bạn cần chạy mã trong một môi trường có thể chuyển đổi việc thực thi qua lại giữa các phần khác nhau của mã khi nó đang chạy. Điều này thường được hiện thực bằng những thứ như fiber, thread (luồng) và process (tiến trình).

To have parallelism, you need hardware that can do two things at once. This might be multiple cores in a CPU, multiple CPUs in a computer, or multiple computers connected together.

Để có được tính song song, bạn cần phần cứng có thể làm hai việc cùng một lúc. Đó có thể là nhiều nhân (core) trong một CPU, nhiều CPU trong một máy tính, hay nhiều máy tính nối lại với nhau.

Everything Is Concurrent

Mọi Thứ Đều Đồng Thời

It's almost impossible to write code in a decent-sized system that doesn't have concurrent aspects to it. They may be explicit, or they may be buried inside a library. Concurrency is a requirement if you want your application to be able to deal with the real world, where things are **asynchronous**: users are interacting, data is being fetched, external services are being called, all at the same time. If you force this **process** to be serial, with one thing happening, then the next, and so on, your system feels sluggish and you're probably not taking full advantage of the power of the hardware on which it runs.

Gần như không thể nào viết mã cho một hệ thống cỡ khá khá mà lại không có khía cạnh đồng thời nào trong đó. Chúng có thể hiện rõ, hoặc cũng có thể nằm chôn sâu bên trong một thư viện. Xử lý đồng thời là một đòi hỏi nếu bạn muốn ứng dụng của mình có thể xoay xở với thế giới thực, nơi mọi thứ diễn ra bất đồng bộ: người dùng đang tương tác, dữ liệu đang được lấy về, các dịch vụ bên ngoài đang được gọi đến — tất cả cùng một lúc. Nếu bạn ép quy trình này phải tuần tự, làm xong một việc rồi mới đến việc kế tiếp, cứ thế, thì hệ thống của bạn sẽ ì ạch, và có lẽ bạn cũng chưa tận dụng hết sức mạnh của phần cứng đang chạy nó.

In this chapter we'll look at concurrency and parallelism.

Trong chương này, chúng ta sẽ xem xét cả xử lý đồng thời lẫn tính song song.

Developers often talk about **coupling** between chunks of code. They're referring to dependencies, and how those dependencies make things hard to change. But there's another form of coupling. **Temporal coupling** happens when your code imposes a sequence on things that is not required to solve the problem at hand. Do you depend on the “tick” coming before the “tock”? Not if you want to stay flexible. Does your code access multiple back-end services sequentially, one after the other? Not if you want to keep your customers. In **Topic 33, Breaking Temporal Coupling**, we'll look at ways of identifying this kind of temporal coupling.

Lập trình viên thường nói về sự ghép nối (coupling) giữa các khúc mã với nhau. Họ đang nói tới các phụ thuộc, và chuyện những phụ thuộc đó khiến mọi thứ khó thay đổi ra sao. Nhưng còn một dạng ghép nối khác nữa. Ghép nối thời gian (temporal coupling) xảy ra khi mã của bạn áp đặt một trình tự cho các sự việc, trong khi trình tự ấy chẳng cần thiết để giải quyết vấn đề trước mắt. Bạn có đang phụ thuộc vào việc tiếng “tích” phải đến trước tiếng “tắc” không? Đừng, nếu bạn muốn giữ được sự linh hoạt. Mã của bạn có truy cập nhiều dịch vụ back-end một cách tuần tự, cái này sau cái kia không? Đừng, nếu bạn muốn giữ chân khách hàng. Ở Mục 33, *Phá vỡ Ghép nối Thời gian*, chúng ta sẽ tìm cách nhận diện kiểu ghép nối thời gian này.

Why is writing concurrent and parallel code so difficult? One reason is that we learned to program using sequential systems, and our languages have features that are relatively safe when used sequentially but become a liability once two things can

Vì sao viết mã đồng thời và song song lại khó đến vậy? Một lý do là chúng ta học lập trình bằng các hệ thống tuần tự, và các ngôn ngữ của chúng ta có những tính năng tương đối an toàn khi dùng tuần tự, nhưng lại trở thành mối hiểm họa một khi có hai việc có thể

happen at the same time. One of the biggest culprits here is **shared state**. This doesn't just mean global variables: any time two or more chunks of code hold references to the same piece of mutable data, you have shared **state**. And Topic 34, **Shared State Is Incorrect State**. The section describes a number of workarounds for this, but ultimately they're all error prone.

xảy ra cùng lúc. Một trong những thủ phạm lớn nhất ở đây là trạng thái chia sẻ (shared state). Điều này không chỉ có nghĩa là biến toàn cục: bất cứ khi nào hai hay nhiều khúc mã cùng nắm giữ tham chiếu đến cùng một mẫu dữ liệu khả biến, là bạn đã có trạng thái chia sẻ. Và Mục 34, *Trạng thái Chia sẻ là Trạng thái Sai*. Mục đó mô tả một số cách lách vấn đề này, nhưng rốt cuộc chúng đều dễ sinh lỗi.

If that makes you feel sad, nil desperandum! There are better ways to construct concurrent applications. One of these is using the **actor** model, where independent processes, which share no data, communicate over channels using defined, simple, semantics. We talk about both the theory and practice of this approach in Topic 35, *Actors and Processes*.

Nếu điều đó làm bạn thấy buồn, thì *nil desperandum* — đừng tuyệt vọng! Có những cách tốt hơn để dựng nên các ứng dụng đồng thời. Một trong số đó là dùng mô hình actor, nơi các tiến trình độc lập, không chia sẻ dữ liệu, giao tiếp với nhau qua các kênh (channel) bằng những ngữ nghĩa đơn giản, đã định rõ. Chúng ta sẽ bàn cả lý thuyết lẫn thực hành của cách tiếp cận này ở Mục 35, *Actor và Tiến trình*.

Finally, we'll look at Topic 36, *Blackboards*. These are systems which act like a combination of an object store and a smart **publish/subscribe** broker. In their original form, they never really took off. But today we're seeing more and more implementations of middleware layers with **blackboard**-like

semantics. Used correctly, these types of systems offer a serious amount of **decoupling**.

Cuối cùng, chúng ta sẽ xem xét Mục 36, *Bảng đen*. Đây là những hệ thống hoạt động như sự kết hợp giữa một kho lưu đối tượng và một broker publish/subscribe thông minh. Ở hình hài nguyên thủy, chúng chưa bao giờ thực sự cất cánh. Nhưng ngày nay chúng ta đang thấy ngày càng nhiều hiện thực của các lớp middleware mang ngữ nghĩa kiểu bảng đen. Nếu dùng đúng cách, những loại hệ thống này mang lại một mức gỡ ghép nổi đáng kể.

Concurrent and parallel code used to be exotic. Now it is required.

Mã đồng thời và song song trước đây từng là thứ xa lạ, kỳ dị. Giờ thì nó là điều bắt buộc.

Topic 33. Breaking Temporal Coupling — Topic 33. Phá vỡ Ghép nối Thời gian

“What is temporal coupling all about?”, you may ask. It’s about time.

“Ghép nối thời gian rốt cuộc là về cái gì?”, có thể bạn sẽ hỏi vậy. Nó là về thời gian.

Time is an often ignored aspect of software architectures. The only time that preoccupies us is the time on the schedule, the time left until we ship—but this is not what we’re talking about here. Instead, we are talking about the role of time as a design element of the software itself. There are two aspects of time that are important to us: concurrency (things happening at the same time) and ordering (the relative positions of things in time).

Thời gian là một khía cạnh thường bị bỏ quên trong kiến trúc phần mềm. Thứ thời gian duy nhất khiến chúng ta bận tâm là thời gian trên lịch tiến độ, thời gian còn lại trước khi giao hàng — nhưng đó không phải là điều chúng ta đang nói tới ở đây. Thay vào đó, chúng ta đang nói về vai trò của thời gian như một yếu tố thiết kế của chính phần mềm. Có hai khía cạnh của thời gian quan trọng với chúng ta: tính đồng thời (những việc xảy ra cùng lúc) và thứ tự (vị trí tương đối của các sự việc trong dòng thời gian).

We don’t usually approach programming with either of these aspects in mind. When people first sit down to design an architecture or write a program, things tend to be linear. That’s the way most people think— do this and then always do that . But thinking this way leads to temporal coupling : coupling in time. Method A must always be called before method B ; only one report can be run at a time; you must wait for the screen to redraw before the button click is received. Tick must happen before tock.

Chúng ta thường không tiếp cận việc lập trình với một trong hai khía cạnh ấy trong đầu. Khi người ta lần đầu ngồi xuống thiết kế một kiến trúc hay viết một chương trình, mọi thứ có xu hướng tuyến tính. Đó là cách phần lớn mọi người suy nghĩ — làm việc này rồi thì luôn luôn làm việc kia. Nhưng nghĩ theo lối đó dẫn tới ghép nối thời gian (temporal coupling): ghép nối trong thời gian. Phương thức A luôn phải được gọi trước phương thức B; mỗi lần chỉ chạy được đúng một báo cáo; bạn phải chờ màn hình vẽ lại xong thì cú nhấp chuột mới được tiếp nhận. Tiếng “tích” phải xảy ra trước tiếng “tắc”.

This approach is not very flexible, and not very realistic.

Cách tiếp cận này không linh hoạt cho lắm, mà cũng chẳng thực tế cho lắm.

We need to allow for concurrency and to think about decoupling any time or order dependencies. In doing so, we can gain flexibility and reduce any time-based dependencies in many areas of development: workflow analysis, architecture, design,

Chúng ta cần chừa chỗ cho sự đồng thời và phải nghĩ tới việc gỡ bỏ mọi phụ thuộc về thời gian hay thứ tự. Khi làm vậy, chúng ta có thể giành được sự linh hoạt và giảm bớt các phụ thuộc dựa trên thời gian ở nhiều lĩnh vực phát triển: phân tích luồng công việc, kiến trúc, thiết kế,

and deployment. The result will be systems that are easier to reason about, that potentially respond faster and more reliably.

và triển khai. Kết quả sẽ là những hệ thống dễ suy luận hơn, có tiềm năng phản hồi nhanh hơn và đáng tin cậy hơn.

Looking For Concurrency — Tìm kiếm Sự Đồng thời

On many projects, we need to model and analyze the application workflows as part of the design. We'd like to find out what can happen at the same time, and what must happen in a strict order. One way to do this is to capture the workflow using a notation such as the activity diagram . [46]

Ở nhiều dự án, ta cần mô hình hóa và phân tích các luồng công việc của ứng dụng như một phần của thiết kế. Ta muốn tìm ra điều gì có thể xảy ra cùng lúc, và điều gì buộc phải xảy ra theo đúng một thứ tự nghiêm ngặt. Một cách để làm việc đó là ghi lại luồng công việc bằng một ký pháp như sơ đồ hoạt động (activity diagram). [46]

Tip 56: Analyze Workflow to Improve Concurrency

Tip 56: Phân Tích Luồng Công Việc Để Cải Thiện Tính Đồng Thời

An activity diagram consists of a set of actions drawn as rounded boxes. The arrow leaving an action leads to either another action (which can start once the first action completes) or to a thick line called a synchronization bar . Once all the actions leading into a synchronization bar are complete, you can then proceed along any arrows leaving the bar. An action with no arrows leading into it can be started at any time.

Một sơ đồ hoạt động gồm một tập các hành động được vẽ thành các ô bo góc. Mũi tên rời khỏi một hành động sẽ dẫn tới hoặc một hành động khác (có thể bắt đầu khi hành động đầu tiên hoàn tất), hoặc tới một đường kẻ đậm gọi là *thanh đồng bộ* (synchronization bar). Một khi tất cả các hành động dẫn vào một thanh đồng bộ đã hoàn tất, bạn mới có thể đi tiếp theo bất kỳ mũi tên nào rời khỏi thanh đó. Một hành động không có mũi tên nào dẫn vào thì có thể được bắt đầu vào bất cứ lúc nào.

You can use activity diagrams to maximize parallelism by identifying activities that could be performed in parallel, but aren't.

Bạn có thể dùng sơ đồ hoạt động để tối đa hóa tính song song bằng cách nhận diện những hoạt động *có thể* được làm song song, nhưng hiện chưa được làm vậy.

For instance, we may be writing the software for a robotic piña colada maker. We're told that the steps are:

Chẳng hạn, có thể chúng ta đang viết phần mềm cho một cái máy pha piña colada bằng robot. Người ta bảo chúng ta rằng các bước là:

1. Open blender 1. Close blender
 1. Mở máy xay 1. Đậy máy xay
2. Open piña colada mix 2. Liquefy for 1 minute
 2. Mở hỗn hợp piña colada 2. Xay nhuyễn trong 1 phút

3. Put mix in blender 3. Open blender
3. Cho hỗn hợp vào máy xay 3. Mở máy xay
4. Measure 1/2 cup white rum 4. Get glasses
4. Đong 1/2 cốc rượu rum trắng 4. Lấy ly
5. Pour in rum 5. Get pink umbrellas
5. Rót rum vào 5. Lấy mấy cây dù màu hồng
6. Add 2 cups of ice 6. Serve
6. Thêm 2 cốc đá 6. Phục vụ

However, a bartender would lose their job if they followed these steps, one by one, in order. Even though they describe these actions serially, many of them could be performed in parallel. We'll use the following activity diagram to capture and reason about potential concurrency.

Tuy nhiên, một bartender mà cứ làm theo các bước này, từng bước một, đúng thứ tự, thì sẽ bị mất việc. Dù chúng được mô tả một cách tuần tự, nhiều bước trong số đó hoàn toàn có thể làm song song. Chúng ta sẽ dùng sơ đồ hoạt động sau đây để ghi lại và suy luận về khả năng đồng thời tiềm tàng.

images/pina-colada.png

images/pina-colada.png

It can be eye-opening to see where the dependencies really exist. In this instance, the top-level tasks (1, 2, 4, 10, and 11) can all happen concurrently, up front. Tasks 3, 5, and 6 can happen in parallel later. If you were in a piña colada-making contest, these optimizations may make all the difference.

Thấy được các phụ thuộc thực sự nằm ở đâu có thể khiến ta mở mang tầm mắt. Trong trường hợp này, các tác vụ ở tầng trên cùng (1, 2, 4, 10 và 11) đều có thể diễn ra đồng thời, ngay từ đầu. Các tác vụ 3, 5 và 6 có thể diễn ra song song về sau. Nếu bạn đang thi đấu pha piña colada, những tối ưu này có thể tạo nên khác biệt một trời một vực.

Faster Formatting

Định dạng Nhanh hơn

This book is written in **plain text**. To build the version to be printed, or an ebook, or whatever, that text is fed through a **pipeline** of processors. Some look for particular constructs (bibliography citations, index entries, special markup for tips, and so on). Other processors operate on the document as a whole.

Cuốn sách này được viết bằng văn bản thuần (plain text). Để dựng nên phiên bản đem in, hay một cuốn ebook, hay gì gì đó, văn bản ấy được đưa qua một đường ống (pipeline) các bộ xử lý. Một số bộ tìm kiếm những cấu trúc cụ thể (trích dẫn thư mục, mục từ chỉ mục, đánh dấu đặc biệt cho các mẹo, v.v.). Các bộ xử lý khác thì thao tác trên toàn bộ tài liệu.

Many of the processors in the pipeline have to access external information (reading files, writing files, piping through external programs). All this relatively slow speed work gives us the opportunity to exploit concurrency: in fact each step in the pipeline executes concurrently, reading from the previous step and writing to the next.

Nhiều bộ xử lý trong đường ống phải truy cập thông tin bên ngoài (đọc tệp, ghi tệp, dẫn qua các chương trình ngoài). Tất cả những công việc tương đối chậm này cho chúng ta cơ

hội khai thác tính đồng thời: thực tế, mỗi bước trong đường ống thực thi đồng thời, đọc từ bước trước và ghi vào bước sau.

In addition, some parts of the process are relatively processor intensive. One of these is the conversion of mathematical formulae. For various historical reasons each equation can take up to 500ms to convert. To speed things up, we take advantage of parallelism. Because each formula is independent of the others, we convert each in its own parallel process and collect the results back into the book as they become available.

Thêm vào đó, một số phần của quy trình thì tương đối tốn bộ xử lý. Một trong số đó là việc chuyển đổi các công thức toán học. Vì nhiều lý do lịch sử khác nhau, mỗi phương trình có thể tốn tới 500ms để chuyển đổi. Để tăng tốc, chúng tôi tận dụng tính song song. Vì mỗi công thức độc lập với những công thức khác, chúng tôi chuyển đổi từng cái trong tiến trình song song riêng của nó, rồi thu kết quả về lại cuốn sách ngay khi chúng sẵn sàng.

As a result, the book builds much, much faster on multicore machines.

Nhờ vậy, cuốn sách được dựng nên nhanh hơn rất, rất nhiều trên các máy đa nhân.

(And, yes, we did indeed discover a number of concurrency errors in our pipeline along the way...)

(Và, đúng vậy, đúng là chúng tôi đã phát hiện ra kha khá lỗi đồng thời trong đường ống của mình trên đường đi....)

Opportunities For Concurrency — Cơ hội cho Sự Đồng thời

Activity diagrams show the potential areas of concurrency, but have nothing to say about whether these areas are worth exploiting. For example, in the piña colada example, a bartender would need five hands to be able to run all the potential initial tasks at once.

Sơ đồ hoạt động cho thấy các khu vực tiềm tàng có thể đồng thời, nhưng chẳng nói gì về chuyện những khu vực đó có đáng để khai thác hay không. Ví dụ, trong ví dụ piña colada, một bartender sẽ cần tới năm cái tay mới có thể chạy hết tất cả các tác vụ khởi đầu tiềm năng cùng một lúc.

And that's where the design part comes in. When we look at the activities, we realize that number 8, liquify, will take a minute. During that time, our bartender can get the glasses and umbrellas (activities 10 and 11) and probably still have time to serve another **customer**.

Và đó là lúc phần thiết kế bước vào. Khi nhìn vào các hoạt động, chúng ta nhận ra rằng tác vụ số 8, xay nhuyễn, sẽ tốn một phút. Trong khoảng thời gian đó, bartender của chúng ta có thể đi lấy ly và dù (tác vụ 10 và 11), và có lẽ vẫn còn dư thời gian để phục vụ một vị khách khác.

And that's what we're looking for when we're designing for concurrency. We're hoping to find activities that take time, but not time in our code. Querying a database, accessing an external

Và đó chính là thứ chúng ta đang tìm khi thiết kế cho tính đồng thời. Chúng ta hy vọng tìm được những hoạt động tốn thời gian, nhưng không tốn thời gian trong mã của ta. Truy vấn cơ sở dữ liệu, truy cập một dịch vụ

service, waiting for user input: all these things would normally stall our program until they complete. And these are all opportunities to do something more productive than the CPU equivalent of twiddling one's thumbs.

bên ngoài, chờ người dùng nhập liệu: tất cả những thứ này thông thường sẽ làm chương trình của ta đình trệ cho đến khi chúng hoàn tất. Và đây đều là những cơ hội để làm điều

gì đó hữu ích hơn là cái việc tương đương ở phía CPU của chuyện ngồi vờ ngón tay cái cho qua giờ.

Opportunities For Parallelism — Cơ hội cho Tính Song song

Remember the distinction: concurrency is a software mechanism, and parallelism is a hardware concern. If we have multiple processors, either locally or remotely, then if we can split work out among them we can reduce the overall time things take.

Hãy nhớ sự phân biệt này: xử lý đồng thời là một cơ chế phần mềm, còn tính song song là chuyện của phần cứng. Nếu ta có nhiều bộ xử lý, dù là cục bộ hay ở xa, thì nếu ta có thể chia nhỏ công việc ra cho chúng, ta có thể giảm tổng thời gian mọi thứ cần đến.

The ideal things to split this way are pieces of work that are relatively independent—where each can proceed without waiting for anything from the others. A common pattern is to take a large piece of work, split it into independent chunks, process each in parallel, then combine the results.

Những thứ lý tưởng để chia theo cách này là các phần công việc tương đối độc lập — nơi mỗi phần có thể tiến hành mà không phải chờ bất cứ thứ gì từ những phần khác. Một mẫu hình phổ biến là lấy một khối công việc lớn, chia nó thành các khúc độc lập, xử lý từng khúc song song, rồi gộp các kết quả lại.

An interesting example of this in practice is the way the compiler for the Elixir language works. When it starts, it splits the project it is building into modules, and compiles each in parallel. Sometimes a module depends on another, in which case its compilation pauses until the results of the other module's build become available. When the top-level module completes, it means that all dependencies have been compiled. The result is a speedy compilation that takes advantage of all the cores available.

Một ví dụ thú vị về điều này trong thực tế là cách bộ biên dịch của ngôn ngữ Elixir hoạt động. Khi khởi động, nó chia dự án đang được dựng ra thành các module, rồi biên dịch từng module song song. Đôi khi một module phụ thuộc vào một module khác, khi đó việc biên dịch nó tạm dừng cho đến khi có được kết quả dựng của module kia. Khi module ở tầng cao nhất hoàn tất, nghĩa là tất cả các phụ thuộc đã được biên dịch xong. Kết quả là một quá trình biên dịch nhanh chóng, tận dụng được toàn bộ các nhân hiện có.

Identifying Opportunities Is The Easy Part — Nhận diện Cơ hội Mới là Phần Dễ

Back to your applications. We've identified places where it will benefit from concurrency and parallelism. Now for the tricky part: how can we implement it safely. That's the topic of the rest

Quay lại với các ứng dụng của bạn. Chúng ta đã nhận diện được những chỗ mà nó sẽ hưởng lợi từ xử lý đồng thời và tính song song. Giờ tới phần hóc búa: làm sao hiện thực được điều đó một cách an toàn. Đó là chủ đề của phần còn lại

of the chapter.

của chương này.

Related Sections Include — Các mục liên quan

Topic 10, **Orthogonality**

Mục 10, Tính trực giao

Topic 26, How to Balance Resources

Mục 26, Cách Cân bằng Tài nguyên

Topic 28, Decoupling

Mục 28, Gỡ ghép nối

Topic 36, Blackboards

Mục 36, Bảng đen

Challenges — Thử thách

How many tasks do you perform in parallel when you get ready for work in the morning? Could you express this in a UML activity diagram? Can you find some way to get ready more quickly by increasing concurrency?

Bạn thực hiện bao nhiêu tác vụ song song khi chuẩn bị đi làm vào buổi sáng? Bạn có thể diễn đạt điều này bằng một sơ đồ hoạt động UML không? Bạn có tìm được cách nào để chuẩn bị nhanh hơn bằng cách tăng tính đồng thời không?

Topic 34. Shared State Is Incorrect State — Topic 34. Trạng Thái Chia Sẻ Là Trạng Thái Sai

You're in your favorite diner. You finish your main course, and ask your server if there's any apple pie left. He looks over his shoulder, sees one piece in the display case, and says yes. You order it and sigh contentedly.

Bạn đang ngồi trong quán ăn yêu thích của mình. Vừa ăn xong món chính, bạn hỏi người phục vụ xem còn miếng bánh táo nào không. Anh ta ngoái đầu nhìn lại, thấy còn đúng một miếng trong tủ kính, và trả lời là còn. Bạn gọi luôn rồi thở phào khoan khoái.

Meanwhile, on the other side of the restaurant, another **customer** asks their server the same question. She also looks, confirms there's a piece, and that customer orders.

Trong khi đó, ở phía bên kia nhà hàng, một thực khách khác hỏi người phục vụ của họ đúng câu hỏi ấy. Cô phục vụ cũng nhìn vào tủ, xác nhận là còn một miếng, và vị khách kia cũng đặt món.

One of the customers is going to be disappointed.

Một trong hai vị khách sắp phải thất vọng.

Swap the display case for a joint bank account, and turn the waitstaff into point-of-sale devices. You and your partner both decide to buy a new phone at the same time, but there's only enough in the account for one. Someone—the bank, the store, or you—is going to be very unhappy.

Giờ hãy thay cái tủ kính bằng một tài khoản ngân hàng chung, và biến những người phục vụ thành các thiết bị thanh toán điểm bán hàng (point-of-sale). Bạn và người bạn đời cùng quyết định mua một chiếc điện thoại mới vào đúng cùng một lúc, nhưng trong tài khoản chỉ đủ tiền cho một chiếc. Một ai đó — ngân hàng, cửa hàng, hay chính bạn — sắp sửa rất bực bội.

Tip 57: Shared State** Is Incorrect State**

Tip 57: Trạng Thái Chia Sẻ Là Trạng Thái Sai

The problem is the shared **state**. Each server in the restaurant looked into the display case without regard for the other. Each point-of-sale device looked at an account balance without regard for the other.

Vấn đề nằm ở trạng thái chia sẻ (shared state). Mỗi người phục vụ trong nhà hàng nhìn vào tủ kính mà chẳng đoái hoài gì đến người kia. Mỗi thiết bị thanh toán nhìn vào số dư tài khoản mà chẳng đoái hoài gì đến thiết bị kia.

Nonatomic Updates — Cập nhật không nguyên tử

Let's look at our diner example as if it were code:

Hãy nhìn ví dụ quán ăn của chúng ta như thể nó là một đoạn mã:

images/pie_case.png

images/pie_case.png

The two waiters operate concurrently (and, in real life, in parallel). Let's look at their code:

Hai người phục vụ làm việc đồng thời (concurrently) (và, trong đời thực, là song song).

Hãy xem đoạn mã của họ:

```
if display_case.pie_count > 0 promise_pie_to_customer() display_case.take_pie() give_pie_to_customer()
end
```

```
if display_case.pie_count > 0 promise_pie_to_customer() display_case.take_pie() give_pie_to_customer()
end
```

Waiter 1 gets the current pie count, and finds that it is one. He promises the pie to the customer. But at that point, waiter 2 runs. She also sees the pie count is one and makes the same promise to her customer. One of the two then grabs the last piece of pie, and the other waiter enters some kind of error state (which probably involves much grovelling).

Người phục vụ 1 lấy số lượng bánh hiện tại và thấy là một. Anh ta hứa miếng bánh đó cho khách của mình. Nhưng ngay lúc đó, người phục vụ 2 cũng chạy. Cô ấy cũng thấy số lượng bánh là một và cũng hứa hẹn y như vậy với khách của mình. Rồi một trong hai người vớ lấy miếng bánh cuối cùng, còn người kia rơi vào trạng thái lỗi nào đó (mà chắc là kèm theo rất nhiều lời xin lỗi rồi rít).

The problem here is not that two processes can write to the same memory. The problem is that neither **process** can guarantee that its view of that memory is consistent. Effectively, when a waiter executes `display_case.pie_count()`, they copy the value from the display case into their own memory. If the value in the display case changes, their memory (which they are using to make decisions) is now out of date.

Vấn đề ở đây không phải là hai tiến trình có thể cùng ghi vào một vùng nhớ. Vấn đề là chẳng tiến trình nào bảo đảm được rằng cái nhìn của mình về vùng nhớ ấy là nhất quán. Trên thực tế, khi một người phục vụ thực thi `display_case.pie_count()`, họ sao chép giá trị từ tủ bánh vào bộ nhớ của riêng mình. Nếu giá trị trong tủ bánh thay đổi, thì bộ nhớ của họ (thứ họ đang dùng để ra quyết định) đã lỗi thời mất rồi.

This is all because the fetching and then updating the pie count is not an **atomic operation**: the underlying value can change in the middle.

Tất cả là vì việc lấy ra rồi cập nhật số lượng bánh không phải là một thao tác nguyên tử (atomic): giá trị bên dưới có thể thay đổi ngay giữa chừng.

So how can we make it atomic?

Vậy làm sao để biến nó thành nguyên tử?

Semaphores and Other Forms of **Mutual Exclusion** A **semaphore** is simply a thing that only one person can own at a time. You can create a semaphore and then use it to control access to some other resource. In our example, we could create a semaphore to control access to the pie case, and adopt the convention that anyone who wants to update the pie case contents can only do so if they are holding that semaphore.

Cờ hiệu (semaphore) và các dạng loại trừ tương hỗ khác Một cờ hiệu đơn giản là một thứ mà tại mỗi thời điểm chỉ một người được sở hữu. Bạn có thể tạo ra một cờ hiệu rồi dùng nó để kiểm soát quyền truy cập vào một tài nguyên nào đó. Trong ví dụ của chúng ta, ta có thể tạo một cờ hiệu để kiểm soát quyền truy cập vào tủ bánh, và đặt ra quy ước rằng ai muốn cập nhật nội dung tủ bánh thì chỉ được làm khi đang giữ cờ hiệu đó.

Say the diner decides to fix the pie problem with a physical semaphore. They place a plastic Leprechaun on the pie case. Before any waiter can sell a pie, they have to be holding the Leprechaun in their hand. Once their order has been completed (which means delivering the pie to the table) they can return the Leprechaun to its place guarding the treasure of the pies, ready to mediate the next order.

Giả sử quán quyết định giải quyết vấn đề bánh trái bằng một cờ hiệu vật lý. Họ đặt một chú Leprechaun bằng nhựa lên trên tủ bánh. Trước khi bất kỳ người phục vụ nào bán bánh, họ phải cầm chú Leprechaun trong tay. Khi đơn hàng đã hoàn tất (nghĩa là bánh đã được mang ra bàn) họ trả chú Leprechaun về chỗ cũ để canh giữ kho báu bánh trái, sẵn sàng điều phối đơn hàng kế tiếp.

Let's look at this in code. Classically, the operation to grab the semaphore was called P , and the operation to release it was called V . [47] Today we use terms such as lock/unlock , claim/release , and so on.

Hãy nhìn chuyện này dưới dạng mã. Theo lối cổ điển, thao tác chiếm cờ hiệu được gọi là P, còn thao tác giải phóng nó gọi là V. [47] Ngày nay ta dùng những thuật ngữ như lock/unlock, claim/release, v.v.

```
case_semaphore.lock()
  case_semaphore.lock()
if display_case.pie_count > 0 promise_pie_to_customer() display_case.take_pie() give_pie_to_customer()
end
  if display_case.pie_count > 0 promise_pie_to_customer() display_case.take_pie() give_pie_to_customer()
end
case_semaphore.unlock()
  case_semaphore.unlock()
```

This code assumes that a semaphore has already been created and stored in the variable `case_semaphore` .

Đoạn mã này giả định rằng một cờ hiệu đã được tạo sẵn từ trước và lưu trong biến `case_semaphore`.

Let's assume both waiters execute the code at the same time. They both try to lock the semaphore, but only one succeeds. The one that gets the semaphore continues to run as normal. The one that doesn't get the semaphore is suspended until the semaphore becomes available (the waiter waits...). When the first waiter completes the order they unlock the semaphore and the second waiter continues running. They now see there's no pie in the case, and apologize to the customer.

Giả sử cả hai người phục vụ cùng thực thi đoạn mã một lúc. Cả hai đều cố khóa cờ hiệu, nhưng chỉ một người thành công. Người chiếm được cờ hiệu tiếp tục chạy bình thường. Người không chiếm được thì bị tạm dừng cho đến khi cờ hiệu rảnh trở lại (người phục vụ phải chờ...). Khi người phục vụ đầu tiên hoàn thành đơn hàng, họ mở khóa cờ hiệu và người phục vụ thứ hai chạy tiếp. Lúc này cô ấy thấy trong tủ không còn bánh nữa, và đành xin lỗi khách.

There are some problems with this approach. Probably the most significant is that it only works because everyone who accesses the pie case agrees on the convention of using the semaphore. If someone forgets (that is, some developer writes code that doesn't follow the convention) then we're back in chaos.

Cách làm này có vài vấn đề. Vấn đề lớn nhất có lẽ là nó chỉ hoạt động vì tất cả mọi người truy cập tủ bánh đều đồng thuận với quy ước dùng cờ hiệu. Nếu ai đó quên (tức là có một lập trình viên nào đó viết mã không tuân theo quy ước) thì ta lại quay về cảnh hỗn loạn.

Make the Resource Transactional The current design is poor because it delegates responsibility for protecting access to the pie case to the people who use it. Let's change it to centralize that control. To do this, we have to change the API so that waiters can check the count and also take a slice of pie in a single call:

Biến tài nguyên thành giao dịch Thiết kế hiện tại rất tệ vì nó giao phó trách nhiệm bảo vệ quyền truy cập tủ bánh cho chính những người sử dụng nó. Hãy thay đổi để tập trung hóa việc kiểm soát ấy. Để làm được, ta phải thay đổi API sao cho người phục vụ có thể kiểm tra số lượng và lấy luôn một miếng bánh chỉ trong một lệnh gọi duy nhất:

```
slice = display_case.get_pie_if_available() if slice give_pie_to_customer() end
```

```
slice = display_case.get_pie_if_available() if slice give_pie_to_customer() end
```

To make this work, we need to write a method that runs as part of the display case itself:

Để làm được điều này, ta cần viết một phương thức chạy như một phần của chính cái tủ kính:

```
def get_pie_if_available() ##### if @slices.size > 0 # update_sales_data( :pie ) # return @slices.shift # else # incorrect code! false # end # end #####
```

```
def get_pie_if_available() ##### if @slices.size > 0 # update_sales_data( :pie ) # return @slices.shift # else # mã sai! false # end # end #####
```

This code illustrates a common misconception. We've moved the resource access into a central place, but our method can still be called from multiple concurrent threads, so we still need to protect it with a semaphore:

Đoạn mã này minh họa một ngộ nhận phổ biến. Ta đã dời việc truy cập tài nguyên vào một chỗ tập trung, nhưng phương thức của ta vẫn có thể bị gọi từ nhiều luồng đồng thời, nên ta vẫn cần bảo vệ nó bằng một cờ hiệu:

```
def get_pie_if_available() @case_semaphore.lock()
```

```
def get_pie_if_available() @case_semaphore.lock()
```

```

if @slices.size > 0 update_sales_data( :pie ) return @slices.shift else false end
  if @slices.size > 0 update_sales_data( :pie ) return @slices.shift else false end
@case_semaphore.unlock() end
  @case_semaphore.unlock() end

```

Even this code might not be correct. If `update_sales_data` raises an **exception**, the semaphore will never get unlocked, and all future access to the pie case will hang indefinitely. We need to handle this:

Ngay cả đoạn mã này cũng có thể chưa đúng. Nếu `update_sales_data` ném ra một ngoại lệ, cờ hiệu sẽ không bao giờ được mở khóa, và mọi lần truy cập tủ bánh về sau sẽ treo vô thời hạn. Ta cần xử lý chuyện này:

```

def get_pie_if_available() @case_semaphore.lock()
  def get_pie_if_available() @case_semaphore.lock()
try { if @slices.size > 0 update_sales_data( :pie ) return @slices.shift else false
  try { if @slices.size > 0 update_sales_data( :pie ) return @slices.shift else false
end } ensure { @case_semaphore.unlock() } end
  end } ensure { @case_semaphore.unlock() } end

```

Because this is such a common mistake, many languages provide libraries that handle this for you:

Vì đây là một lỗi quá phổ biến, nhiều ngôn ngữ cung cấp sẵn thư viện lo việc này hộ bạn:

```

def get_pie_if_available() @case_semaphore.protect() { if @slices.size > 0 update_sales_data( :pie )
return @slices.shift else false end } end

def get_pie_if_available() @case_semaphore.protect() { if @slices.size > 0 update_sales_data(
:pie ) return @slices.shift else false end } end

```

Multiple Resource Transactions — Giao dịch trên nhiều tài nguyên

Our diner just installed an ice cream freezer. If a customer orders pie à la mode , the waiter will need to check that both pie and ice cream are available.

Quán ăn của chúng ta vừa lắp một tủ đông kem. Nếu một khách gọi bánh táo ăn kèm kem (pie à la mode), người phục vụ sẽ phải kiểm tra xem có sẵn cả bánh lẫn kem hay không.

We could change the waiter code to something like:

Ta có thể đổi đoạn mã của người phục vụ thành đại loại như thế này:

```

slice = display_case.get_pie_if_available() scoop = freezer.get_ice_cream_if_available()
  slice = display_case.get_pie_if_available() scoop = freezer.get_ice_cream_if_available()
if slice && scoop give_order_to_customer() end
  if slice && scoop give_order_to_customer() end

```

This won't work, though. What happens if we claim a slice of pie, but when we try to get a scoop of ice cream we find out there isn't any? We're now left holding some pie that we can't do anything with (because our customer must have ice cream). And

Tuy nhiên cách này sẽ không ổn. Chuyện gì xảy ra nếu ta chiếm được một miếng bánh, nhưng khi định lấy một viên kem thì lại phát hiện không còn kem? Giờ ta đang cầm một miếng bánh mà chẳng làm gì được với nó (vì khách của ta nhất định phải có kem). Và

the fact we're holding the pie means it isn't in the case, so it isn't available to some other customer who (being a purist) doesn't want ice cream with it.

việc ta đang cầm miếng bánh đồng nghĩa nó không còn nằm trong tủ, nên nó cũng không sẵn cho một vị khách khác nào đó (kiểu người thuần túy) chỉ muốn ăn bánh mà không cần kem.

We could fix this by adding a method to the case that lets us return a slice of pie. We'll need to add exception handling to ensure we don't keep resources if something fails:

Ta có thể khắc phục điều này bằng cách thêm vào tủ kính một phương thức cho phép ta trả lại miếng bánh. Ta sẽ cần thêm xử lý ngoại lệ để bảo đảm không giữ khư khư tài nguyên nếu có gì đó hỏng:

```
slice = display_case.get_pie_if_available()
```

```
slice = display_case.get_pie_if_available()
```

```
if slice try { scoop = freezer.get_ice_cream_if_available() if scoop try { give_order_to_customer() } rescue { freezer.give_back(scoop) } end } rescue { display_case.give_back(slice) } end
```

```
if slice try { scoop = freezer.get_ice_cream_if_available() if scoop try { give_order_to_customer() } rescue { freezer.give_back(scoop) } end } rescue { display_case.give_back(slice) } end
```

Again, this is less than ideal. The code is now really ugly: working out what it actually does is difficult: the business logic is buried in all the housekeeping.

Một lần nữa, cách này vẫn chưa lý tưởng. Đoạn mã giờ thực sự xấu xí: hiểu được nó rốt cuộc làm gì là chuyện khó khăn — logic nghiệp vụ bị chôn vùi giữa đồng việc dọn dẹp lặt vặt.

Previously we fixed this by moving the resource handling code into the resource itself. Here, though, we have two resources. Should we put the code in the display case or the freezer?

Trước đó ta đã sửa chuyện này bằng cách chuyển phần mã quản lý tài nguyên vào trong chính tài nguyên. Nhưng ở đây ta có tới hai tài nguyên. Ta nên đặt phần mã vào tủ kính hay tủ đông?

We think the answer is “no” to both options. The **pragmatic** approach would be to say that “apple pie à la mode” is its own resource. We'd move this code into a new module, and then the

Chúng tôi cho rằng câu trả lời cho cả hai lựa chọn đều là “không”. Cách tiếp cận thực dụng sẽ là coi “bánh táo ăn kèm kem” như một tài nguyên riêng của nó. Ta sẽ chuyển phần mã này vào một module mới, rồi khi đó

client could just say “get me apple pie with ice cream” and it either succeeds or fails.

phía client chỉ cần nói “lấy cho tôi bánh táo kèm kem” và việc đó hoặc thành công hoặc thất bại.

Of course, in the real world there are likely to be many composite dishes like this, and you wouldn't want to write new modules for each. Instead, you'd probably want some kind of menu item which contained references to its components, and then have a generic `get_menu_item` method that does the resource dance with each.

Tất nhiên, ngoài đời thực nhiều khả năng sẽ có cả đồng món ghép kiểu này, và bạn chẳng muốn viết một module mới cho từng món. Thay vào đó, có lẽ bạn sẽ muốn một dạng món trong thực đơn (menu item) chứa các tham chiếu tới những thành phần của nó, rồi có một phương thức tổng quát `get_menu_item` lo màn khiêu vũ tài nguyên với từng thành phần.

Non-Transactional Updates — Cập nhật phi giao dịch

A lot of attention is given to shared memory as a source of **concurrency** problems, but in fact the problems can pop up anywhere where your application code shares mutable resources: files, databases, external services, and so on. Whenever two or more instances of your code can access some resource at the same time, you're looking at a potential problem.

Người ta dành rất nhiều sự chú ý cho bộ nhớ chia sẻ như một nguồn gốc của các vấn đề về xử lý đồng thời, nhưng thực ra vấn đề có thể nảy sinh ở bất cứ đâu mà mã ứng dụng của bạn chia sẻ những tài nguyên có thể thay đổi: tệp tin, cơ sở dữ liệu, dịch vụ ngoài, v.v. Bất cứ khi nào có hai hay nhiều thể hiện của mã cùng truy cập một tài nguyên vào cùng một lúc, là bạn đang đối mặt với một vấn đề tiềm tàng.

Sometimes, the resource isn't all that obvious. While writing this edition of the book we updated the toolchain to do more work in parallel using threads. This caused the build to fail, but in bizarre ways and random places. A common thread through all the errors was that files or directories could not be found, even though they were really in exactly the right place.

Đôi khi, tài nguyên ấy chẳng hề hiển nhiên. Trong lúc viết phiên bản này của cuốn sách, chúng tôi đã cập nhật toolchain để làm nhiều việc song song hơn bằng cách dùng các luồng (threads). Việc này khiến quá trình build thất bại, nhưng theo những cách kỳ quái và ở những chỗ ngẫu nhiên. Một điểm chung xuyên suốt mọi lỗi là không tìm thấy tệp hay thư mục nào đó, mặc dù chúng thực sự nằm đúng chỗ.

We tracked this down to a couple of places in the code which temporarily changed the current directory. In the nonparallel version, the fact that this code restored the directory back was good enough. But in the parallel version, one thread would change the directory and then, while in that directory, another thread would start running. That thread would expect to be in the original directory, but because the current directory is

Chúng tôi lần ra nguyên nhân ở vài chỗ trong mã có tạm thời đổi thư mục hiện hành. Ở phiên bản không song song, việc đoạn mã này khôi phục lại thư mục cũ là đủ tốt. Nhưng ở phiên bản song song, một luồng sẽ đổi thư mục rồi, ngay khi đang ở trong thư mục đó, một luồng khác bắt đầu chạy. Luồng đó mong rằng mình đang ở thư mục ban đầu, nhưng vì thư mục hiện hành được

shared between threads, that wasn't the case.

chia sẻ giữa các luồng, nên thực tế không phải vậy.

The nature of this problem prompts another **tip**:

Bản chất của vấn đề này gợi ra một mẹo nữa:

Tip 58: Random Failures Are Often Concurrency Issues

Tip 58: Lỗi Ngẫu Nhiên Thường Là Vấn Đề Xử Lý Đồng Thời

Other Kinds Of Exclusive Access — Các dạng truy cập độc quyền khác

Most languages have library support for some kind of exclusive access to shared resources. They may call it mutexes (for mutual exclusion), monitors, or semaphores. These are all implemented as libraries.

Hầu hết các ngôn ngữ đều có thư viện hỗ trợ một dạng truy cập độc quyền nào đó vào tài nguyên chia sẻ. Chúng có thể gọi đó là mutex (loại trừ tương hỗ), monitor, hay semaphore (cờ hiệu). Tất cả đều được hiện thực dưới dạng thư viện.

However, some languages have concurrency support built into the language itself. Rust, for example, enforces the concept of data ownership; only one variable or parameter can hold a reference to any particular piece of mutable data at a time.

Tuy nhiên, một số ngôn ngữ tích hợp sẵn hỗ trợ xử lý đồng thời ngay trong bản thân ngôn ngữ. Rust, chẳng hạn, áp đặt khái niệm quyền sở hữu dữ liệu; tại mỗi thời điểm chỉ một biến hay một tham số được phép giữ tham chiếu tới một mẫu dữ liệu có thể thay đổi cụ thể nào đó.

You could also argue that functional languages, with their tendency to make all data immutable, make concurrency simpler. However, they still face the same **challenges**, because at some point they are forced to step into the real, mutable world.

Bạn cũng có thể lập luận rằng các ngôn ngữ hàm (functional), với xu hướng khiến mọi dữ liệu trở nên bất biến, làm cho việc xử lý đồng thời đơn giản hơn. Tuy vậy, chúng vẫn đối mặt với chính những thách thức ấy, bởi đến một lúc nào đó chúng buộc phải bước vào thế giới thực, nơi mọi thứ đều có thể thay đổi.

Doctor, It Hurts... — Bác sĩ ơi, đau quá...

If you take nothing else away from this section, take this: concurrency in a shared resource environment is difficult, and managing it yourself is fraught with challenges.

Nếu bạn chẳng rút ra được gì khác từ mục này, thì hãy nhớ lấy điều này: xử lý đồng thời trong một môi trường tài nguyên chia sẻ là chuyện khó, và tự mình quản lý nó thì đầy rẫy thách thức.

Which is why we're recommending the punchline to the old joke:

Đó là lý do chúng tôi muốn giới thiệu phần kết của câu chuyện cười xưa cũ:

Doctor, it hurts when I do this.

Bác sĩ ơi, tôi cứ làm thế này là đau.

Then don't do that.

Vậy thì đừng làm thế nữa.

The next couple of sections suggest alternative ways of getting the benefits of concurrency without the pain.

Vài mục tiếp theo sẽ gợi ý những cách khác để gặt hái lợi ích của xử lý đồng thời mà không phải chịu đau đớn.

Related Sections Include — Các mục liên quan

Topic 10, Orthogonality

Topic 10, Tính trực giao

Topic 28, **Decoupling**

Topic 28, Gỡ ghép nối

Topic 38, **Programming by Coincidence**

Topic 38, Lập trình theo may rủi

Without writers, stories would not be written, Without actors, stories could not be brought to life. Angie-Marie Delsante

Không có người viết, những câu chuyện sẽ chẳng được viết ra. Không có diễn viên, những câu chuyện chẳng thể được mang đến với cuộc đời. Angie-Marie Delsante

Topic 35. Actors and Processes — Topic 35. Actor và Tiến trình

Actors and processes offer interesting ways of implementing **concurrency** without the burden of synchronizing access to shared memory.

Actor và tiến trình (process) mở ra những cách thú vị để hiện thực hóa việc xử lý đồng thời (concurrency) mà không phải gánh nặng đồng bộ hóa quyền truy cập vào bộ nhớ chia sẻ.

Before we get into them, however, we need to define what we mean. And this is going to sound academic. Never fear, we'll be working through it all in a short while.

Tuy nhiên, trước khi bàn sâu, chúng ta cần định nghĩa rõ mình đang nói về cái gì. Và nghe sẽ có vẻ hàn lâm đấy. Đừng sợ, chỉ một lát nữa thôi chúng ta sẽ mổ xẻ tường tận tất cả.

An **actor** is an independent virtual processor with its own local (and private) **state**. Each actor has a mailbox. When a message appears in the mailbox and the actor is idle, it kicks into life and processes the message. When it finishes processing, it processes another message in the mailbox, or, if the mailbox is empty, it goes back to sleep. When processing a message, an actor can create other actors, send messages to other actors that it knows about, and create a new state that will become the **current state** when the next message is processed.

Một actor là một bộ xử lý ảo độc lập, sở hữu trạng thái cục bộ (và riêng tư) của riêng nó. Mỗi actor có một hộp thư (mailbox). Khi một thông điệp xuất hiện trong hộp thư mà actor đang rảnh rỗi, nó bừng tỉnh và xử lý thông điệp đó. Xử lý xong, nó xử lý tiếp thông điệp kế trong hộp thư, hoặc, nếu hộp thư rỗng, nó quay lại ngủ. Khi đang xử lý một thông điệp, actor có thể tạo ra các actor khác, gửi thông điệp tới những actor mà nó biết, và tạo ra một trạng thái mới — trạng thái này sẽ trở thành trạng thái hiện hành khi thông điệp tiếp theo được xử lý.

A **process** is typically a more general-purpose virtual processor, often implemented by the operating system to facilitate concurrency. Processes can be constrained (by convention) to behave like actors, and that's the type of process we mean here.

Một tiến trình thường là một bộ xử lý ảo có mục đích tổng quát hơn, thường do hệ điều hành hiện thực để hỗ trợ xử lý đồng thời. Tiến trình có thể bị ràng buộc (theo quy ước) để hành xử như một actor, và đó chính là loại tiến trình mà chúng tôi muốn nói đến ở đây.

Actors Can Only Be Concurrent — Actor Bắt Buộc Phải Đồng Thời

There are a few things that you won't find in the definition of actors:

Có một vài điều bạn sẽ không tìm thấy trong định nghĩa về actor:

There's no single thing that's in control. Nothing schedules what happens next, or orchestrates the transfer of information from the raw data to the final output.

Không hề có một thực thể duy nhất nào nắm quyền điều khiển. Chẳng có gì lên lịch xem chuyện gì xảy ra kế tiếp, hay điều phối việc chuyển thông tin từ dữ liệu thô đến kết quả cuối cùng.

The only state in the system is held in messages and in the local state of each actor. Messages cannot be examined except by being read by their recipient, and local state is inaccessible outside the actor.

Trạng thái duy nhất trong hệ thống nằm ở các thông điệp và ở trạng thái cục bộ của từng actor. Không thể soi xét thông điệp nếu không phải bằng cách người nhận đọc nó, còn trạng thái cục bộ thì không thể truy cập từ bên ngoài actor.

All messages are one way—there's no concept of replying. If you want an actor to return a response, you include your own mailbox address in the message you send it, and it will (eventually) send the response as just another message to that mailbox.

Mọi thông điệp đều là một chiều — không hề có khái niệm trả lời. Nếu bạn muốn một actor gửi lại phản hồi, bạn phải đính kèm địa chỉ hộp thư của chính mình vào thông điệp gửi đi, và (sớm muộn) nó sẽ gửi phản hồi như một thông điệp khác đến hộp thư đó.

An actor processes each message to completion, and only processes one message at a time.

Một actor xử lý trọn vẹn từng thông điệp, và mỗi lần chỉ xử lý đúng một thông điệp.

As a result, actors execute concurrently, asynchronously, and share nothing. If you had enough physical processors, you could run an actor on each. If you have a single processor, then some runtime can handle the switching of **context** between them. Either way, the code running in the actors is the same.

Hệ quả là các actor thực thi đồng thời, bất đồng bộ, và không chia sẻ gì cả. Nếu có đủ bộ xử lý vật lý, bạn có thể chạy mỗi actor trên một bộ xử lý. Nếu chỉ có một bộ xử lý, thì một runtime nào đó sẽ lo việc chuyển ngữ cảnh qua lại giữa chúng. Dù theo cách nào, đoạn mã chạy trong các actor vẫn y hệt nhau.

Tip 59: Use Actors For Concurrency Without Shared State

Tip 59: Dùng Actor Để Xử Lý Đồng Thời Mà Không Cần Trạng Thái Chia Sẻ

A Simple Actor — Một Actor Đơn Giản

Let's implement our diner using actors. In this case, we'll have three (the **customer**, the waiter, and the pie case).

Hãy hiện thực cái quán ăn của chúng ta bằng actor. Lần này, ta sẽ có ba actor (khách, bồi bàn, và tủ bánh).

The overall message flow will look like this:

Tổng thể luồng thông điệp sẽ trông như sau:

We (as some kind of external, God-like being) tell the customer that

Chúng ta (trong vai một đấng tối cao, kiểu thần thánh bên ngoài) bảo khách rằng
they are hungry
họ đang đói

In response, they'll ask the waiter for pie

Đáp lại, khách sẽ hỏi bồi bàn xin bánh

The waiter will ask the pie case to get some pie to the customer

Bồi bàn sẽ nhờ tủ bánh đưa ít bánh cho khách

If the pie case has a slice available, it will send it to the customer, and also notify the waiter to add it to the bill

Nếu tủ bánh còn một miếng, nó sẽ gửi miếng bánh cho khách, đồng thời báo cho bồi bàn ghi vào hóa đơn

If there is no pie, the case tells the waiter, and the waiter apologizes to the customer

Nếu hết bánh, tủ bánh báo cho bồi bàn, và bồi bàn xin lỗi khách

We've chosen to implement the code in JavaScript using the Nact library. [48] We've added a little **wrapper** to this that lets us write actors as simple objects, where the keys are the message types that it receives and the values are functions to run when that particular message is received. (Most actor systems have a similar kind of structure, but the details depend on the **host language**.)

Chúng tôi chọn hiện thực đoạn mã bằng JavaScript với thư viện Nact. [48] Chúng tôi có bọc thêm một lớp nhỏ để có thể viết actor dưới dạng object đơn giản: khóa là các loại thông điệp mà actor nhận được, còn giá trị là các hàm sẽ chạy khi thông điệp đó tới. (Đa số hệ thống actor đều có cấu trúc đại loại như vậy, nhưng chi tiết thì tùy vào ngôn ngữ chủ.)

Let's start with the customer. The customer can receive three messages:

Hãy bắt đầu với khách. Khách có thể nhận ba thông điệp:

You're hungry (sent by the external context)

Bạn đang đói (do ngữ cảnh bên ngoài gửi)

There's pie on the table (sent by the pie case)

Có bánh trên bàn (do tủ bánh gửi)

Sorry, there's no pie (sent by the waiter)

Xin lỗi, hết bánh (do bồi bàn gửi)

Here's the code:

Đây là đoạn mã:

concurrency/actors/index.js

concurrency/actors/index.js

```
const customerActor = { 'hungry for pie' : (msg, ctx, state) => { return dispatch(state.waiter, { type: "order", customer: ctx.self, wants: 'pie' }) },
```

```
const customerActor = { 'hungry for pie' : (msg, ctx, state) => { return dispatch(state.waiter, { type: "order", customer: ctx.self, wants: 'pie' }) },
```

```
‘put on table’: (msg, ctx, _state) => console.log( ${ctx.self.name} sees " ${msg.food} " appear on the table ),
```

```
‘put on table’: (msg, ctx, _state) => console.log( ${ctx.self.name} sees " ${msg.food} " appear on the table ),
```

```
‘no pie left’: (_msg, ctx, _state) => console.log( ${ctx.self.name} sulks... ) }
```

```
‘no pie left’: (_msg, ctx, _state) => console.log( ${ctx.self.name} sulks... ) }
```

The interesting case is when we receive a “hungry for pie” message, where we then send a message off to the waiter. (We’ll see how the customer knows about the waiter actor shortly.)

Trường hợp đáng chú ý là khi nhận được thông điệp “hungry for pie”, lúc đó ta gửi một thông điệp đến cho bồi bàn. (Chắc nữa ta sẽ thấy làm sao khách biết được về actor bồi bàn.)

Here’s the waiter’s code:

Đây là đoạn mã của bồi bàn:

concurrency/actors/index.js

concurrency/actors/index.js

```
const waiterActor = { “order”: (msg, ctx, state) => { if (msg.wants == “pie” ) { dispatch(state.pieCase, { type: “get slice” , customer: msg.customer, waiter: ctx.self }) } else { console.dir( Don't know how to order ${msg.wants} ); } },
```

```
const waiterActor = { “order”: (msg, ctx, state) => { if (msg.wants == “pie” ) { dispatch(state.pieCase, { type: “get slice” , customer: msg.customer, waiter: ctx.self }) } else { console.dir( Don't know how to order ${msg.wants} ); } },
```

```
“add to order”: (msg, ctx) => console.log( Waiter adds ${msg.food} to ${msg.customer.name} 's order ),
```

```
“add to order”: (msg, ctx) => console.log( Waiter adds ${msg.food} to ${msg.customer.name} 's order ),
```

```
“error”: (msg, ctx) => { dispatch(msg.customer, { type: ‘no pie left’ , msg: msg.msg }); console.log( \nThe waiter apologizes to ${msg.customer.name} : ${msg.msg} ) }
```

```
“error”: (msg, ctx) => { dispatch(msg.customer, { type: ‘no pie left’ , msg: msg.msg }); console.log( \nThe waiter apologizes to ${msg.customer.name} : ${msg.msg} ) }
```

```
};
```

```
};
```

When it receives the ‘order’ message from the customer, it checks to see if the request is for pie. If so, it sends a request to the pie

Khi nhận được thông điệp ‘order’ từ khách, nó kiểm tra xem yêu cầu có phải là bánh không. Nếu đúng, nó gửi một yêu cầu tới tủ

case, passing references both to itself and the customer.

bánh, kèm theo tham chiếu đến cả chính nó lẫn khách.

The pie case has state: an array of all the slices of pie it holds. (Again, we see how that gets set up shortly.) When it receives a ‘get slice’ message from the waiter, it sees if it has any slices left. If it does,

it passes the slice to the customer, tells the waiter to update the order, and finally returns an updated state, containing one less slice. Here's the code:

Tủ bánh có trạng thái: một mảng chứa tất cả các miếng bánh mà nó đang giữ. (Lát nữa ta sẽ thấy mảng này được thiết lập thế nào.) Khi nhận được thông điệp 'get slice' từ bồi bàn, nó xem còn miếng nào không. Nếu còn, nó chuyển miếng bánh cho khách, bảo bồi bàn cập nhật hóa đơn, và cuối cùng trả về một trạng thái mới, ít hơn một miếng. Đây là đoạn mã:

concurrency/actors/index.js

concurrency/actors/index.js

```
const pieCaseActor = { 'get slice' : (msg, context, state) => { if (state.slices.length == 0) { dispatch(msg.waiter, { type: 'error' , msg: "no pie left" , customer: msg.customer }) return state } else { var slice = state.slices.shift() + " pie slice" ; dispatch(msg.customer, { type: 'put on table' , food: slice }); dispatch(msg.waiter, { type: 'add to order' , food: slice, customer: msg.customer }); return state; } } }
```

```
const pieCaseActor = { 'get slice' : (msg, context, state) => { if (state.slices.length == 0) { dispatch(msg.waiter, { type: 'error' , msg: "no pie left" , customer: msg.customer }) return state } else { var slice = state.slices.shift() + " pie slice" ; dispatch(msg.customer, { type: 'put on table' , food: slice }); dispatch(msg.waiter, { type: 'add to order' , food: slice, customer: msg.customer }); return state; } } }
```

Although you'll often find that actors are started dynamically by other actors, in our case we'll keep it simple and start our actors manually. We will also pass each some initial state:

Mặc dù bạn thường thấy các actor được khởi tạo động bởi những actor khác, ở đây ta cứ làm cho đơn giản, khởi tạo các actor một cách thủ công. Ta cũng truyền cho mỗi actor một ít trạng thái khởi đầu:

The pie case gets the initial list of pie slices it contains

Tủ bánh nhận được danh sách ban đầu các miếng bánh mà nó chứa

We'll give the waiter a reference to the pie case

Ta sẽ trao cho bồi bàn một tham chiếu đến tủ bánh

We'll give the customers a reference to the waiter

Ta sẽ trao cho các khách một tham chiếu đến bồi bàn

concurrency/actors/index.js

concurrency/actors/index.js

```
const actorSystem = start();
```

```
const actorSystem = start();
```

```
let pieCase = start_actor( actorSystem, 'pie-case' , pieCaseActor, { slices: [ "apple" , "peach" , "cherry" ] });
```

```
let pieCase = start_actor( actorSystem, 'pie-case' , pieCaseActor, { slices: [ "apple" , "peach" , "cherry" ] });
```

```
let waiter = start_actor( actorSystem, 'waiter' , waiterActor, { pieCase: pieCase });
```

```
let waiter = start_actor( actorSystem, 'waiter' , waiterActor, { pieCase: pieCase });
```

```
let c1 = start_actor(actorSystem, 'customer1' , customerActor, { waiter: waiter }); let c2 = start_actor(actorSystem, 'customer2' , customerActor, { waiter: waiter });
```

```
let c1 = start_actor(actorSystem, 'customer1' , customerActor, { waiter: waiter }); let c2 = start_actor(actorSystem, 'customer2' , customerActor, { waiter: waiter });
```

And finally we kick it off. Our customers are greedy. Customer 1 asks for three slices of pie, and customer 2 asks for two:

Và cuối cùng ta khởi động mọi thứ. Các vị khách của ta đều tham ăn. Khách 1 đòi ba miếng bánh, còn khách 2 đòi hai miếng:

concurrency/actors/index.js

concurrency/actors/index.js

```
dispatch(c1, { type: 'hungry for pie' , waiter: waiter }); dispatch(c2, { type: 'hungry for pie' , waiter: waiter }); dispatch(c1, { type: 'hungry for pie' , waiter: waiter }); dispatch(c2, { type: 'hungry for pie' , waiter: waiter }); dispatch(c1, { type: 'hungry for pie' , waiter: waiter }); sleep(500) .then(() => { stop(actorSystem); })
```

When we run it, we can see the actors communicating. [49] The order you see may well be different:

```
dispatch(c1, { type: 'hungry for pie' , waiter: waiter }); dispatch(c2, { type: 'hungry for pie' , waiter: waiter }); dispatch(c1, { type: 'hungry for pie' , waiter: waiter }); dispatch(c2, { type: 'hungry for pie' , waiter: waiter }); dispatch(c1, { type: 'hungry for pie' , waiter: waiter }); sleep(500) .then(() => { stop(actorSystem); })
```

Khi chạy chương trình, ta thấy được các actor trao đổi với nhau. [49] Thứ tự bạn thấy rất có thể sẽ khác:

\$ node index.js customer1 sees “apple pie slice” appear on the table customer2 sees “peach pie slice” appear on the table

\$ node index.js customer1 sees “apple pie slice” appear on the table customer2 sees “peach pie slice” appear on the table

Waiter adds apple pie slice to customer1’s order Waiter adds peach pie slice to customer2’s order customer1 sees “cherry pie slice” appear on the table Waiter adds cherry pie slice to customer1’s order

Waiter adds apple pie slice to customer1’s order Waiter adds peach pie slice to customer2’s order customer1 sees “cherry pie slice” appear on the table Waiter adds cherry pie slice to customer1’s order

The waiter apologizes to customer1: no pie left customer1 sulks...

The waiter apologizes to customer1: no pie left customer1 sulks...

The waiter apologizes to customer2: no pie left customer2 sulks...

The waiter apologizes to customer2: no pie left customer2 sulks...

No Explicit Concurrency — Không Cần Xử Lý Đồng Thời Tường Minh

In the actor model, there’s no need to write any code to handle concurrency, as there is no **shared state**. There’s also no need to code in explicit end-to-end “do this, do that” logic, as the actors work it out for themselves based on the messages they receive.

Trong mô hình actor, bạn chẳng cần viết bất kỳ đoạn mã nào để xử lý chuyện đồng thời, bởi không hề có trạng thái chia sẻ. Cũng chẳng cần viết logic kiểu “làm cái này, rồi làm cái

kia” một cách tường minh từ đầu đến cuối, vì bản thân các actor tự xoay sở lấy dựa trên những thông điệp chúng nhận được.

There’s also no mention of the underlying architecture. This set of components work equally well on a single processor, on multiple cores, or on multiple networked machines.

Cũng không hề có một dòng nào nhắc đến kiến trúc bên dưới. Tập hợp các thành phần này chạy tốt như nhau dù trên một bộ xử lý, trên nhiều lõi, hay trên nhiều máy nối mạng với nhau.

Erlang Sets The Stage — Erlang Mở Màn

The Erlang language and runtime are great examples of an actor implementation (even though the inventors of Erlang hadn’t read the original Actor’s paper). Erlang calls actors processes, but they aren’t regular operating system processes. Instead, just like the actors we’ve been discussing, Erlang processes are lightweight (you can run millions of them on a single machine), and they communicate by sending messages. Each is isolated from the others, so there is no sharing of state.

Ngôn ngữ và runtime Erlang là những ví dụ tuyệt vời về một hiện thực của mô hình actor (dấu cho những người phát minh ra Erlang khi đó chưa hề đọc bài báo gốc về Actor). Erlang gọi actor là tiến trình (process), nhưng đó không phải là tiến trình thông thường của hệ điều hành. Trái lại, giống hệt các actor mà ta đang bàn, tiến trình của Erlang rất nhẹ (bạn có thể chạy hàng triệu cái trên một máy), và chúng giao tiếp bằng cách gửi thông điệp. Mỗi tiến trình cô lập với những tiến trình khác, nên không hề có chuyện chia sẻ trạng thái.

In addition, the Erlang runtime implements a supervision system, which manages the lifetimes of processes, potentially restarting a process or set of processes in case of failure. And Erlang also offers hot-code loading: you can replace code in a

Ngoài ra, runtime của Erlang còn hiện thực một hệ thống giám sát (supervision), lo việc quản lý vòng đời của các tiến trình, có thể khởi động lại một tiến trình hay một nhóm tiến trình khi gặp sự cố. Erlang cũng cho phép nạp mã nóng (hot-code loading): bạn có thể thay thế mã trong một hệ thống

running system without stopping that system. And the Erlang system runs some of the world’s most reliable code, often citing nine nines availability.

đang chạy mà không cần dừng hệ thống đó lại. Và hệ thống Erlang đang chạy một số đoạn mã đáng tin cậy bậc nhất thế giới, thường được dẫn chứng với độ sẵn sàng chín số chín (nine nines).

But Erlang (and its progeny Elixir) aren’t unique—there are actor implementations for most languages. Consider using them for your concurrent implementations.

Nhưng Erlang (cùng hậu duệ của nó là Elixir) không phải là độc nhất vô nhị — có những hiện thực mô hình actor cho hầu hết các ngôn ngữ. Hãy cân nhắc dùng chúng cho các phần hiện thực xử lý đồng thời của bạn.

Related Sections Include — Các mục liên quan

Topic 28, Decoupling

Topic 28, Gỡ ghép nối

Topic 30, Transforming Programming

Topic 30, Lập trình bằng phép biến đổi

Topic 36, Blackboards

Topic 36, Bảng đen

Challenges — Thử thách

Do you currently have code that uses **mutual exclusion** to protect shared data. Why not try a **prototype** of the same code written using actors?

Hiện bạn có đoạn mã nào dùng loại trừ tương hỗ (mutual exclusion) để bảo vệ dữ liệu chia sẻ không? Sao không thử làm một bản mẫu cho chính đoạn mã đó nhưng viết bằng actor xem?

The actor code for the diner only supports ordering slices of pie. Extend it to let customers order pie à la mode, with separate agents managing the pie slices and the scoops of ice cream. Arrange things so that it handles the situation where one or the other runs out.

Đoạn mã actor cho quán ăn mới chỉ hỗ trợ đặt từng miếng bánh. Hãy mở rộng nó để khách có thể gọi bánh kèm kem (pie à la mode), với các tác nhân (agent) riêng quản lý số miếng bánh và số viên kem. Hãy sắp xếp sao cho nó xử lý được tình huống một trong hai thứ — bánh hoặc kem — hết hàng.

The writing is on the wall... Daniel 5 (ref)

Lời sấm truyền đã hiện trên tường... Daniel 5 (ref)

Topic 36. Blackboards — Topic 36. Bảng đen

Consider how detectives might use a **blackboard** to coordinate and solve a murder investigation. The chief inspector starts off by setting up a large blackboard in the conference room. On it, she writes a single question:

Hãy hình dung các thám tử có thể dùng một tấm bảng đen (blackboard) để phối hợp và phá một vụ án mạng như thế nào. Vị thanh tra trưởng bắt đầu bằng việc dựng một tấm bảng đen lớn trong phòng họp. Trên đó, bà viết một câu hỏi duy nhất:

H. Dumpty (Male, Egg): Accident? Murder?

H. Dumpty (Nam, Trứng): Tai nạn? Án mạng?

Did Humpty really fall, or was he pushed? Each detective may make contributions to this potential murder mystery by adding facts, statements from witnesses, any forensic evidence that might arise, and so on. As the data accumulates, a detective might notice a connection and post that observation or speculation as well. This **process** continues, across all shifts, with many different people and agents, until the case is closed. A sample blackboard is shown in the figure .

Humpty thực sự bị ngã, hay bị ai đó xô? Mỗi thám tử có thể góp phần vào vụ án mạng tiềm tàng này bằng cách thêm vào các sự kiện, lời khai nhân chứng, mọi bằng chứng pháp y nảy sinh, v.v. Khi dữ liệu tích tụ dần, một thám tử có thể nhận ra mối liên hệ nào đó và dán luôn quan sát hay phỏng đoán ấy lên bảng. Quá trình này tiếp diễn qua mọi ca trực, với nhiều người và tác nhân khác nhau, cho đến khi vụ án khép lại. Một tấm bảng đen mẫu được minh họa ở hình dưới.

images/blackboard.png

images/blackboard.png

Figure 2. Someone found a connection between Humpty's gambling debts and the phone logs. Perhaps he was getting threatening phone calls.

Hình 2. Có người tìm ra mối liên hệ giữa các khoản nợ cờ bạc của Humpty và bản ghi cuộc gọi điện thoại. Có lẽ ông ta đang nhận những cuộc gọi đe dọa.

Some key features of the blackboard approach are:

Vài đặc điểm cốt lõi của phương pháp bảng đen là:

None of the detectives needs to know of the existence of any other detective—they watch the board for new information, and add their findings.

Không thám tử nào cần biết đến sự tồn tại của bất kỳ thám tử nào khác — họ chỉ theo dõi tấm bảng để tìm thông tin mới, rồi bổ sung những gì mình tìm được.

The detectives may be trained in different disciplines, may have different levels of education and expertise, and may not even work in the same precinct. They share a desire to solve the case, but

Các thám tử có thể được đào tạo trong những chuyên ngành khác nhau, có trình độ học vấn và mức độ chuyên môn khác nhau, thậm chí không cùng làm ở một khu vực cảnh sát. Họ chia sẻ chung mong muốn phá án, nhưng

that's all.

chỉ có vậy thôi.

Different detectives may come and go during the course of the process, and may work different shifts.

Những thám tử khác nhau có thể đến rồi đi trong suốt quá trình, và có thể làm những ca trực khác nhau.

There are no restrictions on what may be placed on the blackboard. It may be pictures, sentences, physical evidence, and so on.

Không có ràng buộc nào về thứ có thể đặt lên bảng đen. Đó có thể là hình ảnh, câu chữ, vật chứng, v.v.

This is a form of *laissez faire* **concurrency**. The detectives are independent processes, agents, actors, and so on. Some store facts on the blackboard. Others take facts off the board, maybe combining or processing them, and add more information to the board. Gradually the board helps them come to a conclusion.

Đây là một dạng xử lý đồng thời (concurrency) kiểu “thả nổi” (*laissez faire*). Các thám tử là những tiến trình, tác nhân, actor độc lập, v.v. Một số người lưu sự kiện lên bảng. Những người khác lấy sự kiện ra khỏi bảng, có lẽ kết hợp hay xử lý chúng, rồi thêm thông tin mới lên bảng. Dần dần, tấm bảng giúp họ đi đến một kết luận.

Computer-based blackboard systems were originally used in artificial intelligence applications where the problems to be solved were large and complex—speech recognition, knowledgebased reasoning systems, and so on.

Các hệ thống bảng đen dựa trên máy tính ban đầu được dùng trong những ứng dụng trí tuệ nhân tạo, nơi các bài toán cần giải vừa lớn vừa phức tạp — nhận dạng giọng nói, hệ suy luận dựa trên tri thức, v.v.

One of the first blackboard systems was David Gelernter's Linda. It stored facts as typed tuples. Applications could write new tuples into Linda, and query for existing tuples using a form of pattern matching.

Một trong những hệ bảng đen đầu tiên là Linda của David Gelernter. Nó lưu các sự kiện dưới dạng bộ (tuple) có kiểu. Các ứng dụng có thể ghi tuple mới vào Linda, và truy vấn các tuple đã có bằng một dạng so khớp mẫu (pattern matching).

Later came distributed blackboard-like systems such as JavaSpaces and T Spaces. With these systems, you can store active Java objects—not just data—on the blackboard, and retrieve them by partial matching of fields (via templates and wildcards) or by subtypes. For example, suppose you had a type `Author`, which is a subtype of `Person`. You could search a blackboard containing `Person` objects by using an `Author` template with a `lastName` value of “Shakespeare.” You'd get Bill Shakespeare the author, but not Fred Shakespeare the gardener.

Về sau xuất hiện những hệ phân tán kiểu bảng đen như JavaSpaces và T Spaces. Với các hệ này, bạn có thể lưu trên bảng đen cả những đối tượng Java sống động — chứ không chỉ dữ liệu — rồi lấy chúng ra bằng cách so khớp một phần các trường (qua template và ký tự đại diện) hoặc theo kiểu con. Chẳng hạn, giả sử bạn có kiểu `Author`, là kiểu con của `Person`. Bạn có thể tìm trong một tấm bảng đen chứa các đối tượng `Person` bằng một template `Author` với `lastName` là “Shakespeare”. Bạn sẽ nhận được Bill Shakespeare nhà văn, chứ không phải Fred Shakespeare bác làm vườn.

These systems never really took off, we believe, in part, because the need for the kind of concurrent cooperative processing hadn't yet developed.

Chúng tôi tin rằng những hệ này chưa bao giờ thực sự cất cánh, một phần vì nhu cầu về kiểu xử lý hợp tác đồng thời như vậy lúc ấy vẫn chưa hình thành.

A Blackboard In Action — Bảng đen trong thực chiến

Suppose we are writing a program to accept and process mortgage or loan applications. The laws that govern this area are odiously complex, with federal, **state**, and local governments all having their say. The lender must prove they have disclosed certain things, and must ask for certain information—but must not ask certain other questions, and so on, and so on.

Giả sử chúng ta đang viết một chương trình để tiếp nhận và xử lý đơn vay thế chấp hoặc đơn vay vốn. Luật lệ chi phối lĩnh vực này phức tạp đến phát ngán, khi chính quyền liên bang, tiểu bang lẫn địa phương đều có tiếng nói. Bên cho vay phải chứng minh đã công bố một số điều nhất định, và phải hỏi một số thông tin nhất định — nhưng lại không được hỏi một số câu khác, v.v. và v.v.

Beyond the miasma of applicable law, we also have the following problems to contend with:

Ngoài cái đầm lầy luật lệ đang áp dụng đó, chúng ta còn phải đối phó với những vấn đề sau:

Responses can arrive in any order. For instance, queries for a credit check or title search may take a substantial amount of time, while items such as name and address may be available immediately.

Phản hồi có thể đến theo bất kỳ thứ tự nào. Chẳng hạn, các truy vấn kiểm tra tín dụng hay tra cứu quyền sở hữu có thể mất khá nhiều thời gian, trong khi những thứ như tên và địa chỉ thì có ngay tức thì.

Data gathering may be done by different people, distributed across different offices, in different time zones.

Việc thu thập dữ liệu có thể do nhiều người khác nhau thực hiện, rải rác qua các văn phòng khác nhau, ở những múi giờ khác nhau.

Some data gathering may be done automatically by other systems. This data may arrive asynchronously as well.

Một số dữ liệu có thể được các hệ thống khác thu thập tự động. Dữ liệu này cũng có thể đến một cách bất đồng bộ.

Nonetheless, certain data may still be dependent on other data. For instance, you may not be able to start the title search for a car until you get proof of ownership or insurance.

Tuy vậy, một số dữ liệu vẫn có thể phụ thuộc vào dữ liệu khác. Ví dụ, bạn không thể bắt đầu tra cứu quyền sở hữu cho một chiếc xe cho đến khi có được bằng chứng sở hữu hoặc bảo hiểm.

The arrival of new data may raise new questions and policies. Suppose the credit check comes back with a less than glowing report; now you need these five extra forms and perhaps a blood sample.

Sự xuất hiện của dữ liệu mới có thể làm nảy sinh những câu hỏi và chính sách mới. Giả sử kết quả kiểm tra tín dụng trả về không lấy gì làm sáng sủa; giờ thì bạn cần thêm năm biểu mẫu này và có lẽ cả một mẫu máu nữa.

You can try to handle every possible combination and circumstance using a workflow system. Many such systems

Bạn có thể cố gắng xử lý mọi tổ hợp và tình huống khả dĩ bằng một hệ thống quy trình công việc (workflow). Nhiều hệ thống như vậy

exist, but they can be complex and programmer intensive. As regulations change, the workflow must be reorganized: people may have to change their procedures and hard-wired code may have to be rewritten.

tồn tại, nhưng chúng có thể phức tạp và tốn nhiều công sức lập trình. Khi quy định thay đổi, quy trình công việc phải được tổ chức lại: con người có thể phải đổi quy trình làm việc, và mã được “đi dây cứng” có thể phải viết lại.

A blackboard, in combination with a rules engine that encapsulates the legal **requirements**, is an elegant solution to the difficulties found here. Order of data arrival is irrelevant: when a fact is posted it can trigger the appropriate rules. Feedback is easily handled as well: the output of any set of rules can post to the blackboard and cause the triggering of yet more applicable rules.

Một tấm bảng đen, kết hợp với một bộ máy luật lệ (rules engine) đóng gói các yêu cầu pháp lý, là một giải pháp thanh thoát cho những khó khăn ở đây. Thứ tự dữ liệu đến không còn quan trọng: khi một sự kiện được đăng lên, nó có thể kích hoạt những luật lệ thích hợp. Phản hồi cũng được xử lý dễ dàng: đầu ra của bất kỳ tập luật nào cũng có thể đăng lên bảng đen và làm kích hoạt thêm những luật lệ áp dụng được khác.

Tip 60: Use Blackboards to Coordinate Workflow

Tip 60: Dùng Bảng Đen Để Điều Phối Quy Trình Công Việc

Messaging Systems Can Be Like Blackboards — Hệ thống nhắn tin có thể giống như bảng đen

As we're writing this second edition, many applications are constructed using small, decoupled services, all communicating via some form of messaging system. These messaging systems (such as

Kafka and NATS) do far more than simply send data from A to B. In particular, they offer persistence (in the form of an **event log**) and the ability to retrieve messages through a form of pattern matching. This means you can use them both as a blackboard system and/or as a platform on which you can run a bunch of actors.

Vào lúc chúng tôi viết ấn bản thứ hai này, nhiều ứng dụng được dựng nên từ các dịch vụ nhỏ, ít ghép nối, tất cả giao tiếp qua một dạng hệ thống nhắn tin (messaging system) nào đó. Những hệ nhắn tin này (như Kafka và NATS) làm được nhiều thứ hơn nhiều so với chỉ chuyển dữ liệu từ A sang B. Cụ thể, chúng cung cấp khả năng lưu trữ bền vững (dưới dạng nhật ký sự kiện) và khả năng truy xuất thông điệp qua một dạng so khớp mẫu. Điều này nghĩa là bạn có thể dùng chúng vừa như một hệ bảng đen và/hoặc như một nền tảng để chạy một loạt actor.

But It'S Not That Simple... — Nhưng đời không đơn giản vậy đâu...

The **actor** and/or blackboard and/or microservice approach to architecture removes a whole class of potential concurrency problems from your applications. But that benefit comes at a cost. These approaches are harder to reason about, because a lot

Cách tiếp cận kiến trúc theo kiểu actor và/hoặc bảng đen và/hoặc microservice loại bỏ cả một lớp các vấn đề xử lý đồng thời tiềm tàng khỏi ứng dụng của bạn. Nhưng lợi ích ấy đi kèm một cái giá. Những cách tiếp cận này khó suy luận hơn, bởi phần lớn

of the action is indirect. You'll find it helps to keep a central repository of message formats and/or APIs, particularly if the repository can generate the code and documentation for you. You'll also need good tooling to be able to trace messages and facts as they progress through the system. (A useful technique is to add a unique trace id when a particular business function is initiated and then propagate it to all the actors involved. You'll then be able to reconstruct what happens from the log files.)

hành động diễn ra một cách gián tiếp. Bạn sẽ thấy việc duy trì một kho tập trung chứa các định dạng thông điệp và/hoặc API là rất có ích, nhất là khi cái kho ấy có thể sinh ra mã và tài liệu giúp bạn. Bạn cũng sẽ cần công cụ tốt để truy vết các thông điệp và sự kiện khi chúng di chuyển qua hệ thống. (Một kỹ thuật hữu ích là gắn một mã truy vết (trace id) duy nhất khi một chức năng nghiệp vụ cụ thể được khởi động, rồi lan truyền nó tới tất cả các actor liên quan. Khi đó bạn sẽ có thể dựng lại chuyện gì đã xảy ra từ các tệp nhật ký.)

Finally, these kinds of system can be more troublesome to deploy and manage, as there are more moving parts. To some extent this is offset by the fact that the system is more granular, and can be updated by replacing individual actors, and not the whole system.

Cuối cùng, những loại hệ thống này có thể rắc rối hơn khi triển khai và quản lý, vì có nhiều bộ phận chuyển động hơn. Phần nào điều này được bù lại bởi thực tế là hệ thống mịn hơn, và có thể cập nhật bằng cách thay thế từng actor riêng lẻ, chứ không phải cả hệ thống.

Related Sections Include — Các mục liên quan

Topic 28, Decoupling

Topic 28, Gỡ ghép nối

Topic 29, Juggling the Real World

Topic 29, Tung hứng với thế giới thực

Topic 33, Breaking **Temporal Coupling**

Topic 33, Phá vỡ ghép nối thời gian

Topic 35, Actors and Processes

Topic 35, Actor và tiến trình

Exercises — Bài tập

Exercise 24 (possible answer)

Bài tập 24 (một đáp án khả dĩ)

Would a blackboard-style system be appropriate for the following applications? Why, or why not?

Một hệ thống kiểu bảng đen có phù hợp cho những ứng dụng sau không? Vì sao có, hoặc vì sao không?

Image processing. You'd like to have a number of parallel processes grab chunks of an image, process them, and put the completed chunk back.

Xử lý ảnh. Bạn muốn có một số tiến trình song song giành lấy từng mảnh của một bức ảnh, xử lý chúng, rồi đặt mảnh đã hoàn thành trở lại.

Group calendaring. You've got people scattered across the globe, in different time zones, and speaking different languages, trying to schedule a meeting.

Lịch nhóm. Bạn có những con người rải rác khắp toàn cầu, ở các múi giờ khác nhau, nói những ngôn ngữ khác nhau, đang cố sắp xếp một cuộc họp.

Network monitoring tool. The system gathers performance statistics and collects trouble reports, which agents use to look for trouble in the system.

Công cụ giám sát mạng. Hệ thống thu thập số liệu hiệu năng và gom các báo cáo sự cố, để các tác nhân dùng đó mà truy tìm trục trặc trong hệ thống.

Challenges — Thử thách

Do you use blackboard systems in the real world—the message board by the refrigerator, or the big whiteboard at work? What makes them effective? Are messages ever posted with a consistent format? Does it matter?

Bạn có dùng các hệ bảng đen trong đời thực không — tấm bảng tin cạnh tủ lạnh, hay tấm bảng trắng to ở nơi làm việc? Điều gì làm cho chúng hiệu quả? Các mẫu tin có bao giờ được đăng theo một định dạng nhất quán không? Điều đó có quan trọng không?

Footnotes

Chú thích

[46] Although UML has gradually faded, many of its individual diagrams still exist in one form or another, including the very useful activity diagram . For more information on all of the UML diagram types, see UML Distilled: A Brief Guide to the Standard Object Modeling Language [Fow04] .

[46] Dù UML đã dần phai nhạt, nhiều biểu đồ riêng lẻ của nó vẫn còn tồn tại dưới dạng này hay dạng khác, bao gồm cả biểu đồ hoạt động (activity diagram) rất hữu dụng. Để biết thêm về tất cả các loại biểu đồ UML, xem UML Distilled: A Brief Guide to the Standard Object Modeling Language [Fow04] .

[47] The names P and V come from the initial letters of Dutch words. However there is some discussion about exactly which words. The inventor of the technique, Edsger Dijkstra, has suggested both *passering* and *prolaag* for P, and *vrijgave* and possibly *verhogen* for V.

[47] Hai cái tên P và V lấy từ chữ cái đầu của các từ tiếng Hà Lan. Tuy nhiên vẫn còn tranh cãi xem chính xác là những từ nào. Chính người phát minh ra kỹ thuật này, Edsger Dijkstra, đã từng nhắc đến cả *passering* lẫn *prolaag* cho P, và *vrijgave*, có lẽ cả *verhogen*, cho V.

[48] <https://github.com/ncthbrr/nact>

[48] <https://github.com/ncthbrr/nact>

[49] In order to run this code you'll also need our **wrapper** functions, which are not shown here. You can download them from <https://media.pragprog.com/titles/tpp20/code/concurrency/actors/index.js>

[49] Để chạy được đoạn mã này, bạn cũng sẽ cần các hàm bao của chúng tôi, vốn không được trình bày ở đây. Bạn có thể tải chúng về từ <https://media.pragprog.com/titles/tpp20/code/concurrency/actors/index.js>

Copyright © 2020 Pearson Education, Inc.

Copyright © 2020 Pearson Education, Inc.

While You Are Coding — Khi Bạn Đang Viết Mã

Conventional wisdom says that once a project is in the coding phase, the work is mostly mechanical, transcribing the design into executable statements. We think that this attitude is the single biggest reason that software projects fail, and many systems end up ugly, inefficient, poorly structured, unmaintainable, or just plain wrong.

Lẽ thường vẫn nói rằng một khi dự án đã bước vào giai đoạn viết mã (coding) thì công việc gần như chỉ còn mang tính máy móc, đơn thuần là chép lại bản thiết kế thành những câu lệnh chạy được. Chúng tôi cho rằng chính lối nghĩ này là nguyên nhân lớn nhất khiến các dự án phần mềm thất bại, và khiến biết bao hệ thống cuối cùng trở nên xấu xí, kém hiệu quả, cấu trúc lủng củng, không thể bảo trì, hoặc đơn giản là sai bét.

Coding is not mechanical. If it were, all the CASE tools that people pinned their hopes on way back in the early 1980s would have replaced programmers long ago. There are decisions to be made every minute—decisions that require careful thought and judgment if the resulting program is to enjoy a long, accurate, and productive life.

Viết mã không phải chuyện máy móc. Nếu đúng vậy thì tất cả những công cụ CASE mà người ta đặt bao kỳ vọng vào hồi đầu thập niên 1980 hẳn đã thay thế lập trình viên từ đời nào rồi. Có những quyết định phải đưa ra từng phút một — những quyết định đòi hỏi suy nghĩ và phán đoán cẩn trọng nếu muốn chương trình làm ra được sống một cuộc đời dài lâu, chính xác và hữu ích.

Not all decisions are even conscious. You can better harness your instincts and nonconscious thoughts when you **Topic 37**, Listen to Your Lizard Brain . We'll see how to listen more carefully and look at ways of actively responding to these sometimes niggling thoughts.

Không phải quyết định nào cũng diễn ra một cách ý thức. Bạn có thể khai thác bản năng và những suy nghĩ vô thức của mình tốt hơn ở Mục 37, Lắng Nghe Bộ Não Bò Sát Của Bạn. Chúng ta sẽ xem cách lắng nghe kỹ hơn và tìm hiểu những cách chủ động đáp lại những suy nghĩ đôi khi cứ gặm nhấm âm ỉ ấy.

But listening to your instincts doesn't mean you can just fly on autopilot. Developers who don't actively think about their code are **programming by coincidence**—the code might work, but

Nhưng lắng nghe bản năng không có nghĩa là bạn cứ thế bay trên chế độ tự lái. Những lập trình viên không chủ động suy nghĩ về mã của mình đang lập trình theo may rủi (programming by coincidence) — mã có thể chạy được, nhưng

there's no particular reason why. In **Topic 38**, Programming by Coincidence , we advocate a more positive involvement with the coding **process**.

chẳng có lý do cụ thể nào giải thích vì sao. Ở Mục 38, Lập Trình Theo May Rủi, chúng tôi cố vũ một thái độ dẫn thân tích cực hơn vào quá trình viết mã.

While most of the code we write executes quickly, we occasionally develop algorithms that have the potential to bog down even the fastest processors. In Topic 39, **Algorithm Speed**, we discuss ways to estimate the speed of code, and we give some tips on how to spot potential problems before they happen.

Tuy phần lớn mã chúng ta viết chạy rất nhanh, đôi khi ta lại tạo ra những thuật toán có khả năng làm sa lầy cả những bộ xử lý nhanh nhất. Ở Mục 39, Tốc Độ Thuật Toán, chúng ta bàn về những cách ước lượng tốc độ của mã, và đưa ra vài mẹo để phát hiện các vấn đề tiềm ẩn trước khi chúng xảy ra.

Pragmatic Programmers think critically about all code, including our own. We constantly see room for improvement in our programs and our designs. In Topic 40, **Refactoring**, we look at techniques that help us fix up existing code continuously as we go.

Lập trình viên Thực dụng (Pragmatic Programmer) tư duy phản biện về mọi đoạn mã, kể cả mã của chính mình. Chúng ta luôn nhìn thấy chỗ còn có thể cải thiện trong chương trình và thiết kế của mình. Ở Mục 40, Tái Cấu Trúc, chúng ta xem xét những kỹ thuật giúp ta liên tục chỉnh sửa mã hiện có ngay trong lúc làm.

Testing is not about finding bugs, it's about getting feedback on your code: aspects of design, the API, **coupling**, and so on. That means that the major benefits of testing happen when you think about and write the tests, not just when you run them. We'll explore this idea in Topic 41, **Test to Code**.

Kiểm thử không phải để tìm lỗi, mà để thu phản hồi về mã của bạn: về các khía cạnh thiết kế, về API, về sự ghép nối (coupling), v.v. Điều đó nghĩa là lợi ích lớn nhất của kiểm thử đến khi bạn nghĩ về và viết các bài kiểm thử, chứ không chỉ khi chạy chúng. Chúng ta sẽ khai thác ý tưởng này ở Mục 41, Kiểm Thử Hướng Mã.

But of course when you test your own code, you might bring your own biases to the task. In Topic 42, **Property-Based Testing** we'll see how to have the computer do some wideranging testing for you and how to handle the inevitable bugs that come up.

Nhưng dĩ nhiên khi tự kiểm thử mã của mình, bạn có thể mang theo cả những thiên kiến của bản thân vào việc đó. Ở Mục 42, Kiểm Thử Dựa Trên Thuộc Tính, chúng ta sẽ xem cách để máy tính tự thực hiện một số kiểm thử trên diện rộng giúp bạn, và cách xử lý những lỗi tất yếu sẽ nảy sinh.

It's critical that you write code that is readable and easy to reason about. It's a harsh world out there, filled with bad actors who are actively trying to break into your system and cause harm. We'll discuss some very basic techniques and approaches to help you Topic 43, Stay Safe Out There.

Việc bạn viết mã dễ đọc và dễ suy luận là điều cực kỳ quan trọng. Ngoài kia là một thế giới khắc nghiệt, đầy rẫy những kẻ xấu đang ráo riết tìm cách đột nhập vào hệ thống của bạn và gây hại. Chúng ta sẽ bàn về một vài kỹ thuật và cách tiếp cận hết sức căn bản giúp bạn ở Mục 43, Hãy Cẩn Thận Ngoài Kia.

Finally, one of the hardest things in software development is Topic 44, **Naming Things**. We have to name a lot of things, and in many ways the names we choose define the reality we create. You need to stay aware of any potential semantic drift while you are coding.

Cuối cùng, một trong những việc khó nhất trong phát triển phần mềm là Mục 44, Đặt Tên Cho Mọi Thứ. Chúng ta phải đặt tên cho vô số thứ, và ở nhiều khía cạnh, những cái tên ta chọn định hình nên cái thực tại mà ta tạo ra. Bạn cần luôn để ý đến mọi sự trượt nghĩa (semantic drift) tiềm tàng trong lúc viết mã.

Most of us can drive a car largely on autopilot; we don't explicitly command our foot to press a pedal, or our arm to turn the wheel—we just think “slow down and turn right.” However, good, safe drivers are constantly reviewing the situation, checking for potential problems, and putting themselves into good positions in case the unexpected happens. The same is true of coding—it may be largely routine, but keeping your wits about you could well prevent a disaster.

Hầu hết chúng ta đều có thể lái xe gần như ở chế độ tự lái; ta đâu có ra lệnh rõ ràng cho chân đạp phanh hay cho tay đánh vô-lăng — ta chỉ nghĩ “giảm tốc rồi rẽ phải”. Tuy vậy, những tài xế giỏi và an toàn luôn liên tục quan sát tình huống, dò tìm vấn đề tiềm ẩn và đặt mình vào vị thế tốt phòng khi điều bất ngờ xảy ra. Viết mã cũng vậy — phần lớn có thể là chuyện thường nhật, nhưng giữ cho đầu óc tỉnh táo hoàn toàn có thể ngăn được một thảm họa.

Only human beings can look directly at something, have all the information they need to make an accurate prediction, perhaps even momentarily make the accurate prediction, and then say that it isn't so. Gavin de Becker, *The Gift of Fear*

Chỉ có con người mới có thể nhìn thẳng vào một điều gì đó, có đủ mọi thông tin để đưa ra một dự đoán chính xác, thậm chí trong khoảnh khắc còn đưa ra đúng dự đoán ấy, rồi lại nói rằng nó không phải thế. Gavin de Becker, *The Gift of Fear*

Topic 37. Listen to Your Lizard Brain — Topic 37. Lắng Nghe Bộ Não Bò Sát Của Bạn

Gavin de Becker's life's work is helping people to protect themselves. His book, *The Gift of Fear: And Other Survival Signals That Protect Us from Violence* [de 98], encapsulates his message. One of the key themes running through the book is that as sophisticated humans we have learned to ignore our more animal side; our instincts, our lizard brain. He claims that most people who are attacked in the street are aware of feeling uncomfortable or nervous before the attack. These people just tell themselves they're being silly. Then the figure emerges from the dark doorway....

Sự nghiệp cả đời của Gavin de Becker là giúp con người tự bảo vệ mình. Cuốn sách của ông, *The Gift of Fear: And Other Survival Signals That Protect Us from Violence* [de 98], đúc kết thông điệp ấy. Một trong những chủ đề chính chạy xuyên suốt cuốn sách là: với tư cách những con người tinh vi, chúng ta đã học cách phớt lờ phần con vật hơn trong mình; phớt lờ bản năng, phớt lờ bộ não bò sát của mình. Ông cho rằng phần lớn những người bị tấn công trên phố đều cảm thấy bất an hay bồn chồn trước khi bị tấn công. Họ chỉ tự nhủ rằng mình đang ngớ ngẩn. Rồi cái bóng kia bước ra từ khung cửa tối....

Instincts are simply a response to patterns packed into our nonconscious brain. Some are innate, others are learned through repetition. As you gain experience as a programmer, your brain is laying down layers of tacit knowledge: things that work, things that don't work, the probable causes of a type of error, all the things you notice throughout your days. This is the part of your brain that hits the save file key when you stop to chat with someone, even when you don't realize that you're

Bản năng chẳng qua là một phản ứng trước những khuôn mẫu đã được nén chặt vào bộ não vô thức của chúng ta. Có thứ là bẩm sinh, có thứ học được qua sự lặp đi lặp lại. Khi bạn tích lũy kinh nghiệm với tư cách một lập trình viên, bộ não bạn đang đắp dần từng lớp tri thức ngầm: những gì hiệu quả, những gì không, nguyên nhân khả dĩ của một loại lỗi nào đó, tất cả những điều bạn để ý thấy trong suốt ngày dài. Đó chính là phần não bấm phím lưu file khi bạn dừng lại tán gẫu với ai đó, dù chính bạn còn chẳng nhận ra là mình

doing it.

đang làm vậy.

Whatever their source, instincts share one thing: they have no words. Instincts make you feel, not think. And so when an instinct is triggered, you don't see a flashing lightbulb with a banner wrapped around it. Instead, you get nervous, or queasy, or feel like this is just too much work.

Dù xuất xứ từ đâu, các bản năng đều có chung một điểm: chúng không có lời. Bản năng khiến bạn cảm, chứ không khiến bạn nghĩ. Và thế nên khi một bản năng được kích hoạt, bạn chẳng thấy bóng đèn nào lóe sáng với một dải băng-rôn quấn quanh. Thay vào đó, bạn thấy bồn chồn, nôn nao, hoặc cảm giác như việc này sao mà nhọc nhằn quá đỗi.

The trick is first to notice it is happening, and then to work out why. Let's look first at a couple of common situations in which your inner lizard is trying to tell you something. Then we'll discuss how you can let that instinctive brain out of its protective **wrapper**.

Cái khéo là trước hết phải nhận ra rằng nó đang diễn ra, rồi sau đó tìm cho ra lý do. Trước tiên hãy cùng xem vài tình huống thường gặp khi con bò sát bên trong bạn đang cố nói điều gì đó với bạn. Rồi chúng ta sẽ bàn cách thả cái bộ não bản năng ấy ra khỏi lớp vỏ bảo vệ của nó.

Fear Of The Blank Page — Nỗi Sợ Trang Giấy Trắng

Everyone fears the empty screen, the lonely blinking cursor surrounded by a whole bunch of nothing. Starting a new project (or even a new module in an existing project) can be an unnerving experience. Many of us would prefer to put off making the initial commitment of starting.

Ai cũng sợ cái màn hình trống trơn, con trỏ cô đơn nhấp nháy giữa một vùng trắng xóa chẳng có gì. Bắt đầu một dự án mới (hay thậm chí một mô-đun mới trong một dự án đang có) có thể là một trải nghiệm khiến ta chùn lòng. Nhiều người trong chúng ta thà trì hoãn cái cam kết khởi đầu của việc bắt tay vào còn hơn.

We think that there are two problems that cause this, and that both have the same solution.

Chúng tôi cho rằng có hai vấn đề gây ra chuyện này, và cả hai đều có chung một lời giải.

One problem is that your lizard brain is trying to tell you something; there's some kind of doubt lurking just below the surface of perception. And that's important.

Một vấn đề là bộ não bò sát của bạn đang cố nói với bạn điều gì đó; có một mối nghi ngại nào đó đang lẩn quất ngay dưới bề mặt nhận thức. Và điều đó rất quan trọng.

As a developer, you've been trying things and seeing which worked and which didn't. You've been accumulating experience and wisdom. When you feel a nagging doubt, or experience some reluctance when faced with a task, it might be that experience trying to speak to you. Heed it. You may not be able

Là một lập trình viên, bạn đã luôn thử các thứ và xem cái nào hiệu quả, cái nào không. Bạn đã tích lũy kinh nghiệm và sự khôn ngoan. Khi bạn cảm thấy một mối nghi ngại cứ day dứt, hay thấy chần chừ trước một nhiệm vụ, rất có thể đó là chính kinh nghiệm của bạn đang cố lên tiếng. Hãy để tâm đến nó. Bạn có thể chưa chỉ ra được

to put your finger on exactly what's wrong, but give it time and your doubts will probably crystallize into something more solid, something you can address. Let your instincts contribute to your performance.

chính xác cái gì không ổn, nhưng cứ cho nó thời gian, rồi những nghi ngại của bạn sẽ kết tinh lại thành một điều gì đó cụ thể hơn, một điều bạn có thể xử lý được. Hãy để bản năng góp phần vào hiệu suất của bạn.

The other problem is a little more prosaic: you might simply be afraid that you'll make a mistake.

Vấn đề còn lại thì có phần trần trụi hơn: đơn giản là bạn có thể đang sợ mình sẽ phạm sai lầm.

And that's a reasonable fear. We developers put a lot of ourselves into our code; we can take errors in that code as reflections on our competence. Perhaps there's an element of imposter syndrome, too; we may think that this project is beyond us. We can't see our way through to the end; we'll get so far and then be forced to admit that we're lost.

Và đó là một nỗi sợ hợp lý. Cánh lập trình viên chúng ta gửi gắm rất nhiều bản thân mình vào mã; ta có thể coi những lỗi trong mã ấy như phản ánh năng lực của chính mình. Có lẽ ở đây còn có chút hội chứng kẻ mạo danh (imposter syndrome) nữa; ta có thể nghĩ rằng dự án này vượt quá sức mình. Ta chẳng thấy được lối đi nào đến tận cùng; rồi ta sẽ làm được đến một mức nào đó để rồi buộc phải thừa nhận là mình đã lạc lối.

Fighting Yourself — Khi Bạn Đánh Vật Với Chính Mình

Sometimes code just flies from your brain into the editor: ideas become bits with seemingly no effort.

Có những lúc mã cứ bay vù vù từ não bạn vào trình soạn thảo: ý tưởng hóa thành những con bit mà tưởng như chẳng tốn chút công sức nào.

Other days, coding feels like walking uphill in mud. Taking each step requires tremendous effort, and every three steps you slide back two.

Có những ngày khác, viết mã lại như lội bùn leo dốc. Mỗi bước đi đòi hỏi nỗ lực khủng khiếp, và cứ ba bước tiến thì lại tụt lại hai.

But, being a professional, you soldier on, taking step after muddy step: you have a job to do. Unfortunately, that's probably the exact opposite of what you should do.

Nhưng, vì là dân chuyên nghiệp, bạn vẫn gồng mình tiến tới, bước thêm từng bước lầy lội: bạn có việc phải làm mà. Đáng tiếc thay, đó có lẽ lại là điều hoàn toàn ngược với những gì bạn nên làm.

Your code is trying to tell you something. It's saying that this is harder than it should be. Maybe the structure or design is wrong, maybe you're solving the wrong problem, or maybe you're just creating an ant farm's worth of bugs. Whatever the reason, your lizard brain is sensing feedback from the code, and it's desperately trying to get you to listen.

Mã của bạn đang cố nói với bạn điều gì đó. Nó đang nói rằng việc này khó hơn mức đáng lẽ phải có. Có thể cấu trúc hay thiết kế đã sai, có thể bạn đang giải sai bài toán, hoặc có thể bạn đang gây dựng cả một trại kiến đầy lỗi. Dù lý do là gì, bộ não bò sát của bạn đang cảm nhận được phản hồi từ mã, và nó đang tuyệt vọng cố khiến bạn chịu lắng nghe.

How To Talk Lizard — Cách Trò Chuyện Với Con Bò Sát

We talk a lot about listening to your instincts, to your nonconscious, lizard brain. The techniques are always the same.

Chúng tôi nói rất nhiều về việc lắng nghe bản năng, lắng nghe bộ não vô thức, bộ não bò sát của bạn. Các kỹ thuật thì lúc nào cũng vậy.

Tip 61: Listen to Your Inner Lizard

Tip 61: Hãy Lắng Nghe Con Bò Sát Bên Trong Bạn

First, stop what you're doing. Give yourself a little time and space to let your brain organize itself. Stop thinking about the code, and do something that is fairly mindless for a while, away from a keyboard. Take a walk, have lunch, chat with someone. Maybe sleep on it. Let the ideas percolate up through the layers of your brain on their own: you can't force it. Eventually they may bubble up to the conscious level, and you have one of those a ha! moments.

Trước hết, hãy dừng việc bạn đang làm. Cho bản thân chút thời gian và không gian để bộ não tự sắp xếp lại. Ngừng nghĩ về mã, và làm một việc gì đó khá vô thưởng vô phạt trong chốc lát, tránh xa bàn phím. Đi dạo, ăn trưa, tán gẫu với ai đó. Có khi cứ để qua một giấc ngủ. Hãy để các ý tưởng tự ngấm dần lên qua các tầng lớp của bộ não: bạn không thể ép được. Rồi cuộc chúng có thể sủi lên tới tầng ý thức, và thế là bạn có một trong những khoảnh khắc à há!

If that's not working, try externalizing the issue. Make doodles about the code you're writing, or explain it to a coworker (preferably one who isn't a programmer), or to your rubber duck. Expose different parts of your brain to the issue, and see if any of them have a better handle on the thing that's troubling you. We've lost track of the number of conversations we've had where one of us was explaining a problem to the other and suddenly went "Oh! Of course!" and broke off to fix it.

Nếu cách đó không ăn thua, hãy thử đưa vấn đề ra ngoài. Vẽ vài hình nguệch ngoạc về đoạn mã bạn đang viết, hoặc giải thích nó cho một đồng nghiệp (tốt nhất là người không phải lập trình viên), hoặc cho con vịt cao su của bạn. Hãy phơi bày vấn đề ra cho những phần khác nhau của bộ não, và xem liệu có phần nào nắm bắt được cái điều đang làm bạn rối trí hay không. Chúng tôi đã chẳng còn đếm xuể số cuộc trò chuyện mà một trong hai chúng tôi đang giải thích một vấn đề cho người kia thì bỗng "Ồ! Tất nhiên rồi!" và cắt ngang để chạy đi sửa.

But maybe you've tried these things, and you're still stuck. It's time for action. We need to tell your brain that what you're about to do doesn't matter. And we do that by prototyping.

Nhưng có khi bạn đã thử mấy thứ này rồi mà vẫn tắc tịt. Đã đến lúc phải hành động. Chúng ta cần nói cho bộ não bạn biết rằng cái việc bạn sắp làm chẳng quan trọng gì cả. Và ta làm điều đó bằng cách dựng bản mẫu (prototype).

It'S Playtime! — Đến Giờ Chơi Rồi!

Andy and Dave have both spent hours looking at empty editor buffers. We'll type in some code, then look at the ceiling, then get yet another drink, then type in some more code, then go

Andy và Dave đều đã từng ngồi hàng giờ nhìn chăm chăm vào những vùng soạn thảo trống không. Chúng tôi gõ vào ít mã, rồi ngược nhìn trần nhà, rồi đi rót thêm cốc nước nữa, rồi gõ thêm ít mã, rồi đi

read a funny story about a cat with two tails, then type some more code, then do select-all/delete and start again. And again. And again.

đọc một mẩu chuyện vui về con mèo có hai cái đuôi, rồi gõ thêm ít mã nữa, rồi chọn-tất-cả/xóa và bắt đầu lại. Lại lần nữa. Và lại lần nữa.

And over the years we've found a brain hack that seems to work. Tell yourself you need to **prototype** something. If you're facing a blank screen, then look for some aspect of the project that you want to explore. Maybe you're using a new framework, and want to see how it does data binding. Or maybe it's a new algorithm, and you want to explore how it works on edge cases. Or maybe you want to try a couple of different styles of user interaction.

Và qua bao năm tháng, chúng tôi đã tìm ra một mẹo lừa bộ não có vẻ hữu hiệu. Hãy tự nhủ rằng bạn cần dựng bản mẫu cho một thứ gì đó. Nếu bạn đang đối diện với màn hình trống, hãy tìm một khía cạnh nào đó của dự án mà bạn muốn khám phá. Có thể bạn đang dùng một framework mới và muốn xem nó liên kết dữ liệu (data binding) thế nào. Hoặc có thể đó là một thuật toán mới, và bạn muốn khám phá xem nó hoạt động ra sao với các trường hợp biên. Hoặc có thể bạn muốn thử vài kiểu tương tác người dùng khác nhau.

If you're working on existing code and it's pushing back, then stash it away somewhere and prototype up something similar instead.

Nếu bạn đang làm với mã sẵn có và nó cứ chống cự lại, thì hãy cất nó đi đâu đó rồi dựng bản mẫu cho một thứ tương tự để thay thế.

Do the following.

Hãy làm như sau.

1. Write "I'm prototyping" on a sticky note, and stick it on the side of your screen. 2. Remind yourself that prototypes are meant to fail. And remind yourself that prototypes get thrown away, even if they don't fail. There is no downside to doing this. 3. In your empty editor buffer, create a comment describing in one sentence what you want to learn or do. 4. Start coding.

1. Viết "Tôi đang dựng bản mẫu" lên một mẫu giấy nhớ, rồi dán nó vào cạnh màn hình.
2. Tự nhắc mình rằng bản mẫu sinh ra là để thất bại. Và tự nhắc rằng bản mẫu rồi sẽ bị vứt đi, kể cả khi nó không thất bại. Làm vậy thì chẳng mất mát gì. 3. Trong vùng soạn thảo trống của bạn, tạo một dòng chú thích mô tả trong một câu điều bạn muốn học hay muốn làm. 4. Bắt đầu viết mã.

If you start to have doubts, look at the sticky note.

Nếu bạn bắt đầu nảy sinh nghi ngại, hãy nhìn vào mẫu giấy nhớ.

If, in the middle of coding, that nagging doubt suddenly crystallizes into a solid concern, then address it.

Nếu, ngay giữa lúc viết mã, mối nghi ngại day dứt ấy bỗng kết tinh thành một nỗi lo cụ thể, thì hãy xử lý nó.

If you get to the end of the experiment and you still feel uneasy, start again with the walk and the talk and the time off.

Nếu đến cuối cuộc thử nghiệm mà bạn vẫn thấy bất an, hãy bắt đầu lại từ chuyện đi dạo, trò chuyện và nghỉ ngơi.

But, in our experience, at some point during the first prototype you will be surprised to find yourself humming along with your music, enjoying the feeling of creating code. The nervousness will have evaporated, replaced by a feeling of urgency: let's get this done!

Nhưng, theo kinh nghiệm của chúng tôi, đến một lúc nào đó trong bản mẫu đầu tiên, bạn sẽ ngạc nhiên thấy mình ngân nga theo điệu nhạc, tận hưởng cảm giác tạo ra mã. Sự bồn chồn lúc ấy đã bốc hơi, thay vào đó là một cảm giác cấp bách: làm cho xong cái này nào!

At this stage, you know what to do. Delete all the prototype code, throw away the sticky note, and fill that empty editor buffer with bright, shiny new code.

Đến giai đoạn này, bạn biết phải làm gì rồi. Xóa hết mã bản mẫu, vứt mẫu giấy nhớ đi, và lấp đầy cái vùng soạn thảo trống ấy bằng những dòng mã mới sáng bóng tinh tươm.

Not Just Your Code — Không Chỉ Mã Của Bạn

A large part of our job is dealing with existing code, often written by other people. Those people will have different instincts to you, and so the decisions they made will be different. Not necessarily worse; just different.

Một phần lớn công việc của chúng ta là làm việc với mã sẵn có, thường do người khác viết. Những người ấy có bản năng khác bạn, và vì thế những quyết định họ đưa ra cũng sẽ khác. Không nhất thiết là tệ hơn; chỉ là khác đi.

You can read their code mechanically, slogging through it making notes on stuff that seems important. It's a chore, but it works.

Bạn có thể đọc mã của họ một cách máy móc, liệt kê qua đó và ghi chú lại những thứ có vẻ quan trọng. Đó là một việc nhàm chán, nhưng có tác dụng.

Or you can try an experiment. When you spot things done in a way that seems strange, jot it down. Continue doing this, and look for patterns. If you can see what drove them to write code that way, you may find that the job of understanding it becomes a lot easier. You'll be able consciously to apply the patterns that they applied tacitly.

Hoặc bạn có thể thử một cuộc thử nghiệm. Khi bạn bắt gặp những thứ được làm theo một cách có vẻ kỳ quặc, hãy ghi nhanh lại. Cứ tiếp tục như vậy, và tìm xem có khuôn mẫu nào không. Nếu bạn nhìn ra được điều gì đã thúc họ viết mã theo cách ấy, bạn có thể thấy việc hiểu nó trở nên dễ dàng hơn nhiều. Bạn sẽ có thể áp dụng một cách có ý thức những khuôn mẫu mà họ đã áp dụng một cách ngầm định.

And you might just learn something new along the way.

Và biết đâu trên đường đi bạn lại học được điều gì đó mới mẻ.

Not Just Code — Không Chỉ Mã

Learning to listen to your gut when coding is an important skill to foster. But it applies to the bigger picture as well. Sometimes a design just feels wrong, or some requirement makes you feel uneasy. Stop and analyze these feelings. If you're in a supportive environment, express them out loud. Explore them. The chances are that there's something lurking in that dark doorway. Listen to your instincts and avoid the problem before it jumps out at you.

Học cách lắng nghe trực giác khi viết mã là một kỹ năng quan trọng cần nuôi dưỡng. Nhưng nó còn áp dụng được cho cả bức tranh lớn hơn. Đôi khi một thiết kế cứ có cảm giác sai sai, hoặc một yêu cầu nào đó khiến bạn thấy bất an. Hãy dừng lại và phân tích những cảm giác đó. Nếu bạn đang ở trong một môi trường biết lắng nghe, hãy nói chúng ra thành lời. Hãy khám phá chúng. Nhiều khả năng có điều gì đó đang lẩn khuất trong khung cửa tối ấy. Hãy lắng nghe bản năng của bạn và né được vấn đề trước khi nó vọt ra chộp lấy bạn.

Related Sections Include — Các mục liên quan

Topic 13, Prototypes and Post-it Notes

Mục 13, Bản Mẫu và Giấy Nhớ Post-it

Topic 22, Engineering Daybooks

Mục 22, Nhật Ký Kỹ Thuật

Topic 46, Solving Impossible Puzzles

Mục 46, Giải Những Câu Đố Bất Khả

Challenges — Thử thách

Is there something you know you should do, but have put off because it feels a little scary, or difficult? Apply the techniques in this section. Time box it to an hour, maybe two, and promise yourself that when the bell rings you'll delete what you did. What did you learn?

Có điều gì bạn biết là mình nên làm, nhưng cứ trì hoãn mãi vì nó hơi đáng sợ, hay hơi khó không? Hãy áp dụng các kỹ thuật trong mục này. Khoanh thời gian lại trong một giờ, có lẽ hai giờ, và hứa với bản thân rằng khi chuông reo bạn sẽ xóa hết những gì mình đã làm. Bạn đã học được điều gì?

Topic 38. Programming by Coincidence — Topic 38. Lập trình theo may rủi

Do you ever watch old black-and-white war movies? The weary soldier advances cautiously out of the brush. There's a clearing ahead: are there any land mines, or is it safe to cross? There aren't any indications that it's a minefield—no signs, barbed wire, or craters. The soldier pokes the ground ahead of him with his bayonet and winces, expecting an explosion. There isn't one. So he proceeds painstakingly through the field for a while, prodding and poking as he goes. Eventually, convinced that the field is safe, he straightens up and marches proudly forward, only to be blown to pieces.

Bạn có bao giờ xem mấy bộ phim chiến tranh đen trắng cũ chưa? Người lính mệt mỏi thận trọng tiến ra khỏi bụi rậm. Phía trước là một khoảng đất trống: liệu có mìn không, hay băng qua thì an toàn? Chẳng có dấu hiệu nào cho thấy đó là bãi mìn — không biển báo, không dây thép gai, không hố bom. Người lính lấy lưỡi lê chọc xuống đất phía trước rồi nhả mặt, chờ đợi một tiếng nổ. Chẳng có gì cả. Thế là anh ta nhọc nhằn tiến qua bãi đất một hồi, vừa đi vừa chọc, vừa dò. Cuối cùng, tin chắc rằng bãi đất an toàn, anh ta đứng thẳng dậy và hiên ngang sải bước về phía trước — để rồi bị thổi tung thành nhiều mảnh.

The soldier's initial probes for mines revealed nothing, but this was merely lucky. He was led to a false conclusion—with disastrous results.

Những lần dò mìn ban đầu của người lính chẳng phát hiện ra gì, nhưng đó chỉ thuần túy là may mắn. Anh ta bị dẫn tới một kết luận sai lầm — với hậu quả thảm khốc.

As developers, we also work in minefields. There are hundreds of traps waiting to catch us each day. Remembering the soldier's tale, we should be wary of drawing false conclusions. We should avoid **programming by coincidence**—relying on luck and accidental successes—in favor of programming deliberately.

Là lập trình viên, chúng ta cũng làm việc trong những bãi mìn. Mỗi ngày có hàng trăm cái bẫy chờ tóm lấy chúng ta. Nhớ tới câu chuyện người lính, chúng ta nên cảnh giác với việc rút ra những kết luận sai lầm. Chúng ta nên tránh lập trình theo may rủi

(programming by coincidence) — dựa dẫm vào may mắn và những thành công tình cờ — mà thay vào đó hãy lập trình một cách có chủ đích.

How To Program By Coincidence — Cách lập trình theo may rủi

Suppose Fred is given a programming assignment. Fred types in some code, tries it, and it seems to work. Fred types in some more code, tries it, and it still seems to work. After several weeks of coding this way, the program suddenly stops working, and after hours of trying to fix it, he still doesn't know why. Fred

Giả sử Fred được giao một bài tập lập trình. Fred gõ vào ít mã, chạy thử, và nó có vẻ chạy được. Fred gõ thêm ít mã nữa, chạy thử, và nó vẫn có vẻ chạy được. Sau vài tuần code theo kiểu này, chương trình đột nhiên ngừng hoạt động, và sau hàng giờ cố sửa, anh ta vẫn không biết tại sao. Fred

may well spend a significant amount of time chasing this piece of code around without ever being able to fix it. No matter what he does, it just doesn't ever seem to work right.

rất có thể sẽ tốn một lượng thời gian đáng kể để rượt theo đoạn mã này khắp nơi mà chẳng bao giờ sửa được nó. Bất kể anh ta làm gì, nó cũng cứ trục trặc không tài nào chạy đúng.

Fred doesn't know why the code is failing because he didn't know why it worked in the first place. It seemed to work, given the limited “testing” that Fred did, but that was just a coincidence. Buoyed by false confidence, Fred charged ahead into oblivion. Now, most intelligent people may know someone like Fred, but we know better. We don't rely on coincidences— do we?

Fred không biết tại sao đoạn mã bị lỗi vì ngay từ đầu anh ta đã chẳng biết tại sao nó chạy được. Nó có vẻ chạy được, với mấy phép “kiểm thử” hạn chế mà Fred làm, nhưng đó chỉ là sự trùng hợp. Được tiếp sức bởi sự tự tin hão, Fred xông thẳng tới vực thẳm. Giờ thì, hầu hết những người thông minh hẳn đều quen biết một ai đó giống Fred, nhưng chúng ta thì khôn hơn. Chúng ta đâu có dựa vào sự may rủi — đúng không?

Sometimes we might. Sometimes it can be pretty easy to confuse a happy coincidence with a purposeful plan. Let's look at a few examples.

Đôi khi thì có đấy. Đôi khi rất dễ nhầm lẫn giữa một sự trùng hợp may mắn với một kế hoạch có chủ đích. Hãy cùng xem vài ví dụ.

Accidents of Implementation Accidents of implementation are things that happen simply because that's the way the code is currently written. You end up relying on undocumented error or boundary conditions.

Tai nạn do cách triển khai (Accidents of Implementation) Tai nạn do cách triển khai là những thứ xảy ra đơn giản chỉ vì hiện tại mã được viết theo kiểu đó. Bạn rốt cuộc lại dựa dẫm vào những điều kiện lỗi hoặc điều kiện biên không hề được ghi lại trong tài liệu.

Suppose you call a routine with bad data. The routine responds in a particular way, and you code based on that response. But the author didn't intend for the routine to work that way—it was never even considered. When the routine gets “fixed,” your code may break. In the most extreme case, the routine you called may not even be designed to do what you want, but it seems to work okay. Calling things in the wrong order, or in the wrong **context**, is a related problem.

Giả sử bạn gọi một thủ tục với dữ liệu xấu. Thủ tục phản hồi theo một cách nhất định, và bạn viết mã dựa trên phản hồi đó. Nhưng tác giả không hề có ý định để thủ tục hoạt động theo kiểu ấy — họ thậm chí chưa bao giờ tính đến nó. Khi thủ tục được “sửa”, mã của bạn có thể vỡ. Trong trường hợp cực đoan nhất, thủ tục bạn gọi thậm chí không hề được thiết

kế để làm việc bạn muốn, nhưng nó có vẻ vẫn ổn. Gọi mọi thứ sai thứ tự, hoặc sai ngữ cảnh, là một vấn đề có liên quan.

Here it looks like Fred is desperately trying to get something out on the screen using some particular GUI rendering framework:

Ở đây có vẻ Fred đang tuyệt vọng cố đẩy thứ gì đó lên màn hình bằng một framework dựng giao diện (GUI rendering framework) cụ thể nào đó:

```
paint(); invalidate(); validate(); revalidate(); repaint(); paintImmediately();
```

```
paint(); invalidate(); validate(); revalidate(); repaint(); paintImmediately();
```

But these routines were never designed to be called this way; although they seem to work, that's really just a coincidence.

Nhưng những thủ tục này không hề được thiết kế để gọi theo kiểu đó; dù chúng có vẻ chạy được, thì thực ra đó chỉ là sự trùng hợp.

To add insult to injury, when the scene finally does get drawn, Fred won't try to go back and take out the spurious calls. "It works now, better leave well enough alone...."

Tệ hại hơn nữa, khi cảnh cuối cùng cũng được vẽ ra, Fred sẽ chẳng buồn quay lại bỏ bớt mấy lời gọi thừa thãi đó. "Giờ nó chạy rồi, tốt nhất là đừng động vào nữa..."

It's easy to be fooled by this line of thought. Why should you take the risk of messing with something that's working? Well, we can think of several reasons:

Rất dễ bị lối suy nghĩ này đánh lừa. Tại sao bạn lại phải mạo hiểm nghịch ngợm một thứ đang chạy ngon lành? Chà, chúng tôi có thể nghĩ ra vài lý do:

It may not really be working—it might just look like it is.

Có thể nó không thực sự chạy được — chỉ là trông như vậy thôi.

The boundary condition you rely on may be just an accident. In different circumstances (a different screen resolution, more CPU cores), it might behave differently.

Điều kiện biên mà bạn dựa vào có thể chỉ là một sự ngẫu nhiên. Trong hoàn cảnh khác (độ phân giải màn hình khác, nhiều nhân CPU hơn), nó có thể hành xử khác đi.

Undocumented behavior may change with the next release of the library.

Hành vi không được ghi trong tài liệu có thể thay đổi ở bản phát hành kế tiếp của thư viện.

Additional and unnecessary calls make your code slower.

Những lời gọi thêm và không cần thiết khiến mã của bạn chạy chậm hơn.

Additional calls increase the risk of introducing new bugs of their own.

Những lời gọi thêm làm tăng nguy cơ tự gây ra các lỗi mới.

For code you write that others will call, the basic principles of good modularization and of hiding implementation behind small, well-documented interfaces can all help. A well-specified contract (see **Topic 23, Design by Contract**) can help eliminate misunderstandings.

Với mã bạn viết để người khác gọi, các nguyên tắc cơ bản về module hóa tốt và về việc che giấu cách triển khai sau những giao diện nhỏ, được ghi tài liệu rõ ràng đều có thể giúp ích. Một hợp đồng được đặc tả tốt (xem Topic 23, Thiết kế theo hợp đồng) có thể giúp loại bỏ những hiểu lầm.

For routines you call, rely only on documented behavior. If you can't, for whatever reason, then document your assumption well.

Với các thủ tục bạn gọi, chỉ dựa vào hành vi đã được ghi trong tài liệu. Nếu vì lý do nào đó mà bạn không thể, thì hãy ghi lại giả định của mình cho thật rõ.

Close Enough Isn't We once worked on a large project that reported on data fed from a very large number of hardware data collection units out in the field. These units spanned states and time zones, and for various logistical and historical reasons, each unit was set to local time. [50] As a result of conflicting time zone interpretations and inconsistencies in Daylight Savings Time policies, results were almost always wrong, but only off by one. The developers on the project had gotten into the habit of just adding one or subtracting one to get the correct answer, reasoning that it was only off by one in this one situation. And then the next function would see the value as off by the one the other way, and change it back.

Gần đúng thì vẫn chưa đúng (Close Enough Isn't) Chúng tôi từng làm một dự án lớn chuyên báo cáo dữ liệu lấy từ rất nhiều thiết bị thu thập dữ liệu phần cứng đặt ngoài hiện trường. Các thiết bị này trải khắp nhiều bang và nhiều múi giờ, và vì đủ thứ lý do hậu cần lẫn lịch sử, mỗi thiết bị lại được đặt theo giờ địa phương. [50] Kết quả là, do cách diễn giải múi giờ mâu thuẫn nhau và do các chính sách giờ mùa hè (Daylight Savings Time) thiếu nhất quán, kết quả gần như lúc nào cũng sai, nhưng chỉ lệch đúng một đơn vị. Các lập trình viên trong dự án đã đâm ra quen thói cứ cộng thêm một hoặc trừ đi một để ra đáp án đúng, với lý lẽ rằng trong tình huống này nó chỉ lệch có một mà thôi. Rồi hàm kế tiếp lại thấy giá trị bị lệch một theo chiều ngược lại, và đổi nó về như cũ.

But the fact that it was “only” off by one some of the time was a coincidence, masking a deeper and more fundamental flaw. Without a proper model of time handling, the entire large code base had devolved over time to an untenable mass of +1 and -1 statements. Ultimately, none of it was correct and the project was scrapped.

Nhưng việc “chỉ” lệch một đơn vị trong một số trường hợp lại chính là một sự trùng hợp, che giấu một khiếm khuyết sâu xa và căn bản hơn. Không có một mô hình xử lý thời gian đáng hoàng, cả cái kho mã đồ sộ ấy theo thời gian đã thoái hóa thành một mớ +1 và -1 không tài nào chống đỡ nổi. Rốt cuộc, chẳng có chỗ nào đúng cả, và dự án bị vứt bỏ.

Phantom Patterns Human beings are designed to see patterns and causes, even when it's just a coincidence. For example, Russian leaders always alternate between being bald and hairy: a bald (or obviously balding) **state** leader of Russia has succeeded a nonbald (“hairy”) one, and vice versa, for nearly 200 years. [51]

Những khuôn mẫu ma (Phantom Patterns) Con người được tạo ra để nhìn thấy khuôn mẫu và nguyên nhân, ngay cả khi đó chỉ là sự trùng hợp. Ví dụ, các nhà lãnh đạo Nga luôn xen kẽ nhau giữa hói và có tóc: suốt gần 200 năm, một nguyên thủ Nga hói (hoặc hói rõ rệt) lại kế nhiệm một người không hói (“có tóc”), và ngược lại. [51]

But while you wouldn't write code that depended on the next

Nhưng trong khi bạn sẽ chẳng viết mã phụ thuộc vào việc vị lãnh đạo Nga

Russian leader being bald or hairy, in some domains we think that way all the time. Gamblers imagine patterns in lottery numbers, dice games, or roulette, when in fact these are statistically independent events. In finance, stock and bond trading are similarly rife with coincidence instead of actual, discernible patterns.

kế tiếp hói hay có tóc, thì ở một số lĩnh vực chúng ta lại suy nghĩ theo kiểu đó suốt. Dân cờ bạc tưởng tượng ra các khuôn mẫu trong số xổ số, trong các ván xúc xắc, hay trong cò

quay roulette, trong khi thực ra đó là những sự kiện độc lập về mặt thống kê. Trong tài chính, việc giao dịch cổ phiếu và trái phiếu cũng đầy rẫy những sự trùng hợp thay vì các khuôn mẫu thực sự, có thể nhận biết được.

A log file that shows an intermittent error every 1,000 requests may be a difficult-to-diagnose **race condition**, or may be a plain old bug. Tests that seem to pass on your machine but not on the server might indicate a difference between the two environments, or maybe it's just a coincidence.

Một tệp log thể hiện một lỗi chập chờn cứ mỗi 1.000 yêu cầu có thể là một điều kiện tranh đua (race condition) khó chẩn đoán, mà cũng có thể chỉ là một con bug bình thường cũ rích. Các bài kiểm thử có vẻ pass trên máy bạn nhưng lại không pass trên server có thể cho thấy một khác biệt giữa hai môi trường, mà cũng có thể chỉ là sự trùng hợp.

Don't assume it, prove it.

Đừng giả định, hãy chứng minh.

Accidents of Context You can have “accidents of context” as well. Suppose you are writing a utility module. Just because you are currently coding for a GUI environment, does the module have to rely on a GUI being present? Are you relying on English-speaking users? Literate users? What else are you relying on that isn't guaranteed?

Tai nạn do ngữ cảnh (Accidents of Context) Bạn cũng có thể gặp “tai nạn do ngữ cảnh”. Giả sử bạn đang viết một module tiện ích. Chỉ vì hiện tại bạn đang code cho một môi trường giao diện đồ họa, liệu module đó có nhất thiết phải dựa vào việc có sẵn một giao diện đồ họa không? Bạn có đang dựa vào việc người dùng nói tiếng Anh không? Người dùng biết chữ? Bạn còn đang dựa vào những gì khác mà không hề được bảo đảm?

Are you relying on the current directory being writable? On certain environment variables or **configuration** files being present? On the time on the server being accurate—within what tolerance? Are you relying on network availability and speed?

Bạn có đang dựa vào việc thư mục hiện hành có quyền ghi không? Vào việc một số biến môi trường hay tệp cấu hình nhất định có mặt? Vào việc thời gian trên server chính xác — chính xác trong khoảng dung sai nào? Bạn có đang dựa vào sự sẵn sàng và tốc độ của mạng không?

When you copied code from the first answer you found on the net, are you sure your context is the same? Or are you building “**cargo cult**” code, merely imitating form without content? [52]

Khi bạn chép đoạn mã từ câu trả lời đầu tiên tìm thấy trên mạng, bạn có chắc ngữ cảnh của mình cũng y như vậy không? Hay bạn đang dựng nên thứ mã “sùng bái hàng hóa” (cargo cult), chỉ bắt chước cái vỏ mà không có cái ruột? [52]

Finding an answer that happens to fit is not the same as the

Tìm được một đáp án tình cờ vừa khít không giống với việc tìm được
right answer.

đáp án đúng.

Tip 62: Don't Program by Coincidence

Tip 62: Đừng Lập Trình Theo May Rủi

Implicit Assumptions Coincidences can mislead at all levels—from generating **requirements** through to testing. Testing is particularly fraught with false causalities and **coincidental** outcomes. It's easy to assume that X causes Y, but as we said in Topic 20, **Debugging**: don't assume it, prove it.

Những giả định ngầm (Implicit Assumptions) Sự trùng hợp có thể đánh lừa ở mọi cấp độ — từ khâu sinh ra yêu cầu cho tới khâu kiểm thử. Kiểm thử đặc biệt đầy rẫy những quan hệ nhân quả giả và những kết cục tình cờ. Rất dễ giả định rằng X gây ra Y, nhưng như chúng tôi đã nói ở Topic 20, Gỡ lỗi: đừng giả định, hãy chứng minh.

At all levels, people operate with many assumptions in mind— but these assumptions are rarely documented and are often in conflict between different developers. Assumptions that aren't based on well-established facts are the bane of all projects.

Ở mọi cấp độ, người ta làm việc với vô số giả định trong đầu — nhưng những giả định này hiếm khi được ghi lại và thường mâu thuẫn nhau giữa các lập trình viên khác nhau. Những giả định không dựa trên các sự thật đã được xác lập vững chắc chính là tai họa của mọi dự án.

How To Program Deliberately — Cách lập trình có chủ đích

We want to spend less time churning out code, catch and fix errors as early in the development cycle as possible, and create fewer errors to begin with. It helps if we can **program deliberately**:

Chúng ta muốn dành ít thời gian hơn cho việc nhả ra mã, bắt và sửa lỗi càng sớm càng tốt trong chu trình phát triển, và ngay từ đầu tạo ra ít lỗi hơn. Sẽ hữu ích nếu ta có thể lập trình một cách có chủ đích:

Always be aware of what you are doing. Fred let things get slowly out of hand, until he ended up boiled, like the frog here .

Luôn ý thức được mình đang làm gì. Fred đã để mọi thứ từ từ vượt khỏi tầm tay, cho tới khi rốt cuộc bị luộc chín, như con ếch ở đây.

Can you explain the code, in detail, to a more junior programmer? If not, perhaps you are relying on coincidences.

Bạn có thể giải thích đoạn mã, một cách chi tiết, cho một lập trình viên non tay hơn không? Nếu không, có lẽ bạn đang dựa vào những sự trùng hợp.

Don't code in the dark. Build an application you don't fully grasp, or use a technology you don't understand, and you'll likely be bitten by coincidences. If you're not sure why it works, you won't know why it fails.

Đừng code trong bóng tối. Hãy xây một ứng dụng mà bạn không hiểu thấu đáo, hoặc dùng một công nghệ bạn không hiểu, và bạn rất có thể sẽ bị cắn bởi sự trùng hợp ngẫu nhiên. Nếu bạn không chắc tại sao nó chạy, bạn sẽ không biết tại sao nó hỏng.

Proceed from a plan, whether that plan is in your head, on the back of a cocktail napkin, or on a whiteboard.

Hãy xuất phát từ một kế hoạch, dù kế hoạch đó nằm trong đầu bạn, trên mặt sau một tấm khăn giấy quán bar, hay trên một tấm bảng trắng.

Rely only on reliable things. Don't depend on assumptions. If you can't tell if something is reliable, assume the worst.

Chỉ dựa vào những thứ đáng tin cậy. Đừng phụ thuộc vào các giả định. Nếu bạn không thể biết một thứ có đáng tin hay không, hãy giả định điều tệ nhất.

Document your assumptions. Topic 23, Design by Contract , can help clarify your assumptions in your own mind, as well as help communicate them to others.

Hãy ghi lại các giả định của bạn. Topic 23, Thiết kế theo hợp đồng, có thể giúp làm rõ các giả định ngay trong đầu bạn, cũng như giúp truyền đạt chúng cho người khác.

Don't just test your code, but test your assumptions as well. Don't guess; actually try it. Write an **assertion** to test your assumptions (see Topic 25, **Assertive Programming**). If your assertion is right, you have improved the documentation in your code. If you discover your assumption is wrong, then count yourself lucky.

Đừng chỉ kiểm thử mã của bạn, mà hãy kiểm thử cả các giả định của bạn nữa. Đừng đoán; hãy thực sự thử nó. Hãy viết một assertion để kiểm tra các giả định của bạn (xem Topic 25, Lập trình phòng vệ bằng assertion). Nếu assertion của bạn đúng, bạn đã cải thiện được tài liệu trong mã của mình. Còn nếu bạn phát hiện giả định của mình sai, thì hãy tự coi mình là người may mắn.

Prioritize your effort. Spend time on the important aspects; more than likely, these are the hard parts. If you don't have fundamentals or infrastructure correct, brilliant bells and whistles will be irrelevant.

Hãy ưu tiên công sức. Dành thời gian cho những khía cạnh quan trọng; nhiều khả năng đó chính là những phần khó. Nếu bạn chưa làm đúng phần nền tảng hay hạ tầng, thì những thứ chuông khánh hào nhoáng cũng chẳng có nghĩa lý gì.

Don't be a slave to history. Don't let existing code dictate future code. All code can be replaced if it is no longer appropriate. Even within one program, don't let what you've already done constrain what you do next—be ready to refactor (see Topic 40, **Refactoring**). This decision may impact the project schedule. The assumption is that the impact will be less than the cost of not making the change. [53]

Đừng làm nô lệ cho lịch sử. Đừng để mã hiện có quy định mã tương lai. Mọi đoạn mã đều có thể bị thay thế nếu nó không còn phù hợp. Ngay cả trong một chương trình, đừng để những gì bạn đã làm trói buộc những gì bạn làm tiếp theo — hãy sẵn sàng tái cấu trúc (xem Topic 40, Tái cấu trúc). Quyết định này có thể tác động tới lịch trình dự án. Giả định ở đây là tác động đó sẽ nhỏ hơn cái giá của việc không thực hiện thay đổi. [53]

So next time something seems to work, but you don't know why, make sure it isn't just a coincidence.

Vậy nên, lần tới khi một thứ gì đó có vẻ chạy được, nhưng bạn không biết tại sao, hãy chắc chắn rằng đó không chỉ là một sự trùng hợp.

Related Sections Include — Các mục liên quan

Topic 4, **Stone Soup and Boiled Frogs**

Topic 4, Súp đá và Những con ếch bị luộc

Topic 9, **DRY**—The Evils of **Duplication**

Topic 9, DRY — Cái ác của Sự trùng lặp

Topic 23, Design by Contract

Topic 23, Thiết kế theo hợp đồng

Topic 34, **Shared State Is Incorrect State**

Topic 34, Trạng thái chia sẻ là Trạng thái sai

Topic 43, Stay Safe Out There

Topic 43, Hãy giữ an toàn ở ngoài kia

Exercises — Bài tập

Exercise 25 (possible answer)

Bài tập 25 (đáp án khả dĩ)

A data feed from a vendor gives you an array of tuples representing key-value pairs. The key of DepositAccount will hold a string of the account number in the corresponding value:

Một luồng dữ liệu từ nhà cung cấp trả về cho bạn một mảng các tuple biểu diễn những cặp khóa-giá trị. Khóa DepositAccount sẽ chứa chuỗi số tài khoản trong giá trị tương ứng:

```
[ ... { :DepositAccount , "564-904-143-00" } ... ]
```

```
[ ... { :DepositAccount , "564-904-143-00" } ... ]
```

It worked perfectly in test on the 4-core developer laptops and on the 12-core build machine, but on the production servers running in containers, you keep getting the wrong account numbers. What's going on?

Nó chạy hoàn hảo khi kiểm thử trên các laptop 4 nhân của lập trình viên và trên cỗ máy build 12 nhân, nhưng trên các server sản xuất chạy trong container, bạn cứ liên tục nhận về sai số tài khoản. Chuyện gì đang xảy ra vậy?

Exercise 26 (possible answer)

Bài tập 26 (đáp án khả dĩ)

You're coding an autodialer for voice alerts, and have to manage a database of contact information. The ITU specifies that phone numbers should be no longer than 15 digits, so you store the contact's phone number in a numeric field guaranteed to hold at least 15 digits. You've tested in thoroughly throughout North America and everything seems fine, but suddenly you're getting a rash of complaints from other parts of the world. Why?

Bạn đang code một bộ tự động quay số (autodialer) cho các cảnh báo bằng giọng nói, và phải quản lý một cơ sở dữ liệu thông tin liên hệ. ITU quy định rằng số điện thoại không được dài quá 15 chữ số, nên bạn lưu số điện thoại của liên hệ vào một trường số được bảo đảm chứa được ít nhất 15 chữ số. Bạn đã kiểm thử kỹ lưỡng khắp Bắc Mỹ và mọi thứ có vẻ ổn, nhưng đột nhiên bạn nhận được một loạt than phiền từ các nơi khác trên thế giới. Tại sao?

Exercise 27 (possible answer)

Bài tập 27 (đáp án khả dĩ)

You have written an app that scales up common recipes for a cruise ship dining room that seats 5,000. But you're getting complaints that the conversions aren't precise. You check, and the code uses the conversion formula of 16 cups to a gallon.

Bạn đã viết một ứng dụng nhân khẩu phần các công thức nấu ăn thông dụng cho một phòng ăn trên du thuyền có sức chứa 5.000 người. Nhưng bạn nhận được than phiền rằng các phép quy đổi không chính xác. Bạn kiểm tra, và mã dùng công thức quy đổi 16 cốc (cup) cho một gallon.

That's right, isn't it?

Đúng vậy mà, phải không?

Topic 39. Algorithm Speed — Topic 39. Tốc độ thuật toán

In **Topic 15, Estimating**, we talked about estimating things such as how long it takes to walk across town, or how long a project will take to finish. However, there is another kind of estimating that **Pragmatic** Programmers use almost daily: estimating the resources that algorithms use—time, processor, memory, and so on.

Ở Topic 15, Ước lượng, chúng ta đã bàn về việc ước lượng những thứ như mất bao lâu để đi bộ ngang qua thành phố, hay một dự án sẽ mất bao lâu để hoàn thành. Tuy nhiên, còn một kiểu ước lượng khác mà các Lập trình viên Thực dụng (Pragmatic Programmer) sử dụng gần như hằng ngày: ước lượng (estimating) tài nguyên mà các thuật toán tiêu tốn—thời gian, bộ xử lý, bộ nhớ, v.v.

This kind of estimating is often crucial. Given a choice between two ways of doing something, which do you pick? You know how long your program runs with 1,000 records, but how will it scale to 1,000,000? What parts of the code need optimizing?

Kiểu ước lượng này thường có ý nghĩa sống còn. Khi có hai cách để làm cùng một việc, bạn chọn cách nào? Bạn biết chương trình của mình chạy mất bao lâu với 1.000 bản ghi, nhưng nó sẽ co giãn ra sao với 1.000.000? Phần nào của mã cần tối ưu?

It turns out that these questions can often be answered using common sense, some analysis, and a way of writing approximations called the **Big-O notation**.

Hóa ra những câu hỏi này thường có thể trả lời được bằng lẽ thường, một chút phân tích, và một cách viết các giá trị xấp xỉ gọi là ký hiệu Big-O.

What Do We Mean By Estimating Algorithms? — Ước lượng thuật toán nghĩa là gì?

Most nontrivial algorithms handle some kind of variable input— sorting strings, inverting an matrix, or decrypting a message with an n -bit key. Normally, the size of this input will affect the algorithm: the larger the input, the longer the running time or the more memory used.

Hầu hết các thuật toán không tầm thường đều xử lý một loại đầu vào biến thiên nào đó— sắp xếp chuỗi, nghịch đảo một ma trận, hay giải mã một thông điệp bằng khóa n -bit. Thông thường, kích thước của đầu vào này sẽ ảnh hưởng đến thuật toán: đầu vào càng lớn thì thời gian chạy càng lâu hoặc bộ nhớ tiêu tốn càng nhiều.

If the relationship were always linear (so that the time increased in direct proportion to the value of n), this section wouldn't be important. However, most significant algorithms are not linear. The good news is that many are sublinear. A **binary search**, for

Nếu quan hệ này luôn tuyến tính (tức thời gian tăng tỉ lệ thuận trực tiếp với giá trị đầu vào) thì mục này chẳng có gì quan trọng. Tuy nhiên, hầu hết các thuật toán đáng kể đều không tuyến tính. Tin vui là nhiều thuật toán dưới-tuyến-tính (sublinear). Chẳng hạn, một phép tìm kiếm nhị phân

example, doesn't need to look at every candidate when finding a match. The bad news is that other algorithms are considerably worse than linear; runtimes or memory **requirements** increase far faster than n . An algorithm that takes a minute to **process** ten items may take a lifetime to process 100.

không cần xem xét mọi ứng viên khi tìm phần tử khớp. Tin buồn là có những thuật toán khác tệ hơn tuyến tính rất nhiều; thời gian chạy hay nhu cầu bộ nhớ tăng nhanh hơn n

rất nhiều. Một thuật toán mất một phút để xử lý mười phần tử có thể mất cả đời người để xử lý 100 phần tử.

We find that whenever we write anything containing loops or recursive calls, we subconsciously check the runtime and memory requirements. This is rarely a formal process, but rather a quick confirmation that what we're doing is sensible in the circumstances. However, we sometimes do find ourselves performing a more detailed analysis. That's when Big-O notation comes in handy.

Chúng tôi nhận thấy mỗi khi viết bất cứ thứ gì chứa vòng lặp hay lời gọi đệ quy, mình đều vô thức kiểm tra nhu cầu về thời gian chạy và bộ nhớ. Đây hiếm khi là một quy trình chính thức, mà chỉ là một sự xác nhận chớp nhoáng rằng điều mình đang làm là hợp lý trong hoàn cảnh đó. Tuy nhiên, đôi khi chúng tôi cũng thấy mình phải làm một phân tích chi tiết hơn. Đó là lúc ký hiệu Big-O trở nên hữu dụng.

Big-O Notation — Ký hiệu Big-O

The Big-O notation, written $O()$, is a mathematical way of dealing with approximations. When we write that a particular sort routine sorts records in time, we are simply saying that the worst-case time taken will vary as the square of n . Double the number of records, and the time will increase roughly fourfold. Think of the $O()$ as meaning on the order of.

Ký hiệu Big-O, viết là $O()$, là một cách toán học để xử lý các giá trị xấp xỉ. Khi viết rằng một thường trình sắp xếp nào đó sắp xếp n bản ghi trong thời gian $O(n^2)$, chúng ta đơn giản nói rằng thời gian trong trường hợp xấu nhất sẽ biến thiên theo bình phương của n . Tăng gấp đôi số bản ghi thì thời gian sẽ tăng khoảng gấp bốn. Hãy nghĩ về chữ O như nghĩa là “vào cỡ” (on the order of).

The notation puts an upper bound on the value of the thing we're measuring (time, memory, and so on). If we say a function takes time, then we know that the upper bound of the time it takes will not grow faster than $O()$. Sometimes we come up with fairly complex functions, but because the highest-order term will dominate the value as n increases, the convention is to remove all low-order terms, and not to bother showing any constant multiplying factors:

Ký hiệu này đặt một cận trên cho giá trị của thứ ta đang đo (thời gian, bộ nhớ, v.v.). Nếu nói một hàm mất $O(n^2)$ thời gian, thì ta biết cận trên của thời gian nó tốn sẽ không tăng nhanh hơn n^2 . Đôi khi ta thu được những hàm khá phức tạp, nhưng vì số hạng bậc cao nhất sẽ chiếm ưu thế khi n tăng, nên quy ước là loại bỏ mọi số hạng bậc thấp, và không bận tâm hiển thị bất kỳ hệ số nhân hằng số nào:

This is actually a feature of the notation—one algorithm may be 1,000 times faster than another algorithm, but you won't know it from the notation. Big-O is never going to give you actual numbers for time or memory or whatever: it simply tells you how these values will change as the input changes.

Đây thực ra là một đặc tính của ký hiệu này—một thuật toán có thể nhanh hơn thuật toán khác 1.000 lần, nhưng bạn sẽ không nhận ra điều đó từ ký hiệu. Big-O không bao giờ cho bạn con số thực về thời gian, bộ nhớ hay bất cứ gì: nó chỉ cho bạn biết các giá trị ấy sẽ thay đổi ra sao khi đầu vào thay đổi.

Figure 3, Runtimes of various algorithms shows several common notations you'll come across, along with a graph comparing running times of algorithms in each category. Clearly, things quickly start getting out of hand once we get over $n=10$.

Hình 3, Thời gian chạy của các thuật toán khác nhau trình bày một số ký hiệu thường gặp mà bạn sẽ bắt gặp, kèm một đồ thị so sánh thời gian chạy của các thuật toán trong từng

nhóm. Rõ ràng mọi thứ nhanh chóng vượt khỏi tầm kiểm soát một khi ta vượt qua $O(n^2)$.

For example, suppose you've got a routine that takes one second to process 100 records. How long will it take to process 1,000? If your code is , then it will still take one second. If it's , then you'll probably be waiting about three seconds. will show a linear increase to ten seconds, while an will take some 33 seconds. If you're unlucky enough to have an routine, then sit back for 100 seconds while it does its stuff. And if you're using an exponential algorithm , you might want to make a cup of coffee—your routine should finish in about years. Let us know how the universe ends.

Ví dụ, giả sử bạn có một thường trình mất một giây để xử lý 100 bản ghi. Nó sẽ mất bao lâu để xử lý 1.000? Nếu mã của bạn là $O(1)$ thì vẫn mất một giây. Nếu là $O(\lg n)$ thì có lẽ bạn sẽ chờ khoảng ba giây. $O(n)$ sẽ cho thấy mức tăng tuyến tính lên mười giây, còn $O(n \lg n)$ sẽ mất khoảng 33 giây. Nếu xui xẻo có một thường trình $O(n^2)$ thì hãy ngả lưng chờ 100 giây trong lúc nó làm việc. Và nếu bạn dùng một thuật toán hàm mũ $O(2^n)$, bạn có lẽ nên đi pha một tách cà phê—thường trình của bạn sẽ xong sau khoảng 10^{277} năm. Nhớ báo cho chúng tôi biết vũ trụ kết thúc ra sao nhé.

The notation doesn't apply just to time; you can use it to represent any other resources used by an algorithm. For example, it is often useful to be able to model memory consumption (see the **exercises** for an example).

Ký hiệu này không chỉ áp dụng cho thời gian; bạn có thể dùng nó để biểu diễn bất kỳ tài nguyên nào khác mà một thuật toán sử dụng. Ví dụ, thường rất hữu ích khi mô hình hóa được mức tiêu thụ bộ nhớ (xem phần bài tập để có một ví dụ).

Constant (access element in array, simple statements)

$O(1)$ Hằng số (truy cập phần tử trong mảng, các câu lệnh đơn giản)

Logarithmic (binary search). The base of the logarithm doesn't

$O(\lg n)$ Logarit (tìm kiếm nhị phân). Cơ số của logarit không

matter, so this is equivalent .

quan trọng, nên cách viết này tương đương $O(\log n)$.

Linear (sequential search)

$O(n)$ Tuyến tính (tìm kiếm tuần tự)

Worse than linear, but not much worse. (Average runtime of quicksort, heapsort)

$O(n \lg n)$ Tệ hơn tuyến tính, nhưng không tệ hơn nhiều. (Thời gian chạy trung bình của quicksort, heapsort)

Square law (selection and insertion sorts)

$O(n^2)$ Quy luật bình phương (sắp xếp chọn và sắp xếp chèn)

Cubic (multiplication of two matrices)

$O(n^3)$ Bậc ba (nhân hai ma trận $n \times n$)

Exponential (traveling salesman problem, set partitioning)

$O(2^n)$ Hàm mũ (bài toán người bán hàng rong, phân hoạch tập hợp)

images/big-o.png

images/big-o.png

Figure 3. Runtimes of various algorithms

Hình 3. Thời gian chạy của các thuật toán khác nhau

Common Sense Estimation — Ước lượng bằng lẽ thường

You can estimate the order of many basic algorithms using common sense.

Bạn có thể ước lượng bậc của nhiều thuật toán cơ bản chỉ bằng lẽ thường.

Simple loops If a simple loop runs from a to b , then the algorithm is likely to be $O(b - a)$ —time increases linearly with $b - a$. Examples include exhaustive searches, finding the maximum value in an array, and generating checksums.

Vòng lặp đơn Nếu một vòng lặp đơn chạy từ 1 đến n thì thuật toán nhiều khả năng là $O(n)$ —thời gian tăng tuyến tính theo n . Ví dụ gồm tìm kiếm vét cạn, tìm giá trị lớn nhất trong một mảng, và sinh checksum.

Nested loops If you nest a loop inside another, then your algorithm becomes $O(m \times n)$, where m and n are the two loops' limits. This commonly occurs in simple sorting algorithms, such as bubble sort, where the outer loop scans each element in the array in turn, and the inner loop works out where to place that element in the sorted result. Such sorting algorithms tend to be $O(n^2)$.

Vòng lặp lồng nhau Nếu bạn lồng một vòng lặp vào trong một vòng lặp khác thì thuật toán của bạn trở thành $O(m \times n)$, với m và n là giới hạn của hai vòng lặp. Điều này thường xảy ra trong các thuật toán sắp xếp đơn giản, chẳng hạn bubble sort, nơi vòng lặp ngoài lần lượt quét từng phần tử trong mảng, còn vòng lặp trong tìm ra chỗ đặt phần tử đó vào kết quả đã sắp xếp. Những thuật toán sắp xếp như vậy thường là $O(n^2)$.

Binary chop If your algorithm halves the set of things it considers each time around the loop, then it is likely to be logarithmic, $O(\lg n)$. A binary search of a sorted list, traversing a binary tree, and finding the first set bit in a machine word can all be $O(\lg n)$.

Chia đôi Nếu thuật toán của bạn chia đôi tập đối tượng nó xem xét sau mỗi vòng lặp thì nó nhiều khả năng là logarit, $O(\lg n)$. Một phép tìm kiếm nhị phân trên danh sách đã sắp xếp, duyệt một cây nhị phân, và tìm bit được bật đầu tiên trong một từ máy đều có thể là $O(\lg n)$.

Divide and conquer Algorithms that partition their input work on the two halves independently, and then combine the result can be $O(n \lg n)$. The classic example is quicksort, which works by partitioning the data into two halves and recursively sorting each. Although technically $O(n^2)$, because its behavior degrades when it is fed sorted input, the average

runtime of quicksort is $O(n \lg n)$. Chia để trị Những thuật toán phân hoạch đầu vào của mình, xử lý hai nửa một cách độc lập rồi gộp kết quả lại có thể là $O(n \lg n)$. Ví dụ kinh điển là quicksort, hoạt động bằng cách phân hoạch dữ liệu thành hai nửa rồi sắp xếp đệ quy mỗi nửa. Mặc dù về mặt kỹ thuật là $O(n^2)$ vì hành vi của nó suy giảm khi được nạp đầu vào đã sắp xếp, thời gian chạy

runtime of quicksort is $O(n \lg n)$.

trung bình của quicksort là $O(n \lg n)$.

Combinatoric Whenever algorithms start looking at the permutations of things, their running times may get out of hand. This is because permutations involve factorials (there are $5!$ permutations of the digits from 1 to 5). Time a combinatoric algorithm for five elements: it will take six times longer to run it for six, and 42 times longer for seven. Examples include algorithms for many of the acknowledged hard problems—the traveling salesman problem, optimally packing things into

a container, partitioning a set of numbers so that each set has the same total, and so on. Often, heuristics are used to reduce the running times of these types of algorithms in particular problem domains.

Tổ hợp Bất cứ khi nào thuật toán bắt đầu xét đến các hoán vị của các đối tượng, thời gian chạy của chúng có thể vượt khỏi tầm kiểm soát. Đó là vì hoán vị kéo theo giai thừa (có 120 hoán vị của các chữ số từ 1 đến 5). Hãy bấm giờ một thuật toán tổ hợp với năm phần tử: nó sẽ mất thời gian lâu gấp sáu lần để chạy với sáu phần tử, và lâu gấp 42 lần với bảy phần tử. Ví dụ gồm các thuật toán cho nhiều bài toán được công nhận là khó—bài toán người bán hàng rong, đóng gói tối ưu các vật vào một thùng chứa, phân hoạch một tập số sao cho mỗi tập con có cùng tổng, v.v. Người ta thường dùng các phép suy nghiệm (heuristic) để giảm thời gian chạy của những kiểu thuật toán này trong các miền bài toán cụ thể.

Algorithm Speed In Practice — Tốc độ thuật toán trong thực tế

It's unlikely that you'll spend much time during your career writing sort routines. The ones in the libraries available to you will probably outperform anything you may write without substantial effort. However, the basic kinds of algorithms we've described earlier pop up time and time again. Whenever you find yourself writing a simple loop, you know that you have an algorithm. If that loop contains an inner loop, then you're looking at . You should be asking yourself how large these values can get. If the numbers are bounded, then you'll know how long the code will take to run. If the numbers depend on external factors (such as the number of records in an overnight batch run, or the number of names in a list of people), then you might want to stop and consider the effect that large values may have on your running time or memory consumption.

Bạn khó có khả năng dành nhiều thời gian trong sự nghiệp để viết các thường trình sắp xếp. Những thường trình sẵn có trong các thư viện bạn dùng nhiều khả năng sẽ vượt trội hơn bất cứ thứ gì bạn tự viết mà không bỏ ra nỗ lực đáng kể. Tuy nhiên, các loại thuật toán cơ bản chúng tôi mô tả ở trên cứ lặp đi lặp lại hết lần này đến lần khác. Mỗi khi thấy mình đang viết một vòng lặp đơn, bạn biết mình có một thuật toán $O(n)$. Nếu vòng lặp ấy chứa một vòng lặp con thì bạn đang nhìn vào $O(n^2)$. Bạn nên tự hỏi những giá trị này có thể lớn đến đâu. Nếu các con số bị chặn thì bạn sẽ biết mã chạy mất bao lâu. Nếu các con số phụ thuộc vào những yếu tố bên ngoài (như số bản ghi trong một lượt chạy theo lô qua đêm, hay số tên trong một danh sách người), thì có lẽ bạn nên dừng lại và cân nhắc tác động mà các giá trị lớn có thể gây ra cho thời gian chạy hay mức tiêu thụ bộ nhớ của mình.

Tip 63: Estimate the Order of Your Algorithms

Tip 63: Hãy Ước Lượng Bậc Của Thuật Toán

There are some approaches you can take to address potential problems. If you have an algorithm that is , try to find a divide-and-conquer approach that will take you down to .

Có vài cách tiếp cận bạn có thể dùng để xử lý các vấn đề tiềm tàng. Nếu bạn có một thuật toán $O(n^2)$, hãy thử tìm một cách tiếp cận chia-để-trị đưa bạn xuống $O(n \lg n)$.

If you're not sure how long your code will take, or how much memory it will use, try running it, varying the input record count or whatever is likely to impact the runtime. Then plot the results. You should soon get a good idea of the shape of the curve. Is it curving upward, a straight line, or flattening off as the input size increases? Three or four points should give you an idea.

Nếu không chắc mã của mình mất bao lâu, hay tốn bao nhiêu bộ nhớ, hãy thử chạy nó, thay đổi số lượng bản ghi đầu vào hoặc bất cứ thứ gì có khả năng tác động đến thời gian chạy. Rồi vẽ đồ thị kết quả. Bạn sẽ sớm nắm được hình dạng của đường cong. Nó cong vồng

lên, là một đường thẳng, hay phẳng dần khi kích thước đầu vào tăng? Ba bốn điểm là đủ để bạn hình dung.

Also consider just what you're doing in the code itself. A simple loop may well perform better than a complex, one for smaller values of n , particularly if the algorithm has an expensive inner loop.

Cũng hãy cân nhắc chính xác bạn đang làm gì trong bản thân đoạn mã. Một vòng lặp đơn rất có thể chạy tốt hơn một vòng lặp phức tạp với các giá trị n nhỏ, đặc biệt nếu thuật toán có một vòng lặp trong tổt kém.

In the middle of all this theory, don't forget that there are practical considerations as well. Runtime may look like it increases linearly for small input sets. But feed the code millions of records and suddenly the time degrades as the system starts to thrash. If you test a sort routine with random input keys, you may be surprised the first time it encounters ordered input. Try to cover both the theoretical and practical bases. After all this estimating, the only timing that counts is the speed of your code, running in the production environment, with real data. This leads to our next **tip**.

Giữa tất cả lý thuyết này, đừng quên rằng cũng có những cân nhắc thực tế. Thời gian chạy có thể trông như tăng tuyến tính với các tập đầu vào nhỏ. Nhưng hãy nạp cho mã hàng triệu bản ghi và đột nhiên thời gian suy giảm khi hệ thống bắt đầu thrashing. Nếu bạn kiểm thử một thường trình sắp xếp bằng các khóa đầu vào ngẫu nhiên, bạn có thể ngạc nhiên lần đầu nó gặp đầu vào đã sắp xếp sẵn. Hãy cố che cả phương diện lý thuyết lẫn thực tế. Sau tất cả việc ước lượng này, thước đo thời gian duy nhất có ý nghĩa là tốc độ của mã bạn, chạy trong môi trường production, với dữ liệu thật. Điều này dẫn đến mẹo tiếp theo của chúng tôi.

Tip 64: Test Your Estimates

Tip 64: Hãy Kiểm Thử Các Ước Lượng Của Bạn

If it's tricky getting accurate timings, use code profilers to count the number of times the different steps in your algorithm get executed, and plot these figures against the size of the input.

Nếu việc đo thời gian chính xác là khó, hãy dùng trình profiler để đếm số lần các bước khác nhau trong thuật toán được thực thi, rồi vẽ các con số này theo kích thước đầu vào.

Best Isn't Always Best You also need to be pragmatic about choosing appropriate algorithms—the fastest one is not always the best for the job. Given a small input set, a straightforward insertion sort will perform just as well as a quicksort, and will take you less time to write and debug. You also need to be careful if the algorithm you choose has a high setup cost. For small input sets, this setup may dwarf the running time and make the algorithm inappropriate.

Tốt nhất không phải lúc nào cũng là tốt nhất Bạn cũng cần thực dụng khi chọn thuật toán phù hợp—cái nhanh nhất không phải lúc nào cũng là cái tốt nhất cho công việc. Với một tập đầu vào nhỏ, một phép sắp xếp chèn đơn giản sẽ chạy tốt chẳng kém gì quicksort, mà lại tốn ít thời gian hơn để viết và gỡ lỗi. Bạn cũng cần cẩn trọng nếu thuật toán bạn chọn có chi phí khởi tạo cao. Với các tập đầu vào nhỏ, chi phí khởi tạo này có thể lấn át thời gian chạy và khiến thuật toán trở nên không phù hợp.

Also be wary of **premature optimization**. It's always a good idea to make sure an algorithm really is a bottleneck before investing your precious time trying to improve it.

Cũng hãy dè chừng việc tối ưu hóa sớm (premature optimization). Luôn là một ý hay khi đảm bảo một thuật toán thực sự là điểm nghẽn trước khi đầu tư thời gian quý báu của bạn để cố cải thiện nó.

Related Sections Include — Các mục liên quan

Topic 15, Estimating

[Topic 15, Ước lượng](#)

Challenges — Thử thách

Every developer should have a feel for how algorithms are designed and analyzed. Robert Sedgewick has written a series of accessible books on the subject (Algorithms [SW11] An Introduction to the Analysis of Algorithms [SF13] and others). We recommend adding one of his books to your collection, and making a point of reading it.

Mọi nhà phát triển nên có cảm nhận về cách các thuật toán được thiết kế và phân tích. Robert Sedgewick đã viết một loạt sách dễ tiếp cận về chủ đề này (Algorithms [SW11], An Introduction to the Analysis of Algorithms [SF13] và những cuốn khác). Chúng tôi khuyên bạn thêm một cuốn của ông vào tủ sách của mình, và quyết tâm đọc nó.

For those who like more detail than Sedgewick provides, read Donald Knuth’s definitive Art of Computer Programming books, which analyze a wide range of algorithms.

Với những ai thích chi tiết hơn mức Sedgewick cung cấp, hãy đọc bộ Art of Computer Programming kinh điển của Donald Knuth, phân tích một phạm vi rộng các thuật toán.

The Art of Computer Programming, Volume 1:

[The Art of Computer Programming, Tập 1:](#)

Fundamental Algorithms [Knu98]

[Fundamental Algorithms \[Knu98\]](#)

The Art of Computer Programming, Volume 2: Seminumerical Algorithms [Knu98a]

[The Art of Computer Programming, Tập 2: Seminumerical Algorithms \[Knu98a\]](#)

The Art of Computer Programming, Volume 3: Sorting and Searching [Knu98b]

[The Art of Computer Programming, Tập 3: Sorting and Searching \[Knu98b\]](#)

The Art of Computer Programming, Volume 4A: Combinatorial Algorithms, Part 1 [Knu11] .

[The Art of Computer Programming, Tập 4A: Combinatorial Algorithms, Part 1 \[Knu11\].](#)

In the first exercise that follows we look at sorting arrays of long integers. What is the impact if the keys are more complex, and the overhead of key comparison is high? Does the key structure affect the efficiency of the sort algorithms, or is the fastest sort always fastest?

Trong bài tập đầu tiên ngay sau đây, chúng ta xem xét việc sắp xếp các mảng số nguyên dài. Tác động sẽ ra sao nếu các khóa phức tạp hơn, và phí tổn so sánh khóa là cao? Cấu trúc khóa có ảnh hưởng đến hiệu quả của các thuật toán sắp xếp không, hay phép sắp xếp nhanh nhất luôn là nhanh nhất?

Exercises — Bài tập

Exercise 28 (possible answer)

[Bài tập 28 \(đáp án khả dĩ\)](#)

We coded a set of simple sort routines [54] in Rust. Run them on various machines available to you. Do your figures follow the expected curves? What can you deduce about the relative speeds of your machines? What are the effects of various compiler optimization settings?

Chúng tôi đã viết một bộ các thường trình sắp xếp đơn giản [54] bằng Rust. Hãy chạy chúng trên những máy khác nhau mà bạn có. Các con số của bạn có bám theo những đường cong như dự đoán không? Bạn suy ra được điều gì về tốc độ tương đối giữa các máy của mình? Các mức tùy chọn tối ưu hóa khác nhau của trình biên dịch gây ra tác động gì?

Exercise 29 (possible answer)

Bài tập 29 (đáp án khả dĩ)

In Common Sense **Estimation** , we claimed that a binary chop is . Can you prove this?

Trong mục Ước lượng bằng lẽ thường, chúng tôi đã khẳng định rằng một phép chia đôi là $O(\lg n)$. Bạn có thể chứng minh điều này không?

Exercise 30 (possible answer)

Bài tập 30 (đáp án khả dĩ)

In Figure 3, Runtimes of various algorithms , we claimed that is the same as (or indeed logarithms to any base). Can

Trong Hình 3, Thời gian chạy của các thuật toán khác nhau, chúng tôi đã khẳng định rằng $O(\lg n)$ cũng giống như $O(\log n)$ (hay thực ra là logarit với cơ số bất kỳ). Bạn

you explain why?

có thể giải thích vì sao không?

Change and decay in all around I see... H. F. Lyte, Abide With Me

Đổi thay và tàn lụi, quanh tôi đâu đâu cũng thấy... H. F. Lyte, Abide With Me

Topic 40. Refactoring — Topic 40. Tái cấu trúc

As a program evolves, it will become necessary to rethink earlier decisions and rework portions of the code. This **process** is perfectly natural. Code needs to evolve; it's not a static thing.

Khi một chương trình tiến hóa, sẽ đến lúc cần xem xét lại những quyết định trước đó và làm lại từng phần mã. Quá trình này hoàn toàn tự nhiên. Mã cần tiến hóa; nó không phải thứ tĩnh tại.

Unfortunately, the most common metaphor for software development is building construction. Bertrand Meyer's classic work Object-Oriented Software Construction [Mey97] uses the term "Software Construction," and even your humble authors edited the Software Construction column for IEEE Software in the early 2000s. [55]

Đáng tiếc là ẩn dụ phổ biến nhất cho việc phát triển phần mềm lại là xây dựng nhà cửa. Cuốn sách kinh điển Object-Oriented Software Construction [Mey97] của Bertrand Meyer dùng thuật ngữ "Software Construction" (xây dựng phần mềm), và ngay cả hai tác giả khiêm nhường của bạn cũng từng biên tập chuyên mục Software Construction cho tạp chí IEEE Software hồi đầu những năm 2000. [55]

But using construction as the guiding metaphor implies the following steps:

Nhưng lấy việc xây dựng làm ẩn dụ dẫn đường thì ngầm hiểu các bước sau:

1. An architect draws up blueprints. 2. Contractors dig the foundation, build the superstructure, wire and plumb, and apply finishing touches. 3. The tenants move in and live happily ever after, calling building maintenance to fix any problems.

1. Một kiến trúc sư vẽ bản thiết kế. 2. Các nhà thầu đào móng, dựng phần kết cấu, đi dây điện và đường nước, rồi hoàn thiện. 3. Người thuê dọn vào và sống hạnh phúc mãi mãi, chỉ cần gọi bộ phận bảo trì khi có sự cố.

Well, software doesn't quite work that way. Rather than construction, software is more like gardening—it is more organic than concrete. You plant many things in a garden according to an initial plan and conditions. Some thrive, others are destined to end up as compost. You may move plantings

Chà, phần mềm không vận hành kiểu đó đâu. Thay vì xây dựng, phần mềm giống làm vườn hơn — nó hữu cơ hơn là bê tông. Bạn trồng nhiều thứ trong vườn dựa theo một kế hoạch và điều kiện ban đầu. Có cây phát triển tốt, cây khác lại định sẵn phải thành phân ủ. Bạn có thể dời các luống cây

relative to each other to take advantage of the interplay of light and shadow, wind and rain. Overgrown plants get split or pruned, and colors that clash may get moved to more aesthetically pleasing locations. You pull weeds, and you fertilize plantings that are in need of some extra help. You constantly monitor the health of the garden, and make adjustments (to the soil, the plants, the layout) as needed.

so với nhau để tận dụng sự tương tác giữa ánh sáng và bóng râm, gió và mưa. Cây mọc um tùm thì được tách ra hoặc tỉa bớt, còn những màu sắc xung khắc nhau thì có thể dời tới chỗ ưa nhìn hơn. Bạn nhổ cỏ dại, và bón phân cho những cây cần được giúp thêm chút. Bạn liên tục theo dõi sức khỏe khu vườn, và điều chỉnh (đất, cây, cách bố trí) khi cần.

Business people are comfortable with the metaphor of building construction: it is more scientific than gardening, it's repeatable, there's a rigid reporting hierarchy for management, and so on. But we're not building skyscrapers—we aren't as constrained by the boundaries of physics and the real world.

Dân kinh doanh thấy thoải mái với ẩn dụ xây dựng nhà cửa: nó khoa học hơn làm vườn, lặp lại được, có hệ thống báo cáo phân cấp chặt chẽ cho cấp quản lý, vân vân. Nhưng chúng ta đâu có xây nhà chọc trời — chúng ta không bị bó buộc bởi giới hạn của vật lý và thế giới thực như thế.

The gardening metaphor is much closer to the realities of software development. Perhaps a certain routine has grown too large, or is trying to accomplish too much—it needs to be split into two. Things that don't work out as planned need to be weeded or pruned.

Ẩn dụ làm vườn gần với thực tế phát triển phần mềm hơn nhiều. Có lẽ một thường trình (routine) nào đó đã phình quá to, hoặc cố ôm đồm quá nhiều việc — nó cần được tách làm hai. Những thứ không diễn ra như kế hoạch thì cần nhổ cỏ hoặc tỉa bớt.

Rewriting, reworking, and re-architecting code is collectively known as restructuring . But there's a subset of that activity that has become practiced as **refactoring** .

Viết lại, làm lại và tái kiến trúc mã, gộp chung lại, được gọi là tái dựng (restructuring). Nhưng có một nhánh con của hoạt động đó đã được thực hành dưới tên gọi tái cấu trúc (refactoring).

Refactoring [Fow19] is defined by Martin Fowler as a:

Tái cấu trúc [Fow19] được Martin Fowler định nghĩa là:

disciplined technique for restructuring an existing body of code, altering its **internal** structure without changing its external behavior.

một kỹ thuật có kỷ luật để tái dựng một thân mã hiện có, thay đổi cấu trúc nội tại của nó mà không làm thay đổi hành vi bên ngoài.

The critical parts of this definition are that:

Những phần then chốt của định nghĩa này là:

1. The activity is disciplined, not a free-for-all
 1. Hoạt động này có kỷ luật, không phải kiểu mạnh ai nấy làm
2. External behavior does not change; this is not the time to add features
 2. Hành vi bên ngoài không thay đổi; đây không phải lúc để thêm tính năng

Refactoring is not intended to be a special, high-ceremony, once-in-a-while activity, like plowing under the whole garden in order to replant. Instead, refactoring is a day-to-day activity, taking low-risk small steps, more like weeding and raking. Instead of a free-for-all, wholesale rewrite of the codebase, it's a targeted, precision approach to help keep the code easy to change.

Tái cấu trúc không nhằm trở thành một hoạt động đặc biệt, rình rang, thỉnh thoảng mới làm một lần, kiểu cày xới cả khu vườn lên để trồng lại. Trái lại, tái cấu trúc là hoạt động thường nhật, đi từng bước nhỏ ít rủi ro, giống nhổ cỏ và cào lá hơn. Thay vì mạnh ai nấy làm, viết lại toàn bộ mã nguồn, nó là cách tiếp cận có mục tiêu, chính xác, giúp giữ cho mã dễ thay đổi.

In order to guarantee that the external behavior hasn't changed, you need good, automated unit testing that validates the behavior of the code.

Để đảm bảo hành vi bên ngoài không thay đổi, bạn cần kiểm thử đơn vị (unit test) tự động tốt, xác nhận được hành vi của mã.

When Should You Refactor? — Khi nào nên tái cấu trúc?

You refactor when you've learned something; when you understand something better than you did last year, yesterday, or even just ten minutes ago.

Bạn tái cấu trúc khi đã học được điều gì đó; khi bạn hiểu một thứ rõ hơn so với hồi năm ngoái, hôm qua, hay thậm chí chỉ mười phút trước.

Perhaps you've come across a stumbling block because the code doesn't quite fit anymore, or you notice two things that should really be merged, or anything else at all strikes you as being "wrong," don't hesitate to change it. There's no time like the present. Any number of things may cause code to qualify for refactoring:

Có thể bạn vấp phải một chướng ngại vì đoạn mã không còn vừa khít nữa, hoặc bạn nhận ra hai thứ thực ra nên được gộp lại, hay bất cứ điều gì khác khiến bạn thấy "sai sai" — đừng ngần ngại sửa nó. Còn lúc nào tốt hơn hiện tại đâu. Vô số lý do có thể khiến mã đủ điều kiện để tái cấu trúc:

Duplication You've discovered a violation of the **DRY** principle.

Sự trùng lặp Bạn phát hiện một vi phạm nguyên tắc DRY.

Nonorthogonal design You've discovered something that could be made more orthogonal.

Thiết kế phi trực giao Bạn phát hiện thứ gì đó có thể làm cho trực giao hơn.

Outdated knowledge Things change, **requirements** drift, and your knowledge of the problem increases. Code needs to keep up.

Tri thức đã lỗi thời Mọi thứ thay đổi, yêu cầu trôi dạt, và hiểu biết của bạn về vấn đề tăng lên. Mã cần bắt kịp.

Usage As the system gets used by real people under real circumstances, you realize some features are now more important than previously thought, and “must have” features perhaps weren’t.

Cách dùng Khi hệ thống được người thật sử dụng trong hoàn cảnh thật, bạn nhận ra một số tính năng giờ quan trọng hơn so với tưởng tượng trước đây, còn những tính năng “bắt buộc phải có” thì có lẽ lại không phải vậy.

Performance You need to move functionality from one area of the system to another to improve performance.

Hiệu năng Bạn cần dời chức năng từ vùng này sang vùng khác của hệ thống để cải thiện hiệu năng.

The Tests Pass Yes. Seriously. We did say that refactoring should be a small scale activity, backed up by good tests. So when you’ve added a small amount of code, and that one extra test passes, you now have a great opportunity to dive in and tidy up what you just wrote.

Các kiểm thử đều qua Đúng vậy. Nghiêm túc đấy. Chúng tôi đã nói rằng tái cấu trúc nên là hoạt động quy mô nhỏ, được chống lưng bởi các kiểm thử tốt. Vậy nên khi bạn vừa thêm một lượng mã nhỏ, và cái kiểm thử bổ sung đó vừa qua, giờ bạn có một cơ hội tuyệt vời để nhảy vào dọn dẹp lại đúng thứ mình vừa viết.

Refactoring your code—moving functionality around and updating earlier decisions—is really an exercise in pain management . Let’s face it, changing source code around can be pretty painful: it was working, maybe it’s better to leave well enough alone. Many developers are reluctant to go in and reopen a piece of code just because it isn’t quite right.

Tái cấu trúc mã — dời chức năng đi chỗ khác và cập nhật những quyết định trước đó — thực ra là một bài tập về quản lý nỗi đau. Phải thừa nhận thôi, thay đổi mã nguồn lung tung có thể khá đau: nó đang chạy tốt mà, hay là cứ để yên cho lành. Nhiều lập trình viên ngại nhảy vào mở lại một đoạn mã chỉ vì nó chưa được ổn lắm.

Real-World Complications So you go to your teammates or **client** and say, “This code works, but I need another week to completely refactor it.”

Những rắc rối ngoài đời thực Thế là bạn đến gặp đồng đội hoặc khách hàng và nói: “Mã này chạy được, nhưng tôi cần thêm một tuần nữa để tái cấu trúc toàn bộ.”

We can’t print their reply.

Câu họ đáp lại thì chúng tôi không in ra được.

Time pressure is often used as an excuse for not refactoring. But this excuse just doesn’t hold up: fail to refactor now, and there’ll be a far greater time investment to fix the problem down the road—when there are more dependencies to reckon with. Will there be more time available then? Nope.

Áp lực thời gian thường bị viện làm cớ để không tái cấu trúc. Nhưng cái cớ này không đứng vững: không tái cấu trúc bây giờ, thì sau này sẽ phải đầu tư thời gian lớn hơn nhiều để sửa vấn đề — khi đã có thêm nhiều phụ thuộc phải tính tới. Liệu lúc đó có sẵn nhiều thời gian hơn không? Không hề.

You might want to explain this principle to others by using a medical analogy: think of the code that needs refactoring as “a growth.” Removing it requires invasive surgery. You can go in now, and take it

out while it is still small. Or, you could wait while it grows and spreads—but removing it then will be both more expensive and more dangerous. Wait even longer, and you may lose the patient entirely.

Bạn có thể muốn giải thích nguyên tắc này cho người khác bằng một phép ẩn dụ y học: hãy hình dung đoạn mã cần tái cấu trúc như “một khối u”. Cắt bỏ nó đòi hỏi phẫu thuật xâm lấn. Bạn có thể vào ngay bây giờ và lấy nó ra khi nó còn nhỏ. Hoặc, bạn có thể chờ trong lúc nó lớn lên và lan rộng — nhưng cắt bỏ lúc đó sẽ vừa tốn kém hơn vừa nguy hiểm hơn. Chờ lâu hơn nữa, và bạn có thể mất luôn bệnh nhân.

Tip 65: Refactor Early, Refactor Often

Tip 65: Hãy Tái Cấu Trúc Sớm, Tái Cấu Trúc Thường Xuyên

Collateral damage in code can be just as deadly over time (see **Topic 3, Software Entropy**). Refactoring, as with most things, is easier to do while the issues are small, as an ongoing activity while coding. You shouldn’t need “a week to refactor” a piece of code—that’s a full-on rewrite. If that level of disruption is necessary, then you might well not be able to do it immediately. Instead, make sure that it gets placed on the schedule. Make sure that users of the affected code know that it is scheduled to be rewritten and how this might affect them.

Thiệt hại liên đới trong mã, theo thời gian, cũng có thể chết người y như vậy (xem Topic 3, Sự rối loạn phần mềm). Tái cấu trúc, như hầu hết mọi thứ, dễ làm hơn khi vấn đề còn nhỏ, như một hoạt động diễn ra liên tục trong lúc viết mã. Lẽ ra bạn không nên cần “một tuần để tái cấu trúc” một đoạn mã — đó là viết lại toàn bộ rồi. Nếu thực sự cần mức xáo trộn đó, thì có lẽ bạn không thể làm ngay lập tức. Thay vào đó, hãy chắc chắn nó được đưa vào lịch trình. Hãy chắc chắn những người dùng đoạn mã bị ảnh hưởng biết rằng nó đã được lên lịch để viết lại và điều này có thể tác động tới họ ra sao.

How Do You Refactor? — Tái cấu trúc như thế nào?

Refactoring started out in the Smalltalk community, and had just started to gain a wider audience when we wrote the first edition of this book, probably thanks to the first major book on refactoring (*Refactoring: Improving the Design of Existing Code* [Fow19] , now in its second edition).

Tái cấu trúc khởi nguồn từ cộng đồng Smalltalk, và vừa mới bắt đầu được biết đến rộng rãi hơn khi chúng tôi viết ấn bản đầu tiên của cuốn sách này, có lẽ nhờ cuốn sách lớn đầu tiên về tái cấu trúc (*Refactoring: Improving the Design of Existing Code* [Fow19], nay đã đến ấn bản thứ hai).

At its heart, refactoring is redesign. Anything that you or others on your team designed can be redesigned in light of new facts, deeper understandings, changing requirements, and so on. But if you proceed to rip up vast quantities of code with wild abandon, you may find yourself in a worse position than when you started.

Về cốt lõi, tái cấu trúc chính là thiết kế lại. Bất cứ thứ gì bạn hay người khác trong nhóm đã thiết kế đều có thể được thiết kế lại dựa trên những sự thật mới, hiểu biết sâu hơn, yêu cầu thay đổi, vân vân. Nhưng nếu bạn lao vào xé toang hàng đống mã một cách bạt mạng, bạn có thể thấy mình rơi vào tình thế tệ hơn lúc khởi đầu.

Clearly, refactoring is an activity that needs to be undertaken slowly, deliberately, and carefully. Martin Fowler offers the following simple tips on how to refactor without doing more harm than good: [56]

Rõ ràng, tái cấu trúc là hoạt động cần được tiến hành chậm rãi, có chủ đích và cẩn trọng. Martin Fowler đưa ra những lời khuyên đơn giản sau về cách tái cấu trúc mà không gây

hại nhiều hơn lợi: [56]

1. Don't try to refactor and add functionality at the same time. 2. Make sure you have good tests before you begin refactoring. Run the tests as often as possible. That way you will know quickly if your changes have broken anything. 3. Take short, deliberate steps: move a field from one class to another, split a method, rename a variable. Refactoring often involves making many localized changes that result in a larger-scale change. If you keep your steps small, and test after each step, you will avoid prolonged **debugging**. [57]

1. Đừng cố tái cấu trúc và thêm chức năng cùng một lúc. 2. Hãy chắc chắn bạn có các kiểm thử tốt trước khi bắt tay tái cấu trúc. Hãy chạy kiểm thử càng thường xuyên càng tốt. Nhờ vậy bạn sẽ biết ngay nếu thay đổi của mình làm hỏng thứ gì đó. 3. Hãy đi những bước ngắn, có chủ đích: dời một trường từ lớp này sang lớp khác, tách một phương thức, đổi tên một biến. Tái cấu trúc thường bao gồm nhiều thay đổi cục bộ dẫn tới một thay đổi ở quy mô lớn hơn. Nếu bạn giữ cho các bước của mình nhỏ, và kiểm thử sau mỗi bước, bạn sẽ tránh được những phen gỡ lỗi kéo dài. [57]

Automatic Refactoring Back in the first edition we noted that, “this technology has yet to appear outside of the Smalltalk world, but this is likely to change....” And indeed, it did, as automatic refactoring is available in many IDEs and for most mainstream languages.

Tái cấu trúc tự động Hồi ấn bản đầu chúng tôi đã ghi rằng “công nghệ này vẫn chưa xuất hiện bên ngoài thế giới Smalltalk, nhưng điều này có lẽ sẽ thay đổi....” Và quả thực nó đã thay đổi, khi tái cấu trúc tự động giờ đã có mặt trong nhiều IDE và cho hầu hết các ngôn ngữ phổ thông.

These IDEs can rename variables and methods, split a long routine into smaller ones, automatically propagating the required changes, drag and drop to assist you in moving code, and so on.

Các IDE này có thể đổi tên biến và phương thức, tách một thường trình dài thành những thường trình nhỏ hơn, tự động lan truyền các thay đổi cần thiết, kéo-thả để hỗ trợ bạn dời mã, vãn vãn.

We'll talk more about testing at this level in Topic 41, **Test to Code**, and larger-scale testing in Ruthless and Continuous Testing, but Mr. Fowler's point of maintaining good regression

Chúng ta sẽ bàn thêm về kiểm thử ở mức này trong Topic 41, Kiểm thử so với Mã, và kiểm thử ở quy mô lớn hơn trong Kiểm thử Tàn nhẫn và Liên tục, nhưng điểm mấu chốt mà ngài Fowler nêu — duy trì các kiểm thử hồi quy

tests is the key to refactoring safely.

tốt — chính là chìa khóa để tái cấu trúc an toàn.

If you have to go beyond refactoring and end up changing external behavior or interfaces, then it can help to deliberately break the build: old clients of this code should fail to compile. That way you'll know what needs updating. Next time you see a piece of code that isn't quite as it should be, fix it. Manage the pain: if it hurts now, but is going to hurt even more later, you might as well get it over with. Remember the lessons of Topic 3, Software **Entropy**: don't live with **broken windows**.

Nếu bạn buộc phải đi xa hơn tái cấu trúc và rốt cuộc thay đổi hành vi bên ngoài hoặc các giao diện, thì có thể hữu ích khi cố ý làm hỏng bản build: những đoạn mã cũ vốn dùng đoạn mã này lẽ ra phải biên dịch thất bại. Như vậy bạn sẽ biết những gì cần cập nhật. Lần tới khi bạn thấy một đoạn mã không hẳn được như nó nên có, hãy sửa nó. Quản lý nỗi đau: nếu giờ đã đau, mà về sau còn đau hơn, thì chi bằng giải quyết cho xong. Hãy nhớ bài học của Topic 3, Sự rối loạn phần mềm: đừng sống chung với cửa sổ vỡ (broken windows).

Related Sections Include — Các mục liên quan

Topic 3, Software Entropy

Topic 3, Sự rối loạn phần mềm

Topic 9, DRY—The Evils of Duplication

Topic 9, DRY — Tai họa của Sự trùng lặp

Topic 12, **Tracer Bullets**

Topic 12, Đạn vạch đường

Topic 27, Don't Outrun Your Headlights

Topic 27, Đừng vượt quá tầm đèn pha

Topic 44, **Naming Things**

Topic 44, Đặt tên cho mọi thứ

Topic 48, The Essence of **Agility**

Topic 48, Tinh túy của Sự linh hoạt

Topic 41. Test to Code — Topic 41. Kiểm thử để viết mã

The first edition of this book was written in more primitive times, when most developers wrote no tests—why bother, they thought, the world was going to end in the year 2000 anyway.

Ấn bản đầu tiên của cuốn sách này được viết vào một thời kỳ còn sơ khai hơn, khi mà phần lớn lập trình viên chẳng viết bài kiểm thử (test) nào — bận tâm làm gì, họ nghĩ thế, chẳng nào thế giới cũng tận thế vào năm 2000 cơ mà.

In that book, we had a section on how to build code that was easy to test. It was a sneaky way of convincing developers to actually write tests.

Trong cuốn sách đó, chúng tôi có một mục bàn về cách xây dựng mã sao cho dễ kiểm thử. Đó là một cách tinh ranh để thuyết phục lập trình viên thực sự ngồi viết test.

These are more enlightened times. If there are any developers still not writing tests, they at least know that they should be.

Giờ đã là thời khai sáng hơn nhiều. Nếu vẫn còn lập trình viên nào không viết test, thì ít ra họ cũng biết rằng mình nên làm điều đó.

But there's still a problem. When we ask developers why they write tests, they look at us as if we just asked if they still coded using punched cards and they'd say “to make sure the code works,” with an unspoken “you dummy” at the end. And we think that's wrong.

Nhưng vẫn còn một vấn đề. Khi chúng tôi hỏi các lập trình viên vì sao họ viết test, họ nhìn chúng tôi như thể chúng tôi vừa hỏi liệu họ có còn lập trình bằng thẻ đục lỗ hay không, rồi đáp “để chắc chắn mã chạy đúng”, với một câu “đồ ngốc” ngầm hiểu ở cuối. Và chúng tôi cho rằng như vậy là sai.

So what do we think is important about testing? And how do we think you should go about it?

Vậy chúng tôi nghĩ điều gì mới thật sự quan trọng về kiểm thử? Và chúng tôi nghĩ bạn nên tiếp cận nó ra sao?

Let's start with the bold statement:

Hãy bắt đầu bằng một tuyên bố táo bạo:

Tip 66: Testing Is Not About Finding Bugs

Tip 66: Kiểm Thử Không Phải Là Để Tìm Bug

We believe that the major benefits of testing happen when you think about and write the tests, not when you run them.

Chúng tôi tin rằng lợi ích lớn nhất của kiểm thử đến vào lúc bạn suy nghĩ về và viết ra các bài test, chứ không phải lúc bạn chạy chúng.

Thinking About Tests — Suy nghĩ về các bài kiểm thử

It's a Monday morning and you settle in to start work on some new code. You have to write something that queries the database to return a list of people who watch more than 10 videos a week on your “world's funniest dishwashing videos” site.

Đó là một buổi sáng thứ Hai và bạn ngồi vào bàn để bắt đầu viết một đoạn mã mới. Bạn phải viết một thứ gì đó truy vấn cơ sở dữ liệu để trả về danh sách những người xem hơn 10 video mỗi tuần trên trang “những video rửa bát hài hước nhất thế giới” của bạn.

You fire up your editor, and start by writing the function that performs the query:

Bạn mở trình soạn thảo lên và bắt đầu bằng việc viết hàm thực hiện truy vấn:

```
def return_avid_viewers do # ... hmmm ... end  
  def return_avid_viewers do # ... hmmm ... end
```

Stop! How do you know that what you're about to do is a good thing?

Khoan đã! Làm sao bạn biết được rằng cái bạn sắp làm là một điều tốt?

The answer is that you can't know that. No one can. But thinking about tests can make it more likely. Here's how that works.

Câu trả lời là bạn không thể biết được. Chẳng ai biết cả. Nhưng việc suy nghĩ về các bài test có thể khiến khả năng đó cao hơn. Đây là cách nó hoạt động.

Start by imagining that you'd finished writing the function and now had to test it. How would you do that? Well, you'd want to use some test data, which probably means you want to work in a database you control. Now some frameworks can handle that for you, running tests against a test database, but in our case that means we should be passing the database instance into our function rather than using some global one, as that allows us to change it while testing:

Hãy bắt đầu bằng việc tưởng tượng rằng bạn đã viết xong hàm và bây giờ phải kiểm thử nó. Bạn sẽ làm điều đó thế nào? Chà, bạn sẽ muốn dùng một ít dữ liệu kiểm thử, mà điều đó nhiều khả năng có nghĩa là bạn muốn làm việc trên một cơ sở dữ liệu do mình kiểm soát. Một số framework có thể lo việc đó hộ bạn, chạy test trên một cơ sở dữ liệu thử nghiệm, nhưng trong trường hợp của chúng ta thì điều đó có nghĩa là ta nên truyền instance cơ sở dữ liệu vào hàm thay vì dùng một biến toàn cục nào đó, vì như thế ta mới có thể thay đổi nó khi kiểm thử:

```
def return_avid_users(db) do  
  def return_avid_users(db) do
```

Then we have to think about how we'd populate that test data. The requirement asks for a “list of people who watch more than 10 videos a week.” So we look at the database **schema** for fields

Rồi ta phải nghĩ xem làm sao để đổ đầy dữ liệu kiểm thử ấy. Yêu cầu hỏi về một “danh sách những người xem hơn 10 video mỗi tuần”. Vậy nên ta nhìn vào lược đồ cơ sở dữ liệu để tìm các trường

that might help. We find two likely fields in a table of whowatched-what: `opened_video` and `completed_video`. To write our test data, we need to know which field to use. But we don't know what the requirement means, and our business contact is out. Let's just cheat and pass in the name of the field (which will allow us to test what we have, and potentially change it later):

có thể giúp ích. Ta tìm được hai trường có vẻ hợp lý trong một bảng ai-đã-xem-cái-gì: `opened_video` và `completed_video`. Để viết dữ liệu kiểm thử, ta cần biết phải dùng trường nào. Nhưng ta lại không biết yêu cầu ấy có nghĩa gì, mà người phụ trách nghiệp vụ thì đang vắng mặt. Thôi thì cứ ăn gian một chút và truyền vào tên của trường (như thế ta vẫn kiểm thử được cái mình đang có, và sau này có thể đổi lại):

```
def return_avid_users(db, qualifying_field_name) do
```

```
  def return_avid_users(db, qualifying_field_name) do
```

We started by thinking about our tests, and without writing a line of code, we've already made two discoveries and used them to change the API of our method.

Ta bắt đầu bằng việc suy nghĩ về các bài test của mình, và chưa viết lấy một dòng mã, ta đã có hai phát hiện và dùng chúng để thay đổi API của phương thức.

Tests Drive Coding — Kiểm thử dẫn dắt việc viết mã

In the previous example, thinking about testing made us reduce **coupling** in our code (by passing in a database connection rather than using a global one) and increase flexibility (by making the name of the field we test a parameter). Thinking about writing a test for our method made us look at it from the outside, as if we were a **client** of the code, and not its author.

Trong ví dụ trước, việc suy nghĩ về kiểm thử đã khiến ta giảm sự ghép nối (coupling) trong mã (bằng cách truyền vào một kết nối cơ sở dữ liệu thay vì dùng biến toàn cục) và tăng tính linh hoạt (bằng cách biến tên của trường ta cần kiểm thử thành một tham số). Việc suy nghĩ về chuyện viết test cho phương thức đã buộc ta nhìn nó từ bên ngoài, như thể ta là người dùng đoạn mã đó, chứ không phải tác giả của nó.

Tip 67: A Test Is the First User of Your Code

Tip 67: Bài Test Là Người Dùng Đầu Tiên Của Mã Bạn Viết

We think this is probably the biggest benefit offered by testing: testing is vital feedback that guides your coding.

Chúng tôi nghĩ đây có lẽ là lợi ích lớn nhất mà kiểm thử mang lại: kiểm thử là phản hồi sống còn dẫn lối cho việc viết mã của bạn.

A function or method that is tightly coupled to other code is hard to test, because you have to set up all that environment before you can even run your method. So making your stuff testable also reduces its coupling.

Một hàm hay phương thức bị ghép nối chặt với mã khác thì khó kiểm thử, vì bạn phải dựng lên toàn bộ môi trường đó trước khi có thể chạy nổi phương thức của mình. Vậy nên việc làm cho mã của bạn dễ kiểm thử cũng đồng thời giảm sự ghép nối của nó.

And before you can test something, you have to understand it. That sounds silly, but in reality we've all launched into a piece of

Và trước khi có thể kiểm thử một thứ gì đó, bạn phải hiểu nó. Nghe có vẻ ngớ ngẩn, nhưng thực tế tất cả chúng ta đều từng lao vào một đoạn

code based on a nebulous understanding of what we had to do. We assure ourselves that we'll work it out as we go along. Oh, and we'll add all the code to support the boundary conditions later, too. Oh, and the error handling. And the code ends up five times longer than it should because it's full of conditional logic and special cases. But shine the light of a test on that code, and things become clearer. If you think about testing boundary conditions and how that will work before you start coding, you may well find the patterns in the logic that'll simplify the function. If you think about the error conditions you'll need to test, you'll structure your function accordingly.

mã dựa trên một sự hiểu biết mơ hồ về việc mình phải làm gì. Ta tự trấn an rằng ta sẽ vừa làm vừa nghĩ ra. À, và ta sẽ thêm toàn bộ phần mã xử lý các điều kiện biên sau. À, và cả phần xử lý lỗi nữa. Rồi đoạn mã rốt cuộc dài gấp năm lần mức cần thiết vì nó đầy rẫy logic điều kiện và các trường hợp đặc biệt. Nhưng hãy rọi ánh sáng của một bài test vào đoạn mã đó, và mọi thứ trở nên sáng tỏ hơn. Nếu bạn suy nghĩ về việc kiểm thử các điều kiện biên và cách chúng vận hành trước khi bắt đầu viết mã, rất có thể bạn sẽ tìm ra các khuôn mẫu trong logic giúp đơn giản hóa hàm. Nếu bạn nghĩ về các điều kiện lỗi cần kiểm thử, bạn sẽ cấu trúc hàm của mình một cách tương ứng.

Test-Driven Development There's a school of programming that says that, given all the benefits of thinking about tests up front, why not go ahead and write them up front too? They practice something called testdriven development or TDD . You'll also see this called **test-first** development . [58]

Phát triển Hướng Kiểm thử (Test-Driven Development) Có một trường phái lập trình cho rằng, đã thấy hết những lợi ích của việc suy nghĩ về các bài test ngay từ đầu, thì sao không viết luôn chúng ngay từ đầu? Họ thực hành cái gọi là phát triển hướng kiểm thử (test-driven development), hay TDD. Bạn cũng sẽ thấy nó được gọi là phát triển kiểm thử trước (test-first development). [58]

The basic cycle of TDD is:

Chu trình cơ bản của TDD là:

1. Decide on a small piece of functionality you want to add. 2. Write a test that will pass once that functionality is implemented. 3. Run all tests. Verify that the only failure is the one you just wrote. 4. Write the smallest amount of code needed to get the test to pass, and verify that the tests now run cleanly. 5. Refactor your code: see if there is a way to improve on what you just wrote (the test or the function). Make sure the tests still pass when you're done.
1. Quyết định một mẫu chức năng nhỏ bạn muốn thêm vào. 2. Viết một bài test sẽ vượt qua (pass) một khi chức năng đó được hiện thực. 3. Chạy toàn bộ test. Xác nhận rằng lỗi duy nhất là cái bạn vừa viết. 4. Viết lượng mã nhỏ nhất cần thiết để bài test vượt qua, rồi xác nhận rằng các test giờ chạy sạch sẽ. 5. Tái cấu trúc (refactor) mã của bạn: xem có cách nào cải thiện thứ bạn vừa viết không (bài test hoặc hàm). Đảm bảo các test vẫn vượt qua khi bạn xong.

The idea is that this cycle should be very short: a matter of minutes, so that you're constantly writing tests and then getting them to work.

Ý tưởng là chu trình này phải rất ngắn: chỉ vài phút, để bạn liên tục viết test rồi làm cho chúng chạy được.

We see a major benefit in TDD for people just starting out with testing. If you follow the TDD workflow, you'll guarantee that you always have tests for your code. And that means you'll always be thinking about your tests.

Chúng tôi thấy TDD mang lại một lợi ích lớn cho những người mới chập chững làm quen với kiểm thử. Nếu bạn theo quy trình TDD, bạn đảm bảo rằng mã của mình luôn luôn có test. Và điều đó nghĩa là bạn sẽ luôn suy nghĩ về các bài test của mình.

However, we've also seen people become slaves to TDD. This manifests itself in a number of ways:

Tuy nhiên, chúng tôi cũng từng thấy người ta trở thành nô lệ của TDD. Điều này biểu hiện theo nhiều cách:

They spend inordinate amounts of time ensuring that they always have 100% test coverage.

Họ tốn lượng thời gian vô độ để bảo đảm rằng họ luôn có 100% độ phủ kiểm thử (test coverage).

They have lots of redundant tests. For example, before writing a class for the first time, many TDD adherents will first write a failing test that simply references the class's name. It fails, then they write an empty class definition and it passes. But now you have a test that does absolutely nothing; the next test you write will also reference the class, and so it makes the first unnecessary. There's more stuff to change if the class name changes later. And this is just a trivial example.

Họ có rất nhiều bài test thừa thãi. Ví dụ, trước khi viết một lớp (class) lần đầu, nhiều tín đồ TDD trước hết sẽ viết một bài test thất bại chỉ đơn thuần tham chiếu đến tên của lớp. Nó thất bại, rồi họ viết một định nghĩa lớp rỗng và nó vượt qua. Nhưng giờ bạn có một bài test chẳng làm gì cả; bài test tiếp theo bạn viết cũng sẽ tham chiếu đến lớp đó, và như thế khiến cái đầu tiên thành thừa. Sẽ có nhiều thứ phải sửa hơn nếu tên lớp đổi sau này. Và đây mới chỉ là một ví dụ vật vãnh.

Their designs tend to start at the bottom and work their way up. (See Bottom-Up vs. Top-Down vs. The Way You Should Do It.)

Thiết kế của họ có xu hướng bắt đầu từ dưới và đi dần lên trên. (Xem Từ dưới lên vs. Từ trên xuống vs. Cách bạn nên làm.)

Bottom-Up vs. Top-Down vs. The Way You Should Do It

Từ dưới lên vs. Từ trên xuống vs. Cách bạn nên làm

Back when computing was young and carefree, there were two schools of design: topdown and bottom-up. The top-down folks said you should start with the overall problem you're trying to solve and break it into a small number of pieces. Then break each of these into smaller pieces, and so on, until you end up with pieces small enough to express in code.

Hồi máy tính còn non trẻ và vô tư, có hai trường phái thiết kế: từ trên xuống (top-down) và từ dưới lên (bottom-up). Phe từ trên xuống nói rằng bạn nên bắt đầu với bài toán tổng thể mình đang cố giải và chia nó thành một số ít mảnh nhỏ. Rồi chia mỗi mảnh đó thành những mảnh nhỏ hơn, và cứ thế, cho đến khi bạn có những mảnh đủ nhỏ để diễn đạt bằng mã.

The bottom-up folks build code like you'd build a house. They start at the bottom, producing a layer of code that gives them some abstractions that are closer to the problem they are trying to solve. Then they add another layer, with higher-level abstractions. They keep on until the final layer is an abstraction that solves the problem. "Make it so...."

Phe từ dưới lên xây dựng mã như cách bạn xây một ngôi nhà. Họ bắt đầu từ dưới đáy, tạo ra một tầng mã cung cấp cho họ một số trừu tượng (abstraction) gần với bài toán họ đang cố giải hơn. Rồi họ thêm một tầng nữa, với các trừu tượng cấp cao hơn. Họ cứ thế cho đến khi tầng cuối cùng là một trừu tượng giải được bài toán. “Cứ thế mà làm....”

Neither school actually works, because both ignore one of the most important aspects of software development: we don’t know what we’re doing when we start. The top-down folks assume they can express the whole requirement up front: they can’t. The bottomup folks assume they can build a list of abstractions which will take them eventually to a

Chẳng trường phái nào thực sự hiệu quả, vì cả hai đều bỏ qua một trong những khía cạnh quan trọng nhất của phát triển phần mềm: ta không biết mình đang làm gì khi mới bắt đầu. Phe từ trên xuống cho rằng họ có thể diễn đạt toàn bộ yêu cầu ngay từ đầu: họ không thể. Phe từ dưới lên cho rằng họ có thể dựng nên một danh sách các trừu tượng rất cuộc sẽ đưa họ đến

single top-level solution, but how can they decide on the functionality of layers when they don’t know where they are heading?

một lời giải đơn nhất ở tầng cao nhất, nhưng làm sao họ quyết định được chức năng của các tầng khi họ không biết mình đang hướng về đâu?

Tip 68: Build End-to-End, Not Top-Down or Bottom Up

Tip 68: Hãy Xây Theo Kiểu Đầu-Cuối, Đừng Từ Trên Xuống Hay Từ Dưới Lên

We strongly believe that the only way to build software is incrementally. Build small pieces of end-to-end functionality, learning about the problem as you go. Apply this learning as you continue to flesh out the code, involve the **customer** at each step, and have them guide the **process**.

Chúng tôi tin chắc rằng cách duy nhất để xây dựng phần mềm là làm tăng dần (incrementally). Hãy xây những mẫu chức năng đầu-cuối nhỏ, vừa làm vừa học về bài toán. Áp dụng những điều học được khi bạn tiếp tục đắp đầy đặn cho đoạn mã, lôi kéo khách hàng vào ở từng bước, và để họ dẫn dắt quá trình.

By all means practice TDD. But, if you do, don’t forget to stop every now and then and look at the **big picture**. It is easy to become seduced by the green “tests passed” message, writing lots of code that doesn’t actually get you closer to a solution.

Bằng mọi giá, hãy cứ thực hành TDD. Nhưng nếu làm vậy, đừng quên thỉnh thoảng dừng lại và nhìn vào bức tranh toàn cảnh. Rất dễ bị dòng chữ xanh “tests passed” quyến rũ, để rồi viết ra cả đống mã mà thật ra chẳng đưa bạn đến gần lời giải hơn chút nào.

Tdd: You Need To Know Where You’re Going — TDD: Bạn cần biết mình đang đi về đâu

The old joke asks “How do you eat an elephant?” The punchline: “One bite at a time.” And this idea is often touted as a benefit of TDD. When you can’t comprehend the whole problem, take small steps, one test at a time. However, this approach can mislead you, encouraging you to focus on and endlessly polish the easy problems while ignoring the real reason you’re coding. An interesting example of this happened in 2006, when Ron Jeffries, a leading figure in the **agility** movement, started a series of blog posts which documented his test-driven coding of a Sudoku solver. [59] After five posts, he’d refined the representation of the underlying board, **refactoring** a number of times until he was happy with the object model. But then he abandoned the project. It’s interesting to read the blog posts in order,

and watch how a clever person can get sidetracked by the minutia, reinforced by the glow of passing tests.

Câu đùa xưa hỏi rằng “Làm sao ăn hết một con voi?” Câu chốt: “Ăn từng miếng một.” Và ý tưởng này thường được tung hô như một lợi ích của TDD. Khi bạn chưa thể nắm bắt trọn vẹn bài toán, hãy đi từng bước nhỏ, mỗi lần một bài test. Tuy nhiên, cách tiếp cận này có thể đánh lừa bạn, khuyến khích bạn chăm chăm vào và mãi giữa mãi không thôi những bài toán dễ, trong khi phớt lờ lý do thực sự khiến bạn ngồi xuống viết mã. Một ví dụ thú vị về chuyện này xảy ra vào năm 2006, khi Ron Jeffries, một nhân vật hàng đầu của phong trào agile, khởi động một loạt bài blog ghi lại quá trình ông viết một chương trình giải Sudoku theo lối hướng kiểm thử. [59] Sau năm bài, ông đã tinh chỉnh xong cách biểu diễn bàn cờ bên dưới, tái cấu trúc nhiều lần cho tới khi hài lòng với mô hình đối tượng. Nhưng rồi ông bỏ dở dự án. Đọc loạt bài blog ấy theo thứ tự thì rất thú vị, để thấy một người thông minh có thể bị những tiểu tiết kéo chệch hướng ra sao, lại còn được tiếp sức bởi ánh hào quang của những bài test vượt qua.

As a contrast, Peter Norvig describes an alternative approach [60]

Ngược lại, Peter Norvig mô tả một cách tiếp cận khác [60]

which feels very different in character: rather than being driven by tests, he starts with a basic understanding of how these kinds of problems are traditionally solved (using constraint propagation), and then focuses on refining his algorithm. He addresses board representation in a dozen lines of code that flow directly from his discussion of notation.

mang một tính chất rất khác biệt: thay vì để các bài test dẫn dắt, ông bắt đầu với một sự hiểu biết cơ bản về việc những loại bài toán kiểu này thường được giải ra sao theo truyền thống (dùng phương pháp lan truyền ràng buộc - constraint propagation), rồi tập trung vào việc tinh chỉnh thuật toán của mình. Ông giải quyết phần biểu diễn bàn cờ trong khoảng chục dòng mã, tuôn ra trực tiếp từ phần thảo luận của ông về cách ký hiệu.

Tests can definitely help drive development. But, as with every drive, unless you have a destination in mind, you can end up going in circles.

Các bài test chắc chắn có thể giúp dẫn dắt việc phát triển. Nhưng, như với mọi chuyến lái xe, trừ phi bạn có đích đến trong đầu, bạn có thể rốt cuộc cứ chạy lòng vòng.

Back To The Code — Quay lại với mã

Component-based development has long been a lofty goal of software development. [61] The idea is that generic software components should be available and combined just as easily as common integrated circuits (ICs) are combined. But this works only if the components you are using are known to be reliable, and if you have common voltages, interconnect standards, timing, and so on.

Phát triển dựa trên thành phần (component-based development) từ lâu đã là một mục tiêu cao cả của ngành phát triển phần mềm. [61] Ý tưởng là các thành phần phần mềm tổng quát nên có sẵn và có thể ghép lại với nhau dễ dàng y như cách người ta ghép các vi mạch tích hợp (IC) thông dụng. Nhưng điều này chỉ hiệu quả nếu những thành phần bạn dùng đã được biết là đáng tin cậy, và nếu bạn có chung điện áp, chuẩn kết nối, định thời, vân vân.

Chips are designed to be tested—not just at the factory, not just when they are installed, but also in the field when they are deployed. More complex chips and systems may have a full Built-In Self Test (BIST) feature that runs some base-level diagnostics internally, or a Test Access Mechanism (TAM) that

provides a test harness that allows the external environment to provide stimuli and collect responses from the chip.

Các con chip được thiết kế để kiểm thử được — không chỉ tại nhà máy, không chỉ khi được lắp đặt, mà còn cả ngoài thực địa khi chúng được triển khai. Những con chip và hệ thống phức tạp hơn có thể có hẳn một tính năng Tự Kiểm Tra Tích Hợp (Built-In Self Test, BIST) chạy một số chẩn đoán cơ bản từ bên trong, hoặc một Cơ Chế Truy Cập Kiểm Thử (Test Access Mechanism, TAM) cung cấp một bộ giàn kiểm thử cho phép môi trường bên ngoài đưa kích thích vào và thu thập phản hồi từ con chip.

We can do the same thing in software. Like our hardware colleagues, we need to build testability into the software from the very beginning, and test each piece thoroughly before trying to wire them together.

Ta có thể làm điều tương tự trong phần mềm. Giống như các đồng nghiệp phần cứng, ta cần xây dựng tính kiểm thử được vào trong phần mềm ngay từ đầu, và kiểm thử kỹ lưỡng từng mảnh trước khi cố nối chúng lại với nhau.

Unit Testing — Kiểm thử đơn vị

Chip-level testing for hardware is roughly equivalent to unit testing in software—testing done on each module, in isolation, to verify its behavior. We can get a better feeling for how a module will react in the big wide world once we have tested it thoroughly under controlled (even contrived) conditions.

Kiểm thử ở cấp con chip đối với phần cứng đại khái tương đương với kiểm thử đơn vị (unit testing) trong phần mềm — kiểm thử được thực hiện trên từng mô-đun, một cách độc lập, để xác minh hành vi của nó. Ta sẽ có cảm nhận tốt hơn về việc một mô-đun sẽ phản ứng ra sao ngoài thế giới rộng lớn một khi đã kiểm thử nó kỹ lưỡng trong những điều kiện được kiểm soát (thậm chí được dàn dựng).

A software unit test is code that **exercises** a module. Typically, the unit test will establish some kind of artificial environment, then invoke routines in the module being tested. It then checks the results that are returned, either against known values or against the results from previous runs of the same test (regression testing).

Một bài kiểm thử đơn vị trong phần mềm là đoạn mã vận hành một mô-đun. Thông thường, bài test đơn vị sẽ thiết lập một loại môi trường nhân tạo nào đó, rồi gọi các thường trình (routine) trong mô-đun đang được kiểm thử. Sau đó nó kiểm tra các kết quả trả về, hoặc đối chiếu với các giá trị đã biết, hoặc đối chiếu với kết quả từ những lần chạy trước của cùng bài test (kiểm thử hồi quy - regression testing).

Later, when we assemble our “software ICs” into a complete system, we’ll have confidence that the individual parts work as expected, and then we can use the same unit test facilities to test the system as a whole. We talk about this large-scale checking of the system in Ruthless and Continuous Testing

Về sau, khi ta lắp ráp các “IC phần mềm” của mình thành một hệ thống hoàn chỉnh, ta sẽ tự tin rằng từng bộ phận riêng lẻ hoạt động đúng như mong đợi, và rồi ta có thể dùng chính các tiện ích kiểm thử đơn vị ấy để kiểm thử cả hệ thống. Chúng tôi bàn về việc kiểm tra hệ thống ở quy mô lớn này trong Kiểm thử Tàn nhẫn và Liên tục.

Before we get that far, however, we need to decide what to test at the unit level. Historically, programmers threw a few random bits of data at the code, looked at the print statements, and called it tested. We can do much better.

Tuy nhiên, trước khi đi xa đến mức đó, ta cần quyết định kiểm thử cái gì ở cấp đơn vị. Trong lịch sử, lập trình viên ném vài mẫu dữ liệu ngẫu nhiên vào mã, nhìn các câu lệnh in ra, rồi tuyên bố là đã kiểm thử xong. Ta có thể làm tốt hơn thế nhiều.

Testing Against Contract — Kiểm thử theo hợp đồng

We like to think of unit testing as testing against contract (see **Topic 23, Design by Contract**). We want to write test cases that ensure that a given unit honors its contract. This will tell us two things: whether the code meets the contract, and whether the contract means what we think it means. We want to test that the module delivers the functionality it promises, over a wide range

Chúng tôi thích nghĩ về kiểm thử đơn vị như là kiểm thử theo hợp đồng (testing against contract — xem Topic 23, Thiết kế theo hợp đồng). Ta muốn viết các trường hợp kiểm thử nhằm bảo đảm rằng một đơn vị nhất định tôn trọng hợp đồng của nó. Điều này cho ta biết hai chuyện: liệu mã có đáp ứng hợp đồng hay không, và liệu hợp đồng có đúng nghĩa như ta tưởng hay không. Ta muốn kiểm thử rằng mô-đun trao đúng chức năng nó hứa, trên một dải rộng

of test cases and boundary conditions.

các trường hợp kiểm thử và điều kiện biên.

What does this mean in practice? Let's start with a simple, numerical example: a square root routine. Its documented contract is simple:

Điều này có nghĩa gì trong thực tế? Hãy bắt đầu với một ví dụ đơn giản, mang tính số học: một thường trình tính căn bậc hai. Hợp đồng được ghi chép của nó rất đơn giản:

pre-conditions: $\text{argument} \geq 0$;

điều kiện tiên quyết (pre-conditions): $\text{argument} \geq 0$;

post-conditions: $(\text{result} * \text{result}) - \text{argument} \leq \text{epsilon} * \text{argument}$;

điều kiện hậu nghiệm (post-conditions): $(\text{result} * \text{result}) - \text{argument} \leq \text{epsilon} * \text{argument}$;

This tells us what to test:

Điều này cho ta biết cần kiểm thử những gì:

Pass in a negative argument and ensure that it is rejected.

Truyền vào một đối số âm và bảo đảm rằng nó bị từ chối.

Pass in an argument of zero to ensure that it is accepted (this is the boundary value).

Truyền vào đối số bằng không để bảo đảm rằng nó được chấp nhận (đây là giá trị biên).

Pass in values between zero and the maximum expressible argument and verify that the difference between the square of the result and the original argument is less than some small fraction of the argument (epsilon).

Truyền vào các giá trị nằm giữa không và đối số lớn nhất biểu diễn được, và xác minh rằng hiệu giữa bình phương của kết quả và đối số ban đầu nhỏ hơn một phần nhỏ nào đó của đối số (epsilon).

Armed with this contract, and assuming that our routine does its own preand postcondition checking, we can write a basic test script to exercise the square root function.

Trang bị bằng hợp đồng này, và giả định rằng thường trình của ta tự kiểm tra điều kiện tiên quyết và hậu nghiệm của chính nó, ta có thể viết một kịch bản kiểm thử cơ bản để vận hành hàm căn bậc hai.

Then we can call this routine to test our square root function:

Rồi ta có thể gọi thường trình này để kiểm thử hàm căn bậc hai của mình:

```
assertWithinEpsilon(my_sqrt(0), 0) assertWithinEpsilon(my_sqrt(2.0), 1.4142135624) assertWithinEpsilon(my_sqrt(8.0), 2.8284271247)
assertWithinEpsilon(my_sqrt(1.0e7), 3162.2776602) assertRaisesException fn => my_sqrt(-4.0)
end
```

```
assertWithinEpsilon(my_sqrt(0), 0) assertWithinEpsilon(my_sqrt(2.0), 1.4142135624)
assertWithinEpsilon(my_sqrt(64.0), 8.0) assertWithinEpsilon(my_sqrt(1.0e7), 3162.2776602)
assertRaisesException fn => my_sqrt(-4.0) end
```

This is a pretty simple test; in the real world, any nontrivial module is likely to be dependent on a number of other modules,

Đây là một bài test khá đơn giản; trong thế giới thực, bất kỳ mô-đun nào không tầm thường đều dễ phụ thuộc vào một số mô-đun khác,

so how do we go about testing the combination?

vậy ta làm sao để kiểm thử cái tổ hợp ấy?

Suppose we have a module A that uses a DataFeed and a LinearRegression . In order, we would test:

Giả sử ta có một mô-đun A dùng đến DataFeed và LinearRegression. Theo thứ tự, ta sẽ kiểm thử:

1. DataFeed 's contract, in full
 2. LinearRegression 's contract, in full
 3. A 's contract, which relies on the other contracts but does not directly expose them
1. Hợp đồng của DataFeed, một cách đầy đủ
 2. Hợp đồng của LinearRegression, một cách đầy đủ
 3. Hợp đồng của A, vốn dựa vào các hợp đồng kia nhưng không phơi bày chúng ra trực tiếp

This style of testing requires you to test subcomponents of a module first. Once the subcomponents have been verified, then the module itself can be tested.

Lối kiểm thử này đòi hỏi bạn phải kiểm thử các thành phần con của một mô-đun trước. Một khi các thành phần con đã được xác minh, thì bản thân mô-đun mới có thể được kiểm thử.

If DataFeed and LinearRegression 's tests passed, but A 's test failed, we can be pretty sure that the problem is in A , or in A 's use of one of those subcomponents. This technique is a great way to reduce **debugging** effort: we can quickly concentrate on the likely source of the problem within module A , and not waste time reexamining its subcomponents.

Nếu các bài test của DataFeed và LinearRegression đều vượt qua, mà bài test của A lại thất bại, ta có thể khá chắc rằng vấn đề nằm ở A, hoặc ở cách A sử dụng một trong những thành phần con ấy. Kỹ thuật này là một cách tuyệt vời để giảm công sức gỡ lỗi: ta có thể nhanh chóng tập trung vào nguồn gốc khả dĩ của vấn đề bên trong mô-đun A, thay vì lãng phí thời gian xem xét lại các thành phần con của nó.

Why do we go to all this trouble? Above all, we want to avoid creating a “time bomb”—something that sits around unnoticed and blows up at an awkward moment later in the project. By emphasizing testing against contract, we can try to avoid as many of those downstream disasters as possible.

Tại sao ta lại lảm chuyện đến thế? Trên hết, ta muốn tránh tạo ra một “quả bom hẹn giờ” — thứ nằm im không ai để ý rồi nổ tung vào một thời điểm ngặt nghèo sau này trong dự án. Bằng cách đề cao kiểm thử theo hợp đồng, ta có thể cố tránh được càng nhiều thảm họa ở hạ nguồn càng tốt.

Tip 69: Design to Test

Tip 69: Hãy Thiết Kế Sao Cho Kiểm Thử Được

Ad Hoc Testing — Kiểm thử tùy biến (ad hoc)

Not to be confused with “odd hack,” ad-hoc testing is when we

Đừng nhầm với “odd hack” (mẹo kỳ quặc), kiểm thử tùy biến (ad-hoc testing) là khi ta run poke at our code manually. This may be as simple as a `console.log()`, or a piece of code entered interactively in a debugger, IDE environment, or REPL.

chạy thử và chọc ngoáy vào mã của mình một cách thủ công. Việc này có thể đơn giản chỉ là một lệnh `console.log()`, hoặc một mẫu mã được gõ tương tác trong trình gỡ lỗi, môi trường IDE, hay REPL.

At the end of the debugging session, you need to formalize this ad hoc test. If the code broke once, it is likely to break again. Don’t just throw away the test you created; add it to the existing unit test arsenal.

Vào cuối phiên gỡ lỗi, bạn cần chính thức hóa bài test tùy biến này. Nếu đoạn mã đã hỏng một lần, nó dễ hỏng lần nữa. Đừng vứt đi bài test bạn vừa tạo; hãy thêm nó vào kho vũ khí kiểm thử đơn vị hiện có.

Build A Test Window — Xây một cửa sổ kiểm thử

Even the best sets of tests are unlikely to find all the bugs; there’s something about the damp, warm conditions of a production environment that seems to bring them out of the woodwork.

Ngay cả những bộ test tốt nhất cũng khó tìm ra hết mọi lỗi; có cái gì đó trong những điều kiện ẩm ướt, ấm áp của môi trường vận hành (production) dường như dụ lũ lỗi bò ra khỏi các khe gỗ.

This means you’ll often need to test a piece of software once it has been deployed—with real-world data flowing through its veins. Unlike a circuit board or chip, we don’t have test pins in software, but we can provide various views into the **internal state** of a module, without using the debugger (which may be inconvenient or impossible in a production application).

Điều này nghĩa là bạn thường sẽ cần kiểm thử một phần mềm sau khi nó đã được triển khai — với dữ liệu thế giới thực chảy trong huyết quản của nó. Khác với một bo mạch hay con chip, trong phần mềm ta không có các chân kiểm thử, nhưng ta có thể cung cấp nhiều góc nhìn khác nhau vào trạng thái nội bộ của một mô-đun, mà không cần dùng trình gỡ lỗi (vốn có thể bất tiện hoặc bất khả thi trong một ứng dụng đang vận hành).

Log files containing trace messages are one such mechanism. Log messages should be in a regular, consistent format; you may want to parse them automatically to deduce processing time or logic paths that the program took. Poorly or inconsistently formatted diagnostics are just so much “spew”—they are difficult to read and impractical to parse.

Các tệp nhật ký (log file) chứa thông điệp truy vết là một cơ chế như vậy. Thông điệp log nên ở một định dạng đều đặn, nhất quán; bạn có thể muốn phân tích cú pháp chúng một cách tự động để suy ra thời gian xử lý hoặc các đường đi logic mà chương trình đã chạy qua. Những chẩn đoán định dạng tệ hoặc thiếu nhất quán chỉ là một mớ “phun trào” — chúng khó đọc và bất khả thi để phân tích cú pháp.

Another mechanism for getting inside running code is the “hotkey” sequence or magic URL. When this particular combination of keys is pressed, or the URL is accessed, a diagnostic control window pops up with status messages and so on. This isn’t

Một cơ chế khác để chui vào bên trong đoạn mã đang chạy là chuỗi “phím nóng” (hotkey) hoặc URL phép thuật. Khi tổ hợp phím cụ thể này được bấm, hoặc URL này được truy cập, một cửa sổ điều khiển chẩn đoán sẽ bật lên với các thông điệp trạng thái và vân vân. Đây không phải

something you normally would reveal to end users, but it can be very handy for the help desk.

thứ bạn thường phơi bày cho người dùng cuối, nhưng nó có thể rất tiện cho bộ phận hỗ trợ (help desk).

More generally, you could use a feature switch to enable extra diagnostics for a particular user or class of users.

Tổng quát hơn, bạn có thể dùng một công tắc tính năng (feature switch) để bật thêm các chẩn đoán cho một người dùng cụ thể hoặc một nhóm người dùng cụ thể.

A Confession

Một lời thú nhận

I (Dave) have been known to tell people that I no longer write tests. Partly I do it to shake the faith of those who have turned testing into a religion. And partly I say it because it is (somewhat) true.

Tôi (Dave) từng được biết đến là người hay nói với mọi người rằng tôi không còn viết test nữa. Một phần tôi làm vậy để lung lay niềm tin của những kẻ đã biến kiểm thử thành một tôn giáo. Và một phần tôi nói thế vì nó (phần nào) đúng.

I’ve been coding for 45 years, and writing automated tests for more than 30 of them. Thinking about testing is built in to the way I approach coding. It felt comfortable. And my personality insists that when something starts to feel comfortable I should move on to something else.

Tôi đã lập trình 45 năm, và viết test tự động hơn 30 năm trong số đó. Việc suy nghĩ về kiểm thử đã ăn sâu vào cách tôi tiếp cận việc viết mã. Nó từng cảm thấy thoải mái. Và tính cách của tôi cứ khẳng khái rằng khi một thứ gì đó bắt đầu trở nên thoải mái thì tôi nên chuyển sang thứ khác.

In this case I decided to stop writing tests for a couple of months and see what it did to my code. To my surprise, the answer was “not a lot.” So I spent some time working out why.

Trong trường hợp này tôi quyết định ngừng viết test trong vài tháng để xem nó tác động thế nào đến mã của mình. Khiến tôi ngạc nhiên, câu trả lời là “chẳng mấy”. Nên tôi dành chút thời gian tìm hiểu lý do.

I believe the answer is that (for me) most of the benefit of testing comes from thinking about the tests and their impact on the code. And, after doing it for so long, I could do that thinking without actually writing tests. My code was still testable; it just wasn’t tested.

Tôi tin câu trả lời là (đối với tôi) phần lớn lợi ích của kiểm thử đến từ việc suy nghĩ về các bài test và tác động của chúng lên mã. Và, sau khi làm việc đó lâu đến vậy, tôi có thể suy

nghĩ như thế mà không thực sự viết ra test. Mã của tôi vẫn kiểm thử được; nó chỉ là không được kiểm thử mà thôi.

But that ignores the fact that tests are also a way of communicating with other developers, so I now do write tests on code shared with others or that relies on the peculiarities of external dependencies.

Nhưng điều đó bỏ qua sự thật rằng test cũng là một cách giao tiếp với các lập trình viên khác, nên giờ tôi vẫn viết test cho mã được chia sẻ với người khác hoặc mã dựa vào những đặc tính kỹ cục của các phụ thuộc bên ngoài.

Andy says I shouldn't include this sidebar. He worries it will tempt inexperienced developers not to test. Here's my compromise:

Andy bảo tôi không nên đưa cái khung lề (sidebar) này vào. Anh ấy lo nó sẽ cám dỗ các lập trình viên non kinh nghiệm bỏ kiểm thử. Đây là sự thỏa hiệp của tôi:

Should you write tests? Yes. But after you've been doing it for 30 years, feel free to experiment a little to see where the benefit lies for you.

Bạn có nên viết test không? Có. Nhưng sau khi đã làm việc đó được 30 năm, cứ thoải mái thử nghiệm một chút để xem lợi ích nằm ở đâu đối với riêng bạn.

A Culture Of Testing — Một văn hóa kiểm thử

All software you write will be tested—if not by you and your team, then by the eventual users—so you might as well plan on testing it thoroughly. A little forethought can go a long way toward minimizing maintenance costs and help-desk calls.

Mọi phần mềm bạn viết đều sẽ được kiểm thử — nếu không phải bởi bạn và nhóm của bạn, thì bởi những người dùng cuối — vậy nên bạn cũng nên dự tính kiểm thử nó cho kỹ lưỡng luôn đi. Một chút lo xa có thể giúp ích rất nhiều trong việc giảm thiểu chi phí bảo trì và những cuộc gọi đến bộ phận hỗ trợ.

You really only have a few choices:

Bạn thực sự chỉ có vài lựa chọn:

Test First

Kiểm thử Trước (Test First)

Test During

Kiểm thử Trong khi (Test During)

Test Never

Không bao giờ Kiểm thử (Test Never)

Test First, including Test-Driven Design, is probably your best choice in most circumstances, as it ensures that testing happens. But sometimes that's not as convenient or useful, so Test During coding can be a good fallback, where you write some code, fiddle with it, write the tests for it, then move on to the next bit. The worst choice is often called "Test Later," but who are you kidding? "Test Later" really means "Test Never."

Kiểm thử Trước, bao gồm cả Thiết kế Hướng Kiểm thử (Test-Driven Design), có lẽ là lựa chọn tốt nhất của bạn trong hầu hết hoàn cảnh, vì nó bảo đảm rằng việc kiểm thử có diễn ra. Nhưng đôi khi điều đó không tiện hoặc không hữu dụng lắm, nên Kiểm thử Trong khi viết mã có thể là một phương án dự phòng tốt, trong đó bạn viết một ít mã, mày mò với

nó, viết test cho nó, rồi chuyển sang mẫu tiếp theo. Lựa chọn tệ nhất thường được gọi là “Kiểm thử Sau” (Test Later), nhưng bạn đang lừa ai vậy? “Kiểm thử Sau” thực ra có nghĩa là “Không bao giờ Kiểm thử”.

A culture of testing means all the tests pass all the time. Ignore a spew of tests that “always fail” makes it easier to ignore all the tests, and the vicious spiral begins (see Topic 3, **Software Entropy**).

Một văn hóa kiểm thử nghĩa là mọi test đều vượt qua mọi lúc. Mặc kệ một mớ test cứ “luôn luôn thất bại” khiến ta dễ phớt lờ luôn cả các test khác, và vòng xoáy luẩn quẩn bắt đầu (xem Topic 3, Sự rối loạn phần mềm).

Treat test code with the same care as any production code. Keep it decoupled, clean, and robust. Don’t rely on unreliable things (see Topic 38, **Programming by Coincidence**) like the absolute position of widgets in a GUI system, or exact timestamps in a server log, or the exact wording of error messages. Testing for these sorts of things will result in fragile tests.

Hãy đối xử với mã kiểm thử cẩn thận như với bất kỳ mã vận hành nào. Giữ nó gọn ghẽ, nối, sạch sẽ, và bền bỉ. Đừng dựa vào những thứ không đáng tin (xem Topic 38, Lập trình theo may rủi) như vị trí tuyệt đối của các widget trong một hệ thống GUI, hay các dấu thời gian chính xác trong một bản log máy chủ, hay câu chữ chính xác của các thông điệp lỗi. Kiểm thử cho những thứ kiểu này sẽ tạo ra các bài test mong manh.

Tip 70: Test Your Software, or Your Users Will

Tip 70: Hãy Kiểm Thử Phần Mềm Của Bạn, Nếu Không Người Dùng Sẽ Kiểm Thử Họ

Make no mistake, testing is part of programming. It’s not something left to other departments or staff.

Đừng nhầm lẫn, kiểm thử là một phần của lập trình. Nó không phải thứ để giao phó cho các bộ phận hay nhân sự khác.

Testing, design, coding—it’s all programming.

Kiểm thử, thiết kế, viết mã — tất cả đều là lập trình.

Related Sections Include — Các mục liên quan

Topic 27, Don’t Outrun Your Headlights

Topic 27, Đừng phóng nhanh quá tầm đèn pha

Topic 51, **Pragmatic Starter Kit**

Topic 51, Bộ khởi đầu Thực dụng

Доверяй , но проверяй (Trust, but verify) Russian proverb

Доверяй, но проверяй (Tin tưởng, nhưng phải kiểm chứng) Tục ngữ Nga

Topic 42. Property-Based Testing — Topic 42. Kiểm thử dựa trên thuộc tính

We recommend writing unit tests for your functions. You do that by thinking about typical things that might be a problem, based on your knowledge of the thing you’re testing.

Chúng tôi khuyên bạn nên viết kiểm thử đơn vị (unit test) cho các hàm của mình. Bạn làm điều đó bằng cách nghĩ về những tình huống điển hình có thể gây trục trặc, dựa trên hiểu biết của bạn về thứ đang được kiểm thử.

There's a small but potentially significant problem lurking in that paragraph, though. If you write the original code and you write the tests, is it possible that an incorrect assumption could be expressed in both? The code passes the tests, because it does what it is supposed to based on your understanding.

Tuy nhiên, có một vấn đề nhỏ nhưng có khả năng rất đáng kể ẩn nấp trong đoạn vừa rồi. Nếu bạn viết cả mã gốc lẫn các bài kiểm thử, liệu có khả năng một giả định sai lầm được thể hiện trong cả hai không? Mã vượt qua các bài kiểm thử, bởi vì nó làm đúng những gì lẽ ra phải làm theo cách bạn hiểu.

One way around this is to have different people write tests and the code under test, but we don't like this: as we said in **Topic 41, Test to Code**, one of the biggest benefits of thinking about tests is the way it informs the code you write. You lose that when the work of testing is split from the coding.

Một cách để né tránh chuyện này là để người này viết kiểm thử còn người kia viết mã cần kiểm thử, nhưng chúng tôi không thích cách đó: như đã nói ở Topic 41, Kiểm thử trước, một trong những lợi ích lớn nhất của việc suy nghĩ về kiểm thử chính là cách nó định hình mã mà bạn viết. Bạn sẽ mất đi lợi ích đó khi tách công việc kiểm thử ra khỏi việc lập trình.

Instead, we favor an alternative, where the computer, which doesn't share your preconceptions, does some testing for you.

Thay vào đó, chúng tôi nghiêng về một lựa chọn khác: để cho máy tính — vốn không mang sẵn những định kiến của bạn — kiểm thử giúp bạn.

Contracts, Invariants, And Properties — Hợp đồng, bất biến và thuộc tính

In Topic 23, **Design by Contract**, we talked about the idea that code has contracts that it meets: you meet the conditions when you feed it input, and it will make certain guarantees about the

Ở Topic 23, Thiết kế theo hợp đồng, chúng ta đã bàn về ý tưởng rằng mã có những hợp đồng mà nó phải tuân thủ: bạn đáp ứng các điều kiện khi đưa đầu vào cho nó, và nó sẽ đảm bảo một số điều nhất định về

outputs it produces.

đầu ra mà nó tạo ra.

There are also code invariants, things that remain true about some piece of **state** when it's passed through a function. For example, if you sort a list, the result will have the same number of elements as the original—the length is invariant.

Cũng có những bất biến (invariant) của mã, tức những điều luôn đúng về một mảng trạng thái nào đó khi nó được truyền qua một hàm. Ví dụ, nếu bạn sắp xếp một danh sách, kết quả sẽ có cùng số phần tử như ban đầu — độ dài là bất biến.

Once we work out our contracts and invariants (which we're going to lump together and call properties) we can use them to automate our testing. What we end up doing is called propertybased testing.

Một khi đã xác định được các hợp đồng và bất biến (mà chúng ta sẽ gom chung lại và gọi là thuộc tính — property), ta có thể dùng chúng để tự động hóa việc kiểm thử. Thứ ta thu được chính là kiểm thử dựa trên thuộc tính (property-based testing).

Use Property-Based Tests to Validate Your — Dùng kiểm thử dựa trên thuộc tính để kiểm chứng

Tip 71: Assumptions

Tip 71: các giả định của bạn

As an artificial example, we can build some tests for our sorted list. We've already established one property: the sorted list is the same size as the original. We can also state that no element in the result can be greater than the one that follows it.

Lấy một ví dụ giả định, ta có thể dựng vài bài kiểm thử cho danh sách đã sắp xếp của mình. Ta đã thiết lập được một thuộc tính: danh sách đã sắp xếp có cùng kích thước với danh sách gốc. Ta cũng có thể phát biểu rằng không phần tử nào trong kết quả được phép lớn hơn phần tử đứng ngay sau nó.

We can now express that in code. Most languages have some kind of **property-based testing** framework. This example is in Python, and uses the Hypothesis tool and pytest, but the principles are pretty universal.

Giờ ta có thể diễn đạt điều đó bằng mã. Phần lớn ngôn ngữ đều có một dạng framework kiểm thử dựa trên thuộc tính nào đó. Ví dụ này viết bằng Python, dùng công cụ Hypothesis và pytest, nhưng các nguyên tắc thì khá phổ quát.

Here is the full source of the tests:

Đây là toàn bộ mã nguồn của các bài kiểm thử:

```
proptest/sort.py
```

```
proptest/sort.py
```

```
from hypothesis import given import hypothesis.strategies as some
```

```
from hypothesis import given import hypothesis.strategies as some
```

```
@given (some.lists(some.integers())) def test_list_size_is_invariant_across_sorting(a_list):
```

```
@given (some.lists(some.integers())) def test_list_size_is_invariant_across_sorting(a_list):
```

```
original_length = len(a_list) a_list.sort() assert len(a_list) == original_length
```

```
original_length = len(a_list) a_list.sort() assert len(a_list) == original_length
```

```
@given (some.lists(some.text())) def test_sorted_result_is_ordered(a_list): a_list.sort() for i in range(len(a_list) - 1): assert a_list[i] <= a_list[i + 1]
```

```
@given (some.lists(some.text())) def test_sorted_result_is_ordered(a_list): a_list.sort() for i in range(len(a_list) - 1): assert a_list[i] <= a_list[i + 1]
```

Here's what happens when we run it:

Đây là những gì xảy ra khi ta chạy nó:

```
$ pytest sort.py ===== test session starts ===== ... plugins: hypothesis-4.14.0
```

```
$ pytest sort.py ===== test session starts ===== ... plugins: hypothesis-4.14.0
```

```
sort.py .. [100%]
```

```
sort.py .. [100%]
```

===== 2 passed in 0.95 seconds =====

===== 2 passed in 0.95 seconds =====

Not much drama there. But, behind the scenes, Hypothesis ran both of our tests one hundred times, passing in a different list each time. The lists will have varying lengths, and will have different contents. It's as if we'd cooked up 200 individual tests with 200 random lists.

Chẳng có gì kịch tính. Nhưng ở hậu trường, Hypothesis đã chạy mỗi bài kiểm thử của ta một trăm lần, mỗi lần truyền vào một danh sách khác nhau. Các danh sách có độ dài thay đổi, và có nội dung khác nhau. Cứ như thể ta đã pha chế ra 200 bài kiểm thử riêng lẻ với 200 danh sách ngẫu nhiên.

Test Data Generation — Sinh dữ liệu kiểm thử

Like most property-based testing libraries, Hypothesis gives you a minilanguage for describing the data it should generate. The language is based around calls to functions in the `hypothesis.strategies` module, which we aliased as `some`, just because it reads better.

Giống như hầu hết các thư viện kiểm thử dựa trên thuộc tính, Hypothesis cho bạn một ngôn ngữ thu nhỏ (mini-language) để mô tả dữ liệu mà nó cần sinh ra. Ngôn ngữ này xoay quanh các lời gọi hàm trong module `hypothesis.strategies`, mà ta đặt bí danh là `some`, đơn giản vì đọc lên nghe xuôi hơn.

If we wrote:

Nếu ta viết:

```
@given(some.integers())
```

```
@given(some.integers())
```

Our test function would run multiple times. Each time, it would be passed a different integer. If instead we wrote the following:

Hàm kiểm thử của ta sẽ chạy nhiều lần. Mỗi lần, nó được truyền vào một số nguyên khác nhau. Còn nếu thay vào đó ta viết như sau:

```
@given(some.integers(min_value=5, max_value=10).map(lambda x: x * 2))
```

```
@given(some.integers(min_value=5, max_value=10).map(lambda x: x * 2))
```

then we'd get the even numbers between 10 and 20.

thì ta sẽ thu được các số chẵn nằm giữa 10 và 20.

You can also compose types, so that

Bạn cũng có thể kết hợp các kiểu, sao cho

```
@given(some.lists(some.integers(min_value=1), max_size=100))
```

```
@given(some.lists(some.integers(min_value=1), max_size=100))
```

will be lists of natural numbers that are at most 100 elements long.

sẽ cho ra các danh sách số tự nhiên dài tối đa 100 phần tử.

This isn't supposed to be a tutorial on any particular framework, so we'll skip a bunch of cool details and instead look at a realworld example.

Đây không phải là một bài hướng dẫn về bất kỳ framework cụ thể nào, nên ta sẽ bỏ qua một loạt chi tiết thú vị và thay vào đó xem một ví dụ thực tế.

Finding Bad Assumptions — Phát hiện những giả định sai lầm

We’re writing a simple order processing and stock control system (because there’s always room for one more). It models the stock levels with a Warehouse object. We can query a warehouse to see if something is in stock, remove things from stock, and get the current stock levels.

Ta đang viết một hệ thống xử lý đơn hàng và quản lý kho đơn giản (bởi lúc nào cũng còn chỗ cho thêm một cái nữa). Nó mô hình hóa mức tồn kho bằng một đối tượng Warehouse. Ta có thể hỏi một kho xem một món hàng còn hay không, lấy hàng ra khỏi kho, và xem mức tồn kho hiện tại.

Here’s the code:

Đây là mã:

proptest/stock.py

proptest/stock.py

```
class Warehouse: def init(self, stock): self.stock = stock
```

```
class Warehouse: def init(self, stock): self.stock = stock
```

```
def in_stock(self, item_name):
```

```
def in_stock(self, item_name):
```

```
return (item_name in self.stock) and (self.stock[item_name] > 0)
```

```
return (item_name in self.stock) and (self.stock[item_name] > 0)
```

```
def take_from_stock(self, item_name, quantity): if quantity <= self.stock[item_name]: self.stock[item_name] -= quantity else : raise Exception( “Oversold {}” .format(item_name))
```

```
def take_from_stock(self, item_name, quantity): if quantity <= self.stock[item_name]: self.stock[item_name] -= quantity else : raise Exception( “Oversold {}” .format(item_name))
```

```
def stock_count(self, item_name): return self.stock[item_name]
```

```
def stock_count(self, item_name): return self.stock[item_name]
```

We wrote a basic unit test, which passes:

Ta viết một kiểm thử đơn vị cơ bản, và nó vượt qua:

proptest/stock.py

proptest/stock.py

```
def test_warehouse(): wh = Warehouse({ “shoes” : 10, “hats” : 2, “umbrellas” : 0}) assert wh.in_stock( “shoes” ) assert wh.in_stock( “hats” ) assert not wh.in_stock( “umbrellas” )
```

```
def test_warehouse(): wh = Warehouse({ “shoes” : 10, “hats” : 2, “umbrellas” : 0}) assert wh.in_stock( “shoes” ) assert wh.in_stock( “hats” ) assert not wh.in_stock( “umbrellas” )
```

```
wh.take_from_stock( “shoes” , 2) assert wh.in_stock( “shoes” )
```

```
wh.take_from_stock( “shoes” , 2) assert wh.in_stock( “shoes” )
```

```
wh.take_from_stock( “hats” , 2) assert not wh.in_stock( “hats” )
```



```
wh.take_from_stock( "hats" , 2) assert not wh.in_stock( "hats" )
```

Then we wrote a function that processes a request to order items from the warehouse. It returns a tuple where the first element is either “ok” or “not available” , followed by the item and requested quantity. We also wrote some tests, and they pass:

Rồi ta viết một hàm xử lý yêu cầu đặt hàng từ kho. Nó trả về một tuple mà phần tử đầu tiên là “ok” hoặc “not available” , theo sau là món hàng và số lượng được yêu cầu. Ta cũng viết vài bài kiểm thử, và chúng vượt qua:

```
proptest/stock.py
```

```
proptest/stock.py
```

```
def order(warehouse, item, quantity): if warehouse.in_stock(item): warehouse.take_from_stock(item, quantity) return ( "ok" , item, quantity ) else : return ( "not available" , item, quantity )
```

```
def order(warehouse, item, quantity): if warehouse.in_stock(item): warehouse.take_from_stock(item, quantity) return ( "ok" , item, quantity ) else : return ( "not available" , item, quantity )
```

```
proptest/stock.py
```

```
proptest/stock.py
```

```
def test_order_in_stock(): wh = Warehouse({ "shoes" : 10, "hats" : 2, "umbrellas" : 0}) status, item, quantity = order(wh, "hats" , 1) assert status == "ok" assert item == "hats" assert quantity == 1 assert wh.stock_count( "hats" ) == 1
```

```
def test_order_in_stock(): wh = Warehouse({ "shoes" : 10, "hats" : 2, "umbrellas" : 0}) status, item, quantity = order(wh, "hats" , 1) assert status == "ok" assert item == "hats" assert quantity == 1 assert wh.stock_count( "hats" ) == 1
```

```
def test_order_not_in_stock(): wh = Warehouse({ "shoes" : 10, "hats" : 2, "umbrellas" : 0}) status, item, quantity = order(wh, "umbrellas" , 1) assert status == "not available" assert item == "umbrellas" assert quantity == 1 assert wh.stock_count( "umbrellas" ) == 0
```

```
def test_order_not_in_stock(): wh = Warehouse({ "shoes" : 10, "hats" : 2, "umbrellas" : 0}) status, item, quantity = order(wh, "umbrellas" , 1) assert status == "not available" assert item == "umbrellas" assert quantity == 1 assert wh.stock_count( "umbrellas" ) == 0
```

```
def test_order_unknown_item(): wh = Warehouse({ "shoes" : 10, "hats" : 2, "umbrellas" : 0}) status, item, quantity = order(wh, "bagel" , 1) assert status == "not available" assert item == "bagel" assert quantity == 1
```

```
def test_order_unknown_item(): wh = Warehouse({ "shoes" : 10, "hats" : 2, "umbrellas" : 0}) status, item, quantity = order(wh, "bagel" , 1) assert status == "not available" assert item == "bagel" assert quantity == 1
```

On the surface, everything looks fine. But before we ship the code, let’s add some property tests.

Nhìn bề ngoài, mọi thứ có vẻ ổn. Nhưng trước khi đem mã đi giao, hãy thêm vài bài kiểm thử thuộc tính.

One thing we know is that stock cannot appear and disappear across our transaction. This means that if we take some items from the warehouse, the number we took plus the number currently in the warehouse should be the same as the number originally in the warehouse. In the following test, we run our test with the item parameter chosen randomly from “hat” or “shoe” and the quantity chosen from 1 to 4:

Một điều ta biết chắc là hàng không thể tự nhiên xuất hiện hay biến mất qua giao dịch của ta. Nghĩa là nếu ta lấy một số món ra khỏi kho, thì số đã lấy cộng với số hiện còn trong kho phải bằng đúng số ban đầu có trong kho. Ở bài kiểm thử sau, ta chạy kiểm thử với tham số item được chọn ngẫu nhiên từ “hat” hoặc “shoe” và quantity được chọn từ 1 đến 4:

proptest/stock.py

proptest/stock.py

```
@given (item = some.sampled_from([ "shoes" , "hats" ]), quantity = some.integers(min_value=1,
max_value=4))
```

```
@given (item = some.sampled_from([ "shoes" , "hats" ]), quantity = some.integers(min_value=1,
max_value=4))
```

```
def test_stock_level_plus_quantity_equals_original_stock_level(item, quantity): wh = Warehouse({
"shoes" : 10, "hats" : 2, "umbrellas" : 0}) initial_stock_level = wh.stock_count(item) (status, item,
quantity) = order(wh, item, quantity) if status == "ok" : assert wh.stock_count(item) + quantity ==
initial_stock_level
```

```
def test_stock_level_plus_quantity_equals_original_stock_level(item, quantity): wh =
Warehouse({ "shoes" : 10, "hats" : 2, "umbrellas" : 0}) initial_stock_level = wh.stock_count(item)
(status, item, quantity) = order(wh, item, quantity) if status == "ok" : assert wh.stock_count(item)
+ quantity == initial_stock_level
```

Let's run it:

Hãy chạy nó:

```
$ pytest stock.py ... stock.py:72: _____ stock.py:76: in
test_stock_level_plus_quantity_equals_original_stock_level (status, item, quantity) = order(wh, item,
quantity) stock.py:40: in order warehouse.take_from_stock(item, quantity) _____
```

```
_____
$ pytest stock.py ... stock.py:72: _____ stock.py:76:
in test_stock_level_plus_quantity_equals_original_stock_level (status, item, quantity)
= order(wh, item, quantity) stock.py:40: in order warehouse.take_from_stock(item,
quantity) _____
```

```
self = <stock.Warehouse object at 0x10cf97cf8>, item_name = 'hats' quantity = 3
```

```
self = <stock.Warehouse object at 0x10cf97cf8>, item_name = 'hats' quantity = 3
```

```
def take_from_stock(self, item_name, quantity): if quantity <= self.stock[item_name]: self.stock[item_name]
-= quantity else: > raise Exception( "Oversold {}" .format(item_name)) E Exception: Oversold hats
```

```
def take_from_stock(self, item_name, quantity): if quantity <= self.stock[item_name]:
self.stock[item_name] -= quantity else: > raise Exception( "Oversold {}" .format(item_name))
E Exception: Oversold hats
```

```
stock.py:16: Exception _____ Hypothesis _____ Falsifying example:
test_stock_level_plus_quantity_equals_original_stock_level( item='hats', quantity=3)
```

```
stock.py:16: Exception _____ Hypothesis _____ Falsifying
example: test_stock_level_plus_quantity_equals_original_stock_level( item='hats',
quantity=3)
```

It blew up in warehouse.take_from_stock : we tried to remove three hats from the warehouse, but it only has two in stock.

Nó nổ tung trong `warehouse.take_from_stock` : ta thử lấy ba chiếc mũ ra khỏi kho, nhưng kho chỉ còn hai chiếc.

Our property testing found a faulty assumption: our `in_stock` function only checks that there's at least one of the given item in

Bài kiểm thử thuộc tính của ta đã tìm ra một giả định sai: hàm `in_stock` của ta chỉ kiểm tra rằng có ít nhất một món hàng đã cho trong

stock. Instead we need to make sure we have enough to fill the order:

kho. Đáng lẽ ta phải đảm bảo có đủ hàng để đáp ứng đơn đặt:

proptest/stock1.py

proptest/stock1.py

```
def in_stock(self, item_name, quantity): » return (item_name in self.stock) and (self.stock[item_name]
>= quantity)
```

```
def in_stock(self, item_name, quantity): » return (item_name in self.stock) and
(self.stock[item_name] >= quantity)
```

And we change the order function, too:

Và ta cũng sửa luôn hàm order:

proptest/stock1.py

proptest/stock1.py

```
def order(warehouse, item, quantity): » if warehouse.in_stock(item, quantity): warehouse.take_from_stock(item,
quantity) return ( "ok" , item, quantity ) else : return ( "not available" , item, quantity )
```

```
def order(warehouse, item, quantity): » if warehouse.in_stock(item, quantity): warehouse.take_from_stock(item,
quantity) return ( "ok" , item, quantity ) else : return ( "not available" , item, quantity )
```

And now our property test passes.

Và giờ bài kiểm thử thuộc tính của ta vượt qua.

Property-Based Tests Often Surprise You — Kiểm thử dựa trên thuộc tính thường khiến bạn bất ngờ

In the previous example, we used a property-based test to check that stock levels were adjusted properly. The test found a bug, but it wasn't to do with stock level adjustment. Instead, it found a bug in our `in_stock` function.

Ở ví dụ trước, ta đã dùng một bài kiểm thử dựa trên thuộc tính để kiểm tra rằng mức tồn kho được điều chỉnh đúng. Bài kiểm thử tìm ra một lỗi, nhưng lỗi đó lại chẳng liên quan gì đến việc điều chỉnh mức tồn kho. Thay vào đó, nó tìm ra một lỗi trong hàm `in_stock` của ta.

This is both the power and the frustration of property-based testing. It's powerful because you set up some rules for generating inputs, set up some assertions for validating output, and then just let it rip. You never quite know what will happen. The test may pass. An **assertion** may fail. Or the code may fail totally because it couldn't handle the inputs it was given.

Đây vừa là sức mạnh vừa là nỗi bức bối của kiểm thử dựa trên thuộc tính. Nó mạnh ở chỗ bạn thiết lập vài quy tắc để sinh đầu vào, thiết lập vài khẳng định (assertion) để kiểm

chứng đầu ra, rồi cứ thế thả cho nó chạy. Bạn chẳng bao giờ biết chắc chuyện gì sẽ xảy ra. Bài kiểm thử có thể vượt qua. Một khẳng định có thể thất bại. Hoặc mã có thể đổ vỡ hoàn toàn vì nó không xử lý nổi đầu vào được đưa cho.

The frustration is that it can be tricky to pin down what failed.

Nỗi bực bội nằm ở chỗ có thể khá nhằn để truy ra chính xác cái gì đã thất bại.

Our suggestion is that when a property-based test fails, find out what parameters it was passing to the test function, and then use those values to create a separate, regular, unit test. That unit test does two things for you. First, it lets you focus in on the problem without all the additional calls being made into your code by the property-based testing framework. Second, that unit test acts as a regression test. Because property-based tests generate random values that get passed to your test, there's no guarantee that the same values will be used the next time you run tests. Having a unit test that forces those values to be used ensures that this bug won't slip through.

Gợi ý của chúng tôi là: khi một bài kiểm thử thuộc tính thất bại, hãy tìm xem nó đã truyền những tham số nào vào hàm kiểm thử, rồi dùng các giá trị đó để tạo một bài kiểm thử đơn vị riêng, thông thường. Bài kiểm thử đơn vị đó làm hai việc cho bạn. Thứ nhất, nó cho phép bạn tập trung vào vấn đề mà không bị nhiễu bởi tất cả những lời gọi bổ sung mà framework kiểm thử dựa trên thuộc tính tổng vào mã của bạn. Thứ hai, bài kiểm thử đơn vị đó đóng vai trò một kiểm thử hồi quy (regression test). Bởi vì kiểm thử dựa trên thuộc tính sinh ra các giá trị ngẫu nhiên để truyền vào bài kiểm thử, không có gì đảm bảo cùng những giá trị đó sẽ được dùng lại ở lần chạy tiếp theo. Có một bài kiểm thử đơn vị ép buộc dùng đúng các giá trị đó sẽ đảm bảo lỗi này không lọt lưới.

Property-Based Tests Also Help Your Design — Kiểm thử dựa trên thuộc tính cũng giúp ích cho thiết kế của bạn

When we talked about unit testing, we said that one of the major benefits was the way it made you think about your code: a unit test is the first **client** of your API.

Khi bàn về kiểm thử đơn vị, ta đã nói rằng một trong những lợi ích lớn của nó là cách nó buộc bạn phải suy nghĩ về mã của mình: một bài kiểm thử đơn vị là khách hàng đầu tiên của API của bạn.

The same is true of property-based tests, but in a slightly different way. They make you think about your code in terms of invariants and contracts; you think about what must not change, and what must be true. This extra insight has a magical effect on your code, removing edge cases and highlighting functions that leave data in an inconsistent state.

Điều đó cũng đúng với kiểm thử dựa trên thuộc tính, nhưng theo một cách hơi khác. Chúng buộc bạn nghĩ về mã của mình dưới góc độ bất biến và hợp đồng; bạn nghĩ về cái gì không được phép thay đổi, và cái gì phải luôn đúng. Sự thấu hiểu thêm này có một tác dụng kỳ diệu lên mã của bạn, loại bỏ các trường hợp biên và làm lộ ra những hàm để lại dữ liệu ở trạng thái không nhất quán.

We believe that property-based testing is complementary to unit testing: they address different concerns, and each brings its own benefits. If you're not currently using them, give them a go.

Chúng tôi tin rằng kiểm thử dựa trên thuộc tính bổ trợ cho kiểm thử đơn vị: chúng giải quyết những mối quan tâm khác nhau, và mỗi loại mang lại lợi ích riêng. Nếu hiện giờ bạn chưa dùng chúng, hãy thử xem.

Related Sections Include — Các mục liên quan

Topic 23, Design by Contract

Topic 23, Thiết kế theo hợp đồng

Topic 25, **Assertive Programming**

Topic 25, Lập trình phòng vệ bằng assertion

Topic 45, The **Requirements** Pit

Topic 45, Cái hố yêu cầu

Exercises — Bài tập

Exercise 31 (possible answer)

Bài tập 31 (đáp án gợi ý)

Look back at the warehouse example. Are there any other properties that you can test?

Nhìn lại ví dụ về kho hàng. Còn thuộc tính nào khác mà bạn có thể kiểm thử không?

Exercise 32 (possible answer)

Bài tập 32 (đáp án gợi ý)

Your company ships machinery. Each machine comes in a crate, and each crate is rectangular. The crates vary in size. Your job is to write some code to pack as many crates as possible in a single layer that fits in the delivery truck. The output of your code is a list of all the crates. For each crate, the list gives the location in the truck, along with the width and height. What properties of the output could be tested?

Công ty bạn vận chuyển máy móc. Mỗi cỗ máy đựng trong một thùng, và mỗi thùng đều hình chữ nhật. Các thùng có kích thước khác nhau. Nhiệm vụ của bạn là viết mã để xếp được càng nhiều thùng càng tốt trong một lớp đơn vừa khít sàn xe tải giao hàng. Đầu ra của mã là danh sách tất cả các thùng. Với mỗi thùng, danh sách cho biết vị trí trong xe tải, kèm chiều rộng và chiều cao. Những thuộc tính nào của đầu ra có thể được kiểm thử?

Challenges — Thử thách

Think about the code you're currently working on. What are the properties: the contracts and invariants? Can you use propertybased testing framework to verify these automatically?

Hãy nghĩ về đoạn mã bạn đang làm việc hiện tại. Đây là các thuộc tính: các hợp đồng và bất biến? Bạn có thể dùng framework kiểm thử dựa trên thuộc tính để kiểm chứng chúng một cách tự động không?

Good fences make good neighbors. Robert Frost, Mending Wall

Hàng rào tốt tạo nên láng giềng tốt. Robert Frost, Mending Wall

Topic 43. Stay Safe Out There — Topic 43. Hãy An Toàn Ngoài Kia

In the first edition's discussion of code **coupling** we made a bold and naive statement: “we don't need to be as paranoid as spies or dissidents.” We were wrong. In fact, you do need to be that paranoid, every day.

Trong lần thảo luận về sự ghép nối (coupling) của mã ở ấn bản đầu tiên, chúng tôi đã đưa ra một tuyên bố táo bạo và ngây thơ: “chúng ta không cần phải hoang tưởng như gián điệp hay những kẻ bất đồng chính kiến.” Chúng tôi đã sai. Thực tế, bạn cần phải hoang tưởng đến mức đó, mỗi ngày.

As we write this, the daily news is filled with stories of devastating data breaches, hijacked systems, and cyberfraud. Hundreds of millions of records stolen at once, billions and billions of dollars in losses and remediation—and these numbers are growing rapidly each year. In the vast majority of cases, it’s not because the attackers were terribly clever, or even vaguely competent.

Vào lúc chúng tôi viết những dòng này, tin tức hằng ngày tràn ngập những câu chuyện về các vụ rò rỉ dữ liệu thảm khốc, hệ thống bị chiếm quyền, và lừa đảo qua mạng. Hàng trăm triệu bản ghi bị đánh cắp cùng lúc, hàng tỷ tỷ đô la thiệt hại và chi phí khắc phục — và những con số này tăng nhanh chóng mỗi năm. Trong đại đa số các trường hợp, không phải vì kẻ tấn công cực kỳ thông minh, hay thậm chí chỉ là kha khá có năng lực.

It’s because the developers were careless.

Mà vì các lập trình viên đã bất cẩn.

The Other 90% — 90% Còn Lại

When coding, you may go through several cycles of “it works!” and “why isn’t that working?” with the occasional “there’s no way that could have happened...” [62] After several hills and bumps on this uphill climb, it’s easy to say to yourself, “pew, it all works!” and proclaim the code done. Of course, it’s not done yet. You’re 90% done, but now you have the other 90% to consider.

Khi lập trình, bạn có thể trải qua vài vòng “chạy được rồi!” và “sao cái đó lại không chạy?”, thỉnh thoảng xen vào “không đời nào chuyện đó lại xảy ra...” [62] Sau vài đợt lên dốc xuống dốc trên con đường leo núi này, thật dễ để tự nhủ “phù, xong hết rồi!” và tuyên bố mã đã hoàn tất. Dĩ nhiên là chưa xong đâu. Bạn mới xong 90%, nhưng giờ bạn còn phải tính đến 90% còn lại.

The next thing you have to do is analyze the code for ways it can go wrong and add those to your test suite. You’ll consider things such as passing in bad parameters, leaking or unavailable resources; that sort of thing.

Việc tiếp theo bạn phải làm là phân tích mã để tìm những cách nó có thể hỏng và thêm các trường hợp đó vào bộ kiểm thử. Bạn sẽ cân nhắc những thứ như truyền vào tham số xấu, rò rỉ hay không sẵn có tài nguyên; đại loại như vậy.

In the good old days, this evaluation of **internal** errors may have been sufficient. But today that’s only the beginning, because in addition to errors from internal causes, you need to consider how an external **actor** could deliberately screw up the system. But perhaps you protest, “Oh, no one will care about this code, it’s not important, no one even knows about this server...” It’s a big world out there, and most of it is connected. Whether it’s a bored kid on the other side of the planet, **state**-sponsored terrorism, criminal gangs, corporate espionage, or even a vengeful ex, they are out there and aiming for you. The survival time of an unpatched, outdated system on the open net is measured in minutes—or even less.

Vào thời xưa tốt đẹp, việc đánh giá các lỗi nội bộ này có lẽ là đủ. Nhưng ngày nay đó mới chỉ là khởi đầu, bởi vì ngoài các lỗi từ nguyên nhân nội bộ, bạn còn cần cân nhắc xem một tác nhân bên ngoài có thể cố tình phá hỏng hệ thống ra sao. Nhưng có lẽ bạn sẽ phản đối: “Ồ, chẳng ai thèm quan tâm đến đoạn mã này đâu, nó có quan trọng gì, thậm chí chẳng ai biết đến cái server này...”. Thế giới ngoài kia rộng lớn lắm, và phần lớn nó được kết nối

với nhau. Dù là một đứa trẻ buồn chán ở phía bên kia hành tinh, khủng bố do nhà nước bảo trợ, các băng nhóm tội phạm, gián điệp doanh nghiệp, hay thậm chí một người yêu cũ đầy thù hận, thì họ đều ngoài kia và đang nhắm vào bạn. Thời gian sống sót của một hệ thống chưa vá lỗi, lỗi thời trên mạng công khai được đo bằng phút — hoặc thậm chí ít hơn.

Security through obscurity just doesn't work.

An ninh bằng cách giấu nhem (security through obscurity) đơn giản là không hiệu quả.

Security Basic Principles — Những Nguyên Tắc An Ninh Cơ Bản

Pragmatic Programmers have a healthy amount of paranoia. We know we have faults and limitations, and that external attackers will seize on any opening we leave to compromise our systems. Your particular development and deployment environments will have their own security-centric needs, but there are a handful of basic principles that you should always bear in mind:

Các Lập trình viên Thực dụng (Pragmatic Programmer) có một liều lượng hoang tưởng lành mạnh. Chúng tôi biết mình có khuyết điểm và giới hạn, và rằng kẻ tấn công bên ngoài sẽ chớp lấy bất kỳ khe hở nào ta để lại để xâm phạm hệ thống. Môi trường phát triển và triển khai cụ thể của bạn sẽ có những nhu cầu an ninh riêng, nhưng có một nhóm nguyên tắc cơ bản mà bạn nên luôn ghi nhớ:

1. Minimize **Attack Surface** Area
2. **Principle of Least Privilege**
3. **Secure Defaults**
4. Encrypt Sensitive Data
5. Maintain Security Updates

1. Tối thiểu hóa Diện tích Bề mặt Tấn công
2. Nguyên tắc Đặc quyền Tối thiểu
3. Mặc định An toàn
4. Mã hóa Dữ liệu Nhạy cảm
5. Duy trì Cập nhật Bảo mật

Let's take a look at each of these.

Hãy cùng xem qua từng nguyên tắc.

Minimize Attack Surface Area The attack surface area of a system is the sum of all access points where an attacker can enter data, extract data, or invoke execution of a service. Here are a few examples:

Tối thiểu hóa Diện tích Bề mặt Tấn công Diện tích bề mặt tấn công (attack surface area) của một hệ thống là tổng hợp tất cả các điểm truy cập nơi kẻ tấn công có thể nhập dữ liệu vào, trích dữ liệu ra, hoặc kích hoạt việc thực thi một dịch vụ. Sau đây là vài ví dụ:

Code complexity leads to attack vectors Code complexity makes the attack surface larger, with more opportunities for unanticipated side effects. Think of complex code as making the surface area more porous and open to infection. Once again, simple, smaller code is better. Less code means fewer bugs, fewer opportunities for a crippling security hole. Simpler, tighter, less complex code is easier to reason about, easier to spot potential weaknesses.

Độ phức tạp của mã dẫn đến các vector tấn công Độ phức tạp của mã làm cho bề mặt tấn công lớn hơn, với nhiều cơ hội hơn cho các tác dụng phụ ngoài dự tính. Hãy nghĩ về mã phức tạp như thứ làm cho bề mặt thêm xốp rỗng và dễ bị nhiễm bệnh. Một lần nữa, mã đơn giản, nhỏ gọn hơn vẫn tốt hơn. Ít mã hơn nghĩa là ít lỗi hơn, ít cơ hội xuất hiện một lỗ hổng bảo mật chí mạng hơn. Mã đơn giản, gọn chặt, ít phức tạp thì dễ suy luận hơn, dễ phát hiện những điểm yếu tiềm tàng hơn.

Input data is an **attack vector** Never trust data from an external entity, always **sanitize** it before passing it on to a database, view rendering, or other processing. [63] Some languages can help with this. In Ruby, for example, variables holding external input are **tainted**, which limits what

operations can be performed on them. For example, this code apparently uses the `wc` utility to report on the number of characters in a file whose name is supplied at runtime:

Dữ liệu đầu vào là một vector tấn công. Đừng bao giờ tin dữ liệu đến từ một thực thể bên ngoài, luôn làm sạch nó trước khi chuyển tiếp vào cơ sở dữ liệu, kết xuất giao diện, hay bất kỳ khâu xử lý nào khác. [63] Một số ngôn ngữ có thể giúp bạn chuyển này. Chẳng hạn trong Ruby, các biến chứa dữ liệu đầu vào từ bên ngoài bị đánh dấu nhiễm bẩn (tainted), điều này giới hạn những thao tác có thể thực hiện trên chúng. Ví dụ, đoạn mã sau đây có vẻ như dùng tiện ích `wc` để báo cáo số ký tự trong một tập tin mà tên của nó được cung cấp lúc chạy:

safety/taint.rb

safety/taint.rb

```
puts "Enter a file name to count:" name = gets system( "wc -c #{ name }" )
```

```
puts "Enter a file name to count:" name = gets system( "wc -c #{ name }" )
```

A nefarious user could do damage like this:

Một người dùng có ý đồ xấu có thể gây thiệt hại như thế này:

```
Enter a file name to count: test.dat; rm -rf /
```

```
Enter a file name to count: test.dat; rm -rf /
```

However, setting the `SAFE` level to 1 will taint external data, which means it can't be used in dangerous contexts:

Tuy nhiên, đặt mức `SAFE` bằng 1 sẽ đánh dấu nhiễm bẩn cho dữ liệu bên ngoài, nghĩa là nó không thể dùng trong các bối cảnh nguy hiểm:

safety/taint.rb

safety/taint.rb

```
» $SAFE = 1
```

```
» $SAFE = 1
```

```
puts "Enter a file name to count:" name = gets system( "wc -c #{ name }" )
```

```
puts "Enter a file name to count:" name = gets system( "wc -c #{ name }" )
```

```
~~~ session $ ruby taint.rb Enter a file name to count: test.dat; rm -rf /
```

```
~~~ session $ ruby taint.rb Enter a file name to count: test.dat; rm -rf /
```

```
code/safety/taint.rb:5:in system': Insecure operation - system (SecurityError) from code/safety/taint.rb:5:in main' ~~~
```

```
code/safety/taint.rb:5:in system': Insecure operation - system (SecurityError) from code/safety/taint.rb:5:in main' ~~~
```

Unauthenticated services are an attack vector. By their very nature, any user anywhere in the world can call unauthenticated services, so barring any other handling or limiting you've immediately created an opportunity for a **denial-of-service** attack at the very least. Quite a few of highly public data breaches recently were caused by developers accidentally putting data in unauthenticated, publicly readable data stores in the cloud.

Dịch vụ không xác thực là một vector tấn công Về bản chất, bất kỳ người dùng nào ở bất kỳ đâu trên thế giới đều có thể gọi các dịch vụ không xác thực, nên nếu không có cách xử lý hay giới hạn nào khác, bạn đã ngay lập tức tạo ra cơ hội cho một cuộc tấn công từ chối dịch vụ — ít nhất là vậy. Khá nhiều vụ rò rỉ dữ liệu đình đám gần đây là do các lập trình viên vô tình đặt dữ liệu vào những kho lưu trữ không xác thực, có thể đọc công khai trên đám mây.

Authenticated services are an attack vector Keep the number of authorized users at an absolute minimum. Cull unused, old, or outdated users and services. Many net-enabled devices have been found to contain simple default passwords or unused, unprotected administrative accounts. If an account with deployment

Dịch vụ có xác thực là một vector tấn công Hãy giữ số lượng người dùng được cấp quyền ở mức tối thiểu tuyệt đối. Loại bỏ những người dùng và dịch vụ không còn dùng, cũ kỹ, hay lỗi thời. Nhiều thiết bị có kết nối mạng đã bị phát hiện chứa mật khẩu mặc định đơn giản hoặc các tài khoản quản trị không dùng đến, không được bảo vệ. Nếu một tài khoản có quyền triển khai

credentials is compromised, your entire product is compromised.

bị xâm phạm, thì toàn bộ sản phẩm của bạn cũng bị xâm phạm.

Output data is an attack vector There's a (possibly apocryphal) story about a system that dutifully reported the error message Password is used by another user . Don't give away information. Make sure that the data you report is appropriate for the authorization of that user. Truncate or obfuscate potentially risky information such as Social Security or other government ID numbers.

Dữ liệu đầu ra là một vector tấn công Có một câu chuyện (có thể chỉ là truyền miệng) về một hệ thống đã rất mực tận tụy báo lại thông điệp lỗi Password is used by another user . Đừng cho không thông tin. Hãy chắc chắn rằng dữ liệu bạn báo ra là phù hợp với mức phân quyền của người dùng đó. Hãy cắt bớt hoặc che mờ những thông tin tiềm ẩn rủi ro như số An sinh Xã hội hay các số định danh do chính phủ cấp.

Debugging info is an attack vector There's nothing as heartwarming as seeing a full stack trace with data on your local ATM machine, an airport kiosk, or crashing web page. Information designed to make debugging easier can make breaking in easier as well. Make sure any "test window" (discussed here) and runtime **exception** reporting is protected from spying eyes. [64]

Thông tin gỡ lỗi là một vector tấn công Chẳng có gì ấm lòng bằng việc nhìn thấy một stack trace đầy đủ kèm dữ liệu trên máy ATM ở khu bạn ở, trên một ki-ốt sân bay, hay trên một trang web vừa đổ vỡ. Thông tin được thiết kế để việc gỡ lỗi dễ hơn cũng có thể khiến việc đột nhập dễ hơn. Hãy đảm bảo mọi "cửa sổ kiểm thử" (bàn ở đây) và mọi báo cáo ngoại lệ lúc chạy đều được che chắn khỏi những con mắt dòm ngó. [64]

Tip 72: Keep It Simple and Minimize Attack Surfaces

Tip 72: Giữ Đơn Giản và Tối thiểu hóa Bề mặt Tấn công

Principle of Least Privilege Another key principle is to use the least amount of privilege for the shortest time you can get away with. In other words, don't automatically grab the highest permission level, such as root or Administrator. If that high level is needed, take it, do the minimum amount of work, and relinquish your permission quickly to reduce the risk. This principle dates back to the early 1970s:

Nguyên tắc Đặc quyền Tối thiểu Một nguyên tắc then chốt khác là hãy dùng lượng đặc quyền ít nhất trong khoảng thời gian ngắn nhất mà bạn có thể xoay sở được. Nói cách khác, đừng mặc định chộp lấy mức quyền cao nhất, kiểu như root hay Administrator. Nếu

quả thật cần đến mức cao đó, hãy lấy nó, làm lượng việc tối thiểu, rồi trả lại quyền thật nhanh để giảm rủi ro. Nguyên tắc này có từ đầu thập niên 1970:

Every program and every privileged user of the system should operate using the least amount of privilege necessary to complete the job.— Jerome Saltzer, Communications of the ACM, 1974.

Mỗi chương trình và mỗi người dùng có đặc quyền của hệ thống nên hoạt động bằng lượng đặc quyền ít nhất cần thiết để hoàn thành công việc.— Jerome Saltzer, Communications of the ACM, 1974.

Take the login program on Unix-derived systems. It initially executes with root privileges. As soon as it finishes authenticating the correct user, though, it drops the high level privilege to that of the user.

Hãy lấy chương trình login trên các hệ thống họ Unix làm ví dụ. Ban đầu nó chạy với đặc quyền root. Nhưng ngay khi xác thực xong đúng người dùng, nó lập tức hạ mức đặc quyền cao đó xuống mức của chính người dùng ấy.

This doesn't just apply to operating system privilege levels. Does your application implement different levels of access? Is it a blunt tool, such as “administrator” vs. “user?” If so, consider something more finely grained, where your sensitive resources are partitioned into different categories, and individual users have permissions for only certain of those categories.

Điều này không chỉ áp dụng cho các mức đặc quyền của hệ điều hành. Ứng dụng của bạn có triển khai các mức truy cập khác nhau không? Nó có phải là một công cụ thô thiển kiểu “quản trị viên” đối lập với “người dùng” không? Nếu vậy, hãy cân nhắc thứ gì đó chi tiết hơn, nơi các tài nguyên nhạy cảm được phân thành các nhóm khác nhau, và từng người dùng chỉ có quyền với một số nhóm nhất định.

This technique follows the same sort of idea as minimizing surface area—reducing the scope of attack vectors, both by time and by privilege level. In this case, less is indeed more.

Kỹ thuật này theo cùng một ý tưởng với việc tối thiểu hóa bề mặt — thu hẹp phạm vi của các vector tấn công, cả về thời gian lẫn về mức đặc quyền. Trong trường hợp này, ít quả thật là nhiều.

Secure Defaults The default settings on your app, or for your users on your site, should be the most secure values. These might not be the most user-friendly or convenient values, but it's better to let each individual decide for themselves the trade-offs between security and convenience.

Mặc định An toàn Các thiết lập mặc định trên ứng dụng của bạn, hoặc cho người dùng trên trang của bạn, nên là những giá trị an toàn nhất. Chúng có thể không phải là những giá trị thân thiện hay tiện lợi nhất với người dùng, nhưng tốt hơn là để mỗi cá nhân tự quyết định cho mình sự đánh đổi giữa an toàn và tiện lợi.

For example, the default for password entry might be to hide the password as entered, replacing each character with an asterisk. If you're entering a password in a crowded public place, or projected before a large audience, that's a sensible default. But some users might want to see the password spelled

Ví dụ, mặc định cho việc nhập mật khẩu có thể là che mật khẩu khi gõ vào, thay mỗi ký tự bằng một dấu hoa thị. Nếu bạn đang nhập mật khẩu ở một nơi công cộng đông người, hoặc đang chiếu lên trước một đám đông, thì đó là một mặc định hợp lý. Nhưng một số người dùng có thể muốn thấy mật khẩu hiện rõ ra,

out, perhaps for accessibility. If there's little risk someone is looking over their shoulder, that's a reasonable choice for them.

có lẽ vì lý do trợ năng. Nếu hầu như không có rủi ro ai đó đang nhìn qua vai họ, thì đó là một lựa chọn hợp lý đối với họ.

Encrypt Sensitive Data Don't leave personally identifiable information, financial data, passwords, or other credentials in **plain text**, whether in a database or some other external file. If the data gets exposed, encryption offers an additional level of safety.

Mã hóa Dữ liệu Nhạy cảm Đừng để thông tin định danh cá nhân, dữ liệu tài chính, mật khẩu, hay các thông tin xác thực khác ở dạng văn bản thuần (plain text), dù là trong cơ sở dữ liệu hay một tập tin bên ngoài nào đó. Nếu dữ liệu bị lộ, mã hóa mang lại thêm một lớp an toàn.

In **Topic 19, Version Control** we strongly recommend putting everything needed for the project under version control. Well, almost everything. Here's one major exception to that rule:

Trong Topic 19, Quản lý Phiên bản, chúng tôi mạnh mẽ khuyến nghị đưa mọi thứ cần thiết cho dự án vào quản lý phiên bản (version control). À, gần như mọi thứ. Đây là một ngoại lệ lớn cho quy tắc đó:

Don't check in secrets, API keys, SSH keys, encryption passwords or other credentials alongside your source code in version control.

Đừng đưa các bí mật (secret), khóa API, khóa SSH, mật khẩu mã hóa hay các thông tin xác thực khác vào quản lý phiên bản cùng với mã nguồn của bạn.

Keys and secrets need to be managed separately, generally via config files or environment variables as part of build and deployment.

Các khóa và bí mật cần được quản lý riêng, thường là qua tập tin cấu hình hoặc biến môi trường như một phần của quá trình build và triển khai.

Password Antipatterns

Phản mẫu Mật khẩu

One of the fundamental problems with security is that oftentimes good security runs counter to common sense or common practice. For example, you might think that strict password **requirements** would increase security for your application or site. You'd be wrong.

Một trong những vấn đề cốt lõi của bảo mật là rất thường xuyên, bảo mật tốt lại đi ngược với lẽ thường hay thực hành phổ biến. Ví dụ, bạn có thể nghĩ rằng các yêu cầu mật khẩu nghiêm ngặt sẽ tăng cường bảo mật cho ứng dụng hay trang của bạn. Bạn sẽ lầm.

Strict password policies will actually lower your security. Here's a short list of very bad [65] ideas, along with some recommendations from the NIST:

Các chính sách mật khẩu nghiêm ngặt thực chất sẽ hạ thấp bảo mật của bạn. Sau đây là một danh sách ngăn những ý tưởng cực tệ [65], kèm theo vài khuyến nghị từ NIST:

Do not restrict password length to less than 64 characters. NIST recommends 256 as a good maximum length. Do not truncate the user's chosen password.

Đừng giới hạn độ dài mật khẩu dưới 64 ký tự. NIST khuyến nghị 256 là một độ dài tối đa tốt. Đừng cắt bớt mật khẩu mà người dùng đã chọn.

Do not restrict special characters such as ;&%\$# or / . See the note about Bobby Tables earlier in this section. If special characters in your password will compromise your system, you have bigger problems. The NIST says to accept all printing ASCII characters, space, and Unicode.

Đừng cấm các ký tự đặc biệt như ;&%\$# hay / . Xem lại ghi chú về Bobby Tables ở phần trước trong mục này. Nếu ký tự đặc biệt trong mật khẩu làm hệ thống của bạn sụp đổ, thì bạn đang có vấn đề lớn hơn nhiều. NIST khuyên nên chấp nhận toàn bộ ký tự ASCII in được, dấu cách, và Unicode.

Do not provide password hints to unauthenticated users, or prompt for specific types of information (e.g., “what was the name of your first pet?”). Do not disable the paste function in the browser. Crippling the functionality of the browser and password managers does not make your system more secure, in fact it drives users to create simpler, shorter passwords that are much easier to compromise. Both the NIST in the US and the National Cyber Security Centre in the UK specifically require verifiers to allow paste functionality for this reason.

Đừng cung cấp gợi ý mật khẩu cho người dùng chưa xác thực, cũng đừng hỏi những loại thông tin cụ thể (ví dụ “tên con thú cưng đầu tiên của bạn là gì?”). Đừng vô hiệu hóa chức năng dán (paste) trong trình duyệt. Làm tê liệt chức năng của trình duyệt và các trình quản lý mật khẩu không làm hệ thống của bạn an toàn hơn, thực tế nó đẩy người dùng đến chỗ tạo ra những mật khẩu đơn giản hơn, ngắn hơn, dễ bị xâm phạm hơn nhiều. Cả NIST ở Mỹ lẫn Trung tâm An ninh Mạng Quốc gia ở Anh đều cụ thể yêu cầu các bộ xác thực phải cho phép chức năng dán vì lý do này.

Do not impose other composition rules. For example, do not mandate any particular mix of upper and lower case, numerics, or special characters, or prohibit repeating characters, and so on.

Đừng áp đặt các quy tắc cấu thành khác. Ví dụ, đừng bắt buộc một sự pha trộn cụ thể nào đó giữa chữ hoa và chữ thường, chữ số, hay ký tự đặc biệt, cũng đừng cấm ký tự lặp lại, v.v.

Do not arbitrarily require users to change their passwords after some length of time. Only do this for a valid reason (e.g., if there has been a breach).

Đừng tùy tiện bắt người dùng đổi mật khẩu sau một khoảng thời gian nào đó. Chỉ làm vậy khi có lý do chính đáng (ví dụ, nếu đã có một vụ rò rỉ).

You want to encourage long, random passwords with a high degree of **entropy**. Putting artificial constraints limits entropy and encourages bad password habits, leaving your user’s accounts vulnerable to takeover.

Bạn muốn khuyến khích những mật khẩu dài, ngẫu nhiên với độ ngẫu nhiên (entropy) cao. Đặt ra những ràng buộc nhân tạo sẽ giới hạn entropy và khuyến khích những thói quen mật khẩu xấu, khiến tài khoản người dùng của bạn dễ bị chiếm đoạt.

Maintain Security Updates Updating computer systems can be a huge pain. You need that security patch, but as a side effect it breaks some portion of your application. You could decide to wait, and defer the update until later. That’s a terrible idea, because now your system is vulnerable to a known exploit.

Duy trì Cập nhật Bảo mật Cập nhật hệ thống máy tính có thể là một nỗi đau lớn. Bạn cần bản vá bảo mật đó, nhưng tác dụng phụ của nó lại làm hỏng một phần ứng dụng của bạn. Bạn có thể quyết định chờ đợi, hoãn việc cập nhật lại sau. Đó là một ý tưởng tồi tệ, vì giờ hệ thống của bạn dễ tổn thương trước một lỗ hổng đã được biết đến.

Tip 73: Apply Security Patches Quickly

Tip 73: Áp dụng Bản vá Bảo mật Nhanh chóng

This **tip** affects every net-connected device, including phones, cars, appliances, personal laptops, developer machines, build machines, production servers, and cloud images. Everything. And if you

think that this doesn't really matter, just remember that the largest data breaches in history (so far) were caused by systems that were behind on their updates.

Mẹo này ảnh hưởng đến mọi thiết bị có kết nối mạng, bao gồm điện thoại, ô tô, thiết bị gia dụng, laptop cá nhân, máy của lập trình viên, máy build, máy chủ vận hành (production server), và các image trên đám mây. Tất cả mọi thứ. Và nếu bạn nghĩ chuyện này chẳng quan trọng lắm, hãy nhớ rằng những vụ rò rỉ dữ liệu lớn nhất trong lịch sử (cho đến nay) đều do các hệ thống chậm trễ trong việc cập nhật gây ra.

Don't let it happen to you.

Đừng để chuyện đó xảy đến với bạn.

Common Sense Vs. Crypto — Lễ Thường Đối Đầu Mật Mã Học

It's important to keep in mind that common sense may fail you when it comes to matters of **cryptography**. The first and most important rule when it comes to **crypto** is never do it yourself. [66] Even for something as simple as passwords, common practices are wrongheaded (see the sidebar Password Antipatterns). Once you get into the world of crypto, even the tiniest, most insignificant-looking error can compromise everything: your clever new, home-made encryption algorithm can probably be broken by an expert in minutes. You don't want to do encryption yourself.

Điều quan trọng cần ghi nhớ là lễ thường có thể phản bội bạn khi đụng tới chuyện mật mã học (cryptography). Quy tắc đầu tiên và quan trọng nhất khi nói tới mật mã là đừng bao giờ tự làm. [66] Ngay cả với thứ đơn giản như mật khẩu, các thực hành phổ biến cũng đã sai lầm (xem khung phụ Phản mẫu Mật khẩu). Một khi đã bước vào thế giới mật mã, ngay cả lỗi nhỏ nhất, trông tầm thường nhất cũng có thể làm sụp đổ tất cả: thuật toán mã hóa mới mẻ, tự chế thông minh của bạn có lẽ sẽ bị một chuyên gia phá vỡ trong vài phút. Bạn không muốn tự làm mã hóa đâu.

As we've said elsewhere, rely only on reliable things: wellvetted, thoroughly examined, well-maintained, frequently updated, preferably open source libraries and frameworks.

Như chúng tôi đã nói ở chỗ khác, hãy chỉ dựa vào những thứ đáng tin cậy: các thư viện và framework đã được kiểm chứng kỹ, soi xét thấu đáo, bảo trì tốt, cập nhật thường xuyên, và tốt nhất là mã nguồn mở.

Beyond simple encryption tasks, take a hard look at other security-related features of your site or application. Take **authentication**, for instance.

Vượt ra ngoài các tác vụ mã hóa đơn giản, hãy nhìn thật kỹ vào các tính năng liên quan tới bảo mật khác của trang hay ứng dụng của bạn. Lấy việc xác thực (authentication) làm ví dụ.

In order to implement your own login with password or biometric authentication, you need to understand how hashes and salts work, how crackers use things like Rainbow tables, why you shouldn't use MD5 or SHA1, and a host of other concerns. And even if you get all that right, at the end of the day you're still responsible for holding onto the data and keeping it secure, subject to whatever new legislation and legal obligations come up.

Để tự triển khai phần đăng nhập bằng mật khẩu hoặc xác thực sinh trắc học, bạn cần hiểu cách các hàm băm (hash) và muối (salt) hoạt động, cách những kẻ bẻ khóa dùng những thứ như bảng Rainbow, vì sao bạn không nên dùng MD5 hay SHA1, cùng một loạt mối lo khác. Và ngay cả khi bạn làm đúng hết tất cả những thứ đó, rốt cuộc bạn vẫn phải chịu

trách nhiệm nắm giữ dữ liệu và giữ nó an toàn, chịu sự chi phối của bất kỳ luật lệ và nghĩa vụ pháp lý mới nào nảy sinh.

Or, you could take the Pragmatic approach and let someone else worry about it and use a third-party authentication provider. This may be an off-the-shelf service you run in-house, or it could be a third party in the cloud. Authentication services are

Hoặc, bạn có thể chọn cách tiếp cận Thực dụng và để người khác lo chuyện đó, dùng một nhà cung cấp xác thực bên thứ ba. Đây có thể là một dịch vụ đóng gói sẵn mà bạn tự vận hành nội bộ, hoặc có thể là một bên thứ ba trên đám mây. Các dịch vụ xác thực

often available from email, phone, or social media providers, which may or may not be appropriate for your application. In any case, these folks spend all their days keeping their systems secure, and they're better at it than you are.

thường có sẵn từ các nhà cung cấp email, điện thoại, hoặc mạng xã hội, dù có thể phù hợp hay không với ứng dụng của bạn. Trong mọi trường hợp, những người này dành cả ngày để giữ cho hệ thống của họ an toàn, và họ giỏi việc đó hơn bạn.

Stay safe out there.

Hãy an toàn ngoài kia.

Related Sections Include — Các mục liên quan

Topic 23, **Design by Contract**

Topic 23, Thiết kế theo Hợp đồng

Topic 24, Dead Programs Tell No Lies

Topic 24, Chương trình Đã chết Không Nói Dối

Topic 25, **Assertive Programming**

Topic 25, Lập trình Phòng vệ bằng Assertion

Topic 38, **Programming by Coincidence**

Topic 38, Lập trình theo May rủi

Topic 45, The Requirements Pit

Topic 45, Cái Hố Yêu cầu

The beginning of wisdom is to call things by their proper name. Confucius

Khởi đầu của trí tuệ là gọi sự vật bằng đúng tên của nó. Khổng Tử

Topic 44. Naming Things — Topic 44. Đặt Tên Cho Mọi Thứ

What's in a name? When we're programming, the answer is "everything!"

Cái tên thì có gì quan trọng? Khi lập trình, câu trả lời là "tất cả!"

We create names for applications, subsystems, modules, functions, variables—we're constantly creating new things and bestowing names on them. And those names are very, very important, because they reveal a lot about your intent and belief.

Chúng ta tạo ra tên cho ứng dụng, hệ thống con, mô-đun, hàm, biến — chúng ta không ngừng tạo ra những thứ mới và ban cho chúng cái tên. Và những cái tên ấy rất, rất quan trọng, bởi chúng tiết lộ rất nhiều về ý định và niềm tin của bạn.

We believe that things should be named according to the role they play in your code. This means that, whenever you create something, you need to pause and think “what is my motivation to create this?”

Chúng tôi tin rằng mọi thứ nên được đặt tên theo đúng vai trò mà nó đảm nhận trong mã của bạn. Điều này có nghĩa là, mỗi khi tạo ra một thứ gì đó, bạn cần dừng lại và tự hỏi “động cơ nào khiến mình tạo ra cái này?”

This is a powerful question, because it takes you out of the immediate problem-solving mindset and makes you look at the bigger picture. When you consider the role of a variable or function, you’re thinking about what is special about it, about what it can do, and what it interacts with. Often, we find ourselves realizing that what we were about to do made no sense, all because we couldn’t come up with an appropriate name.

Đây là một câu hỏi đầy sức mạnh, bởi nó kéo bạn ra khỏi tâm thế giải quyết vấn đề trước mắt và buộc bạn nhìn vào bức tranh toàn cảnh hơn. Khi cân nhắc vai trò của một biến hay một hàm, bạn đang nghĩ về điều gì khiến nó đặc biệt, nó làm được gì, và nó tương tác với những gì. Rất nhiều lần, chúng tôi chợt nhận ra rằng cái mình sắp làm chẳng có nghĩa lý gì cả, tất cả chỉ vì không nghĩ ra nổi một cái tên cho ra hồn.

There’s some science behind the idea that names are deeply

Có cả cơ sở khoa học cho ý tưởng rằng những cái tên mang ý nghĩa

meaningful. It turns out that the brain can read and understand words really fast: faster than many other activities. This means that words have a certain priority when we try to make sense of something. This can be demonstrated using the Stroop effect. [67]

thực sự có ý nghĩa. Hóa ra bộ não có thể đọc và hiểu từ ngữ cực nhanh: nhanh hơn nhiều hoạt động khác. Điều này có nghĩa là từ ngữ được ưu tiên nhất định khi ta cố hiểu một điều gì đó. Có thể minh họa chuyện này bằng hiệu ứng Stroop. [67]

Look at the following panel. It has a list of color names or shades, and each is shown in a color or shade. But the names and colors don’t necessarily match. Here’s part one of the challenge—say aloud the name of each color as written: [68]

Hãy nhìn bảng dưới đây. Nó gồm một danh sách các tên màu hoặc sắc độ, và mỗi cái được hiển thị bằng một màu hoặc một sắc độ. Nhưng tên gọi và màu sắc không nhất thiết khớp nhau. Đây là phần một của thử thách — hãy đọc to tên của từng màu đúng như chữ viết: [68]

images/stroop_color.png

images/stroop_color.png

Now repeat this, but instead say aloud the color used to draw the word. Harder, eh? It’s easy to be fluent when reading, but way harder when trying to recognize colors.

Bây giờ làm lại, nhưng lần này hãy đọc to cái màu được dùng để vẽ chữ. Khó hơn nhỉ? Đọc chữ thì trôi chảy dễ dàng, nhưng nhận diện màu thì khó hơn nhiều.

Your brain treats written words as something to be respected. We need to make sure the names we use live up to this.

Bộ não của bạn coi chữ viết là thứ đáng được tôn trọng. Chúng ta cần đảm bảo những cái tên mình dùng xứng đáng với sự tôn trọng ấy.

Let's look at a couple of examples:

Hãy xem qua vài ví dụ:

We're authenticating people who access our site that sells jewelry made from old graphics cards:

Chúng ta đang xác thực những người truy cập trang web bán đồ trang sức làm từ card đồ họa cũ:

```
let user = authenticate(credentials)
```

```
let user = authenticate(credentials)
```

The variable is user because it's always user . But why? It means

Biến này tên là user bởi vì lúc nào nó chẳng là user . Nhưng tại sao? Nó chẳng có nghĩa

nothing. How about **customer** , or buyer ? That way we get constant reminders as we code of what this person is trying to do, and what that means to us.

lý gì cả. Hay là customer , hoặc buyer thì sao? Như thế, trong lúc lập trình ta sẽ liên tục được nhắc nhở rằng con người này đang cố làm gì, và điều đó có ý nghĩa gì với ta.

We have an instance method that discounts an order:

Chúng ta có một phương thức (instance method) thực hiện chiết khấu cho một đơn hàng:

```
public void deductPercent( double amount) // ...
```

```
public void deductPercent( double amount) // ...
```

Two things here. First, deductPercent is what it does and not why it does it . Then the name of the parameter amount is at best misleading: is it an absolute amount, a percentage? Perhaps this would be better:

Có hai vấn đề ở đây. Thứ nhất, deductPercent nói lên nó làm gì chứ không nói vì sao nó làm thế . Thứ hai, tên tham số amount thì nhẹ nhất cũng là gây hiểu lầm: đó là một con số tuyệt đối, hay một tỷ lệ phần trăm? Có lẽ viết thế này sẽ tốt hơn:

```
public void applyDiscount(Percentage discount) // ...
```

```
public void applyDiscount(Percentage discount) // ...
```

The method name now makes its intent clear. We've also changed the parameter from a double to a Percentage , a type we've defined. We don't know about you, but when dealing with percentages we never know if the value is supposed to be between 0 and 100 or 0.0 and 1.0. Using a type documents what the function expects.

Giờ thì tên phương thức đã nói rõ ý định của nó. Ta cũng đổi tham số từ double thành Percentage , một kiểu do ta tự định nghĩa. Chúng tôi không biết bạn thế nào, chứ khi làm việc với phần trăm thì chúng tôi chẳng bao giờ đoán nổi giá trị đó đáng lẽ nằm giữa 0 và 100 hay giữa 0.0 và 1.0. Dùng một kiểu dữ liệu chính là tài liệu hóa những gì hàm mong đợi.

We have a module that does interesting things with Fibonacci numbers. One of those things is to calculate the number in the sequence. Stop and think what you'd call this function. Most people we ask would call it fib . Seems reasonable, but remember it will normally be called in the **context** of its module, so the call would be Fib.fib(n) . How about calling it of or nth instead:

Ta có một mô-đun làm những chuyện thú vị với các số Fibonacci. Một trong những chuyện đó là tính số thứ n trong dãy. Hãy dừng lại và nghĩ xem bạn sẽ gọi hàm này là gì. Phần lớn những người chúng tôi hỏi đều đặt tên nó là `fib`. Nghe cũng hợp lý, nhưng nhớ rằng nó thường được gọi trong ngữ cảnh của mô-đun chứa nó, nên lời gọi sẽ là `Fib.fib(n)`. Hay là đặt tên nó là `of` hoặc `nth` thì sao:

```
Fib.of(0) # => 0 Fib.nth(20) # => 4181
```

```
Fib.of(0) # => 0 Fib.nth(20) # => 4181
```

When **naming things**, you're constantly looking for ways of clarifying what you mean, and that act of clarification will lead you to a better understanding of your code as you write it.

Khi đặt tên cho mọi thứ, bạn liên tục tìm cách làm rõ ý mình muốn nói, và chính hành động làm rõ ấy sẽ dẫn bạn tới một hiểu biết sâu sắc hơn về mã của mình ngay khi đang viết.

However, not all names have to be candidates for a literary prize.

Tuy nhiên, không phải cái tên nào cũng cần phải là ứng viên cho một giải thưởng văn học.

The **Exception** That Proves the Rule

Ngoại Lệ Chứng Minh Quy Tắc

While we strive for clarity in code, branding is a different matter entirely.

Trong khi ta nỗ lực để mã được rõ ràng, thì việc đặt thương hiệu (branding) lại là chuyện hoàn toàn khác.

There's a well-established tradition that projects and project teams should have obscure, "clever" names. Names of Pokémon, Marvel superheroes, cute mammals, Lord of the Rings characters, you name it.

Có một truyền thống đã ăn sâu rằng các dự án và các đội dự án nên có những cái tên tối nghĩa, "nghe cho ngẫu nhiên". Tên các Pokémon, các siêu anh hùng Marvel, các loài thú dễ thương, các nhân vật trong Chúa Tể Những Chiếc Nhẫn (Lord of the Rings), gì cũng được.

Literally.

Đúng theo nghĩa đen.

Honor The Culture — Tôn Trọng Văn Hóa

Most introductory computer texts will admonish you never to use single letter variables such as `i`, `j`, or `k`. [69]

Phần lớn giáo trình nhập môn máy tính sẽ răn bạn rằng đừng bao giờ dùng biến một chữ cái như `i`, `j`, hay `k`. [69]

We think they're wrong. Sort of.

Chúng tôi cho rằng họ sai. Đại loại thế.

In fact, it depends on the culture of that particular programming language or environment. In the C programming language, `i`, `j`, and `k` are traditionally used as loop increment variables, `s` is used for a character string, and so on. If you program in that environment, that's what you are used to seeing and it would be jarring (and hence wrong) to violate that norm. On the other hand, using that convention in a different environment where it's not expected is just as wrong. You'd never do something heinous like this Clojure example which assigns a string to variable `i`:

Thật ra, chuyện đó phụ thuộc vào văn hóa của từng ngôn ngữ hay môi trường lập trình cụ thể. Trong ngôn ngữ C, theo truyền thống i, j, và k được dùng làm biến đếm vòng lặp, s được dùng cho chuỗi ký tự, và cứ thế. Nếu bạn lập trình trong môi trường đó, đấy là những gì bạn quen mắt, và vi phạm chuẩn mực ấy sẽ gây chối mắt (và do đó là sai). Ngược lại, dùng quy ước ấy trong một môi trường khác, nơi người ta không trông đợi nó, thì cũng sai y như vậy. Bạn sẽ chẳng bao giờ làm chuyện ghê rợn như ví dụ Clojure dưới đây, gán một chuỗi cho biến i :

```
(let [i "Hello World"] (println i))
```

```
(let [i "Hello World"] (println i))
```

Some language communities prefer camelCase, with embedded capital letters, while others prefer snake_case with embedded underscores to separate words. The languages themselves will of course accept either, but that doesn't make it right. Honor the local culture.

Một số cộng đồng ngôn ngữ chuộng camelCase, với các chữ hoa nằm chen bên trong, trong khi số khác lại chuộng snake_case với dấu gạch dưới ngăn cách các từ. Bản thân các ngôn ngữ dĩ nhiên chấp nhận cả hai, nhưng điều đó không có nghĩa là làm gì cũng đúng. Hãy tôn trọng văn hóa bản địa.

Some languages allow a subset of Unicode in names. Get a sense of what the community expects before going all cute with names like `æsn` or `εξέρχεται`.

Một số ngôn ngữ cho phép dùng một tập con của Unicode trong tên gọi. Hãy cảm nhận xem cộng đồng trông đợi điều gì trước khi giờ trở đáng yêu với những cái tên kiểu `æsn` hay `εξέρχεται`.

Consistency — Tính Nhất Quán

Emerson is famous for writing “A foolish consistency is the hobgoblin of little minds...,” but Emerson wasn't on a team of programmers.

Emerson nổi tiếng với câu “Sự nhất quán ngu xuẩn là con ngáo ộp của những đầu óc nhỏ bé...”, nhưng Emerson đâu có ở trong một đội lập trình viên.

Every project has its own vocabulary: jargon words that have a special meaning to the team. “Order” means one thing to a team creating an online store, and something very different to a team whose app charts the lineage of religious groups. It's important that everyone on the team knows what these words mean, and that they use them consistently.

Mỗi dự án đều có vốn từ vựng riêng: những từ lóng (jargon) mang ý nghĩa đặc biệt với cả đội. “Order” (đơn hàng) nghĩa một đằng với đội đang dựng một cửa hàng trực tuyến, và nghĩa rất khác với một đội mà ứng dụng của họ vẽ phả hệ của các nhóm tôn giáo. Điều quan trọng là mọi người trong đội đều biết những từ này nghĩa là gì, và dùng chúng một cách nhất quán.

One way is to encourage a lot of communication. If everyone pair programs, and pairs switch frequently, then jargon will spread osmotically.

Một cách là khuyến khích trao đổi thật nhiều. Nếu mọi người đều lập trình theo cặp (pair programming), và các cặp thường xuyên đổi chỗ cho nhau, thì từ lóng sẽ lan ra theo kiểu thẩm thấu.

Another way is to have a project glossary, listing the terms that have special meaning to the team. This is an informal document, possibly maintained on a wiki, possibly just index cards on a wall

somewhere.

Một cách khác là có một bảng thuật ngữ (glossary) cho dự án, liệt kê những thuật ngữ mang ý nghĩa đặc biệt với cả đội. Đây là một tài liệu không chính thức, có thể duy trì trên một wiki, mà cũng có thể chỉ là những tấm thẻ ghi chú dán đâu đó trên tường.

After a while, the project jargon will take on a life of its own. As everyone gets comfortable with the vocabulary, you'll be able to use the jargon as a shorthand, expressing a lot of meaning accurately and concisely. (This is exactly what a pattern language is.)

Sau một thời gian, từ lóng của dự án sẽ tự sống cuộc đời riêng của nó. Khi mọi người đã thấy thoải mái với vốn từ vựng đó, bạn sẽ có thể dùng nó như một dạng viết tắt, diễn đạt thật nhiều ý nghĩa một cách chính xác và súc tích. (Đây chính xác là một ngôn ngữ mẫu — pattern language.)

Renaming Is Even Harder — Đổi Tên Còn Khó Hơn

No matter how much effort you put in up front, things change. Code is refactored, usage shifts, meaning becomes subtly altered. If you aren't vigilant about updating names as you go, you can quickly descend into a nightmare much worse than meaningless names: misleading names. Have you ever had someone explain inconsistencies in code such as, "The routine called `getData` really writes data to an archive file"?

Dù bạn có bỏ ra bao nhiêu công sức ngay từ đầu đi nữa, mọi thứ vẫn thay đổi. Mã được tái cấu trúc, cách dùng dịch chuyển, ý nghĩa bị biến đổi một cách tinh vi. Nếu bạn không cảnh giác cập nhật tên gọi trong lúc đi tới, bạn sẽ nhanh chóng rơi vào một cơn ác mộng tệ hơn nhiều so với những cái tên vô nghĩa: những cái tên gây hiểu lầm. Đã bao giờ bạn nghe ai đó giải thích một chỗ bất nhất trong mã kiểu như "cái routine tên là `getData` thật ra ghi dữ liệu vào một tập tin lưu trữ" chưa?

As we discuss in **Topic 3, Software Entropy**, when you spot a problem, fix it—right here and now. When you see a name that no longer expresses the intent, or is misleading or confusing, fix it. You've got full regression tests, so you'll spot any instances you may have missed.

Như đã bàn ở Topic 3, Sự Rối Loạn Phần Mềm (Software Entropy), khi bạn phát hiện một vấn đề, hãy sửa nó — ngay tại đây và ngay bây giờ. Khi bạn thấy một cái tên không còn thể hiện đúng ý định, hoặc gây hiểu lầm hay rối rắm, hãy sửa nó. Bạn đã có bộ kiểm thử hồi quy đầy đủ, nên sẽ phát hiện được mọi chỗ có thể bị bỏ sót.

Tip 74: Name Well; Rename When Needed

Tip 74: Đặt Tên Cho Tốt; Đổi Tên Khi Cần

If for some reason you can't change the now-wrong name, then you've got a bigger problem: an **ETC** violation (see Topic 8, The Essence of Good Design). Fix that first, then change the offending name. Make renaming easy, and do it often.

Nếu vì lý do nào đó bạn không thể đổi cái tên giờ-đã-sai, thì bạn đang gặp một vấn đề lớn hơn: một vi phạm ETC (xem Topic 8, Tinh Túy Của Thiết Kế Tốt). Hãy sửa cái đó trước, rồi mới đổi cái tên đáng ghét kia. Hãy làm cho việc đổi tên trở nên dễ dàng, và làm điều đó thường xuyên.

Otherwise you'll have to explain to the new folks on the team that `getData` really writes data to a file, and you'll have to do it with a straight face.

Nếu không, bạn sẽ phải giải thích với những người mới vào đội rằng `getData` thật ra ghi dữ liệu ra tập tin, và bạn sẽ phải làm điều đó với vẻ mặt tỉnh bơ.

Related Sections Include — Các mục liên quan

Topic 3, Software **Entropy**

Topic 3, Sự Rối Loạn Phần Mềm

Topic 40, **Refactoring**

Topic 40, Tái Cấu Trúc

Topic 45, The **Requirements** Pit

Topic 45, Cái Hố Yêu Cầu

Challenges — Thử thách

When you find a function or method with an overly generic name, try and rename it to express all the things it really does. Now it's an easier target for refactoring.

Khi bạn tìm thấy một hàm hay một phương thức có cái tên quá chung chung, hãy thử đổi tên nó để diễn đạt hết tất cả những gì nó thực sự làm. Giờ thì nó là một mục tiêu dễ nhắm hơn cho việc tái cấu trúc.

In our examples, we suggested using more specific names such as `buyer` instead of the more traditional and generic `user`. What other names do you habitually use that could be better?

Trong các ví dụ của mình, chúng tôi đã gợi ý dùng những cái tên cụ thể hơn như `buyer` thay cho `user` truyền thống và chung chung. Còn những cái tên nào khác mà bạn quen dùng nhưng có thể đặt tốt hơn không?

Are the names in your system congruent with user terms from the domain? If not, why? Does this cause a Stroop-effect style cognitive dissonance for the team?

Các tên trong hệ thống của bạn có ăn khớp với những từ ngữ mà người dùng trong miền (domain) hay dùng không? Nếu không, tại sao? Liệu điều đó có gây ra một dạng nghịch lý nhận thức kiểu hiệu ứng Stroop cho cả đội không?

Are names in your system hard to change? What can you do to fix that particular broken window?

Các tên trong hệ thống của bạn có khó đổi không? Bạn có thể làm gì để sửa chính cái cửa sổ vỡ (broken window) đó?

Footnotes

Chú thích cuối trang

[50] Note from the battle-scarred: UTC is there for a reason. Use it.

[50] Lời nhắn từ kẻ đầy thương tích trận mạc: UTC có lý do tồn tại của nó. Hãy dùng nó.

[51] https://en.wikipedia.org/wiki/Correlation_does_not_imply_causation

[51] https://en.wikipedia.org/wiki/Correlation_does_not_imply_causation

[52] See Topic 50, Coconuts Don't Cut It .

[52] Xem Topic 50, Coconuts Don't Cut It .

[53] You can also go too far here. We once knew a developer who rewrote all source he was given because he had his own naming conventions.

[53] Bạn cũng có thể đi quá xa ở đây. Chúng tôi từng quen một lập trình viên viết lại toàn bộ mã nguồn được giao chỉ vì anh ta có quy ước đặt tên của riêng mình.

[54] https://mediaorigin.pragprog.com/titles/tpp20/code/algorithm_speed/sort/src/main.rs

[54] https://mediaorigin.pragprog.com/titles/tpp20/code/algorithm_speed/sort/src/main.rs

[55] And yes, we did voice our concerns over the title.

[55] Và đúng vậy, chúng tôi đã lên tiếng bày tỏ lo ngại về cái tựa đề.

[56] Originally spotted in UML Distilled: A Brief Guide to the Standard Object Modeling Language [Fow00] .

[56] Ban đầu được phát hiện trong UML Distilled: A Brief Guide to the Standard Object Modeling Language [Fow00] .

[57] This is excellent advice in general (see Topic 27, Don't Outrun Your Headlights).

[57] Đây là lời khuyên tuyệt vời nói chung (xem Topic 27, Don't Outrun Your Headlights).

[58] Some folks argue that **test-first** and test-driven development are two different things, saying that the intents of the two are different. However, historically, test-first (which comes from **eXtreme Programming**) was identical to what people now call TDD.

[58] Một số người lập luận rằng test-first và test-driven development là hai thứ khác nhau, cho rằng ý đồ của hai cái là khác biệt. Tuy nhiên, xét về lịch sử, test-first (vốn ra đời từ eXtreme Programming) đồng nhất với cái mà ngày nay người ta gọi là TDD.

[59] <https://ronjeffries.com/categories/sudoku> . A big “thank you” to Ron for letting us use this story.

[59] <https://ronjeffries.com/categories/sudoku> . Một lời “cảm ơn” lớn gửi tới Ron vì đã cho phép chúng tôi dùng câu chuyện này.

[60] <http://norvig.com/sudoku.html>

[60] <http://norvig.com/sudoku.html>

We've been trying since at least 1986, when Cox and Novobilski coined the term [61] “software IC” in their Objective-C book Object-Oriented Programming Object-Oriented Programming: An Evolutionary Approach [CN91] .

Chúng tôi đã thử kể từ ít nhất năm 1986, khi Cox và Novobilski đặt ra thuật ngữ [61] “software IC” trong cuốn sách Objective-C của họ Object-Oriented Programming Object-Oriented Programming: An Evolutionary Approach [CN91] .

[62] See Topic 20, **Debugging** .

[62] Xem Topic 20, **Gỡ Lỗi** .

[63] Remember our good friend, little Bobby Tables (<https://xkcd.com/327>)? While you're reminiscing have a look at <https://bobby-tables.com> , which lists ways of sanitizing data passed to database queries.

[63] Còn nhớ người bạn tốt của chúng ta, cậu nhóc Bobby Tables (<https://xkcd.com/327>) không? Nhân lúc đang hoài niệm, hãy ghé qua <https://bobby-tables.com> , nơi liệt kê các cách làm sạch dữ liệu được truyền vào các truy vấn cơ sở dữ liệu.

[64] This technique has proven to be successful at the CPU chip level, where well-known exploits target debugging and administrative facilities. Once cracked, the entire machine is left exposed.

[64] Kỹ thuật này đã được chứng minh là thành công ở cấp độ chip CPU, nơi các lỗ hổng nổi tiếng nhắm vào các tiện ích gỡ lỗi và quản trị. Một khi bị bẻ khóa, toàn bộ cỗ máy bị phơi bày.

[65] NIST Special Publication 800-63B: Digital Identity Guidelines: **Authentication** and Lifecycle Management , available free online at <https://doi.org/10.6028/NIST.SP.800-63b>

[65] NIST Special Publication 800-63B: Digital Identity Guidelines: Authentication and Lifecycle Management , có sẵn miễn phí trực tuyến tại <https://doi.org/10.6028/NIST.SP.800-63b>

[66] Unless you have a PhD in **cryptography**, and even then only with major peer review, extensive field trials with a bug bounty, and budget for long-term maintenance.

[66] Trừ khi bạn có bằng tiến sĩ về mật mã học, mà ngay cả khi đó cũng chỉ với điều kiện được bình duyệt kỹ lưỡng, thử nghiệm thực địa rộng rãi kèm chương trình thưởng săn lỗi (bug bounty), và ngân sách cho việc bảo trì dài hạn.

[67] Studies of Interference in Serial Verbal Reactions [Str35]

[67] Studies of Interference in Serial Verbal Reactions [Str35]

[68] We have two versions of this panel. One uses different colors, and the other uses shades of gray. If you're seeing this in black and white and want the color version, or if you're having trouble distinguishing colors and want to try the grayscale version, pop over to <https://pragprog.com/the-pragmatic-programmer/stroop-effect> .

[68] Chúng tôi có hai phiên bản của bảng này. Một dùng các màu khác nhau, còn cái kia dùng các sắc độ xám. Nếu bạn đang xem bản này dưới dạng đen trắng và muốn có bản màu, hoặc nếu bạn gặp khó khăn trong việc phân biệt màu và muốn thử bản thang xám, hãy ghé qua <https://pragprog.com/the-pragmatic-programmer/stroop-effect> .

[69] Do you know why `i` is commonly used as a loop variable? The answer comes from over 60 years ago, when variables starting with `I` through `N` were integers in the original FORTRAN. And FORTRAN was in turn influenced by algebra.

[69] Bạn có biết vì sao `i` thường được dùng làm biến vòng lặp không? Câu trả lời đến từ hơn 60 năm trước, khi các biến bắt đầu bằng chữ `I` đến `N` là số nguyên trong FORTRAN nguyên thủy. Và đến lượt mình, FORTRAN lại chịu ảnh hưởng từ đại số.

Copyright © 2020 Pearson Education, Inc.

Copyright © 2020 Pearson Education, Inc.

Before the Project — Trước Khi Bắt Đầu Dự Án

At the very beginning of a project, you and the team need to learn the **requirements**. Simply being told what to do or listening to users is not enough: read **Topic 45**, The Requirements Pit and learn how to avoid the common traps and pitfalls.

Ngay ở thời điểm khởi đầu của một dự án, bạn và cả nhóm cần phải nắm được các yêu cầu (requirements). Chỉ đơn thuần được người ta bảo phải làm gì hay ngồi nghe người dùng nói thôi thì chưa đủ: hãy đọc Topic 45, Cái Hố Yêu Cầu, và học cách né tránh những cạm bẫy thường gặp.

Conventional wisdom and constraint management are the topics of Topic 46, Solving Impossible Puzzles . Whether you are performing requirements, analysis, coding, or testing, difficult problems will crop up. Most of the time, they won't be as difficult as they first appear to be.

Trí khôn thông thường và việc quản lý ràng buộc là chủ đề của Topic 46, Giải Những Câu Đố Bất Khả Thi. Dù bạn đang làm yêu cầu, phân tích, viết mã hay kiểm thử, những bài toán hóc búa kiểu gì cũng sẽ lộ mặt ra. Phần lớn thời gian, chúng chẳng khó như vẻ ngoài lúc đầu của chúng đâu.

And when that impossible project comes up, we like to turn to our secret weapon: Topic 47, **Working Together** . And by “working together” we don't mean sharing a massive requirements document, flinging heavily cc'd emails or enduring endless meetings. We mean solving problems together while coding. We'll show you who you need and how to start.

Và khi cái dự án bất khả thi ấy ập tới, chúng tôi thích viện đến vũ khí bí mật của mình: Topic 47, Làm Việc Cùng Nhau. Và khi nói “làm việc cùng nhau”, chúng tôi không có ý là cùng chia sẻ một bản tài liệu yêu cầu đồ sộ, quăng qua quăng lại những email cc cho cả thiên hạ, hay chịu đựng những cuộc họp dài lê thê. Chúng tôi muốn nói đến chuyện cùng nhau giải quyết vấn đề trong lúc viết mã. Chúng tôi sẽ chỉ cho bạn cần những ai và bắt đầu từ đâu.

Even though the **Agile** Manifesto begins with “Individuals and interactions over processes and tools,” virtually all “agile” projects begin with an ironic discussion of which **process** and which tools they'll use. But no matter how well thought out it is,

Mặc dù Tuyên ngôn Agile (Agile Manifesto) mở đầu bằng câu “Cá nhân và tương tác hơn là quy trình và công cụ”, hầu như mọi dự án “agile” đều khởi sự bằng một cuộc tranh luận đầy mỉa mai về việc sẽ dùng quy trình nào và công cụ nào. Nhưng dù cho nó được nghĩ ra chu đáo đến đâu,

and regardless of which “best practices” it includes, no method can replace thinking . You don't need

any particular process or tool, what you do need is the Topic 48, The Essence of **Agility**.

và bất kể nó gói ghém bao nhiêu “thực hành tốt nhất” (best practices), không phương pháp nào thay thế được việc tư duy. Bạn chẳng cần một quy trình hay công cụ cụ thể nào cả, cái bạn thật sự cần là Topic 48, Tinh Túy Của Sự Linh Hoạt.

With these critical issues sorted out before the project gets under way, you can be better positioned to avoid “**analysis paralysis**” and actually begin—and complete—your successful project.

Khi đã giải quyết ổn thỏa những vấn đề then chốt này trước lúc dự án lăn bánh, bạn sẽ ở vào vị thế tốt hơn để tránh “tê liệt vì phân tích” (analysis paralysis) và thực sự bắt đầu—rồi hoàn thành—dự án thành công của mình.

Perfection is achieved, not when there is nothing left to add but when there is nothing left to take away... Antoine de St. Exupery, Wind, Sand, and Stars, 1939

Sự hoàn hảo đạt được không phải khi chẳng còn gì để thêm vào, mà khi chẳng còn gì để bớt đi nữa... Antoine de St. Exupery, Wind, Sand, and Stars, 1939

Topic 45. The Requirements Pit — Topic 45. Cái Hố Yêu Cầu

Many books and tutorials refer to **requirements gathering** as an early phase of the project. The word “gathering” seems to imply a tribe of happy analysts, foraging for nuggets of wisdom that are lying on the ground all around them while the Pastoral Symphony plays gently in the background. “Gathering” implies that the requirements are already there— you need merely find them, place them in your basket, and be merrily on your way.

Nhiều sách và hướng dẫn nhắc tới việc thu thập yêu cầu như một giai đoạn sớm của dự án. Chữ “thu thập” (gathering) dường như ngụ ý một bộ tộc các nhà phân tích hớn hởi đi nhặt nhạnh những hạt ngọc trí tuệ nằm rải rác trên mặt đất xung quanh họ, trong khi bản Giao hưởng Đồng quê dịu dặt cất lên làm nền. “Thu thập” hàm ý rằng yêu cầu vốn đã có sẵn ở đó rồi—bạn chỉ việc tìm thấy chúng, bỏ vào giỏ, rồi vui vẻ lên đường.

It doesn’t quite work that way. Requirements rarely lie on the surface. Normally, they’re buried deep beneath layers of assumptions, misconceptions, and politics. Even worse, often they don’t really exist at all.

Mọi chuyện chẳng diễn ra đúng như thế đâu. Yêu cầu hiếm khi nằm phơi trên bề mặt. Bình thường, chúng bị chôn sâu dưới những tầng giả định, ngộ nhận, và chính trị. Tệ hơn nữa, nhiều khi chúng còn chẳng thực sự tồn tại.

Tip 75: No One Knows Exactly What They Want

Tip 75: Không Ai Biết Chính Xác Mình Muốn Gì

The Requirements Myth — Huyền Thoại Về Yêu Cầu

In the early days of software, computers were more valuable (in terms of amortized cost per hour) than the people who worked with them. We saved money by trying to get things correct the

Vào những ngày đầu của ngành phần mềm, máy tính có giá trị (xét theo chi phí khấu hao tính theo giờ) hơn cả những con người làm việc với chúng. Chúng tôi tiết kiệm tiền bằng cách cố làm cho mọi thứ đúng ngay từ

first time. Part of that process was trying to specify exactly what we were going to get the machine to do. We’d start by getting a specification of the requirements, parlay that into a design document,

then into flowcharts and pseudo code, and finally into code. Before feeding it into a computer, though, we'd spend time desk checking it.

lần đầu. Một phần của quá trình ấy là cố đặc tả thật chính xác cái mà chúng tôi sắp bắt cỗ máy làm. Chúng tôi sẽ bắt đầu bằng cách lập một bản đặc tả các yêu cầu, biến nó thành một tài liệu thiết kế, rồi thành lưu đồ và mã giả, và cuối cùng thành mã thật. Tuy nhiên, trước khi nạp nó vào máy tính, chúng tôi sẽ dành thời gian rà soát nó trên giấy.

It cost a lot of money. And that cost meant that people only tried to automate something when they knew exactly what they wanted. As early machines were fairly limited, the scope of problems they solved was constrained: it was actually possible to understand the whole problem before you started.

Việc đó tốn rất nhiều tiền. Và cái chi phí ấy có nghĩa là người ta chỉ cố tự động hóa một thứ gì đó khi họ biết chính xác mình muốn gì. Vì những cỗ máy thời ấy khá hạn chế, phạm vi các bài toán mà chúng giải quyết được cũng bị bó hẹp: thực ra ta hoàn toàn có thể hiểu trọn vẹn cả bài toán trước khi bắt đầu.

But that is not the real world. The real world is messy, conflicted, and unknown. In that world, exact specifications of anything are rare, if not downright impossible.

Nhưng đó không phải là thế giới thực. Thế giới thực thì lộn xộn, đầy mâu thuẫn và chưa biết trước được. Trong cái thế giới ấy, đặc tả chính xác cho bất cứ thứ gì là chuyện hiếm, nếu không nói là hoàn toàn bất khả thi.

That's where we programmers come in. Our job is to help people understand what they want. In fact, that's probably our most valuable attribute. And it's worth repeating:

Và đó chính là lúc cánh lập trình viên chúng ta xuất hiện. Việc của chúng ta là giúp người ta hiểu được họ muốn gì. Thực ra, đó có lẽ là phẩm chất quý giá nhất của chúng ta. Và điều này đáng được nhắc lại:

Programmers Help People Understand What They — Lập Trình Viên Giúp Người Ta Hiểu Được Họ

Tip 76: Want

Tip 76: Muốn Gì

Programming As Therapy — Lập Trình Như Một Liệu Pháp Tâm Lý

Let's call the people who ask us to write software our clients.

Hãy gọi những người nhờ chúng ta viết phần mềm là khách hàng (client) của mình.

The typical **client** comes to us with a need. The need may be strategic, but it is just as likely to be a tactical issue: a response to a current problem. The need may be for a change to an existing system or it may ask for something new. The need will sometimes be expressed in business terms, and sometimes in

Vị khách hàng điển hình tìm đến chúng ta với một nhu cầu. Nhu cầu ấy có thể mang tính chiến lược, nhưng cũng rất có khả năng là một vấn đề chiến thuật: một phản ứng trước trục trặc đang xảy ra. Nhu cầu có thể là thay đổi một hệ thống đang có, hoặc đòi hỏi một thứ gì đó mới toanh. Đôi khi nhu cầu được diễn đạt bằng ngôn ngữ kinh doanh, đôi khi bằng

technical ones.

ngôn ngữ kỹ thuật.

The mistake new developers often make is to take this statement of need and implement a solution for it.

Sai lầm mà các lập trình viên non kinh nghiệm thường mắc phải là cầm lấy lời phát biểu nhu cầu này rồi hiện thực ngay một giải pháp cho nó.

In our experience, this initial statement of need is not an absolute requirement. The client may not realize this, but it is really an invitation to explore.

Theo kinh nghiệm của chúng tôi, lời phát biểu nhu cầu ban đầu này không phải là một yêu cầu tuyệt đối. Khách hàng có thể không nhận ra điều đó, nhưng thực chất nó là một lời mời gọi cùng khám phá.

Let's take a simple example.

Hãy lấy một ví dụ đơn giản.

You work for a **publisher** of paper and electronic books. You're given a new requirement:

Bạn làm việc cho một nhà xuất bản sách giấy và sách điện tử. Bạn được giao một yêu cầu mới:

Shipping should be free on all orders costing \$50 or more.

Mọi đơn hàng từ 50 đô-la trở lên đều được miễn phí vận chuyển.

Stop for a second and imagine yourself in that position. What's the first thing that comes to mind?

Hãy dừng lại một giây và tưởng tượng bạn đang ở vào vị trí đó. Điều đầu tiên nảy ra trong đầu bạn là gì?

The chances are very good that you had questions:

Nhiều khả năng là bạn đã nảy ra mấy câu hỏi:

Does the \$50 include tax?

50 đô-la đó đã bao gồm thuế chưa?

Does the \$50 include current shipping charges?

50 đô-la đó đã bao gồm phí vận chuyển hiện hành chưa?

Does the \$50 have to be for paper books, or can the order also include ebooks?

50 đô-la đó có bắt buộc phải là sách giấy không, hay đơn hàng cũng có thể bao gồm cả ebook?

What kind of shipping is offered? Priority? Ground?

Loại hình vận chuyển nào được áp dụng? Ưu tiên? Đường bộ?

What about international orders?

Còn đơn hàng quốc tế thì sao?

How often will the \$50 limit change in the future?

Ngưỡng 50 đô-la này về sau sẽ thay đổi bao nhiêu lần?

That's what we do. When given something that seems simple, we annoy people by looking for edge cases and asking about

Đó chính là việc chúng ta làm. Khi được giao một thứ trông có vẻ đơn giản, chúng ta làm phiền người khác bằng cách sẫm soi các trường hợp biên rồi hỏi về

them.

chúng.

The chances are the client will have already thought of some of these, and just assumed that the implementation would work that way. Asking the question just flushes that information out.

Nhiều khả năng là khách hàng đã nghĩ tới một số câu hỏi trong đó rồi, và chỉ đơn giản mặc định rằng phần hiện thực sẽ chạy đúng theo cách ấy. Việc đặt câu hỏi chỉ là để moi cái thông tin đó ra mà thôi.

But other questions will likely be things that the client hadn't previously considered. That's where things get interesting, and where a good developer learns to be diplomatic.

Nhưng những câu hỏi khác thì có lẽ là những điều khách hàng chưa từng cân nhắc tới trước đây. Đó là lúc mọi chuyện trở nên thú vị, và là lúc một lập trình viên giỏi học cách ăn nói khéo léo.

You: We were wondering about the \$50 total. Does that include what we'd normally charge for shipping?

Bạn: Chúng tôi đang thắc mắc về tổng giá trị 50 đô-la. Con số đó có bao gồm khoản phí vận chuyển mà chúng ta vẫn thường tính không?

Client: Of course. It's the total they'd pay us.

Khách hàng: Tất nhiên rồi. Đó là tổng số tiền họ phải trả cho chúng ta mà.

You: That's nice and simple for our customers to understand: I can see the attraction. But I can see some less scrupulous customers trying to game that system.

Bạn: Quy định ấy đơn giản, dễ hiểu cho khách hàng của ta: tôi thấy được sức hấp dẫn của nó. Nhưng tôi cũng thấy một số khách hàng kém lành đạo có thể tìm cách lách hệ thống này.

Client: How so?

Khách hàng: Lách bằng cách nào?

You: Well, let's say they buy a book for \$25, and then select overnight shipping, the most expensive option. That'll likely be about \$30, making the whole order \$55. We'd then make the shipping free, and they'd get overnight shipping on a \$25 book for just \$25.

Bạn: À, cứ cho là họ mua một cuốn sách giá 25 đô-la, rồi chọn vận chuyển hỏa tốc trong đêm, là tùy chọn đắt nhất. Khoản đó nhiều khả năng vào tầm 30 đô-la, khiến tổng đơn hàng thành 55 đô-la. Lúc ấy chúng ta sẽ miễn phí vận chuyển, và họ được giao trong đêm một cuốn sách 25 đô-la chỉ với đúng 25 đô-la.

(At this point the experienced developer stops. Deliver

(Đến đây thì lập trình viên dày dạn dừng lại. Hãy trình bày

facts, and let the client make the decisions,)

dữ kiện, rồi để khách hàng tự ra quyết định,)

Client: Ouch. That certainly wasn't what I intended; we'd lose money on those orders. What are the options?

Khách hàng: Ồi. Đó chắc chắn không phải điều tôi định làm; chúng ta sẽ lỡ với những đơn hàng kiểu đó. Vậy có những lựa chọn nào?

And this starts an exploration. Your role in this is to interpret what the client says and to feed back to them the implications. This is both an intellectual process and a creative one: you're thinking on your feet and you're contributing to a solution that is likely to be better than one that either you or the client would have produced alone.

Và thế là một cuộc khám phá bắt đầu. Vai trò của bạn ở đây là diễn giải những gì khách hàng nói và phản hồi lại cho họ những hệ quả kéo theo. Đây vừa là một quá trình trí tuệ, vừa là một quá trình sáng tạo: bạn đang vừa nghĩ vừa làm tại chỗ, và bạn đang góp phần tạo ra một giải pháp mà nhiều khả năng còn tốt hơn cái mà một mình bạn hoặc một mình khách hàng có thể nghĩ ra.

Requirements Are A Process — Yêu Cầu Là Một Quá Trình

In the previous example, the developer took the requirements and fed-back a consequence to the client. This initiated the exploration. During that exploration, you are likely to come up with more feedback as the client plays with different solutions. This is the reality of all requirements gathering:

Trong ví dụ vừa rồi, lập trình viên cầm lấy yêu cầu và phản hồi lại cho khách hàng một hệ quả. Điều đó khởi động cuộc khám phá. Trong quá trình khám phá ấy, nhiều khả năng bạn sẽ nảy ra thêm nhiều phản hồi nữa khi khách hàng nghịch thử với các giải pháp khác nhau. Đây mới là thực tế của mọi công việc thu thập yêu cầu:

Tip 77: Requirements Are Learned in a Feedback Loop

Tip 77: Yêu Cầu Được Học Trong Một Vòng Phản Hồi

Your job is to help the client understand the consequences of their stated requirements. You do that by generating feedback, and letting them use that feedback to refine their thinking.

Việc của bạn là giúp khách hàng hiểu được những hệ quả của các yêu cầu mà họ đã phát biểu. Bạn làm điều đó bằng cách tạo ra phản hồi, và để họ dùng phản hồi ấy mà mài giũa lại tư duy của mình.

In the previous example, the feedback was easy to express in words. Sometimes that's not the case. And sometimes you honestly won't know enough about the domain to be as specific as that.

Trong ví dụ trước, phản hồi rất dễ diễn đạt bằng lời. Đôi khi mọi chuyện không như vậy. Và đôi khi nói thật là bạn cũng chẳng hiểu đủ về lĩnh vực đó để có thể cụ thể được như thế.

In those cases, **Pragmatic** Programmers rely on the “is this what

Trong những trường hợp ấy, các Lập trình viên Thực dụng (Pragmatic Programmer) dựa vào trường phái phản hồi “ý anh là

you meant?” school of feedback. We produce mockups and prototypes, and let the client play with them. Ideally the things we produce are flexible enough that we can change them during our discussions with the client, letting us respond to “that isn't what I meant” with “so more like this?”

thế này phải không?”. Chúng tôi dựng ra các bản phác (mockup) và bản mẫu (prototype), rồi để khách hàng nghịch thử với chúng. Lý tưởng nhất là những thứ chúng tôi tạo ra phải đủ linh hoạt để chúng tôi có thể chỉnh sửa ngay trong lúc trao đổi với khách hàng, cho phép chúng tôi đáp lại câu “đó không phải ý tôi” bằng câu “vậy đại loại thế này hả?”.

Sometimes these mockups can be thrown together in an hour or so. They are obviously just hacks to get an idea across.

Đôi khi những bản phác này có thể được chấp vá lại trong vòng độ một tiếng đồng hồ. Rõ ràng chúng chỉ là những thứ làm tạm bợ để truyền đạt một ý tưởng.

But the reality is that all of the work we do is actually some form of **mockup**. Even at the end of a project we're still interpreting what our client wants. In fact, by that point we're likely to have more clients: the QA people, operations, marketing, and maybe even test groups of customers.

Nhưng thực tế là toàn bộ công việc chúng ta làm thật ra đều là một dạng bản phác nào đó. Ngay cả ở cuối dự án, chúng ta vẫn còn đang diễn giải điều khách hàng muốn. Thực vậy, đến lúc đó nhiều khả năng chúng ta sẽ có thêm nhiều “khách hàng” hơn: nhóm QA, nhóm vận hành, nhóm tiếp thị, và có khi cả những nhóm khách hàng dùng thử.

So the **Pragmatic Programmer** looks at all of the project as a requirements gathering exercise. That's why we prefer short iterations; ones that end with direct client feedback. This keeps us on track, and makes sure that if we do go in the wrong direction, the amount of time lost is minimized.

Thế nên Lập trình viên Thực dụng nhìn cả dự án như một bài tập thu thập yêu cầu. Đó là lý do chúng tôi ưa các vòng lặp ngắn; những vòng lặp kết thúc bằng phản hồi trực tiếp từ khách hàng. Điều này giữ chúng tôi đi đúng hướng, và đảm bảo rằng nếu lỡ có đi nhầm đường, lượng thời gian mất đi cũng được giảm thiểu.

Walk In Your Client'S Shoes — Hãy Đi Trong Đôi Giày Của Khách Hàng

There's a simple technique for getting inside your clients' heads that isn't used often enough: become a client. Are you writing a system for the help desk? Spend a couple of days monitoring the phones with an experienced support person. Are you automating a manual stock control system? Work in the warehouse for a week. [70]

Có một kỹ thuật đơn giản để chui vào đầu khách hàng mà người ta ít dùng đến: hãy trở thành khách hàng. Bạn đang viết một hệ thống cho bộ phận hỗ trợ (help desk)? Hãy dành vài ngày trực điện thoại cùng một nhân viên hỗ trợ giàu kinh nghiệm. Bạn đang tự động hóa một hệ thống quản lý kho thủ công? Hãy vào làm trong kho một tuần. [70]

As well as giving you insight into how the system will really be used, you'd be amazed at how the request “May I sit in for a week while you do your job?” helps build trust and establishes a basis for communication with your clients. Just remember not

Bên cạnh việc cho bạn cái nhìn thấu đáo về cách hệ thống sẽ thực sự được dùng, bạn sẽ kinh ngạc khi thấy lời đề nghị “Tôi ngồi cùng anh một tuần trong lúc anh làm việc được không?” giúp xây dựng lòng tin và tạo nền tảng cho việc giao tiếp với khách hàng đến mức nào. Chỉ cần nhớ là đừng

to get in the way!

có cản đường người ta!

Tip 78: Work with a User to Think Like a User

Tip 78: Làm Việc Cùng Người Dùng Để Nghĩ Như Người Dùng

Gathering feedback is also the time to start to build a rapport with your client base, learning their expectations and hopes for the system you are building. See Topic 52, Delight Your Users , for more.

Thu thập phản hồi cũng là lúc bắt đầu gây dựng mối thân tình với cộng đồng khách hàng của bạn, tìm hiểu kỳ vọng và mong mỏi của họ về hệ thống mà bạn đang xây. Xem thêm ở Topic 52, Làm Hải Lòng Người Dừng.

Requirements Vs. Policy — Yêu Cầu So Với Chính Sách

Let's imagine that while discussing a Human Resources system, a client says "Only an employee's supervisors and the personnel department may view that employee's records." Is this statement truly a requirement? Perhaps today, but it embeds business **policy** in an absolute statement.

Hãy hình dung rằng trong lúc thảo luận về một hệ thống Nhân sự, một khách hàng nói "Chỉ cấp trên của một nhân viên và phòng nhân sự mới được xem hồ sơ của nhân viên đó." Lời này liệu có thực sự là một yêu cầu không? Có lẽ là yêu cầu của ngày hôm nay, nhưng nó nhồi một chính sách kinh doanh vào trong một phát biểu tuyệt đối.

Business policy? Requirement? It's a relatively subtle distinction, but it's one that will have profound implications for the developers. If the requirement is stated as "Only supervisors and personnel can view an employee record," the developer may end up coding an explicit test every time the application accesses this data. However, if the statement is "Only authorized users may access an employee record," the developer will probably design and implement some kind of access control system. When policy changes (and it will), only the **metadata** for that system will need to be updated. In fact, gathering requirements in this way naturally leads you to a system that is well factored to support metadata.

Chính sách kinh doanh? Yêu cầu? Đó là một sự phân biệt khá tinh tế, nhưng là sự phân biệt sẽ có những hệ quả sâu xa cho cánh lập trình viên. Nếu yêu cầu được phát biểu là "Chỉ cấp trên và nhân sự mới được xem hồ sơ nhân viên," lập trình viên có thể sẽ rất cuốc phải viết một phép kiểm tra tường minh mỗi lần ứng dụng truy cập dữ liệu này. Tuy nhiên, nếu phát biểu là "Chỉ những người dùng được cấp quyền mới được truy cập hồ sơ nhân viên," thì lập trình viên có lẽ sẽ thiết kế và hiện thực một loại hệ thống kiểm soát truy cập nào đó. Khi chính sách thay đổi (và nó sẽ thay đổi), chỉ cần cập nhật phần metadata của hệ thống ấy là đủ. Thực ra, thu thập yêu cầu theo cách này sẽ tự nhiên dẫn bạn đến một hệ thống được phân tách khéo léo để hỗ trợ metadata.

In fact, there's a general rule here:

Thực vậy, ở đây có một quy tắc chung:

Tip 79: Policy Is Metadata

Tip 79: Chính Sách Là Metadata

Implement the general case, with the policy information as an example of the type of thing the system needs to support.

Hãy hiện thực trường hợp tổng quát, với thông tin chính sách đóng vai một ví dụ về loại thứ mà hệ thống cần hỗ trợ.

Requirements Vs. Reality — Yêu Cầu So Với Thực Tế

In a January 1999 Wired magazine article, [71] producer and musician Brian Eno described an incredible piece of technology —the ultimate mixing board. It does anything to sound that can be done. And yet, instead of letting musicians make better music, or produce a recording faster or less expensively, it gets in the way; it disrupts the creative process.

Trong một bài báo trên tạp chí Wired số tháng 1 năm 1999, [71] nhà sản xuất kiêm nhạc sĩ Brian Eno mô tả một món công nghệ phi thường — chiếc bàn trộn âm thanh tối thượng. Nó làm được với âm thanh mọi thứ có thể làm. Vậy mà thay vì giúp các nhạc sĩ làm ra thứ âm nhạc hay hơn, hay thu âm nhanh hơn và rẻ hơn, nó lại cản đường; nó phá vỡ quá trình sáng tạo.

To see why, you have to look at how recording engineers work. They balance sounds intuitively. Over the years, they develop an innate **feedback loop** between their ears and their fingertips—sliding faders, rotating knobs, and so on. However, the interface to the new mixer didn't leverage off those abilities. Instead, it forced its users to type on a keyboard or click a mouse. The functions it provided were comprehensive, but they were packaged in unfamiliar and exotic ways. The functions the engineers needed were sometimes hidden behind obscure names, or were achieved with nonintuitive combinations of basic facilities.

Để hiểu vì sao, bạn phải nhìn vào cách các kỹ sư thu âm làm việc. Họ cân chỉnh âm thanh bằng trực giác. Qua bao năm tháng, họ phát triển một vòng phản hồi bẩm sinh giữa đôi tai và mấy đầu ngón tay—gạt cần trượt, vặn núm xoay, vân vân. Tuy nhiên, giao diện của chiếc bàn trộn mới không tận dụng được những khả năng đó. Thay vào đó, nó buộc người dùng phải gõ bàn phím hoặc bấm chuột. Các chức năng nó cung cấp thì toàn diện, nhưng chúng được đóng gói theo những cách xa lạ và quái dị. Những chức năng mà các kỹ sư cần đôi khi lại bị giấu sau những cái tên khó hiểu, hoặc phải đạt được bằng những tổ hợp phản trực giác của các tính năng cơ bản.

This example also illustrates our belief that successful tools adapt to the hands that use them. Successful requirements gathering takes this into account. And this is why early feedback, with prototypes or **tracer bullets**, will let your clients say “yes, it does what I want, but not how I want.”

Ví dụ này cũng minh họa cho niềm tin của chúng tôi rằng những công cụ thành công là những công cụ thích nghi với đôi tay sử dụng chúng. Thu thập yêu cầu thành công có tính đến điều này. Và đó là lý do vì sao phản hồi sớm, bằng các bản mẫu hay đạn vạch đường (tracer bullets), sẽ cho phép khách hàng của bạn nói “đúng rồi, nó làm đúng cái tôi muốn, nhưng không theo cách tôi muốn.”

Documenting Requirements — Tài Liệu Hóa Yêu Cầu

We believe that the best requirements documentation, perhaps

Chúng tôi tin rằng tài liệu yêu cầu tốt nhất, mà có lẽ
the only requirements documentation, is working code.

là tài liệu yêu cầu duy nhất, chính là mã đang chạy được.

But that doesn't mean that you can get away without documenting your understanding of what the client wants. It just means that those documents are not a deliverable: they are not something that you give to a client to sign off on. Instead, they are simply mileposts to help guide the implementation process.

Nhưng điều đó không có nghĩa là bạn có thể trốn được việc tài liệu hóa cách hiểu của mình về điều khách hàng muốn. Nó chỉ có nghĩa là những tài liệu ấy không phải là sản phẩm bàn giao: chúng không phải là thứ bạn đưa cho khách hàng để họ ký duyệt. Thay vào đó, chúng chỉ đơn giản là những cột mốc giúp dẫn đường cho quá trình hiện thực.

Requirements Documents Are Not for Clients In the past, both Andy and Dave have been on projects that produced incredibly detailed requirements. These substantial documents expanded on the

client's initial two-minute explanation of what was wanted, producing inch-thick masterpieces full of diagrams and tables. Things were specified to the point where there was almost no room for ambiguity in the implementation. Given sufficiently powerful tools, the document could actually be the final program.

Tài Liệu Yêu Cầu Không Phải Dành Cho Khách Hàng Trong quá khứ, cả Andy lẫn Dave đều từng tham gia những dự án sản sinh ra các bản yêu cầu chi tiết đến không tưởng. Những tài liệu đồ sộ ấy bồi đắp thêm vào lời giải thích vồn vện hai phút ban đầu của khách hàng về điều họ muốn, để ra những kiệt tác dày cả tắc đầy sơ đồ và bảng biểu. Mọi thứ được đặc tả tới mức gần như chẳng còn chỗ cho sự mơ hồ nào trong khâu hiện thực. Nếu có công cụ đủ mạnh, cái tài liệu ấy thực ra có thể chính là chương trình cuối cùng.

Creating these documents was a mistake for two reasons. First, as we've discussed, the client doesn't really know what they want up front. So when we take what they say and expand it into what is almost a legal document, we are building an incredibly complex castle on quicksand.

Việc tạo ra những tài liệu này là một sai lầm vì hai lý do. Thứ nhất, như đã bàn, khách hàng thực ra chẳng biết ngay từ đầu họ muốn gì. Thế nên khi chúng ta cầm lấy lời họ nói rồi nở nó ra thành một thứ gần như một văn bản pháp lý, là ta đang xây một tòa lâu đài cực kỳ phức tạp trên nền cát lún.

You might say "but then we take the document to the client and they sign off on it. We're getting feedback." And that leads us to the second problem with these requirement specifications: the client never reads them.

Bạn có thể nói "nhưng rồi chúng ta mang tài liệu tới cho khách hàng và họ ký duyệt mà. Ta đang nhận được phản hồi đấy chứ." Và điều đó dẫn chúng ta tới vấn đề thứ hai với các bản đặc tả yêu cầu này: khách hàng chẳng bao giờ đọc chúng.

The client uses programmers because, while the client is motivated by solving a high-level and somewhat nebulous problem, programmers are interested in all the details and

Khách hàng dùng lập trình viên vì, trong khi khách hàng có động lực bởi việc giải quyết một bài toán ở tầm cao và phân nào mơ hồ, thì lập trình viên lại quan tâm tới mọi chi tiết và

nuances. The requirements document is written for developers, and contains information and subtleties that are sometimes incomprehensible and frequently boring to the client.

sắc thái. Tài liệu yêu cầu được viết cho cánh lập trình viên, và chứa đựng những thông tin cùng những điều tinh tế mà đối với khách hàng đôi khi là khó hiểu và thường là chán phèo.

Submit a 200-page requirements document, and the client will likely heft it to decide if it weighs enough to be important, they may read the first couple of paragraphs (which is why the first two paragraphs are always titled Management Summary), and they may flick through the rest, sometimes stopping when there's a neat diagram.

Hãy nộp một tài liệu yêu cầu dày 200 trang, và khách hàng nhiều khả năng sẽ nhấc thử nó lên để xem nó có đủ nặng để mà quan trọng hay không, họ có thể đọc vài đoạn đầu (đó là lý do hai đoạn đầu luôn được đặt tiêu đề là Tóm Tắt Cho Lãnh Đạo), và họ có thể lật lướt qua phần còn lại, thì thoảng dừng lại ở chỗ có một sơ đồ trông gọn gàng đẹp mắt.

This isn't putting the client down. But giving them a large technical document is like giving the average developer a copy of the Iliad in Homeric Greek and asking them to code the video game from it.

Đây không phải là chê bai khách hàng đâu. Nhưng đưa cho họ một tài liệu kỹ thuật đồ sộ chẳng khác nào đưa cho một lập trình viên trung bình một cuốn Iliad bằng tiếng Hy Lạp thời Homer rồi bảo họ lập trình ra trò chơi điện tử từ đó.

Requirements Documents Are for Planning So we don't believe in the monolithic, heavy-enough-to-stun-an-ox, requirements document. We do, however, know that requirements have to be written down, simply because developers on a team need to know what they'll be doing.

Tài Liệu Yêu Cầu Là Để Lập Kế Hoạch Vậy nên chúng tôi không tin vào cái tài liệu yêu cầu đồ sộ, nặng-đến-mức-quạt-ngã-một-con-bò ấy. Tuy nhiên, chúng tôi biết rằng yêu cầu vẫn phải được viết ra, đơn giản vì các lập trình viên trong một nhóm cần biết họ sẽ phải làm gì.

What form does this take? We favor something that can fit on a real (or virtual) **index card**. These short descriptions are often called user stories. They describe what a small portion of the application should do from the perspective of a user of that functionality.

Việc này mang hình thức nào? Chúng tôi ưa thứ gì đó có thể vừa vặn trên một tấm thẻ ghi chú (index card) thật (hoặc ảo). Những mô tả ngắn này thường được gọi là câu chuyện người dùng (user story). Chúng mô tả một phần nhỏ của ứng dụng nên làm gì, dưới góc nhìn của một người dùng tính năng đó.

When written this way, the requirements can be placed on a board and moved around to show both status and priority.

Khi được viết theo cách này, các yêu cầu có thể được đặt lên một tấm bảng và di chuyển qua lại để thể hiện cả trạng thái lẫn độ ưu tiên.

You might think that a single index card can't hold the information needed to implement a component of the

Bạn có thể nghĩ rằng một tấm thẻ ghi chú đơn lẻ không thể chứa đủ thông tin cần để hiện thực một thành phần của

application. You'd be right. And that's part of the point. By keeping this statement of requirements short, you're encouraging developers to ask clarifying questions. You're enhancing the feedback process between clients and coders before and during the creation of each piece of code.

ứng dụng. Bạn nghĩ đúng đấy. Và đó chính là một phần của vấn đề. Bằng cách giữ cho lời phát biểu yêu cầu này ngắn gọn, bạn đang khuyến khích lập trình viên đặt câu hỏi làm rõ. Bạn đang củng cố quá trình phản hồi giữa khách hàng và người viết mã, cả trước lẫn trong lúc tạo ra từng mẩu mã.

Overspecification — Đặc Tả Quá Mức

Another big danger in producing a requirements document is being too specific. Good requirements are abstract. Where requirements are concerned, the simplest statement that accurately reflects the business need is best. This doesn't mean you can be vague—you must capture the underlying semantic invariants as requirements, and document the specific or current work practices as policy.

Một mối nguy lớn khác khi tạo ra một tài liệu yêu cầu là quá cụ thể. Yêu cầu tốt thì trừu tượng. Khi nói đến yêu cầu, lời phát biểu đơn giản nhất mà vẫn phản ánh chính xác nhu cầu kinh doanh là tốt nhất. Điều này không có nghĩa là bạn được phép mơ hồ—bạn phải nắm bắt các bất biến ngữ nghĩa nền tảng dưới dạng yêu cầu, và ghi lại những cách làm việc cụ thể hoặc hiện hành dưới dạng chính sách.

Requirements are not architecture. Requirements are not design, nor are they the user interface. Requirements are need .

Yêu cầu không phải là kiến trúc. Yêu cầu không phải là thiết kế, cũng chẳng phải giao diện người dùng. Yêu cầu là nhu cầu.

Just One More Wafer-Thin Mint... — Chỉ Một Viên Kẹo Bạc Hà Mỏng Dính Nửa Thôi Mà...

Many project failures are blamed on an increase in scope—also known as **feature bloat**, creeping featurism, or **requirements creep**. This is an aspect of the boiled-frog syndrome from Topic 4, **Stone Soup and Boiled Frogs** . What can we do to prevent requirements from creeping up on us?

Nhiều thất bại của dự án bị quy cho sự phình to phạm vi—còn được gọi là phình tính năng (feature bloat), bệnh thêm tính năng lan dần, hay sự trườn yêu cầu (requirements creep). Đây là một khía cạnh của hội chứng ếch luộc từ Topic 4, Súp Đá Và Con Ếch Bị Luộc. Chúng ta có thể làm gì để ngăn yêu cầu trườn dần lên chúng ta?

The answer (again) is feedback. If you're working with the client in iterations with constant feedback, then the client will experience first-hand the impact of “just one more feature.” They'll see another story card go up on the board, and they'll get to help choose another card to move into the next iteration to make room. Feedback works both ways.

Câu trả lời (lại một lần nữa) là phản hồi. Nếu bạn làm việc với khách hàng theo các vòng lặp với phản hồi liên tục, thì khách hàng sẽ tận mắt nếm trải tác động của chuyện “chỉ thêm một tính năng nữa thôi”. Họ sẽ thấy một tấm thẻ câu chuyện nữa được dán lên bảng, và họ sẽ được góp phần chọn một tấm thẻ khác để bỏ ra cho vòng lặp tiếp theo nhằm nhường chỗ. Phản hồi thì có tác dụng cả hai chiều.

Maintain A Glossary — Hãy Duy Trì Một Bảng Thuật Ngữ

As soon as you start discussing requirements, users and domain experts will use certain terms that have specific meaning to them. They may differentiate between a “client” and a “**customer**,” for example. It would then be inappropriate to use either word casually in the system.

Ngay khi bạn bắt đầu bàn về yêu cầu, người dùng và các chuyên gia lĩnh vực sẽ dùng những thuật ngữ nhất định mang ý nghĩa riêng đối với họ. Chẳng hạn, họ có thể phân biệt giữa “client” và “customer”. Khi đó, sẽ là không ổn nếu dùng từ nào trong hai từ ấy một cách tùy tiện trong hệ thống.

Create and maintain a project glossary—one place that defines all the specific terms and vocabulary used in a project. All participants in the project, from end users to support staff, should use the glossary to ensure consistency. This implies that the glossary needs to be widely accessible—a good argument for online documentation.

Hãy tạo và duy trì một bảng thuật ngữ dự án—một nơi duy nhất định nghĩa tất cả các thuật ngữ và từ vựng chuyên biệt được dùng trong một dự án. Mọi thành viên tham gia dự án, từ người dùng cuối đến nhân viên hỗ trợ, đều nên dùng bảng thuật ngữ này để bảo đảm sự nhất quán. Điều này hàm ý rằng bảng thuật ngữ cần được tiếp cận rộng rãi—một lý lẽ tốt cho việc lập tài liệu trực tuyến.

Tip 80: Use a Project Glossary

Tip 80: Hãy Dùng Bảng Thuật Ngữ Dự Án

It's hard to succeed on a project if users and developers call the same thing by different names or, even worse, refer to different things by the same name.

Thật khó mà thành công với một dự án nếu người dùng và lập trình viên gọi cùng một thứ bằng những cái tên khác nhau, hoặc tệ hơn nữa, gọi những thứ khác nhau bằng cùng một cái tên.

Related Sections Include — Các mục liên quan

Topic 5, **Good-Enough Software**

Topic 5, Phần Mềm Đủ Tốt

Topic 7, **Communicate!**

Topic 7, Giao Tiếp Đi!

Topic 11, **Reversibility**

Topic 11, Tính Khả Nghịch

Topic 13, Prototypes and Post-it Notes

Topic 13, Bản Mẫu Và Giấy Nhớ Post-it

Topic 23, **Design by Contract**

Topic 23, Thiết Kế Theo Hợp Đồng

Topic 43, Stay Safe Out There

Topic 43, Hãy Giữ An Toàn Ngoài Kia

Topic 44, **Naming Things**

Topic 44, Đặt Tên Mọi Thứ

Topic 46, Solving Impossible Puzzles

Topic 46, Giải Những Câu Đố Bất Khả Thi

Topic 52, Delight Your Users

Topic 52, Làm Hài Lòng Người Dùng

Exercises — Bài tập

Exercise 33 (possible answer)

Bài tập 33 (đáp án khả dĩ)

Which of the following are probably genuine requirements? Restate those that are not to make them more useful (if possible).

Trong các phát biểu sau, cái nào có lẽ là yêu cầu đích thực? Hãy phát biểu lại những cái không phải để chúng hữu ích hơn (nếu có thể).

1. The response time must be less than ~500ms.
2. Modal windows will have a gray background.
3. The application will be organized as a number of front-end processes and a back-end server.
4. If a user enters non-numeric characters in a numeric field, the system will flash the field background and not accept them.
5. The code and data for this embedded application must fit within 32Mb.

1. Thời gian phản hồi phải nhỏ hơn ~500ms.
2. Cửa sổ modal sẽ có nền màu xám.
3. Ứng dụng sẽ được tổ chức thành một số tiến trình front-end và một máy chủ back-end.
4. Nếu người dùng nhập ký tự không phải số vào một trường số, hệ thống sẽ làm nhấp nháy nền của trường đó và không chấp nhận chúng.
5. Mã và dữ liệu của ứng dụng nhúng này phải nằm gọn trong 32Mb.

Challenges — Thử thách

Can you use the software you are writing? Is it possible to have a good feel for requirements without being able to use the software yourself?

Bạn có dùng được phần mềm mà chính mình đang viết không? Liệu có thể nắm vững yêu cầu mà không tự mình dùng được phần mềm hay không?

Pick a non-computer-related problem you currently need to solve. Generate requirements for a noncomputer solution.

Hãy chọn một bài toán không liên quan tới máy tính mà bạn hiện đang cần giải. Hãy tạo ra các yêu cầu cho một giải pháp phi máy tính.

Gordius, the King of Phrygia, once tied a knot that no one could untie. It was said that whoever solved the riddle of the Gordian Knot would rule all of Asia. So along comes Alexander the Great, who chops the knot to bits with his sword. Just a little different interpretation of the requirements, that's all.... And he did end up ruling most of Asia.

Gordius, vua xứ Phrygia, từng thắt một nút thắt mà không ai cởi được. Người ta đồn rằng ai giải được câu đố của Nút thắt Gordius sẽ thống trị toàn cõi châu Á. Thế rồi Alexander Đại đế xuất hiện, chém phăng nút thắt thành từng mảnh bằng thanh kiếm của mình. Chỉ là một cách diễn giải yêu cầu hơi khác một chút thôi mà... Và rốt cuộc ông ấy thống trị được phần lớn châu Á thật.

Topic 46. Solving Impossible Puzzles — Topic 46. Giải những câu đố tưởng chừng bất khả thi

Every now and again, you will find yourself embroiled in the middle of a project when a really tough puzzle comes up: some piece of engineering that you just can't get a handle on, or perhaps some bit of code that is turning out to be much harder to write than you thought. Maybe it looks impossible. But is it really as hard as it seems?

Thỉnh thoảng, bạn sẽ thấy mình mắc kẹt giữa một dự án khi một bài toán thật sự hóc búa xuất hiện: một mảnh kỹ thuật nào đó mà bạn cứ loay hoay không nắm bắt nổi, hoặc có lẽ một đoạn mã hóa ra khó viết hơn nhiều so với bạn tưởng. Có khi trông nó như bất khả thi. Nhưng liệu nó có thật sự khó như vẻ bề ngoài không?

Consider real-world puzzles— those devious little bits of wood, wrought iron, or plastic that seem to turn up as Christmas presents or at garage sales. All you have to do is remove the ring, or fit the Tshaped pieces in the box, or whatever.

Hãy nghĩ tới những câu đố ngoài đời thực — mấy món đồ gỗ, sắt rèn hay nhựa tinh quái nhỏ xíu hay xuất hiện làm quà Giáng sinh hoặc bày bán ở mấy phiên chợ xép. Việc của bạn chỉ là tháo cái vòng ra, hoặc nhét những mảnh hình chữ T vào hộp, hay đại loại thế.

So you pull on the ring, or try to put the Ts in the box, and quickly discover that the obvious solutions just don't work. The puzzle can't be solved that way. But even though it's obvious, that doesn't stop people from trying the same thing—over and over— thinking there must be a way.

Thế là bạn kéo cái vòng, hoặc thử nhét mấy mảnh chữ T vào hộp, và nhanh chóng phát hiện rằng những lời giải hiển nhiên đơn giản là không ăn thua. Câu đố không giải được kiểu đó. Nhưng dù đã rõ ràng rành, điều ấy vẫn chẳng ngăn được người ta cứ thử đi thử lại cùng một cách — hết lần này tới lần khác — mà đinh ninh rằng phải có một lối nào đó.

Of course, there isn't. The solution lies elsewhere. The secret to solving the puzzle is to identify the real (not imagined) constraints, and find a solution therein. Some constraints are absolute ; others are merely preconceived notions . Absolute constraints must be honored, however distasteful or stupid they may appear to be.

Tất nhiên là chẳng có. Lời giải nằm ở chỗ khác. Bí quyết để giải câu đố là nhận diện được những ràng buộc thực sự (chứ không phải tưởng tượng), rồi tìm lời giải bên trong đó. Một số ràng buộc là tuyệt đối; số khác chỉ là những định kiến có sẵn. Ràng buộc tuyệt đối thì buộc phải tôn trọng, dù chúng có vẻ khó chịu hay ngớ ngẩn đến đâu.

On the other hand, as Alexander proved, some apparent constraints may not be real constraints at all. Many software problems can be just as sneaky.

Mặt khác, như Alexander đã chứng minh, một số ràng buộc tưởng như có thật có khi lại chẳng phải ràng buộc gì cả. Nhiều bài toán phần mềm cũng tinh quái y hệt.

Degrees Of Freedom — Các bậc tự do

The popular buzz-phrase “thinking outside the box” encourages us to recognize constraints that might not be applicable and to ignore them. But this phrase isn't entirely accurate. If the “box” is the boundary of constraints and conditions, then the trick is to find the box, which may be considerably larger than you think.

Cụm từ thời thượng “nghĩ ra ngoài chiếc hộp” (thinking outside the box) khuyến khích ta nhận ra những ràng buộc có thể không còn áp dụng được và bỏ qua chúng. Nhưng cụm từ này không hoàn toàn chính xác. Nếu “chiếc hộp” là ranh giới của các ràng buộc và điều kiện, thì mẹo nằm ở chỗ tìm ra chiếc hộp ấy — và nó có thể lớn hơn bạn nghĩ rất nhiều.

The key to solving puzzles is both to recognize the constraints placed on you and to recognize the degrees of freedom you do have, for in those you'll find your solution. This is why some puzzles are so effective; you may dismiss potential solutions too readily.

Chìa khóa để giải câu đố vừa là nhận ra những ràng buộc đặt lên bạn, vừa là nhận ra những bậc tự do mà bạn thực sự có, bởi lời giải nằm ở chính những bậc tự do ấy. Đây là lý do vì sao một số câu đố lại hiệu nghiệm đến thế; bạn có thể đã gạt bỏ những lời giải tiềm năng quá vội vàng.

For example, can you connect all of the dots in the following puzzle and return to the starting point with just three straight lines—without lifting your pen from the paper or retracing your steps (Math Puzzles & Games [Hol92])?

Ví dụ, bạn có thể nối tất cả các chấm trong câu đố sau và quay về điểm xuất phát chỉ với ba đường thẳng — mà không nhấc bút khỏi giấy hay tô lại đường đã vẽ không (Math Puzzles & Games [Hol92])?

You must challenge any preconceived notions and evaluate whether or not they are real, hard-and-fast constraints.

Bạn phải chất vấn mọi định kiến có sẵn và đánh giá xem chúng có thực sự là những ràng buộc cứng nhắc, bất di bất dịch hay không.

It's not whether you think inside the box or outside the box. The problem lies in finding the box—identifying the real constraints.

Vấn đề không phải bạn nghĩ trong hộp hay ngoài hộp. Vấn đề nằm ở chỗ tìm ra chiếc hộp — nhận diện những ràng buộc thực sự.

Tip 81: Don't Think Outside the Box— Find the Box

Tip 81: Đừng Nghĩ Bên Ngoài Chiếc Hộp — Hãy Tìm Ra Chiếc Hộp

When faced with an intractable problem, enumerate all the possible avenues you have before you. Don't dismiss anything, no matter how unusable or stupid it sounds. Now go through the list and explain why a certain path cannot be taken. Are you sure? Can you prove it?

Khi đối mặt với một bài toán nan giải, hãy liệt kê tất cả những con đường bạn có thể đi. Đừng gạt bỏ bất cứ điều gì, dù nó nghe vô dụng hay ngớ ngẩn đến đâu. Giờ hãy duyệt qua danh sách và giải thích vì sao một con đường nào đó không đi được. Bạn chắc chứ? Bạn chứng minh được không?

Consider the Trojan horse—a novel solution to an intractable problem. How do you get troops into a walled city without being discovered? You can bet that “through the front door” was initially dismissed as suicide.

Hãy nghĩ tới con ngựa thành Troy — một lời giải mới mẻ cho một bài toán nan giải. Làm sao đưa quân vào một thành trì có tường bao bọc mà không bị phát hiện? Bạn cá là “qua cổng trước” ban đầu đã bị gạt phăng vì coi đó là hành động tự sát.

Categorize and prioritize your constraints. When woodworkers begin a project, they cut the longest pieces first, then cut the smaller pieces out of the remaining wood. In the same manner, we want to identify the most restrictive constraints first, and fit the remaining constraints within them.

Hãy phân loại và xếp thứ tự ưu tiên cho các ràng buộc của bạn. Khi thợ mộc bắt đầu một dự án, họ cắt những thanh dài nhất trước, rồi cắt các thanh nhỏ hơn từ phần gỗ còn lại. Tương tự như vậy, ta muốn nhận diện những ràng buộc chặt chẽ nhất trước, rồi sắp các ràng buộc còn lại vừa khít vào trong chúng.

By the way, a solution to the Four Posts puzzle is shown at the

Nhân tiện, lời giải cho câu đố Bốn Cọc được trình bày ở

end of the book .

cuối sách.

Get Out Of Your Own Way! — Đừng cản đường chính mình!

Sometimes you will find yourself working on a problem that seems much harder than you thought it should be. Maybe it feels like you're going down the wrong path—that there must be an easier way than this! Perhaps you are running late on the schedule now, or even despair of ever getting the system to work because this particular problem is “impossible.”

Đôi khi bạn sẽ thấy mình đang vật lộn với một bài toán có vẻ khó hơn nhiều so với mức đáng ra phải có. Có lẽ bạn cảm thấy mình đang đi sai đường — rằng phải có cách dễ hơn thế này chứ! Có lẽ giờ bạn đang trễ tiến độ, hoặc thậm chí tuyệt vọng chẳng biết bao giờ mới làm cho hệ thống chạy được, bởi cái bài toán cụ thể này nó “bất khả thi”.

This is an ideal time to do something else for a while. Work on something different. Go walk the dog. Sleep on it.

Đây là thời điểm lý tưởng để làm việc khác một lúc. Hãy làm một thứ gì đó khác đi. Đi dắt chó đi dạo. Ngủ một giấc rồi tính.

Your conscious brain is aware of the problem, but your conscious brain is really pretty dumb (no offense). So it's time to give your real brain, that amazing associative neural net that lurks below your consciousness, some space. You'll be amazed how often the answer will just pop into your head when you deliberately distract yourself.

Bộ não ý thức của bạn biết về bài toán đấy, nhưng bộ não ý thức thì thật ra khá đần (không có ý xúc phạm đâu). Vậy nên đã đến lúc nhường chỗ cho bộ não thật của bạn — cái mạng nơ-ron liên tưởng kỳ diệu ẩn náu bên dưới ý thức. Bạn sẽ ngạc nhiên khi thấy lời giải bật ra trong đầu thường xuyên đến mức nào lúc bạn cố tình làm cho mình mất tập trung.

If that sounds too mystical for you, it isn't. Psychology Today [72] reports:

Nếu điều đó nghe có vẻ huyền bí quá với bạn, thì không đâu. Tạp chí Psychology Today [72] cho biết:

To put it plainly—people who were distracted did better on a complex problem-solving task than people who put in conscious effort.

Nói thẳng ra — những người bị làm cho xao nhãng lại làm tốt hơn ở một bài toán giải quyết vấn đề phức tạp so với những người dồn sức nỗ lực có ý thức.

If you're still not willing to drop the problem for a while, the next best thing is probably finding someone to explain it to. Often, the distraction of simply talking about it will lead you to enlightenment.

Nếu bạn vẫn chưa sẵn lòng buông bài toán xuống một lúc, thì điều tốt nhất kế tiếp có lẽ là tìm một người nào đó để giải thích bài toán cho họ nghe. Thường thì chỉ riêng sự xao nhãng khi nói về nó cũng đủ dẫn bạn tới giác ngộ.

Have them ask you questions such as:

Hãy nhờ họ đặt cho bạn những câu hỏi như:

Why are you solving this problem?

Vì sao bạn lại giải bài toán này?

What's the benefit of solving it?

Giải được nó thì lợi gì?

Are the problems you're having related to edge cases? Can you eliminate them?

Những trục trặc bạn gặp có liên quan tới các trường hợp biên không? Bạn có loại bỏ chúng được không?

Is there a simpler, related problem you can solve?

Có một bài toán liên quan nào đơn giản hơn mà bạn giải được không?

This is another example of Rubber Ducking in practice.

Đây lại là một ví dụ nữa về kỹ thuật Vịt Cao Su (Rubber Ducking) trong thực tế.

Fortune Favors The Prepared Mind — Vận may mỉm cười với những bộ óc có chuẩn bị

Louis Pasteur is reported to have said:

Tương truyền Louis Pasteur từng nói:

Dans les champs de l'observation le hasard ne favorise que les esprits préparés. (When it comes to observation, fortune favors the prepared mind.)

Dans les champs de l'observation le hasard ne favorise que les esprits préparés. (Trong lĩnh vực quan sát, vận may chỉ mỉm cười với những bộ óc có chuẩn bị.)

That is true for problem solving, too. In order to have those eureka! moments, your nonconscious brain needs to have plenty of raw material; prior experiences that can contribute to an answer.

Điều đó cũng đúng với việc giải quyết vấn đề. Để có được những khoảnh khắc eureka! ấy, bộ não vô thức của bạn cần có thật nhiều nguyên liệu thô; những kinh nghiệm trước đây có thể góp phần tạo nên lời giải.

A great way to feed your brain is to give it feedback on what works and what doesn't work as you do your daily job. And we describe a great way to do that using an **Engineering Daybook** (**Topic 22**, Engineering Daybooks).

Một cách tuyệt vời để nuôi dưỡng bộ não là cho nó phản hồi về điều gì hiệu quả và điều gì không trong lúc bạn làm công việc hằng ngày. Và chúng tôi có mô tả một cách tuyệt vời để làm điều đó bằng Nhật ký Kỹ thuật (engineering daybook) (**Topic 22**, Nhật ký Kỹ thuật).

And always remember the advice on the cover of The Hitchhiker's Guide to the Galaxy : DON'T PANIC.

Và luôn nhớ lời khuyên in trên bìa cuốn The Hitchhiker's Guide to the Galaxy: ĐỪNG HOẢNG LOẠN.

Related Sections Include — Các mục liên quan

Topic 5, **Good-Enough Software**

Topic 5, Phần mềm Đủ Tốt

Topic 37, Listen to Your Lizard Brain

Topic 37, Lắng nghe Bộ não Bò sát của bạn

Topic 45, The **Requirements** Pit

Topic 45, Cái Hố Yêu cầu

Andy wrote an entire book about this kind of thing: **Pragmatic Thinking and Learning: Refactor Your Wetware** [Hun08] .

Andy đã viết hẳn một cuốn sách về kiểu chuyện này: Pragmatic Thinking and Learning: Refactor Your Wetware [Hun08].

Challenges — Thử thách

Take a hard look at whatever difficult problem you are embroiled in today. Can you cut the Gordian knot? Do you have to do it this way? Do you have to do it at all?

Hãy nhìn thật kỹ vào bất kỳ bài toán hóc búa nào bạn đang vật lộn hôm nay. Bạn có chặt được nút thắt Gordian không? Bạn có buộc phải làm theo cách này không? Bạn có buộc phải làm nó chút nào không?

Were you handed a set of constraints when you signed on to your current project? Are they all still applicable, and is the interpretation of them still valid?

Bạn có được giao một bộ ràng buộc khi gia nhập dự án hiện tại không? Liệu chúng có còn áp dụng được hết không, và cách diễn giải chúng có còn hợp lệ không?

I've never met a human being who would want to read 17,000 pages of documentation, and if there was, I'd kill him to get him out of the gene pool. Joseph Costello, President of Cadence

Tôi chưa từng gặp ai lại muốn đọc 17.000 trang tài liệu, và nếu có một người như thế, tôi sẽ giết hắn để loại hắn khỏi vốn gen của loài người. Joseph Costello, Chủ tịch Cadence

Topic 47. Working Together — Topic 47. Làm Việc Cùng Nhau

It was one of those “impossible” projects, the kind you hear about that sounds both exhilarating and terrifying at the same time. An ancient system was approaching end-of-life, the hardware was physically going away, and a brand-new system had to be crafted that would match the (often undocumented) behavior exactly. Many hundreds of millions of dollars of other people's money would pass through this system, and the deadline from inception to deployment was on the order of months.

Đó là một trong những dự án “bất khả thi”, kiểu dự án mà bạn nghe kể lại và thấy vừa hứng khởi vừa rợn người cùng lúc. Một hệ thống cũ kỹ đang tiến gần hồi kết, phần cứng thì sắp bị tháo bỏ về mặt vật lý, còn một hệ thống hoàn toàn mới phải được dựng nên sao cho khớp y hệt hành vi (thường chẳng có tài liệu nào) của hệ cũ. Hàng trăm triệu đô-la tiền của người khác sẽ chảy qua hệ thống này, mà hạn chót từ lúc khởi xướng đến lúc triển khai chỉ tính bằng vài tháng.

And that is where Andy and Dave first met. An impossible project with a ridiculous deadline. There was only one thing that made the project a roaring success. The expert who had managed this system for years was sitting right there in her office, just across the hall from our broom closet-sized development room. Continuously available for questions, clarifications, decisions, and demos.

Và đó cũng là nơi Andy và Dave gặp nhau lần đầu. Một dự án bất khả thi với một hạn chót lỗ bịch. Chỉ có duy nhất một điều khiến dự án thành công vang dội. Vị chuyên gia đã quản lý hệ thống này suốt nhiều năm ngồi ngay trong văn phòng của bà, chỉ cách phòng phát triển bé tí xíu như cái nhà kho chứa chổi của chúng tôi một hành lang. Lúc nào cũng sẵn sàng cho mọi câu hỏi, mọi sự làm rõ, mọi quyết định và mọi buổi demo.

Throughout this book we recommend working closely with users; they are part of your team. On that first project together,

Xuyên suốt cuốn sách này, chúng tôi khuyên bạn nên làm việc sát sao với người dùng; họ là một phần của nhóm bạn. Trong dự án đầu tiên cùng nhau ấy,

we practiced what now might be called **pair programming** or **mob programming** : one person typing code while one or more other team members comment, ponder, and solve problems together. It's a powerful way of **working together** that transcends endless meetings, memos, and overstuffed legalistic documentation prized for weight over usefulness.

chúng tôi đã thực hành cái mà bây giờ có lẽ được gọi là lập trình cặp (pair programming) hay lập trình đám đông (mob programming): một người gõ mã trong khi một hay nhiều thành viên khác trong nhóm bình luận, suy ngẫm và cùng nhau giải quyết vấn đề. Đó là một cách làm việc cùng nhau đầy sức mạnh, vượt xa những cuộc họp triền miên, những bản ghi nhớ và những đồng tài liệu pháp lý phình to được coi trọng vì cân nặng hơn là vì hữu ích.

And that's what we really mean by “working with”: not just asking questions, having discussions, and taking notes, but asking questions and having discussions while you're actually coding .

Và đó mới là điều chúng tôi thực sự muốn nói khi dùng từ “làm việc cùng”: không chỉ là đặt câu hỏi, thảo luận và ghi chú, mà là đặt câu hỏi và thảo luận ngay trong lúc bạn đang thực sự viết mã.

Conway's Law

Định luật Conway (Conway's Law)

In 1967, Melvin Conway introduced an idea in *How do Committees Invent?* [Con68] which would become known as Conway's Law:

Năm 1967, Melvin Conway giới thiệu một ý tưởng trong bài *How do Committees Invent?* [Con68], về sau được biết đến với tên Định luật Conway:

Organizations which design systems are constrained to produce designs which are copies of the communication structures of these organizations.

Những tổ chức thiết kế hệ thống bị ràng buộc phải tạo ra các thiết kế sao chép lại cấu trúc giao tiếp của chính những tổ chức đó.

That is, the social structures and communication pathways of the team and the organization will be mirrored in the application, website, or product being developed. Various studies have shown strong support for this idea. We've witnessed it first-hand countless times—for example, in teams where no one talks to each other at all, resulting in siloed, “stove-pipe” systems. Or teams that were split into two, resulting in a **client**/server or frontend/backend division.

Nghĩa là, cấu trúc xã hội và các đường giao tiếp của nhóm và của tổ chức sẽ được phản chiếu vào ứng dụng, website hay sản phẩm đang được phát triển. Nhiều nghiên cứu đã cho thấy bằng chứng mạnh mẽ ủng hộ ý tưởng này. Chính chúng tôi cũng đã tận mắt chứng kiến vô số lần — chẳng hạn, ở những nhóm mà chẳng ai nói chuyện với ai, kết quả là những hệ thống biệt lập, kiểu “ống khói” (stove-pipe). Hay những nhóm bị tách làm đôi, dẫn đến sự phân chia client/server hoặc frontend/backend.

Studies also offer support for the reverse principle: you can deliberately structure your team the way you want your code to look. For example, geographically distributed teams are shown to tend toward more modular, distributed software.

Các nghiên cứu cũng đưa ra bằng chứng cho nguyên lý ngược lại: bạn có thể chủ ý sắp xếp cấu trúc nhóm theo đúng cách bạn muốn mã của mình trông như thế nào. Ví dụ, người ta thấy các nhóm phân tán về mặt địa lý có xu hướng tạo ra phần mềm mô-đun hơn, phân tán hơn.

But most importantly, development teams that include users will produce software that clearly reflects that involvement, and teams that don't bother will reflect that, too.

Nhưng quan trọng nhất, những nhóm phát triển có bao gồm cả người dùng sẽ tạo ra phần mềm phản ánh rõ rệt sự tham gia ấy, còn những nhóm chẳng buồn làm vậy thì cũng sẽ phản ánh điều đó luôn.

Pair Programming — Lập trình cặp (Pair Programming)

Pair programming is one of the practices of **eXtreme Programming** that has become popular outside of XP itself. In pair programming, one developer operates the keyboard, and

Lập trình cặp là một trong những thực hành của eXtreme Programming đã trở nên phổ biến ra ngoài cả bản thân XP. Trong lập trình cặp, một lập trình viên điều khiển bàn phím, còn

the other does not. Both work on the problem together, and can switch typing duties as needed.

người kia thì không. Cả hai cùng làm việc trên bài toán, và có thể đổi phiên gõ phím cho nhau khi cần.

There are many benefits to pair programming. Different people bring different backgrounds and experience, different problemsolving techniques and approaches, and differing levels of focus and attention to any given problem. The developer acting as **typist** must focus on the low-level details of syntax and coding style, while the other developer is free to consider higher-level issues and scope. While that might sound like a small distinction, remember that we humans have only so much brain bandwidth. Fiddling around with typing esoteric words and symbols that the compiler will grudgingly accept takes a fair bit of our own processing power. Having a second developer's full brain available during the task brings a lot more mental power to bear.

Lập trình cặp có rất nhiều lợi ích. Mỗi người mang đến một nền tảng và kinh nghiệm khác nhau, những kỹ thuật và cách tiếp cận giải quyết vấn đề khác nhau, cùng những mức độ tập trung và chú tâm khác nhau cho bất kỳ bài toán nào. Người đóng vai người gõ phím phải tập trung vào những chi tiết cấp thấp của cú pháp và phong cách viết mã, trong khi người kia rảnh trí để cân nhắc những vấn đề và phạm vi ở tầm cao hơn. Nghe thì có vẻ đó là một khác biệt nhỏ, nhưng hãy nhớ rằng chúng ta — con người — chỉ có một lượng băng thông não bộ nhất định. Cứ loay hoay gõ những từ và ký hiệu khó hiểu mà trình biên dịch sẽ miễn cưỡng chấp nhận đã ngốn kha khá năng lực xử lý của chính ta rồi. Có thêm bộ não trọn vẹn của một lập trình viên thứ hai sẵn sàng trong suốt nhiệm vụ sẽ mang đến nhiều sức mạnh tư duy hơn hẳn.

The inherent peer-pressure of a second person helps against moments of weakness and bad habits of naming variables *foo* and *such*. You're less inclined to take a potentially embarrassing shortcut when someone is actively watching, which also results in higher-quality software.

Áp lực đồng nghiệp tự nhiên từ người thứ hai giúp ta chống lại những phút yếu lòng và những thói xấu kiểu đặt tên biến là *foo* và đại loại thế. Bạn sẽ ít có xu hướng đi đường tắt một cách dễ xấu hổ khi có người đang chăm chú quan sát, và điều đó cũng cho ra phần mềm chất lượng cao hơn.

Mob Programming — Lập trình đám đông (Mob Programming)

And if two heads are better than one, what about having a dozen diverse people all working on the same problem at the same time, with one typist?

Và nếu hai cái đầu tốt hơn một, thì sao không thử cả tá người với đủ kiểu khác nhau cùng làm việc trên cùng một bài toán cùng một lúc, với chỉ một người gõ phím?

Mob programming, despite the name, does not involve torches or pitchforks. It's an extension of pair programming that involves more than just two developers. Proponents report great results using mobs to solve hard problems. Mobs can easily include people not usually considered part of the development

Lập trình đám đông, bất chấp cái tên, chẳng dính dáng gì đến đuốc đuốc hay chia ba đầu. Nó là phần mở rộng của lập trình cặp với hơn hai lập trình viên. Những người ủng hộ kể lại những kết quả tuyệt vời khi dùng đám đông để giải các bài toán học búa. Đám đông có thể dễ dàng bao gồm cả những người thường không được coi là một phần của nhóm phát triển,

team, including users, project sponsors, and testers. In fact, in our first “impossible” project together, it was a common sight for one of us to be typing while the other discussed the issue with our business expert. It was a small mob of three.

bao gồm người dùng, nhà tài trợ dự án và người kiểm thử. Thực tế, trong dự án “bất khả thi” đầu tiên của chúng tôi cùng nhau, cảnh một người gõ phím trong khi người kia bàn bạc vấn đề với chuyên gia nghiệp vụ của chúng tôi là chuyện thường thấy. Đó là một đám đông nhỏ gồm ba người.

You might think of mob programming as tight collaboration with live coding.

Bạn có thể hình dung lập trình đám đông như sự cộng tác chặt chẽ kết hợp với viết mã trực tiếp.

What Should I Do? — Vậy Tôi Nên Làm Gì?

If you’re currently only programming solo, maybe try pair programming. Give it a minimum of two weeks, only a few hours at a time, as it will feel strange at first. To brainstorm new ideas or diagnose thorny issues, perhaps try a mob programming session.

Nếu hiện giờ bạn chỉ đang lập trình một mình, có lẽ hãy thử lập trình cặp. Hãy cho nó tối thiểu hai tuần, mỗi lần chỉ vài tiếng thôi, vì lúc đầu bạn sẽ thấy nó lạ lẫm lắm. Để động não tìm ý tưởng mới hay chẩn đoán những vấn đề gai góc, có lẽ hãy thử một buổi lập trình đám đông.

If you are already pairing or mobbing, who’s included? Is it just developers, or do you allow members of your extended team to participate: users, testers, sponsors...?

Nếu bạn đã đang làm cặp hay làm đám đông rồi, thì có những ai tham gia? Chỉ toàn lập trình viên, hay bạn cho phép các thành viên trong nhóm mở rộng của mình cùng góp mặt: người dùng, người kiểm thử, nhà tài trợ...?

And as with all collaboration, you need to manage the human aspects of it as well as the technical. Here are just a few tips to get started:

Và như với mọi sự cộng tác, bạn cần quản lý cả khía cạnh con người lẫn khía cạnh kỹ thuật của nó. Dưới đây chỉ là vài mẹo để bắt đầu:

Build the code, not your ego. It’s not about who’s brightest; we all have our moments, good and bad.

Hãy dựng nên mã, đừng dựng nên cái tôi của bạn. Vấn đề không phải ai thông minh nhất; ai trong chúng ta cũng có lúc này lúc khác, lúc hay lúc dở.

Start small. Mob with only 4-5 people, or start with just a few pairs, in short sessions.

Bắt đầu từ nhỏ. Làm đám đông chỉ với 4-5 người, hoặc bắt đầu với chỉ vài cặp, trong những buổi ngắn.

Criticize the code, not the person. “Let’s look at this block” sounds much better than “you’re wrong.”

Phê bình mã, đừng phê bình con người. Câu “Cùng xem khối này nào” nghe dễ chịu hơn nhiều so với “cậu sai rồi.”

Listen and try to understand others' viewpoints. Different isn't wrong.

Lắng nghe và cố gắng hiểu quan điểm của người khác. Khác biệt không có nghĩa là sai.

Conduct frequent retrospectives to try and improve for next time.

Tổ chức những buổi nhìn lại (retrospective) thường xuyên để cố cải thiện cho lần sau.

Coding in the same office or remote, alone, in pairs, or in mobs, are all effective ways of working together to solve problems. If you and your team have only ever done it one way, you might want to experiment with a different style. But don't just jump in with a naive approach: there are rules, suggestions, and guidelines for each of these development styles. For instance, with mob programming you swap out the typist every 5-10 minutes.

Viết mã trong cùng một văn phòng hay từ xa, một mình, theo cặp, hay theo đám đông — tất cả đều là những cách làm việc cùng nhau hiệu quả để giải quyết vấn đề. Nếu bạn và nhóm của mình xưa nay chỉ làm theo đúng một kiểu, có lẽ bạn nên thử nghiệm một phong cách khác. Nhưng đừng cứ thế nhảy vào một cách ngây thơ: mỗi phong cách phát triển này đều có những quy tắc, gợi ý và hướng dẫn riêng. Chẳng hạn, với lập trình đám đông, bạn đổi người gõ phím mỗi 5-10 phút một lần.

Do some reading and research, from both textbook and experience reports, and get a feel for the advantages and pitfalls you may encounter. You might want to start by coding a simple exercise, and not just jump straight into your toughest production code.

Hãy đọc và tìm hiểu một chút, từ cả sách giáo khoa lẫn những bài kể lại kinh nghiệm thực tế, để cảm nhận được những lợi thế và những cạm bẫy bạn có thể gặp phải. Có lẽ bạn nên bắt đầu bằng cách viết mã một bài tập đơn giản, đừng nhảy thẳng vào đoạn mã sản xuất khó nhằn nhất của mình.

But however you go about it, let us suggest one final piece of advice:

Nhưng dù bạn tiếp cận theo cách nào, hãy để chúng tôi gợi ý một lời khuyên cuối cùng:

Tip 82: Don't Go into the Code Alone

Tip 82: Đừng Một Mình Xông Vào Mã

You keep using that word, I do not think it means what you think it means. Inigo Montoya, The Princess Bride

Anh cứ dùng mãi cái từ đó. Tôi không nghĩ nó có nghĩa như anh tưởng đâu. Inigo Montoya, The Princess Bride

Topic 48. The Essence of Agility — Topic 48. Tinh túy của sự linh hoạt

Agile is an adjective: it's how you do something. You can be an agile developer. You can be on a team that adopts agile practices, a team that responds to change and setbacks with **agility**. Agility is your style, not you.

Agile là một tính từ: nó nói về cách bạn làm một việc gì đó. Bạn có thể là một lập trình viên linh hoạt (agile). Bạn có thể ở trong một nhóm áp dụng các thực hành linh hoạt, một nhóm phản ứng với thay đổi và trắc trở một cách linh hoạt. Sự linh hoạt là phong cách của bạn, chứ không phải là bản thân bạn.

Tip 83: Agile Is Not a Noun; Agile Is How You Do Things

Tip 83: Agile Không Phải Là Một Danh Từ; Agile Là Cách Bạn Làm Mọi Việc

As we write this, almost 20 years after the inception of the Manifesto for Agile Software Development, [73] we see many, many developers successfully applying its values. We see many fantastic teams who find ways to take these values and use them to guide what they do, and how they change what they do.

Vào lúc chúng tôi viết những dòng này, gần 20 năm sau khi Tuyên ngôn Phát triển Phần mềm Linh hoạt (Manifesto for Agile Software Development) ra đời, [73] chúng tôi thấy rất, rất nhiều lập trình viên đang áp dụng thành công các giá trị của nó. Chúng tôi thấy nhiều nhóm tuyệt vời tìm được cách lấy những giá trị ấy và dùng chúng để dẫn lối cho những gì mình làm, và cho cách mình thay đổi những gì mình làm.

But we also see another side of agility. We see teams and companies eager for off-the-shelf solutions: Agile-in-a-Box. And we see many consultants and companies all too happy to sell them what they want. We see companies adopting more layers of management, more formal reporting, more specialized developers, and more fancy job titles which just mean “someone with a clipboard and a stopwatch.” [74]

Nhưng chúng tôi cũng thấy một mặt khác của sự linh hoạt. Chúng tôi thấy những nhóm và những công ty háo hức đi tìm giải pháp làm sẵn: Agile-in-a-Box, agile đóng hộp. Và chúng tôi thấy vô số nhà tư vấn cùng công ty quá đổi vui lòng bán cho họ đúng cái họ muốn. Chúng tôi thấy các công ty thêm vào nhiều tầng quản lý hơn, nhiều báo cáo hình thức hơn, nhiều lập trình viên chuyên biệt hơn, và nhiều chức danh kêu hơn mà rốt cuộc chỉ có nghĩa là “một người cầm bảng kẹp giấy và cái đồng hồ bấm giờ.” [74]

We feel that many people have lost sight of the true meaning of agility, and we’d like to see folks return to the basics.

Chúng tôi cảm thấy nhiều người đã đánh mất ý nghĩa thật sự của sự linh hoạt, và chúng tôi muốn thấy mọi người quay về với những điều căn bản.

Remember the values from the manifesto:

Hãy nhớ lại các giá trị trong bản tuyên ngôn:

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

Chúng tôi đang khám phá những cách tốt hơn để phát triển phần mềm bằng cách tự mình làm và giúp người khác làm. Qua quá trình này, chúng tôi đã đi đến chỗ coi trọng:

Individuals and interactions over processes and tools

Cá nhân và tương tác hơn là quy trình và công cụ

Working software over comprehensive documentation

Phần mềm chạy được hơn là tài liệu đầy đủ

Customer collaboration over contract negotiation

Cộng tác với khách hàng hơn là đàm phán hợp đồng

Responding to change over following a plan

Phản ứng với thay đổi hơn là bám theo một kế hoạch

That is, while there is value in the items on the right, we value the items on the left more.

Nghĩa là, dù những thứ ở vế phải vẫn có giá trị, chúng tôi coi trọng những thứ ở vế trái hơn.

Anyone selling you something that increases the importance on things on the right over things on the left clearly doesn't value the same things that we and the other manifesto writers did.

Bất kỳ ai bán cho bạn một thứ làm tăng tầm quan trọng của những điều ở vế phải so với những điều ở vế trái thì rõ ràng không coi trọng cùng những thứ mà chúng tôi và những người viết tuyên ngôn khác đã coi trọng.

And anyone selling you a solution-in-a-box hasn't read the introductory statement. The values are motivated and informed by the continuous act of uncovering better ways to produce software. This is not a static document. It's suggestions for a generative **process**.

Và bất kỳ ai bán cho bạn một giải-pháp-trong-hộp đều chưa đọc câu mở đầu. Các giá trị được thúc đẩy và soi sáng bởi hành động liên tục khám phá những cách tốt hơn để tạo ra phần mềm. Đây không phải một văn bản tĩnh. Nó là những gợi ý cho một quá trình sinh sôi.

There Can Never Be An Agile Process — Không bao giờ có thể tồn tại một “quy trình Agile”

In fact, whenever someone says “do this, and you'll be agile,” they are wrong. By definition.

Thực ra, mỗi khi có ai đó nói “làm thế này đi, rồi bạn sẽ linh hoạt,” họ đã sai. Sai về mặt định nghĩa.

Because agility, both in the physical world and in software development, is all about responding to change, responding to the unknowns you encounter after you set out. A running gazelle doesn't go in a straight line. A gymnast makes hundreds of corrections a second as they respond to changes in their environment and minor errors in their foot placement.

Bởi vì sự linh hoạt, dù trong thế giới vật lý hay trong phát triển phần mềm, đều xoay quanh việc phản ứng với thay đổi, phản ứng với những điều chưa biết mà bạn gặp phải sau khi đã lên đường. Một con linh dương đang chạy không chạy theo đường thẳng. Một vận động viên thể dục dụng cụ thực hiện hàng trăm điều chỉnh mỗi giây khi họ phản ứng với những thay đổi trong môi trường và những sai lệch nhỏ ở vị trí đặt chân.

So it is with teams and individual developers. There is no single plan you can follow when you develop software. Three of the four values tell you that. They're all about gathering and responding to feedback.

Với các nhóm và từng lập trình viên cũng vậy. Không có một kế hoạch duy nhất nào bạn có thể bám theo khi phát triển phần mềm. Ba trong bốn giá trị nói cho bạn điều đó. Tất cả chúng đều xoay quanh việc thu thập và phản ứng với phản hồi.

The values don't tell you what to do. They tell you what to look for when you decide for yourself what to do.

Các giá trị không bảo bạn phải làm gì. Chúng bảo bạn cần nhìn vào đâu khi tự mình quyết định phải làm gì.

These decisions are always contextual: they depend on who you are, the nature of your team, your application, your tooling, your company, your customer, the outside world; an incredibly large number of factors, some major and some trivial. No fixed, static plan can survive this uncertainty.

Những quyết định này luôn phụ thuộc vào bối cảnh: chúng phụ thuộc vào bạn là ai, bản chất nhóm của bạn, ứng dụng của bạn, công cụ của bạn, công ty của bạn, khách hàng của bạn, thế giới bên ngoài; vô số yếu tố, có cái lớn và có cái vụn vặt. Không một kế hoạch cố định, tính tại nào có thể sống sót qua sự bất định này.

So What Do We Do? — Vậy chúng ta làm gì?

No one can tell you what to do. But we think we can tell you something about the spirit with which you do it. It all boils down to how you deal with uncertainty. The manifesto suggests that you do this by gathering and acting on feedback. So here's our recipe for working in an agile way:

Không ai có thể bảo bạn phải làm gì. Nhưng chúng tôi nghĩ mình có thể nói cho bạn nghe đôi điều về cái tinh thần mà bạn dùng để làm việc đó. Tất cả rút lại thành cách bạn đối mặt với sự bất định. Bản tuyên ngôn gợi ý rằng bạn làm điều này bằng cách thu thập phản hồi và hành động dựa trên đó. Vậy nên đây là công thức của chúng tôi để làm việc theo lối linh hoạt:

1. Work out where you are.
 1. Xác định xem bạn đang ở đâu.
2. Make the smallest meaningful step towards where you want to be.
 2. Bước bước nhỏ nhất nhưng có ý nghĩa về phía nơi bạn muốn tới.
3. Evaluate where you end up, and fix anything you broke.
 3. Đánh giá xem mình kết thúc ở đâu, và sửa lại bất cứ thứ gì bạn đã làm hỏng.

Repeat these steps until you're done. And use them recursively, at every level of everything you do.

Lặp lại các bước này cho đến khi xong. Và dùng chúng một cách đệ quy, ở mọi cấp độ của mọi việc bạn làm.

Sometimes even the most trivial-seeming decision becomes important when you gather feedback.

Đôi khi ngay cả một quyết định trông có vẻ tầm thường nhất cũng trở nên quan trọng khi bạn thu thập phản hồi.

“Now my code needs to get the account owner.

“Giờ mã của tôi cần lấy chủ tài khoản.

```
let user = accountOwner(accountID);
```

```
let user = accountOwner(accountID);
```

Hmmm... user is a useless name. I'll make it owner .

Hừm... user là một cái tên vô dụng. Tôi sẽ đổi nó thành owner .

```
let owner = accountOwner(accountID);
```

```
let owner = accountOwner(accountID);
```

But now that feels a little redundant. What am I actually trying to do here? The story says that I'm sending this person an email, so I need to find their email address. Maybe I don't need the whole account owner at all.

Nhưng giờ cái này lại hơi thừa. Tôi thật sự đang cố làm gì ở đây? Câu chuyện nói rằng tôi đang gửi cho người này một email, nên tôi cần tìm địa chỉ email của họ. Có lẽ tôi chẳng cần đến cả đối tượng chủ tài khoản đầy đủ làm gì.

```
let email = emailOfAccountOwner(accountID);
```

```
let email = emailOfAccountOwner(accountID);
```

By applying the **feedback loop** at a really low level (the naming of a variable) we've actually improved the design of the overall system, reducing the **coupling** between this code and the code that deals with accounts.

Bằng cách áp dụng vòng phản hồi ở một cấp độ thật sự thấp (việc đặt tên cho một biến), chúng ta đã thật sự cải thiện thiết kế của cả hệ thống, giảm bớt sự ghép nối giữa đoạn mã này với đoạn mã lo chuyện tài khoản.

The feedback loop also applies at the highest level of a project. Some of our most successful work has happened when we started working on a **client's requirements**, took a single step, and realized that what we were about to do wasn't necessary, that the best solution didn't even involve software.

Vòng phản hồi cũng áp dụng được ở cấp độ cao nhất của một dự án. Một số công việc thành công nhất của chúng tôi đã diễn ra khi chúng tôi bắt tay vào các yêu cầu của khách hàng, bước một bước, rồi nhận ra rằng thứ chúng tôi sắp làm là không cần thiết, rằng giải pháp tốt nhất thậm chí chẳng dính dáng gì đến phần mềm.

This loop applies outside the scope of a single project. Teams should apply it to review their process and how well it worked. A team that doesn't continuously experiment with their process is not an agile team.

Vòng lặp này áp dụng được ra ngoài phạm vi của một dự án đơn lẻ. Các nhóm nên áp dụng nó để rà soát quy trình của mình và xem nó hoạt động tốt đến đâu. Một nhóm không liên tục thử nghiệm với quy trình của mình thì không phải là một nhóm linh hoạt.

And This Drives Design — Và điều này dẫn dắt thiết kế

In **Topic 8**, The Essence of Good Design we assert that the measure of design is how easy the result of that design is to change: a good design produces something that's easier to change than a bad design.

Trong Topic 8, Tinh túy của thiết kế tốt, chúng tôi khẳng định rằng thước đo của một thiết kế là kết quả của thiết kế đó dễ thay đổi đến mức nào: một thiết kế tốt tạo ra thứ dễ thay đổi hơn so với một thiết kế tồi.

And this discussion about agility explains why that's the case.

Và cuộc thảo luận về sự linh hoạt này giải thích vì sao lại như vậy.

You make a change, and discover you don't like it. Step 3 in our list says we have to be able to fix what we break. To make our feedback loop efficient, this fix has to be as painless as possible. If it isn't, we'll be tempted to shrug it off and leave it unfixed. We talk about this effect in Topic 3, **Software Entropy**. To make this whole agile thing work, we need to practice good design, because good design makes things easy to change. And if it's easy to change, we can adjust, at every level, without any hesitation.

Bạn thực hiện một thay đổi, rồi phát hiện ra mình không thích nó. Bước 3 trong danh sách của chúng tôi nói rằng ta phải có khả năng sửa lại những gì mình đã làm hỏng. Để vòng phản hồi của ta hiệu quả, việc sửa này phải càng nhẹ nhàng càng tốt. Nếu không, ta sẽ bị cám dỗ nhún vai cho qua và để nó nguyên trạng. Chúng tôi nói về hiệu ứng này trong Topic 3, Sự rối loạn phần mềm. Để toàn bộ chuyện agile này hoạt động, ta cần thực hành

thiết kế tốt, bởi thiết kế tốt làm cho mọi thứ dễ thay đổi. Và nếu nó dễ thay đổi, ta có thể điều chỉnh, ở mọi cấp độ, mà không chần chừ gì.

That is agility.

Đó chính là sự linh hoạt.

Related Sections Include — Các mục liên quan

Topic 27, Don't Outrun Your Headlights

Topic 27, Đừng vượt quá tầm đèn pha

Topic 40, **Refactoring**

Topic 40, Tái cấu trúc

Topic 50, Coconuts Don't Cut It

Topic 50, Quả dừa không xài được đâu

Challenges — Thử thách

The simple feedback loop isn't just for software. Think of other decisions you've made recently. Could any of them have been improved by thinking about how you might be able to undo them if things didn't take you in the direction you were going? Can you think of ways you can improve what you do by gathering and acting on feedback?

Vòng phản hồi đơn giản không chỉ dành cho phần mềm. Hãy nghĩ về những quyết định khác bạn đã đưa ra gần đây. Có quyết định nào trong số đó đã có thể tốt hơn nếu bạn nghĩ tới chuyện làm sao có thể hoàn tác chúng phòng khi mọi việc không đi theo hướng bạn đang đi không? Bạn có thể nghĩ ra những cách cải thiện việc mình làm bằng cách thu thập phản hồi và hành động dựa trên đó không?

Footnotes

Chú thích

[70] Does a week sound like a long time? It really isn't, particularly when you're looking at processes in which management and workers occupy different worlds. Management will give you one view of how things operate, but when you get down on the floor, you'll find a very different reality—one that will take time to assimilate.

[70] Một tuần nghe có vẻ là khoảng thời gian dài? Thật ra không hề, nhất là khi bạn đang nhìn vào những quy trình mà ở đó ban quản lý và người lao động sống trong hai thế giới khác nhau. Ban quản lý sẽ đưa cho bạn một góc nhìn về cách mọi thứ vận hành, nhưng khi xuống tận sàn làm việc, bạn sẽ thấy một thực tế rất khác — một thực tế cần thời gian để thấm thía.

[71] <https://www.wired.com/1999/01/eno/>

[71] <https://www.wired.com/1999/01/eno/>

[72] <https://www.psychologytoday.com/us/blog/your-brain-work/201209/stop-tryingsolve-problems>

[72] <https://www.psychologytoday.com/us/blog/your-brain-work/201209/stop-tryingsolve-problems>

[73] <https://agilemanifesto.org>

[73] <https://agilemanifesto.org>

[74] For more on just how bad that approach can be, see The Tyranny of Metrics [Mul18] .

[74] Để hiểu thêm cách tiếp cận đó có thể tệ đến mức nào, xem The Tyranny of Metrics [Mul18].

Copyright © 2020 Pearson Education, Inc.

Copyright © 2020 Pearson Education, Inc.

Pragmatic Projects — Dự án Thực dụng

As your project gets under way, we need to move away from issues of individual philosophy and coding to talk about larger, project-sized issues. We aren't going to go into specifics of project management, but we will talk about a handful of critical areas that can make or break any project.

Khi dự án của bạn đã khởi động, chúng ta cần rời khỏi những vấn đề về triết lý cá nhân và viết code để bàn tới những vấn đề lớn hơn, ở quy mô cả dự án. Chúng tôi sẽ không đi sâu vào chi tiết quản lý dự án, nhưng sẽ nói về một số ít lĩnh vực then chốt có thể làm nên hoặc phá hỏng bất kỳ dự án nào.

As soon as you have more than one person working on a project, you need to establish some ground rules and delegate parts of the project accordingly. In **Topic 49, Pragmatic Teams**, we'll show how to do this while honoring the Pragmatic philosophy.

Ngay khi bạn có hơn một người cùng làm việc trong một dự án, bạn cần thiết lập vài quy tắc nền tảng và phân chia các phần của dự án cho phù hợp. Ở Mục 49 (Topic 49), Nhóm Thực dụng (Pragmatic Teams), chúng tôi sẽ chỉ cho bạn cách làm điều đó trong khi vẫn tôn trọng triết lý Thực dụng.

The purpose of a software development method is to help people work together. Are you and your team doing what works well for you, or are you only investing in the trivial surface artifacts, and not getting the real benefits you deserve? We'll see why Topic 50, Coconuts Don't Cut It and offer the true secret to success.

Mục đích của một phương pháp phát triển phần mềm là giúp con người làm việc cùng nhau. Liệu bạn và nhóm của bạn có đang làm những điều thật sự hiệu quả với mình, hay bạn chỉ đang đầu tư vào những thứ hình thức hời hợt mà không gặt hái được lợi ích thực sự mà mình xứng đáng có? Chúng ta sẽ thấy tại sao ở Mục 50 (Topic 50), Dừa Thì Không Ăn Thua (Coconuts Don't Cut It), và đưa ra bí quyết thành công thật sự.

And of course none of that matters if you can't deliver software consistently and reliably. That's the basis of the magic trio of **version control**, testing, and automation: the Topic 51, Pragmatic **Starter Kit**.

Và dĩ nhiên, chẳng điều nào trong số đó có ý nghĩa nếu bạn không thể bàn giao phần mềm một cách nhất quán và đáng tin cậy. Đó là nền tảng của bộ ba kỳ diệu gồm quản lý phiên bản (version control), kiểm thử, và tự động hóa: Mục 51 (Topic 51), Bộ Khởi đầu Thực dụng (Pragmatic Starter Kit).

Ultimately, though, success is in the eye of the beholder—the sponsor of the project. The perception of success is what counts, and in Topic 52, Delight Your Users we'll show you how to delight every

project's sponsor.

Tuy nhiên, suy cho cùng, thành công nằm trong mắt người nhìn nhận nó — nhà tài trợ của dự án. Chính cảm nhận về thành công mới là điều quan trọng, và ở Mục 52 (Topic 52), Làm Người Dùng Mê Mẩn (Delight Your Users), chúng tôi sẽ chỉ cho bạn cách làm hài lòng nhà tài trợ của mọi dự án.

The last **tip** in the book is a direct consequence of all the rest. In Topic 53, *Pride and Prejudice*, we ask you to sign your work, and to take pride in what you do.

Mẹo cuối cùng trong cuốn sách là hệ quả trực tiếp của tất cả những điều còn lại. Ở Mục 53 (Topic 53), Tự hào và Định kiến (*Pride and Prejudice*), chúng tôi đề nghị bạn ký tên lên sản phẩm của mình, và tự hào về những gì mình làm.

At Group L, Stoffel oversees six first-rate programmers, a managerial challenge roughly comparable to herding cats. *The Washington Post Magazine*, June 9, 1985

Ở Group L, Stoffel giám sát sáu lập trình viên hạng nhất, một thử thách quản lý đại khái cũng ngang với việc lừa mèo vào bẫy. *The Washington Post Magazine*, ngày 9 tháng 6 năm 1985

Topic 49. Pragmatic Teams — Topic 49. Nhóm Thực dụng

Even in 1985, the joke about herding cats was getting old. By the time of the first edition at the turn of the century, it was positively ancient. Yet it persists, because it has a ring of truth to it. Programmers are a bit like cats: intelligent, strong willed, opinionated, independent, and often worshiped by the net.

Ngay cả vào năm 1985, câu đùa về chuyện lừa mèo đã bắt đầu cũ. Đến thời ấn bản đầu tiên ở khúc giao thế kỷ, nó đã thành cổ lỗ sĩ hần hoi. Vậy mà nó vẫn còn dai dẳng, bởi vì nó có một chút sự thật trong đó. Lập trình viên hơi giống mèo: thông minh, ý chí cứng cỏi, lảm chính kiến, độc lập, và thường được cả mạng tôn sùng.

So far in this book we've looked at pragmatic techniques that help an individual be a better programmer. Can these methods work for teams as well, even for teams of strong-willed, independent people? The answer is a resounding "yes!" There are advantages to being a pragmatic individual, but these advantages are multiplied manyfold if the individual is working on a **pragmatic team**.

Cho tới giờ trong cuốn sách này, chúng ta đã xem xét những kỹ thuật thực dụng giúp một cá nhân trở thành lập trình viên giỏi hơn. Liệu những phương pháp ấy có hiệu quả với cả nhóm hay không, kể cả với những nhóm gồm những con người ý chí cứng cỏi và độc lập? Câu trả lời là một tiếng "có!" vang dội. Làm một cá nhân thực dụng đã có lợi rồi, nhưng những lợi ích ấy còn được nhân lên gấp bội nếu cá nhân đó làm việc trong một nhóm thực dụng (pragmatic team).

A team, in our view, is a small, mostly stable entity of its own. Fifty people aren't a team, they're a horde. [75] Teams where members are constantly being pulled onto other assignments and no one knows each other aren't a team either, they are merely strangers temporarily sharing a bus stop in the rain.

Theo cách nhìn của chúng tôi, một nhóm là một thực thể nhỏ, tự thân và phần lớn thời gian giữ được sự ổn định. Năm mươi người không phải là một nhóm, họ là một đám ô hợp. [75] Những nhóm mà thành viên liên tục bị kéo sang việc khác và chẳng ai biết ai cũng

chẳng phải là nhóm, họ chỉ là những người xa lạ tạm trú chung một trạm xe buýt dưới cơn mưa.

A pragmatic team is small, under 10-12 or so members.

Một nhóm thực dụng thì nhỏ, chừng dưới 10–12 thành viên.

Members come and go rarely. Everyone knows everyone well, trusts each other, and depends on each other.

Thành viên ít khi ra vào. Mọi người hiểu rõ nhau, tin tưởng lẫn nhau, và dựa vào nhau.

Tip 84: Maintain Small, Stable Teams

Tip 84: Hãy duy trì những nhóm nhỏ và ổn định

In this section we'll look briefly at how pragmatic techniques can be applied to teams as a whole. These notes are only a start. Once you've got a group of pragmatic developers working in an enabling environment, they'll quickly develop and refine their own team dynamics that work for them.

Trong mục này, chúng ta sẽ xem qua một cách ngắn gọn việc áp dụng các kỹ thuật thực dụng cho cả nhóm như thế nào. Những ghi chú này chỉ là điểm khởi đầu. Một khi bạn đã có một nhóm các lập trình viên thực dụng làm việc trong một môi trường tạo điều kiện thuận lợi, họ sẽ nhanh chóng phát triển và mài giũa nên động lực nội nhóm của riêng mình, phù hợp với chính họ.

Let's recast some of the previous sections in terms of teams.

Hãy diễn giải lại một số mục trước đó dưới góc nhìn của nhóm.

No Broken Windows — Không Cửa sổ Vỡ

Quality is a team issue. The most diligent developer placed on a team that just doesn't care will find it difficult to maintain the enthusiasm needed to fix niggling problems. The problem is further exacerbated if the team actively discourages the developer from spending time on these fixes.

Chất lượng là vấn đề của cả nhóm. Lập trình viên cần mẫn nhất khi bị đặt vào một nhóm chẳng buồn quan tâm cũng sẽ thấy khó mà giữ được nhiệt huyết cần thiết để sửa những vấn đề lặt vặt. Vấn đề càng trầm trọng hơn nếu nhóm còn tích cực cản trở việc lập trình viên ấy dành thời gian cho những lần sửa chữa đó.

Teams as a whole should not tolerate **broken windows**—those small imperfections that no one fixes. The team must take responsibility for the quality of the product, supporting developers who understand the no broken windows philosophy we describe in Topic 3, **Software Entropy**, and encouraging those who haven't yet discovered it.

Cả nhóm không nên dung thứ những cửa sổ vỡ (broken windows) — những khuyết điểm nhỏ mà chẳng ai chịu sửa. Nhóm phải nhận trách nhiệm về chất lượng sản phẩm, ủng hộ những lập trình viên thấm nhuần triết lý không cửa sổ vỡ mà chúng tôi mô tả ở Mục 3 (Topic 3), Sự Rối loạn Phần mềm (Software Entropy), và động viên những ai chưa nhận ra điều đó.

Some team methodologies have a “quality officer”—someone to whom the team delegates the responsibility for the quality of the deliverable. This is clearly ridiculous: quality can come only from the individual contributions of all team members. Quality is built in, not bolted on.

Một số phương pháp luận đội nhóm có “cán bộ chất lượng” — người được nhóm giao phó trách nhiệm về chất lượng của sản phẩm bàn giao. Điều này rõ ràng là vô lý: chất lượng

chỉ có thể đến từ đóng góp của từng cá nhân trong mọi thành viên của nhóm. Chất lượng được dựng nên từ bên trong, chứ không phải lắp vào từ bên ngoài.

Boiled Frogs — Những Con Ếch Bị Luộc

Remember the apocryphal frog in the pan of water, back in Topic 4, **Stone Soup and Boiled Frogs** ? It doesn't notice the gradual change in its environment, and ends up cooked. The same can happen to individuals who aren't vigilant. It can be difficult to keep an eye on your overall environment in the heat of project development.

Còn nhớ con ếch huyền thoại trong nồi nước, ở Mục 4 (Topic 4), Súp Đá và Ếch Bị Luộc (Stone Soup and Boiled Frogs) chứ? Nó không nhận ra sự thay đổi từ từ của môi trường quanh mình, và rồi cuộc bị luộc chín. Điều tương tự cũng có thể xảy ra với những cá nhân thiếu cảnh giác. Quả là khó mà để mắt tới toàn cảnh môi trường của mình giữa lúc nóng bỏng của quá trình phát triển dự án.

It's even easier for teams as a whole to get boiled. People assume that someone else is handling an issue, or that the team leader must have OK'd a change that your user is requesting. Even the best-intentioned teams can be oblivious to significant changes in their projects.

Và còn dễ hơn nữa khi cả nhóm bị luộc chín. Người ta cho rằng ai đó khác đang lo việc này, hoặc rằng trưởng nhóm hẳn đã đồng ý với cái thay đổi mà người dùng của bạn đang yêu cầu. Ngay cả những nhóm có thiện chí nhất cũng có thể không nhận ra những thay đổi đáng kể trong dự án của mình.

Fight this. Encourage everyone to actively monitor the environment for changes. Stay awake and aware for increased scope, decreased time scales, additional features, new environments—anything that wasn't in the original understanding. Keep metrics on new **requirements**. [76] The team needn't reject changes out of hand—you simply need to be aware that they're happening. Otherwise, it'll be you in the hot water.

Hãy chống lại điều đó. Hãy khuyến khích mọi người chủ động theo dõi môi trường xem có gì thay đổi. Hãy luôn tỉnh táo và cảnh giác trước phạm vi phình ra, thời hạn co lại, tính năng phát sinh, môi trường mới — bất cứ thứ gì không nằm trong thỏa thuận ban đầu. Hãy giữ số liệu về các yêu cầu mới. [76] Nhóm không nhất thiết phải gạt phăng mọi thay đổi — bạn chỉ đơn giản là cần biết rằng chúng đang xảy ra. Nếu không, người ngồi trong nồi nước nóng sẽ là bạn.

Schedule Your Knowledge Portfolio — Lên Lịch cho Danh mục Tri thức của Bạn

In Topic 6, **Your Knowledge Portfolio** we looked at ways you should invest in your personal **Knowledge Portfolio** on your own time. Teams that want to succeed need to consider their knowledge and skill investments as well.

Ở Mục 6 (Topic 6), Danh mục Tri thức của Bạn (Your Knowledge Portfolio), chúng ta đã xem xét những cách bạn nên đầu tư vào danh mục tri thức (knowledge portfolio) cá nhân bằng thời gian của chính mình. Những nhóm muốn thành công cũng cần cân nhắc đầu tư vào tri thức và kỹ năng của họ.

If your team is serious about improvement and innovation, you need to schedule it. Trying to get things done “whenever there’s a free moment” means they will never happen. Whatever sort of

Nếu nhóm của bạn thực sự nghiêm túc về việc cải thiện và đổi mới, bạn cần lên lịch cho nó. Cứ cố làm những việc đó “mỗi khi rảnh được chút nào” thì có nghĩa là chúng sẽ chẳng bao giờ xảy ra. Dù bạn đang làm việc với kiểu

backlog or task list or flow you’re working with, don’t reserve it for only feature development. The team works on more than just new features. Some possible examples include:

backlog, danh sách công việc hay luồng công việc nào đi nữa, đừng dành nó riêng cho việc phát triển tính năng. Nhóm làm nhiều thứ chứ không chỉ những tính năng mới. Một vài ví dụ khả dĩ gồm:

Old Systems Maintenance While we love working on the shiny new system, there’s likely maintenance work that needs to be done on the old system. We’ve met teams who try and shove this work in the corner. If the team is charged with doing these tasks, then do them—for real.

Bảo trì Hệ thống Cũ Dù chúng ta yêu thích làm việc trên hệ thống mới bóng bẩy, vẫn rất có thể còn việc bảo trì cần làm trên hệ thống cũ. Chúng tôi từng gặp những nhóm cố nhét việc này vào một xó. Nếu nhóm được giao làm những việc đó, thì hãy làm chúng — làm cho ra hồn.

Process Reflection and Refinement Continuous improvement can only happen when you take the time to look around, figure out what’s working and not, and then make changes (see Topic 48, The Essence of **Agility**). Too many teams are so busy bailing out water that they don’t have time to fix the leak. Schedule it. Fix it.

Suy ngẫm và Tinh chỉnh Quy trình Cải tiến liên tục chỉ có thể xảy ra khi bạn dành thời gian nhìn quanh, xác định cái gì đang hiệu quả và cái gì không, rồi tạo ra thay đổi (xem Mục 48 (Topic 48), Cốt lõi của Sự Linh hoạt (The Essence of Agility)). Quá nhiều nhóm bận tát nước đến mức chẳng có thời gian bịt chỗ rò rỉ. Hãy lên lịch cho nó. Hãy sửa nó.

New tech experiments Don’t adopt new tech, frameworks, or libraries just because “everyone is doing it,” or based on something you saw at a conference or read online. Deliberately vet candidate technologies with prototypes. Put tasks on the schedule to try the new things and analyze results.

Thử nghiệm công nghệ mới Đừng áp dụng công nghệ, framework hay thư viện mới chỉ vì “ai cũng đang làm vậy,” hay dựa trên thứ gì đó bạn thấy ở một hội nghị hoặc đọc trên mạng. Hãy chủ động thẩm định các công nghệ ứng viên bằng những bản mẫu (prototype). Đưa lên lịch các công việc để thử những thứ mới và phân tích kết quả.

Learning and skill improvements Personal learning and improvements are a great start, but many skills are more effective when spread teamwide. Plan to do it, whether it’s the informal brown-bag lunch or more formal training sessions.

Học hỏi và nâng cao kỹ năng Việc học và cải thiện của cá nhân là một khởi đầu tuyệt vời, nhưng nhiều kỹ năng sẽ phát huy hiệu quả hơn khi được lan tỏa khắp nhóm. Hãy lên kế hoạch làm điều đó, dù là buổi “brown-bag lunch” thân mật hay những buổi đào tạo bài bản hơn.

Tip 85: Schedule It to Make It Happen

Tip 85: Lên lịch cho nó thì nó mới xảy ra

Communicate Team Presence — Truyền tải Sự Hiện diện của Nhóm

It’s obvious that developers in a team must talk to each other. We gave some suggestions to facilitate this in Topic 7, **Communicate!**. However, it’s easy to forget that the team itself has a presence within

the organization. The team as an entity needs to communicate clearly with the rest of the world.

Hiển nhiên là các lập trình viên trong một nhóm phải trò chuyện với nhau. Chúng tôi đã đưa ra vài gợi ý để hỗ trợ việc này ở Mục 7 (Topic 7), Hãy Giao tiếp! (Communicate!). Tuy nhiên, ta dễ quên rằng bản thân nhóm cũng có một sự hiện diện trong tổ chức. Nhóm với tư cách một thực thể cần giao tiếp một cách rõ ràng với phần còn lại của thế giới.

To outsiders, the worst project teams are those that appear sullen and reticent. They hold meetings with no structure, where no one wants to talk. Their emails and project documents are a mess: no two look the same, and each uses different terminology.

Trong mắt người ngoài, những nhóm dự án tệ nhất là những nhóm tỏ ra ủ ê và kiêu lời. Họ tổ chức những cuộc họp không có cấu trúc, nơi chẳng ai muốn nói gì. Email và tài liệu dự án của họ thì lộn xộn: không hai cái nào giống nhau, và mỗi cái dùng một thuật ngữ khác nhau.

Great project teams have a distinct personality. People look forward to meetings with them, because they know that they'll see a well-prepared performance that makes everyone feel good. The documentation they produce is crisp, accurate, and consistent. The team speaks with one voice. [77] They may even have a sense of humor.

Những nhóm dự án tuyệt vời có một cá tính rất riêng. Người ta mong chờ được họp với họ, bởi họ biết mình sẽ được xem một màn trình diễn chuẩn bị kỹ càng khiến ai nấy đều thấy dễ chịu. Tài liệu nhóm ấy làm ra thì gọn ghẽ, chính xác và nhất quán. Cả nhóm nói chung một giọng. [77] Thậm chí họ còn có thể có cả khiếu hài hước.

There is a simple marketing trick that helps teams communicate as one: generate a brand. When you start a project, come up with a name for it, ideally something off-the-wall. (In the past, we've named projects after things such as killer parrots that prey on sheep, optical illusions, gerbils, cartoon characters, and mythical cities.) Spend 30 minutes coming up with a zany logo, and use it. Use your team's name liberally when talking with people. It sounds silly, but it gives your team an identity to build on, and the world something memorable to associate with your work.

Có một mẹo tiếp thị đơn giản giúp các nhóm giao tiếp như một thể thống nhất: tạo ra một thương hiệu. Khi khởi động một dự án, hãy nghĩ ra một cái tên cho nó, lý tưởng nhất là cái gì đó ngộ nghĩnh. (Trước đây, chúng tôi từng đặt tên dự án theo những thứ như loài vẹt sát thủ chuyên săn cừu, ảo ảnh thị giác, chuột nhảy, nhân vật hoạt hình, và những thành phố thần thoại.) Dành ra 30 phút nghĩ ra một logo điên rồ, và dùng nó. Hãy dùng tên nhóm của bạn một cách thoải mái khi nói chuyện với người khác. Nghe có vẻ ngớ ngẩn, nhưng nó cho nhóm bạn một bản sắc để xây dựng nên, và cho thế giới một thứ đáng nhớ để gắn với công việc của bạn.

Don'T Repeat Yourself — Đừng Lặp Lại Chính Mình

In Topic 9, **DRY**—The Evils of **Duplication**, we talked about the difficulties of eliminating duplicated work between members of a team. This duplication leads to wasted effort, and can result in a maintenance nightmare. “Stovepipe” or “siloe” systems are common in these teams, with little sharing and a lot of duplicated functionality.

Ở Mục 9 (Topic 9), DRY — Cái Ác của Sự Trùng lặp (DRY—The Evils of Duplication), chúng ta đã bàn về những khó khăn trong việc loại bỏ công việc bị làm trùng lặp giữa các thành viên trong một nhóm. Sự trùng lặp này dẫn tới công sức lãng phí, và có thể gây ra một cơn ác mộng bảo trì. Những hệ thống kiểu “ống khói” (stovepipe) hay “silo” rất phổ biến ở các nhóm này, với rất ít sự chia sẻ và rất nhiều chức năng bị làm trùng.

Good communication is key to avoiding these problems. And by “good” we mean instant and frictionless .

Giao tiếp tốt là chìa khóa để tránh những vấn đề này. Và “tốt” ở đây nghĩa là tức thì và không ma sát.

You should be able to ask a question of team members and get a more-or-less instant reply. If the team is co-located, this might be as simple as poking your head over the cube wall or down the hall. For remote teams, you may have to rely on a messaging app or other electronic means.

Bạn phải có khả năng đặt câu hỏi cho các thành viên trong nhóm và nhận được câu trả lời gần như tức thì. Nếu cả nhóm ngồi cùng chỗ, việc này có thể đơn giản như thò đầu qua vách ngăn hay nói vọng xuống cuối hành lang. Với những nhóm làm việc từ xa, có lẽ bạn phải trông cậy vào một ứng dụng nhắn tin hay phương tiện điện tử khác.

If you have to wait a week for the team meeting to ask your question or share your status, that’s an awful lot of friction. [78] Frictionless means it’s easy and low-ceremony to ask questions, share your progress, your problems, your insights and learnings, and to stay aware of what your teammates are doing.

Nếu bạn phải chờ cả tuần đến buổi họp nhóm mới đặt được câu hỏi hay chia sẻ được tình hình của mình, thì đó là một mức ma sát khủng khiếp. [78] Không ma sát nghĩa là việc đặt câu hỏi, chia sẻ tiến độ, khó khăn, những phát hiện và bài học của bạn, cũng như việc nắm được đồng đội đang làm gì, đều dễ dàng và ít nghi thức.

Maintain awareness to stay DRY.

Hãy duy trì sự nắm bắt lẫn nhau để giữ DRY.

Team Tracer Bullets — Dạn Vạch Đường của Nhóm

A project team has to accomplish many different tasks in different areas of the project, touching a lot of different technologies. Understanding requirements, designing architecture, coding for frontend and server, testing, all have to happen. But it’s a common misconception that these activities and tasks can happen separately, in isolation. They can’t.

Một nhóm dự án phải hoàn thành nhiều công việc khác nhau ở những mảng khác nhau của dự án, đụng tới rất nhiều công nghệ khác nhau. Hiểu yêu cầu, thiết kế kiến trúc, viết code cho frontend lẫn cho phía máy chủ, kiểm thử — tất cả đều phải diễn ra. Nhưng có một quan niệm sai lầm phổ biến rằng những hoạt động và công việc này có thể diễn ra tách rời, biệt lập. Không thể như vậy được.

Some methodologies advocate all sort of different roles and titles within the team, or create separate specialized teams entirely. But the problem with that approach is that it introduces gates and handoffs . Now instead of a smooth flow from the team to deployment, you have artificial gates where the work stops. Handoffs that have to wait to be accepted. Approvals. Paperwork. The Lean folks call this waste , and strive to actively eliminate it.

Một số phương pháp luận cổ vũ đủ kiểu vai trò và chức danh khác nhau bên trong nhóm, hoặc tạo hẳn ra những nhóm chuyên trách riêng biệt. Nhưng vấn đề của cách tiếp cận đó là nó đưa vào những cánh cổng và những lần bàn giao. Giờ đây, thay vì một dòng chảy mượt mà từ nhóm đến lúc triển khai, bạn lại có những cánh cổng nhân tạo nơi công việc dừng lại. Những lần bàn giao phải chờ được phê duyệt. Những phê chuẩn. Những giấy tờ. Dân Lean gọi đây là lãng phí, và nỗ lực chủ động loại bỏ nó.

All of these different roles and activities are actually different views of the same problem, and artificially separating them can cause a boatload of trouble. For example, programmers who are two or three levels removed from the actual users of their code are unlikely to be aware of the **context** in which their work is used. They will not be able to make informed decisions.

Tất cả những vai trò và hoạt động khác nhau này thực ra chỉ là những góc nhìn khác nhau về cùng một bài toán, và việc tách chúng ra một cách giả tạo có thể gây ra cả mớ rắc rối. Chẳng hạn, những lập trình viên cách xa hai hay ba tầng so với người dùng thật sự của code mình viết thì khó lòng nhận thức được bối cảnh nơi sản phẩm của họ được dùng tới. Họ sẽ không thể đưa ra những quyết định có hiểu biết.

With Topic 12, **Tracer Bullets**, we recommend developing individual features, however small and limited initially, that go end-to-end through the entire system. That means that you need all the skills to do that within the team: frontend, UI/UX, server, DBA, QA, **etc.**, all comfortable and accustomed to working with each other. With a tracer bullet approach, you can implement very small bits of functionality very quickly, and get immediate feedback on how well your team communicates and delivers. That creates an environment where you can make changes and tune your team and process quickly and easily.

Với Mục 12 (Topic 12), Đạn Vạch Đường (Tracer Bullets), chúng tôi khuyên nên phát triển từng tính năng riêng lẻ — dù ban đầu nhỏ bé và hạn chế tới đâu — chạy xuyên suốt từ đầu đến cuối qua toàn bộ hệ thống. Điều đó nghĩa là bạn cần có đủ mọi kỹ năng để làm việc ấy ngay trong nhóm: frontend, UI/UX, phía máy chủ, DBA, QA, v.v., tất cả đều thoải mái và quen làm việc cùng nhau. Với cách tiếp cận đạn vạch đường (tracer bullets), bạn có thể hiện thực những mẫu chức năng rất nhỏ một cách rất nhanh, và nhận phản hồi tức thì về việc nhóm bạn giao tiếp và bàn giao tốt đến đâu. Điều đó tạo ra một môi trường nơi bạn có thể thay đổi và điều chỉnh nhóm cùng quy trình của mình một cách nhanh chóng và dễ dàng.

Tip 86: Organize Fully Functional Teams

Tip 86: Hãy tổ chức những nhóm đầy đủ chức năng

Build teams so you can build code end-to-end, incrementally and iteratively.

Hãy xây dựng các nhóm sao cho bạn có thể xây code từ đầu đến cuối, theo từng bước nhỏ và lặp đi lặp lại.

Automation — Tự động hóa

A great way to ensure both consistency and accuracy is to automate everything the team does. Why struggle with code formatting standards when your editor or IDE can do it for you automatically? Why do manual testing when the continuous build can run tests automatically? Why deploy by hand when automation can do it the same way every time, repeatably and reliably?

Một cách tuyệt vời để bảo đảm cả tính nhất quán lẫn tính chính xác là tự động hóa mọi thứ mà nhóm làm. Có gì phải vật lộn với các chuẩn định dạng code khi trình soạn thảo hay IDE của bạn có thể tự động làm việc đó cho bạn? Có gì phải kiểm thử thủ công khi quá trình build liên tục có thể chạy kiểm thử một cách tự động? Có gì phải triển khai bằng tay khi tự động hóa có thể làm y hệt như nhau mỗi lần, có thể lặp lại và đáng tin cậy?

Automation is an essential component of every project team. Make sure the team has skills at tool building to construct and deploy the tools that automate the project development and production deployment.

Tự động hóa là một thành phần thiết yếu của mọi nhóm dự án. Hãy bảo đảm rằng nhóm có những kỹ năng xây dựng công cụ để tạo ra và triển khai những công cụ tự động hóa quá trình phát triển dự án lẫn việc triển khai lên môi trường sản xuất.

Know When To Stop Adding Paint — Biết Khi nào Nên Dừng Quét Thêm Sơn

Remember that teams are made up of individuals. Give each member the ability to shine in their own way. Give them just enough structure to support them and to ensure that the project delivers value. Then, like the painter in Topic 5, **Good-Enough Software**, resist the temptation to add more paint.

Hãy nhớ rằng các nhóm được tạo nên từ những cá nhân. Hãy cho mỗi thành viên khả năng tỏa sáng theo cách riêng của họ. Hãy cho họ vừa đủ cấu trúc để nâng đỡ họ và để bảo đảm dự án mang lại giá trị. Rồi thì, giống như người họa sĩ ở Mục 5 (Topic 5), Phần mềm Đủ Tốt (Good-Enough Software), hãy cưỡng lại cám dỗ quét thêm sơn.

Related Sections Include — Các mục liên quan

Topic 2, **The Cat Ate My Source Code**

Mục 2 (Topic 2), Con Mèo Đã Ăn Mất Mã Nguồn Của Tôi

Topic 7, **Communicate!**

Mục 7 (Topic 7), Hãy Giao tiếp!

Topic 12, **Tracer Bullets**

Mục 12 (Topic 12), Đạn Vạch Đường

Topic 19, **Version Control**

Mục 19 (Topic 19), Quản lý Phiên bản

Topic 50, **Coconuts Don't Cut It**

Mục 50 (Topic 50), Dừa Thì Không Ăn Thua

Topic 51, **Pragmatic Starter Kit**

Mục 51 (Topic 51), Bộ Khởi đầu Thực dụng

Challenges — Thử thách

Look around for successful teams outside the area of software development. What makes them successful? Do they use any of the processes discussed in this section?

Hãy nhìn quanh để tìm những nhóm thành công bên ngoài lĩnh vực phát triển phần mềm. Điều gì làm họ thành công? Họ có dùng quy trình nào trong số những quy trình bàn ở mục này không?

Next time you start a project, try convincing people to brand it. Give your organization time to become used to the idea, and then do a quick audit to see what difference it made, both within the team and externally.

Lần tới khi bạn khởi động một dự án, hãy thử thuyết phục mọi người tạo thương hiệu cho nó. Hãy cho tổ chức của bạn thời gian để làm quen với ý tưởng đó, rồi thực hiện một cuộc kiểm tra nhanh để xem nó tạo ra khác biệt gì, cả bên trong nhóm lẫn ra bên ngoài.

You were probably once given problems such as “If it takes 4 workers 6 hours to dig a ditch, how long would it take 8 workers?” In real life, however, what factors affect the answer if the workers were writing code instead? In how many scenarios is the time actually reduced?

Có lẽ bạn từng được giao những bài toán kiểu như “Nếu 4 công nhân mất 6 giờ để đào một con mương, thì 8 công nhân sẽ mất bao lâu?” Tuy nhiên, trong đời thực, những yếu tố nào ảnh hưởng đến đáp án nếu các công nhân đó đang viết code thay vì đào mương? Trong bao nhiêu kịch bản thì thời gian thực sự được rút ngắn?

Read *The Mythical Man Month* [Bro96] by Frederick Brooks. For extra credit, buy two copies so you can read it twice as fast.

Hãy đọc *The Mythical Man Month* [Bro96] của Frederick Brooks. Để được điểm thưởng, hãy mua hai bản để đọc nhanh gấp đôi.

Topic 50. Coconuts Don’t Cut It — Topic 50. Dừa Thì Không Ăn Thua

The native islanders had never seen an airplane before, or met people such as these strangers. In return for use of their land, the strangers provided mechanical birds that flew in and out all day long on a “runway,” bringing incredible material wealth to their island home. The strangers mentioned something about war and fighting. One day it was over and they all left, taking their strange riches with them.

Những người dân bản địa trên đảo trước nay chưa từng thấy máy bay, cũng chưa từng gặp ai như đám người lạ này. Đổi lấy quyền sử dụng đất của họ, đám người lạ mang đến những con chim cơ khí ngày ngày bay ra bay vào trên một “đường băng”, đem theo của cải vật chất khó tin về với hòn đảo quê hương họ. Đám người lạ có nhắc đến gì đó về chiến tranh và đánh nhau. Rồi một ngày họ kết thúc mọi chuyện kết thúc, tất cả bọn họ ra đi, mang theo cả những thứ của cải kỳ lạ kia.

The islanders were desperate to restore their good fortunes, and re-built a facsimile of the airport, control tower, and equipment using local materials: vines, coconut shells, palm fronds, and such. But for some reason, even though they had everything in place, the planes didn’t come. They had imitated the form, but not the content. Anthropologists call this a **cargo cult**.

Người dân trên đảo tha thiết muốn khôi phục lại vận may của mình, nên đã dựng lại một bản sao của sân bay, tháp điều khiển và đủ thứ thiết bị bằng vật liệu địa phương: dây leo, gáo dừa, lá cọ, và những thứ tương tự. Nhưng chẳng hiểu vì sao, dù đã bày biện đâu vào đấy, máy bay vẫn không tới. Họ đã bắt chước được cái vỏ, nhưng không bắt chước được cái ruột. Các nhà nhân học gọi đây là sự sùng bái hàng hóa (cargo cult).

All too often, we are the islanders.

Quá thường xuyên, chính chúng ta là những người dân trên đảo ấy.

It’s easy and tempting to fall into the cargo cult trap: by investing in and building up the easily-visible artifacts, you hope to attract the underlying, working magic. But as with the original cargo cults of Melanesia, [79] a fake airport made out of coconut shells is no substitute for the real thing.

Sa vào cái bẫy sùng bái hàng hóa (cargo cult) thì vừa dễ vừa đầy cám dỗ: bằng cách đầu tư vào và dựng lên những thứ tạo tác dễ thấy, bạn hy vọng sẽ hút được thứ ma thuật thật sự đang vận hành bên dưới. Nhưng cũng như với những cargo cult nguyên bản ở Melanesia, [79] một sân bay giả làm bằng gáo dừa chẳng thể nào thay thế được đồ thật.

For example, we have personally seen teams that claim to be using Scrum. But, upon closer examination, it turned out they were doing a daily stand up meeting once a week, with fourweek iterations that often turned into sixor eight-week

Lấy ví dụ, đích thân chúng tôi đã chứng kiến những nhóm tuyên bố rằng họ đang dùng Scrum. Nhưng khi xem xét kỹ hơn, hóa ra họ làm cuộc họp stand-up hằng ngày một tuần một lần, với những vòng lặp bốn tuần mà thường kéo dài thành sáu

iterations. They felt that this was okay because they were using a popular “**agile**” scheduling tool. They were only investing in the superficial artifacts—and even then, often in name only, as if “stand up” or “iteration” were some sort of incantation for the superstitious. Unsurprisingly, they, too, failed to attract the real magic.

hay tám tuần. Họ thấy thế là ổn vì họ đang dùng một công cụ lập lịch “agile” đang thịnh hành. Họ chỉ đầu tư vào những thứ bề ngoài hời hợt—mà ngay cả thế, nhiều khi cũng chỉ có cái tên, cứ như “stand-up” hay “iteration” là một thứ thần chú nào đó cho đám người mê tín. Chẳng có gì ngạc nhiên khi họ cũng thất bại trong việc thu hút thứ ma thuật thật sự.

Context Matters — Bối Cảnh Mới Là Điều Quan Trọng

Have you or your team fallen in this trap? Ask yourself, why are you even using that particular development method? Or that framework? Or that testing technique? Is it actually well-suited for the job at hand? Does it work well for you? Or was it adopted just because it was being used by the latest internet-fueled success story?

Bạn hay nhóm của bạn đã từng sa vào cái bẫy này chưa? Hãy tự hỏi mình, rốt cuộc tại sao bạn lại dùng đúng cái phương pháp phát triển đó? Hay cái framework đó? Hay cái kỹ thuật kiểm thử đó? Liệu nó có thật sự phù hợp với công việc trước mắt không? Nó có thật sự hiệu quả với bạn không? Hay nó được chọn chỉ vì nó đang được dùng bởi câu chuyện thành công mới nhất mà internet thổi phồng lên?

There’s a current trend to adopt the policies and processes of successful companies such as Spotify, Netflix, Stripe, GitLab, and others. Each have their own unique take on software development and management. But consider the **context**: are you in the same market, with the same constraints and opportunities, similar expertise and organization size, similar management, and similar culture? Similar user base and **requirements**?

Hiện đang có một trào lưu áp dụng chính sách và quy trình của những công ty thành công như Spotify, Netflix, Stripe, GitLab, và nhiều công ty khác. Mỗi công ty có cách tiếp cận riêng độc đáo của họ về phát triển và quản lý phần mềm. Nhưng hãy nghĩ về bối cảnh: bạn có ở trong cùng một thị trường, với cùng những ràng buộc và cơ hội, chuyên môn và quy mô tổ chức tương tự, cách quản lý tương tự, và văn hóa tương tự không? Cùng tập người dùng và yêu cầu tương tự không?

Don’t fall for it. Particular artifacts, superficial structures, policies, processes, and methods are not enough.

Đừng mắc bẫy. Những thứ tạo tác cụ thể, những cấu trúc bề ngoài, chính sách, quy trình và phương pháp—chừng đó là không đủ.

Tip 87: Do What Works, Not What’s Fashionable

Tip 87: Hãy Làm Thứ Có Hiệu Quả, Không Phải Thứ Đang Là Mốt

How do you know “what works”? You rely on that most fundamental of **Pragmatic** techniques:

Làm sao bạn biết “cái gì hiệu quả”? Bạn dựa vào kỹ thuật căn bản nhất của người Thực dụng:

Try it.

Thử nó đi.

Pilot the idea with a small team or set of teams. Keep the good bits that seem to work well, and discard anything else as waste or overhead. No one will downgrade your organization because it operates differently from Spotify or Netflix, because even they didn't follow their current processes while they were growing. And years from now, as those companies mature and pivot and continue to thrive, they'll be doing something different yet again.

Hãy thử nghiệm ý tưởng đó với một nhóm nhỏ hoặc một vài nhóm. Giữ lại những phần hay có vẻ chạy tốt, và vứt bỏ mọi thứ còn lại như rác rưởi hay gánh nặng dư thừa. Sẽ chẳng ai đánh giá thấp tổ chức của bạn vì nó vận hành khác Spotify hay Netflix đâu, bởi ngay cả những công ty đó cũng đâu có theo các quy trình hiện tại của họ hồi còn đang lớn lên. Và nhiều năm nữa, khi những công ty đó trưởng thành, xoay trục và tiếp tục phát đạt, họ lại sẽ làm một cái gì đó khác nữa.

That's the actual secret to their success.

Đó mới chính là bí quyết thật sự cho thành công của họ.

One Size Fits No One Well — Một Cỡ Chẳng Vừa Với Ai Cả

The purpose of a software development methodology is to help people work together. As we discuss in **Topic 48, The Essence of Agility**, there is no single plan you can follow when you develop software, especially not a plan that someone else came up with at another company.

Mục đích của một phương pháp luận phát triển phần mềm là giúp con người làm việc cùng nhau. Như chúng tôi bàn ở Topic 48, The Essence of Agility, không hề có một kế hoạch duy nhất nào để bạn có thể tuân theo khi phát triển phần mềm, đặc biệt không phải là cái kế hoạch mà ai đó ở một công ty khác nghĩ ra.

Many certification programs are actually even worse than that: they are predicated on the student being able to memorize and follow the rules. But that's not what you want. You need the ability to see beyond the existing rules and exploit possibilities for advantage. That's a very different mindset from “but Scrum/Lean/Kanban/XP/agile does it this way...” and so on.

Nhiều chương trình cấp chứng chỉ thực ra còn tệ hơn thế: chúng dựa trên việc người học có thể ghi nhớ và tuân theo các quy tắc. Nhưng đó không phải thứ bạn muốn. Bạn cần khả năng nhìn xa hơn những quy tắc hiện có và khai thác những cơ hội để giành lợi thế. Đó là một lối tư duy rất khác với kiểu “nhưng Scrum/Lean/Kanban/XP/agile làm theo cách này cơ mà...” và đại loại thế.

Instead, you want to take the best pieces from any particular methodology and adapt them for use. No one size fits all, and current methods are far from complete, so you'll need to look at more than just one popular method.

Thay vào đó, bạn muốn lấy những phần hay nhất từ bất kỳ phương pháp luận cụ thể nào và điều chỉnh chúng cho phù hợp để dùng. Không có cỡ nào vừa cho tất cả, và các phương pháp hiện thời còn lâu mới hoàn chỉnh, nên bạn sẽ cần nhìn vào nhiều hơn chỉ một phương pháp đang thịnh hành.

For example, Scrum defines some project management

Lấy ví dụ, Scrum định nghĩa một số thực hành quản lý dự án, practices, but Scrum by itself doesn't provide enough guidance at the technical level for teams or at the portfolio/governance level for leadership. So where do you start?

nhưng bản thân Scrum không cung cấp đủ chỉ dẫn ở tầng kỹ thuật cho các nhóm, cũng như ở tầng danh mục dự án/quản trị cho ban lãnh đạo. Vậy bạn bắt đầu từ đâu?

Be Like Them!

Hãy Giống Như Họ!

We frequently hear software development leaders tell their staff, “We should operate like Netflix” (or one of these other leading companies). Of course you could do that.

Chúng tôi thường nghe các lãnh đạo phát triển phần mềm bảo nhân viên của mình, “Chúng ta nên vận hành như Netflix” (hoặc một trong những công ty hàng đầu khác này). Tất nhiên là bạn có thể làm thế.

First, get yourself a few hundred thousand servers and tens of millions of users...

Trước tiên, hãy sắm cho mình vài trăm nghìn máy chủ và vài chục triệu người dùng đã...

The Real Goal — Mục Tiêu Thật Sự

The goal of course isn't to “do Scrum,” “do agile,” “do Lean,” or what-have-you. The goal is to be in a position to deliver working software that gives the users some new capability at a moment's notice. Not weeks, months, or years from now, but now . For many teams and organizations, continuous delivery feels like a lofty, unattainable goal, especially if you're saddled with a **process** that restricts delivery to months, or even weeks. But as with any goal, the key is to keep aiming in the right direction.

Mục tiêu tất nhiên không phải là “làm Scrum”, “làm agile”, “làm Lean”, hay đại loại thế. Mục tiêu là ở vào vị thế có thể giao phần mềm chạy được—đem lại cho người dùng một năng lực mới nào đó—ngay lập tức bất cứ lúc nào. Không phải vài tuần, vài tháng, hay vài năm nữa, mà là ngay bây giờ. Với nhiều nhóm và tổ chức, giao hàng liên tục cảm giác như một mục tiêu xa vời, không thể với tới, nhất là khi bạn đang gánh trên lưng một quy trình giới hạn việc giao hàng xuống còn vài tháng, hay thậm chí vài tuần một lần. Nhưng cũng như với mọi mục tiêu, mấu chốt là cứ tiếp tục nhắm đúng hướng.

images/delivery_times.png

images/delivery_times.png

If you're delivering in years, try and shorten the cycle to months. From months, cut it down to weeks. From a four-week sprint, try two. From a two week sprint, try one. Then daily. Then, finally, on demand. Note that being able to deliver on demand does not mean you are forced to deliver every minute of every day. You deliver when the users need it, when it makes business sense to do so.

Nếu bạn đang giao hàng theo năm, hãy thử rút ngắn chu kỳ xuống còn tháng. Từ tháng, cắt xuống còn tuần. Từ một sprint bốn tuần, thử hai. Từ một sprint hai tuần, thử một. Rồi hằng ngày. Rồi, cuối cùng, theo yêu cầu. Lưu ý rằng việc có thể giao hàng theo yêu cầu không có nghĩa là bạn bị buộc phải giao hàng mỗi phút của mỗi ngày. Bạn giao hàng khi người dùng cần nó, khi điều đó hợp lý về mặt kinh doanh.

Tip 88: Deliver When Users Need It

Tip 88: Hãy Giao Hàng Khi Người Dùng Cần

In order to move to this style of continuous development, you need a rock-solid infrastructure, which we discuss in the next topic, Topic 51, Pragmatic **Starter Kit**. You do development in the main trunk of your **version control** system, not in branches, and use techniques such as feature switches to roll out test features to users selectively.

Để chuyển sang phong cách phát triển liên tục này, bạn cần một hạ tầng vững như bàn thạch, thứ mà chúng tôi sẽ bàn ở mục tiếp theo, Topic 51, Pragmatic Starter Kit. Bạn làm việc phát triển trên nhánh chính (main trunk) của hệ thống quản lý phiên bản, chứ không phải trên các nhánh, và dùng những kỹ thuật như feature switch để tung ra các tính năng thử nghiệm cho người dùng một cách có chọn lọc.

Once your infrastructure is in order, you need to decide how to

Một khi hạ tầng của bạn đã đầu vào đấy, bạn cần quyết định cách

organize the work. Beginners might want to start with Scrum for project management, plus the technical practices from **eXtreme Programming** (XP). More disciplined and experienced teams might look to Kanban and Lean techniques, both for the team and perhaps for larger governance issues.

tổ chức công việc. Người mới có thể muốn bắt đầu với Scrum để quản lý dự án, cộng thêm các thực hành kỹ thuật từ eXtreme Programming (XP). Những nhóm kỷ luật hơn và dày dạn hơn có thể nhắm tới các kỹ thuật Kanban và Lean, cả cho nhóm lẫn có lẽ cho cả những vấn đề quản trị ở quy mô lớn hơn.

But don't take our word for it, investigate and try these approaches for yourself. Be careful, though, in overdoing it. Overly investing in any particular methodology can leave you blind to alternatives. You get used to it. Soon it becomes hard to see any other way. You've become calcified, and now you can't adapt quickly anymore.

Nhưng đừng tin lời chúng tôi, hãy tự mình tìm hiểu và thử những cách tiếp cận này. Tuy nhiên, hãy cẩn thận, đừng làm quá tay. Đầu tư quá mức vào bất kỳ phương pháp luận cụ thể nào cũng có thể khiến bạn mù quáng trước các lựa chọn khác. Bạn quen với nó. Chẳng mấy chốc khó mà thấy được bất kỳ con đường nào khác. Bạn đã trở nên xơ cứng, và giờ bạn không còn thích nghi nhanh được nữa.

Might as well be using coconuts.

Thế thì cũng chẳng khác gì đang dùng dừa.

Related Sections Include — Các mục liên quan

Topic 12, **Tracer Bullets**

Topic 12, Tracer Bullets

Topic 27, Don't Outrun Your Headlights

Topic 27, Don't Outrun Your Headlights

Topic 48, The Essence of Agility

Topic 48, The Essence of Agility

Topic 49, Pragmatic Teams

Topic 49, Pragmatic Teams

Topic 51, Pragmatic Starter Kit

Topic 51, Pragmatic Starter Kit

Civilization advances by extending the number of important operations we can perform without thinking. Alfred North Whitehead

Nền văn minh tiến lên bằng cách mở rộng số lượng những thao tác quan trọng mà ta có thể thực hiện mà không cần phải suy nghĩ. Alfred North Whitehead

Topic 51. Pragmatic Starter Kit — Topic 51. Bộ Khởi Đầu Thực Dụng

Back when cars were a novelty, the instructions for starting a Model-T Ford were more than two pages long. With modern cars, you just push a button—the starting procedure is automatic and foolproof. A person following a list of instructions might flood the engine, but the automatic starter won't.

Hồi ô tô còn là thứ mới lạ, bản hướng dẫn khởi động một chiếc Ford Model-T dài hơn hai trang giấy. Với xe hiện đại, bạn chỉ cần ấn một nút — quy trình khởi động là tự động và không thể làm sai. Người làm theo một danh sách hướng dẫn có thể làm ngập động cơ, nhưng bộ khởi động tự động thì không.

Although software development is still an industry at the Model-T stage, we can't afford to go through two pages of instructions again and again for some common operation. Whether it is the build and release procedure, testing, project paperwork, or any other recurring task on the project, it has to be automatic and repeatable on any capable machine.

Dù phát triển phần mềm vẫn còn là một ngành công nghiệp ở giai đoạn Model-T, chúng ta không thể cứ lặp đi lặp lại hai trang hướng dẫn cho một thao tác thường gặp nào đó. Dù đó là quy trình build và phát hành (release), kiểm thử, giấy tờ dự án, hay bất kỳ tác vụ lặp lại nào trên dự án, thì nó phải tự động và lặp lại được trên mọi máy đủ năng lực.

In addition, we want to ensure consistency and repeatability on the project. Manual procedures leave consistency up to chance; repeatability isn't guaranteed, especially if aspects of the procedure are open to interpretation by different people.

Thêm nữa, chúng ta muốn đảm bảo tính nhất quán và khả năng lặp lại trên dự án. Quy trình thủ công phó mặc sự nhất quán cho may rủi; khả năng lặp lại không được đảm bảo, nhất là khi có những khía cạnh của quy trình bị bỏ ngỏ cho mỗi người diễn giải một kiểu.

After we wrote the first edition of **The Pragmatic Programmer**, we wanted to create more books to help teams develop software. We figured we should start at the beginning: what are the most

Sau khi viết xong ấn bản đầu của **The Pragmatic Programmer**, chúng tôi muốn viết thêm nhiều cuốn nữa để giúp các nhóm phát triển phần mềm. Chúng tôi nghĩ nên bắt đầu từ đầu: đâu là những thành phần cơ

basic, most important elements that every team needs regardless of methodology, language, or technology stack. And so the idea of the **Pragmatic Starter Kit** was born, covering these three critical and interrelated topics:

bản nhất, quan trọng nhất mà mọi nhóm đều cần, bất kể phương pháp luận, ngôn ngữ hay ngăn xếp công nghệ. Và thế là ý tưởng về Bộ Khởi Đầu Thực Dụng (Pragmatic Starter Kit) ra đời, bao quát ba chủ đề mấu chốt và gắn bó với nhau này:

Version Control

Quản lý phiên bản (Version Control)

Regression Testing

Kiểm thử hồi quy (Regression Testing)

Full Automation

Tự động hóa toàn diện (Full Automation)

These are the three legs that support every project. Here's how.

Đây là ba chân để chống đỡ cho mọi dự án. Sau đây là cách chúng làm điều đó.

Drive With Version Control — Lái Dự Án Bằng Quản Lý Phiên Bản

As we said in **Topic 19**, Version Control, you want to keep everything needed to build your project under version control. That idea becomes even more important in the **context** of the project itself.

Như chúng tôi đã nói ở Topic 19, Quản lý phiên bản, bạn muốn giữ mọi thứ cần để build dự án dưới sự quản lý phiên bản (version control). Ý tưởng đó còn trở nên quan trọng hơn nữa trong bối cảnh của chính dự án.

First, it allows build machines to be **ephemeral**. Instead of one hallowed, creaky machine in the corner of the office that everyone is afraid to touch, [80] build machines and/or clusters are created on demand as spot instances in the cloud. Deployment **configuration** is under version control as well, so releasing to production can be handled automatically.

Thứ nhất, nó cho phép các máy build trở nên phù du (ephemeral). Thay vì một cỗ máy thiêng liêng, cột kẹt nằm trong góc văn phòng mà ai cũng sợ đụng vào, [80] các máy build và/hoặc cụm máy build được tạo ra theo nhu cầu dưới dạng spot instance trên cloud. Cấu hình triển khai cũng nằm dưới quản lý phiên bản, nên việc phát hành lên production có thể được xử lý tự động.

And that's the important part: at the project level, version control drives the build and release **process**.

Và đó mới là phần quan trọng: ở cấp độ dự án, quản lý phiên bản chính là thứ điều khiển quy trình build và phát hành.

Use Version Control to Drive Builds, Tests, and — Dùng Quản Lý Phiên Bản Để Điều Khiển Build, Test, Và

Tip 89: Releases

Tip 89: Phát Hành (Release)

That is, build, test, and deployment are triggered via commits or

Tức là, build, test và triển khai được kích hoạt qua các lần commit hoặc

pushes to version control, and built in a container in the cloud. Release to staging or production is specified by using a tag in your version control system. Releases then become a much more low-ceremony part of every day life—true continuous delivery, not tied to any one build machine or developer's machine.

push lên hệ quản lý phiên bản, và được build trong một container trên cloud. Việc phát hành lên môi trường staging hay production được chỉ định bằng cách dùng một tag trong

hệ quản lý phiên bản của bạn. Khi đó các bản phát hành trở thành một phần ít nghi thức hơn hẳn của cuộc sống thường ngày — đúng nghĩa giao hàng liên tục (continuous delivery), không bị trói buộc vào bất kỳ máy build hay máy của lập trình viên nào.

Ruthless And Continuous Testing — Kiểm Thử Tàn Nhẫn Và Liên Tục

Many developers test gently, subconsciously knowing where the code will break and avoiding the weak spots. Pragmatic Programmers are different. We are driven to find our bugs now, so we don't have to endure the shame of others finding our bugs later.

Nhiều lập trình viên kiểm thử nhẹ tay, trong tiềm thức biết chỗ nào code sẽ vỡ và tránh né những điểm yếu đó. Lập trình viên Thực dụng (Pragmatic Programmer) thì khác. Chúng tôi bị thôi thúc phải tìm ra lỗi của mình ngay bây giờ, để khỏi phải chịu nỗi nhục bị người khác tìm ra lỗi của mình về sau.

Finding bugs is somewhat like fishing with a net. We use fine, small nets (unit tests) to catch the minnows, and big, coarse nets (integration tests) to catch the killer sharks. Sometimes the fish manage to escape, so we patch any holes that we find, in hopes of catching more and more slippery defects that are swimming about in our project pool.

Tìm lỗi có phần giống như đánh cá bằng lưới. Chúng tôi dùng lưới mịn, mắt nhỏ (unit test) để bắt cá con, và lưới to, mắt thô (integration test) để bắt mấy con cá mập sát thủ. Đôi khi cá vẫn xoay sở thoát ra được, nên chúng tôi vá lại bất kỳ lỗ thủng nào tìm thấy, với hy vọng tóm được ngày càng nhiều những con lỗi trơn tuột đang bơi lội trong hồ dự án của mình.

Tip 90: Test Early, Test Often, Test Automatically

Tip 90: Hãy Kiểm Thử Sớm, Kiểm Thử Thường Xuyên, Kiểm Thử Tự Động

We want to start testing as soon as we have code. Those tiny minnows have a nasty habit of becoming giant, man-eating sharks pretty fast, and catching a shark is quite a bit harder. So we write unit tests. A lot of unit tests.

Chúng tôi muốn bắt đầu kiểm thử ngay khi vừa có code. Mấy con cá con tí hon ấy có cái thói rất khó chịu là chỉ một thoáng đã hóa thành cá mập khổng lồ ăn thịt người, mà bắt một con cá mập thì khó hơn nhiều. Vậy nên chúng tôi viết unit test. Rất nhiều unit test.

In fact, a good project may well have more test code than production code. The time it takes to produce this test code is worth the effort. It ends up being much cheaper in the long run, and you actually stand a chance of producing a product with

Thực ra, một dự án tốt hoàn toàn có thể có nhiều code kiểm thử hơn cả code production. Thời gian bỏ ra để viết đồng code kiểm thử này là xứng đáng. Rốt cuộc về lâu dài nó rẻ hơn nhiều, và bạn thực sự có cơ may làm ra một sản phẩm với

close to zero defects.

số lỗi gần như bằng không.

Additionally, knowing that you've passed the test gives you a high degree of confidence that a piece of code is "done."

Thêm vào đó, biết rằng mình đã vượt qua bài kiểm thử cho bạn một mức độ tự tin cao rằng một đoạn code đã "xong".

Tip 91: Coding Ain't Done 'Til All the Tests Run

Tip 91: Code Chưa Xong Cho Đến Khi Mọi Bài Test Chạy Qua

The automatic build runs all available tests. It's important to aim to “test for real,” in other words, the test environment should match the production environment closely. Any gaps are where bugs breed.

Bản build tự động chạy hết mọi bài test có sẵn. Điều quan trọng là phải nhắm tới việc “test cho ra trò”, nói cách khác, môi trường test phải khớp sát với môi trường production. Mọi khe hở chính là nơi lỗi sinh sôi.

The build may cover several major types of software testing: unit testing; integration testing; validation and verification; and performance testing.

Bản build có thể bao quát vài loại kiểm thử phần mềm chính: unit testing; integration testing; thẩm định và kiểm chứng (validation and verification); và kiểm thử hiệu năng (performance testing).

This list is by no means complete, and some specialized projects will require various other types of testing as well. But it gives us a good starting point.

Danh sách này hoàn toàn chưa đầy đủ, và một số dự án chuyên biệt sẽ còn đòi hỏi nhiều loại kiểm thử khác nữa. Nhưng nó cho chúng ta một điểm khởi đầu tốt.

Unit Testing A unit test is code that **exercises** a module. We covered this in Topic 41, **Test to Code**. Unit testing is the foundation of all the other forms of testing that we'll discuss in this section. If the parts don't work by themselves, they probably won't work well together. All of the modules you are using must pass their own unit tests before you can proceed.

Unit Testing Một unit test là đoạn code vận hành một mô-đun. Chúng tôi đã bàn về điều này ở Topic 41, Kiểm thử trước. Unit testing là nền móng cho mọi hình thức kiểm thử khác mà chúng ta sẽ thảo luận trong mục này. Nếu từng bộ phận tự nó đã không chạy, thì hợp lại với nhau chắc cũng chẳng chạy tốt. Mọi mô-đun bạn đang dùng đều phải vượt qua unit test của riêng nó trước khi bạn có thể tiến tiếp.

Once all of the pertinent modules have passed their individual tests, you're ready for the next stage. You need to test how all the modules use and interact with each other throughout the system.

Một khi tất cả các mô-đun liên quan đã vượt qua bài kiểm thử riêng của chúng, bạn đã sẵn sàng cho giai đoạn kế tiếp. Bạn cần kiểm thử cách tất cả các mô-đun sử dụng và tương tác với nhau xuyên suốt cả hệ thống.

Integration Testing Integration testing shows that the major subsystems that make up the project work and play well with each other. With good contracts in place and well tested, any integration issues can be detected easily. Otherwise, integration becomes a fertile breeding ground for bugs. In fact, it is often the single largest source of bugs in the system.

Integration Testing Kiểm thử tích hợp cho thấy rằng các hệ con (subsystem) chính cấu thành dự án vận hành và hợp tác ăn ý với nhau. Khi đã có những hợp đồng (contract) tốt được đặt đúng chỗ và kiểm thử kỹ, mọi vấn đề tích hợp đều có thể được phát hiện dễ dàng. Bằng không, tích hợp trở thành mảnh đất màu mỡ cho lỗi sinh sôi. Thực tế, nó thường là nguồn sản sinh lỗi đơn lẻ lớn nhất trong hệ thống.

Integration testing is really just an extension of the unit testing we've described—you're just testing how entire subsystems honor their contracts.

Integration testing thực ra chỉ là phần mở rộng của unit testing mà chúng tôi đã mô tả — bạn chỉ đang kiểm thử xem các hệ con trọn vẹn tôn trọng hợp đồng của chúng ra sao.

Validation and Verification As soon as you have an executable user interface or **prototype**, you need to answer an all-important question: the users told you what they wanted, but is it what they need?

Validation and Verification Ngay khi bạn có một giao diện người dùng hay bản mẫu (prototype) chạy được, bạn cần trả lời một câu hỏi tối quan trọng: người dùng đã nói cho bạn biết họ muốn gì, nhưng đó có phải là thứ họ cần không?

Does it meet the functional **requirements** of the system? This, too, needs to be tested. A bug-free system that answers the wrong question isn't very useful. Be conscious of end-user access patterns and how they differ from developer test data (for an example, see the story about brush strokes here).

Nó có đáp ứng các yêu cầu chức năng của hệ thống không? Điều này cũng cần được kiểm thử. Một hệ thống không lỗi nhưng trả lời sai câu hỏi thì chẳng mấy hữu dụng. Hãy lưu tâm đến cách người dùng cuối truy cập và sự khác biệt của nó so với dữ liệu test của lập trình viên (xem ví dụ trong câu chuyện về nét cọ vẽ ở đây).

Performance Testing Performance or stress testing may be important aspects of the project as well.

Performance Testing Kiểm thử hiệu năng hay kiểm thử chịu tải (stress testing) cũng có thể là những khía cạnh quan trọng của dự án.

Ask yourself if the software meets the performance requirements under real-world conditions—with the expected number of users, or connections, or transactions per second. Is it scalable?

Hãy tự hỏi liệu phần mềm có đáp ứng các yêu cầu hiệu năng trong điều kiện thực tế hay không — với số lượng người dùng, kết nối, hay giao dịch mỗi giây như dự kiến. Nó có khả năng mở rộng (scalable) không?

For some applications, you may need specialized testing

Với một số ứng dụng, bạn có thể cần phần cứng hoặc phần mềm kiểm thử

hardware or software to simulate the load realistically.

chuyên dụng để mô phỏng tải một cách thực tế.

Testing the Tests Because we can't write perfect software, it follows that we can't write perfect test software either. We need to test the tests.

Kiểm Thử Chính Các Bài Test Vì chúng ta không thể viết phần mềm hoàn hảo, suy ra chúng ta cũng không thể viết phần mềm kiểm thử hoàn hảo. Chúng ta cần kiểm thử chính các bài test.

Think of our set of test suites as an elaborate security system, designed to sound the alarm when a bug shows up. How better to test a security system than to try to break in?

Hãy hình dung bộ test suite của chúng ta như một hệ thống an ninh tinh vi, được thiết kế để hú còi báo động khi một con lỗi xuất hiện. Còn cách nào kiểm tra một hệ thống an ninh tốt hơn là thử đột nhập vào nó?

After you have written a test to detect a particular bug, cause the bug deliberately and make sure the test complains. This ensures that the test will catch the bug if it happens for real.

Sau khi viết một bài test để phát hiện một lỗi cụ thể, hãy cố tình gây ra lỗi đó và đảm bảo bài test kêu ca. Việc này đảm bảo rằng bài test sẽ tóm được con lỗi nếu nó xảy ra thật.

Tip 92: Use Saboteurs to Test Your Testing

Tip 92: Dùng Kẻ Phá Hoại Để Kiểm Thử Chính Việc Kiểm Thử Của Bạn

If you are really serious about testing, take a separate branch of the source tree, introduce bugs on purpose, and verify that the tests will catch them. At a higher level, you can use something like Netflix’s Chaos Monkey [81] to disrupt (i.e., “kill”) services and test your application’s resilience.

Nếu bạn thực sự nghiêm túc về kiểm thử, hãy tách riêng một nhánh của cây mã nguồn, cố tình cài lỗi vào, và xác nhận rằng các bài test sẽ tóm được chúng. Ở cấp cao hơn, bạn có thể dùng thứ gì đó như Chaos Monkey của Netflix [81] để gây rối (tức là “giết”) các dịch vụ và kiểm thử khả năng chống chịu của ứng dụng.

When writing tests, make sure that alarms sound when they should.

Khi viết test, hãy đảm bảo còi báo động hú lên đúng lúc đáng phải hú.

Testing Thoroughly Once you are confident that your tests are correct, and are finding bugs you create, how do you know if you have tested the code base thoroughly enough?

Kiểm Thử Cho Thật Kỹ Một khi bạn đã tự tin rằng các bài test của mình là đúng, và đang tóm được những con lỗi bạn tạo ra, thì làm sao biết liệu bạn đã kiểm thử nền mã đủ kỹ hay chưa?

The short answer is “you don’t,” and you never will. You might look to try coverage analysis tools that watch your code during

Câu trả lời ngắn gọn là “bạn không biết được”, và sẽ chẳng bao giờ biết. Bạn có thể thử dùng các công cụ phân tích độ phủ (coverage) theo dõi code trong lúc

testing and keep track of which lines of code have been executed and which haven’t. These tools help give you a general feel for how comprehensive your testing is, but don’t expect to see 100% coverage. [82]

kiểm thử và ghi lại xem những dòng code nào đã được thực thi và những dòng nào thì chưa. Những công cụ này giúp bạn có cảm nhận chung về mức độ toàn diện của việc kiểm thử, nhưng đừng trông mong thấy độ phủ 100%. [82]

Even if you do happen to hit every line of code, that’s not the whole picture. What is important is the number of states that your program may have. States are not equivalent to lines of code. For instance, suppose you have a function that takes two integers, each of which can be a number from 0 to 999:

Ngay cả khi bạn tình cờ chạm tới mọi dòng code, đó vẫn chưa phải toàn bộ bức tranh. Điều quan trọng là số lượng trạng thái mà chương trình của bạn có thể có. Trạng thái không tương đương với dòng code. Chẳng hạn, giả sử bạn có một hàm nhận hai số nguyên, mỗi số có thể từ 0 đến 999:

```
int test( int a, int b) { return a / (a + b); }
```

```
int test( int a, int b) { return a / (a + b); }
```

In theory, this three-line function has 1,000,000 logical states, 999,999 of which will work correctly and one that will not (when $a + b$ equals zero). Simply knowing that you executed this line of code doesn’t tell you that—you would need to identify all possible states of the program. Unfortunately, in general this is a really hard problem. Hard as in, “The sun will be a cold hard lump before you can solve it.”

Về lý thuyết, hàm ba dòng này có 1.000.000 trạng thái logic, trong đó 999.999 trạng thái sẽ chạy đúng và một trạng thái thì không (khi $a + b$ bằng không). Chỉ biết rằng bạn đã thực thi dòng code này không nói cho bạn biết điều đó — bạn sẽ cần nhận diện mọi trạng thái khả dĩ của chương trình. Khốn nỗi, nói chung đây là một bài toán cực khó. Khó kiểu như “Mặt trời sẽ thành một cục lạnh ngắt rồi mà bạn vẫn chưa giải xong nó.”

Tip 93: Test State Coverage, Not Code Coverage****

Tip 93: Kiểm Thử Độ Phủ Trạng Thái, Không Phải Độ Phủ Dòng Code

Property-Based Testing A great way to explore how your code handles unexpected states is to have a computer generate those states.

Property-Based Testing Một cách tuyệt vời để khám phá xem code xử lý các trạng thái bất ngờ ra sao là để máy tính tự sinh ra những trạng thái đó.

Use property-based testing techniques to generate test data according to the contracts and invariants of the code under test. We cover this topic in detail in Topic 42, Property-Based Testing .

Hãy dùng các kỹ thuật kiểm thử dựa trên thuộc tính (property-based testing) để sinh dữ liệu test theo đúng các hợp đồng và bất biến (invariant) của đoạn code đang được kiểm thử. Chúng tôi bàn kỹ chủ đề này ở Topic 42, Kiểm thử dựa trên thuộc tính.

Tightening The Net — Siết Chặt Lưới Lại

Finally, we'd like to reveal the single most important concept in testing. It is an obvious one, and virtually every textbook says to do it this way. But for some reason, most projects still do not.

Cuối cùng, chúng tôi muốn tiết lộ khái niệm quan trọng nhất trong kiểm thử. Nó là một điều hiển nhiên, và hầu như mọi cuốn giáo trình đều bảo phải làm theo cách này. Nhưng vì lý do nào đó, phần lớn dự án vẫn không làm.

If a bug slips through the net of existing tests, you need to add a new test to trap it next time.

Nếu một con lỗi lọt qua tấm lưới của các bài test hiện có, bạn cần thêm một bài test mới để chộp nó ở lần sau.

Tip 94: Find Bugs Once

Tip 94: Tìm Lỗi Một Lần Duy Nhất

Once a human tester finds a bug, it should be the last time a human tester finds that bug. The automated tests should be modified to check for that particular bug from then on, every time, with no exceptions, no matter how trivial, and no matter how much the developer complains and says, “Oh, that will never happen again.”

Một khi người kiểm thử tìm ra một con lỗi, đó nên là lần cuối cùng có một người phải tìm ra con lỗi đó. Từ đó trở đi, các bài test tự động phải được chỉnh sửa để kiểm tra chính con lỗi cụ thể ấy, mọi lúc, không ngoại lệ, dù nó tầm thường đến đâu, và dù lập trình viên có cầu nhàu phản đối đến mấy rằng “Ôi dào, chuyện đó sẽ chẳng bao giờ xảy ra lại đâu.”

Because it will happen again. And we just don't have the time to go chasing after bugs that the automated tests could have found for us. We have to spend our time writing new code—and new bugs.

Bởi vì nó sẽ xảy ra lại. Và chúng ta đơn giản là không có thời gian để chạy đuổi theo những con lỗi mà các bài test tự động lẽ ra đã tìm hộ chúng ta. Chúng ta phải dành thời gian viết code mới — và lỗi mới.

Full Automation — Tự Động Hóa Toàn Diện

As we said at the beginning of this section, modern development relies on scripted, automatic procedures. Whether you use something as simple as **shell** scripts with **rsync** and **ssh**, or full-featured solutions such as **Ansible**, **Puppet**, **Chef**, or **Salt**, just don't rely on any manual intervention.

Như chúng tôi đã nói ở đầu mục này, phát triển hiện đại dựa vào các quy trình tự động, được viết thành script. Dù bạn dùng thứ đơn giản như shell script với rsync và ssh, hay các giải pháp đầy đủ tính năng như Ansible, Puppet, Chef, hay Salt, chỉ cần đừng dựa vào bất kỳ sự can thiệp thủ công nào.

Once upon a time, we were at a **client** site where all the developers were using the same IDE. Their system administrator gave each developer a set of instructions on

Ngày xưa ngày xưa, chúng tôi đến chỗ một khách hàng nơi tất cả lập trình viên đều dùng cùng một IDE. Quản trị viên hệ thống của họ đưa cho mỗi lập trình viên một bộ hướng dẫn về cách

installing add-on packages to the IDE. These instructions filled many pages—pages full of click here, scroll there, drag this, double-click that, and do it again.

cài các gói bổ trợ (add-on) cho IDE. Bộ hướng dẫn này dài đến mấy trang — những trang chỉ chít nào là bấm vào đây, cuộn xuống kia, kéo cái này, nhấp đúp cái nọ, rồi lại làm lại lần nữa.

Not surprisingly, every developer’s machine was loaded slightly differently. Subtle differences in the application’s behavior occurred when different developers ran the same code. Bugs would appear on one machine but not on others. Tracking down version differences of any one component usually revealed a surprise.

Chẳng có gì lạ khi máy của lập trình viên nào cũng được nạp khác nhau một chút. Những khác biệt tinh vi trong hành vi ứng dụng nảy sinh khi các lập trình viên khác nhau chạy cùng một đoạn code. Lỗi xuất hiện trên một máy mà không trên máy khác. Truy lùng khác biệt phiên bản của bất kỳ thành phần nào thường lại hé lộ một bất ngờ.

Tip 95: Don’t Use Manual Procedures

Tip 95: Đừng Dùng Quy Trình Thủ Công

People just aren’t as repeatable as computers are. Nor should we expect them to be. A shell script or program will execute the same instructions, in the same order, time after time. It is under version control itself, so you can examine changes to the build/release procedures over time as well (“but it used to work...”).

Con người đơn giản là không lặp lại nhất quán được như máy tính. Mà chúng ta cũng chẳng nên kỳ vọng họ làm được. Một shell script hay chương trình sẽ thực thi cùng những lệnh ấy, theo cùng thứ tự ấy, hết lần này đến lần khác. Bản thân nó cũng nằm dưới quản lý phiên bản, nên bạn còn có thể xem lại các thay đổi của quy trình build/phát hành theo thời gian (“ơ nhưng trước nó vẫn chạy mà...”).

Everything depends on automation. You can’t build the project on an anonymous cloud server unless the build is fully automatic. You can’t deploy automatically if there are manual steps involved. And once you introduce manual steps (“just for this one part...”) you’ve broken a very large window. [83]

Mọi thứ đều phụ thuộc vào tự động hóa. Bạn không thể build dự án trên một máy chủ cloud vô danh trừ khi bản build là hoàn toàn tự động. Bạn không thể triển khai tự động nếu vẫn còn những bước thủ công dính vào. Và một khi bạn đưa vào các bước thủ công (“chỉ mỗi chỗ này thôi mà...”), là bạn đã đập vỡ một ô cửa sổ rất lớn. [83]

With these three legs of version control, ruthless testing, and full automation, your project will have the firm foundation you need so you can concentrate on the hard part: delighting users.

Với ba chân để gồm quản lý phiên bản, kiểm thử tàn nhẫn, và tự động hóa toàn diện, dự án của bạn sẽ có nền móng vững chắc cần thiết để bạn có thể tập trung vào phần khó nhằn:

làm hài lòng người dùng.

Related Sections Include — Các Mục Liên Quan

Topic 11, **Reversibility**

Topic 11, Tính khả nghịch

Topic 12, **Tracer Bullets**

Topic 12, Đạn vạch đường

Topic 17, Shell Games

Topic 17, Trò chơi với Shell

Topic 19, Version Control

Topic 19, Quản lý phiên bản

Topic 41, Test to Code

Topic 41, Kiểm thử trước

Topic 49, Pragmatic Teams

Topic 49, Nhóm thực dụng

Topic 50, Coconuts Don't Cut It

Topic 50, Dừa Đâu Có Giải Quyết Được Gì

Challenges — Thử Thách

Are your nightly or continuous builds automatic, but deploying to production isn't? Why? What's special about that server?

Các bản build hằng đêm hay build liên tục của bạn đã tự động, nhưng việc triển khai lên production thì chưa? Tại sao? Có gì đặc biệt ở cái máy chủ ấy?

Can you automatically test your project completely? Many teams are forced to answer “no.” Why? Is it too hard to define the acceptable results? Won't this make it hard to prove to the sponsors that the project is “done”?

Bạn có thể kiểm thử dự án của mình một cách tự động và trọn vẹn không? Nhiều nhóm buộc phải trả lời “không”. Tại sao? Có phải vì quá khó để định nghĩa kết quả nào là chấp nhận được? Chẳng phải điều này khiến việc chứng minh cho nhà tài trợ rằng dự án đã “xong” trở nên khó khăn sao?

Is it too hard to test the application logic independent of the GUI? What does this say about the GUI? About **coupling**?

Có phải kiểm thử logic ứng dụng tách biệt khỏi GUI là quá khó không? Điều đó nói lên gì về GUI? Về sự ghép nối (coupling)?

When you enchant people, your goal is not to make money from them or to get them to do what you want, but to fill them with great delight. Guy Kawasaki

Khi bạn làm người ta mê đắm, mục tiêu của bạn không phải kiếm tiền từ họ hay khiến họ làm theo ý mình, mà là rót đầy lòng họ niềm vui sướng tột cùng. Guy Kawasaki

Topic 52. Delight Your Users — Topic 52. Làm Hài Lòng Người Dùng

Our goal as developers is to delight users. That's why we're here. Not to mine them for their data, or count their eyeballs or empty their wallets. Nefarious goals aside, even delivering working software in a timely manner isn't enough. That alone won't delight them.

Mục tiêu của chúng ta với tư cách lập trình viên là làm hài lòng người dùng. Đó là lý do chúng ta có mặt ở đây. Không phải để khai thác dữ liệu của họ, đếm lượt mắt nhìn hay vét sạch ví tiền của họ. Gạt những mục đích mờ ám ấy sang một bên, thì ngay cả việc giao phần mềm chạy được đúng hạn cũng chưa đủ. Chỉ chừng đó thôi sẽ không làm họ hài lòng đâu.

Your users are not particularly motivated by code. Instead, they have a business problem that needs solving within the **context** of their objectives and budget. Their belief is that by working with your team they'll be able to do this.

Người dùng của bạn chẳng mấy hào hứng với code. Thay vào đó, họ có một bài toán nghiệp vụ cần giải quyết trong khuôn khổ mục tiêu và ngân sách của mình. Họ tin rằng khi làm việc với nhóm của bạn, họ sẽ làm được điều đó.

Their expectations are not software related. They aren't even implicit in any specification they give you (because that specification will be incomplete until your team has iterated through it with them several times).

Kỳ vọng của họ không liên quan gì đến phần mềm. Chúng thậm chí cũng chẳng nằm sẵn, dù ngầm hiểu, trong bất kỳ bản đặc tả nào họ đưa cho bạn (bởi bản đặc tả ấy sẽ còn dở dang cho tới khi nhóm của bạn cùng họ lặp đi lặp lại qua nó vài lần).

How do you unearth their expectations, then? Ask a simple question:

Vậy làm sao để đào ra những kỳ vọng đó? Hãy hỏi một câu đơn giản:

How will you know that we've all been successful a month (or a year, or whatever) after this project is done?

Làm thế nào để bạn biết tất cả chúng ta đã thành công, một tháng (hoặc một năm, hoặc bất cứ mốc nào) sau khi dự án này kết thúc?

You may well be surprised by the answer. A project to improve product recommendations might actually be judged in terms of **customer** retention; a project to consolidate two databases might be judged in terms of data quality, or it might be about cost savings. But it's these expectations of business value that really count—not just the software project itself. The software is only a means to these ends.

Bạn rất có thể sẽ bất ngờ với câu trả lời. Một dự án nhằm cải thiện gợi ý sản phẩm thực ra có thể lại được đánh giá qua mức độ giữ chân khách hàng; một dự án gộp hai cơ sở dữ liệu lại có thể được đánh giá qua chất lượng dữ liệu, hoặc cũng có thể là về tiết kiệm chi phí. Nhưng chính những kỳ vọng về giá trị nghiệp vụ này mới thực sự đáng kể — chứ không phải bản thân dự án phần mềm. Phần mềm chỉ là phương tiện để đạt tới những đích đến ấy.

And now that you've surfaced some of the underlying expectations of value behind the project, you can start thinking about how you can deliver against them:

Và giờ khi bạn đã làm lộ ra một số kỳ vọng giá trị nằm bên dưới dự án, bạn có thể bắt đầu nghĩ xem mình có thể đáp ứng chúng ra sao:

Make sure everyone on the team is totally clear about these expectations.

Hãy chắc chắn mọi người trong nhóm đều hoàn toàn rõ ràng về những kỳ vọng này.

When making decisions, think about which path forward moves closer to those expectations.

Khi ra quyết định, hãy nghĩ xem con đường nào tiến gần hơn tới những kỳ vọng đó.

Critically analyze the user **requirements** in light of the expectations. On many projects we've discovered that the stated "requirement" was in fact just a guess at what could be done by technology: it was actually an amateur implementation plan dressed up as a requirements document. Don't be afraid to make suggestions that change the requirement if you can demonstrate that they will move the project closer to the objective.

Hãy phân tích phê phán các yêu cầu của người dùng dưới ánh sáng của những kỳ vọng. Ở rất nhiều dự án, chúng tôi đã phát hiện ra rằng cái gọi là "yêu cầu" được nêu ra hóa ra chỉ là một phỏng đoán về những gì công nghệ có thể làm được: nó thực chất là một bản kế hoạch triển khai nghiệp dư khoác lên mình bộ áo của tài liệu yêu cầu. Đừng ngại đưa ra những đề xuất làm thay đổi yêu cầu, nếu bạn chứng minh được rằng chúng sẽ đưa dự án tiến gần hơn tới mục tiêu.

Continue to think about these expectations as you progress through the project.

Hãy tiếp tục nghĩ về những kỳ vọng này trong suốt quá trình bạn triển khai dự án.

We've found that as our knowledge of the domain increases, we're better able to make suggestions on other things that could be done to address the underlying business issues. We strongly believe that developers, who are exposed to many different aspects of an organization, can often see ways of weaving different parts of the business together that aren't always obvious to individual departments.

Chúng tôi nhận ra rằng khi hiểu biết của mình về lĩnh vực ngày một sâu hơn, chúng tôi càng có khả năng đưa ra đề xuất cho những việc khác có thể làm để xử lý các vấn đề nghiệp vụ cốt lõi. Chúng tôi tin chắc rằng các lập trình viên, những người được tiếp xúc với nhiều khía cạnh khác nhau của một tổ chức, thường có thể nhìn ra cách đan kết những phần khác nhau của doanh nghiệp lại với nhau — điều không phải lúc nào cũng hiển hiện với từng phòng ban riêng lẻ.

Tip 96: Delight Users, Don't Just Deliver Code

Tip 96: Hãy Làm Người Dùng Vui Lòng, Đừng Chỉ Giao Code

If you want to delight your **client**, forge a relationship with them where you can actively help solve their problems. Even though your title might be some variation of "Software Developer" or "Software Engineer," in truth it should be "Problem Solver." That's what we do, and that's the essence of a **Pragmatic Programmer**.

Nếu bạn muốn làm hài lòng khách hàng, hãy gây dựng một mối quan hệ trong đó bạn chủ động giúp họ giải quyết các bài toán của mình. Dù chức danh của bạn có thể là một biến thể nào đó của "Lập trình viên Phần mềm" hay "Kỹ sư Phần mềm", thì thực ra nó nên là "Người Giải Quyết Vấn Đề". Đó là việc chúng ta làm, và đó là cốt lõi của một Lập trình viên Thực dụng (Pragmatic Programmer).

We solve problems.

Chúng ta giải quyết vấn đề.

Related Sections Include — Các mục liên quan

Topic 12, Tracer Bullets

Topic 12, Dạn vạch đường (Tracer Bullets)

Topic 13, Prototypes and Post-it Notes

Topic 13, Bản mẫu và Giấy nhớ (Prototypes and Post-it Notes)

Topic 45, The Requirements Pit

Topic 45, Cái hố Yêu cầu (The Requirements Pit)

You have delighted us long enough. Jane Austen, *Pride and Prejudice*

Bạn đã làm chúng tôi hài lòng đủ lâu rồi đấy. Jane Austen, *Kiêu hãnh và Định kiến*

Topic 53. *Pride and Prejudice* — Topic 53. Niềm tự hào và Định kiến

Pragmatic Programmers don't shirk from responsibility. Instead, we rejoice in accepting **challenges** and in making our expertise well known. If we are responsible for a design, or a piece of code, we do a job we can be proud of.

Lập trình viên Thực dụng (Pragmatic Programmer) không né tránh trách nhiệm. Ngược lại, chúng tôi vui mừng đón nhận thử thách và cho cả thế giới biết về chuyên môn của mình. Nếu chịu trách nhiệm cho một thiết kế, hay một đoạn mã, chúng tôi làm việc đó theo cách khiến mình có thể tự hào.

Tip 97: Sign Your Work

Tip 97: Hãy Ký Tên Lên Tác Phẩm Của Bạn

Artisans of an earlier age were proud to sign their work. You should be, too.

Những người thợ thủ công thời xưa luôn tự hào ký tên lên tác phẩm của mình. Bạn cũng nên như vậy.

Project teams are still made up of people, however, and this rule can cause trouble. On some projects, the idea of code ownership can cause cooperation problems. People may become territorial, or unwilling to work on common foundation elements. The project may end up like a bunch of insular little fiefdoms. You become prejudiced in favor of your code and against your coworkers.

Tuy nhiên, các nhóm dự án vẫn được tạo thành từ con người, và quy tắc này có thể gây rắc rối. Trong một số dự án, ý niệm về quyền sở hữu mã (code ownership) có thể gây ra vấn đề trong hợp tác. Người ta có thể trở nên cát cứ, hoặc không chịu đựng tay vào các thành phần nền tảng dùng chung. Dự án rốt cuộc có thể giống như một mớ những lãnh địa nhỏ bé khép kín. Bạn đam ra thiên vị mã của mình và mang định kiến với đồng nghiệp.

That's not what we want. You shouldn't jealously defend your code against interlopers; by the same token, you should treat other people's code with respect. The Golden Rule ("Do unto others as you would have them do unto you") and a foundation of mutual respect among the developers is critical to make this **tip** work.

Đó không phải điều chúng tôi mong muốn. Bạn không nên khư khư bảo vệ mã của mình trước những kẻ xâm phạm; cùng lẽ ấy, bạn nên đối xử với mã của người khác bằng sự tôn trọng. Quy tắc Vàng ("Hãy đối xử với người khác theo cách bạn muốn họ đối xử với mình")

và một nền tảng tôn trọng lẫn nhau giữa các lập trình viên là điều then chốt để mọy này phát huy tác dụng.

Anonymity, especially on large projects, can provide a breeding ground for sloppiness, mistakes, sloth, and bad code. It becomes too easy to see yourself as just a cog in the wheel, producing lame excuses in endless status reports instead of good code.

Tính ẩn danh, nhất là trong các dự án lớn, có thể trở thành mảnh đất màu mỡ cho sự cầu thả, sai sót, lười nhác và mỗ tời. Quá dễ để bạn tự xem mình chỉ là một mắt xích nhỏ trong guồng máy, sản xuất ra những lời bao biện vô tận trong các bản báo cáo tiến độ thay vì sản xuất ra mỗ tốt.

While code must be owned, it doesn't have to be owned by an individual. In fact, Kent Beck's **eXtreme Programming** [84] recommends communal ownership of code (but this also requires additional practices, such as **pair programming**, to guard against the dangers of anonymity).

Dù mã nguồn phải có chủ, nó không nhất thiết phải thuộc về một cá nhân. Thực tế, eXtreme Programming của Kent Beck [84] khuyến nghị sở hữu mã theo kiểu cộng đồng (nhưng điều này cũng đòi hỏi thêm những thực hành khác, chẳng hạn lập trình cặp (pair programming), để phòng ngừa những nguy cơ đến từ sự vô danh).

We want to see pride of ownership. "I wrote this, and I stand behind my work." Your **signature** should come to be recognized as an indicator of quality. People should see your name on a piece of code and expect it to be solid, well written, tested, and documented. A really professional job. Written by a professional.

Chúng tôi muốn thấy niềm tự hào về quyền sở hữu. "Tôi đã viết cái này, và tôi đứng sau công việc của mình." Chữ ký của bạn rồi sẽ được công nhận như một dấu hiệu của chất lượng. Người ta thấy tên bạn trên một đoạn mã và mong đợi nó vững chãi, viết tốt, đã kiểm thử và có tài liệu. Một công việc thực sự chuyên nghiệp. Được viết bởi một người chuyên nghiệp.

A Pragmatic Programmer.

Một Lập trình viên Thực dụng.

Thank you.

Cảm ơn bạn.

images/dave_and_andy.png

images/dave_and_andy.png

In the long run, we shape our lives, and we shape ourselves. The **process** never ends until we die. And the choices we make are ultimately our own responsibility. Eleanor Roosevelt

Về lâu dài, chúng ta định hình cuộc đời mình, và chúng ta định hình chính mình. Quá trình ấy chẳng bao giờ dừng lại cho đến khi ta nhắm mắt. Và những lựa chọn ta đưa ra, rốt cuộc, là trách nhiệm của chính ta. Eleanor Roosevelt

Footnotes

Footnotes

[75] As team size grows, communication paths grow at the rate of n^2 , where n is the number of team members. On larger teams, communication begins to break down and becomes ineffective.

[75] Khi quy mô nhóm tăng lên, số đường truyền giao tiếp tăng theo tốc độ n^2 , trong đó là số thành viên nhóm. Với các nhóm lớn hơn, giao tiếp bắt đầu đổ vỡ và trở nên kém hiệu quả.

[76] A burnup chart is better for this than the more usual burndown chart. With a burnup chart, you can clearly see how the additional features move the goalposts.

[76] Biểu đồ burnup phù hợp cho việc này hơn biểu đồ burndown thông thường. Với biểu đồ burnup, bạn có thể thấy rõ những tính năng phát sinh đã dịch chuyển cột mốc đích như thế nào.

[77] The team speaks with one voice—externally. Internally, we strongly encourage lively, robust debate. Good developers tend to be passionate about their work.

[77] Nhóm phát ngôn bằng một tiếng nói chung—với bên ngoài. Còn nội bộ, chúng tôi hết sức khuyến khích tranh luận sôi nổi, gay gắt. Lập trình viên giỏi thường đầy nhiệt huyết với công việc của mình.

[78] Andy has met teams who conduct their daily Scrum standups on Fridays.

[78] Andy từng gặp những nhóm tổ chức buổi họp đứng Scrum hằng ngày vào... thứ Sáu.

[79] See https://en.wikipedia.org/wiki/Cargo_cult .

[79] Xem https://en.wikipedia.org/wiki/Cargo_cult .

[80] We've seen this first-hand more times than you'd think.

[80] Chúng tôi đã tận mắt chứng kiến điều này nhiều lần hơn bạn tưởng.

[81] <https://netflix.github.io/chaosmonkey>

[81] <https://netflix.github.io/chaosmonkey>

[82] For an interesting study of the correlation between test coverage and defects, see Mythical Unit Test Coverage [ADSS18] .

[82] Để có một nghiên cứu thú vị về mối tương quan giữa độ phủ kiểm thử và lỗi, xem Mythical Unit Test Coverage [ADSS18] .

[83] Always remember **Topic 3, Software Entropy** . Always.

[83] Luôn nhớ Topic 3, Sự rối loạn phần mềm . Luôn luôn.

[84] <http://www.extremeprogramming.org>

[84] <http://www.extremeprogramming.org>

Copyright © 2020 Pearson Education, Inc.

Copyright © 2020 Pearson Education, Inc.

Postface — Lời bạt

In the twenty years leading up to the first edition, we were part of the evolution of the computer from a peripheral curiosity to a modern imperative for businesses. In the twenty years since then, software has grown beyond mere business machines and has truly taken over the world. But what does that really mean for us?

Trong hai mươi năm dẫn tới lần xuất bản đầu tiên, chúng tôi đã là một phần của cuộc tiến hóa của máy tính, từ một thứ kỳ lạ ngoài lề trở thành một thứ thiết yếu cho doanh nghiệp hiện đại. Hai mươi năm kể từ đó, phần mềm đã vượt khỏi khuôn khổ những cỗ máy phục vụ kinh doanh đơn thuần và thực sự thống lĩnh cả thế giới. Nhưng điều đó thực ra có ý nghĩa gì với chúng ta?

In *The Mythical Man-Month: Essays on Software Engineering* [Bro96], Fred Brooks said “The programmer, like the poet, works only slightly removed from pure thought-stuff. He builds his castles in the air, from air, creating by exertion of the imagination.” We start with a blank page, and we can create pretty much anything we can imagine. And the things we create can change the world.

Trong cuốn *The Mythical Man-Month: Essays on Software Engineering* [Bro96], Fred Brooks viết: “Lập trình viên, cũng như nhà thơ, làm việc chỉ cách thuần chất tư duy có một chút. Anh ta xây những lâu đài trên không, từ không khí, sáng tạo bằng nỗ lực của trí tưởng tượng.” Chúng ta khởi đầu với một trang giấy trắng, và có thể tạo ra gần như bất cứ thứ gì ta tưởng tượng ra. Và những thứ ta tạo ra có thể thay đổi cả thế giới.

From Twitter helping people plan revolutions, to the processor in your car working to stop you skidding, to the smartphone which means we no longer have to remember pesky daily details, our programs are everywhere. Our imagination is everywhere.

Từ Twitter giúp người ta lên kế hoạch cho các cuộc cách mạng, đến bộ xử lý trong chiếc xe của bạn đang gắng sức ngăn bạn trượt bánh, cho tới chiếc điện thoại thông minh nhờ đó ta không còn phải nhớ những chi tiết vụn vặt hằng ngày — các chương trình của chúng ta hiện diện khắp nơi. Trí tưởng tượng của chúng ta hiện diện khắp nơi.

We developers are incredibly privileged. We are truly building the future. It’s an extraordinary amount of power. And with that

Chúng ta, những lập trình viên, là những người đặc quyền đến khó tin. Chúng ta thực sự đang xây dựng tương lai. Đó là một quyền năng phi thường. Và đi cùng quyền năng

power comes an extraordinary responsibility.

ấy là một trách nhiệm cũng phi thường không kém.

How often do we stop to think about that? How often do we discuss, both among ourselves and with a more general audience, what this means?

Chúng ta dừng lại để suy ngẫm về điều đó bao nhiêu lần? Chúng ta bàn luận về ý nghĩa của nó bao nhiêu lần, với nhau cũng như với một đối tượng độc giả rộng hơn?

Embedded devices use an order of magnitude more computers than those used in laptops, desktops, and data centers. These embedded computers often control life-critical systems, from power plants to cars to medical equipment. Even a simple central heating control system or home appliance can kill someone if it is poorly designed or implemented. When you develop for these devices, you take on a staggering responsibility.

Các thiết bị nhúng dùng số máy tính nhiều hơn hẳn một bậc độ lớn so với số máy tính trong laptop, desktop và trung tâm dữ liệu. Những máy tính nhúng này thường điều khiển các hệ thống mà sự sống phụ thuộc vào, từ nhà máy điện đến ô tô đến thiết bị y tế. Ngay cả một hệ thống điều khiển sưởi trung tâm hay thiết bị gia dụng đơn giản cũng có thể giết người nếu nó được thiết kế hoặc lập trình tệ. Khi bạn phát triển cho những thiết bị này, bạn gánh lấy một trách nhiệm đến choáng váng.

Many nonembedded systems can also do both great good and great harm. Social media can promote peaceful revolution or foment ugly hate. Big data can make shopping easier, and it can destroy any vestige of privacy you might think you have. Banking systems make loan decisions that change people's lives. And just about any system can be used to snoop on its users.

Nhiều hệ thống không nhúng cũng có thể làm điều tốt lớn lao lẫn điều hại khủng khiếp. Mạng xã hội có thể cổ vũ những cuộc cách mạng ôn hòa hoặc nhen lên thứ thù hận xấu xí. Dữ liệu lớn (big data) có thể khiến việc mua sắm dễ dàng hơn, mà cũng có thể hủy hoại mọi chút riêng tư mà bạn ngỡ mình còn có. Các hệ thống ngân hàng đưa ra những quyết định cho vay làm thay đổi cuộc đời người ta. Và gần như bất kỳ hệ thống nào cũng có thể bị dùng để rình mò chính người dùng của nó.

We've seen hints of the possibilities of a utopian future, and examples of unintended consequences leading to nightmare dystopias. The difference between the two outcomes might be more subtle than you think. And it's all in your hands.

Chúng ta đã thoáng thấy những khả năng của một tương lai không tưởng tươi đẹp, và cả những ví dụ về hậu quả ngoài ý muốn dẫn tới những viễn cảnh phản địa đàng như ác mộng. Khác biệt giữa hai kết cục ấy có thể tinh vi hơn bạn nghĩ. Và tất cả nằm trong tay bạn.

The Moral Compass

La Bàn Đạo Đức

The price of this unexpected power is vigilance. Our actions directly affect people. No longer the hobby program on the 8-bit CPU in the garage, the isolated batch business **process** on the mainframe in the data center, or even just the desktop PC; our software weaves the very fabric of daily modern life.

Cái giá của thứ quyền năng bất ngờ này là sự cảnh giác. Hành động của chúng ta tác động trực tiếp tới con người. Không còn là chương trình nghiệp dư chạy trên con CPU 8-bit trong ga-ra, không còn là tiến trình nghiệp vụ theo lô đơn độc trên chiếc mainframe trong trung tâm dữ liệu, hay thậm chí chỉ là chiếc máy tính để bàn nữa; phần mềm của chúng ta dệt nên chính tấm vải của đời sống hiện đại thường nhật.

We have a duty to ask ourselves two questions about every piece of code we deliver:

Chúng ta có nghĩa vụ tự hỏi mình hai câu hỏi về mỗi đoạn mã ta giao đi:

1. Have I protected the user?
2. Would I use this myself?

1. Mình đã bảo vệ người dùng chưa? 2. Bản thân mình có dùng thứ này không?

First, you should ask “Have I done my best to protect the users of this code from harm?” Have I made provisions to apply ongoing security patches to that simple baby monitor? Have I ensured that however the automatic central heating thermostat fails the **customer** will still have manual control? Am I storing only the data I need, and encrypting anything personal?

Trước hết, bạn nên hỏi: “Mình đã làm hết sức để bảo vệ người dùng của đoạn mã này khỏi bị tổn hại chưa?” Mình đã trù liệu để có thể liên tục vá lỗi bảo mật cho cái máy theo dõi em bé giản đơn kia chưa? Mình đã bảo đảm rằng dù cái van điều nhiệt sưởi trung tâm tự động có hỏng cách nào đi nữa thì khách hàng vẫn còn quyền điều khiển bằng tay chưa? Mình có chỉ lưu đúng dữ liệu mình cần, và mã hóa mọi thứ thuộc về cá nhân không?

No one is perfect; everyone misses things now and then. But if you can’t truthfully say that you tried to list all the consequences, and made sure to protect the users from them, then you bear some responsibility when things go bad.

Chẳng ai hoàn hảo cả; ai rồi cũng có lúc bỏ sót điều gì đó. Nhưng nếu bạn không thể thành thật nói rằng mình đã cố liệt kê mọi hậu quả và bảo đảm bảo vệ người dùng khỏi chúng, thì bạn phải chịu một phần trách nhiệm khi mọi sự đổ bể.

Tip 98: First, Do No Harm

Tip 98: Trước Hết, Đừng Gây Hại

Second, there’s a judgment related to the Golden Rule: would I be happy to be a user of this software? Do I want my details

Thứ hai, có một phán xét gắn với Quy Tắc Vàng: liệu mình có vui khi là người dùng của phần mềm này không? Mình có muốn thông tin của mình

shared? Do I want my movements to be given to retail outlets? Would I be happy to be driven by this autonomous vehicle? Am I comfortable doing this?

bị đem chia sẻ không? Mình có muốn lộ trình di chuyển của mình bị giao cho các cửa hàng bán lẻ không? Mình có vui khi để chiếc xe tự hành này chở đi không? Mình có thấy thoải mái khi làm việc này không?

Some inventive ideas begin to skirt the bounds of ethical behavior, and if you’re involved in that project, you are just as responsible as the sponsors. No matter how many degrees of separation you might rationalize, one rule remains true:

Vài ý tưởng đầy sáng tạo bắt đầu lượn sát ranh giới của hành vi đạo đức, và nếu bạn dính líu vào dự án đó, bạn cũng phải chịu trách nhiệm y như những kẻ chống lưng. Dù bạn có biện hộ rằng mình cách xa bao nhiêu nấc đi nữa, có một quy tắc vẫn đúng:

Tip 99: Don’t Enable Scumbags

Tip 99: Đừng Tiếp Tay Cho Lũ Khốn

Imagine the Future you Want

Hãy Mường Tượng Tương Lai Bạn Muốn

It’s up to you. It’s your imagination, your hopes, your concerns that provide the pure thought-stuff that builds the next twenty years and beyond.

Tất cả tùy thuộc vào bạn. Chính trí tưởng tượng, niềm hy vọng, những trăn trở của bạn là thứ thuần chất tư duy dựng nên hai mươi năm tới và xa hơn nữa.

You are building the future, for yourselves and for your descendants. Your duty is to make it a future that we'd all want to inhabit. Recognize when you're doing something against this ideal, and have the courage to say "no!" Envision the future we could have, and have the courage to create it. Build castles in the air every day.

Bạn đang xây dựng tương lai, cho chính mình và cho con cháu mình. Nghĩa vụ của bạn là làm cho nó thành một tương lai mà tất cả chúng ta đều muốn được sống trong đó. Hãy nhận ra khi mình đang làm điều gì đó đi ngược lý tưởng ấy, và có đủ can đảm để nói "không!" Hãy hình dung tương lai ta có thể có, và có đủ can đảm để tạo ra nó. Hãy xây những lâu đài trên không mỗi ngày.

We all have an amazing life.

Tất cả chúng ta đều có một cuộc đời tuyệt diệu.

It's Your Life. — Đó Là Cuộc Đời Của Bạn.

Tip 100: Share it. Celebrate it. Build it.

Tip 100: Hãy chia sẻ nó. Hãy ăn mừng nó. Hãy xây dựng nó.

AND HAVE FUN! — VÀ HÃY VUI LÊN!

Copyright © 2020 Pearson Education, Inc.

Copyright © 2020 Pearson Education, Inc.

Bibliography

[ADSS18 Vard Antinyan, Jesper Derehag, Anna Sandberg, and Mirosław Staron. Mythical Unit Test Coverage. *IEEE J Software* . 35:73-79, 2018. [And10] Jackie Andrade. What does doodling do? *Applied Cognitive Psychology* . 24(1):100-106, 2010, January. [Arm07] Joe Armstrong. *Programming Erlang: Software for a Concurrent World* . The Pragmatic Bookshelf, Raleigh, NC, 2007. [BR89] Albert J. Bernstein and Sydney Craft Rozen. *Dinosaur Brains: Dealing with All Those Impossible People at Work* . John Wiley & Sons, New York, NY, 1989. [Bro96] Frederick P. Brooks, Jr. *The Mythical Man-Month: Essays on Software Engineering* . Addison-Wesley, Reading, MA, Anniversary, 1996. [CN91] Brad J. Cox and Andrew J. Novobilski. *Object-Oriented Programming: An Evolutionary Approach* . Addison- Wesley, Reading, MA, Second, 1991. [Con68] Melvin E. Conway. How do Committees Invent? *Datamation* . 14(5):28-31, 1968, April. [de 98] Gavin de Becker. *The Gift of Fear: And Other Survival Signals That Protect Us from Violence* . Dell Publishing, New York City, 1998. [DL13] Tom DeMacro and Tim Lister. *Peopleware: Productive Projects and Teams* . Addison-Wesley, Boston, MA, Third, 2013. [Fow00] Martin Fowler. *UML Distilled: A Brief Guide to the Standard Object Modeling Language* . Addison-

Wesley, Boston, MA, Second, 2000. [Fow04] Martin Fowler. *UML Distilled: A Brief Guide to the Standard Object Modeling Language* . Addison- Wesley, Boston, MA, Third, 2004. [Fow19] Martin Fowler. *Refactoring: Improving the Design of Existing Code* . Addison-Wesley, Boston, MA, Second, 2019. [GHJV9] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software* . Addison-Wesley, Reading, MA, 1995. [Hol92] Michael Holt. *Math Puzzles & Games* . Dorset House, New York, NY, 1992. [Hun08] Andy Hunt. *Pragmatic Thinking and Learning: Refactor Your Wetware* . The Pragmatic Bookshelf, Raleigh, NC, 2008. [Joi94] T.E. Joiner. Contagious depression: Existence, specificity to depressed symptoms, and the role of reassurance seeking. *Journal of Personality and Social Psychology* . 67(2):287–296, 1994, August. [Knu11] Donald E. Knuth. *The Art of Computer Programming, Volume 4A: Combinatorial Algorithms, Part 1* . Addison-Wesley, Boston, MA, 2011. [Knu98] Donald E. Knuth. *The Art of Computer Programming, Volume 1: Fundamental Algorithms* . Addison-Wesley, Reading, MA, Third, 1998. [Knu98a] Donald E. Knuth. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms* . Addison- Wesley, Reading, MA, Third, 1998. [Knu98b] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching* . Addison-Wesley, Reading, MA, Second, 1998. [KP99] Brian W. Kernighan and Rob Pike. *The Practice of Programming* . Addison-Wesley, Reading, MA, 1999. [Mey97] Bertrand Meyer. *Object-Oriented Software Construction* . Prentice Hall, Upper Saddle River, NJ, Second, 1997. [Mul18] Jerry Z. Muller. *The Tyranny of Metrics* . Princeton University Press, Princeton NJ, 2018.

I would rather have questions that can't be answered than answers that can't be questioned. Richard Feynman

[SF13] Robert Sedgewick and Phillipe Flajolet. *An Introduction to the Analysis of Algorithms* . Addison-Wesley, Boston, MA, Second, 2013. [Str35] James Ridley Stroop. *Studies of Interference*

in Serial Verbal Reactions. *Journal of Experimental Psychology* . 18:643–662, 1935. [SW11] Robert Sedgewick and Kevin Wayne. *Algorithms* . Addison- Wesley, Boston, MA, Fourth, 2011. [Tal10] Nassim Nicholas Taleb. *The Black Swan: Second Edition: The Impact of the Highly Improbable* . Random House, New York, NY, Second, 2010. [WH82] James Q. Wilson and George Helling. The police and neighborhood safety. *The Atlantic Monthly* . 249[3]:29- -38, 1982, March. [YC79] Edward Yourdon and Larry L. Constantine. *Structured Design: Fundamentals of a Discipline of Computer Program and Systems Design* . Prentice Hall, Englewood Cliffs, NJ, 1979. [You95] Edward Yourdon. When good-enough software is best. *IEEE Software* . 1995, May.

Copyright © 2020 Pearson Education, Inc.

Answer 1 (from exercise 1)

To our way of thinking, class `Split2` is more orthogonal. It concentrates on its own task, splitting lines, and ignores details such as where the lines are coming from. Not only does this make the code easier to develop, but it also makes it more flexible. `Split2` can split lines read from a file, generated by another routine, or passed in via the environment.

Answer 2 (from exercise 2)

Let's start with an assertion: you can write good, orthogonal code in just about any language. At the same time, every language has temptations: features that can lead to increased coupling and decreased orthogonality.

In OO languages, features such as multiple inheritance, exceptions, operator overloading, and parent-method overriding (via subclassing) provide ample opportunity to increase coupling in nonobvious ways. There is also a kind of coupling because a class couples code to data. This is normally a good thing (when coupling is good, we call it cohesion). But if you don't make your classes focused enough, it can lead to some pretty ugly interfaces.

In functional languages, you're encouraged to write lots of small, decoupled functions, and to combine them in different ways to solve your problem. In theory this sounds good. In practice it often is. But there's a form of coupling that can happen here, too. These functions typically transform data, which means the result of one function can become the input to another. If you're not careful, making a change to the data format a function generates can result in a failure somewhere down the transformational stream. Languages with good type systems can help mitigate this.

Possible Answers to the Exercises — Đáp án khả dĩ cho các bài tập

Answer 3 (from exercise 3)

Đáp án 3 (từ bài tập 3)

Low-tech to the rescue! Draw a few cartoons with markers on a whiteboard—a car, a phone, and a house. It doesn't have to be great art; stick-figure outlines are fine. Put Post-it notes that describe the contents of target pages on the clickable areas. As the meeting progresses, you can refine the drawings and placements of the Post-it notes.

Công nghệ thấp ra tay cứu nguy! Vẽ vài hình hoạt họa bằng bút lông trên bảng trắng — một chiếc xe, một cái điện thoại, một ngôi nhà. Không cần phải là tác phẩm nghệ thuật; hình que sơ sài là đủ. Dán giấy ghi chú Post-it mô tả nội dung các trang đích lên những vùng có thể bấm vào. Khi cuộc họp diễn ra, bạn có thể tinh chỉnh dần các hình vẽ và vị trí của những tờ Post-it.

Answer 4 (from exercise 4)

Đáp án 4 (từ bài tập 4)

Because we want to make the language extendable, we'll make the **parser** table driven. Each entry in the table contains the command letter, a flag to say whether an argument is required, and the name of the routine to call to handle that particular command.

Vì muốn ngôn ngữ có thể mở rộng được, chúng tôi sẽ làm bộ phân tích cú pháp (parser) theo kiểu điều khiển bằng bảng (table driven). Mỗi mục trong bảng chứa ký tự lệnh, một cờ cho biết lệnh có cần đối số hay không, và tên của hàm cần gọi để xử lý lệnh cụ thể đó.

lang/turtle.c

lang/turtle.c

```
typedef struct { char cmd; /* the command letter / int hasArg; / does it take an argument / void (func)(  
int , int ); /* routine to call */ } Command;
```

```
typedef struct { char cmd; /* the command letter / int hasArg; / does it take an argument /  
void (func)( int , int ); /* routine to call */ } Command;
```

```
static Command cmds[] = { { 'P' , ARG, doSelectPen }, { 'U' , NO_ARG, doPenUp }, { 'D' , NO_ARG,  
doPenDown }, { 'N' , ARG, doPenDir }, { 'E' , ARG, doPenDir }, { 'S' , ARG, doPenDir }, { 'W' , ARG,  
doPenDir } }; The main program is pretty simple: read a line, look up the command, get the argument  
if required, then call the handler
```

```
static Command cmds[] = { { 'P' , ARG, doSelectPen }, { 'U' , NO_ARG, doPenUp }, { 'D' , NO_ARG, doPenDown }, { 'N' , ARG, doPenDir }, { 'E' , ARG, doPenDir }, { 'S' , ARG, doPenDir }, { 'W' , ARG, doPenDir } }; The main program is pretty simple: read a line, look up the command, get the argument if required, then call the handler
```

function.

```
function.
```

```
lang/turtle.c
```

```
lang/turtle.c
```

```
while (fgets(buff, sizeof (buff), stdin)) {
```

```
    while (fgets(buff, sizeof (buff), stdin)) {
```

```
        Command cmd = findCommand(buff);
```

```
        Command cmd = findCommand(buff);
```

```
        if (cmd) { int arg = 0;
```

```
            if (cmd) { int arg = 0;
```

```
            if (cmd->hasArg && !getArg(buff+1, &arg)) { fprintf(stderr, "%c' needs an argument\n" , *buff); continue ; }
```

```
                if (cmd->hasArg && !getArg(buff+1, &arg)) { fprintf(stderr, "%c' needs an argument\n" , *buff); continue ; }
```

```
            cmd->func(*buff, arg); } }
```

```
            cmd->func(*buff, arg); } }
```

The function that looks up a command performs a linear search of the table, returning either the matching entry or NULL.

Hàm tra cứu lệnh thực hiện một phép tìm kiếm tuyến tính trên bảng, trả về hoặc mục khớp hoặc NULL.

```
lang/turtle.c
```

```
lang/turtle.c
```

```
Command *findCommand( int cmd) { int i;
```

```
    Command *findCommand( int cmd) { int i;
```

```
    for (i = 0; i < ARRAY_SIZE(cmds); i++) { if (cmds[i].cmd == cmd) return cmds + i; }
```

```
        for (i = 0; i < ARRAY_SIZE(cmds); i++) { if (cmds[i].cmd == cmd) return cmds + i; }
```

```
    fprintf(stderr, "Unknown command '%c'\n" , cmd); return 0; }
```

```
        fprintf(stderr, "Unknown command '%c'\n" , cmd); return 0; }
```

Finally, reading the numeric argument is pretty simple using sscanf .

Cuối cùng, việc đọc đối số dạng số khá đơn giản, chỉ cần dùng sscanf .

```
lang/turtle.c
```

```
lang/turtle.c
```

```
int getArg( const char buff, int result) { return sscanf(buff, "%d" , result) == 1; }
```

```
int getArg( const char buff, int result) { return sscanf(buff, "%d" , result) == 1; }
```

Answer 5 (from exercise 5)

Đáp án 5 (từ bài tập 5)

Actually, you've already solved this problem in the previous exercise, where you wrote an interpreter for the external language, will contain the **internal** interpreter. In the case of our sample code, this is the doXxx functions.

Thực ra, bạn đã giải xong bài toán này ở bài tập trước, nơi bạn viết một trình thông dịch cho ngôn ngữ bên ngoài, sẽ chứa trình thông dịch nội bộ. Với đoạn mã ví dụ của chúng tôi, đó chính là các hàm doXxx.

Answer 6 (from exercise 6)

Đáp án 6 (từ bài tập 6)

Using BNF, a time specification could be

Dùng BNF, một đặc tả thời gian có thể là

```
time ::= hour ampm | hour : minute ampm | hour : minute
```

```
time ::= hour ampm | hour : minute ampm | hour : minute
```

```
ampm ::= am | pm
```

```
ampm ::= am | pm
```

```
hour ::= digit | digit digit
```

```
hour ::= digit | digit digit
```

```
minute ::= digit digit
```

```
minute ::= digit digit
```

```
digit ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

```
digit ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

A better definition of hour and minute would take into account that an hours can only be from 00 to 23, and a minute from 00 to 59:

Một định nghĩa tốt hơn cho hour và minute sẽ tính đến chuyện giờ chỉ có thể từ 00 đến 23, và phút từ 00 đến 59:

```
hour ::= h-tens digit | digit
```

```
hour ::= h-tens digit | digit
```

```
minute ::= m-tens digit
```

```
minute ::= m-tens digit
```

```
h-tens ::= 0 | 1
```

```
h-tens ::= 0 | 1
```

```
m-tens ::= 0 | 1 | 2 | 3 | 4 | 5
```

```
m-tens ::= 0 | 1 | 2 | 3 | 4 | 5
```

```
digit ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

```
digit ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

Answer 7 (from exercise 7)

Đáp án 7 (từ bài tập 7)

Here's the parser written using the Pegjs JavaScript library:

Đây là bộ phân tích cú pháp viết bằng thư viện JavaScript Pegjs:

```
lang/peg_parser/time_parser.pegjs
```

```
lang/peg_parser/time_parser.pegjs
```

```
time = h:hour offset:ampm { return h + offset } / h:hour ":" m:minute offset:ampm { return h + m + offset } / h:hour ":" m:minute { return h + m }
```

```
time = h:hour offset:ampm { return h + offset } / h:hour ":" m:minute offset:ampm { return h + m + offset } / h:hour ":" m:minute { return h + m }
```

```
ampm = "am" { return 0 } / "pm" { return 12*60 }
```

```
ampm = "am" { return 0 } / "pm" { return 12*60 }
```

```
hour = h:two_hour_digits { return h60 } / h:digit { return h60 }
```

```
hour = h:two_hour_digits { return h60 } / h:digit { return h60 }
```

```
minute = d1:[0-5] d2:[0-9] { return parseInt(d1+d2, 10); }
```

```
minute = d1:[0-5] d2:[0-9] { return parseInt(d1+d2, 10); }
```

```
digit = digit:[0-9] { return parseInt(digit, 10); }
```

```
digit = digit:[0-9] { return parseInt(digit, 10); }
```

```
two_hour_digits = d1:[01] d2:[0-9 ] { return parseInt(d1+d2, 10); } / d1:[2] d2:[0-3] { return parseInt(d1+d2, 10); }
```

```
two_hour_digits = d1:[01] d2:[0-9 ] { return parseInt(d1+d2, 10); } / d1:[2] d2:[0-3] { return parseInt(d1+d2, 10); }
```

The tests show it in use:

Các bài kiểm thử cho thấy nó hoạt động thế nào:

```
lang/peg_parser/test_time_parser.js
```

```
lang/peg_parser/test_time_parser.js
```

```
let test = require( 'tape' ); let time_parser = require( './time_parser.js' );
```

```
let test = require( 'tape' ); let time_parser = require( './time_parser.js' );
```

```
// time ::= hour ampm | // hour : minute ampm | // hour : minute // // ampm ::= am | pm // // hour ::= digit | digit digit // // minute ::= digit digit // // digit ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

```
// time ::= hour ampm | // hour : minute ampm | // hour : minute // // ampm ::= am | pm //  
// hour ::= digit | digit digit // // minute ::= digit digit // // digit ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

```
const h = (val) => val*60; const m = (val) => val; const am = (val) => val; const pm = (val) => val + h(12);
```

```
const h = (val) => val*60; const m = (val) => val; const am = (val) => val; const pm = (val) => val + h(12);
```

```

let tests = {
  let tests = {
    "1am" : h(1), "1pm" : pm(h(1)),
    "1am" : h(1), "1pm" : pm(h(1)),
    "2:30" : h(2) + m(30), "14:30" : pm(h(2)) + m(30), "2:30pm" : pm(h(2)) + m(30),
    "2:30" : h(2) + m(30), "14:30" : pm(h(2)) + m(30), "2:30pm" : pm(h(2)) + m(30),
  }
}

test( 'time parsing' , function (t) { for ( const string in tests) { let result = time_parser.parse(string)
t.equal(result, tests[string], string); } t.end() });

test( 'time parsing' , function (t) { for ( const string in tests) { let result = time_parser.parse(string)
t.equal(result, tests[string], string); } t.end() });

```

Answer 8 (from exercise 8)

Đáp án 8 (từ bài tập 8)

Here's a possible solution in Ruby:

Đây là một lời giải khả dĩ bằng Ruby:

```

lang/re_parser/time_parser.rb
  lang/re_parser/time_parser.rb
TIME_RE = %r{ (?[0-9]){0} (?[0-1]){0} (?[0-6]){0} (? am | pm){0} (? (\g \g) | \g){0} (? \g \g){0}
  TIME_RE = %r{ (?[0-9]){0} (?[0-1]){0} (?[0-6]){0} (? am | pm){0} (? (\g \g) | \g){0} (? \g \g){0}
\A( ( \g \g ) | ( \g : \g \g ) | ( \g : \g ) )\Z
  \A( ( \g \g ) | ( \g : \g \g ) | ( \g : \g ) )\Z
}x
  }x

def parse_time(string) result = TIME_RE.match(string) if result result[ :hour ].to_i * 60 + (result[
:minute ] || "0").to_i + (result[ :ampm ] == "pm" ? 12*60 : 0) end end
  def parse_time(string) result = TIME_RE.match(string) if result result[ :hour ].to_i * 60 +
  (result[ :minute ] || "0").to_i + (result[ :ampm ] == "pm" ? 12*60 : 0) end end

```

(This code uses the trick of defining named patterns at the start of the regular expression, and then referencing them as subpatterns in the actual match.)

(Đoạn mã này dùng mẹo định nghĩa các mẫu (pattern) có tên ở đầu biểu thức chính quy, rồi tham chiếu chúng như các mẫu con trong phần khớp thực sự.)

Answer 9 (from exercise 9)

Đáp án 9 (từ bài tập 9)

Our answer must be couched in several assumptions:

Câu trả lời của chúng tôi phải dựa trên vài giả định:

The storage device contains the information we need to be transferred.

Thiết bị lưu trữ chứa thông tin mà ta cần chuyển đi.

We know the speed at which the person walks.

Ta biết tốc độ đi bộ của người đó.

We know the distance between the machines.

Ta biết khoảng cách giữa hai máy.

We are not accounting for the time it takes to transfer information to and from the storage device.

Ta không tính đến thời gian cần để chuyển thông tin vào và ra khỏi thiết bị lưu trữ.

The overhead of storing data is roughly equal to the overhead of sending it over a communications line.

Phụ phí của việc lưu dữ liệu xấp xỉ bằng phụ phí của việc gửi nó qua đường truyền thông tin.

Answer 10 (from exercise 10)

Đáp án 10 (từ bài tập 10)

Subject to the caveats in the previous answer: A 1TGB tape contains 8×2 , or 2 bits, so a 1Gbps line would have to pump data for about 9,000 seconds, or roughly $2\frac{1}{2}$ hours, to transfer the equivalent amount of information. If the person is walking at a constant $3\frac{1}{2}$ mph, then our two machines would need to be almost 9 miles apart for the communications line to outperform our courier. Otherwise, the person wins.

Với những điều dè dặt ở đáp án trước: Một cuộn băng 1TGB chứa 8×2 , tức 2 bit, nên một đường truyền 1Gbps sẽ phải bơm dữ liệu trong khoảng 9.000 giây, hay chừng $2\frac{1}{2}$ giờ, để chuyển được lượng thông tin tương đương. Nếu người đó đi bộ với vận tốc cố định $3\frac{1}{2}$ dặm/giờ, thì hai máy của chúng ta phải cách nhau gần 9 dặm thì đường truyền mới thắng nổi anh giao liên. Còn không thì con người thắng.

Answer 14 (from exercise 14)

Đáp án 14 (từ bài tập 14)

We'll show the function signatures in Java, with the preand postconditions in comments.

Chúng tôi sẽ thể hiện chữ ký hàm (function signature) bằng Java, với tiền điều kiện và hậu điều kiện đặt trong phần chú thích.

First, the invariant for the class:

Trước tiên, bất biến (invariant) cho lớp:

```
/** * @invariant getSpeed() > 0 * implies isFull() // Don't run empty * * @invariant getSpeed() >= 0 &&  
* getSpeed() < 10 // Range check */
```

```
/** * @invariant getSpeed() > 0 * implies isFull() // Don't run empty * * @invariant  
getSpeed() >= 0 && * getSpeed() < 10 // Range check */
```

Next, the preand postconditions:

Tiếp theo là tiền điều kiện và hậu điều kiện:

```
/** * @pre Math.abs(getSpeed() - x) <= 1 // Only change by one * @pre x >= 0 && x < 10 // Range check  
* @post getSpeed() == x // Honor requested speed */ public void setSpeed( final int x)
```

```

/** * @pre Math.abs(getSpeed() - x) <= 1 // Only change by one * @pre x >= 0 && x < 10
// Range check * @post getSpeed() == x // Honor requested speed */ public void setSpeed(
final int x)

/** * @pre !isFull() // Don't fill it twice * @post isFull() // Ensure it was done */ void fill()

/** * @pre !isFull() // Don't fill it twice * @post isFull() // Ensure it was done */ void fill()

/** * @pre isFull() // Don't empty it twice * @post !isFull() // Ensure it was done */ void empty()

/** * @pre isFull() // Don't empty it twice * @post !isFull() // Ensure it was done */ void
empty()

```

Answer 15 (from exercise 15)

Đáp án 15 (từ bài tập 15)

There are 21 terms in the series. If you said 20, you just experienced a fencepost error (not knowing whether to count the fenceposts or the spaces between them).

Có 21 số hạng trong dãy. Nếu bạn nói 20, bạn vừa mắc phải lỗi cọc hàng rào (fencepost error) — không biết nên đếm các cọc hay đếm các khoảng giữa chúng.

Answer 16 (from exercise 16)

Đáp án 16 (từ bài tập 16)

September, 1752 had only 19 days. This was done to synchronize

Tháng Chín năm 1752 chỉ có 19 ngày. Việc này được làm để đồng bộ

calendars as part of the Gregorian Reformation.

lịch, như một phần của cuộc Cải cách Gregorian.

The directory could have been removed by another **process**, you might not have permission to read it, the drive might not be mounted, ...; you get the picture.

Thư mục có thể đã bị một tiến trình khác xóa mất, bạn có thể không có quyền đọc nó, ổ đĩa có thể chưa được gắn kết (mount), ...; bạn hiểu vấn đề rồi đấy.

We sneakily didn't specify the types of a and b . Operator overloading might have defined + , = , or != to have unexpected behavior. Also, a and b may be aliases for the same variable, so the second assignment will overwrite the value stored in the first. Also, if the program is concurrent and badly written, a might have been updated by the time the addition takes place.

Chúng tôi đã lén không nói rõ kiểu của a và b . Việc nạp chồng toán tử (operator overloading) có thể đã định nghĩa + , = hay != với hành vi ngoài dự kiến. Ngoài ra, a và b có thể là hai bí danh (alias) của cùng một biến, nên phép gán thứ hai sẽ ghi đè giá trị đã lưu ở phép gán thứ nhất. Ngoài ra, nếu chương trình chạy đồng thời mà lại viết ẩu, a có thể đã bị cập nhật vào đúng lúc phép cộng diễn ra.

In non-Euclidean geometry, the sum of the angles of a triangle will not add up to 180°. Think of a triangle mapped on the surface of a sphere.

Trong hình học phi Euclid, tổng các góc của một tam giác sẽ không cộng lại bằng 180°. Hãy nghĩ đến một tam giác vẽ trên bề mặt một hình cầu.

Leap minutes may have 61 or 62 seconds.

Phút nhuận (leap minute) có thể có 61 hoặc 62 giây.

Depending on the language, numeric overflow may leave the result of $a+1$ negative.

Tùy ngôn ngữ, tràn số (numeric overflow) có thể khiến kết quả của $a+1$ hóa ra âm.

Answer 17 (from exercise 17)

Đáp án 17 (từ bài tập 17)

In most C and C++ implementations, there is no way of checking that a pointer actually points to valid memory. A common mistake is to **deallocate** a block of memory and reference that memory later in the program. By then, the memory pointed to may well have been reallocated to some other purpose. By setting the pointer to NULL , the programmers hope to prevent these rogue references—in most cases, dereferencing a NULL pointer will generate a runtime error.

Trong hầu hết các hiện thực C và C++, không có cách nào kiểm tra xem một con trỏ có thực sự trỏ vào vùng nhớ hợp lệ hay không. Một lỗi thường gặp là giải phóng một khối bộ nhớ rồi lại tham chiếu tới vùng nhớ đó ở phần sau của chương trình. Đến lúc ấy, vùng nhớ được trỏ tới rất có thể đã được cấp phát lại cho mục đích khác. Bằng cách gán con trỏ về NULL , lập trình viên hy vọng ngăn được những tham chiếu hoang đường này — trong hầu hết trường hợp, việc giải tham chiếu (dereference) một con trỏ NULL sẽ sinh ra lỗi lúc chạy.

Answer 18 (from exercise 18)

Đáp án 18 (từ bài tập 18)

By setting the reference to NULL , you reduce the number of pointers to the referenced object by one. Once this count

Bằng cách gán tham chiếu về NULL , bạn giảm số con trỏ trỏ tới đối tượng được tham chiếu đi một. Một khi bộ đếm này

reaches zero, the object is eligible for garbage collection. Setting the references to NULL can be significant for long-running programs, where the programmers need to ensure that memory utilization doesn't increase over time.

chạm mốc không, đối tượng đủ điều kiện để bộ thu gom rác (garbage collector) dọn đi. Việc gán các tham chiếu về NULL có thể rất đáng kể với những chương trình chạy dài, nơi lập trình viên cần bảo đảm mức sử dụng bộ nhớ không tăng dần theo thời gian.

Answer 19 (from exercise 19)

Đáp án 19 (từ bài tập 19)

A simple implementation could be:

Một hiện thực đơn giản có thể là:

event/strings_ex_1.rb

event/strings_ex_1.rb

```
class FSM def initialize(transitions, initial_state) @transitions = transitions @**state** = initial_state
end def accept(event) @state, action = TRANSITIONS[@state][event] || TRANSITIONS[@state][:default] end end
```

```
class FSM def initialize(transitions, initial_state) @transitions = transitions @state = initial_state end def accept(event) @state, action = TRANSITIONS[@state][event] || TRANSITIONS[@state][:default] end end
```

(Download this file to get the updated code that uses this new FSM class.)

(Hãy tải tệp này về để lấy đoạn mã đã cập nhật dùng lớp FSM mới này.)

Answer 20 (from exercise 20)

Đáp án 20 (từ bài tập 20)

...three network interface down events within five minutes This could be implemented using a state machine, but it would be trickier than it might first appear: if you get events at minutes 1, 4, 7, and 8, then you should trigger the warning on the fourth event, which means the state machine needs to be able to handle resetting itself. For this reason, event streams would seem to be the technology of choice. There's a reactive function named `buffer` with `size` and `offset` parameters that would let you return each group of three incoming events. You could then look at the timestamps of the first and last event in a group to determine if the alarm should be triggered.

...ba sự kiện giao diện mạng ngừng hoạt động trong vòng năm phút. Việc này có thể hiện thực bằng một máy trạng thái, nhưng sẽ rắc rối hơn thoạt nhìn: nếu bạn nhận được các sự kiện ở phút 1, 4, 7 và 8, thì bạn phải kích hoạt cảnh báo ở sự kiện thứ tư, nghĩa là máy trạng thái phải có khả năng tự đặt lại (reset) chính nó. Vì lý do đó, luồng sự kiện (event stream) có vẻ là công nghệ nên chọn. Có một hàm phản ứng (reactive) tên là `buffer` với tham số `size` và `offset`, cho phép bạn trả về từng nhóm ba sự kiện đến. Sau đó bạn có thể nhìn dấu thời gian của sự kiện đầu và sự kiện cuối trong một nhóm để xác định có nên kích hoạt báo động hay không.

...after sunset, and there is motion detected at the bottom of the stairs followed by motion detected at the top of the stairs... This could probably be implemented using a combination of `pubsub` and state machines. You could use `pubsub` to disseminate events to any number of state machines, and then have the state machines determine what to do.

...sau khi mặt trời lặn, và phát hiện chuyển động ở chân cầu thang, tiếp theo là phát hiện chuyển động ở đầu cầu thang... Việc này có lẽ hiện thực được bằng cách kết hợp `pubsub` và máy trạng thái. Bạn có thể dùng `pubsub` để phát tán sự kiện tới bất kỳ số lượng máy trạng thái nào, rồi để các máy trạng thái tự quyết định phải làm gì.

...notify various reporting systems that an order was completed. This is probably best handled using `pubsub`. You might want to use streams, but that would require that the systems being notified were also **stream** based.

...báo cho nhiều hệ thống lập báo cáo biết rằng một đơn hàng đã hoàn tất. Việc này có lẽ tốt nhất là dùng `pubsub`. Bạn có thể muốn dùng stream, nhưng như vậy đòi hỏi các hệ thống được thông báo cũng phải dựa trên stream.

...three backend services and wait for the responses. This is similar to our example that used streams to fetch user data.

...gọi ba dịch vụ phía sau (backend service) và chờ phản hồi. Việc này tương tự ví dụ của chúng tôi dùng stream để lấy dữ liệu người dùng.

Answer 21 (from exercise 21)

Đáp án 21 (từ bài tập 21)

1. Shipping and sales tax are added to an order:

1. Phí vận chuyển và thuế bán hàng được thêm vào một đơn hàng:

basic order → finalized order

đơn hàng cơ bản → đơn hàng đã chốt

In conventional code, it's likely you'd have a function that calculated shipping costs and another that calculated tax. But we're thinking about transformations here, so we transform an order with just items into a new kind of thing: an order that can be shipped. 2. Your application loads **configuration** information from a named file:

Trong mã truyền thống, nhiều khả năng bạn sẽ có một hàm tính phí vận chuyển và một hàm khác tính thuế. Nhưng ở đây chúng ta đang nghĩ theo lối biến đổi (transformation), nên ta biến đổi một đơn hàng chỉ gồm các mặt hàng thành một thứ kiểu mới: một đơn hàng có thể giao được. 2. Ứng dụng của bạn nạp thông tin cấu hình từ một tệp có tên cho trước:

file name → configuration structure

tên tệp → cấu trúc cấu hình

3. Someone logs in to a web application:

3. Có người đăng nhập vào một ứng dụng web:

user credentials → session

thông tin đăng nhập của người dùng → phiên (session)

Answer 22 (from exercise 22)

Đáp án 22 (từ bài tập 22)

The high-level **transformation**:

Phép biến đổi ở mức cao:

field contents as string

nội dung trường dưới dạng chuỗi

→ [validate & convert] → { :ok, value } | { :error, reason }

→ [kiểm tra & chuyển đổi] → { :ok, value } | { :error, reason }

could be broken down into:

có thể được chia nhỏ thành:

field contents as string → [convert string to integer] → [check value >= 18] → [check value <= 150]
→ { :ok, value } | { :error, reason }

nội dung trường dưới dạng chuỗi → [chuyển chuỗi thành số nguyên] → [kiểm tra giá trị >= 18] → [kiểm tra giá trị <= 150] → { :ok, value } | { :error, reason }

This assumes that you have an error-handling **pipeline**.

Điều này giả định rằng bạn có một đường ống xử lý lỗi (error-handling pipeline).

Answer 23 (from exercise 23)

Đáp án 23 (từ bài tập 23)

Let's answer the second part first: we prefer the first piece of code.

Hãy trả lời phần thứ hai trước: chúng tôi thích đoạn mã thứ nhất hơn.

In the second chunk of code, each step returns an object that implements the next function we call: the object returned by content_of must implement find_matching_lines , and so on.

Ở đoạn mã thứ hai, mỗi bước trả về một đối tượng có hiện thực hàm mà ta gọi kế tiếp: đối tượng do `content_of` trả về phải hiện thực `find_matching_lines`, và cứ thế.

This means that the object returned by `content_of` is coupled to our code. Imagine the requirement changed, and we have to ignore lines starting with a `#` character. In the transformation style, that would be easy:

Điều đó nghĩa là đối tượng do `content_of` trả về bị ghép nối với mã của chúng ta. Hãy tưởng tượng yêu cầu thay đổi, và ta phải bỏ qua những dòng bắt đầu bằng ký tự `#`. Theo lối biến đổi (transformation), việc đó sẽ rất dễ:

```
const content = File.read(file_name); const no_comments = remove_comments(content) const lines = find_matching_lines(no_comments, pattern) const result = truncate_lines(lines)
```

```
const content = File.read(file_name); const no_comments = remove_comments(content)
const lines = find_matching_lines(no_comments, pattern) const result = truncate_lines(lines)
```

We could even swap the order of `remove_comments` and `find_matching_lines` and it would still work.

Ta thậm chí có thể hoán đổi thứ tự của `remove_comments` và `find_matching_lines` mà nó vẫn chạy tốt.

But in the chained style, this would be more difficult. Where should our `remove_comments` method live: in the object returned by

Nhưng ở lối nối chuỗi (chaining), việc này sẽ khó hơn. Phương thức `remove_comments` của ta nên đặt ở đâu: trong đối tượng do

`content_of` or the object returned by `find_matching_lines`? And what other code will we break if we change that object? This **coupling** is why the method chaining style is sometimes called a train wreck

`content_of` trả về, hay trong đối tượng do `find_matching_lines` trả về? Và ta sẽ làm hỏng những đoạn mã nào khác nếu thay đổi đối tượng đó? Chính sự ghép nối này là lý do lối nối chuỗi phương thức đôi khi bị gọi là train wreck (thảm họa tàu hỏa).

Answer 24 (from exercise 24)

Đáp án 24 (từ bài tập 24)

Image processing. For simple scheduling of a workload among the parallel processes, a shared work queue may be more than adequate. You might want to consider a **blackboard** system if there is feedback involved—that is, if the results of one processed chunk affect other chunks, as in machine vision applications, or complex 3D image-warp transforms.

Xử lý ảnh. Để lập lịch đơn giản cho một khối lượng công việc trải đều giữa các tiến trình song song, một hàng đợi công việc dùng chung (shared work queue) có lẽ là quá đủ. Bạn có thể cân nhắc hệ thống bảng đen (blackboard) nếu có yếu tố phản hồi — tức là nếu kết quả của một mảnh đã xử lý ảnh hưởng tới các mảnh khác, như trong các ứng dụng thị giác máy (machine vision), hay các phép biến đổi cong ảnh 3D phức tạp.

Group calendaring This might be a good fit. You can post scheduled meetings and availability to the blackboard. You have entities functioning autonomously, feedback from decisions is important, and participants may come and go.

Lập lịch họp nhóm. Đây có thể là một sự phù hợp tốt. Bạn có thể đăng các cuộc họp đã lên lịch và tình trạng rảnh/bận lên bảng đen. Bạn có các thực thể hoạt động tự chủ, phản hồi từ các quyết định là quan trọng, và người tham gia có thể đến rồi đi.

You might want to consider partitioning this kind of blackboard system depending on who is searching: junior staff may care about only the immediate office, human resources may want only English-speaking offices worldwide, and the CEO may want the whole enchilada.

Bạn có thể cân nhắc phân vùng (partition) loại hệ thống bảng đen này tùy theo ai đang tìm kiếm: nhân viên cấp dưới có lẽ chỉ quan tâm tới văn phòng ngay gần, phòng nhân sự có thể chỉ muốn các văn phòng nói tiếng Anh trên toàn cầu, còn CEO có thể muốn nguyên cả mâm cỗ.

There is also some flexibility on data formats: we are free to ignore formats or languages we don't understand. We have to understand different formats only for those offices that have meetings with each other, and we do not

Cũng có chút linh hoạt về định dạng dữ liệu: ta được tự do bỏ qua những định dạng hay ngôn ngữ mà ta không hiểu. Ta chỉ cần hiểu các định dạng khác nhau với những văn phòng có họp với nhau, và ta không

need to expose all participants to a full transitive closure of all possible formats. This reduces coupling to where it is necessary, and does not constrain us artificially.

cần phơi bày toàn bộ người tham gia ra một bao đóng bắc cầu (transitive closure) đầy đủ của mọi định dạng khả dĩ. Việc này giảm ghép nối xuống chỉ còn ở nơi cần thiết, và không ràng buộc ta một cách giả tạo.

Network monitoring tool This is very similar to the mortgage/loan application program . You've got trouble reports sent in by users and statistics reported automatically, all posting to the blackboard. A human or software agent can analyze the blackboard to diagnose network failures: two errors on a line might just be cosmic rays, but 20,000 errors and you've got a hardware problem. Just as the detectives solve the murder mystery, you can have multiple entities analyzing and contributing ideas to solve the network problems.

Công cụ giám sát mạng. Cái này rất giống chương trình xử lý đơn vay/thế chấp. Bạn có các báo cáo sự cố do người dùng gửi vào và thống kê được báo cáo tự động, tất cả đều đăng lên bảng đen. Một tác nhân là người hay phần mềm có thể phân tích bảng đen để chẩn đoán hỏng hóc mạng: hai lỗi trên một đường truyền có thể chỉ là tia vũ trụ, nhưng 20.000 lỗi thì bạn đã có vấn đề phần cứng rồi. Hệt như các thám tử phá án mạng người, bạn có thể cho nhiều thực thể cùng phân tích và đóng góp ý tưởng để giải quyết các trục trặc mạng.

Answer 25 (from exercise 25)

Đáp án 25 (từ bài tập 25)

The assumption with a list of key-value pairs is generally that the key is unique, and **hash** libraries typically enforce that either by the behavior of the hash itself or with explicit error messages for duplicated keys. However, an array typically does not have those constraints, and will happily store duplicate keys unless you code it specifically not to. So in this case, the first key found that matches DepositAccount wins, and any remaining matching entries are ignored. The order of entries is not guaranteed, so sometimes it works and sometimes it doesn't.

Với một danh sách các cặp khóa-giá trị, giả định thông thường là khóa phải duy nhất, và các thư viện hash thường bắt buộc điều đó, hoặc bằng chính hành vi của hash, hoặc bằng thông báo lỗi tường minh khi khóa bị trùng. Tuy nhiên, một mảng thường không có những ràng buộc ấy, và sẽ vui vẻ lưu các khóa trùng nhau trừ khi bạn viết mã để chặn chuyện đó. Nên trong trường hợp này, khóa đầu tiên khớp với DepositAccount sẽ thắng, còn mọi mục khớp còn lại đều bị bỏ qua. Thứ tự các mục lại không được bảo đảm, nên lúc thì nó chạy đúng, lúc thì không.

And what about the difference in machines from development and production? It's just a coincidence.

Còn chuyện khác biệt máy móc giữa môi trường phát triển và môi trường sản xuất thì sao?
Chỉ là trùng hợp ngẫu nhiên thôi.

Answer 26 (from exercise 26)

Đáp án 26 (từ bài tập 26)

The fact that a purely numeric field works in the US, Canada, and the Caribbean is a coincidence. Per the ITU spec, international call format starts with a literal + sign. The * character is also used in some locales, and more commonly, leading zeros can be a part of the number. Never store a phone number in a numeric field.

Việc một trường thuần số vẫn dùng được ở Mỹ, Canada và vùng Caribbean chỉ là chuyện trùng hợp ngẫu nhiên. Theo đặc tả của ITU, định dạng gọi quốc tế bắt đầu bằng đúng một dấu + . Ký tự * cũng được dùng ở vài nơi, và phổ biến hơn nữa, các số 0 đứng đầu có thể là một phần của số điện thoại. Đừng bao giờ lưu số điện thoại trong một trường kiểu số.

Answer 27 (from exercise 27)

Đáp án 27 (từ bài tập 27)

Depends on where you are. In the US, volume measures are based on the gallon, which is the volume of a cylinder 6 inches high and 7 inches in diameter, rounded to the nearest cubic inch.

Tùy bạn ở đâu. Ở Mỹ, các đơn vị đo thể tích dựa trên gallon, vốn là thể tích của một hình trụ cao 6 inch và đường kính 7 inch, làm tròn tới inch khối gần nhất.

In Canada, “one cup” in a recipe could mean any of

Ở Canada, “một cup” trong một công thức nấu ăn có thể là bất kỳ giá trị nào trong số

1/5 of an imperial quart, or 227ml

1/5 của một imperial quart, tức 227ml

1/4 of a US quart, or 236ml

1/4 của một US quart, tức 236ml

16 metric tablespoons, or 240ml

16 muỗng canh hệ mét, tức 240ml

1/4 of a liter, or 250ml

1/4 lít, tức 250ml

Unless you're talking about a rice cooker, in which case “one cup” is 180ml. That derives from the koku , which was the estimated volume of **dry** rice required to feed one person for one year: apparently, around 180L. Rice cooker cups are 1 g ō , which is 1/1000 of a koku. So, roughly the amount of rice a person would eat at a single meal. [85]

Trừ khi bạn đang nói về nồi cơm điện, trường hợp đó “một cup” là 180ml. Con số ấy bắt nguồn từ đơn vị koku , vốn là thể tích gạo khô ước tính đủ nuôi một người trong một năm: nghe nói vào khoảng 180L. Cup của nồi cơm điện là 1 gō , tức 1/1000 koku. Nói nôm na là lượng gạo một người ăn hết trong một bữa. [85]

Answer 28 (from exercise 28)

Đáp án 28 (từ bài tập 28)

Clearly, we can't give any absolute answers to this exercise. However, we can give you a couple of pointers.

Rõ ràng chúng tôi không thể đưa ra câu trả lời tuyệt đối nào cho bài tập này. Tuy nhiên, chúng tôi có thể cho bạn vài gợi ý.

If you find that your results don't follow a smooth curve, you might want to check to see if some other activity is using some of your processor's power. You probably won't get good figures if background processes periodically take cycles away from your programs. You might also want to check memory: if the application starts using swap space, performance will nose dive.

Nếu bạn thấy kết quả của mình không theo một đường cong mượt mà, bạn có thể nên kiểm tra xem có hoạt động nào khác đang ngốn bớt sức của bộ xử lý không. Bạn có lẽ sẽ không có được con số đẹp nếu các tiến trình nền định kỳ giành chu kỳ xử lý khỏi chương trình của bạn. Bạn cũng nên kiểm tra bộ nhớ: nếu ứng dụng bắt đầu dùng tới không gian trao đổi (swap space), hiệu năng sẽ cắm đầu lao dốc.

Here's a graph of the results of running the code on one of our machines:

Đây là đồ thị kết quả khi chạy đoạn mã trên một trong các máy của chúng tôi:

images/alg-speed-rust-results.png

images/alg-speed-rust-results.png

Answer 29 (from exercise 29)

Đáp án 29 (từ bài tập 29)

There are a couple of ways of getting there. One is to turn the problem on its head. If the array has just one element, we don't iterate around the loop. Each additional iteration doubles the size of the array we can search. The general formula for the array size is therefore 2^i , where i is the number of iterations. If you take logs to the base 2 of each side, you get $i = \log_2 n$, which by the definition of logs becomes

Có vài cách để đến đích. Một cách là lật ngược bài toán lại. Nếu mảng chỉ có một phần tử, ta không lặp quanh vòng lặp lần nào. Mỗi lần lặp thêm sẽ nhân đôi kích thước mảng mà ta có thể tìm kiếm. Vậy công thức tổng quát cho kích thước mảng là, trong đó là số lần lặp. Nếu lấy logarit cơ số 2 cả hai vế, bạn được, theo định nghĩa logarit thì trở thành.

Answer 30 (from exercise 30)

Đáp án 30 (từ bài tập 30)

This is probably too much of a flashback to secondary school math, but the formula for converting a logarithm in base b to one in base a is:

Cái này có lẽ hơi gợi nhớ tới toán cấp hai quá mức cần thiết, nhưng công thức chuyển một logarit ở cơ số này sang cơ số kia là:

Because c is a constant, then we can ignore it inside a Big-O result.

Vì là một hằng số, nên ta có thể bỏ qua nó trong một kết quả Big-O.

Answer 31 (from exercise 31)

Đáp án 31 (từ bài tập 31)

One property we can test is that an order succeeds if the warehouse has enough items on hand. We can generate orders for random quantities of items, and verify that an “OK” tuple is returned if the warehouse had stock.

Một thuộc tính ta có thể kiểm thử là: một đơn hàng sẽ thành công nếu kho còn đủ hàng. Ta có thể sinh ra các đơn hàng với số lượng mặt hàng ngẫu nhiên, rồi kiểm chứng rằng một tuple “OK” được trả về nếu kho có hàng.

Answer 32 (from exercise 32)

Đáp án 32 (từ bài tập 32)

This is a good use of **property-based testing**. The unit tests can focus on individual cases where you’ve worked out the result by

Đây là một trường hợp dùng tốt cho kiểm thử dựa trên thuộc tính (property-based testing). Các bài kiểm thử đơn vị (unit test) có thể tập trung vào những ca riêng lẻ mà bạn đã tính ra kết quả bằng

some other means, and the property tests can focus on things like:

một cách nào đó khác, còn các bài kiểm thử thuộc tính có thể tập trung vào những thứ như:

Do any two crates overlap?

Có hai thùng nào chồng lên nhau không?

Does any part of any crate exceed the width or length of the truck?

Có phần nào của thùng nào vượt quá chiều rộng hay chiều dài của xe tải không?

Is the packing density (area used by crates divided by the area of the truck bed) less than or equal to 1?

Mật độ xếp (diện tích các thùng chia cho diện tích sàn xe) có nhỏ hơn hoặc bằng 1 không?

If it’s part of the requirement, does the packing density exceed the minimum acceptable density?

Nếu là một phần của yêu cầu, mật độ xếp có vượt quá mật độ tối thiểu chấp nhận được không?

Answer 33 (from exercise 33)

Đáp án 33 (từ bài tập 33)

1. This statement sounds like a real requirement: there may be constraints placed on the application by its environment. 2. On its own, this statement isn’t really a requirement. But to find out what’s really required, you have to ask the magic question, “Why?” It may be that this is a corporate standard, in which case the actual requirement should be something like “all UI elements must conform to the MegaCorp User Interface Standards, V12.76.” It may be that this is a color that the design team happen to like. In that case, you should think about the way the design team also likes to change their minds, and phrase the requirement as “the background color of all modal windows must be configurable. As shipped, the color will be gray.” Even better would be the broader statement “All visual elements of the application (colors, fonts, and languages) must be configurable.” Or it may simply mean that the user needs to be able to distinguish modal and nonmodal windows. If that’s the case, some more discussions are needed. 3. This statement is not a requirement, it’s architecture. When faced with something like this, you have to dig deep to find out what the user is thinking. Is this a scaling issue?

Or performance? Cost? Security? The answers will inform your design. 4. The underlying requirement is probably something closer to “The system will prevent the user from making invalid entries in fields,

1. Phát biểu này nghe như một yêu cầu thật: có thể có những ràng buộc do môi trường đặt lên ứng dụng. 2. Đứng riêng một mình, phát biểu này không thực sự là một yêu cầu. Nhưng để tìm ra điều gì thực sự được yêu cầu, bạn phải hỏi câu hỏi thần kỳ: “Tại sao?” Có thể đây là một chuẩn của công ty, trong trường hợp đó yêu cầu thật sự nên là kiểu như “mọi thành phần giao diện người dùng phải tuân theo Chuẩn Giao diện Người dùng MegaCorp, V12.76.” Có thể đây là một màu mà nhóm thiết kế tình cờ thích. Trong trường hợp đó, bạn nên nghĩ tới chuyện nhóm thiết kế cũng hay thay đổi ý định, và diễn đạt yêu cầu thành “màu nền của mọi cửa sổ chế độ (modal window) phải có thể cấu hình được. Khi giao hàng, màu sẽ là xám.” Còn hay hơn nữa là phát biểu rộng hơn “Mọi yếu tố trực quan của ứng dụng (màu sắc, phong chữ và ngôn ngữ) phải có thể cấu hình được.” Hoặc nó có thể đơn giản chỉ có nghĩa là người dùng cần phân biệt được cửa sổ chế độ và cửa sổ không chế độ. Nếu vậy, cần thêm vài cuộc thảo luận nữa. 3. Phát biểu này không phải là một yêu cầu, mà là kiến trúc. Khi đối mặt với thứ như thế này, bạn phải đào sâu để tìm ra người dùng đang nghĩ gì. Đây là vấn đề mở rộng quy mô (scaling)? Hay hiệu năng? Chi phí? Bảo mật? Câu trả lời sẽ định hướng cho thiết kế của bạn. 4. Yêu cầu nền tảng có lẽ gần với “Hệ thống sẽ ngăn người dùng nhập các mục không hợp lệ vào các trường,

and will warn the user when these entries are made.” 5. This statement is probably a hard requirement, based on some hardware limitation.

và sẽ cảnh báo người dùng khi những mục nhập này được tạo.” 5. Phát biểu này có lẽ là một yêu cầu cứng, dựa trên một hạn chế phần cứng nào đó.

And here’s a solution to the four-dots problem:

Và đây là lời giải cho bài toán bốn chấm:

images/four_dots_answer.png

images/four_dots_answer.png

Footnotes

Cước chú

[85] Thanks for this bit of trivia goes to Avi Bryant (@avibryant)

[85] Cảm ơn Avi Bryant (@avibryant) đã cung cấp mẫu kiến thức vật này

Copyright © 2020 Pearson Education, Inc.

Copyright © 2020 Pearson Education, Inc.